R for Health Economics & Outcomes Research and Health Technology Assessment



Dr Aswath Karunakaran

ONE MUST IMAGINE SISYPHUS HAPPY.

ALBERT CAMUS

R for Health Economics and Outcomes Research (HEOR) and Health Technology Assessment (HTA)

A practical, reproducible guide to decision-analytic modelling, evidence synthesis, and HTA submission workflows in R

Dr Aswath Karunakaran

Draft edition — compiled progressively, chapter by chapter

Foreword

Health technology assessment sits at an uncomfortable junction: the discipline demands the statistical rigour of a biostatistics department, the structural clarity of an actuarial model, and the practical honesty of a budget-holder who has to live with the decision an analysis recommends. Few analysts arrive at HEOR already fluent in all three. Most arrive, as this book's author did, from a single adjacent discipline — public health, epidemiology, clinical medicine, economics, or data science — and build the rest on the job, one model, one submission, one committee critique at a time.

This book exists because that on-the-job build is harder than it should be. The canonical references on health economic modelling (Briggs, Claxton and Sculpher's **Decision Modelling for Health Economic Evaluation**; Drummond and colleagues' **Methods for the Economic Evaluation of Health Care Programmes**) are rightly foundational, but they were not written as R programming references, and R's own documentation was not written with HTA submission requirements in mind. The gap between "I understand what a partitioned survival model is" and "I can build one in R that a NICE evidence review group will accept without a technical query" is where most working analysts spend their first several difficult years.

What follows is an attempt to close that gap directly: every statistical and economic concept is paired with working R code; every modelling chapter is grounded in what the major HTA agencies actually ask to see; every method is introduced with the assumption that the reader is capable and motivated, not that the reader has already sat through a two-year health economics master's programme. Readers coming from applied public health and programmatic data work — TB programme analysis, national disease registries, LMIC health systems — will find this book written with their vantage point as an explicit first-class concern, not an afterthought bolted onto a Western-payer-market default.

No single book makes anyone a health economist. This one aims to make the R side of the job considerably less lonely.

Preface

Why R for HEOR and HTA

Health economic models have historically lived in Excel and TreeAge, with SAS handling the clinical-trial statistics feeding into them. That world is changing, and not slowly. R now has a mature, purpose-built ecosystem for every stage of the HEOR workflow — `flexsurv` and `survival` for extrapolation, `heemod` and `hesim` for cohort and individual-level state-transition models, `BCEA` and `voi` for uncertainty and value-of-information analysis, `netmeta` and `gemtc` for network meta-analysis, `shiny` for submission-ready interactive model front-ends. HTA agencies, NICE chief among them, now routinely receive and review R-based submissions, and several publish explicit methodological guidance (the NICE DSU Technical Support Documents) that assumes a modeller capable of implementing the methods in code, not just describing them in a Word document.

R's advantages over the Excel/TreeAge default are not merely aesthetic. Version control is native. A model's entire parameter set, structural logic, and reporting pipeline can live in one reviewable, re-runnable repository instead of a spreadsheet with hidden formulas and undocumented macro buttons. Probabilistic sensitivity analysis that would take an Excel model minutes per thousand iterations runs in seconds. Reproducibility — increasingly a condition of serious HTA submission, and squarely the subject of the CHEERS 2022 reporting checklist — is close to automatic in R and close to aspirational in Excel.

None of this makes R effortless. It trades spreadsheet transparency (anyone can click a cell and see the formula) for code transparency (anyone who can read R can see the logic, but first they have to be able to read R). This book's purpose is to make that trade pay off quickly rather than slowly.

Who this book is for

This book assumes:

- Working competence in R at the level of being comfortable with data frames, basic data wrangling, and writing simple functions — not mastery, and Chapter 3 exists specifically to shore up anything shaky here.
- No prior formal health economics training. Chapter 4 builds the core concepts — QALYs, ICERs, discounting, perspective — from first principles, with R code alongside each concept as it is introduced.
- A practical, applied motivation: you need to build, review, critique, or commission health economic models and evidence syntheses as part of real work, whether in a pharmaceutical HEOR team, an HTA agency, an academic health economics group, a consultancy, or — a case this book treats as a first-class audience rather than a footnote — a public health programme in a low- or middle-income country assessing interventions against a national reference case and a genuinely binding budget constraint.

Readers coming from applied public health, programmatic disease surveillance, or clinical data analysis backgrounds will recognise much of the data-quality and real-world-evidence material in Part II and Part III. That recognition is intentional: the skills that make someone good at program-

matic TB, HIV, or immunisation data analysis — scepticism about data completeness, comfort with messy field data, an instinct for what a confidence interval is actually telling you — transfer directly into HEOR, and this book tries to make that transfer explicit rather than leaving the reader to discover it unaided.

How to use this book

The book is organised to be read front-to-back by someone starting from zero, but each Part is also written to stand largely on its own for a reader who already has foundations and wants to go straight to, say, network meta-analysis (Part III) or decision-tree and Markov modelling (Part IV). Every chapter opens with explicit learning objectives and closes with exercises; "HEOR in practice" boxes throughout ground the method in a real submission scenario, "Common pitfalls" boxes flag the mistakes that recur most often in actual agency technical queries, and "Agency perspective" boxes state plainly what a reviewer at NICE, CADTH, PBAC, or HTAIn is likely to be checking for.

Part VIII's end-to-end case studies, including a TB programme cost-effectiveness analysis built for an LMIC setting, are deliberately placed last: by that point every technique used in them has already been introduced on its own, and the case studies exist to show how the pieces assemble into a real, submission-shaped analysis.

A note on how this edition was produced

This draft edition is being written progressively, chapter by chapter, with R code and HTA-practice content checked against standard references (ISPOR-SMDM good-practice guidance, NICE DSU Technical Support Documents, Briggs/Claxton/Sculpher, Drummond et al.) as each chapter is completed, rather than delivered as a single finished manuscript. Readers working through an early draft chapter should treat package version numbers and function signatures as subject to verification against the live CRAN package before production use in a real submission — Chapter 2's `renv` and reproducibility guidance exists precisely so that this kind of drift between a book's print date and a package's current behaviour never silently breaks a real model.

Companion Website, Code Repository and Data Sets

All R code in this book is intended to live in a companion GitHub repository, organised to mirror the book's chapter structure (`ch01/`, `ch02/`, … `ch35/`), with each chapter's code runnable standalone given the chapter's own `renv.lock` file (see Chapter 2 §2.3 for why each chapter — not just the book as a whole — pins its own package versions).

Planned repository contents, to be populated as each chapter is finalised:

- `/code/` — one subfolder per chapter, containing every code listing in that chapter as a runnable `.R` or `.qmd` file, not just the excerpts shown in the printed text.
- `/data/` — the synthetic and openly-licensed example datasets used throughout (see Appendix F for the full data dictionary); no real patient-level or commercially sensitive trial data is used anywhere in this book.
- `/appendix-c-template/` — the full template R project structure from Appendix C, ready to fork for a new real-world analysis.
- `/solutions/` — worked solutions to end-of-chapter exercises (Appendix G).

Until the repository is published, each chapter's code lives alongside its Typst source in this project's own `typst_project/` folder and the parallel `Chapter Markdowns/` summaries, which together serve as the working source of truth during drafting. A working local draft of the `/code/` folder itself — one subfolder per chapter, auto-extracted from each chapter's typeset code blocks and kept current after every edit — already exists locally and is what gets pushed into the public repository once it is published.

Reporting errors

Health economic models are unforgiving of small coding errors — a flipped sign in a discounting formula or a mis-specified transition matrix can silently produce a materially wrong ICER. Any reader who finds an error in this book's code or its statistical claims should treat that as worth flagging immediately rather than working around it quietly; the companion repository's issue tracker will be the designated channel once published.

Conventions, Package Versions and Reproducibility Notes

Typographic conventions

- Inline R code, function names, and package names appear in `monospace`: e.g. `heemod::run_model()`, `flexsurv`.
- Standalone code listings are set in full-width monospace blocks with syntax-relevant elements distinguishable by weight; output shown beneath a listing is marked `#>` per R console convention.
- Mathematical notation follows standard health-economics usage: discount rate as r, time horizon as T, incremental cost as ΔC, incremental effect as ΔE, and the incremental cost-effectiveness ratio as $\text{ICER} = \Delta\frac{C}{\Delta}E$.
- Callout boxes recur across every methods chapter in three forms: **HEOR in practice** (a real-submission-shaped scenario illustrating the method just introduced), **Common pitfall** (a mistake that recurs often enough in actual agency technical-engagement queries to be worth naming explicitly), and **Agency perspective** (what a named HTA agency's published guidance or typical reviewer behaviour says about the method).

Package versions

Each chapter's code is developed and checked against the CRAN package versions current at the time that chapter is drafted; the exact versions are pinned in that chapter's own `renv.lock` in the companion repository (§ Companion Website, above) rather than hard-coded into the printed text, because HEOR-relevant packages — `flexsurv`, `hesim`, `heemod`, `BCEA`, `netmeta`, `mice`, among others — update frequently enough that a version number printed in a book is stale within a year or two. Where a function's behaviour has changed in a way that would break a book example, the companion repository's `CHANGES.md` is the authority, not the printed page.

Reproducibility baseline

Every worked example and case study in this book is built to run from a clean R session given only: base R, the packages named in that chapter, and the `renv.lock` for that chapter. No worked example depends on data, credentials, or infrastructure that a reader cannot access independently — real patient-level trial data is never used; synthetic or openly-licensed datasets (Appendix F) stand in throughout, chosen specifically to reproduce the statistical features (skewness, censoring, missingness patterns) that make the real-world versions of these analyses hard.

A note on country and agency defaults

Where a chapter must choose a concrete worked example — a willingness- to-pay threshold, a reference case, a named agency's submission template — this book deliberately rotates among NICE (UK), CADTH (Canada), PBAC (Australia), and HTAIn (India) rather than defaulting silently

to a single high-income reference case throughout. Chapter 31 lays out the comparison explicitly; earlier chapters that use a single agency's convention for a worked example say so directly rather than implying it as the universal default.

Abbreviations

ADaM	Analysis Data Model (CDISC)
AE	Adverse Event
AFT	Accelerated Failure Time (survival model)
AIC	Akaike Information Criterion
AMNOG	Arzneimittelmarktneuordnungsgesetz (German early benefit assessment process)
API	Application Programming Interface
BCEA	Bayesian Cost-Effectiveness Analysis (R package)
BIC	Bayesian Information Criterion
BLAS	Basic Linear Algebra Subprograms
CADTH	Canadian Agency for Drugs and Technologies in Health
CBA	Cost-Benefit Analysis
CCA	Cost-Consequence Analysis
CDISC	Clinical Data Interchange Standards Consortium
CEA	Cost-Effectiveness Analysis
CEAC	Cost-Effectiveness Acceptability Curve
CHEERS	Consolidated Health Economic Evaluation Reporting Standards
CI	Confidence Interval
CIF	Cumulative Incidence Function
CMA	Cost-Minimisation Analysis
CONSORT	Consolidated Standards of Reporting Trials
COI	Cost-of-Illness (study)
CPT	Current Procedural Terminology
CRAN	Comprehensive R Archive Network
CRO	Contract Research Organisation
CUA	Cost-Utility Analysis
DALY	Disability-Adjusted Life Year
DBI	R's DataBase Interface package
DCE	Discrete Choice Experiment

DHS	Demographic and Health Surveys
DIC	Deviance Information Criterion
DL	Deep Learning
DOI	Digital Object Identifier
DPDPA	Digital Personal Data Protection Act (India, 2023)
DR-TB	Drug-Resistant Tuberculosis
DSU	Decision Support Unit (NICE)
DTA	Diagnostic Test Accuracy
ECOG	Eastern Cooperative Oncology Group (performance status scale)
EHR	Electronic Health Record
EM	Expectation-Maximisation (algorithm)
EMB	Expectation-Maximisation with Bootstrapping (Amelia algorithm)
EORTC	European Organisation for Research and Treatment of Cancer
EQ-5D	EuroQol 5-Dimension (health utility instrument)
ESS	Effective Sample Size
FDA	Food and Drug Administration (United States)
GCP	Good Clinical Practice
GDP	Gross Domestic Product
GDPR	General Data Protection Regulation (EU)
GLM	Generalized Linear Model
HAS	Haute Autorité de Santé (France)
HCPCS	Healthcare Common Procedure Coding System
HIPAA	Health Insurance Portability and Accountability Act (United States)
HR	Hazard Ratio
HTA	Health Technology Assessment
HTAIn	Health Technology Assessment in India
HUI	Health Utilities Index
ICD	International Classification of Diseases
ICER	Incremental Cost-Effectiveness Ratio
ICH	International Council for Harmonisation of Technical Requirements for Pharmaceuticals for Human Use
IDE	Integrated Development Environment

IPCW	Inverse Probability of Censoring Weighting
IPD	Individual Patient Data
IQWiG	Institute for Quality and Efficiency in Health Care (Germany)
ISPOR	International Society for Pharmacoeconomics and Outcomes Research
ITC	Indirect Treatment Comparison
ITT	Intention-to-Treat
JAGS	Just Another Gibbs Sampler
KM	Kaplan-Meier (survival estimator)
LMIC	Low- and Middle-Income Country
LOCF	Last Observation Carried Forward
LTFU	Lost to Follow-Up
MAE	Mean Absolute Error
MAIC	Matching-Adjusted Indirect Comparison
MAR	Missing At Random
MCAR	Missing Completely At Random
MCMC	Markov Chain Monte Carlo
MDR-TB	Multidrug-Resistant Tuberculosis
MI	Multiple Imputation
ML	Machine Learning
ML-NMR	Multilevel Network Meta-Regression
MNAR	Missing Not At Random
NHB	Net Health Benefit
NHS	National Health Service (United Kingdom)
NICE	National Institute for Health and Care Excellence (United Kingdom)
NMA	Network Meta-Analysis
NMB	Net Monetary Benefit
OLS	Ordinary Least Squares
OR	Odds Ratio
PBAC	Pharmaceutical Benefits Advisory Committee (Australia)
PH	Proportional Hazards
PICO	Population, Intervention, Comparator, Outcome

PM-JAY	Pradhan Mantri Jan Arogya Yojana (India)
PMDA	Pharmaceuticals and Medical Devices Agency (Japan)
PPP	Purchasing Power Parity
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
PRISMA-NMA	PRISMA extension for Network Meta-Analyses
PSA	Probabilistic Sensitivity Analysis
QALY	Quality-Adjusted Life Year
RCT	Randomised Controlled Trial
REML	Restricted Maximum Likelihood
RIS	Research Information Systems (bibliographic file format)
ROBINS-I	Risk Of Bias In Non-randomised Studies of Interventions
RR	Risk Ratio (Relative Risk)
RMSE	Root Mean Squared Error
RWE	Real-World Evidence
SAS	Statistical Analysis System (software)
SDTM	Study Data Tabulation Model (CDISC)
SE	Standard Error
SF-6D	Short Form 6-Dimension (health utility instrument)
SG	Standard Gamble
SLR	Systematic Literature Review
SMD	Standardised Mean Difference
SPSS	Statistical Package for the Social Sciences (software)
SQL	Structured Query Language
SROC	Summary Receiver Operating Characteristic
STC	Simulated Treatment Comparison
SUCRA	Surface Under the Cumulative RAnking curve
TB	Tuberculosis
TEAE	Treatment-Emergent Adverse Event
TSD	Technical Support Document (NICE DSU)
TTO	Time Trade-Off
VOI	Value of Information

WHO	World Health Organization
WTP	Willingness-To-Pay
XDR-TB	Extensively Drug-Resistant Tuberculosis
YLD	Years Lived with Disability
YLL	Years of Life Lost

Table of Contents

Foreword iii
Preface iv
Why R for HEOR and HTA iv
Who this book is for iv
How to use this book v
A note on how this edition was produced v
Companion Website, Code Repository and Data Sets vi
Reporting errors vi
Conventions, Package Versions and Reproducibility Notes vii
Typographic conventions vii
Package versions vii
Reproducibility baseline vii
A note on country and agency defaults vii
Abbreviations ix
PART I
Foundations
..... 1
Chapter 1. Introduction to HEOR and HTA 2
What HEOR is and what it is not 2
The HTA ecosystem 4
Evidence needs across the product lifecycle 6
Typical deliverables 7
Why R: the case for open-source and reproducible HEOR 8
Where R fits in the HEOR workflow 9
Chapter 2. Setting Up an R Environment for HEOR 12
2.1 Installing R, RStudio/Positron and Quarto 12
2.2 Project structure for HEOR analyses 13
2.3 Package management with `renv` 14
2.4 Version control with Git and GitHub 16
2.5 Working with Stan, JAGS and C++ toolchains 17
2.6 Containers (Docker) for reproducible models 20
2.7 Recommended package ecosystem (overview map) 22
Chapter 3. R Essentials for Health Economists 25
3.1 Data structures: vectors, lists, data frames and tibbles, arrays and matrices 25
3.2 Data wrangling with `tidyverse` and `data.table` 27
3.3 Functional programming with `purrr` and vectorization 29
3.4 Writing functions, parameter lists and defensive programming 31
3.5 Performance: profiling, `Rcpp`, parallelisation (`future`, `furrr`) 34

3.6 Dates, durations and time-to-event data (lubridate) 37
3.7 Debugging and error handling 38
Chapter 4. Core Concepts in Health Economics (with R Illustrations) 40
4.1 Costs, effects and the opportunity cost of health care 40
4.2 Types of economic evaluation 42
4.3 QALYs, DALYs and life-years 44
4.4 ICERs, net monetary benefit (NMB) and net health benefit (NHB) 46
4.5 Willingness-to-pay thresholds, including country-specific and LMIC thresholds 49
4.6 Perspective, time horizon, discounting and half-cycle correction 51
4.7 Dominance, extended dominance and the efficiency frontier 54
4.8 Coding a first simple CEA in R 56

PART II

Data for HEOR

..... 62
Chapter 5. Data Sources and Data Management 63
5.1 Clinical trial data (CDISC SDTM/ADaM concepts) 63
5.2 Registries, claims, electronic health records and surveys 66
5.3 Published literature as a data source 68
5.4 Importing data: readr, readxl, haven, DBI, APIs 70
Delimited and flat files with readr 71
Spreadsheets with readxl 71
Clinical trial and survey formats with haven 72
Databases with DBI 73
Public and institutional APIs 74
5.5 Data dictionaries, metadata and codebooks 75
What a data dictionary must contain 75
Practical R approaches 76
5.6 Data quality checks (pointblank, assertr, validate) 77
5.7 Data governance, privacy and de-identification 80
Chapter 6. Systematic Literature Review (SLR) Support in R 82
6.1 Search strategy documentation 82
6.2 Deduplication and screening workflows 84
6.3 PRISMA flow diagrams in R 87
6.4 Data extraction templates and extraction QA 90
6.5 Risk of bias visualisation (robvis) 92
6.6 Text mining and ML-assisted screening 95
Chapter 7. Missing Data 97
7.1 Mechanisms: MCAR, MAR, MNAR 97
7.2 Exploring missingness (naniar, visdat, Shiny-based explorers) 99
7.3 Complete-case versus imputation 100
7.4 Multiple imputation (mice, Amelia, missForest) 102
7.5 Sensitivity analysis, including tipping-point and pattern-mixture approaches 105

7.6 Missing data in cost and QALY data 107
7.7 Reporting missing data handling for HTA 109
Chapter 8. Health-Related Quality of Life and Utilities 112
8.1 Generic and disease-specific instruments (EQ-5D, SF-6D, HUI, etc.) 112
8.2 Value sets and scoring in R (eq5d) 114
8.3 Mapping algorithms (cross-walks and regression-based mapping) 115
8.4 Analysing utility data: censoring, bounded distributions, beta regression, mixture models 117
8.5 Utility decrements, age-adjusted utilities and background mortality 120
8.6 Time trade-off, standard gamble and discrete choice experiments 122
Chapter 9. Costs and Resource Use 125
9.1 Identifying, measuring and valuing resources 125
9.2 Unit costs and inflation or currency conversion (PPP adjustments) 127
9.3 Cost data features: skewness, zero-inflation, heavy tails 129
9.4 Regression models for costs: GLMs, two-part models, gamma models 132
Generalised linear models on the cost scale 132
Two-part (hurdle-style) models for zero-inflated costs 133
Smooths and flexible mean functions 134
9.5 Cost-of-illness and micro-costing studies 135
9.6 Handling censored cost data 137

PART III

Statistical Methods for Evidence Synthesis and Analysis

..... 140
Chapter 10. Survival Analysis for Economic Evaluation 141
10.1 Kaplan-Meier and life tables 142
10.2 Parametric models 144
10.3 Flexible and spline-based models 146
10.4 Mixture-cure and non-mixture cure models 148
10.5 Piecewise and landmark models 151
10.6 Model selection: AIC/BIC, visual fit, clinical plausibility 153
10.7 Proportional hazards diagnostics and non-PH alternatives 157
Testing the proportional hazards assumption 157
When PH fails: alternative hazard structures 158
10.8 Covariate-adjusted and treatment-effect-specific models 159
Covariates on the location/scale parameter 159
Covariates on ancillary (shape) parameters 160
Treatment-effect-specific extrapolation 160
10.9 Extrapolation: external data, background mortality, expert elicitation 160
Anchoring with external data 161
Background mortality as a plausibility bound 161

Expert elicitation for structural and tail uncertainty ... 162
10.10 Competing risks and multi-state models ... 162
Competing risks: cause-specific vs. subdistribution hazards ... 163
Data preparation and fitting with `mstate` ... 163
Parametric extrapolation of transition-specific hazards with `flexsurv` ... 164
10.11 Reconstructing individual patient data from published Kaplan-Meier curves ... 164
The Guyot et al. (2012) reconstruction algorithm ... 165
Using `IPDfromKM` ... 165
10.12 Reporting extrapolation following NICE DSU TSD 14 and 21 ... 166
Chapter 11. Meta-Analysis ... 169
11.1 Fixed-effect and random-effects models ... 169
11.2 Heterogeneity and its quantification ... 172
11.3 Publication bias and small-study effects ... 175
11.4 Meta-regression ... 177
11.5 Meta-analysis of time-to-event outcomes ... 178
11.6 Diagnostic test accuracy meta-analysis ... 180
11.7 Meta-analysis of proportions and rates ... 183
Chapter 12. Indirect Treatment Comparisons and Network Meta-Analysis ... 185
12.1 Why indirect comparison is needed ... 185
12.2 Bucher method ... 187
12.2.1 The algebra of anchored indirect comparison ... 187
12.2.2 Why the variance adds ... 187
12.2.3 Worked example: TB regimen comparison ... 188
12.2.4 Assumptions the Bucher method relies on ... 189
12.2.5 When Bucher is enough, and when it is not ... 190
12.3 Frequentist NMA (netmeta package) ... 190
12.3.1 From anchored pairs to a network ... 190
12.3.2 Preparing contrast-level data: `pairwise()` ... 191
12.3.3 Fitting the network: `netmeta()` ... 191
12.3.4 Visualising the evidence network: `netgraph()` ... 192
12.3.5 Ranking treatments: `netrank()` ... 193
12.3.6 Full pairwise results: `netleague()` ... 194
12.4 Bayesian NMA (gemtc, multinma, bnma) ... 196
gemtc: the NICE DSU TSD 2 reference implementation ... 196
multinma: a unified Stan-based alternative ... 197
bnma: a lighter-weight contrast-based option ... 198
12.5 Consistency and inconsistency assessment ... 199
Node-splitting: direct vs indirect evidence on one edge ... 199
Design-by-treatment interaction: a global alternative ... 201
12.6 Network meta-regression ... 202
Mechanics in gemtc ... 202
Interpreting shrinkage of heterogeneity and inconsistency ... 203
12.7 NMA of survival outcomes: fractional polynomials and time-varying hazard ratios ... 206
Why constant hazard ratios are often the wrong assumption ... 206
The Jansen / TSD 18 fractional polynomial NMA approach, conceptually ... 206
Single-study building block with flexsurv, and the bespoke network step ... 207
12.8 Ranking, SUCRA and rankograms ... 210

12.9 Reporting per NICE DSU TSD 2, ISPOR and PRISMA-NMA 212
Chapter 13. Population-Adjusted Indirect Comparisons 214
13.1 When population adjustment is needed 215
13.2 Matching-adjusted indirect comparison (MAIC) 216
13.3 Simulated treatment comparison (STC) 219
13.4 Multilevel network meta-regression (ML-NMR) 222
13.5 Limitations, assumptions and agency expectations (NICE DSU TSD 18) 225
Chapter 14. Bayesian Methods for HEOR 229
14.1 Bayesian thinking for decision makers 230
14.2 Priors: informative, weakly informative and vague priors 231
14.3 MCMC with Stan, JAGS and brms 233
14.4 Diagnostics: convergence, effective sample size, and posterior predictive checks 237
Why convergence is not optional 237
Effective sample size: why 20,000 draws can be worth 200 238
Posterior predictive checks 239
14.5 Bayesian evidence synthesis as a pipeline 241
The general principle: synthesis produces a distribution, not a number 241
Why this matters more as networks get more complex 241
Where this fits in the overall workflow 242
14.6 Bayesian decision analysis: from posterior to PSA 242
Expected value under a posterior 242
Posterior draws as PSA input: the direct pipeline 243
Decision uncertainty is now a direct statement about the evidence 244
14.7 Expert elicitation methods (SHELF) 245
14.8 Bayesian health economic modelling workflows 248
Chapter 15. Real-World Evidence (RWE) and Causal Inference 251
15.1 Fit-for-purpose RWD and RWE frameworks 252
15.2 Study design: cohort, case-control, target trial emulation 254
15.3 Confounding and bias, with DAGs (`dagitty`, `ggdag`) 256
15.4 Propensity scores: matching, weighting, stratification (`MatchIt`, `WeightIt`, `cobalt`) 258
Matching with MatchIt 258
Weighting with WeightIt 259
Checking balance with cobalt 259
A worked numeric example: IPTW ATE by hand 260
15.5 Doubly robust estimators and g-methods 262
G-methods 263
15.6 Instrumental variables and regression discontinuity 265
Instrumental variables 265
Regression discontinuity 267
15.7 Difference-in-differences and interrupted time series 268
Difference-in-differences (DiD) 268
Interrupted time series (ITS) 271
15.8 Treatment switching and crossover adjustment (RPSFTM, IPCW) 272
15.9 E-values and quantitative bias analysis 274
15.10 OHDSI/OMOP workflows in R 276
15.11 Using external control arms and synthetic control arms 278
Chapter 16. Regression Models for HEOR Data 281

16.1 Generalised linear models 281
16.2 Mixed-effects and hierarchical models 283
16.3 Count data and zero-inflated models 285
16.4 Quantile and ordinal regression 287
16.5 Machine learning for prediction and heterogeneity of treatment effect 288
16.6 Subgroup analysis and effect modification 290

PART IV

Decision-Analytic Modelling

..... 293
Chapter 17. Principles of Decision Modelling 294
17.1 Model conceptualisation and problem scoping 294
17.2 Choosing a model type 296
17.3 Structural assumptions and model diagrams 297
17.4 Parameter taxonomy and distributions 298
17.5 Model documentation and transparency 300
17.6 Modelling workflow in R: inputs -> engine -> outputs -> reporting 302
17.7 Good practice guidelines (ISPOR-SMDM, NICE reference case) 303
Chapter 18. Decision Trees 305
18.1 Structure and expected-value calculations 305
18.2 Building decision trees from scratch in R 307
18.3 Using `rdecision` 309
18.4 Short-term models for screening, diagnostics and acute care 311
18.5 Combining a decision tree with a long-term Markov model 312
Chapter 19. Markov Cohort Models 314
19.1 States, transitions and cycles 314
19.2 Time-homogeneous and time-dependent transition matrices 316
19.3 Building a Markov model in base R (matrix-based approach) 316
19.4 Using heemod, hesim and dampack 318
19.5 Applying costs and QALYs, discounting and half-cycle correction 320
19.6 Tunnel states and state residence time 321
19.7 Age-dependent transitions and background mortality 323
19.8 Probabilistic transition matrices (Dirichlet) and Cholesky decomposition 324
19.9 Calibration and validation 325
19.10 Worked example: a chronic-disease CEA 326
Chapter 20. Partitioned Survival Models 329
20.1 Structure and when it is appropriate 329
20.2 Building PSMs from survival extrapolations 330
20.3 Treatment waning and relative-effect assumptions 331
20.4 Modelling with `hesim`, `flexsurv` and custom code 331
20.5 Limitations compared with state-transition models 332
20.6 Worked example: an oncology CEA 333
Chapter 21. Individual-Level State-Transition Models (Microsimulation) 336

21.1 When microsimulation is needed 336
21.2 Patient heterogeneity and history dependence 337
21.3 Efficient implementation: vectorisation, data.table, parallel computing 339
21.4 First-order versus second-order Monte Carlo uncertainty 340
21.5 Common random numbers 342
21.6 Using `hesim` for individual-level models 342
21.7 Worked example: a screening programme 344
Chapter 22. Discrete Event Simulation and Dynamic Models 348
22.1 Concepts: events, queues, resources and attributes 348
22.2 DES with `simmer` 350
22.3 Competing events and time-to-event sampling 352
22.4 Infectious disease transmission models 354
Compartmental models (SIR/SEIR) with `deSolve` 354
Agent-based models and `EpiModel` 356
Linking transmission models to economic evaluation 357
22.5 Choosing between Markov, DES and dynamic models 357
22.6 Worked example: a TB screening/preventive-therapy intervention model 359
Chapter 23. Budget Impact Analysis 362
23.1 Purpose and ISPOR guidance 363
23.2 Population, uptake and market-share structure 364
Eligible population 365
Market share: the reference versus new scenario 365
Uptake and diffusion over the time horizon 366
23.3 Building a budget impact model in R 368
23.4 Scenario analyses 370
23.5 Presenting results to payers 371
Chapter 24. Cost-Benefit, Distributional and Extended Cost-Effectiveness Analysis 374
24.1 Monetary valuation of health 374
Why monetize health at all 375
Value of a statistical life (VSL) 375
Human capital vs. willingness-to-pay approaches 376
24.2 Distributional CEA and equity weighting 378
Why the average can mask unequal distribution 378
Equity weighting and the equity-efficiency impact plane 379
A toy two-subgroup worked example 380
260, so the equity-weighted ICER is $4,000,000 / 260 = **$15,384.62 per QALY** (both figures independently verified by direct calculation, ... 382
24.3 Extended cost-effectiveness analysis: financial risk protection 384
24.4 Multi-criteria decision analysis (MCDA) in R 389
24.5 Severity modifiers, value-based pricing, and managed entry agreements 392

PART V

Uncertainty, Value of Information and Model Validation

. 396
Chapter 25. Handling Uncertainty 397
25.1 Types of uncertainty: parameter, structural, methodological and heterogeneity 397
Parameter uncertainty 398
Structural uncertainty 399
Methodological uncertainty 400
Heterogeneity – not a form of uncertainty 400
25.2 Deterministic sensitivity analysis: one-way, two-way, tornado plots 402
One-way sensitivity analysis 403
Two-way sensitivity analysis 403
Tornado diagrams 403
Worked example: one-way sensitivity analysis on the TB regimen-choice model 404
Worked example: two-way sensitivity analysis 407
Constructing the tornado diagram 408
25.3 Scenario analysis scripts 409
25.4 Probabilistic sensitivity analysis: choosing distributions 413
25.5 Visualising uncertainty: CE plane, CEAC, CEAF, NMB distributions 418
25.6 Threshold analysis 422
25.7 Structural uncertainty and model averaging 425
Two structures for the same decision problem 425
From scenario analysis to model averaging 427
25.8 Analysing patient-level cost-effectiveness trial data and bootstrapping 429
Why trial-based CEA is different 429
The non-parametric bootstrap 429
A parametric alternative, briefly 433
Chapter 26. Value of Information Analysis 434
26.1 Concepts: EVPI, EVPPI, EVSI and ENBS 434
26.2 Computing EVPI and EVPPI (BCEA, voi) 439
26.3 Efficient EVPPI via regression and GAM approximations 441
26.4 EVSI: moment-matching, importance sampling, Gaussian approximation 445
Moment-matching 445
Importance sampling 447
Gaussian approximation 447
26.5 Value of implementation 448
26.6 Using VOI to design research and prioritise data collection 451
Chapter 27. Model Calibration, Validation and Quality Assurance 455
27.1 Calibration methods: random search, Nelder-Mead, Bayesian calibration, ABC, IMIS . 455
Random search 456
Nelder-Mead optimization 457
Bayesian calibration 458
Approximate Bayesian Computation (ABC) 459
Incremental Mixture Importance Sampling (IMIS) 459
27.2 Cross-validation of model structure 460
27.3 Verification: unit testing with `testthat`, extreme-value tests, replication 461
Unit test: transition matrix row-sums equal 1 462
Extreme-value test 462
Replication 463

27.4 Validation frameworks (AdViSHE, TECH-VER, ISPOR-SMDM) 464
27.5 Code review and peer review practices 466
27.6 Face validity, internal, cross and external validity 467

PART VI

Reporting, Communication and Deployment

..... 470
Chapter 28. Reproducible Reporting 471
28.1 Quarto and R Markdown for HEOR reports 471
R Markdown and Quarto: what changed and what didn't 471
A worked example: rendering an HEOR summary report 473
28.2 Parameterised reports 474
Rendering a single parameterised report 475
Looping render calls to generate one report per scenario 475
28.3 Producing Word, PDF, HTML and PowerPoint outputs 476
PDF output and the LaTeX requirement 477
Native editable Word and PowerPoint elements with officer/officedown 478
28.4 Publication-ready tables (gt, gtsummary, flextable, kableExtra) 478
Worked example: an ICER summary table 479
28.5 Publication-ready graphics (ggplot2 themes, colour-blind-safe palettes) 481
A consistent custom theme 482
Colour-blind-safe palettes 482
Saving at print resolution 484
28.6 CHEERS 2022 reporting checklist implementation 485
Building a checklist-tracking table in R 486
28.7 Automating dossier and submission appendices 488
Structuring the dossier as child documents 488
28.8 Automating routine reports across programmes or districts 490
Looping parameterised rendering across districts 490
Scheduling the pipeline for recurring reporting 491
Chapter 29. Interactive Models and Web Applications with Shiny 493
29.1 Shiny fundamentals for model front-ends 493
29.2 Designing a user interface for non-programmers 495
29.3 Modular Shiny with `golem` and `rhino` 497
29.4 Linking Shiny to the model engine and PSA outputs 499
29.5 Downloadable reports and audit trails 501
29.6 Deployment: shinyapps.io, Posit Connect, Docker, servers 503
29.7 Security, validation and agency expectations for web-based models 506
29.8 Case study: a transparent, reviewable submission-ready model app 507
Chapter 30. R Packages and Pipelines for Model Development 511
30.1 Why package your model 511

30.2 Package structure with `usethis` and `devtools` 512
30.3 Documentation with `roxygen2` and `pkgdown` 514
30.4 Pipelines with `targets` and `renv` 515
30.5 Continuous integration with GitHub Actions 517
30.6 Sharing models with HTA agencies and reviewers 519
30.7 Open-source model licensing and the open-source HEOR movement 520
List of Figures 523
List of Tables 529

PART I

Foundations

Chapter 1. Introduction to HEOR and HTA

Learning objectives

- Define Health Economics and Outcomes Research (HEOR) and distinguish it from adjacent disciplines it is often confused with.
- Describe the global Health Technology Assessment (HTA) ecosystem, including the major national and regional agencies and India's HTAIn.
- Map the evidence needs a product generates across its lifecycle, from early pipeline decisions to post-launch re-assessment.
- List the typical analytical deliverables an HEOR team produces and who consumes each one.
- Articulate the case for R as a primary HEOR tool, including where it still lags proprietary alternatives.
- Locate R's role across the end-to-end HEOR workflow that the rest of this book builds on, chapter by chapter.

What HEOR is and what it is not

Health Economics and Outcomes Research (HEOR) is the discipline that generates and synthesises evidence on the clinical, economic and humanistic value of a health intervention, so that payers, clinicians, patients and policymakers can decide whether — and how — it should be funded, prescribed or reimbursed. It sits deliberately between three fields it borrows from but is not reducible to any one of them:

- **Health economics** proper, which studies how scarce health-care resources are allocated, using tools such as welfare economics, demand theory and public finance. HEOR narrows this to intervention- level value questions rather than system-level allocation theory.
- **Biostatistics and epidemiology**, which HEOR leans on heavily (survival analysis, meta-analysis, causal inference — all covered in Part III of this book) but HEOR's output is a decision-relevant value argument, not a hypothesis test in isolation.
- **Health services research**, which studies how care is organised and delivered; HEOR instead asks whether a specific technology is worth paying for, relative to its comparators, in a specific decision context.

A useful operational definition: HEOR produces the evidence base that answers "is this intervention worth it, for whom, compared to what, and under what degree of uncertainty?" Everything else in this book — R code, statistical models, decision trees, Shiny apps — exists only in service of answering that question more rigorously and more reproducibly.

> **Difficult terminology: welfare economics, demand theory, public finance.** These are the three parent disciplines health economics proper draws on, each a large field in its own right. **Welfare economics** studies how to judge whether a policy change makes society better or worse off overall — think of it as the branch of economics concerned with "who wins, who loses, and does the gain outweigh the loss." **Demand theory** models how much of something people buy at different prices (and how that changes as income or preferences shift)

— the supply-and-demand curves from an introductory economics course. **Public finance** studies how governments raise money (taxation) and decide how to spend it across competing priorities (roads, schools, hospitals). HEOR borrows selectively from all three but narrows its focus dramatically: rather than asking "how should a health system allocate its whole budget" (a public-finance question) or "how does demand for healthcare respond to price" (a demand-theory question), HEOR asks a much more contained question — "is **this one** intervention worth paying for, compared to what we'd otherwise do with that money?" You do not need to have studied any of the three to follow this book; they are named here only because an HTA dossier or health-economics textbook may invoke them as HEOR's intellectual ancestry.

Common pitfall. A common conflation, especially among analysts moving into HEOR from general biostatistics or health-services backgrounds, is to treat statistical significance (a p-value below 0.05 for a treatment effect) as equivalent to economic or decision-relevant significance. A statistically significant mean cost difference of $40 per patient across a 20,000-patient budget impact model, or a trial result extrapolated with a parametric survival curve well past the observed follow-up window, can drive a very different reimbursement recommendation than the p-value alone would suggest. HEOR's job is precisely this translation step — from clinical/statistical evidence to decision-relevant value — and it is where most of this book's methodological content concentrates.

Difficult terminology: parametric survival curve (extrapolation). A clinical trial only follows patients for a finite period — typically a few years at most — but a cost-effectiveness model usually needs to know what happens to survival, and therefore to costs and QALYs, over a patient's whole remaining lifetime. "Extrapolation" is the general term for projecting beyond what was actually observed. A **parametric** survival curve is one specific way to do this: instead of just plotting the raw, jagged step-function of who-died-when in the trial (a Kaplan-Meier curve, covered properly in Chapter 10), you fit a smooth mathematical formula — common choices are named after their shape, such as Weibull, log-normal or log-logistic — that describes how the risk of the event changes over time. Once that formula is fitted to the observed trial data, it can be evaluated at any time point you like, including years past the last observed patient, simply by plugging in a larger value for time. The practical analogy is a weather forecast: you have reliable five-day data, but you want a projection for month six, so you fit a model to the pattern you've seen and use it to project forward — the forecast is only as trustworthy as the assumption that the underlying pattern (the shape of the curve) continues to hold, which is exactly why HTA agencies scrutinise which parametric shape was chosen and demand sensitivity analyses across plausible alternatives.

HEOR is also not the same as **market access**, although the two functions collaborate closely in industry settings. Market access is the commercial and regulatory function that negotiates price, reimbursement status and formulary listing; HEOR supplies the evidence (economic models, systematic literature reviews, real-world evidence studies) that market access teams use in those negotiations and that agencies scrutinise in their technical appraisal.

The HTA ecosystem

Health Technology Assessment (HTA) is the formal, usually government-mandated or government-commissioned process by which a jurisdiction decides whether a health technology (a drug, device, diagnostic or procedure) should be covered by public or social insurance funds, and on what terms. HTA agencies are the primary institutional consumers of HEOR output, and the methodological choices in this book are shaped throughout by what these agencies expect to see in a submission.

The major agencies an HEOR analyst will encounter, and the general character of each:

- **NICE** (National Institute for Health and Care Excellence, England and Wales) runs one of the most methodologically prescriptive processes globally, publishing the NICE reference case and a detailed series of Decision Support Unit (DSU) Technical Support Documents (TSDs) that this book cites repeatedly — for survival extrapolation (TSD 14, TSD 21), network meta-analysis (TSD 2) and population-adjusted indirect comparisons (TSD 18), among others.
- **CADTH** (Canada's Drug Agency, historically the Canadian Agency for Drugs and Technologies in Health) performs a broadly similar reimbursement review function for Canada, with its own reference case and reanalysis culture.
- **PBAC** (Pharmaceutical Benefits Advisory Committee, Australia) advises the Australian government on Pharmaceutical Benefits Scheme listing, with a strong tradition of structured uncertainty and stepped economic evaluation.
- **IQWiG** (Institute for Quality and Efficiency in Health Care) and the **G-BA** (Federal Joint Committee) jointly run Germany's early benefit assessment (AMNOG) process, which emphasises patient- relevant outcomes and additional-benefit categorisation ahead of price negotiation.
- **HAS** (Haute Autorité de Santé, France) assesses clinical and economic value (via the Transparency Committee and the Economic and Public Health Evaluation Committee) feeding into price negotiation with the French health insurance system.
- **ICER** (Institute for Clinical and Economic Review, United States) is a non-government, foundation-funded body whose cost-effectiveness reports carry significant influence over US payer and pharmacy-benefit-manager decisions despite having no statutory authority.
- **HTAIn** (Health Technology Assessment in India), housed under the Department of Health Research, Ministry of Health and Family Welfare, is the body most directly relevant to India-based HEOR analysts: it issues an Indian reference case for economic evaluation and informs technology inclusion decisions under schemes such as Ayushman Bharat Pradhan Mantri Jan Arogya Yojana (PM-JAY). HTAIn's methodological guidance is still maturing relative to NICE or CADTH, which in practice means Indian HEOR analyses often triangulate HTAIn's own reference case with ISPOR good-practice guidance and precedent from more established agencies — a pattern Chapter 31 revisits in depth.

Agency	Jurisdiction	Threshold / evaluative approach	Key reference document
NICE	England and Wales	Explicit reference-case, methodologically prescriptive	NICE reference case; DSU TSDs (e.g. TSD 14, TSD 21, TSD 2, TSD 18)
CADTH	Canada	Reference case with reanalysis culture	Guidelines for the Economic Evaluation of Health Technologies
PBAC	Australia	Stepped economic evaluation, strong uncertainty tradition	PBAC Guidelines
IQWiG / G-BA	Germany	Early benefit assessment (AMNOG), patient-relevant outcomes	IQWiG General Methods
HAS	France	Clinical + economic value via dedicated committees	HAS Economic Evaluation Guidance
ICER	United States	Non-government cost-effectiveness reports, no statutory authority	ICER Value Assessment Framework
HTAIn	India	Indian reference case, still maturing	HTAIn Reference Case for Economic Evaluation

Note. Since 2025, EU Joint Clinical Assessment adds a shared clinical-assessment layer; national bodies above still perform their own economic/pricing appraisal.

Table 1: Major HTA agencies/bodies an HEOR analyst routinely encounters, their jurisdiction, general evaluative stance, and a key reference document cited elsewhere in this book.

Agency perspective. Since 2025 the **EU Joint Clinical Assessment** (EU HTA Regulation, in force from January 2025 for oncology and advanced-therapy medicinal products, with wider scope phasing in through 2030) introduces a single European Union-level clinical assessment report that national HTA bodies must then take into account in their own (still separate) appraisals of cost-effectiveness and pricing. This decouples joint **clinical** assessment from still-national **economic** assessment — a structural distinction every analyst preparing a European submission now needs to track, because it changes which evidence goes into which document and on what timeline.

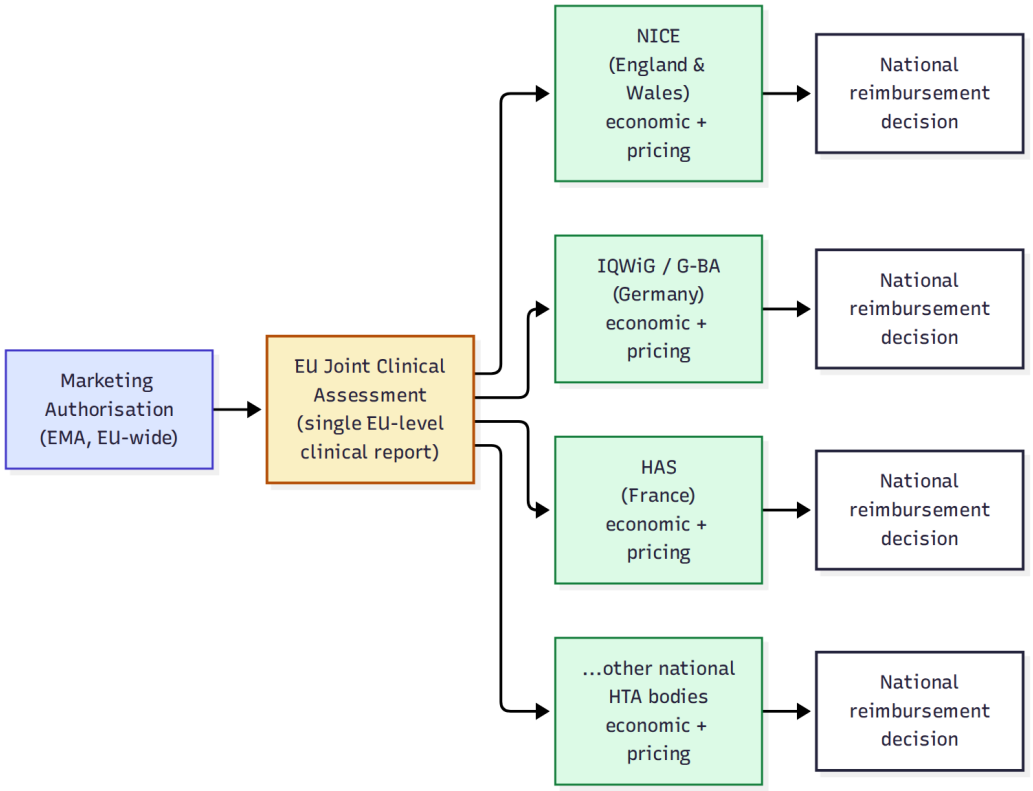


Figure 1: Since 2025, the EU Joint Clinical Assessment produces a single shared clinical-assessment report that feeds into multiple still-separate national economic and pricing appraisals — each national body (NICE, IQWiG/G-BA, HAS, and others) runs its own economic evaluation and reaches its own reimbursement decision from the same upstream clinical evidence base.

A single technology can therefore face half a dozen or more functionally distinct appraisal processes across markets, each with its own reference case, discount rate convention, preferred comparator set and willingness-to-pay stance — one of the strongest practical arguments for building economic models in a flexible, scriptable, re-parameterisable tool rather than a single fixed spreadsheet.

Evidence needs across the product lifecycle

HEOR evidence generation is not a one-off event triggered by a launch date; it is continuous across a technology's life, and the type of evidence that is credible, available and decision-relevant changes at each stage:

1. **Early pipeline / pre-clinical and Phase I-II**: indication selection, early value proposition modelling, target product profile cost-effectiveness thresholds, often using modelled (not yet observed) efficacy assumptions.
2. **Phase III / pre-launch**: pivotal trial-based economic models, systematic literature reviews and network meta-analyses to position the asset against comparators, early health-related quality-of-life (utility) evidence collection alongside the trial.
3. **Launch / initial HTA submission**: the full value dossier — economic model, budget impact model, systematic review, and (where relevant) indirect treatment comparison — submitted to one or more HTA agencies and payers.

4. **Post-launch**: real-world evidence studies to confirm or update trial-based assumptions (particularly long-term survival extrapolation and real-world adherence/persistence), responses to evidence review group critiques, managed entry agreement data collection.
5. **Re-assessment / loss of exclusivity**: re-appraisal as new comparators enter, as biosimilars or generics launch, or as a scheduled HTA review cycle requires updated modelling.

Lifecycle stage	Characteristic evidence generated	R workflow emphasis
Early pipeline / Phase I-II	Indication selection, early value proposition, target product profile thresholds	Quick decision-tree/ Markov prototypes (Ch. 18-19)
Phase III / pre-launch	Pivotal trial-based models, SLRs/ NMAs, early utility collection	Evidence synthesis inputs (Part III)
Launch / initial HTA submission	Full value dossier: economic model, BIM, SLR, ITC	Validated, documented, PSA-ready models (Ch. 25, 27)
Post-launch	RWE studies, ERG critique responses, managed entry agreement data	RWE pipelines, causal inference (Ch. 15)
Re-assessment / LOE	Re-appraisal against new comparators, biosimilars/generics	Updated modelling on existing codebase

Table 2: Evidence generation and typical analytical deliverables by product lifecycle stage.

This sequencing matters for a very practical reason covered throughout this book: at each stage the **type** of R workflow needed differs — early-stage work favours quick, assumption-heavy decision-tree or simple Markov prototypes (Chapters 18-19); launch submissions demand fully validated, documented, PSA-ready models (Chapters 25, 27); post-launch work leans on real-world data pipelines and causal inference methods (Chapter 15) that a pre-launch team may never have needed.

Typical deliverables

An HEOR function, whether inside a pharmaceutical company, a consultancy, a government technical unit or an academic health economics group, typically produces some combination of:

- **Global value dossiers** — the master evidence package (AMCP format or similar) from which country-specific submissions are adapted.
- **Economic models** — cost-effectiveness or cost-utility models (decision trees, Markov models, partitioned survival models, microsimulations — Part IV of this book) that estimate incremental cost per QALY, DALY averted, or other outcome of interest.
- **Budget impact models** — separate from cost-effectiveness models, these forecast the net financial impact on a payer's budget over a defined horizon (Chapter 23), and are frequently the deciding document in resource-constrained settings even when the cost-effectiveness case is favourable.

- **Systematic literature reviews and network meta-analyses** — the evidence synthesis backbone for both the clinical case and the economic model's comparative-effectiveness inputs (Chapters 6, 11-13).
- **Real-world evidence (RWE) studies** — observational analyses using registries, claims or electronic health record data to answer questions trials cannot (Chapter 15).
- **Payer submissions and technical engagement responses** — the formatted documents and rebuttals an agency's evidence review process demands (Chapters 28, 31).

> **Difficult terminology: microsimulation.** Most economic models described so far implicitly track a **cohort** — an average, representative group of patients moving through health states together, where everyone in the "progressed disease" state at year 3 is treated as identical for costing and QALY purposes. A microsimulation instead tracks many individual, simulated patients one at a time, each with their own randomly drawn characteristics and their own randomly drawn sequence of events (does **this** particular patient progress this cycle, or not, based on a random draw against their personal risk?). Run enough individual patients through the model and average their outcomes, and you recover the same population-level answer a cohort model would give — but along the way you also capture patient-to-patient variability (some patients do much better or worse than the average) and can model things a cohort average cannot, such as a patient's own history affecting their future risk. A useful analogy is a flight simulator running thousands of individual passenger journeys with their own delays and connections, rather than reporting only the airline's average on-time performance: both describe the same system, but the individual-level version answers questions ("what fraction of passengers miss their connection?") the average alone cannot.

Tuberculosis programme data quality and analysis for India's National TB Elimination Programme, and workforce-training initiatives built around it, sit most naturally against the budget impact, programme- cost and dynamic/infectious-disease modelling deliverables (Chapters 22-23, 32.7), which this book returns to with TB-specific worked examples wherever the general method permits it.

Why R: the case for open-source and reproducible HEOR

For much of HEOR's history, the dominant modelling tools were Excel (ubiquitous, transparent to reviewers cell-by-cell, but fragile and hard to validate at scale), TreeAge Pro (a widely used commercial decision-modelling GUI), and SAS (entrenched in regulated clinical and RWE pipelines, particularly in the US). R has become a credible, and increasingly preferred, alternative for several concrete reasons:

- **Reproducibility by construction.** An R script, paired with `renv` package-version locking (Chapter 2) and version control (Git), gives an exact, re-runnable record of every assumption and transformation — something a large interlinked Excel workbook with manual copy-paste steps cannot reliably guarantee.
- **Statistical depth.** R's package ecosystem for survival analysis (`flexsurv`), meta-analysis (`meta`, `metafor`, `netmeta`), Bayesian modelling (`brms`, Stan) and causal inference (`MatchIt`, `WeightIt`) is both broader and more actively maintained by the academic biostatistics community than equivalent capability inside TreeAge or Excel, and typically ahead of what is readily available in SAS procedures for newer methods.

- **Cost.** R, RStudio/Positron and essentially the entire package ecosystem used in this book are free and open source, compared to substantial per-seat licensing for TreeAge or SAS — a material consideration for government technical units, academic groups and LMIC HTA bodies including HTAIn-adjacent work.
- **Transparency for reviewers.** A well-structured R project (Chapter 2) lets an external reviewer or evidence review group trace every number in a submitted model back to a specific line of code and a specific input — the standard increasingly expected by NICE's DSU and by open-science-minded reviewers more broadly.
- **Interoperability with reporting.** Quarto and R Markdown (Chapter 28) let the same codebase that runs the model also generate the Word, PDF, HTML or PowerPoint submission documents, removing an entire category of transcription error between "the model" and "the report."

None of this means R is strictly superior in every respect. TreeAge remains attractive for teams who need a point-and-click interface for non-programming stakeholders and fast prototyping without a coding layer; SAS retains deep institutional entrenchment in regulated clinical trial pipelines where validated, audited SAS macros already exist; and Python has a comparably open-source, reproducible profile to R, with particular strength in machine learning and large-scale data engineering (Chapter 34 returns to where Python-R interoperation and newer ML methods fit into an HEOR R workflow).

> **HEOR in practice.** Several HTA agencies now explicitly accept, and in some cases prefer, submissions built with open-source tools, provided the model is adequately documented and validated. NICE's DSU has published R-based worked examples for survival extrapolation (TSD 21) and other technical guidance; several published NICE single-technology appraisals have used R-built partitioned survival and Markov models rather than Excel or TreeAge. The deciding factor for agency acceptance is not the tool itself but whether the model satisfies the agency's validation and transparency expectations (Chapter 27) — R makes that easier to demonstrate, but does not make it automatic.

Where R fits in the HEOR workflow

The remainder of this book is organised around a simple mental model of the HEOR workflow, and each part maps onto a stage of it:

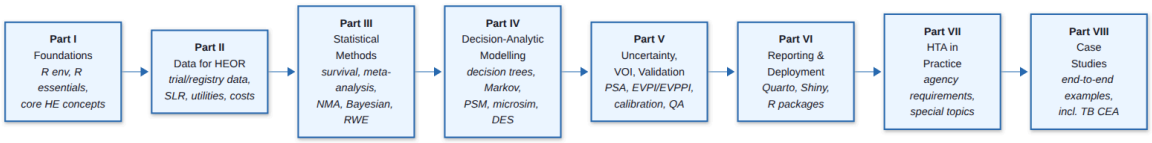


Figure 2: The book's eight-part structure as a cumulative HEOR workflow. Each part assumes fluency with everything upstream of it.

1. **Foundations** (Part I, this part): the R environment, R language essentials, and the core health-economic concepts (QALYs, ICERs, discounting) that every later chapter assumes.
2. **Data** (Part II): getting trial, registry, claims, literature, utility and cost data into a clean, analysis-ready state — the unglamorous but decisive step that determines whether everything downstream is trustworthy.

3. **Statistical methods** (Part III): survival analysis, meta-analysis, indirect comparisons, Bayesian methods and causal inference — the methods that convert raw data into the clinical and comparative- effectiveness parameters a model needs.
4. **Decision-analytic modelling** (Part IV): the model structures themselves — decision trees, Markov cohort models, partitioned survival models, microsimulation, discrete event simulation, budget impact and distributional/extended CEA — that combine those parameters into decision-relevant outputs.
5. **Uncertainty, VOI and validation** (Part V): quantifying how confident we should be in a model's conclusions, and demonstrating that confidence is warranted through calibration and validation.
6. **Reporting, communication and deployment** (Part VI): turning a validated model into a submission-ready, reviewable, and where useful interactive (Shiny) artefact.
7. **HTA in practice** (Part VII): agency-specific requirements, special therapeutic-area considerations, and patient-preference methods that do not fit cleanly into the generic workflow above.
8. **Case studies** (Part VIII): end-to-end worked examples, including a TB programme cost-effectiveness analysis for an LMIC setting, tying every preceding part together in one script-driven project.

Difficult terminology: discrete event simulation. A Markov cohort model (and, usually, a microsimulation built on top of one) advances time in fixed, regular steps — a "cycle" — typically a month or a year, checking at every single cycle boundary whether each patient has moved to a new health state. A discrete event simulation instead advances time irregularly, jumping straight to whenever the **next** thing actually happens (the next relapse, hospital admission, or death, whichever comes first for whichever patient), rather than re-checking everyone on a fixed clock. This matters most when events are rare and irregularly timed, or when the model needs to represent shared, limited resources — a fixed number of hospital beds or specialist appointment slots that patients queue for — something a simple fixed-cycle cohort model struggles to represent naturally. Think of the difference between a factory manager who inspects every machine on the hour regardless of what is happening (a fixed-cycle approach) versus one who is alerted the instant any machine actually breaks down or finishes a job (an event-driven approach); the second is more efficient and more realistic when events are sporadic rather than evenly spaced.

Difficult terminology: distributional / extended cost-effectiveness analysis. A standard cost-effectiveness analysis produces one headline number — the population-average cost per QALY gained — which can quietly hide very different results for different groups within that population. A distributional (or "extended") cost-effectiveness analysis deliberately breaks that average apart, reporting costs, health gains and sometimes financial-risk-protection effects separately by subgroup: by income quintile, by region, by disease severity, or by insurance status, for example. The point is to let a decision-maker see **who** gains and **who** pays, not just whether the average trade-off looks favourable — a programme can look excellent on average while concentrating nearly all of its benefit in one already-advantaged group, which a single pooled ICER would never reveal. A simple analogy: a company's average employee salary can rise even if every individual salary stayed flat, just because a few new high earners joined — the average conceals the distribution. Extended CEA is the health-economics equivalent of

> insisting on seeing the full salary breakdown by group, not just the company-wide average, before judging whether a policy is actually equitable.

Each part builds cumulatively on R skills and HEOR concepts introduced earlier — by the time this book reaches decision-analytic modelling in Part IV, it assumes fluency with the data-wrangling and statistical foundations from Parts I-III, rather than re-explaining them.

Chapter 2. Setting Up an R Environment for HEOR

Learning objectives

- Install and configure R, an IDE (RStudio or Positron), and Quarto for reproducible HEOR reporting.
- Structure a new HEOR analysis project so that raw data, code, and outputs stay cleanly separated and auditable.
- Use `renv` to pin exact package versions so a model can be reproduced byte-for-byte years after submission.
- Set up Git/GitHub version control as both an audit trail and a lightweight QA mechanism mirroring agency technical review.
- Install the C++ toolchain required for Bayesian modelling with Stan/JAGS, and avoid the installation pitfalls that most commonly block analysts moving into Bayesian HEOR work.
- Understand when Docker is worth the overhead for a submission-critical model, versus when `renv` alone is sufficient.
- Navigate the R package ecosystem for HEOR by task area, with forward references to the chapters that cover each package in depth.

2.1 Installing R, RStudio/Positron and Quarto

Every HEOR analysis in this book rests on three pieces of software: R itself, an integrated development environment (IDE), and Quarto for turning analysis into submission-ready documents. Installing these correctly, and understanding the role each plays, prevents a surprising fraction of the reproducibility headaches that plague health-economic modelling teams.

R is distributed through the Comprehensive R Archive Network (CRAN) at `cran.r-project.org`. On Windows, download and run the `.exe` installer; on macOS, use the `.pkg` installer matching your chip architecture (Apple Silicon vs Intel); on Linux, prefer your distribution's package manager or the CRAN apt/yum repositories over a source build, since these handle dependency resolution and security updates automatically. Install the latest patch release of the current major R version unless a specific project mandates otherwise — version pinning is handled later, at the project level, by `renv` (Section 2.3), not by freezing your system R.

For years, RStudio Desktop was the default IDE for R users, and it remains a fully capable, mature choice: integrated console, environment pane, plotting pane, and tight Quarto/R Markdown support. Posit (RStudio's parent company) has since released **Positron**, a VS Code–based IDE built on the same open extensibility model as Visual Studio Code, but with first-class R and Python data-science support built in rather than bolted on. For HEOR work specifically, Positron is gaining traction for several concrete reasons:

- Many HTA submissions now combine R (statistical/economic modelling) with Python (machine-learning-based survival extrapolation, web-scraping literature data, or interfacing with external

APIs) in the same project — Positron handles both languages natively in one workspace, without the friction of RStudio's R-first design.
- It inherits the entire VS Code extension ecosystem: Git integration, remote development over SSH, Docker/dev-container support, and GitHub Copilot-style tooling, which matter increasingly for teams working under version control and CI pipelines.
- Its data viewer, variable inspector, and plot pane are drop-in equivalents to RStudio's, so the learning curve for an RStudio user is shallow.

Positron is still younger than RStudio and a handful of RStudio-specific add-ins have no Positron equivalent yet, so this book treats either IDE as acceptable; screenshots and keybinding references favour RStudio terminology where the two diverge, but all code runs identically in both.

Quarto is Posit's successor to R Markdown and ships either bundled with RStudio/Positron or as a standalone CLI installable from `quarto.org`. It compiles `.qmd` documents mixing prose and executable R (or Python, Julia, Observable JS) code chunks into HTML, PDF, Word, or slide formats — critical for HEOR, where a single analysis often must be rendered as an internal technical report, a submission-ready Word document for a HTA body, and a results dashboard, all from one source file. Confirm your installation with:

```
quarto::quarto_version()
```

> **HEOR in practice.** Standardise on one Quarto version and one R version per **project**, not per analyst's machine. Record both in your project's README and, as Section 2.3 shows, let `renv` pin the R package versions precisely — the Quarto and R engine versions themselves should be noted explicitly since `renv` does not manage them.

2.2 Project structure for HEOR analyses

A consistent folder structure is not bureaucratic overhead — it is the single cheapest thing you can do to make an analysis auditable, handoff-ready, and defensible under HTA scrutiny. HEOR deliverables are frequently revisited months or years after initial submission (in response to reviewer queries, re-submissions, or country adaptations), often by someone other than the original analyst. A predictable layout means that person — possibly future you — can find the raw inputs, the code that transformed them, and the exact outputs without reverse-engineering a one-off script.

A structure that has proven robust across cost-effectiveness models, budget-impact analyses, and systematic literature reviews looks like this:

```
my-heor-project/
├── data-raw/        # original, untouched source files (CSV exports, trial
extracts)
├── data/            # cleaned, analysis-ready datasets (derived, never hand-
edited)
├── R/               # reusable functions (not top-level analysis scripts)
├── analysis/        # scripts/Quarto docs that run the actual analysis
├── output/          # figures, tables, rendered reports — fully disposable/
regenerable
├── renv/            # renv's private package library (Section 2.3)
├── renv.lock        # the package manifest
└── my-heor-project.Rproj
```

The rationale behind each separation matters more than the names themselves:

- **`data-raw/` is immutable.** Nothing in this folder is ever edited by hand or overwritten by a script. If a trial extract or vendor file changes, a new file (or new version) is added — the folder is a permanent, auditable record of what came in. Cleaning logic lives in code under `R/`, not in the data itself.
- **`data/` is derived, not authoritative.** Everything here should be fully reproducible by re-running the cleaning scripts against `data-raw/`. If `data/` were deleted entirely, the project should be able to regenerate it byte-for-byte.
- **`R/` separates logic from execution.** Functions used by more than one analysis script (cost calculations, utility mapping, plotting themes) belong here, loaded via `source()` or, for larger projects, packaged with `devtools::load_all()`. This avoids copy-pasted logic drifting out of sync across scripts.
- **`analysis/` contains the actual narrative** — Quarto documents or scripts that call functions from `R/` against data from `data/` and produce results. Each file here should, in principle, run top-to-bottom on a clean machine and reproduce its own output.
- **`output/` is disposable.** Nothing here is a source of truth; it is always regenerable by re-running `analysis/`. Treating output as disposable is what keeps the project honest — if you can't regenerate a table from scratch, something in the pipeline is undocumented.

A convenient starting scaffold is `usethis::create_project()`, which creates the `.Rproj` file, a sensible `.gitignore`, and an R-project-aware working directory in one call:

```
usethis::create_project("~/projects/my-heor-project")
```

From there, manually add the `data-raw/`, `data/`, `analysis/`, and `output/` directories (a small helper function wrapping `dir.create()` calls is worth writing once and reusing across projects).

> **Common pitfall.** Do not defer this structure until "the analysis is more final." The most common failure mode is a single sprawling script that reads a spreadsheet, cleans it, models it, and plots it in one pass — impossible to audit, and nearly impossible to hand off. Separating concerns from the first commit costs minutes; retrofitting it onto a 2,000-line script before a submission deadline costs days.

2.3 Package management with `renv`

R's package ecosystem evolves continuously, and HEOR analyses routinely have a shelf life of years — a submitted cost-effectiveness model may be queried, re-analysed, or audited long after its packages have moved on to incompatible new versions. Without intervention, simply running `install.packages("survival")` updates the package for **every** project on your machine, silently changing numerical behaviour (confidence interval methods, extrapolation defaults, random number handling) in ways that can alter results computed from the exact same code. `renv` solves this by giving every project its own private, isolated package library and a lockfile recording exact versions.

Initialise `renv` once per project, from the project root:

```
renv::init()
```

This scans your scripts and Quarto documents for `library()`/`require()` calls, builds a private project library under `renv/library/`, and writes an initial `renv.lock` file. From that point, package installs made inside the project (`install.packages()`, `remotes::install_github()`, etc.) are captured in that private library and do not affect your system-wide R installation or other projects.

As the analysis develops and you add or update packages, record the current state with:

```
renv::snapshot()
```

`snapshot()` inspects every package actually used in the project and writes their exact versions — plus, where resolvable, their source (CRAN, Bioconductor, GitHub, with commit hash) — into `renv.lock`. Conversely, to recreate the **exact** environment the lockfile describes, on a new machine or a colleague's laptop, run:

```
renv::restore()
```

`restore()` reads `renv.lock` and installs precisely those package versions into the project's private library, regardless of what is currently installed elsewhere on the system.

The `renv.lock` file itself is plain JSON, version-controllable alongside your code, and human-readable: it records the R version the project was built under, the package repositories consulted, and for each package its name, version, and source hash. A fragment looks like:

```
# (illustrative structure, not a function call)
# {
#   "R": { "Version": "4.3.2" },
#   "Packages": {
#     "survival": { "Package": "survival", "Version": "3.5-7",
#                   "Source": "Repository", "Repository": "CRAN" }
#   }
# }
```

Agency perspective. NICE's Decision Support Unit has published explicit technical support documents (the DSU TSD series referenced throughout this book, e.g. TSD 14 and TSD 21 for survival extrapolation in Chapter 10) that assume a reviewer can inspect and re-run the submitted model's code — NICE's own technology appraisal process guide requires companies to submit the executable model itself, not only a written description of methods, specifically so the Evidence Review Group can verify calculations independently. CADTH's Guidelines for the Economic Evaluation of Health Technologies similarly requires an electronic copy of the model. Neither agency mandates R or `renv` specifically, but both mandate that **whatever** tool is used be inspectable and re-runnable by the reviewing team — which is precisely the gap `renv.lock` closes for an R-based submission. Committing `renv.lock` alongside your analysis code is what makes "re-run this years later" operationally possible, not just a nice-to-have.

Common pitfall. The most common `renv` mistake is treating it as an optional add-on for "later," adopted only once a project is already underway. By the time collaborators have installed ad-hoc package versions by hand, `renv::init()` inherits an already-inconsistent environment, and reconciling it is far more work than starting clean. Run `renv::init()` as the **first** command in a new HEOR project, before a single `install.packages()` call — the

discipline costs nothing up front and prevents an entire category of "it worked on my machine" failures later.

Finally, note that `renv` manages R packages, not the R interpreter version, system libraries, or Quarto itself — pair it with the version documentation practice recommended in Section 2.1 for full environment reproducibility.

2.4 Version control with Git and GitHub

Health economic models are not static deliverables; they are living software artifacts that get re-run dozens of times as evidence changes, reviewers request scenario analyses, and submission deadlines force last-minute parameter updates. Without version control, "the model" quickly fragments into a folder of files named `markov_model_v3_final_FINAL_usethisone.R`, with no record of which assumptions changed between versions or who approved them. Git solves this by tracking every change as a discrete, attributable commit, and GitHub (or an equivalent host) adds the collaborative layer — issues, pull requests, and review history — that turns a personal habit into a team process.

For HEOR work specifically, version control earns its keep in three ways. First, **auditability**: reimbursement submissions to bodies such as NICE, CADTH, or ICER (the US nonprofit, not the cost-effectiveness ratio) are frequently re-opened months later when a reviewer asks "what if the discount rate were 1.5% instead of 3.5%?" A clean commit history lets you answer "what changed between the submitted model and this scenario" with `git diff`, rather than by memory. Second, **branching for scenario analysis**: each deterministic or structural sensitivity analysis — a different time horizon, an alternative comparator, a revised utility source — belongs on its own branch, built from the same validated base model. This keeps the base case stable while exploratory work proceeds in parallel, and it makes clear which scenarios were ever merged back into the primary analysis. Third, **pull requests as quality control**: requiring every change to a submission-track model to go through a PR — with a named reviewer checking the diff line by line — mirrors the technical review step that payer bodies themselves apply to submitted models. It catches hard-coded values, accidental formula changes, and unreviewed shortcuts before they reach a regulator.

HEOR in practice. Treat your Git workflow as a lightweight mirror of agency technical review: no commit to the `main`/`submission` branch without an approving PR from a second analyst. Tag the exact commit hash used to produce the submitted report's figures (`git tag submission-v1 <hash>`) so the evidentiary model can always be reconstructed byte-for-byte, even after further development continues on other branches.

The basic workflow for a new HEOR project is unremarkable but worth stating explicitly, since many analysts come from a non-software background:

```
git init
git add .
git commit -m "Initial commit: project scaffold and renv lockfile"
git checkout -b scenario/alt-discount-rate
# ... make changes ...
```

```
git add R/run_model.R
git commit -m "Add 1.5% discount rate scenario per reviewer request"
git push origin scenario/alt-discount-rate
# open a pull request on GitHub for review before merging
```

GitHub's value beyond raw Git is largely social: it gives external statisticians, academic collaborators, or a payer's technical reviewers a place to comment on specific lines of code, request changes, and approve a merge without needing filesystem access to your machine. Private repositories (or an enterprise GitHub/GitLab instance, which many HTA consultancies and pharmaceutical sponsors require for data governance reasons) support this collaboration while keeping the codebase restricted to named contributors.

A `.gitignore` file needs particular care in this domain. Renv's `renv.lock` file should always be committed — it is the machine-readable record of package versions that makes the environment reproducible and is small, plain-text JSON. By contrast, the `renv/library` folder (the actual installed packages) and any `.Rproj.user` or `.Rhistory` files should be excluded, since they are large, machine-specific, and regenerable from the lockfile.

> **Common pitfall.** Never commit real patient-level data, even temporarily, even on a private repository. A patient-level dataset pulled into a project folder for a quick test and then "removed" in a later commit still exists in Git's history — `git log` and `git show` can retrieve it from any prior commit unless the history itself is rewritten and force-pushed, which is destructive and easy to get wrong. Add `data/raw/`, `*.csv` (or more targeted patterns for your actual raw-data paths), and any patient-level extracts to `.gitignore` from the very first commit, before any such file is ever staged. If synthetic or de-identified data is needed for the repository, generate it programmatically and label it unambiguously as synthetic in the filename and header.

2.5 Working with Stan, JAGS and C++ toolchains

From Chapter 14 onward, this book moves into Bayesian methods — probabilistic sensitivity analysis built on posterior distributions, network meta-analysis, and survival extrapolation models that borrow strength across data sources via hierarchical structure. These techniques are typically fit not with a built-in R function but with a dedicated probabilistic programming language: Stan or, in older or legacy codebases, JAGS (Just Another Gibbs Sampler). Both let you write the statistical model directly as a set of distributional statements and delegate the hard work of sampling from the posterior to an efficient external sampler — Hamiltonian Monte Carlo (via the No-U-Turn Sampler) in Stan's case, and Gibbs sampling in JAGS's case.

> **Difficult terminology: posterior distribution.** In Bayesian statistics, you start with a **prior** — your belief about a parameter (say, a treatment's true effect size) before looking at the current data, which might be deliberately vague ("anywhere from a large harm to a large benefit is plausible") or informed by previous studies. You then combine that prior with the data actually observed in your trial or dataset, using the rules of probability, to get an updated belief: the **posterior distribution**. It is not a single number but a full distribution — a range of plausible values for the parameter, each with an associated probability — because Bayesian

inference treats the parameter itself as uncertain and quantifies exactly how uncertain, before and after seeing data. A simple analogy: if your prior is "this coin is probably close to fair" and you then flip it 20 times and see 15 heads, your posterior shifts the diagnosis-of-fairness towards a biased coin, by an amount that depends on how strong your prior was and how much data you collected. The posterior is usually too mathematically complex to compute an exact formula for in any realistic health-economic model, which is exactly why dedicated samplers like Stan and JAGS exist — they approximate the posterior by drawing thousands of representative random samples from it rather than solving it analytically.

Difficult terminology: hierarchical structure ("borrowing strength"). A hierarchical (or "multi-level") model structure is used when your data naturally comes in related groups — for example, several different trials of similar drugs, or several hospitals reporting the same outcome. Rather than treating each group as completely separate (ignoring that they are related) or pooling all the data together as if the groups didn't exist (ignoring that they might genuinely differ), a hierarchical model does something in between: it estimates a group-specific effect for each group, but constrains those group estimates to be drawn from a shared, overarching distribution that the model also estimates. The practical consequence is "borrowing strength" — a group with very little data of its own (a small trial, or a hospital with few cases) gets pulled gently towards the overall average, because the model recognises it as one draw from the same underlying family as the data-rich groups, instead of relying purely on its own sparse, noisy numbers. A classroom analogy: if you want to estimate how good each of thirty schools' maths teaching is, but some schools only have five students' scores while others have five hundred, a hierarchical model lets the small schools' estimates "borrow" precision from the overall district pattern, rather than wildly over- or under-stating how good that one small school's teaching really is based on five noisy data points alone.

Difficult terminology: Hamiltonian Monte Carlo / No-U-Turn Sampler (NUTS). Sampling from a posterior distribution (see the box above) means drawing many representative random values from it so you can summarise what the model believes, once data has been accounted for. Hamiltonian Monte Carlo (HMC) is a particular, physics-inspired algorithm for doing this efficiently in models with many parameters: rather than wandering around the parameter space in small, nearly random steps (which is slow and can get stuck), it simulates a "puck" sliding across the probability landscape as if under gravity and momentum, following smooth, informed trajectories that explore the most probable regions much faster. The one dial every HMC implementation needs is how far each simulated trajectory should run before stopping; get this wrong and the sampler either wastes time retracing its own path or stops too early to be useful. The No-U-Turn Sampler (NUTS) is the clever auto-tuning addition that solves this: it monitors the simulated trajectory and automatically stops it the moment it starts curving back on itself (making a "U-turn"), so the step length is chosen automatically rather than by manual trial and error. You do not need to understand the physics to use Stan, which implements HMC/NUTS under the hood — the practical takeaway is simply that this is why Stan is usually faster and more reliable than older samplers for complex Bayesian HEOR models, with little manual tuning required.

Difficult terminology: Gibbs sampling. Gibbs sampling is an older, simpler approach to the same sampling problem that Hamiltonian Monte Carlo solves (see above): drawing representative random values from a posterior distribution with several unknown parameters at once. Instead of moving all parameters together, it updates them one at a time, in turn: hold every other parameter fixed at its current value, draw a new random value for just one parameter conditional on those fixed values, then move on to the next parameter and repeat, cycling through all of them over and over. Over many such cycles, the sequence of values produced converges to representative draws from the full joint posterior. It is conceptually simple to implement and was, for a long time, the workhorse behind BUGS and JAGS-style Bayesian software in health economics and epidemiology, but it can be noticeably slower to explore complex models than Hamiltonian Monte Carlo/NUTS, particularly when parameters are strongly correlated with each other — one reason newer R-based Bayesian HEOR work has shifted increasingly towards Stan.

The practical consequence for an R-based HEOR workflow is that Stan model code is compiled to C++ before it runs. The R packages `rstan` and `cmdstanr` are interfaces that generate this C++, hand it to a compiler, and then call the resulting executable — they are not pure-R implementations. This means your machine needs a working C++ toolchain, which is rarely installed by default and is the single most common installation obstacle analysts moving from base R or `survival`/`flexsurv`-style modelling report when they first try Bayesian HEOR work.

The toolchain requirement differs by operating system:

- **Windows**: requires Rtools, a separate installer (matched to your R version, e.g. Rtools42 for R 4.2.x) that provides a `gcc`-based compiler and the Unix-like build utilities R's compilation toolchain expects. It is not installed by running `install.packages()`; it is a standalone download from CRAN's Rtools page and must be on the system `PATH`.
- **macOS**: requires the Xcode Command Line Tools, installed with `xcode-select --install` from a terminal, which provides `clang` and related build tools without requiring the full Xcode IDE.
- **Linux**: requires the standard GNU build toolchain, typically installed via the distribution's package manager, e.g. `sudo apt-get install build-essential` on Debian/Ubuntu, which supplies `gcc`, `g++`, and `make`.

```
# macOS
xcode-select --install

# Debian/Ubuntu
sudo apt-get update && sudo apt-get install build-essential
```

Once the toolchain is present, `cmdstanr` is the interface this book recommends over the older `rstan`. `cmdstanr` is a lightweight wrapper around CmdStan, the Stan command-line interface, and is updated in step with the core Stan language; `rstan` bundles its own, often older, copy of Stan directly into the R package and has historically lagged behind in supporting newer Stan features and compiler versions. Installing `cmdstanr` additionally requires a one-time call to `cmdstanr::install_cmdstan()`, which downloads and compiles CmdStan itself — a step that can take several minutes and depends on the same toolchain described above.

```
install.packages("cmdstanr",
  repos = c("https://mc-stan.org/r-packages/", getOption("repos")))
```

```
cmdstanr::check_cmdstan_toolchain()
cmdstanr::install_cmdstan()
```

JAGS, used in some legacy HEOR network meta-analysis scripts via the `rjags` or `R2jags` packages, is architecturally different: it is a standalone program installed outside of R (from SourceForge, or via the distribution's package manager on Linux) rather than a C++ code generator, so it does not share Stan's compiler requirement — but it does require that the JAGS binary itself be installed and discoverable before `rjags` will load.

Agency perspective. If a submission package to a payer body includes Bayesian NMA or survival extrapolation results, document the exact Stan/CmdStan or JAGS version used, alongside the R package versions captured in `renv.lock`. NICE DSU TSD 2 (the technical support document governing network meta-analysis methodology, covered in depth in Chapter 12) explicitly requires reporting of the software and convergence diagnostics used to fit a Bayesian NMA — not just the point estimates — because posterior samples from MCMC are not guaranteed bit-identical across sampler versions even with a fixed seed, and a reviewer reproducing your analysis on a different machine needs to know which toolchain generated the submitted numbers.

Common pitfall. A frequent failure mode is installing `rstan` or `cmdstanr` successfully but hitting cryptic compiler errors only when the first model is actually compiled — because `install.packages()` succeeds without ever invoking the C++ compiler, while fitting a model does. Run `cmdstanr::check_cmdstan_toolchain()` (or `pkgbuild::has_build_tools(debug = TRUE)` for `rstan`) immediately after installing the toolchain, before writing any model code, so toolchain problems are caught early rather than mid-analysis under deadline pressure.

2.6 Containers (Docker) for reproducible models

`renv` (Section 2.3) solves a specific problem well: it pins the versions of R packages and records where they came from. But it solves only part of the reproducibility problem. The R packages installed by `renv::restore()` still have to compile or link against something — a particular R interpreter version, system libraries such as `libcurl`, `zlib`, or `libxml2`, a BLAS/LAPACK implementation for numerical linear algebra, fonts for PDF rendering, and an operating system that provides all of the above in a given configuration. None of this is captured by a `renv.lock` file. If a reviewer, a regulator's technical team, or a colleague two years from now tries to rebuild your analysis on a different Linux distribution, a different R minor version, or a Mac with Apple Silicon instead of an Intel chip, subtle numerical differences can appear even with **identical** package versions — different BLAS implementations alone can shift floating-point results in the eighth decimal place, which is usually immaterial for an ICER but can matter in a bootstrap or Markov trace with many iterations.

Docker addresses this by pinning the entire execution environment: the base operating system, the system libraries, the R version itself, and then the R packages on top. A container built from a `Dockerfile` today will — barring extreme edge cases such as architecture changes — produce bit-for-bit the same computational environment years later, regardless of what has changed on the host machine or in CRAN's current package index.

A minimal, HEOR-appropriate `Dockerfile` built on the Rocker Project's `rocker/r-ver` image, which pins an exact R version, looks like this:

```
FROM rocker/r-ver:4.3.2

# System libraries needed by common HEOR packages
# (XML, curl, and SSL for reporting/network packages;
# a Fortran compiler for survival/optimisation routines)
RUN apt-get update && apt-get install -y --no-install-recommends \
    libxml2-dev \
    libcurl4-openssl-dev \
    libssl-dev \
    gfortran \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /model

# Bring in the renv lockfile and restore from it
COPY renv.lock renv.lock
RUN Rscript -e "install.packages('renv'); renv::restore()"

COPY . .

CMD ["Rscript", "run_model.R"]
```

This pairs the two tools rather than treating them as alternatives: `renv.lock` still governs exactly which package versions are installed, while the `Dockerfile` governs everything renv cannot see.

> **HEOR in practice.** A practical rule of thumb for submission-critical models (NICE, ICER, CADTH, or similar HTA body submissions): if the deliverable includes an executable model that a third party is expected to independently re-run and reproduce your base-case and sensitivity-analysis outputs, build and ship a Docker image alongside the R code, not instead of it. NICE's technology appraisal process explicitly provides for the Evidence Review Group to request the submitted model and re-run it during the technical engagement stage — a containerized model removes "it depends on my machine's configuration" as a possible point of failure during that review. Tag the image with the model version and archive it (e.g. as a `.tar` export via `docker save`, or pushed to a registry with access controls) with the rest of the submission package. "It ran on my machine" is not an acceptable reproducibility standard when a cost-effectiveness claim underpins a reimbursement decision.

Docker is not always worth the overhead, however. For exploratory or internal analysis — scoping work, early-stage model structuring, or a one-off piece of academic research where the main deliverable is a paper rather than an auditable model — a well-maintained `renv.lock` file, used consistently with a documented R version, is usually sufficient and considerably lighter-weight to maintain. Docker adds real friction: a build step, image storage, the need for collaborators to have Docker installed and to understand basic container commands, and slower iteration during active development because every system-library change means a rebuild. The decision point is usually: **will someone outside this project, possibly without R expertise, need to re-run this exact model and get the exact same numbers?** If yes, containerize. If the risk is only "a package update broke something," `renv` alone addresses that.

Common pitfall. Running `apt-get install` or `install.packages()` inside a `Dockerfile` without pinning versions defeats the purpose of using Docker at all — the image will build differently on different days as upstream repositories update. Always pin the base image tag (e.g. `rocker/r-ver:4.3.2`, never `rocker/r-ver:latest`), and let `renv::restore()` — driven by a committed lockfile — control package versions rather than a bare `install.packages()` call.

One further practical point: R package installation inside a container can be slow, because by default `install.packages()` and `renv::restore()` compile packages from source on Linux. Posit Package Manager (formerly RStudio Package Manager, often referenced by its public endpoint `packagemanager.posit.co`, historically "RSPM") serves pre-compiled binary builds of CRAN packages for specific Linux distributions. Pointing your container's R options at a Posit Package Manager binary repository snapshot — rather than CRAN's source-only mirrors — can cut a `renv::restore()` that takes twenty minutes down to under a minute, which matters considerably for CI pipelines and iterative image rebuilds. Posit Package Manager also lets you pin to a **dated snapshot** of CRAN, giving you an additional, complementary layer of reproducibility alongside `renv` and Docker.

Agency perspective. If you are automating image builds (for example, rebuilding a model's Docker image nightly or on every `renv.lock` change via a CI pipeline), have the automation fail loudly — not silently fall back to `latest` tags or an un-pinned package source — the moment a pinned base image or snapshot date becomes unavailable. A silent fallback is exactly the kind of drift Docker was adopted to prevent.

2.7 Recommended package ecosystem (overview map)

The preceding sections set up the plumbing — R itself, RStudio or another IDE, renv, and optionally Docker. What actually gets installed into that environment is the subject of most of the rest of this book. R's package ecosystem for health economics and outcomes research has matured substantially over the past decade, to the point where nearly every stage of a cost-effectiveness analysis — from cleaning trial data through to generating a submission-ready report — has at least one well-maintained, peer-used package behind it. This section is meant as a map you can return to: a single reference point that tells you, for a given task, which package to reach for and which later chapter in this book covers it in depth.

HEOR in practice. Do not install everything in this table at once "just in case." Install packages as your project needs them, and record each one in your `renv.lock` file as you go (Section 2.3). A project's dependency tree should reflect what the analysis actually uses, both for reproducibility and so that a reviewer of your code can infer the analytical approach from the `library()` calls alone.

Task area	Key packages	Covered in
Data wrangling	`tidyverse` (`dplyr`, `tidyr`, `ggplot2`, `purrr`), `data.table`	Ch. 3-5
Survival analysis	`survival`, `flexsurv`, `flexsurvcure`	Ch. 10
Meta-analysis	`meta`, `metafor`	Ch. 11
Network meta-analysis	`netmeta`, `gemtc`, `multinma`	Ch. 12
Bayesian modelling	`brms`, `rstan`, `cmdstanr`	Ch. 14
Decision-tree modelling	`rdecision`, `heemod`	Ch. 18
Markov / cohort models	`heemod`, `hesim`	Ch. 19
Uncertainty analysis / VOI	`BCEA`, `dampack`, `voi`	Ch. 25-26
Missing data	`mice`, `Amelia`, `missForest`	Ch. 25
Reporting	Quarto, `gt`, `gtsummary`, `flextable`	Ch. 28
Interactive apps	`shiny`, `golem`, `rhino`	Ch. 29

Table 3: R package ecosystem for HEOR by task area, with the key packages for each and where in this book they are covered in depth.

A few orientation notes on how these areas relate to one another in practice:

- **Data wrangling is the substrate everything else sits on.** Whether you ultimately build a Markov model with `heemod` or run a network meta-analysis with `netmeta`, the input data almost always passes through `tidyverse` or `data.table` pipelines first. Chapters 3–5 build this foundation before any health-economic modelling is introduced.
- **Survival analysis and decision modelling are tightly coupled.** `flexsurv` fits parametric survival curves to trial data; those fitted distributions frequently become the transition probabilities or extrapolated survival inputs for a `heemod` or `hesim` Markov model. Expect chapters 10, 18, and 19 to cross-reference each other heavily.
- **`heemod` and `hesim` overlap but are not interchangeable.** `heemod` has a gentler learning curve and a clear cohort-based Markov-model API; `hesim` is built for speed at scale (large PSA runs, individual-level microsimulation, and multi-state models) and integrates more directly with `flexsurv` objects. Chapter 19 covers the trade-offs and when to reach for each.
- **Evidence synthesis scales from pairwise to network to Bayesian.** `meta` and `metafor` handle classical pairwise meta-analysis; `netmeta` and `gemtc` extend this to networks of trials; `multinma` and `gemtc` both support Bayesian NMA, with `multinma` in particular designed with HTA-style population-adjustment methods in mind. `brms`, `rstan`, and `cmdstanr` sit one level below these as general-purpose Bayesian engines that some of the above packages call internally, and that chapter 14 also teaches you to use directly for custom Bayesian economic models.
- **Uncertainty analysis is usually the last mile, not a separate project.** `BCEA`, `dampack`, and `voi` all operate on the output of a probabilistic sensitivity analysis already run through a decision model — they do not run the model themselves. Chapters 25–26 assume you arrive with PSA output in hand.
- **Reporting tools are what turns a script into a deliverable.** Quarto (not a package in the traditional sense, but the rendering engine increasingly replacing R Markdown for this purpose)

combined with `gt`, `gtsummary`, and `flextable` is how most modern HEOR teams produce submission-ready tables and reproducible reports directly from R code; chapter 28 builds a full reporting pipeline this way.

Difficult terminology: multi-state model. A simple survival model only tracks one transition — alive to dead — and a basic Markov model usually tracks a small, fixed set of health states (e.g. well, progressed, dead). A multi-state model generalises both: it formally tracks a patient moving between any number of defined health states over time, estimating both the **probability** of each possible transition (e.g. from "stable disease" to "progression" versus to "death" directly) and the **timing** of when it happens, using the same survival-analysis machinery (hazard functions, covered in Chapter 10) applied separately to each possible transition in the diagram. A useful analogy is a subway map with multiple interchangeable lines and stations: a simple survival model only asks "did the train ever reach the depot (death)," while a multi-state model asks, for every station (health state) a train currently sits at, which of the several next stations it's most likely to travel to next, and how long that leg of the journey typically takes. `hesim`'s multi-state-model support (mentioned above) is what lets an R-based HEOR model represent disease pathways that are richer than a simple two- or three-state Markov structure — common, for example, in oncology models with distinct progression, response and relapse states.

Common pitfall. Package names in this space are easy to confuse because several solve adjacent but distinct problems: `meta` and `metafor` are not interchangeable with `netmeta` (pairwise vs. network evidence), and `mice` (multiple imputation by chained equations) is a fundamentally different missing-data strategy from `missForest` (a random-forest-based single imputation method) — the choice between them has real implications for your estimates' standard errors, not just a difference in syntax. Confirm you are using the package appropriate to your actual evidence structure and missingness mechanism before trusting its defaults.

Agency perspective. If you are using an AI coding assistant to scaffold HEOR analysis code, be specific about which package you mean when a generic term like "run a meta-analysis" or "build a decision model" is ambiguous across several packages in the table above — an assistant without HEOR-specific context may default to the most commonly documented general-statistics package (e.g. plain `survival`) rather than the domain-appropriate one (e.g. `flexsurv` for parametric extrapolation), and the resulting code may run without error while answering the wrong question.

Chapter 3. R Essentials for Health Economists

Learning objectives

- Use R's core data structures — vectors, lists, data frames/tibbles, matrices and arrays — fluently, including why each matters for later HEOR modelling work.
- Wrangle tabular data efficiently with both `tidyverse`/`dplyr` and `data.table`, and know when to prefer each.
- Apply vectorization and `purrr`'s functional-programming tools to iterate a calculation across many parameter sets — the exact pattern later used for probabilistic sensitivity analysis.
- Write well-structured, defensively-programmed R functions using named parameter lists, appropriate for models with many inputs.
- Profile R code before optimizing it, and know when `Rcpp` or parallelisation (`future`/`furrr`) is actually warranted.
- Handle dates, durations and time-to-event quantities correctly with `lubridate`, avoiding common international date-format pitfalls.
- Debug R code interactively and handle errors gracefully in long-running simulations such as PSA loops.

3.1 Data structures: vectors, lists, data frames and tibbles, arrays and matrices

Almost everything you will ever compute in a health-economic evaluation — costs, utilities, probabilities, model traces — ultimately lives inside one of a handful of R data structures. Before Chapter 4 introduces any economic concepts, it is worth being fluent in these containers, because the structure you choose determines how easy (or treacherous) later calculations will be.

The atomic vector is the most basic unit. R has no true scalars: even a single number like `42` is a vector of length one. Vectors are homogeneous — every element shares one type (`logical`, `integer`, `double`, `character`) — and R exploits this to **vectorize** arithmetic, meaning operations apply element-by-element without an explicit loop:

```
cost <- c(1200, 3400, 980, 15600)
outcome <- c(0.72, 0.65, 0.81, 0.40)
cost_per_outcome_unit <- cost / outcome
cost_per_outcome_unit
```

This vectorization is not a minor convenience; it is the reason R code that processes thousands of patient records runs in milliseconds rather than requiring hand-written loops. Mixing types in `c()` triggers implicit coercion (the hierarchy is roughly logical → integer → double → character), a frequent source of silent bugs — a logical vector accidentally combined with a character vector becomes character, and comparisons stop working as expected.

Lists relax the homogeneity rule: a list can hold elements of different types and lengths, including other lists. This makes lists the natural container for structured, non-tabular things — most importantly, bundles of model parameters. A typical pattern you will reuse heavily from Chapter 4 onward is a nested list holding every input a model needs:

```
params <- list(
  costs = list(drugA = 1200, drugB = 980, monitoring = 150),
  utilities = list(well = 0.85, sick = 0.55, dead = 0),
  discount_rate = 0.035
)
params$costs$drugA
```

Access with `$` and `[[ ]]` extracts a single element; single-bracket `[ ]` on a list always returns a (sub-)list, even for one element — a distinction that trips up almost everyone at least once.

Tabular data is where `data.frame` and its tidyverse cousin `tibble` (from the `tibble` package, loaded automatically with `library(tidyverse)`) come in. Both store a collection of equal-length vectors as named columns, so a data frame can mix numeric, character and factor columns side by side — exactly the shape of a patient-level dataset with an ID, a cost, and a treatment arm. Tibbles behave identically for most purposes but deliberately tighten base R's looser habits: they print only the first ten rows with column types labeled, never do partial matching of column names, and `[` always returns a tibble (never silently drops to a vector) unlike a data frame, whose `[` drops to a vector when it selects a single column.

> **Common pitfall.** Base `data.frame` objects support **partial matching** on `$` and allow `drop = TRUE` behavior by default when subsetting a single column with `[`. So `df$cos` may silently return the `cost` column if no other column starts with `cos`, and `df[, "cost"]` returns a bare numeric vector, not a one-column data frame. Both behaviors have caused real analysis errors in published health-economic code. `tibble` objects disable partial matching entirely and never implicitly drop dimensions, which is one of the strongest arguments for using `as_tibble()` or `tibble()` constructors throughout your HEOR workflows.

Matrices and arrays generalize vectors to two or more dimensions while keeping a single data type. A matrix is just a vector with a `dim` attribute, built with `matrix()`; an array extends this to three or more dimensions. These matter enormously for the Markov cohort models you will build in Chapter 19: a transition probability matrix is naturally represented as a square matrix where rows sum to 1, and `%*%` (matrix multiplication) propagates a cohort distribution one cycle forward with a single operation.

> **HEOR in practice.** A toy three-state transition structure (Healthy, Sick, Dead) can be sketched now, long before any Markov-model machinery exists:
>
> ```
> states <- c("Healthy", "Sick", "Dead")
> tm <- matrix(
>   c(0.80, 0.15, 0.05,
>     0.00, 0.70, 0.30,
>     0.00, 0.00, 1.00),
>   nrow = 3, byrow = TRUE,
>   dimnames = list(states, states)
> ```

```
)
rowSums(tm)  # should equal 1 for every row
```

Building this habit early — always naming dimensions with `dimnames` and always checking `rowSums()` equals 1 — will save debugging time when transition matrices grow to a dozen states.

Finally, three inspection functions should become reflexive: `class()` reports the object's high-level class (`"data.frame"`, `"matrix"`, `"numeric"`), `typeof()` reports R's internal storage type (`"double"`, `"list"`, `"character"`), and `str()` gives a compact recursive summary of structure and contents — indispensable for peering into a nested parameter list or confirming a column you expected to be numeric did not silently become a factor or character on import.

3.2 Data wrangling with `tidyverse` and `data.table`

Reading in raw data is rarely the hard part; reshaping it into the form an analysis needs almost always is. R offers two mature, widely used approaches to this reshaping, and a working HEOR analyst should be conversant in both: the `tidyverse` (principally the `dplyr` package, plus the pipe) and `data.table`. They are not rivals to be chosen once and for all — they solve the same problem with different trade-offs in readability and speed, and large pharmacoepi or claims-registry projects frequently use both in the same codebase.

The `dplyr` grammar is built from a small set of verbs that each do one thing: `filter()` keeps rows matching a condition, `select()` keeps columns, `mutate()` creates or modifies columns, `group_by()` partitions a data frame into groups for subsequent operations, and `summarise()` collapses each group to one row of aggregate statistics. These verbs are chained with the pipe operator — either `magrittr`'s `%>%` (the long-standing tidyverse convention) or base R's native `|>` (available since R 4.1, functionally equivalent for straightforward chains and increasingly the recommended default). The result reads almost like a sentence describing the transformation step by step:

```
library(tidyverse)

patient_data |>
  filter(arm %in% c("Drug A", "Drug B")) |>
  group_by(arm) |>
  summarise(mean_cost = mean(total_cost), n = n())
```

`data.table` takes a different, more compact approach, built around a single indexing expression `DT[i, j, by]`: `i` selects/filters rows, `j` computes or selects columns, and `by` groups the computation — all inside one set of square brackets, operating on the table **in place** rather than copying it. The equivalent computation looks like this:

```
library(data.table)

patient_dt <- as.data.table(patient_data)
patient_dt[arm %in% c("Drug A", "Drug B"),
           .(mean_cost = mean(total_cost), n = .N),
           by = arm]
```

Both snippets produce the same grouped summary, and once the `DT[i, j, by]` pattern is internalized it is no less readable than the pipe — it is simply denser. The practical difference that matters

is performance and memory behavior at scale. `data.table` modifies data in place (via reference semantics and functions like `setDT()` and `:=` for in-place column assignment) and uses a highly optimized C-level radix sort and hash-based grouping, which makes it dramatically faster and more memory-efficient than `dplyr` once a dataset reaches into the millions of rows — exactly the regime of national claims databases, disease registries, or simulated patient-level microsimulation output with many replications. `dplyr`, by contrast, favors copy-on-modify semantics (it never silently alters your original object), which is safer for interactive, exploratory work but costs more memory and time as data grows.

> **Difficult terminology: radix sort / hash-based grouping.** Both are internal techniques for ordering or grouping rows of data fast, and the reason `data.table` can out-perform more general approaches at large scale. An ordinary comparison-based sort (the kind most languages use by default) repeatedly compares pairs of whole values against each other to decide their order, which gets slower as the dataset grows because the number of comparisons needed grows faster than the data itself. A **radix sort** instead orders data by processing one digit, character, or byte at a time — rather like sorting a stack of exam papers first by surname's first letter into 26 piles, then re-sorting each pile by the second letter, and so on — which scales far better for the kinds of values (integers, short strings, factor-coded categories) common in claims and registry data. **Hash-based grouping** uses a related lookup-table trick for the `by =` step in `DT[i, j, by]`: rather than comparing every row against every other row to see which share a group, it computes a quick numeric "fingerprint" (a hash) for each row's grouping value and uses that fingerprint as a direct lookup key, finding matching rows nearly instantly instead of scanning the whole table repeatedly. Neither technique requires you to write any special code yourself — `data.table` applies them automatically whenever you sort or group — but knowing they exist explains **why** the same `DT[i, j, by]` syntax that looks unremarkable on a 500-row toy dataset becomes dramatically faster than `dplyr` once you are working with millions of claims records.

- Prefer `tidyverse`/`dplyr` when the dataset is small-to-moderate (comfortably fits in memory with room to spare), when the code will be read and maintained by collaborators less fluent in R, or when the pipeline benefits from the broader tidyverse ecosystem (`tidyr` for reshaping, `ggplot2` for plotting the same tibbles downstream).
- Prefer `data.table` when working with claims-scale or registry-scale data (millions of rows, wide microsimulation outputs, repeated resampling such as probabilistic sensitivity analysis replications), when runtime or memory genuinely becomes a bottleneck, or when the same large table is updated repeatedly inside a loop, where `data.table`'s in-place `:=` avoids repeated full-table copies.
- Mixing both is common and acceptable: convert a `data.table` result back with `as_tibble()` or `setDF()` once the heavy aggregation is done, and finish the pipeline in `dplyr`/`ggplot2` for readability.

> **Common pitfall.** `data.table`'s `:=` and in-place modification mean that assigning `dt2 <- dt1` does **not** create an independent copy the way it would for a data frame or tibble — `dt2` and `dt1` point to the same underlying object, and modifying one with `:=` silently modifies the other. If you need an independent copy, use `copy(dt1)` explicitly. This reference-semantics behavior

is the single most common source of "my original data changed and I don't know why" bugs when analysts move from `dplyr` to `data.table`.

Agency perspective. Whichever grammar you choose for a given analysis script, say so explicitly in the script header or project README (Chapter 2's `analysis/` convention) and stay consistent within that script — silently switching between `%>%`, `|>`, and `DT[]` syntax mid-pipeline makes code far harder for a collaborator, or your future self, to audit. The CHEERS 2022 reporting checklist (covered in full in Chapter 28), item 29, explicitly requires that analytic methods be reported in enough detail for a third party to reproduce them — code-style consistency within one script is a small thing that materially supports that requirement in practice, not just a stylistic nicety.

Operation	`dplyr` / tidyverse	`data.table`
Filter rows	`filter(arm == "Drug A")`	`DT[arm == "Drug A"]`
Select columns	`select(id, cost)`	`DT[, .(id, cost)]`
Create/modify column	`mutate(cost2 = cost * 1.1)`	`DT[, cost2 := cost * 1.1]`
Group and summarise	`group_by(arm) |> summarise(mean_cost = mean(cost))`	`DT[, .(mean_cost = mean(cost)), by = arm]`
Chain multiple steps	`|>` or `%>%` pipe between verbs	stacked `[i, j, by]` expressions
Independent copy	implicit (copy-on-modify)	explicit: `copy(dt1)`

Note. `data.table` modifies in place via reference semantics; `dplyr` never silently alters the original object.

Table 4: Common data-wrangling operations: `dplyr`/tidyverse syntax versus the equivalent `data.table` expression.

3.3 Functional programming with `purrr` and vectorization

R is a vectorized language at its core: arithmetic, comparisons, and most mathematical functions operate element-by-element on whole vectors without you writing an explicit loop. This matters for more than style. A `for` loop that grows a result vector by repeated `c()` or indexed assignment inside the loop forces R to reallocate memory on every iteration, and interpreted loop overhead adds up quickly once you are iterating over thousands of patients, cycles, or parameter draws — exactly the scale you hit in probabilistic sensitivity analysis (PSA) later in this book. A vectorized expression, by contrast, dispatches to compiled C code under the hood and processes the whole vector in one pass. The practical rule of thumb: if you find yourself writing `for (i in 1:n)` to apply the same arithmetic to each element of a vector, stop and ask whether the operation can be expressed directly on the vector instead.

```
n_patients <- 1000
cost_per_visit <- rep(185.50, n_patients)
n_visits <- sample(1:12, n_patients, replace = TRUE)
```

```r
# Avoid: explicit loop
total_cost_loop <- numeric(n_patients)
for (i in seq_len(n_patients)) {
  total_cost_loop[i] <- cost_per_visit[i] * n_visits[i]
}

# Prefer: vectorized
total_cost_vec <- cost_per_visit * n_visits
```

Not every problem reduces to simple elementwise arithmetic, though. Often you need to apply a more elaborate function — one with branching logic, multiple inputs, or a non-trivial return type — to each element of a list or each row of a parameter table. This is where the `purrr` package (part of the tidyverse) earns its place in your toolkit, as a more predictable and composable alternative to base R's `sapply()`/`lapply()`.

The core function is `purrr::map()`, which always returns a list, regardless of what you feed it — this predictability is the whole point. Where `sapply()` will silently simplify its output to a vector, a matrix, or a list depending on the data, `map()` never surprises you. When you know the output should be a specific atomic type, use the typed variants: `map_dbl()` returns a double vector (or errors if any element can't be coerced), `map_chr()` a character vector, `map_lgl()` a logical vector, and so on. This type enforcement is itself a form of defensive programming — a `map_dbl()` call that unexpectedly hits a non-numeric result fails loudly instead of quietly returning an untyped list you forget to unlist.

```r
library(purrr)

scenarios <- list(
  base_case  = list(cost_visit = 185.50, visits = 4, n_patients = 250),
  high_cost  = list(cost_visit = 240.00, visits = 4, n_patients = 250),
  high_util  = list(cost_visit = 185.50, visits = 7, n_patients = 250)
)

calc_total_cost <- function(params) {
  params$cost_visit * params$visits * params$n_patients
}

total_costs <- map_dbl(scenarios, calc_total_cost)
total_costs
#>  base_case  high_cost  high_util
#>     185500     240000     324625
```

When you need to iterate over two parallel vectors or lists simultaneously, `map2()` takes two inputs and applies a function pairwise; when you have three or more, `pmap()` takes a single list of input lists/vectors and applies a function across all of them element-by-element. These generalize naturally to the structure of a parameter table, where each row is one scenario and each column is one parameter.

```r
visit_counts <- c(4, 5, 6)
cost_per_visit <- c(185.50, 185.50, 200.00)

map2_dbl(visit_counts, cost_per_visit, ~ .x * .y * 250)

param_grid <- list(
```

```
  cost_visit  = c(185.50, 200.00, 220.00),
  visits      = c(4, 5, 6),
  n_patients  = c(250, 250, 300)
)

pmap_dbl(param_grid, calc_total_cost)
```

> **HEOR in practice.** This `scenarios` list and `pmap_dbl()` pattern is not a toy exercise — it is the literal skeleton of a PSA loop. In Chapter 25, you will replace this three-row `param_grid` with a parameter table of 1,000+ rows, each row a random draw from a cost or utility distribution, and replace `calc_total_cost()` with a full cost-effectiveness model function. The call becomes `pmap(psa_params, run_model)`, and the output list becomes your distribution of ICERs. Learning to think in terms of "a function applied across a parameter table" now will make that chapter feel like a straightforward extension rather than a new paradigm.

3.4 Writing functions, parameter lists and defensive programming

HEOR models rarely take two or three arguments. A Markov cost-effectiveness model might need transition probabilities, cost inputs, utility weights, a discount rate, a time horizon, and a cycle length — easily ten or more parameters. Writing a function that takes these as a long sequence of positional arguments is a recipe for subtle, hard-to-detect bugs: swap the order of `discount_rate` and `cycle_length` in a call several hundred lines away from the function definition, and R will not complain — it will just silently compute the wrong answer.

```
# Fragile: positional arguments, easy to transpose by mistake
run_model <- function(p_progress, p_death, cost_well, cost_sick,
                      util_well, util_sick, discount_rate, horizon) {
  # ...
}
```

The more robust pattern is to bundle related parameters into a single named list (or, for larger models, a dedicated list-of-lists or a small S3/S4 object) and pass that one structured object into the function. Named access by `$` or `[[ ]]` makes every call self-documenting and immune to argument-order mistakes.

> **Difficult terminology: S3 / S4 objects.** R has more than one system for object-oriented programming — ways of bundling data together with the functions ("methods") that know how to act on it, so the data carries its own behaviour rather than living as a bare, anonymous list. S3 is the older, lightweight system: you simply attach a `class` attribute (a label) to an ordinary R object, and then write functions named in a particular pattern (e.g. `print.my_model`) that R automatically dispatches to whenever someone calls `print()` on an object of that class. There is no formal definition enforced anywhere — it works by convention and is easy to get wrong silently. S4 is a stricter, more formal system: you explicitly declare a class's structure in advance (which fields it must have, and what type each one is), and R then checks that any object you create actually conforms to that declaration, catching mistakes earlier. A filing-cabinet analogy helps: S3 is like labelling a folder "Patient Record" and trusting everyone to put the right kinds

of documents inside it; S4 is like a pre-printed form with named, typed boxes for each field, which physically cannot be filled in with the wrong kind of information. Most of this book's own worked examples use plain named lists for simplicity, but model-parameter bundles, `survival` package model-fit objects, and several HEOR packages (`flexsurv`, `heemod`) use S3 or S4 objects internally, so recognising the pattern when you see `class(x)` or `methods(print)` in someone else's code is a practically useful skill even if you rarely need to **write** a new S4 class yourself.

```
model_params <- list(
  p_progress     = 0.15,
  p_death        = 0.05,
  cost_well      = 1200,
  cost_sick      = 8500,
  util_well      = 0.85,
  util_sick      = 0.55,
  discount_rate  = 0.035,
  horizon        = 10
)

run_model <- function(params) {
  stopifnot(is.list(params))
  with(params, {
    # model logic refers to p_progress, cost_sick, etc. by name
  })
}
```

Give arguments sensible default values wherever a standard convention exists — a discount rate of 3.5% per UK NICE guidance, or a cycle length of one year, for instance — using R's native `function(x, discount_rate = 0.035)` syntax. Defaults reduce boilerplate at every call site and document the "normal" assumption directly in the function signature, but be explicit in comments about where a default comes from, since a silently-applied wrong default is just as dangerous as a wrong argument.

The deeper issue is that R, by design, is permissive about types. A probability argument passed as the character string `"0.15"` instead of the numeric `0.15` will often not fail immediately — it may propagate through several calculations, coerced or recycled in ways that produce a number that looks plausible but is wrong, and surface only as an ICER that is subtly off, discovered (if you are lucky) by a reviewer long after submission. This is why defensive programming is not an optional style preference in HEOR work — it is a quality-assurance requirement.

The baseline tool is `stopifnot()`, which checks one or more conditions and halts execution immediately with an error if any is false — cheap insurance placed at the top of every model function.

```
run_model <- function(params) {
  stopifnot(
    is.list(params),
    is.numeric(params$p_progress),
    params$p_progress >= 0, params$p_progress <= 1,
    is.numeric(params$discount_rate),
    params$discount_rate >= 0, params$discount_rate < 1,
```

```
    params$horizon > 0
  )
  # ... model logic ...
}
```

For checks that need a more informative, customized message than `stopifnot()`'s default, use `stop()` directly, and reserve `warning()` for conditions that are suspicious but not necessarily fatal — an unusually high cost input, say, that you want flagged without halting a long-running PSA batch.

```
validate_probability <- function(p, name) {
  if (!is.numeric(p)) {
    stop(sprintf("'%s' must be numeric, got %s.", name, class(p)[1]))
  }
  if (p < 0 || p > 1) {
    stop(sprintf("'%s' must lie in [0, 1], got %s.", name, p))
  }
  if (p > 0.9) {
    warning(sprintf("'%s' = %s is unusually high; check the source.", name, p))
  }
  invisible(p)
}
```

A good discipline is to validate every input once, at the top of the function, rather than scattering `if` checks throughout the body — this keeps the "is this input sane?" logic separate from "what does this input mean for the model?" logic, and makes the function easy to audit.

- State preconditions explicitly with `stopifnot()` before any computation runs.
- Use `stop()` with a formatted, specific message when a precondition fails outright.
- Use `warning()` for implausible-but-not-impossible values you want surfaced, not silenced.
- Bundle 5+ related arguments into a named list rather than a long positional signature.

Common pitfall. R's automatic type coercion is a frequent source of silent errors. `if (p_progress == "0.15")` will happily evaluate as `TRUE` even when `p_progress` is the numeric `0.15`, because R coerces the number to a string for the comparison — no error, no warning, just a wrong path taken. A function that checks `is.numeric()` explicitly, rather than relying on implicit coercion inside a comparison, catches this class of bug before it reaches your cost-effectiveness output.

Agency perspective. When you ask an AI coding assistant to draft a model function for you, always ask it to add `stopifnot()` input validation and informative `stop()`/`warning()` messages in the same pass — generated code tends to implement the "happy path" calculation correctly but omit defensive checks unless specifically prompted, and an unvalidated function is the easiest place for a submission-breaking error to hide. NICE's technology appraisal process (Chapter 2) relies on the Evidence Review Group being able to re-run a submitted model and trust its outputs; a model that silently accepts an out-of-range probability and returns a plausible-looking but wrong ICER is a far worse submission failure than one that errors loudly and gets caught before submission.

3.5 Performance: profiling, `Rcpp`, parallelisation (`future`, `furrr`)

The first rule of optimisation is to not do it until you have evidence. R code that "feels slow" and R code that **is** slow in the place you think it is are often two different programs. Before touching a single line in the name of speed, profile it. `Rprof()` is the base-R sampling profiler — it periodically records the call stack while your code runs, and `summaryRprof()` turns that into a table of where time actually went. `profvis::profvis()` wraps the same mechanism in an interactive flame graph that is far easier to read:

> **Difficult terminology: flame graph.** A flame graph is a specific way of visualising where a program spent its execution time, and it reads differently from an ordinary bar chart. Each horizontal bar represents one function call, and its **width** shows how much total time that call took (wider = slower). Bars are stacked vertically to show the call hierarchy: a function sits directly above the bars representing the functions it called, so following a column straight down from any bar traces that call's entire chain of parent functions back to the top-level script. The name "flame graph" comes from the characteristic look of these stacked bars — varying widths at each level resembling flickering flame tips. Reading one is mostly about scanning for the **widest** bars, especially ones near the bottom of a tall stack (meaning a deeply nested function is eating a disproportionate share of total runtime) — those are the parts of the code actually worth optimising, as opposed to functions that are called often but individually fast, which will show up as many thin bars rather than one wide one.

```
library(profvis)

profvis({
  results <- vector("list", 500)
  for (i in seq_len(500)) {
    results[[i]] <- run_markov_trace(params_list[[i]])
  }
})
```

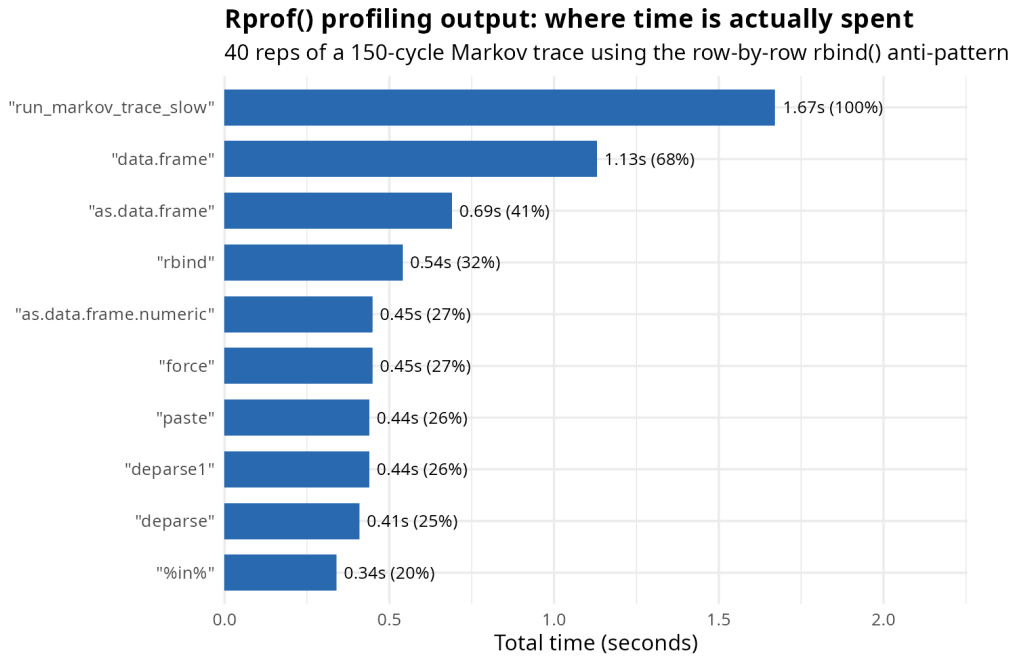


Figure 3: `Rprof()`/`summaryRprof()` output on a deliberately slow Markov-trace loop. The `rbind()`-in-a-loop anti-pattern dominates total time — `data.frame()`, `as.data.frame()` and `rbind()` together account for well over half of it — exactly the kind of result profiling should surface before any hand optimisation begins.

How to report this.

Rank	Function	Why it dominates
1	`rbind()`	Called once per loop iteration; reallocates and copies the growing result object each time
2	`data.frame()`	Invoked internally by `rbind()` on data-frame arguments, compounding the reallocation cost
3	`as.data.frame()`	Coercion overhead triggered repeatedly as part of the same `rbind()` call chain
4+	model/arithmetic logic	Comparatively minor; swamped by the three functions above combined

Note. `data.frame()`, `as.data.frame()` and `rbind()` together account for well over half of total runtime — fix the growing-object pattern before considering `Rcpp` or parallelisation.

Table 5: Top functions by self + total time in the `summaryRprof()` output above, ranked by where profiling attention should go first.

Run this on a **representative** slice of the real workload, not a toy example — bottlenecks in a 500-iteration loop are not always visible at 5 iterations. Nine times out of ten the profiler finds

the slow part in a place you didn't expect: a `data.frame` growing row-by-row inside a loop, an unnecessary `rbind()`, or repeated `read.csv()` calls on the same file. Fix those first; they are free wins that require no new tooling.

Occasionally the profiler confirms that the bottleneck is a genuine hot loop — tight, scalar, repeated millions of times — that vectorisation cannot absorb (e.g., a recursive Markov trace update computed one cycle at a time). This is the narrow case where `Rcpp` earns its keep: you rewrite just that loop in C++, compile it inline, and call it from R as an ordinary function.

```
library(Rcpp)

cppFunction('
  double cycle_sum(NumericVector costs, double discount_rate) {
    double total = 0;
    int n = costs.size();
    for (int i = 0; i < n; i++) {
      total += costs[i] / pow(1 + discount_rate, i);
    }
    return total;
  }
')
```

Reach for this only after profiling has pointed a finger at a specific loop — `Rcpp` adds a C++ toolchain dependency (the one Chapter 2 flagged alongside `cmdstanr`) and a compiled artefact to maintain, so it should replace a measured bottleneck, not a hunch.

More often than not, the real lever is not making one iteration faster but running many iterations at once. This is directly relevant to the probabilistic sensitivity analyses (PSA) built in Chapter 25, where a model is re-run over thousands of parameter draws and each run is independent of the others — an "embarrassingly parallel" problem. The `future` package abstracts **how** code is executed from **what** is executed: you declare a plan once, and the same code then runs sequentially or in parallel without modification.

```
library(future)
library(furrr)

plan(multisession, workers = 4)

psa_results <- future_map(psa_param_draws, run_model, .progress = TRUE)
```

`furrr::future_map()` is a drop-in parallel replacement for `purrr::map()` — same signature, same list-in/list-out contract, but each element is dispatched to a worker process under the active `future` plan. Swapping `map()` for `future_map()` and adding one `plan()` call is often the entire migration.

> **HEOR in practice.** Before parallelising a PSA, make sure each iteration is self-contained — no shared mutable state, no writing to the same file from multiple draws. `future`'s workers are separate R processes; they cannot see objects you forgot to pass in, and silent scoping bugs that worked by accident in a sequential `for` loop will fail (or worse, produce wrong numbers) under `multisession`.

Parallelising a model that has a bug does not fix the bug; it just produces the wrong answer faster, on four cores at once. Profile, fix the real bottleneck if there is one, parallelise last.

3.6 Dates, durations and time-to-event data (`lubridate`)

Health economic models built on trial data live and die by correct date arithmetic — time-to-progression, time-to-death, length of treatment exposure, time between visits, are all date subtractions underneath. Base R's `Date` class handles the simple cases, but `lubridate` makes the parsing, arithmetic and edge cases explicit rather than implicit, which matters when the input is a column of strings from a dozen different trial sites.

Parsing is the first hazard. `lubridate::ymd()`, `dmy()` and `mdy()` each commit to one date order and fail loudly (returning `NA` with a warning) rather than guessing silently:

```
library(lubridate)

enrol_date <- ymd("2021-03-14")
visit_date <- dmy("14/03/2021")

as.Date("2021-03-14")  # base R equivalent, but order must match exactly
```

> **Common pitfall.** International trial data frequently mixes `DD/MM/YYYY` and `MM/DD/YYYY` formats across sites, and spreadsheet exports rarely declare which one they used. `as.Date("03/04/2021")` silently parses as 3rd April under one default and 4th March under another — no error, no warning, just a wrong date fed straight into a survival-time calculation. Always confirm the source format explicitly (check a known out-of-range day like the 25th against the site's convention) and parse with the matching `lubridate` function — `dmy()` or `mdy()` — rather than a bare `as.Date()` that silently assumes one order.

Once dates are parsed correctly, time-to-event quantities reduce to differences. `difftime()` computes a difference with an explicit unit, which avoids the ambiguity of letting R guess whether you meant days or weeks:

```
time_to_progression <- difftime(progression_date, enrol_date, units = "days")
```

For anything beyond a single subtraction — durations spanning leap years, or lengths that should be expressed in calendar months rather than a fixed number of days — `lubridate::interval()` and `time_length()` are the more robust pair. An `interval` is a span between two specific time-points (not just a duration), and `time_length()` converts it into whatever unit you need, correctly accounting for variable month and year lengths:

```
progression_interval <- interval(enrol_date, progression_date)
time_length(progression_interval, unit = "months")
```

This distinction — duration as "an interval between two real dates" versus duration as "a fixed span of time" — becomes important once you reach the parametric survival models in Chapter 10, where time-to-event outcomes are the response variable and getting the units wrong (days vs. months) silently rescales every fitted hazard and every extrapolated curve downstream.

A related trap is censoring: a patient who has not progressed by the data cut-off does not have a missing progression date in the "NA means unknown" sense — they have a **censored** observation, and that status needs to be carried as its own column (commonly a 0/1 event indicator) rather than inferred from an `NA` in the date field. Confusing "no event date because censored" with "no event date because missing" is a common source of silently wrong survival curves, and it starts right here at the data-cleaning stage, well before any survival model is fitted.

3.7 Debugging and error handling

A health economic model that runs to completion and produces a wrong number is more dangerous than one that crashes, because a crash is visible and a silently wrong number is not. This section covers the tools for finding bugs interactively, and — more importantly for the long-running simulations in this book — the discipline of handling errors so that one bad input doesn't either kill a multi-hour run or, worse, let it finish with corrupted results.

When something throws an error, `traceback()` is the first diagnostic: run it immediately after the error to see the full call stack that led to it, innermost call first. This tells you **which** function actually failed, which is not always the function named in the error message:

```
run_model(bad_params)
#> Error in sqrt(variance) : NaN produced
traceback()
```

For errors that are hard to pin down from the stack alone, `debug(fun_name)` flags a function so that the next call to it drops you into an interactive browser at its very first line, letting you step through line by line and inspect every intermediate object. `browser()` does the same thing but by inserting an explicit breakpoint directly in the source, which is more convenient when you already suspect roughly where the problem is:

```
run_model <- function(params) {
  browser()  # execution pauses here; inspect `params` interactively
  variance <- compute_variance(params)
  sqrt(variance)
}
```

Both are for **finding** bugs during development. They are not a substitute for handling errors gracefully in code that will run unattended — and in health economic practice, nothing runs more unattended than a PSA loop iterating over thousands of parameter draws. `tryCatch()` is the tool for that: it lets you catch a specific failure, decide what to do about it, and let execution continue rather than halting the entire run.

```
psa_results <- vector("list", n_iterations)

for (i in seq_len(n_iterations)) {
  psa_results[[i]] <- tryCatch(
    run_model(psa_param_draws[[i]]),
    error = function(e) {
      message(sprintf("Iteration %d failed: %s", i, conditionMessage(e)))
      NA
    }
  )
}
```

Here a single malformed draw — a negative cost sampled from a misspecified distribution, a file read that occasionally fails on a flaky network drive — logs itself and returns `NA` for that one iteration, instead of stopping the other 9,999.

> **HEOR in practice.** Wrap each individual PSA iteration in its own `tryCatch()`, not the whole loop. Wrapping the entire `for` loop in one `tryCatch()` means the **first** bad draw stops everything and you lose every result computed before it; wrapping each iteration means you

lose exactly one data point and keep the other thousands. Always log which iteration failed and why — a silent `NA` with no record of the cause is a debugging problem for tomorrow-you.

Agency perspective. An uncaught error partway through a PSA is the visible failure mode; the one that should worry you more is a **caught** error that is handled too permissively — a blanket `tryCatch(..., error = function(e) 0)` that replaces a failed iteration with zero instead of `NA`. Zero is a plausible-looking number that will quietly bias your cost-effectiveness acceptability curve (CEAC). ISPOR-SMDM good-practice guidance on model transparency (covered in Chapter 27) expects PSA results to be reported alongside the actual number of iterations successfully completed — silently padding a failed run back up to the intended iteration count with substituted zeros defeats that transparency requirement. `NA` at least survives downstream and forces you to notice and account for the dropped iterations before reporting final results.

The underlying principle is the same at every scale, from a single `browser()` session to a 10,000-iteration PSA: an error that stops execution is an inconvenience, but an error that gets silently absorbed into a "successful" result is the one that makes it into a published cost-effectiveness ratio.

Chapter 4. Core Concepts in Health Economics (with R Illustrations)

Learning objectives

- Explain the opportunity-cost foundation of health economics and distinguish the types of costs and effects an economic evaluation must capture.
- Distinguish cost-minimization, cost-effectiveness, cost-utility, cost-benefit, and cost-consequence analysis, and explain why cost-utility analysis with QALYs is the HTA-agency default.
- Construct a QALY from health-state duration and utility weight, and distinguish QALYs from DALYs and plain life-years, including their opposite sign conventions.
- Compute an ICER, and convert it to net monetary benefit (NMB) or net health benefit (NHB) to avoid the ratio's sign-ambiguity problems.
- Describe what a willingness-to-pay threshold represents, and the major methodological traditions (opportunity cost vs stated preference) used to set one, including India's HTAIn approach.
- Apply perspective, time horizon, discounting and half-cycle correction as explicit, justified modelling choices.
- Identify simple and extended dominance and construct an efficiency frontier from a set of competing treatments.
- Write a complete R pipeline that takes cost/QALY data for multiple treatments and produces a dominance-pruned, incrementally-costed frontier table — the book's first executed health-economic calculation.

4.1 Costs, effects and the opportunity cost of health care

Every health system, however wealthy, faces the same brute fact: the budget for health care is finite, but the number of things worth paying for is not. A national payer, an insurance fund, or a hospital formulary committee has a fixed pot of money in any given year. Every pound, dollar, or rupee spent funding one technology is a pound, dollar, or rupee that cannot be spent funding another. This is the economic idea of **scarcity**, and its direct consequence is **opportunity cost**: the value of the best alternative forgone when a choice is made.

This single idea is the foundation of everything in health economic evaluation. If a health system adopts a new oncology drug at a given price, the money for that drug is drawn, directly or indirectly, from the same budget that could have funded extra dialysis capacity, additional health visitor time, or a different medicine entirely. The health gained by patients on the new drug must be weighed against the health that could have been produced by whatever was displaced. A technology is not "cost-effective" in some free-standing, absolute sense — it is cost-effective **relative to** the alternative use of the same resources. This is why every serious economic evaluation is fundamentally comparative: it never asks "is this good?" in isolation, but "is this better use of this money than the next-best thing we could do with it?"

This comparative framing is what separates health economic evaluation from simple clinical evaluation. A clinical trial can show a drug extends survival or shrinks tumours; it says nothing about whether funding that drug is a **good use** of a constrained budget relative to displaced alternatives. Economic evaluation exists precisely to answer that second, harder question.

To do so formally, every analysis needs two ingredients: **costs** and **effects**.

Costs are usually split into three categories:

- **Direct medical costs** — costs borne by the health system in delivering care: drug acquisition, staff time, hospital bed-days, diagnostic tests, outpatient visits, adverse-event management.
- **Direct non-medical costs** — costs borne by the patient or their household in accessing care but not for clinical care itself: travel to appointments, informal carer time, home modifications, over-the-counter aids.
- **Indirect costs** — productivity losses from illness or treatment: time off paid work, reduced workplace output ("presenteeism"), premature mortality's effect on lifetime earnings. Whether indirect costs belong in the analysis at all is a long-standing methodological fault line — most European HTA reference cases (and NICE in particular) exclude productivity costs from the base case on distributional-fairness grounds, while some willingness-to-pay/societal-perspective frameworks (and some CBA applications) include them.

Effects are more subtle, because "effect" in clinical research and "effect" in HEOR are not the same thing. A clinical trial reports **clinical outcomes**: tumour response rate, HbA1c reduction, relapse rate, blood pressure change. These are precise, measurable, and essential — but they are not, on their own, comparable across disease areas, and they carry no information about how much patients **value** the change. A one-point drop in HbA1c and a 10% reduction in stroke risk cannot be traded off against each other using clinical endpoints alone. HEOR needs **value-laden** outcome measures that convert clinical effect into a common currency of health gain, so that a diabetes drug, a cancer drug, and a hip replacement can, in principle, be compared on the same scale. That conversion — from clinical endpoint to a generic, preference-weighted outcome — is the subject of the next two sections.

```
# Illustrating opportunity cost with a fixed budget
budget <- 10e6               # fixed annual budget, GBP 10m
cost_per_qaly_drugA <- 20000  # technology A
cost_per_qaly_drugB <- 12000  # displaced alternative use of the same money

qalys_A <- budget / cost_per_qaly_drugA
qalys_B <- budget / cost_per_qaly_drugB

qalys_A; qalys_B
qalys_B - qalys_A   # health forgone by funding A instead of B
```

Spending the whole £10m budget on technology A buys 500 QALYs; spending it on the displaced alternative B buys 833 QALYs. The 333 QALY difference is not an abstraction — it is real health that could have existed but did not, because the money went elsewhere. This is the opportunity cost of the funding decision, and it is the quantity that cost-effectiveness analysis exists to make visible.

> **HEOR in practice.** When you build cost inputs in R, keep direct medical, direct non-medical, and indirect cost components as **separate columns** in your cost data frame (not a single pre-summed "total cost") for as long as possible in the pipeline. Reviewers and HTA submission

templates routinely ask for a disaggregated cost breakdown, and recomputing it after the fact from a single summed column is error-prone and often impossible if the component data was discarded.

4.2 Types of economic evaluation

Not all economic evaluations measure "effect" the same way, and the choice of effect measure defines the **type** of analysis. Drummond et al.'s taxonomy, now standard across the field, distinguishes five main forms. Understanding the differences is essential, because HTA agencies do not accept any of them interchangeably — most require one specific type as their reference-case default.

- **Cost-minimization analysis (CMA)** — used only when there is good evidence that two or more interventions produce **equivalent** effects (established, for example, by a non-inferiority trial). If effects are genuinely equal, the decision collapses to picking the cheapest option. CMA is fragile: it is valid only as long as the equal-effectiveness assumption holds, and it is frequently misused when effects are merely **untested** for difference rather than **proven** equivalent.
- **Cost-effectiveness analysis (CEA)** — effects are measured in natural clinical units specific to the disease area: life-years gained, cases of disease averted, mmHg reduction, strokes prevented. Results are reported as a cost per unit of natural effect (e.g., cost per life-year gained, cost per case averted). CEA in this narrow sense is useful within a single disease area but cannot compare across different natural units — cost per stroke averted cannot be ranked against cost per cataract corrected.
- **Cost-utility analysis (CUA)** — a specific, standardized form of cost-effectiveness analysis in which effects are expressed in quality-adjusted life years (QALYs), a generic preference-weighted outcome that combines length of life and health-related quality of life into a single number (constructed explicitly in §4.3). Because QALYs are intended to mean the same thing regardless of disease area, CUA is the only one of these methods that supports cross-programme comparison — a cost per QALY for a cancer drug and a cost per QALY for a diabetes drug are, in principle, directly comparable, which is exactly what a national payer deciding across its whole portfolio needs.
- **Cost-benefit analysis (CBA)** — the most ambitious and least commonly used in HTA practice: all effects, including health itself, are monetized (via willingness-to-pay studies, value-of-a-statistical-life estimates, or similar), so that a single net monetary benefit figure summarizes the decision. CBA permits comparison not just across health interventions but across sectors entirely (health spending versus transport safety spending, for instance) but the monetization of health and life is methodologically and ethically contentious, and most health-specific HTA bodies avoid it as a primary reference-case method.
- **Cost-consequence analysis (CCA)** — deliberately avoids collapsing effects into a single ratio. Costs and multiple disaggregated effects (QALYs, specific clinical events, resource-use impacts) are presented side by side in a balance-sheet format, leaving the decision-maker to weigh trade-offs themselves. CCA is often used as a **supplementary** presentation alongside a CUA base case, especially where some important consequences resist being folded sensibly into a single utility number.

Difficult terminology: non-inferiority trial. Most clinical trials are designed as **superiority** trials: they try to show a new treatment works better than an existing one (or than placebo). A non-inferiority trial sets out to answer a deliberately different question — not "is the new treatment better," but "is the new treatment not meaningfully worse," within a pre-agreed margin that both the trial's designers and the relevant regulator accept in advance as clinically unimportant. This design is typically chosen when the new treatment offers some other advantage — cheaper, more convenient to take, fewer side effects, easier to manufacture at scale — so that proving "just as good, clinically" is enough to justify switching, without needing to prove outright superiority on the primary clinical outcome. Because a non-inferiority trial cannot prove two treatments are **exactly** equal (no finite trial can rule out every conceivable tiny difference), its result is always "not worse by more than margin X," not "identical" — a distinction HEOR analysts must keep in mind before treating a non-inferiority result as licence for unqualified cost-minimisation.

Difficult terminology: value-of-a-statistical-life estimate. This sounds like it is pricing one specific, identified person's life, but it is not — it is a population-level monetary figure representing how much a group of people are collectively willing to pay to reduce everyone's risk of death by a small amount. It is typically derived either by observing how much extra pay workers demand to take measurably riskier jobs (a **revealed preference** method, explained further in §4.5 below), or by asking people directly in surveys how much they would pay for a small reduction in their own risk of dying (a **stated preference** method). The number is then extrapolated: if 100,000 people are each willing to pay $50 for a reduction in risk that is expected to prevent one death across that group, the value-of-a-statistical-life estimate for that context is $5 million — not the value of any one real, identified person's life, but the aggregate amount a population puts toward preventing one anonymous, statistical death among them. It is the standard tool cost-benefit analysis (CBA, described just above) uses to convert mortality risk reduction into a monetary figure that can be added alongside other costs and benefits.

In practice, CUA with QALYs has become the de facto default that HTA agencies worldwide demand for formal technology appraisal, and it is worth being explicit about **why**: a finite health budget has to allocate across wildly different disease areas — oncology, rare disease, cardiovascular prevention, mental health — and only a generic outcome measure makes those allocation decisions comparable on a common scale. A natural-units CEA cost per "case averted" cannot inform whether money is better spent on cancer or on diabetes; a QALY-denominated CUA, at least nominally, can.

Agency perspective. NICE's reference case for technology appraisals specifies the QALY, estimated using the EQ-5D (or an NICE-endorsed alternative under its 2022 health technology evaluation methods manual), as the required outcome measure; submissions that present only natural-unit CEA or CCA results without an accompanying QALY-based CUA are routinely asked to supply one. HTAIn's (Health Technology Assessment in India) reference case similarly mandates a QALY-based cost-utility analysis as the base-case outcome measure for the cost-effectiveness threshold it applies, placing India alongside NICE in the CUA-as-default camp despite very different thresholds and population contexts.

Common pitfall. Do not treat "cost-effectiveness analysis" as a synonym for the whole field. In the strict Drummond taxonomy, CEA specifically denotes natural-units analysis, and CUA (QALY-based) is a distinct, narrower category. Many papers and even some agency documents use "cost-effectiveness" loosely to mean "any formal economic evaluation" — when reading or writing a submission, check the actual outcome measure used before assuming which type is meant.

4.3 QALYs, DALYs and life-years

Three outcome metrics recur throughout HEOR, and each solves the "how do we weigh life" problem differently. Historically, the simplest and oldest is the **life-year**: a count of time lived, with no adjustment at all for the quality of that time. A treatment that extends survival by one year scores exactly the same whether that year is spent in full health or bedridden in severe pain. Life-years remain useful for mortality-dominated contexts (vaccination programmes, acute life-saving interventions) and for sensitivity analysis, but their indifference to quality of life is precisely the gap the QALY was built to close.

The **quality-adjusted life year (QALY)** combines duration of time in a health state with a **utility weight** reflecting how good or bad that state is. Formally:

$$\text{QALY} = \sum_i t_i \times u_i$$

where t_i is the time (in years) spent in health state i, and u_i is the utility weight for that state on a scale anchored at $u = 1$ for full health and $u = 0$ for dead. Crucially, the scale is not bounded below at zero: health states judged worse than death (severe, unremitting pain with no prospect of recovery, for example) can carry negative utility weights, reflecting that a respondent would trade some of their remaining life expectancy to avoid that state entirely, or would prefer death to it. Utility weights are typically elicited using standardized preference-based instruments (EQ-5D, SF-6D, HUI) administered to patients or to general-population samples, then converted to utilities via a country-specific value set derived from techniques such as time trade-off or standard gamble.

Difficult terminology: time trade-off / standard gamble. Both are standard techniques for turning "how bad is this health state" into a number between 0 (dead) and 1 (full health) that a QALY calculation can use — they ask a respondent (patient or member of the public) to express that judgement through a concrete choice, rather than simply rating the state on an abstract scale. **Time trade-off** asks: given a fixed number of years to live in the impaired health state being valued, how many of those years would you be willing to give up in exchange for the remaining years being in full health instead? Someone willing to trade 2 years out of 10 to be fully healthy for the remaining 8 implies a utility of 0.8 for that state — the less of their remaining life they'd sacrifice, the closer to 1.0 (full health) the implied utility. **Standard gamble** instead offers a choice between the impaired state for certain, versus a gamble: a probability p of full, instant recovery to perfect health, and a probability $(1 - p)$ of immediate death. The respondent's indifference point — the probability p at which they can no longer decide whether they prefer the certain impaired state or the gamble — is taken as the utility value of that impaired state. A concrete worked example: if a respondent says

they would accept a 25% chance of death in exchange for a 75% chance of being cured, rather than continuing to live with their current health problem for certain, their utility for that health state is read off as 0.75. Both methods are more cognitively demanding than simply asking someone to rate a state "out of 10," but are grounded in a formal economic theory of choice under uncertainty (standard gamble) or time preference (time trade-off), which is why country-specific EQ-5D value sets are built from large population samples answering one or the other.

```
# Simple QALY calculation across two health states
time_in_full_health   <- 2.0   # years
utility_full_health   <- 1.0

time_in_impaired_state <- 3.0  # years
utility_impaired_state  <- 0.6

qalys <- time_in_full_health * utility_full_health +
         time_in_impaired_state * utility_impaired_state
qalys
```

Two years at full health (utility 1.0) contribute 2.0 QALYs; three further years at a utility of 0.6 (reflecting, say, moderate chronic pain and reduced mobility) contribute 1.8 QALYs; the patient's total is 3.8 QALYs over five calendar years of survival — a single number that simultaneously encodes both how long and how well the patient lived. An incremental analysis compares the QALYs accumulated under a new technology against the QALYs accumulated under the comparator, and that QALY difference becomes the denominator of the ICER introduced in Chapter 1. The QALY is explicitly a **good**: more QALYs is always better, and interventions are evaluated by how many QALYs they gain.

The **disability-adjusted life year (DALY)**, by contrast, comes from a different tradition entirely — the WHO/World Bank Global Burden of Disease programme, designed to quantify population-level disease burden rather than to evaluate individual health technologies. A DALY is constructed as:

$$\text{DALY} = \text{YLL} + \text{YLD}$$

where YLL (years of life lost) captures premature mortality relative to a reference life expectancy, and YLD (years lived with disability) captures time lived in a less-than-full-health state, weighted by a disability weight (conceptually similar to, but methodologically distinct from, a QALY utility weight, and typically derived from large multi-country disability-weight surveys rather than patient-elicited utilities). The DALY is explicitly a **bad**: it measures burden, and the policy goal is to minimize DALYs, not maximize them. An intervention's benefit in DALY terms is expressed as "DALYs averted" — the reduction in burden the intervention achieves relative to no intervention — which is a positive good, even though the underlying DALY count itself represents harm.

Agency perspective. WHO's Global Burden of Disease framework uses the DALY as its standard metric for quantifying population-level disease burden and informing global health priority-setting; this is a distinct convention from, and should not be conflated with, the QALY-based cost-utility reference cases used by technology-appraisal HTA bodies such as NICE and HTAIn. Some national HTA systems (several Asian and a number of low- and middle-income country adaptations) do permit DALY-averted as an acceptable alternative effect measure to

the QALY in cost-effectiveness submissions, so the choice between QALY and DALY conventions is itself a jurisdiction-specific methodological decision to check in any agency's reference case before modelling.

Common pitfall. QALYs and DALYs run in **opposite** directions, and mixing up the sign convention is a common and consequential modelling error. A QALY is a good: a successful treatment should **increase** total QALYs relative to the comparator, and the ICER is cost divided by QALYs **gained**. A raw DALY is a burden: a successful treatment should **decrease** total DALYs, and the relevant effectiveness quantity for a cost-effectiveness ratio is DALYs **averted** (comparator DALYs minus intervention DALYs), not the raw DALY count itself. Coding a model that subtracts intervention QALYs from comparator QALYs (the DALY-style direction) — or divides cost by raw DALYs accrued rather than DALYs averted — will silently produce an ICER with the wrong sign or the wrong magnitude. When porting a published DALY-based burden estimate into an R cost-effectiveness model, explicitly convert it to "DALYs averted" and sanity-check the sign before it ever reaches your ICER formula.

4.4 ICERs, net monetary benefit (NMB) and net health benefit (NHB)

Chapter 1 introduced the incremental cost-effectiveness ratio in its canonical form, $\text{ICER} = \frac{\Delta C}{\Delta E}$, where $\Delta C = C_2 - C_1$ is the incremental cost of a new treatment (treatment 2) relative to a comparator (treatment 1), and ΔE is the incremental effect, almost always measured in quality-adjusted life years (QALYs) in HTA work. The ICER answers a narrow but useful question: how much extra does it cost, per unit of extra health gained, to adopt the new treatment instead of the old one?

Consider a worked example. A new oral anticoagulant costs £1,800 per patient per year versus £400 for standard warfarin monitoring, and modelling over a patient's lifetime projects 0.28 additional QALYs for the new drug due to fewer strokes and bleeds.

```
cost_new <- 1800 * 12   # simplified lifetime cost proxy, GBP
cost_comp <- 400 * 12
qaly_new <- 8.42
qaly_comp <- 8.14

delta_C <- cost_new - cost_comp
delta_E <- qaly_new - qaly_comp
icer <- delta_C / delta_E
icer
```

This returns an ICER of £60,000 per QALY gained (ΔC = £16,800, ΔE = 0.28) — a figure a reimbursement committee can compare directly against a threshold (§4.5). At a £20,000/QALY threshold, this drug would not be considered cost-effective on this evidence.

The ratio format, however, conceals as much as it reveals. Two well-known problems arise:

- **Sign ambiguity.** A negative ICER is uninformative on its own: it can arise from a treatment that is more effective and cheaper (dominant — unambiguously good), or from one that is less effective

and more expensive (dominated — unambiguously bad). Both quadrants of the cost-effectiveness plane produce the same algebraic sign, so the ICER alone cannot distinguish "obviously adopt" from "obviously reject."

- **Non-monotonicity near zero.** As ΔE approaches zero from either side, the ICER magnitude explodes toward infinity even for small, economically unimportant differences in cost, making ratios unstable for ranking or for probabilistic sensitivity analysis summaries.

Common pitfall. A reviewer reports: "Drug X is dominated by Drug Y because the ICER is negative (-£2,500/QALY)." This is only sometimes true. Plot the four quadrants: in the **north-west** quadrant (more effective, less costly — Drug X dominant), $\Delta C < 0$ and $\Delta E > 0$, giving a negative ratio that means "adopt." In the **south-east** quadrant (less effective, more costly — Drug X dominated), $\Delta C > 0$ and $\Delta E < 0$, also giving a negative ratio, but meaning "reject." Never report a negative ICER without stating which quadrant it comes from; always accompany it with the disaggregated ΔC and ΔE signs, or better, switch to net benefit.

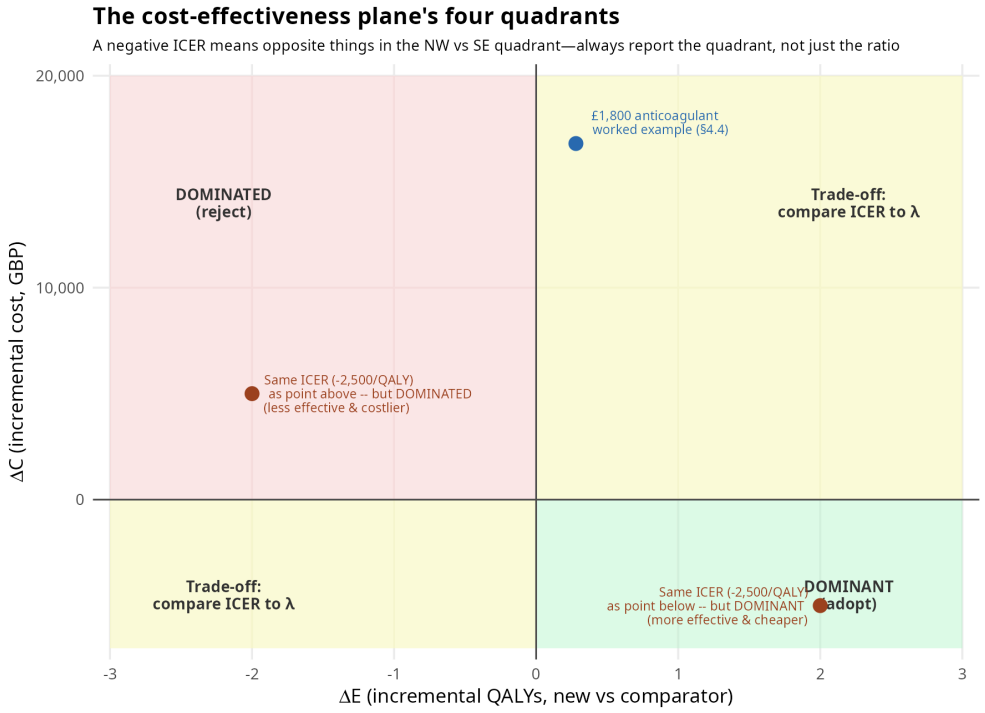


Figure 4: The cost-effectiveness plane's four quadrants. The two orange points share the identical ICER (-£2,500/QALY) but sit in opposite quadrants with opposite real-world meanings — dominant (adopt) versus dominated (reject) — exactly the ambiguity the pitfall above warns against. The blue point plots this section's own anticoagulant worked example (ΔE = 0.28, ΔC = £16,800), which falls in the north-east trade-off quadrant where neither sign alone resolves the decision and the ICER (or NMB) must actually be compared against a threshold.

The standard remedy is to convert the ratio into a linear measure by fixing a willingness-to-pay (WTP) threshold, λ, for one additional QALY. Two equivalent formulations exist:

$$\text{NMB} = (\Delta E \times \lambda) - \Delta C$$

$$\text{NHB} = \Delta E - \frac{\Delta C}{\lambda}$$

NMB expresses net benefit in monetary units; NHB expresses it in health units (QALYs). Because both are linear in ΔC and ΔE, they are well-behaved however small the denominator would have been, they aggregate cleanly across subgroups or probabilistic simulation draws, and — critically — the decision rule becomes unambiguous: **a positive NMB (or NHB) at threshold λ means the intervention is cost-effective at that threshold; negative means it is not.** No quadrant-reading is required.

```
lambda <- 20000   # GBP per QALY, illustrative threshold
nmb <- delta_E * lambda - delta_C
nhb <- delta_E - delta_C / lambda
nmb; nhb
```

With ΔE = 0.28 and ΔC = £16,800: NMB = 0.28 × £20,000 − £16,800 = £5,600 − £16,800 = −£11,200. NMB is negative, so at a £20,000/QALY threshold this drug is **not** cost-effective — exactly the same conclusion the £60,000/QALY ICER already pointed to (£60,000 exceeds the £20,000 threshold). NMB and the ICER decision rule are always mathematically consistent for a single comparator when $\Delta E > 0$ and λ is fixed; NMB's real advantage is not a different conclusion but a **linear**, sign-unambiguous one that remains well-defined even when ΔE is close to zero, which is precisely why it is the quantity averaged across probabilistic sensitivity analysis draws rather than the ICER itself.

How to report this.

Treatment	Cost (GBP)	QALYs	Incremental cost	Incremental QALYs
Standard warfarin monitoring	4,800	8.14	–	–
New oral anticoagulant	21,600	8.42	16,800	0.28
ICER = £60,000 per QALY gained			**NMB** at λ = £20,000/QALY = −£11,200 (not cost-effective at this threshold)	

Table 6: A CHEERS 2022-style cost-effectiveness results table for the anticoagulant worked example — the standard way an ICER/NMB result is reported in a paper or HTA submission, rather than left as bare console output.

HEOR in practice. In practice, NMB is the default output of a Bayesian or bootstrap probabilistic sensitivity analysis: for each of, say, 10,000 simulated parameter draws, compute NMB at the threshold of interest, then summarise the proportion of draws with positive NMB as the "probability cost-effective." This is the basis of cost-effectiveness acceptability curves (CEACs), covered in Chapter 25. Reporting an ICER confidence interval from simulated draws is often nonsensical (ratios of near-zero denominators), whereas an NMB distribution is always well-defined.

4.5 Willingness-to-pay thresholds, including country-specific and LMIC thresholds

A willingness-to-pay (WTP) or cost-effectiveness threshold, denoted λ in §4.4, represents the maximum amount a payer is prepared to pay for one additional unit of health gained — conventionally one QALY. It is the single parameter that converts an ICER (or NMB/NHB) from a descriptive statistic into an actionable reimbursement decision: if ICER $< \lambda$, the intervention is judged cost-effective at that threshold; if ICER $> \lambda$, it is not (subject to the quadrant caveats of §4.4).

The most widely cited threshold range in the HTA literature is NICE's (England and Wales) historical £20,000–£30,000 per QALY range, used since the early 2000s as an informal reference point in appraisals, with values above £30,000 requiring increasingly strong justification (e.g., end-of-life criteria, severity modifiers) and values below £20,000 treated as straightforwardly acceptable. It is worth being explicit that this range was never a hard legislative threshold, was not derived from a formal empirical exercise at the time of adoption, and has been persistently criticised — including by health economists associated with NICE's own methods reviews — as arbitrary and not clearly linked to the opportunity cost of displaced NHS spending. Claxton and colleagues' empirical work estimating the NHS's own marginal productivity (health forgone per pound spent elsewhere in the system) suggested a supply-side threshold closer to £12,000–£15,000 per QALY, considerably below the £20,000–£30,000 range, triggering substantial debate about whether NICE's reference range was too generous.

> **Agency perspective.** NICE's current reference case for threshold use and modifiers is set out in **NICE Health Technology Evaluations: The Manual** (PMG36, 2022), which retains the £20,000–£30,000/QALY range for standard appraisals, allows an additional severity weighting (replacing the older "end-of-life" criteria) for conditions with greater absolute or proportional QALY shortfall, and directs analysts to NICE DSU technical support documents for the quantitative severity-modifier calculations. Students producing a submission for NICE should cite the Manual directly, not a secondary paraphrase of the threshold range.

For low- and middle-income countries (LMICs), the historically dominant heuristic came from the WHO's Choosing Interventions that are Cost-Effective (WHO-CHOICE) programme, which proposed classifying interventions as "very cost-effective" below one times GDP per capita per QALY/DALY averted, and "cost-effective" between one and three times GDP per capita, with anything above three times GDP per capita judged not cost-effective. This 1–3x GDP-per-capita rule was administratively convenient and widely adopted in early LMIC HTA guidance, but it has been substantially abandoned by the WHO itself: a widely cited 2016 WHO Bulletin piece (Bertram et al., "Cost-effectiveness thresholds: pros and cons") explicitly recommended retiring the GDP-multiple heuristic, arguing it has no sound empirical basis, is unrelated to a country's actual health budget or opportunity cost, and tends to label far too many interventions as "cost-effective" relative to what health systems can actually afford.

> **Agency perspective.** WHO's updated position, reflected in WHO-CHOICE guidance and the 2016 Bulletin of the World Health Organization commentary by Bertram, Lauer, Joncheere, Edejer, Baltussen and colleagues, favours deriving thresholds from the **opportunity cost of displaced health spending within the country's own budget** — analogous to Claxton's

NHS-marginal-productivity approach — rather than an arbitrary GDP multiple or a stated willingness-to-pay survey figure.

This opportunity-cost framing sits opposite a second methodological tradition: stated- or revealed-preference willingness-to-pay surveys, which ask members of the public how much they would personally pay (or how much compensation they would require) for a given health gain, typically yielding higher thresholds than supply-side opportunity-cost estimates because they reflect individual valuation of health rather than a health system's binding budget constraint. The debate between these two schools — opportunity cost (what health is displaced if we fund this) versus willingness-to-pay (what society says health is worth) — remains unresolved and is one of the most active areas in health economics methods research; neither is "correct" in an absolute sense, and the choice has first-order consequences for which technologies get funded.

Difficult terminology: revealed preference. "Stated preference" (used just above, and already described elsewhere in this section) means directly asking people what they would pay or accept — a survey. **Revealed preference** takes the opposite approach: instead of asking, it infers how much people value something by observing the actual trade-offs they make in real decisions. The classic example underlying value-of-a-statistical-life estimates (§4.2) is the "wage premium" study: researchers compare otherwise similar jobs that differ mainly in their on-the-job fatality risk (construction versus office work, for instance) and observe how much extra pay workers demand, on average, to accept the riskier job. That observed wage gap, scaled by the size of the risk difference, reveals an implicit price workers are putting on their own safety through real choices they actually made — not through a hypothetical questionnaire answer. The general principle extends beyond wages: any real-world choice where people visibly trade one thing for another (buying a safer but more expensive car, choosing a shorter but costlier commute) can, in principle, reveal an implicit valuation, provided the choice is genuinely free and the trade-off can be isolated from other confounding factors.

India's HTA body, HTAIn (Health Technology Assessment in India), operationalised a country-specific threshold rather than adopting the WHO GDP-multiple rule.

Agency perspective. HTAIn's **Methods and Processes Manual** (operationalised through the Department of Health Research) and subsequent HTAIn-commissioned work (notably India-specific cost-effectiveness threshold studies led by health economics teams around 2018–2021) estimated a supply-side, opportunity-cost-based threshold substantially below one times India's per-capita GDP, reflecting the real resource constraints of India's public health budget rather than a generic multiple. Analysts preparing a submission for HTAIn should reference the current HTAIn threshold value published in its manual, not the superseded WHO 1–3x GDP heuristic.

HEOR in practice. When a jurisdiction's own threshold is unclear or evolving, report results across a **range** of thresholds in a cost-effectiveness acceptability curve (Chapter 25) rather than committing to a single λ. This is standard practice precisely because threshold-setting is contested, and it lets decision-makers apply their own judgement.

Jurisdiction / body	Threshold convention	Approximate value	Methodological basis
NICE (England & Wales)	Fixed reference range, severity-modified	£20,000–£30,000 per QALY	Historical reference range; no formal empirical derivation; Claxton et al. supply-side estimate suggests £12,000–£15,000
CADTH (Canada)	No single published hard threshold	Often benchmarked informally around CAD 50,000 per QALY in practice	Deliberative committee judgement rather than a codified rule; threshold not fixed in CADTH's own methods guidance
HTAIn (India)	Country-specific supply-side threshold	Below 1 times India's per-capita GDP	Opportunity-cost-based estimate from India-specific threshold studies (~2018–2021), not the WHO GDP-multiple rule
WHO-CHOICE (historical)	GDP-per-capita multiple	1x GDP/capita = "very cost-effective"; 1–3x = "cost-effective"	Administrative heuristic; formally recommended for retirement by Bertram et al. 2016 (WHO Bulletin)

Note: The GDP-multiple rule is retained here only as a historical reference point; current WHO guidance favours country-specific opportunity-cost thresholds, consistent with the HTAIn and Claxton/NICE supply-side approaches above.

Table 7: Willingness-to-pay / cost-effectiveness threshold conventions across selected HTA jurisdictions — the kind of comparison table a methods section uses to justify which threshold a submission is benchmarked against.

4.6 Perspective, time horizon, discounting and half-cycle correction

Four further modelling choices — perspective, time horizon, discounting, and (in Markov models) half-cycle correction — jointly determine what counts as a cost or effect, over what period, and how future values are weighted. Each is a source of real controversy and, unlike the ICER/NMB arithmetic above, each requires an explicit judgement call that should be justified and reported transparently.

Perspective defines whose costs and whose effects enter the analysis. The two most common are:

- **Healthcare payer (or healthcare system) perspective**: includes only costs borne by the payer — drug acquisition, administration, monitoring, hospitalisation, adverse event management — and excludes costs falling on patients, families, or other sectors.
- **Societal perspective**: additionally includes productivity losses (time off work, informal caregiver time), patient out-of-pocket costs, transport to appointments, and sometimes costs in other sectors (e.g., social care, criminal justice for some public health interventions).

The choice matters substantially for conditions with large informal-care or productivity burdens (e.g., dementia, severe mental illness): a healthcare-payer analysis can understate the true value of an intervention that reduces caregiver burden even if it does little to reduce direct medical costs. NICE's reference case specifies an NHS-and-Personal-Social-Services perspective for costs (narrower than full societal) paired with QALYs measured across all effects on the patient, while PBAC and others have historically leaned payer-perspective with societal perspective as a supplementary sensitivity analysis.

Time horizon is the period over which costs and effects are modelled. The general principle, stated in essentially all major HTA methods guides, is that the horizon should be long enough to capture **all** costs and effects relevant to the decision — for a one-off curative intervention this might be a few years, but for a chronic, progressive disease (diabetes, chronic kidney disease, most oncology indications) a **lifetime horizon** is typically required, because truncating too early can systematically understate long-run benefits (e.g., avoided complications decades later) or long-run costs (ongoing drug costs, disease progression costs).

> **Common pitfall.** A common and costly modelling error is choosing a short time horizon (e.g., a trial's own follow-up period, often 1–3 years) simply because that is what outcome data are directly observed for, rather than because it genuinely captures the decision-relevant period. For a chronic disease, this almost always biases the ICER against interventions whose main benefit accrues late (e.g., prevention of long-term complications), because the model stops before those benefits are realised while some of the upfront costs are already fully counted. Extrapolation beyond trial follow-up (Chapter 10 and Chapter 19) is then unavoidable and must be justified, not treated as an afterthought.

Discounting addresses the fact that costs and health effects occurring in the future are valued less than those occurring today, reflecting positive time preference (both for money, due to its opportunity cost, and arguably for health itself). A future cost or QALY is converted to its present value by dividing by $(1 + r)^t$, where r is the discount rate and t is the number of years in the future the cost or effect occurs.

```
discount_pv <- function(value, rate, year) {
  value / (1 + rate)^year
}

# A QALY gained in year 10, discounted at NICE's 3.5% rate
discount_pv(value = 1, rate = 0.035, year = 10)
```

> **Agency perspective.** NICE's **Health Technology Evaluations: The Manual** (PMG36, 2022) specifies a discount rate of 3.5% per annum for both costs and health effects as the reference-case base case, consistent with UK Treasury Green Book conventions for public-

sector appraisal, with sensitivity analysis at 1.5% for health effects permitted only in narrowly defined circumstances (very long-term, substantial health gains). This "equal discounting" of costs and QALYs at the same rate is itself a resolved policy choice following an extended methodological debate.

That debate concerns **differential discounting**: some health economists (echoing earlier work by Claxton and colleagues, and reflected in US Panel on Cost-Effectiveness in Health and Medicine recommendations at various points) have argued health effects should be discounted at a **lower** rate than costs, because health does not have the same investment/opportunity-cost properties as money and because discounting health heavily can appear to penalise prevention relative to treatment. Most national HTA bodies, however, have settled on equal discounting of costs and effects at a single rate (3.5% at NICE; comparable single-rate conventions are used by CADTH and others) for consistency and to avoid distortions where differential rates can make almost any sufficiently long-term prevention programme appear cost-saving regardless of its actual value.

HTA body	Reference-case perspective	Time horizon	Discount rate (costs / effects)
NICE (England & Wales)	NHS and Personal Social Services	Long enough to capture all relevant costs/effects; typically lifetime for chronic disease	3.5% / 3.5% (equal); 1.5% on effects only in narrow long-term, large-benefit cases
CADTH (Canada)	Publicly funded health care payer (base case); societal as sensitivity analysis	Lifetime horizon for chronic/progressive conditions	1.5% / 1.5% (equal, per current CADTH guidance)
HTAIn (India)	Health system / payer perspective	Lifetime horizon recommended for chronic conditions	3% / 3% (equal), per HTAIn Methods and Processes Manual

Note: Values shown are reference-case defaults; most agencies also require sensitivity analyses varying perspective, horizon, and discount rate from these defaults. Consult each agency's current methods manual before a live submission – rates and conventions are periodically revised.

Table 8: Perspective, time horizon and discount rate reference-case conventions across selected HTA bodies — the modelling-choices summary a methods section typically tabulates before presenting results.

Half-cycle correction is a technical adjustment specific to Markov cohort models (introduced formally in Chapter 19), noted briefly here because it interacts directly with discounting and time horizon. A Markov model tracks a hypothetical cohort moving between health states in discrete cycles (e.g., annual cycles), and by convention accumulates costs and QALYs as if the entire cohort occupies its cycle-start health state for the whole cycle. In reality, transitions between states (e.g., disease progression, death) occur continuously throughout the cycle, so a naive cycle-start (or cycle-end) accounting overstates (or understates) time spent in each state by roughly half a cycle's

worth. Half-cycle correction adjusts for this by assuming, on average, that transitions occur at the cycle midpoint — commonly implemented as a simple average of the cycle-start and cycle-end state occupancy, or via Simpson's-rule-weighted approaches for greater accuracy with longer cycle lengths. The correction typically has a modest but non-trivial effect on both costs and QALYs, and omitting it is a recognised, checkable error in submitted economic models. Full mechanics, including worked trace calculations and the Simpson's-rule variant, are deferred to Chapter 19′s dedicated treatment of cohort Markov models in R.

Difficult terminology: Simpson's rule (half-cycle correction variant). Simpson's rule is a general numerical technique, unrelated to health economics specifically, for estimating the area under a curve more accurately than a plain average would. The simplest "half-cycle" correction described just above averages only two points per cycle — the state occupancy at the cycle's start and at its end — effectively approximating the true, continuously-curving occupancy over that cycle with a straight line between those two points. Simpson's rule improves on this by also using the occupancy at the cycle's **midpoint** and fitting a short parabolic (curved) segment through all three points rather than a straight line, which tracks a genuinely curving trend much more closely, particularly when cycles are long or the underlying transition probabilities change quickly. A practical analogy: estimating the distance a car travelled using only its speed at the start and end of an hour (the simple average) assumes it accelerated in a straight line; checking its speed at the halfway mark too, and fitting a smooth curve through all three readings (Simpson's rule), gives a noticeably better estimate if the car sped up and slowed down unevenly during that hour rather than changing speed at a constant rate. In R-based Markov models, this usually means sampling state-occupancy at an extra set of mid-cycle time points and combining them with a specific weighted-average formula, rather than performing any genuinely novel calculus.

HEOR in practice. When reviewing someone else's Markov model, check three things together, since they interact: the time horizon is long enough (ideally lifetime for chronic disease), the discount rate matches the target jurisdiction's reference case (e.g., 3.5% equal discounting for a NICE submission), and half-cycle correction (or an equivalent continuous-time correction) has been applied. Getting the first two right while omitting the third still produces a systematically biased ICER.

4.7 Dominance, extended dominance and the efficiency frontier

Section 4.4 gave you the ICER as a ratio, $\Delta C / \Delta E$, and left a loose thread: an ICER only tells you something useful once you know **which two options** belong on either side of the slash. Compare the wrong pair — a brand-new drug against some arbitrary older comparator that was never really in contention — and the number you compute is technically arithmetic but economically meaningless. This section ties that thread off by introducing the idea of **dominance** and the **efficiency frontier**, the two concepts that tell you which comparisons are legitimate in the first place.

Start with the simplest case. Suppose Treatment A costs more than Treatment B **and** delivers fewer QALYs than Treatment B. A is strictly worse on both axes: no rational decision-maker would ever choose it, regardless of budget or willingness to pay. We say A is **dominated** (specifically, **simply**

or **strictly** dominated) by B, and the correct analytical move is to delete A from the comparison set entirely. You do not compute an ICER for a dominated option — dividing a positive cost difference by a negative effect difference produces a number that looks like a ratio but describes a trade nobody would make. Simple dominance is easy to spot on a cost-effectiveness plane (cost on the vertical axis, effect on the horizontal axis): A sits up and to the left of B, in the "worse in every way" quadrant relative to B.

Extended dominance is subtler and is the piece most newcomers miss. A treatment can fail to be dominated by **any single alternative** and still be inefficient, because it is dominated by a **mix** of two other treatments. Picture three non-dominated options B, C, and D, ordered by increasing cost and increasing effect, plotted on the cost-effectiveness plane. Draw a straight line connecting B and D. If C's point lies **above** that line — more costly, for its effect, than some hypothetical blend of B and D obtained by giving a fraction of patients B and the rest D — then C is extendedly dominated. A decision-maker could, in principle, achieve C's average effectiveness at lower average cost by mixing B and D (or, more realistically, by simply recognising that C offers a worse incremental trade than the B-to-D step does directly). C is therefore excluded from the efficient set, even though neither B nor D alone beats it on both cost and effect.

This geometric test — lying above the line connecting the two neighbouring non-dominated points — is exactly equivalent to a numerical test you will use constantly from here on: C is extendedly dominated whenever its ICER against the next-cheaper non-dominated option exceeds its ICER against the next-more-expensive one. When ICERs stop increasing monotonically as you move up the cost ranking, you have found an extended-dominance violation, and the offending option drops out.

Once all simply and extendedly dominated options are removed, what remains is the **efficiency frontier**: the set of treatments for which every additional unit of cost buys you additional effect at a non-decreasing rate, when you walk the ranking from cheapest to most expensive. Plotted on the cost-effectiveness plane, the frontier is a piecewise-linear curve connecting these surviving points, convex from below, with ICERs between adjacent segments that rise (or stay flat) as you move rightward. Any point below this curve is infeasible given the available technologies; any point above it (whether literally dominated or extendedly dominated) is inefficient and has already been pruned.

This is why the rule — compare adjacent alternatives, not an arbitrary comparator — is not a stylistic preference but a structural necessity. An ICER is only interpretable as "the price of the next increment of health" when the two options being compared are **adjacent on the frontier**: nothing cheaper-and-nearly-as-effective, and nothing dearer-and-better, sits between them. Comparing a frontier option against a non-adjacent one, or against something that should already have been eliminated, manufactures a ratio with no decision-relevant meaning, however clean the arithmetic looks.

> **Common pitfall.** A persistent error in student work and, occasionally, in submitted dossiers: computing the ICER of a new treatment against "standard of care" without first checking whether standard of care, or the new treatment, has been dominated or extendedly dominated by something else in the comparator set. Always sort by cost, strip out simply dominated options, check for and strip out extendedly dominated options, **then** compute ICERs only between what remains, moving strictly from one adjacent frontier point to the next. An ICER

against a non-adjacent or dominated comparator is not merely imprecise — it is answering a question nobody asked.

Agency perspective. NICE's health technology evaluation manual (PMG36, 2022) is explicit that analyses submitted for appraisal should present a **fully incremental analysis**: all relevant, non-dominated comparators ranked by cost, with ICERs reported between adjacent options on the efficiency frontier, rather than a series of pairwise comparisons against a single fixed comparator chosen by the manufacturer. The ISPOR-SMDM Modeling Good Research Practices Task Force reports (Caro et al., 2012, published in **Value in Health** and **Medical Decision Making**) reinforce the same expectation for decision-analytic models generally: dominated and extendedly dominated strategies must be identified and excluded before any ICER is presented as evidence for reimbursement decision-making.

4.8 Coding a first simple CEA in R

Three chapters of scaffolding — ICER notation, the HTA agency landscape, the four callout conventions you've been reading throughout this book, and a tour of `dplyr`, `purrr`, and function-writing in R — have all been building toward this moment. This is the first section in the entire book where R code actually **performs a health-economic calculation** rather than illustrating a general-purpose programming idiom. Everything from here on builds on the pattern you are about to write: a tidyverse pipeline that takes raw cost and effect data, prunes it down to an efficiency frontier, and reports incremental results. Treat this as a milestone worth lingering on — type it in, run it, and watch the frontier logic from §4.7 fall out of five `dplyr` verbs.

The toy scenario: a hypothetical chronic condition with four candidate treatments — a generic standard of care and three newer options — each characterised, for this illustration, by a single deterministic cost and a single deterministic QALY estimate (we are deliberately ignoring uncertainty for now; Chapter 25 will put proper probabilistic distributions around every one of these point estimates and propagate them through exactly this kind of frontier logic via probabilistic sensitivity analysis).

```
library(dplyr)
library(tibble)

# Step 1: define the toy treatment data ------------------------------
treatments <- tibble(
  treatment = c("Standard Care", "Drug A", "Drug B", "Drug C"),
  cost      = c(1200,            4800,     5100,     9000),
  qaly      = c(0.62,            0.81,     0.74,     0.95)
)

treatments
```

The data frame has one row per treatment and exactly the two columns a cost-effectiveness analysis needs at minimum: `cost` and `qaly`. Note that Drug B costs more than Drug A but delivers **fewer** QALYs — a hint of trouble we'll let the pipeline catch rather than spotting it by eye.

```r
# Step 2: rank treatments from cheapest to most expensive -------------
ranked <- treatments |>
  arrange(cost)

ranked
```

```r
# Step 3: remove simply dominated treatments --------------------------
# A treatment is simply dominated if some cheaper treatment already
# delivers equal or greater QALYs. We flag and drop these before doing
# any incremental arithmetic at all.
remove_simple_dominance <- function(df) {
  df <- df |> arrange(cost)
  keep <- logical(nrow(df))
  best_qaly_so_far <- -Inf
  for (i in seq_len(nrow(df))) {
    if (df$qaly[i] > best_qaly_so_far) {
      keep[i] <- TRUE
      best_qaly_so_far <- df$qaly[i]
    } else {
      keep[i] <- FALSE   # cheaper option already matches/beats this QALY
    }
  }
  df |> filter(keep)
}

non_dominated <- remove_simple_dominance(ranked)
non_dominated
```

Running this on our four rows drops Drug B: it costs more than Drug A (£5,100 vs £4,800) while delivering fewer QALYs (0.74 vs 0.81) — textbook simple dominance from §4.7. Standard Care, Drug A, and Drug C survive, each cheaper-but-worse or dearer-but-better than its neighbour.

```r
# Step 4: check for extended dominance --------------------------------
# A survivor of simple dominance can still be extendedly dominated if its
# ICER against the next-cheaper option exceeds its ICER against the
# next-more-expensive option (i.e. ICERs are not rising as cost rises).
check_extended_dominance <- function(df) {
  df <- df |>
    arrange(cost) |>
    mutate(
      icer_vs_prev = (cost - lag(cost)) / (qaly - lag(qaly))
    )

  repeat {
    icers <- df$icer_vs_prev
    violation <- which(icers < lag(icers))[1]  # ICER drops vs previous step
    if (is.na(violation) || length(violation) == 0) break
    df <- df |>
      filter(treatment != df$treatment[violation]) |>
      mutate(icer_vs_prev = (cost - lag(cost)) / (qaly - lag(qaly)))
  }
  df
}

frontier <- check_extended_dominance(non_dominated)
```

With only three survivors in this toy example, ICERs happen to rise monotonically already (Standard Care → Drug A → Drug C), so no further rows are removed — but the function is written generally, re-checking after every removal, exactly as it would need to for a larger comparator set where extended dominance bites.

```
# Step 5: build the final frontier table of incremental results ---------
results <- frontier |>
  arrange(cost) |>
  mutate(
    incr_cost = cost - lag(cost),
    incr_qaly = qaly - lag(qaly),
    icer      = incr_cost / incr_qaly
  ) |>
  select(treatment, cost, qaly, incr_cost, incr_qaly, icer)

print(results, digits = 2)
```

```
# A tibble: 3 x 6
  treatment      cost  qaly incr_cost incr_qaly   icer
  <chr>         <dbl> <dbl>     <dbl>     <dbl>  <dbl>
1 Standard Care  1200  0.62        NA        NA     NA
2 Drug A         4800  0.81      3600      0.19  18947
3 Drug C         9000  0.95      4200      0.14  30000
```

Read this table exactly as §4.7 told you to: Standard Care is the frontier's starting point (no comparator cheaper than it exists in this set, so its incremental row is `NA`). Moving to Drug A costs an incremental £18,947 per additional QALY. Moving on from Drug A to Drug C — **not** from Standard Care to Drug C — costs an incremental £30,000 per QALY. Drug B never appears because it was eliminated at Step 3, and no row anywhere compares a treatment to a non-adjacent option. That rising sequence of ICERs, £18,947 then £30,000, is the numerical signature of a valid, extended-dominance-checked efficiency frontier.

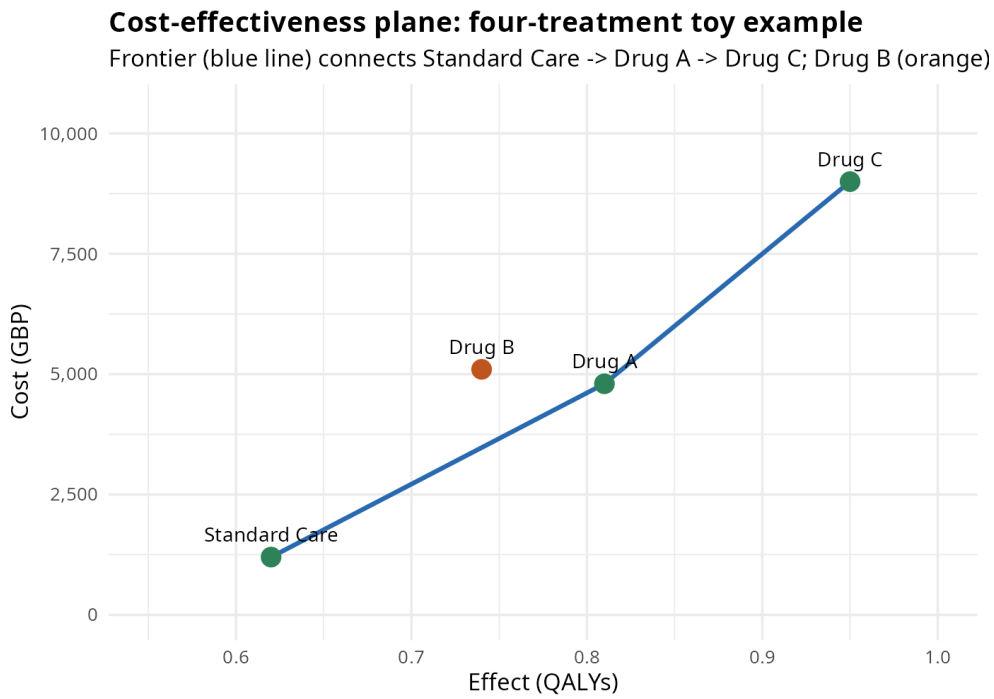


Figure 5: The cost-effectiveness plane for the four-treatment toy example. Drug B (orange) sits above and to the left of Drug A — more expensive, fewer QALYs — and is excluded from the frontier at Step 3. The blue line connects the three surviving, non-dominated treatments in the order the efficiency frontier is built: Standard Care, then Drug A, then Drug C.

How to report this.

Treatment	Cost (GBP)	QALYs	Dominance status	ICER vs next-cheaper (GBP/QALY)
Standard Care	1,200	0.62	On frontier (reference)	–
Drug A	4,800	0.81	On frontier	18,947
Drug B	5,100	0.74	Simply dominated by Drug A (higher cost, fewer QALYs)	Excluded
Drug C	9,000	0.95	On frontier	30,000

Note: ICERs are reported only between adjacent frontier survivors (Standard Care → Drug A → Drug C). Drug B is excluded by simple dominance at Step 3 and never receives an ICER. No extended-dominance violation occurs in this toy data set, since ICERs rise monotonically (£18,947 then £30,000) along the surviving sequence.

Table 9: How the four-treatment frontier analysis would be reported: dominance status, survival of pruning, and the ICER against the next-cheaper non-dominated option.

HEOR in practice. Wrap Steps 3–5 into a single reusable function, say `build_frontier(df)`, that takes any `treatment`/`cost`/`qaly` tibble and returns the pruned, incrementally-costed frontier table. You will reuse this exact pattern — rank by cost, strip simple dominance, strip extended dominance, compute adjacent increments — every time a new decision problem in this book has more than two comparators, including the Markov and partitioned-survival models from Part IV.

Common pitfall. It is tempting to skip straight to `mutate(icer = (cost - cost[1]) / (qaly - qaly[1]))` — comparing everything back to the first row. This silently computes ICERs against a **fixed** baseline rather than the **adjacent** frontier comparator, and it will happily produce a number even for a dominated treatment that should have been deleted at Step 3. Always run the dominance-removal steps first, and always difference against `lag()` on a cost-sorted, already-pruned tibble — never against row 1 by position.

The point estimates in `results` — one cost, one QALY, one ICER per frontier step — are exactly what Chapter 25 will revisit and turn probabilistic: instead of a single `cost = 4800` for Drug A,

you will eventually draw thousands of plausible costs and QALYs from distributions, run this same dominance-and-increment logic on each draw, and summarise the resulting cloud of ICERs with a cost-effectiveness acceptability curve. For now, savor the deterministic version — this five-step `dplyr` pipeline, however small the toy data set, is a complete, decision-grade cost-effectiveness analysis, correctly respecting every rule about dominance and frontiers that §4.7 just laid out.

PART II

Data for HEOR

Chapter 5. Data Sources and Data Management

Learning objectives

- Distinguish CDISC SDTM (collected, domain-organised trial data) from ADaM (analysis-ready derived data), and explain why ADaM datasets are generally preferred as model inputs where available.
- Characterise the distinct strengths, weaknesses and provenance biases of registries, administrative claims, electronic health records, and survey data as real-world data sources.
- Treat published literature as a data source with its own extraction discipline, and explain why publication bias threatens any model parameter pulled from an ad hoc literature search.
- Import health-economic data from flat files, spreadsheets, SAS/Stata/SPSS files, relational databases, and web APIs using `readr`, `readxl`, `haven`, `DBI` and `httr2`.
- Construct and maintain a data dictionary / codebook that documents variable meaning, coding, and derivation logic, including with the `labelled` package.
- Apply programmatic data quality checks (`pointblank`, `assertr`, `validate`) to catch impossible values — negative costs, out-of-range utilities — before they reach a cost-effectiveness pipeline.
- Apply the core concepts of data governance, de-identification, and privacy law (HIPAA, GDPR, India's DPDPA 2023) to patient-level HEOR data handling.

5.1 Clinical trial data (CDISC SDTM/ADaM concepts)

For most HEOR analysts trained in TB programme surveillance or routine public-health M&E, the first encounter with sponsor-provided clinical trial data comes as a jolt. The data are not a single flat line-list you built yourself from a case-based database; they are dozens of interlinked tables governed by a standard you did not write and cannot casually reshape. Understanding that standard — the Clinical Data Interchange Standards Consortium's (CDISC) Study Data Tabulation Model (SDTM) and Analysis Data Model (ADaM) — is a prerequisite for doing economic modelling work that touches regulated trial data, because nearly every phase II/III trial submitted to a regulator in the last fifteen years has been structured this way, and increasingly so are the trials that feed economic submissions to HTA bodies.

SDTM and ADaM solve two different problems, and conflating them is the single most common conceptual error newcomers make. SDTM is the standard for **collected** data: it organises what was observed or recorded during the trial into standardised **domains**, each a rectangular dataset with a controlled set of variables. `DM` holds one row per subject with demographics (age, sex, arm assignment, dates of first/last exposure); `AE` holds one row per adverse event per subject (with standardised terms, severity grades, relatedness, onset/resolution dates); `LB` holds laboratory results in long format (one row per subject per visit per analyte); `EX` holds exposure/dosing records; `VS` holds vital signs; and so on through dozens of other two-letter domain codes. SDTM datasets are deliberately close to raw collection — they are not yet statistically "analysis-ready." A single AE record, for instance, does not by itself tell you whether that event counts toward a "treatment-

emergent adverse event" (TEAE) analysis population, because that determination requires business rules (did the event start after first dose? within a specified window?) that are not SDTM's job to encode.

ADaM is the standard for **analysis-ready** derived data built **from** SDTM, not alongside it or instead of it. The cornerstone ADaM dataset is `ADSL` (Subject-Level Analysis dataset), one row per subject carrying derived flags and population indicators — intent-to-treat flag, safety population flag, treatment duration, baseline disease severity category — that get merged onto every other analysis dataset so that subgroup and population definitions stay consistent across every output in the submission. Built on top of ADSL are domain-specific analysis datasets such as `ADAE` (derived from SDTM `AE` plus ADSL, now carrying the TEAE flag, duration calculations, and toxicity grade shifts) or `ADLB` (derived from SDTM `LB`, now carrying baseline values, change-from-baseline, and shift-table categories attached to each record). The relationship is strictly directional: SDTM feeds ADaM, ADaM feeds statistical analysis and tables/figures/listings, and nothing flows backward. For an HEOR analyst building a cost-effectiveness model off trial data, this matters practically — you want ADaM datasets (particularly ADSL and ADAE/ADTTE-style time-to-event datasets) wherever they exist, because the analysis-ready derivations (populations, flags, time-to-event censoring rules) have already been vetted against the statistical analysis plan. Going back to raw SDTM domains and re-deriving those flags yourself is both more work and a source of discrepancy risk against the trial's own published results.

> **Agency perspective.** CDISC's SDTM and ADaM Implementation Guides are the data standards named explicitly in the FDA's Data Standards Catalog and required for electronic study data submitted to FDA (and, for ICH-aligned submissions, broadly expected by other ICH regulators) for the relevant study start dates covered by that catalog. ICH E3 ("Structure and Content of Clinical Study Reports") separately governs how the **narrative** clinical study report is organised, including appendices that reference the underlying datasets; it does not itself define a data model, but study reports built to ICH E3 are now routinely accompanied by SDTM/ADaM datasets and define-XML metadata. An HEOR analyst requesting trial data from a sponsor's clinical data management or biostatistics group should expect ADaM datasets with accompanying define.xml metadata files describing every variable's derivation — reading that define.xml before touching the data is non-negotiable, because variable names recur across domains with domain-specific meanings.

In practice you will rarely receive true SAS transport (`.xpt`) files directly for ad hoc exploratory modelling work; more often a biostatistics partner exports a derived extract. But knowing the domain structure lets you sanity-check what you're given. A minimal, illustrative read of an ADaM-style subject-level dataset in R might look like this:

```
library(haven)
library(dplyr)

# Read an ADaM subject-level analysis dataset (ADSL) from SAS transport format
adsl <- read_xpt("adsl.xpt")

# ADSL carries one row per subject with population flags already derived
adsl_summary <- adsl |>
  filter(SAFFL == "Y") |>                # safety population flag
  group_by(TRT01P) |>                    # planned treatment, ADaM naming
```

```
convention
  summarise(
    n = n(),
    mean_age = mean(AGE, na.rm = TRUE),
    pct_female = mean(SEX == "F", na.rm = TRUE) * 100
  )

adsl_summary
```

Two ADaM naming conventions are worth internalising early because they recur constantly: variables ending in `FL` are flags (`Y`/`N`/blank), and paired variables like `TRT01P`/`TRT01A` distinguish **planned** treatment (as randomised) from **actual** treatment (as received) — a distinction that matters enormously if your economic model is meant to reflect intention-to-treat effectiveness versus per-protocol or as-treated efficacy assumptions. Getting this pairing wrong is a quiet way to silently misstate your model's analysis population without any error being thrown.

The `adsl_summary` output above is not merely an intermediate check — summarised, by-arm baseline demographics computed straight from ADSL are exactly the content of the "Baseline characteristics" table that opens almost every clinical study report and HTA submission's patient population section.

How to report this.

Treatment arm (TRT01P)	N	Mean age (years)	Female (%)
Placebo	148	61.4	47.3
Standard dose	151	60.8	49.0
High dose	149	62.1	45.6
Safety population (SAFFL = "Y"); age and sex as recorded in ADSL			

Table 10: Baseline demographic summary of the safety population by planned treatment arm (ADSL-derived), the conventional "Table 1" format for clinical study reports and HTA submissions

Common pitfall. Do not merge SDTM domain datasets directly to build your own "ADSL-equivalent" population flags from scratch when ADaM datasets already exist for the trial. SDTM `AE` records, for example, are not pre-filtered to treatment-emergent events, and naively counting all AE rows per arm will systematically overstate adverse event rates relative to the trial's own published safety tables, because it ignores the TEAE windowing logic that the biostatistics team already encoded in `ADAE`. If ADaM datasets are unavailable and you must derive flags from raw SDTM yourself, document every derivation rule and reconcile your subject counts against the trial's CONSORT diagram and published demographic table before proceeding — a mismatch at that stage means your derivation logic is wrong, not the source data.

5.2 Registries, claims, electronic health records and surveys

Outside the tightly governed world of SDTM/ADaM, most of the "real-world" data an HEOR analyst touches comes from four families of sources that look superficially similar — they are all large tables of patient-level information — but differ fundamentally in how they were generated, what they are fit for, and where they will mislead you if you are not careful. The TB-programme instinct to ask "who collected this, for what purpose, and what did they have an incentive to record accurately?" transfers directly here and is, if anything, the single most useful habit you already have.

Disease **registries** are purpose-built, typically prospective, collections of patients with a defined condition, enrolled and followed specifically because they have that condition — a national TB registry, a cancer registry, a rare-disease natural history registry. Because registries are designed around a clinical question, data quality on the variables central to that question (diagnosis confirmation, key outcomes, treatment regimen) tends to be relatively high, and longitudinal follow-up can be genuinely strong. The structural limitation is selection: registry populations are, by construction, the patients who were diagnosed, referred, and consented into the registry, which is rarely a random or representative sample of all patients with the condition — a limitation TB programme analysts will recognise immediately from case-based surveillance versus true population incidence.

Administrative **claims** data — insurance billing records, whether from private payers or public schemes — are generated as a byproduct of the reimbursement process, not for research, and that provenance shapes everything about their strengths and weaknesses. Their great strength is scale and completeness of health-service utilisation: every reimbursed encounter, drug fill, and procedure tends to leave a billing trace, which makes claims data excellent for costing, utilisation patterns, and longitudinal tracking of care pathways across providers — things a single-site registry or trial cannot see. Their great weakness is clinical granularity: claims data record diagnosis and procedure **codes** (ICD, CPT, HCPCS, or local equivalents) attached to a billing event, not direct clinical measurements; there is no lab value, no severity grade, no free-text clinical reasoning, and diagnosis codes are sometimes recorded for billing optimisation rather than clinical precision (a phenomenon generically called "upcoding" that any analyst working with programmatic reporting data will recognise under different names).

Electronic health record (EHR) data sit at nearly the opposite end of the granularity spectrum from claims. EHRs capture rich clinical detail — vital signs, lab trends, clinician notes, imaging reports — generated at the point of care rather than at the point of billing. That richness is also the problem: EHR data are notoriously messy, heterogeneous across institutions and vendor systems, often only locally structured (a lab value field may be clean while diagnosis information lives only in unstructured clinical text requiring natural-language processing to extract), and plagued by interoperability failures when trying to link a patient's record across different health systems or care episodes. Distinguishing "value is genuinely missing" from "value was never measured at this encounter" from "value was recorded somewhere in free text we haven't extracted" is a recurring, unglamorous task in any EHR analysis.

Survey data — whether a dedicated health-utility survey, a national health interview survey, or a patient-reported-outcomes substudy — rely on self-report, and inherit all the familiar biases of self-report instruments: recall bias, social-desirability bias, non-response bias (who agrees to answer a 45-minute survey is rarely a random sample of the target population), and the general gap between

what a patient reports and what a clinical record would show. Surveys remain indispensable, however, for exactly the outcomes that claims and EHR data structurally cannot capture — health-related quality of life, utility weights, out-of-pocket spending, and patient experience — which is precisely why so much of the QALY-generating utility data you'll use in cost-effectiveness modelling (Chapter 8 onward) originates from survey instruments rather than administrative records.

- **Registries**: prospective, condition-defined enrolment; higher data quality on core variables; selected, non-representative populations.
- **Claims**: large-volume, billing-code-based; excellent for utilisation/cost; no direct clinical detail, coding driven partly by reimbursement incentives.
- **EHR**: rich clinical granularity at point of care; messy, heterogeneous, interoperability-limited; extensive missing/unstructured data.
- **Surveys**: only source for genuine self-reported outcomes (utilities, QoL, experience); subject to recall, social-desirability, and non-response bias.

The four source families map onto distinct HEOR modelling use cases rather than being interchangeable, and that mapping is itself the kind of hierarchy a methods section should state explicitly when justifying a parameter's source.

Source type	Typically well suited to	Typically poorly suited to
RCT (SDTM/ ADaM)	Relative treatment effect, trial-period safety and efficacy	Long-term extrapolation beyond follow-up, real-world adherence
Registry	Natural history, long-term outcomes in diagnosed patients	Population-level incidence, undiagnosed/unreferred patients
Administrative claims	Resource use, costing, longitudinal care pathways	Clinical severity, lab-confirmed diagnosis, quality of life
EHR	Clinical event detail, lab trends, comorbidity capture	Cross-system linkage, structured utility/cost extraction
Survey	Health utilities, QALY weights, patient-reported outcomes	Resource-use costing, objective clinical endpoints
Published literature	Comparative effect sizes absent a head-to-head trial, background rates	Patient-level heterogeneity, population-specific calibration

Table 11: Fit-for-purpose mapping of real-world and trial-derived data source types to common HEOR model inputs

A practical first step with any of these four source types in R is the same regardless of provenance: profile missingness and coding structure before doing anything substantive, because the failure modes above show up as specific, checkable patterns in the data.

```
library(dplyr)
library(purrr)

# Generic first-pass profiling for a real-world data extract,
# whether claims, EHR, or registry sourced
profile_source_data <- function(df) {
  map_dfr(names(df), function(col) {
    x <- df[[col]]
```

```
    tibble(
      variable = col,
      pct_missing = round(mean(is.na(x)) * 100, 1),
      n_distinct = n_distinct(x, na.rm = TRUE),
      type = class(x)[1]
    )
  }) |>
    arrange(desc(pct_missing))
}

claims_profile <- profile_source_data(claims_extract)
head(claims_profile, 10)
```

A column with near-zero missingness but a suspiciously small set of distinct values in a claims extract is often a diagnosis code field clustered around a handful of billable categories; a column with 40% missingness in an EHR extract is often a lab value only ordered for a clinical subgroup, not a value missing-at-random — and that distinction has direct consequences for whether later imputation in your model is defensible (a topic picked up again in Chapter 7′s treatment of missing data).

HEOR in practice. When a sponsor or health-system partner hands you a "real-world evidence" extract without documentation, do not take the source label (registry, claims, EHR) at face value — check it against the data's own fingerprints. Billing-code-only fields with no accompanying clinical values and a visit-level grain strongly suggest claims; free-text or heavily missing structured fields alongside visit timestamps suggest EHR; a small number of tightly curated outcome variables with strong longitudinal completeness suggest a purpose-built registry. Getting the provenance right determines which biases you need to defend against in your eventual HTA submission — reviewers will ask.

Agency perspective. NICE's Health Technology Evaluations Manual (PMG36, 2022) explicitly expects companies to justify the choice and quality of real-world data sources used in an evidence submission, including describing data provenance, completeness, and potential biases relative to the decision problem — a registry, claims extract, or EHR cohort cannot simply be asserted as representative of the UK clinical population without supporting justification. Equivalent expectations about real-world evidence transparency and fitness-for-purpose appear in FDA's real-world evidence guidance documents for regulatory decision-making. Analysts should document data source, extraction logic, and known limitations as rigorously as they would document a trial's SDTM/ADaM provenance.

5.3 Published literature as a data source

Not every input to an economic model comes from a dataset you can open in R at all. A large share of the parameters in any cost-effectiveness model — relative treatment effects when no head-to-head trial exists, long-term survival extrapolation anchors, utility decrements for specific health states, background mortality and incidence rates for comparator arms — come instead from the published literature: individual trial publications, systematic reviews, and meta-analyses. Treating published literature as a **data source** in its own right, with its own collection, extraction, and

quality-appraisal discipline, is as important as treating a claims extract or registry with appropriate scepticism about provenance.

The starting point is usually a systematic literature review (SLR), which Chapter 6 covers in depth as a process; what matters here is recognising why that formal process exists rather than simply Google Scholar–searching for a few convenient trials. An ad hoc literature search is vulnerable to exactly the selection problems that plague any non-random sampling exercise: you are more likely to find, read, and cite papers that are highly cited, recently published, available in open access, and — crucially — that report positive, statistically significant results. This last point is the heart of **publication bias**: trials and studies with null or unfavourable results are systematically less likely to be published at all, or are published after longer delay, in lower-visibility journals, or with selective outcome reporting that emphasises secondary endpoints when the primary endpoint was negative. A model built by pulling point estimates from whichever three papers surfaced first on a search is at serious risk of systematically overestimating treatment effect, precisely because positive results are overrepresented in what gets published and found.

Meta-analysis, covered properly in Chapter 11, is the statistical machinery for pooling effect estimates across multiple studies once a systematic review has identified them, and it exists partly as a defence against the limitations of any single trial's precision — but it is not a defence against publication bias itself; a meta-analysis of a biased set of published trials simply produces a precisely wrong pooled estimate. Funnel plots, Egger's test, and trial registry cross-checks (comparing published results against pre-registered trial protocols on registries such as ClinicalTrials.gov) are the standard tools for detecting whether publication bias is likely operating in a given evidence base, and an HEOR analyst pulling a hazard ratio from a published meta-analysis for use in a model should check whether the authors examined this at all before trusting the pooled number uncritically.

> **Difficult terminology: funnel plot / Egger's test.** Both are ways of checking whether a meta-analysis's pooled evidence base looks suspiciously incomplete — specifically, whether small studies with unfavourable results seem to be missing. A **funnel plot** is a scatter chart with each study's estimated effect size (e.g. a risk ratio) on one axis and a measure of that study's precision (often sample size, or the inverse of its standard error) on the other. If publication bias is absent, the plot should look like a roughly symmetric, upside-down funnel: small, imprecise studies scatter widely on both sides of the pooled average, while large, precise studies cluster tightly near it. If small studies showing an unfavourable or null result are more likely to go unpublished, the plot becomes visibly lopsided — small studies appear only on the "favourable" side, leaving a gap on the other, like a funnel with one side sheared off. **Egger's test** is the formal statistical counterpart: rather than eyeballing the plot for asymmetry, it runs a regression of each study's effect size against its precision and tests whether the resulting line is significantly tilted away from zero — a significant tilt is read as statistical evidence of the same asymmetry the funnel plot shows visually. Neither test proves publication bias exists (a real, genuine difference between small and large studies can also cause asymmetry), but both are the field's standard first-line checks before trusting a pooled estimate at face value.

In the interim, before Chapter 6's formal SLR methodology, a reasonable working discipline when extracting parameters from literature for a model is to keep a structured, auditable extraction log rather than copying numbers directly into model code — the same instinct that already governs good practice in extracting data from a clinical dataset applies to extracting data from a PDF.

```
library(tibble)
library(readr)

# A minimal structured extraction log for literature-derived
# model parameters -- auditable, reproducible, and easy to
# cross-check against a later formal SLR in Chapter 6
param_log <- tribble(
  ~parameter,            ~value, ~source_citation,                ~source_type,
~notes,
  "hr_pfs_drugA_vs_soc",   0.62, "Smith et al. 2021, NEJM",         "RCT
publication",  "ITT population, 95% CI 0.48-0.80",
  "utility_progressed",   0.68, "Jones et al. 2019, Qual Life Res", "Utility
study",    "EQ-5D-3L, UK value set",
  "cost_ae_grade3plus",   1450, "National tariff schedule 2023",   "Admin/tariff",
"2023 GBP, not inflated"
)

write_csv(param_log, "literature_parameter_log.csv")
param_log
```

Keeping source type, citation, and context notes alongside each extracted value does two things: it makes the eventual model auditable by a third party (including an HTA reviewer), and it forces you to notice, at extraction time, when a parameter's "source" is actually a secondary citation of a citation rather than the primary study — a surprisingly common and avoidable error.

Common pitfall. Extracting a single point estimate from a published trial or meta-analysis without also recording its confidence interval, the population it applies to, and the follow-up duration is a false economy. When that parameter later needs to flow into a probabilistic sensitivity analysis (Chapter 25), you will either have to re-find the paper under time pressure or, worse, invent a plausible-looking distribution around a number whose true uncertainty you never actually captured. Record the full uncertainty information once, at extraction time, alongside the point estimate.

Agency perspective. NICE's Health Technology Evaluations Manual (PMG36, 2022) requires that evidence submissions describe the search strategy, selection criteria, and quality appraisal used to identify clinical and economic evidence, explicitly so that reviewers can assess the risk of selective or biased evidence identification — a requirement that applies as much to parameters quietly pulled from literature for an economic model as to the clinical evidence base itself. Cochrane's Handbook for Systematic Reviews of Interventions likewise sets out, as the internationally recognised methodological reference, the expectation that a review's search be systematic and its risk-of-bias and publication-bias assessment be reported explicitly rather than assumed absent.

5.4 Importing data: `readr`, `readxl`, `haven`, `DBI`, APIs

A TB programme analyst moving into HEOR work quickly discovers that the hardest part of an analysis is rarely the statistics — it is getting the data into R in a shape you can trust. Cost and outcomes data for HTA submissions arrive in an unglamorous variety of formats: flat files exported

from a surveillance system, Excel workbooks maintained by a finance officer, SAS transport files from a contract research organisation, Stata extracts from a DHS-style household survey, and increasingly, live pulls from a claims warehouse or a public API. This section builds a working toolkit across all five, with an eye toward the habits — explicit column typing, encoding checks, provenance logging — that keep an import pipeline reproducible six months later when a reviewer asks "where did this number come from?"

Delimited and flat files with `readr`

For CSV, TSV, and other delimited files, `readr::read_csv()` is the workhorse. Compared to base R's `read.csv()`, it is faster on large files, guesses column types from a larger sample by default, never silently converts strings to factors, and — critically for reproducibility — prints a column specification you can freeze into the script.

```
library(readr)

# Import quarterly unit-cost data exported from a costing database
unit_costs <- read_csv(
  "data/unit_costs_2023q4.csv",
  col_types = cols(
    facility_id   = col_character(),
    cost_centre   = col_character(),
    unit_cost_usd = col_double(),
    report_date   = col_date(format = "%Y-%m-%d")
  ),
  na = c("", "NA", "N/A", "-99")
)

problems(unit_costs)   # surfaces any parsing failures row-by-row
spec(unit_costs)       # the frozen column specification, paste into the script
```

Pinning `col_types` explicitly rather than accepting `readr`'s guess is the single habit most worth adopting from this section. A silent type mismatch — a patient ID parsed as numeric and losing a leading zero, or a cost column parsed as character because one row contains a stray comma — is exactly the kind of error that survives into a cost-effectiveness model undetected.

> **Common pitfall.** `readr`'s type guessing samples only the first 1,000 rows by default. A numeric-looking ID column that turns alphanumeric at row 1,200, or a currency column where one late row contains `"1,234.56"` with a thousands separator, will parse cleanly up to that point and then silently coerce to `NA` or character from there on. Always pass an explicit `col_types` specification for any file that feeds a submission-facing analysis, and always inspect `problems()` after import — it is empty far less often than you'd expect.

Spreadsheets with `readxl`

Excel remains the lingua franca of health-sector data exchange, and `readxl::read_excel()` reads `.xls` and `.xlsx` without requiring Excel itself or any system dependency — useful on a locked-down analysis server. Its main complications are multi-sheet workbooks, header rows that don't start at row 1, and merged cells that leave ragged blank columns.

```
library(readxl)

# List sheets before committing to one
```

```
excel_sheets("data/registry_extract_2023.xlsx")

registry_raw <- read_excel(
  "data/registry_extract_2023.xlsx",
  sheet = "Enrolment",
  range = "A3:N850",        # skip title rows and a trailing totals row
  col_types = c("text", "text", "date", "numeric", "text",
                "numeric", "numeric", "text", "text", "text",
                "numeric", "text", "text", "numeric"),
  na = c("", "NA", "Not recorded")
)
```

Specifying `range` directly is often more robust than `skip =` plus `n_max =`, because finance and clinical teams frequently insert or delete header rows between workbook versions without warning; a hard-coded range at least fails loudly (wrong number of columns) rather than silently ingesting a title row as data.

Clinical trial and survey formats with `haven`

Clinical trial data exchanged with a CRO, and large household surveys such as DHS or WHO STEPS, are routinely distributed as SAS, Stata, or SPSS files. `haven` reads all three natively and — importantly — preserves value labels and SAS/SPSS "missing" codes as attributes rather than discarding them, which matters a great deal once you reach the codebook work in §5.5.

```
library(haven)

# SAS transport/dataset file (common for SDTM/ADaM exchange)
adsl <- read_sas("data/adsl.sas7bdat")

# Stata .dta (e.g. a DHS-derived household dataset)
survey_dta <- read_dta("data/hh_survey_2022.dta")

# SPSS .sav
qol_spss <- read_sav("data/qol_followup.sav")

# Inspect preserved labelling
attributes(survey_dta$wealth_quintile)
```

A column read with `haven` is typically of class `haven_labelled` — it behaves like a numeric or character vector but carries an attached label and, for SAS/SPSS, a set of user-defined missing codes. Before merging or modelling, convert deliberately rather than letting the labelled class leak into `dplyr` pipelines unexamined:

```
library(dplyr)
library(haven)

survey_clean <- survey_dta |>
  mutate(
    wealth_quintile = as_factor(wealth_quintile),   # labels become factor levels
    age_years       = zap_labels(age_years)         # strip labels, keep plain
numeric
  )
```

Common pitfall. `haven_labelled` columns print deceptively like ordinary numbers in a console or `glimpse()`, but arithmetic and joins on them can behave unexpectedly, and some `tidyverse` verbs will warn or drop the class silently. Decide at import time, column by column, whether you want `as_factor()` (keep the coded meaning) or `zap_labels()` (keep only the numeric value) — don't leave the decision for whoever inherits the script.

Databases with `DBI`

For a claims warehouse, a TB case-based surveillance database, or any registry sitting behind a relational database, `DBI` gives a backend-agnostic connection interface — the same `dbConnect()` / `dbGetQuery()` pattern works whether the backend driver underneath is `RPostgres`, `odbc`, `RSQLite`, or `RMariaDB`. Given your SQL background, this is likely the most immediately comfortable part of the pipeline: write the query exactly as you would in any SQL client, and let `DBI` hand back a data frame.

```
library(DBI)
library(RPostgres)

con <- dbConnect(
  RPostgres::Postgres(),
  dbname   = "claims_warehouse",
  host     = Sys.getenv("CLAIMS_DB_HOST"),
  port     = 5432,
  user     = Sys.getenv("CLAIMS_DB_USER"),
  password = Sys.getenv("CLAIMS_DB_PASSWORD")
)

inpatient_costs <- dbGetQuery(con, "
  SELECT
    patient_id,
    admission_date,
    discharge_date,
    diagnosis_code,
    total_charge_usd
  FROM claims.inpatient_episodes
  WHERE admission_date >= '2022-01-01'
    AND diagnosis_code LIKE 'A15%'   -- respiratory TB, ICD-10
")

dbDisconnect(con)
```

Two habits carry over directly from your SQL work and are worth keeping explicit in R scripts:

- Push filtering and aggregation into the SQL string wherever the database can do it efficiently — pulling an entire multi-year claims table into R memory just to `filter()` it afterward wastes both the database's indexing and your laptop's RAM.
- Never hard-code credentials in a script that might end up in version control; read them from environment variables (`Sys.getenv()`) or a credentials manager, as above.

For parameterised queries — avoiding both SQL injection and the temptation to build query strings with `paste0()` — `DBI::dbGetQuery()` accepts a `params` argument with placeholders:

Difficult terminology: SQL injection. SQL injection is a security flaw that arises whenever a program builds a database query by pasting untrusted text — commonly, text a user typed into a form — directly into the query string, rather than keeping the query's structure and the user-supplied values cleanly separate. If a patient ID field is pasted straight into a query like `"SELECT * FROM patients WHERE id = '" + user_input + "'"`, and a user (malicious or merely careless) enters something like `' OR '1'='1`, the resulting pasted-together query can end up selecting **every** row in the table rather than just one patient's record, or worse, running entirely different commands the original query never intended, because the database cannot tell the difference between "data the user supplied" and "instructions that are part of the query itself" once they've been concatenated into one string. Parameterised queries (as shown just below) avoid this entirely by sending the query's fixed structure and the user-supplied values to the database as two genuinely separate pieces, so a value like `' OR '1'='1` is always treated as a literal, harmless piece of data to search for — never as part of the query's logic — no matter what it contains.

```
dbGetQuery(
  con,
  "SELECT * FROM claims.inpatient_episodes WHERE patient_id = $1",
  params = list("PT-00482")
)
```

HEOR in practice. Many HEOR analyses blend a clinical/claims extract from `DBI` with a published unit-cost list imported via `readxl`, and a trial dataset via `haven`. Keep each source's import in its own clearly commented block with the raw object named by its origin (`claims_raw`, `unitcost_raw`, `trial_raw`) before any joining — it makes the eventual data-management appendix of an HTA submission dossier far easier to write, because you can trace every derived variable back to a single named import step.

Public and institutional APIs

Increasingly, reference data for an HEOR model — WHO Global Health Observatory indicators, World Bank price deflators, or a national TB programme's own API-exposed dashboard — is available as a web API returning JSON rather than a static file. The modern idiomatic approach uses `httr2`, which replaced `httr` as the recommended tidyverse-adjacent HTTP client; the pattern, however, is one you'll recognise from any REST API work.

```
library(httr2)
library(jsonlite)

# Example: WHO Global Health Observatory API (public, no key required)
resp <- request("https://ghoapi.azureedge.net/api") |>
  req_url_path_append("TB_e_inc_num") |>   # TB incidence indicator code
  req_url_query(`$filter` = "SpatialDim eq 'IND'") |>
  req_perform()

resp_check_status(resp)   # raises an informative error on 4xx/5xx

tb_incidence <- resp |>
  resp_body_json(simplifyVector = TRUE)
```

```
tb_incidence_df <- tb_incidence$value   # the JSON payload's tabular "value" array
```

For APIs that require an authentication key, pass it as a header rather than embedding it in the URL, and again source it from an environment variable:

```
resp <- request("https://api.example-registry.org/v1/indicators") |>
  req_headers(Authorization = paste("Bearer", Sys.getenv("REGISTRY_API_KEY"))) |>
  req_perform()
```

`httr2`'s request objects are immutable and pipeable, which makes a multi-step request (path, query parameters, headers, body) read as a clear sequence of transformations rather than a single dense function call — a small ergonomic advantage over the older `httr::GET()` interface, though `httr::GET()` plus `httr::content()` remains perfectly serviceable if you encounter it in an older codebase.

> **Common pitfall.** Public health APIs change their response schema more often than their documentation is updated. Never assume `resp_body_json()`'s structure is stable release to release — print `str()` of the parsed object immediately after a new API integration, and consider caching the raw JSON response to disk (`jsonlite::write_json()`) alongside the parsed data frame, so that a downstream schema change in your parsing code can be debugged against a frozen snapshot rather than a live, possibly-already-changed endpoint.

5.5 Data dictionaries, metadata and codebooks

Importing a file correctly is necessary but not sufficient. Every dataset that feeds an HEOR analysis or HTA submission needs an accompanying data dictionary — sometimes called a codebook — that documents what each variable actually means, independent of the analyst who imported it. This is not bureaucratic overhead; it is the artefact that lets a reviewer, a collaborator, or you yourself in eighteen months reconstruct the logic behind a derived variable without re-reading the whole pipeline.

What a data dictionary must contain

A complete data dictionary, at minimum, documents for every variable:

- **Variable name** as it appears in the dataset, and a short human-readable **label** (e.g. `txn_outcome` → "TB treatment outcome at 12 months").
- **Type and units** — numeric currency in which year's USD, a date in which format, a continuous measure's unit of measurement (mg/dL, QALYs, days).
- **Value code mappings** for categorical variables — e.g. `1 = Cured`, `2 = Completed`, `3 = Failed`, `4 = Died`, `5 = Lost to follow-up`, matching whatever coding scheme the source system used (often a national or WHO-standard classification).
- **Missing-value codes** and their meaning — a claims field coded `-99` for "not applicable" versus `-98` for "known missing" behaves very differently in an analysis, and conflating them silently biases downstream estimates.
- **Derivation logic** for any variable that is not raw input — the exact formula or rule used to compute it, e.g. "`cost_per_episode_usd` = sum of `facility_cost` + `medication_cost` + `transport_reimbursement`, inflated to 2023 USD using the World Bank GDP deflator."

For a dataset built up over several processing stages — raw extract, cleaned, derived-variables-added — it is good practice to maintain the dictionary as a living document updated at each stage, not retrospectively reconstructed at write-up time, which is when details are most likely to be misremembered.

Practical R approaches

The simplest workable codebook is a data frame — one row per variable, with columns for name, label, type, allowed values, and missingness codes — saved alongside the dataset it describes:

```
library(tibble)

codebook <- tribble(
  ~variable,              ~label,                              ~type,
~values,                            ~missing_codes,
  "patient_id",           "Unique patient identifier",          "character", NA,
NA,
  "txn_outcome",          "TB treatment outcome at 12 months",  "integer",
"1=Cured; 2=Completed; 3=Failed; 4=Died; 5=LTFU", "-99=Not applicable;
-98=Unknown",
  "cost_per_episode_usd", "Total treatment cost, 2023 USD",     "double",    NA,
"-99=Not costed"
)

write_csv(codebook, "data/codebooks/tb_outcomes_codebook.csv")
```

This plain-data-frame pattern is transparent, version-controllable, and trivially joinable with the dataset's own column names for an automated consistency check (flagging any dataset column absent from the codebook, or vice versa).

Where the source data already arrives value-labelled — as is typical of `haven`-imported SAS or SPSS files — the `labelled` package lets you preserve and manipulate that labelling natively in R rather than translating it into a separate lookup table:

```
library(labelled)

# Attach value labels directly to a vector, SAS/SPSS style
txn_outcome <- labelled(
  c(1, 2, 3, 1, 4, 5),
  labels = c(Cured = 1, Completed = 2, Failed = 3, Died = 4, "Lost to follow-up" =
5)
)

var_label(txn_outcome) <- "TB treatment outcome at 12 months"

# Generate a human-readable summary straight from the labelled vector
look_for(txn_outcome)        # one-line variable summary with label and value
range
val_labels(txn_outcome)      # the full code -> label mapping

# Convert to factor for modelling, keeping only the chosen labels as levels
txn_outcome_factor <- to_factor(txn_outcome)
```

`labelled` is especially useful when a dataset is re-exported for sharing with a collaborator who expects SPSS- or Stata-style coding; `labelled::write_sav()` and companion functions round-trip

the labels back out into those formats, so the codebook travels with the data rather than living only in a separate document that can drift out of sync.

HEOR in practice. Whichever pattern you choose, generate the codebook programmatically from the dataset wherever possible (column names, types, and observed value ranges pulled directly via code) rather than typed by hand, and re-run that generation step every time the dataset is rebuilt. A hand-maintained codebook reliably drifts out of sync with the data within a few pipeline iterations; a codebook partially generated from the data itself cannot drift on the parts it covers.

Agency perspective. For HTA submissions in particular, data documentation is not optional polish — it is an explicit regulatory and reimbursement-agency expectation. CDISC's **define.xml** standard requires a machine-readable metadata file accompanying SDTM and ADaM datasets submitted to agencies such as FDA and PMDA, documenting every variable's name, label, type, controlled terminology, origin, and derivation algorithm in a structured, agency-parseable format — the formal, standardised cousin of the codebook patterns shown above. NICE's Health Technology Evaluations Manual (PMG36, 2022) likewise sets out expectations that the data and analyses underpinning a submission be transparent and reproducible by the Evidence Review Group, which in practice is unachievable without variable-level documentation of exactly this kind. Treat the codebook as a submission-grade deliverable from the start of an analysis, not an afterthought assembled under deadline pressure once the modelling is finished.

5.6 Data quality checks (`pointblank`, `assertr`, `validate`)

Every dominance and efficiency-frontier computation in Chapter 4 assumed that the input tibble of costs and QALYs was clean: no negative costs, no utilities above 1, no silently missing arms. That assumption rarely survives contact with a real data extract. Claims databases carry transcription errors, trial case report forms arrive with "9999" placeholder codes for missing fields, and TB programme registers accumulate duplicate patient IDs across reporting quarters. Before any cost-effectiveness logic runs, the data must be validated — not eyeballed, but checked programmatically against explicit rules that encode what "valid" means for health-economic data. R has three mature packages for this: `pointblank`, `assertr`, and `validate`, each with a slightly different philosophy but the same goal of turning implicit assumptions into explicit, auditable checks.

`pointblank` builds an **agent** that holds a set of validation rules and then runs them all at once via `interrogate()`, producing a report of which rows and columns failed which rule. This is well suited to a formal, documented QA step — the kind an HTA submission's data management appendix might reference.

```
library(pointblank)
library(dplyr)

# cost_qaly_tbl: one row per treatment arm, as used in Ch.4's
# dominance pipeline (cost, qaly, strategy columns)
cost_qaly_tbl <- tibble::tibble(
  strategy = c("Standard care", "New regimen", "Combination"),
```

```
  cost     = c(1200, 1850, -340),     # note the impossible negative cost
  qaly     = c(0.72, 0.81, 1.04),     # note the impossible utility > 1
  n_missing_followup = c(0, 3, NA)
)

agent <- create_agent(
  tbl = cost_qaly_tbl,
  label = "Cost-QALY QA before dominance analysis"
) |>
  col_vals_gte(columns = cost, value = 0) |>
  col_vals_lte(columns = qaly, value = 1) |>
  col_vals_gte(columns = qaly, value = 0) |>
  col_vals_not_null(columns = c(strategy, cost, qaly)) |>
  interrogate()

agent
# plot/export: get_agent_report(agent, display_table = FALSE)
```

Running `interrogate()` here will flag the "Combination" row twice — once for `cost < 0` and once for `qaly > 1` — **before** a single dominance comparison is attempted. Feeding unvalidated data into the ICER-ranking and dominance logic from Chapter 4 would otherwise produce a plausible-looking but meaningless frontier.

The `pointblank` agent's `interrogate()` report is itself a conventional reporting artefact: HTA submissions' data-management appendices typically summarise exactly this rule-by-rule pass/fail outcome rather than reproducing the raw agent object.

How to report this.

Validation rule	N rows checked	N failed	Pass rate (%)
`cost >= 0` (no negative cost)	3	1	66.7
`qaly <= 1` (utility not exceeding 1)	3	1	66.7
`qaly >= 0` (non-negative utility)	3	0	100.0
`strategy`, `cost`, `qaly` not missing	3	0	100.0
Source: `cost_qaly_tbl`; "Combination" row fails both the cost and utility bounds checks			

Table 12: Data validation summary for the cost-QALY input table, showing rule-by-rule pass/fail counts as reported in a submission's data-management appendix

`assertr` takes a lighter, pipe-native approach, well suited to inline checks inside a `dplyr` pipeline rather than a standalone report:

```
library(assertr)
library(dplyr)

cost_qaly_tbl |>
  verify(nrow(cost_qaly_tbl) > 0) |>
  assert(within_bounds(0, 1), qaly) |>
  assert(within_bounds(0, Inf), cost) |>
```

```
  assert(not_na, strategy, cost, qaly) |>
  # only rows that pass all assertions reach the dominance step
  arrange(cost)
```

`assert()` halts the pipeline with a descriptive error (row index and offending value) the moment a rule is violated, which is useful in an analysis script where you want a hard stop rather than a report to review later. `verify()` checks a logical condition on the whole data frame (e.g. that it is non-empty after a join), and `within_bounds()` is exactly the kind of range check that flags a utility above 1 or a cost below zero.

`validate` separates rule definition from rule application even more explicitly, which scales well when the same rule set needs to be applied to many extracts over time (e.g. successive quarterly TB programme uploads):

```
library(validate)

rules <- validator(
  cost_nonneg   = cost >= 0,
  qaly_bounded  = qaly >= 0 & qaly <= 1,
  strategy_set  = !is.na(strategy),
  followup_ok   = is.na(n_missing_followup) | n_missing_followup >= 0
)

out <- confront(cost_qaly_tbl, rules)
summary(out)
# sapply(out, function(x) sum(!x@expr)) # count fails per rule
```

`confront()` returns a result object that can be summarised, plotted, or written to a log file, giving a persistent audit trail of exactly which rule failed on which row of which dataset version — valuable when a TB programme analyst needs to justify, months later, why a particular district's quarter was excluded from a cost analysis.

HEOR in practice. Treat validation as a pipeline **stage**, not a one-off script you run when something looks wrong. In a reproducible analysis (Chapter 3's `targets` or a plain `source()` chain), the validation step should sit between "load raw extract" and "compute dominance/ICERs," and the pipeline should fail loudly — not produce a frontier plot — when a rule is violated. For HTA submissions, exporting the `pointblank` or `validate` report alongside the analysis code gives reviewers direct evidence that implausible values (negative costs, utilities > 1, impossible QALY decrements) were checked for and excluded or corrected, not missed.

Common pitfall. A validation rule that is too loose is worse than no rule at all, because it creates false confidence. `qaly <= 1` catches an impossible health state but does not catch a QALY of exactly 1.000000 held constant across 40 patient-years of follow-up, which is numerically valid but clinically absurd. Pair range checks with distributional sanity checks (e.g. `col_vals_between()` tightened to a clinically plausible range, or a quick `summary()`/histogram pass) rather than relying on hard mathematical bounds alone.

5.7 Data governance, privacy and de-identification

Validating values is necessary but not sufficient. Before a TB programme dataset, a claims extract, or a trial case report form ever reaches an R script, it has to be handled in a way that respects the people behind the rows — and, for anyone managing patient-level data professionally, that is a legal obligation as much as an ethical one. The vocabulary matters here, because "anonymous" is used loosely in casual conversation but has a precise technical meaning in every major privacy framework:

- **De-identification / anonymization** removes or transforms identifying information such that the remaining data cannot reasonably be linked back to an individual, by anyone, using any reasonably available means. Properly anonymized data generally falls outside the scope of data-protection law entirely — but "properly" is a high bar, especially for small denominators (a district with three TB patients of a rare drug-resistance pattern is not anonymous just because the name column was dropped).
- **Pseudonymization** replaces direct identifiers (name, national ID, exact address) with a code or token, but keeps a way — often a separate, access-controlled key table — to re-identify the record if needed. Pseudonymized data is still personal data under most frameworks, because re-identification remains possible; it reduces risk, but it is not the same as anonymization.

In the US, HIPAA sets out two recognised routes to de-identification. **Safe Harbor** specifies an enumerated list of 18 identifier categories (names, geographic subdivisions smaller than a state, all elements of dates except year, phone/fax numbers, medical record numbers, and so on) that must be removed or generalised; it is mechanical and auditable, but blunt. **Expert Determination** instead has a qualified statistician or scientist formally assess and document that the re-identification risk is "very small," using accepted statistical methods — appropriate when Safe Harbor's blunt removals would destroy analytic value (e.g. needing exact dates to compute time-to-event survival curves). Neither route is specific to US data only in spirit — the underlying logic (enumerate-and-remove versus risk-assess-and-document) is a useful mental model anywhere.

The EU's GDPR applies to any personal data concerning an identified or identifiable natural person, with special-category status (and extra safeguards) for health data specifically. For trial or RWE data originating from or concerning EU subjects, GDPR requires a documented lawful basis for processing, data minimisation, and — critically for anyone sharing analysis code or data extracts across borders — restrictions on transferring personal data outside the EU/EEA without an adequate safeguard. Pseudonymization is explicitly recognised in GDPR as a risk-reducing **technical measure**, not an exemption from the regulation.

India's Digital Personal Data Protection Act (DPDPA), 2023, is the relevant framework for anyone working with government health programme data in India — including TB programme (Ni-kshay and related) patient-level records. It establishes general principles that will shape how such data must be governed going forward: processing personal data only for a specified, lawful purpose with the data principal's consent or another permitted legal basis, collecting no more data than necessary for that purpose, implementing reasonable security safeguards against breach, and providing mechanisms for grievance redress. For a government health data role handling patient-level records, the practical implication is that **data minimisation and purpose limitation are not optional best practices — they are converging with statutory obligation**.

Agency perspective. ICH E6(R2) Good Clinical Practice guidance, in its sections on data handling and essential documents, requires that trial data be attributable, confidential, and protected from unauthorised access throughout the trial and in archived records — confidentiality of subject data is treated as a core GCP obligation, not an afterthought bolted on at publication. Any R-based trial or RWE analysis workflow that touches patient-level data should be able to show, concretely, how access to the raw identifiable extract was restricted and how only de-identified or pseudonymized data reached the analysis environment — this is the kind of evidence a GCP audit or an HTA body's data governance review will actually ask to see.

On the practical, R-side side of this: the single most common and most avoidable governance failure is a patient-level `.csv` ending up in a public (or even a private-but-over-shared) GitHub repository, because Chapter 2's `git add .` habit does not pause to ask what is in the file. Raw extracts with names, exact dates of birth, or facility-linked IDs small enough to re-identify someone must never be committed to version control, full stop — not even to a repo you believe is private, because history persists, forks happen, and access lists change. The workable pattern is the one this book itself uses throughout: write and share R code against small, clearly-labelled **synthetic or simulated** datasets (as in Chapter 4's cost-QALY tibbles), keep any real patient-level extract in a `.gitignore`'d local or access-controlled location, and if a working example genuinely needs to resemble real data's statistical structure, generate it synthetically (resampling distributions, adding noise, or using a package such as `synthpop`) rather than lightly-perturbing real records and hoping that counts as de-identified.

Common pitfall. Dropping the name and ID columns from a spreadsheet is not de-identification. A TB programme extract with facility name, exact diagnosis date, drug-resistance pattern, and sub-district can re-identify a specific patient through the combination of fields even with no name column at all — this is the well-documented "quasi-identifier" problem. Before calling any dataset de-identified, check cell sizes for rare combinations (small-count suppression), generalise or bin dates and geography where exact precision isn't analytically required, and document which identifiers were removed versus generalised versus retained under restricted access.

With the data pipeline now covered end to end — sourcing from clinical, claims, registry, and public-health systems; wrangling and reshaping it reproducibly; and validating and governing it before it ever touches an economic model — the next step in building an evidence base is working backwards from data to the published literature that frames and justifies it. Chapter 6 turns to R's growing toolkit for systematic literature review: searching and deduplicating records across databases, managing screening workflows, and extracting study-level data in a way that is as reproducible and auditable as the data management practices just established here.

Chapter 6. Systematic Literature Review (SLR) Support in R

Learning objectives

- Document a search strategy reproducibly, including the exact Boolean strings run in each database, per PRISMA-S expectations.
- Deduplicate multi-database reference exports and run a documented, agreement-checked two-stage screening workflow.
- Build a PRISMA 2020 flow diagram in R directly from the counts produced by the screening pipeline, not typed by hand.
- Design a structured data extraction template and a programmatic dual-extraction reconciliation workflow.
- Produce Cochrane RoB 2.0 / ROBINS-I traffic-light and summary plots with `robvis`.
- Use text mining to prioritise abstract screening order without weakening Cochrane's dual-independent-screening requirement.

Chapter 5 closed by flagging publication bias as a threat you mostly **detect** statistically, after the evidence is already assembled — funnel plots, Egger's test, trial-registry cross-checks. This chapter is about the earlier stage: building the evidence pool itself in a way that is transparent, reproducible, and auditable from the first search string to the final included-study count. Every one of the workflows below follows the same discipline already familiar from this book's R habits — log everything to a data frame, version it, and let the numbers that end up in a report trace back to a line of code rather than a remembered tally.

6.1 Search strategy documentation

A systematic review lives or dies on whether its search is reproducible. Chapter 5 flagged publication bias as a threat you detect statistically (funnel plots, Egger's test) only **after** the evidence is in hand; the search strategy is where you try to prevent that bias from ever entering the pool, by casting a wide, documented net across databases rather than relying on whatever your usual PubMed habits surface. PRISMA-S, the search-reporting extension to PRISMA 2020, asks for exactly this: database name, platform/interface, full Boolean string, date range, limits applied, and date run, for every source searched — not just the final count you remember.

There's no reason to keep this in a Word table that drifts out of sync with what you actually ran. A tibble, version-controlled alongside your analysis code, does the job better and survives an audit.

```
library(tibble)
library(readr)
library(dplyr)

search_log <- tibble(
  database     = c("PubMed", "Embase", "Cochrane CENTRAL"),
  platform     = c("PubMed.gov", "Ovid", "Cochrane Library"),
  search_string = c(
```

```
    '("tuberculosis"[MeSH] OR "TB"[tiab]) AND ("economic evaluation"[tiab] OR
"cost-effectiveness"[tiab])',
    'exp Tuberculosis/ AND (economic evaluation/ OR "cost effectiveness".ti,ab.)',
    'tuberculosis AND (cost-effectiveness OR "economic evaluation"):ti,ab,kw'
  ),
  date_from    = as.Date(c("2010-01-01", "2010-01-01", "2010-01-01")),
  date_to      = as.Date(rep("2024-06-30", 3)),
  date_run     = as.Date(rep(Sys.Date(), 3)),
  n_hits       = c(1842, 1365, 211),
  limits       = c("English; Humans", "English; Humans", "English")
)

write_csv(search_log, "search_strategy_log.csv")
search_log
```

Keep the raw string exactly as pasted from the database interface — don't "clean it up" for readability. If a reviewer needs to re-run the search in two years, the string in your CSV has to be the string that actually produced those 1,842 hits, syntax quirks and all.

How to report this.

Database	Platform	Limits applied	N hits
PubMed	PubMed.gov	English; Humans	1,842
Embase	Ovid	English; Humans	1,365
Cochrane CENTRAL	Cochrane Library	English	211
Date range 2010-01-01 to 2024-06-30; all searches run and logged the same day (`search_strategy_log.csv`)			

Table 13: Search strategy summary by database, as reported in an SLR methods section or PRISMA-S search appendix

Before you ever get to that final string, you usually need help deciding which terms belong in it. This is where `litsearchr` earns its place in an R-based SLR workflow: it takes a seed set of known-relevant papers (or a naive search's results), extracts candidate keyphrases, and builds a keyword co-occurrence network so you can spot synonyms and related terms you'd otherwise miss — "DR-TB" sitting next to "multidrug-resistant tuberculosis" sitting next to "rifampicin-resistant," for instance.

```
library(litsearchr)

# naive_results is a data frame with title/abstract/keywords columns
# pulled from an initial scoping search
rakedkeywords <- extract_terms(
  text = paste(naive_results$title, naive_results$abstract),
  method = "fakerake",
  min_freq = 3,
  ngrams = TRUE,
  min_n = 1,
  max_n = 3
```

```
)

docs <- paste(naive_results$title, naive_results$abstract)
dfm  <- create_dfm(elements = docs, features = rakedkeywords)
g    <- create_network(search_dfm = dfm, min_studies = 3)

cutoff <- find_cutoff(g, method = "cumulative", percent = 0.80)
reduced_graph <- reduce_graph(g, cutoff_strength = cutoff[1])
search_terms  <- get_keywords(reduced_graph)
search_terms
```

The output isn't a search string you paste in blind — it's a candidate term list you screen against clinical judgment before it enters the Boolean query that then goes into `search_log` above.

HEOR in practice. In HTA submissions, the search strategy appendix is frequently where assessors' first critical comments land — not the clinical evidence itself. A dated, versioned `search_strategy_log.csv` sitting in your project repo, committed alongside the screening scripts, means you can answer "did you search Embase after your PubMed search, or the same week?" without digging through email threads or a collaborator's memory.

Common pitfall. Translating a search string across database syntaxes (PubMed's `[MeSH]` vs. Embase's `exp .../` vs. Cochrane's `:kw`) by eye is a common source of silently different hit counts. Log the **translated** string actually run in each interface, not one master string you assume is "equivalent" everywhere — and sanity-check hit counts against expectations (Embase typically returns more records than PubMed for the same concept; a near-identical count across both is worth a second look).

6.2 Deduplication and screening workflows

Running the same concept across three databases guarantees duplicate records — often 20–40% of a combined pull. Deduplicating by exact title match alone will miss records that differ by punctuation, case, or a trailing journal abbreviation, so you need fuzzy matching on title/author/year, and ideally DOI matching as a first, cheap pass before falling back to string similarity.

`synthesisr` standardizes bibliographic imports (RIS/Bibtex/CSV exports from different databases rarely use identical field names) and performs similarity-based deduplication; `revtools` builds on the same ecosystem and adds an interactive screening interface. `ASySD` (Automated Systematic Search Deduplicator) is purpose-built for the messier reality of SLR exports and handles multi-stage matching (DOI exact match, then title/author fuzzy match) more robustly than a single string-distance cutoff.

```
library(synthesisr)
library(dplyr)

# read_refs() handles RIS exports from PubMed, Embase, and Cochrane
pubmed_refs  <- read_refs("pubmed_export.ris")
embase_refs  <- read_refs("embase_export.ris")
cochrane_refs <- read_refs("cochrane_export.ris")
```

```
all_refs <- merge_columns(pubmed_refs, embase_refs, cochrane_refs) |>
  as_tibble()

# first pass: exact match on standardized title string
dup_exact <- find_duplicates(
  all_refs$title,
  method = "exact",
  to_lower = TRUE,
  rm_punctuation = TRUE
)

# second pass: fuzzy match on anything exact matching missed
dup_fuzzy <- find_duplicates(
  all_refs$title,
  method = "string_osa",
  to_lower = TRUE,
  rm_punctuation = TRUE,
  threshold = 7
)

all_refs_dedup <- all_refs |>
  mutate(dup_id = dup_fuzzy) |>
  extract_unique_references(dup_id)

nrow(all_refs) - nrow(all_refs_dedup)   # duplicates removed, for the PRISMA count
```

Once you have a deduplicated set, screening is a two-stage filter: title/abstract first (fast, generous, errs toward inclusion), then full-text on the survivors (slower, decisive, with documented exclusion reasons). The habit that keeps this defensible is logging every reviewer decision at the record level, not just the final yes/no — and if you have two independent screeners, computing agreement rather than assuming it.

```
library(dplyr)
library(irr)   # kappa2()

screening_log <- all_refs_dedup |>
  mutate(
    reviewer1_ti_ab = sample(c("include", "exclude"), n(), replace = TRUE, prob =
c(0.3, 0.7)),
    reviewer2_ti_ab = sample(c("include", "exclude"), n(), replace = TRUE, prob =
c(0.3, 0.7))
  )

kappa_result <- kappa2(screening_log[, c("reviewer1_ti_ab", "reviewer2_ti_ab")])
kappa_result$value   # Cohen's kappa for title/abstract agreement

conflicts <- screening_log |>
  filter(reviewer1_ti_ab != reviewer2_ti_ab)
nrow(conflicts)   # records requiring reconciliation discussion
```

A kappa below roughly 0.6 is usually a sign the inclusion criteria need tightening before you burn reviewer time on full-text retrieval, not a problem to paper over with a third tie-breaking reviewer alone.

Difficult terminology: Cohen's kappa. Cohen's kappa measures how much two reviewers agree with each other **beyond** what you would expect them to agree on by chance alone. A naive "percent agreement" is misleading here: if 90% of records are obvious excludes, two reviewers who each decide mostly at random (or even by flipping a coin on the hard ones) will still agree with each other roughly 80-90% of the time just by both landing on "exclude" most of the time. Kappa corrects for that baseline chance-agreement rate, so a kappa of 0.71 is a genuinely informative number, not just "71% of decisions matched." The rule-of-thumb bands you will see cited are: below 0.4 poor, 0.4-0.6 moderate, 0.6-0.8 substantial, above 0.8 near-perfect agreement — so 0.71 here is solid but not pristine, which is exactly why 115 conflicts still needed a reconciliation discussion. Think of it the way you'd think of a diagnostic test's performance over the baseline prevalence rate: two clinicians who both tend to diagnose a common condition will agree often just because the condition is common, not because they're applying identical criteria — kappa is the statistic that strips that artefact out.

How to report this.

Metric	Value
Records screened (title/abstract)	3,198
Reviewer 1 — included	960
Reviewer 2 — included	955
Concordant decisions	3,083
Agreement (% records)	96.4%
Cohen's kappa	0.71
Conflicts requiring reconciliation	115

Table 14: Dual-reviewer title/abstract screening agreement, as reported in the SLR methods section

Agency perspective. The Cochrane Handbook for Systematic Reviews of Interventions (Chapter 4, "Searching for and selecting studies") specifies that screening should be done independently by at least two reviewers, with disagreements resolved by discussion or a third reviewer, and that the process — including inter-rater agreement where feasible — should be reported with enough detail for replication. A logged `screening_log` with per-reviewer decision columns and a computed kappa is a direct, auditable response to that requirement, not an optional nicety.

Common pitfall. `find_duplicates()` with a string-distance method is sensitive to the threshold you choose — too loose and you merge genuinely distinct trials with similar titles (common in TB drug trials with similar naming conventions); too tight and you leave duplicates in, inflating your "identified" count and understating how much actual unique evidence you screened. Always spot-check a sample of flagged duplicate pairs by eye before accepting

`extract_unique_references()`'s output as final, and record the method and threshold used in your methods write-up.

6.3 PRISMA flow diagrams in R

Every number that feeds the PRISMA flow diagram should trace back to a line of code you already ran — the row count before `extract_unique_references()`, the duplicate count it removed, the title/abstract exclusions from `screening_log`, the full-text exclusions with reasons. Building the diagram from a hand-typed set of numbers in PowerPoint reintroduces exactly the kind of untraceable discrepancy a reproducible SLR workflow is meant to eliminate.

The `PRISMA2020` package takes a named vector or data frame of stage counts (matching the official PRISMA 2020 flow diagram template) and renders the diagram as an SVG/interactive plot you can embed or export.

```
library(PRISMA2020)
library(dplyr)

prisma_counts <- data.frame(
  stage = c(
    "database_results", "duplicates_removed",
    "records_screened", "records_excluded",
    "full_text_retrieved", "full_text_excluded",
    "studies_included"
  ),
  n = c(
    nrow(all_refs),                     # 3,418 identified
    nrow(all_refs) - nrow(all_refs_dedup),  # duplicates removed
    nrow(all_refs_dedup),               # screened at title/abstract
    sum(screening_log$reviewer1_ti_ab == "exclude" &
          screening_log$reviewer2_ti_ab == "exclude"),
    42,                                  # sent to full-text review
    19,                                  # excluded at full-text, reasons below
    23                                   # final included studies
  )
)

# PRISMA_data() expects its own template row-name structure (stage
# names in column 1, counts in column "n") -- PRISMA2020_data(data) was
# a typo for the real exported function PRISMA_data(data)
data <- PRISMA_data(prisma_counts)
plot <- PRISMA_flowdiagram(
  data,
  interactive = FALSE,
  previous = FALSE,
  other = FALSE,
  fontsize = 12,
  title_colour = "steelblue"
)

# PRISMA2020 has no PRISMA_save(); save a ggplot/DiagrammeR-style object
```

```
# with DiagrammeRsvg + rsvg, the package's own documented export route
svg_code <- DiagrammeRsvg::export_svg(plot)
rsvg::rsvg_svg(charToRaw(svg_code), file = "prisma_flow.svg")
```

Full-text exclusions need reasons, not just a count, because PRISMA 2020 Item 7 and the diagram's exclusion box both expect a breakdown (wrong population, wrong comparator, wrong outcome, conference abstract only, etc.) rather than a single undifferentiated number.

```
exclusion_reasons <- tibble::tibble(
  reason = c("Wrong population", "No cost outcome",
             "Conference abstract only", "Duplicate publication"),
  n = c(7, 6, 4, 2)
)
sum(exclusion_reasons$n)   # must equal full_text_excluded above -- reconcile if
not
```

That reconciliation check — the sum of itemized exclusion reasons equalling the single number on the flow diagram — is worth asserting in code (`stopifnot()` or a simple test) rather than eyeballing, since it's exactly the kind of off-by-one error that surfaces in peer review.

How to report this.

Stage	N records
Records identified across databases	3,418
Duplicates removed	220
Records screened (title/abstract)	3,198
Records excluded at title/abstract	2,238
Full-text reports retrieved	42
Full-text reports excluded	19
Studies included in review	23

Table 15: Screening funnel stage counts underlying the PRISMA 2020 flow diagram, distinct from the itemized full-text exclusion reasons below

How to report this.

Exclusion reason	n
Wrong population	7
No cost outcome	6
Conference abstract only	4
Duplicate publication	2
Total excluded at full-text = 19	

Table 16: Full-text exclusion reasons, reported alongside the PRISMA 2020 flow diagram (Item 7)

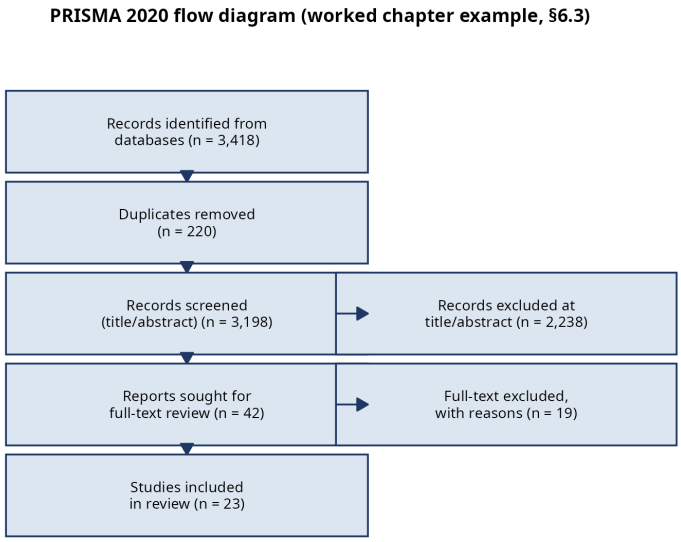


Figure 6: A PRISMA 2020 flow diagram built directly from the stage counts computed in code (§6.3), rather than typed by hand — the reconciliation discipline this section argues for: every number traces back to a line of code, and the itemized exclusion reasons sum to the single full-text-excluded count shown.

HEOR in practice. Submission dossiers to HTA bodies increasingly expect the PRISMA diagram and the underlying counts to be reconcilable on request. Keeping `prisma_counts` as a data frame generated from the same pipeline that produced `search_strategy_log.csv` and `screening_log` means a reviewer's question — "where did the other 19 full-text articles go?" — has a one-line answer: re-run the script and read off `exclusion_reasons`.

Agency perspective. The PRISMA 2020 statement (Page et al., **BMJ** 2021) explicitly requires, in Item 16a, a flow diagram showing the number of records identified, duplicates removed, records screened and excluded, reports sought and not retrieved, reports assessed for eligibil-

ity, and studies included — with reasons given for full-text exclusions. `PRISMA_flowdiagram()` maps directly onto this template field-for-field; using it rather than a freehand diagram is the most mechanical way to guarantee compliance with that specific checklist item.

6.4 Data extraction templates and extraction QA

Once you have a final included-studies set from screening (§6.2-6.3), the next failure point in an SLR is extraction: the manual transcription of population, intervention, comparator, outcomes (PICO), effect sizes, and risk-of-bias inputs from each paper into a structured dataset. For a TB-programme analyst this will feel familiar — it's the same discipline as designing a clean case-report form before you let field staff touch it. A tibble-based extraction template, version-controlled from day one, turns this into reproducible data engineering rather than a spreadsheet someone can silently edit.

Design the template as one row per study-outcome combination (long format), not one row per study, because most SLRs extract multiple outcomes and multiple comparisons per study, and a wide format forces awkward column explosion later.

```
library(tibble)
library(dplyr)

extraction_template <- tibble(
  study_id        = character(),  # e.g. "Nguyen2021"
  reviewer_id     = character(),  # who extracted this row
  population      = character(),
  intervention    = character(),
  comparator      = character(),
  outcome_name    = character(),  # e.g. "treatment_success"
  outcome_timepoint = character(),
  effect_measure  = character(),  # "RR", "OR", "HR", "MD"
  effect_estimate = double(),
  ci_lower        = double(),
  ci_upper        = double(),
  n_intervention  = integer(),
  n_comparator    = integer(),
  rob_domain_1    = character(), # e.g. randomisation process
  rob_domain_2    = character(),
  rob_overall     = character(), # "Low", "Some concerns", "High"
  notes           = character()
)

# one extracted row, reviewer A
extraction_template <- extraction_template %>%
  add_row(
    study_id = "Nguyen2021", reviewer_id = "RA",
    population = "Adults, drug-sensitive PTB",
    intervention = "4-month regimen", comparator = "6-month regimen",
    outcome_name = "treatment_success", outcome_timepoint = "end_of_treatment",
    effect_measure = "RR", effect_estimate = 0.98,
    ci_lower = 0.92, ci_upper = 1.04,
    n_intervention = 310, n_comparator = 308,
    rob_domain_1 = "Low", rob_domain_2 = "Some concerns",
```

```
    rob_overall = "Some concerns", notes = "ITT analysis used"
  )
```

Enforcing a schema up front (explicit column types, controlled vocabularies for `effect_measure` and `rob_overall`) means extraction errors — a reviewer typing "Odds ratio" where another typed "OR" — surface immediately as validation failures rather than silently corrupting a later meta-analysis.

Dual, independent extraction is the standard safeguard against transcription error, mirroring dual independent screening. Have two reviewers populate identical-schema tibbles, then reconcile programmatically rather than by eye:

```
library(waldo)

reviewer_a <- readr::read_csv("extraction_reviewerA.csv")
reviewer_b <- readr::read_csv("extraction_reviewerB.csv")

# align rows on the natural key before comparing
reviewer_a <- reviewer_a %>% arrange(study_id, outcome_name)
reviewer_b <- reviewer_b %>% arrange(study_id, outcome_name)

discrepancies <- waldo::compare(
  reviewer_a, reviewer_b,
  tolerance = 1e-6              # ignore float noise, e.g. 0.980 vs 0.98
)
discrepancies
```

`waldo::compare()` reports exactly which cells differ between the two tibbles — far more actionable for an adjudication meeting than a manual side-by-side read of two spreadsheets. For numeric fields where small rounding differences are expected (confidence intervals read off a forest plot, for instance), a `dplyr`-based tolerance check is often clearer than relying on `waldo`'s defaults:

> **Difficult terminology: forest plot.** A forest plot is the standard way to show a set of effect estimates — one per study — stacked one above the other so they can be compared at a glance. Each row is one study: a square or dot marks that study's point estimate (e.g. a risk ratio or odds ratio), and a horizontal line through it spans its 95% confidence interval, so wider lines mean less-precise studies. The rows are typically ordered by publication date or study size, and when the studies are later statistically pooled (Chapter 11 covers this in full), a diamond shape at the bottom shows the combined estimate, with the diamond's width showing the pooled interval's precision. Visually, it's the same idea as lining up several lab-test results for a patient on one chart with their error bars, so you can see at a glance which results are consistent with each other and which one is the outlier. You will not build a forest plot in this chapter — that machinery lives in Chapter 11 — but the term shows up here already because reconciling reviewer-extracted effect estimates (as in the `waldo::compare()` step above) is precisely the data-cleaning work that feeds into one later.

```
joined <- inner_join(
  reviewer_a, reviewer_b,
  by = c("study_id", "outcome_name"),
  suffix = c("_A", "_B")
)
```

```
flagged <- joined %>%
  mutate(
    estimate_diff = abs(effect_estimate_A - effect_estimate_B),
    rob_mismatch  = rob_overall_A != rob_overall_B
  ) %>%
  filter(estimate_diff > 0.01 | rob_mismatch)

flagged %>% select(study_id, outcome_name, effect_estimate_A, effect_estimate_B,
rob_overall_A, rob_overall_B)
```

Route every row in `flagged` to a third-reviewer adjudication step before it enters the master extraction dataset, and log the adjudicated value with a `reviewer_id = "adjudicated"` row so the audit trail is preserved rather than overwritten.

HEOR in practice. Treat the extraction dataset the way you would treat programme surveillance data destined for a drug-resistance report: commit the CSV or `.rds` files to version control (Git) at each named milestone — "post first-pass extraction", "post reconciliation", "post adjudication" — rather than editing one live file in place. A `git diff` on a CSV extraction file is a legitimate, line-by-line audit trail of who changed what value and when, which is exactly the kind of provenance a submission dossier's SLR appendix may be interrogated on. Avoid storing extraction data only in a shared Excel/Google Sheet with no version history; if the sheet is the master, export it to CSV and commit the export after every extraction session.

Common pitfall. Resist the temptation to resolve discrepancies by simply averaging or eyeballing "which one looks right." A documented discrepancy with no recorded resolution rationale is itself a methodological weakness reviewers will flag. Always log **why** a value was adjudicated the way it was (e.g. "reviewer A misread Table 3 column header — comparator and intervention columns were swapped") alongside the final value.

6.5 Risk of bias visualisation (`robvis`)

Risk-of-bias assessment itself — applying Cochrane RoB 2.0 to RCTs or ROBINS-I to non-randomized studies — is a judgment exercise done by reviewers against the official signalling-question algorithms, not something R automates. What R (via the `robvis` package) does well is turn a completed set of domain-level judgments into the standard traffic-light and weighted-bar-chart figures that PRISMA-compliant reviews and HTA submissions are expected to include.

`robvis` expects a data frame with one row per study, a column per RoB domain plus an overall judgment column, and values restricted to the tool's controlled vocabulary (`"Low"`, `"Some concerns"`/`"Moderate"`, `"High"`/`"Serious"`, `"Critical"`, `"No information"` depending on the tool). Build this from your reconciled extraction tibble:

```
library(robvis)
library(dplyr)

# RoB 2.0 for RCTs: one row per study, domains D1-D5, overall, then a
# final Weight column -- robvis expects the weight as the LAST column
rob2_data <- tibble::tibble(
  Study = c("Nguyen2021", "Alvarez2020", "Okafor2022"),
```

```
  D1 = c("Low", "Some concerns", "Low"),
  D2 = c("Low", "Low", "Some concerns"),
  D3 = c("Some concerns", "Low", "Low"),
  D4 = c("Low", "Low", "Low"),
  D5 = c("Low", "Some concerns", "Low"),
  Overall = c("Some concerns", "Some concerns", "Low"),
  Weight = c(33.3, 33.3, 33.3)
)

rob_traffic_light(
  data = rob2_data,
  tool = "ROB2",
  psize = 8
)

rob_summary(
  data = rob2_data,
  tool = "ROB2",
  overall = TRUE
)
```

`rob_traffic_light()` produces the familiar red/amber/green grid — one row per study, one column per domain — that goes directly into an SLR appendix or a methods annex of an HTA dossier. `rob_summary()` produces the weighted horizontal bar chart showing the proportion of studies at each judgment level per domain, which is the figure typically used in the main body of the review to summarise bias across the whole evidence base at a glance.

For non-randomized studies assessed with ROBINS-I, the domain structure and vocabulary differ, so specify `tool = "ROBINS-I"` and supply the correct domain columns:

```
robins_data <- tibble::tibble(
  Study = c("Mbeki2019", "Santos2021"),
  D1 = c("Low", "Moderate"),
  D2 = c("Moderate", "Low"),
  D3 = c("Low", "Low"),
  D4 = c("Serious", "Moderate"),
  D5 = c("Low", "Low"),
  D6 = c("Low", "Low"),
  D7 = c("Moderate", "Low"),
  Overall = c("Serious", "Moderate"),
  Weight = c(50, 50)
)

rob_traffic_light(data = robins_data, tool = "ROBINS-I")
rob_summary(data = robins_data, tool = "ROBINS-I", overall = TRUE)
```

Both functions return `ggplot2` objects, so they compose with the rest of your usual plotting pipeline — save with `ggsave()`, adjust fonts and sizing for print dossiers, or patch into a combined figure with `patchwork` alongside forest plots from a later meta-analysis (Chapter 11).

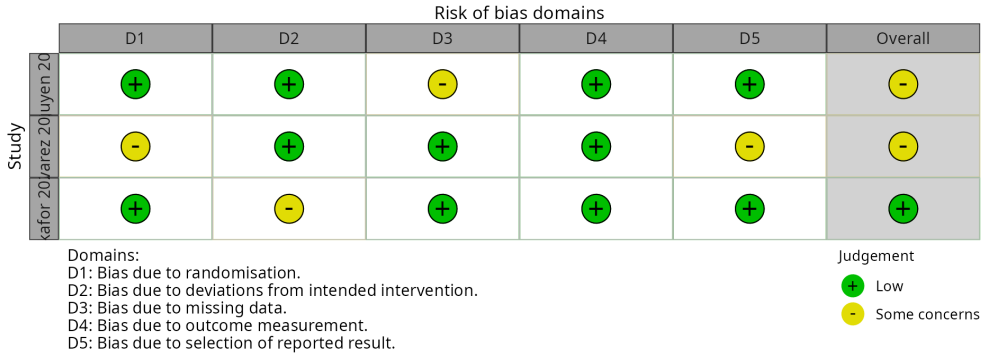


Figure 7: `robvis::rob_traffic_light()` applied to the chapter's worked RoB 2.0 example (`rob2_data`): a red/amber/green grid, one row per study, one column per domain, in the standard format expected in an SLR appendix or HTA methods annex.

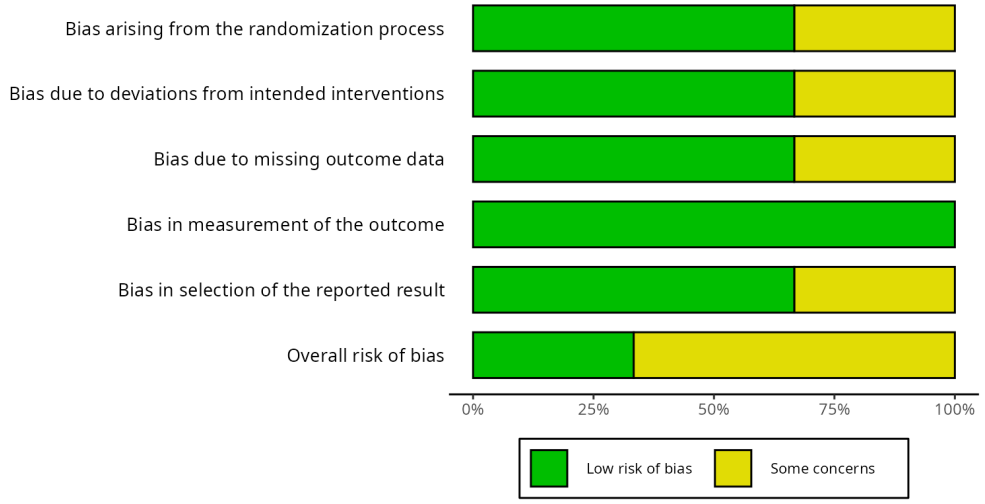


Figure 8: `robvis::rob_summary()` on the same data: the weighted horizontal bar chart summarising the proportion of studies at each judgment level per domain, the figure typically placed in the main body of a review rather than its appendix.

Agency perspective. Cochrane Handbook Chapter 8 (RoB 2.0) and Chapter 25 (ROBINS-I) specify the signalling questions and domain structure that `robvis`'s input vocabulary is built around — do not substitute an informal judgment scale that doesn't map onto those domains, because a reviewer or HTA assessor checking your figure against the Handbook's defined categories will immediately notice a mismatch. PRISMA 2020 item 18 explicitly requires reporting "methods used to assess risk of bias in the included studies," and a traffic-light plot generated this way is standard, recognisable evidence of having done so transparently rather than narratively.

Common pitfall. `robvis` will silently plot whatever judgment strings you give it, including typos or inconsistent capitalisation (`"low"` vs `"Low"`), as a separate category rather than erroring. Always run `unique()` across your domain columns before plotting to catch stray

values that would otherwise produce an extra, meaningless colour band in the traffic light or summary bar.

6.6 Text mining and ML-assisted screening

By the time a search strategy returns several thousand abstracts, manually reading titles in import order is a poor use of two reviewers' time. Text mining can reorder the screening queue so the most plausibly relevant abstracts are screened first, which speeds up time-to-inclusion-decision without changing **what** counts as included — screening decisions themselves remain exclusively human, exactly as in every other stage of this chapter.

A simple term-frequency pass with `tidytext` is often enough to sanity-check that your search strategy actually captured the expected vocabulary before you even start full screening:

```
library(tidytext)
library(dplyr)

abstracts <- tibble::tibble(
  study_id = c("S1", "S2", "S3"),
  abstract = c(
    "Randomized trial of four-month rifapentine-based regimen for pulmonary TB.",
    "Cohort study assessing cardiovascular risk in diabetic patients.",
    "Treatment success and relapse rates in short-course TB chemotherapy."
  )
)

tidy_abstracts <- abstracts %>%
  unnest_tokens(word, abstract) %>%
  anti_join(stop_words, by = "word")

term_freq <- tidy_abstracts %>%
  count(word, sort = TRUE)

term_freq %>% head(10)
```

For a more structured relevance signal across a few thousand abstracts, `quanteda` builds a document-feature matrix and lets you score each abstract against a set of PICO-anchored terms drawn from your protocol's search strategy:

```
library(quanteda)

corp <- corpus(abstracts, text_field = "abstract", docid_field = "study_id")
dfm_abs <- corp %>%
  tokens(remove_punct = TRUE) %>%
  tokens_remove(stopwords("en")) %>%
  dfm()

# relevance proxy: count of protocol-defined key terms per abstract
key_terms <- c("tuberculosis", "rifapentine", "regimen", "relapse", "treatment")
relevance_score <- dfm_abs %>%
  dfm_select(pattern = key_terms) %>%
  rowSums()
```

```r
screening_order <- tibble::tibble(
  study_id = docnames(dfm_abs),
  relevance_score = relevance_score
) %>%
  arrange(desc(relevance_score))

screening_order
```

Feeding `screening_order` to your reviewers as the order in which to work through the abstract list — highest-scoring first — gets likely-includes screened early, which matters operationally when a review has a hard deadline and screening is still incomplete when it arrives.

The `revtools` package goes further, offering topic modelling (via `screen_topics()` / `screen_abstracts()`) specifically built around the SLR screening workflow: it clusters abstracts into latent topics and gives reviewers an interactive interface to mark whole topic clusters as likely relevant or irrelevant, which is useful triage for very large (10,000+) abstract sets before line-by-line screening starts. Purpose-built external screening platforms — Rayyan and Abstrackr are the most commonly cited in SLR methods sections — offer similar machine-learning-assisted prioritisation (active-learning ranking as reviewers screen) and integrate directly with reference managers; R's role here is typically pre- or post-processing (exporting a prioritised order into Rayyan, or pulling Rayyan's decision export back into your extraction pipeline from §6.4) rather than replacing those platforms outright.

> **Common pitfall.** Term-frequency scores, topic clusters, and active-learning rankings are prioritisation aids, not screening decisions, and none of them replace Cochrane's requirement (Cochrane Handbook Chapter 4) for dual independent screening of every title/abstract and full text by two human reviewers with disagreements resolved by discussion or a third reviewer. Using an ML relevance score to **exclude** abstracts outright, or to let a single reviewer screen only the "high-relevance" tier while skipping the rest, breaks that standard and will be flagged by anyone checking your PRISMA flow diagram against the methods text. The only legitimate use of these tools in a Cochrane-conformant review is re-ordering the queue that both independent human reviewers still work through in full.

Chapter 7. Missing Data

Learning objectives

- Distinguish MCAR, MAR, and MNAR missing-data mechanisms and explain why the distinction cannot be confirmed from observed data alone.
- Use `naniar` and `visdat` to visualise and audit missingness patterns before choosing a handling strategy.
- Explain why complete-case analysis and single imputation understate uncertainty, and when each is defensible.
- Perform multiple imputation with `mice` (and know when to reach for `Amelia` or `missForest` instead), pooling estimates correctly via Rubin's rules.
- Conduct tipping-point and pattern-mixture sensitivity analyses to stress-test conclusions against departures from MAR.
- Handle the distinctive challenges of missing cost and QALY data — skewness, bounds, and informative dropout.
- Report missing-data handling for an HTA submission in line with CHEERS 2022 and ICH E9(R1) expectations.

Chapter 6 closed by building a reproducible evidence pool from the published literature; this chapter turns to a problem that shows up just as often in the primary trial, registry, or claims dataset itself — not every cell in that dataset is filled in, and what you do about the gaps can change a cost-effectiveness conclusion as much as any modelling choice later in this book.

7.1 Mechanisms: MCAR, MAR, MNAR

Every HEOR analysis built on trial or real-world data eventually runs into the same quiet problem: some values are simply not there. A patient skips a follow-up visit, a claims extract has a null cost field for a quarter, an EQ-5D questionnaire comes back with three dimensions ticked and two left blank. Before you can decide what to do about it — delete the rows, impute them, or model around them — you need a working theory of **why** the data are missing. Donald Rubin's 1976 framework gives the vocabulary for that theory, and it remains the backbone of every modern missing-data guideline used in health technology assessment.

Rubin defined three mechanisms by asking whether the probability that a value is missing depends on the data itself:

- **MCAR (Missing Completely At Random)**: the probability of a value being missing is unrelated to both the observed and the unobserved data. Missingness is pure noise — equivalent to a coin flip applied independently to every record.
- **MAR (Missing At Random)**: the probability of a value being missing depends only on the **observed** data (other variables you have recorded), not on the missing value itself, once you condition on what you observed.
- **MNAR (Missing Not At Random)**: the probability of a value being missing depends on the **unobserved** value itself, even after conditioning on everything you did observe.

Take a concrete HEOR example and carry it through all three mechanisms: a randomised trial collects EQ-5D-5L questionnaires at 12 weeks to feed a QALY calculation for the economic model, and 15% of participants have a missing follow-up questionnaire.

- If non-response is driven by a random administrative failure — a clinic forgot to mail the form to a random subset of participants regardless of their health status, treatment arm, age, or anything else — that is MCAR. The missing 15% are, in expectation, a random sample of the full cohort, and a complete-case analysis (simply dropping those rows) is unbiased, if inefficient.
- If non-response is higher among patients who missed their clinic visit, and clinic attendance is itself associated with baseline disease severity that you **recorded** at randomisation, then conditioning on baseline severity (which you have) explains the missingness pattern. That is MAR: missingness depends on an observed covariate, not on the unobserved EQ-5D score itself once severity is accounted for.
- If, instead, patients skip the questionnaire specifically **because** their health has deteriorated and they no longer feel well enough to complete it — and that deterioration is not fully captured by anything else in your dataset — then missingness depends on the very value that is missing. That is MNAR, and no amount of conditioning on the data you do have will fix it, because the unobserved health state itself is the driver.

The uncomfortable truth is that these three scenarios can produce an **identical** pattern of missing cells in your dataset. Nothing in the observed rows and columns tells you which story is true — MCAR, MAR, and MNAR are all consistent with the same table of 1s and 0s marking which values are missing. The mechanism is a statement about the **unobserved** values, so it can never be confirmed or refuted by the observed data alone. You can test whether missingness is associated with observed covariates (which can rule out some forms of MCAR, see `mcar_test()` in Section 7.2), but you can never test whether missingness depends on the unobserved value itself — by definition, you do not have that value to check against. The choice between MAR and MNAR is ultimately a domain-knowledge argument, not a statistical result, and should be defended with clinical reasoning (do sicker patients plausibly drop out more?) rather than a p-value.

This matters enormously for HEOR because the three mechanisms license completely different analytic responses:

- Under **MCAR**, a complete-case analysis is unbiased — you lose precision (wider confidence intervals around your ICER) but not accuracy. This is the only mechanism under which "just drop the missing rows" is statistically defensible, and it is rarely plausible for anything beyond trivial administrative dropout.
- Under **MAR**, complete-case analysis is generally biased, but multiple imputation (MI) — drawing plausible values conditional on the observed covariates, as implemented in `mice` or `missForest` — recovers unbiased estimates, because MAR is precisely the assumption that licenses those methods. Most routine MI practice in trial-based cost-effectiveness analysis implicitly assumes MAR.
- Under **MNAR**, no standard imputation model will save you, because the imputation itself needs to encode a mechanism you cannot verify. The accepted response is either explicit mechanism modelling — selection models (a joint model for the outcome and the missingness indicator) or pattern-mixture models (fitting separate outcome models within each missingness pattern and specifying how they differ) — or, more commonly in practice, a structured sensitivity analysis that varies the assumed severity of MNAR departure and reports how the ICER moves.

In practice, no HEOR dataset wears a label saying which mechanism generated it. The discipline is to state your assumption explicitly, justify it with clinical and operational knowledge of why the data went missing, and then stress-test the conclusion against alternative, less convenient assumptions — which is exactly where Section 7.2's exploratory tools and later chapters' sensitivity-analysis techniques come in.

7.2 Exploring missingness (naniar, visdat, Shiny-based explorers)

Before choosing an imputation strategy or defending a mechanism assumption, you need to actually look at your missingness — not just count NAs with `colSums(is.na(df))`, but see where they fall, whether they cluster by variable, by patient, by visit, or by arm, and whether they co-occur in patterns that hint at a shared cause. The `naniar` and `visdat` packages, both built around tidyverse conventions, turn this into a few lines of code rather than a manual audit, and they should be a standard first step on any trial or claims extract destined for a cost-effectiveness model.

`visdat::vis_dat()` gives an overview of an entire data frame in one plot: each column is a vertical strip, coloured by data type (character, numeric, logical), with missing cells picked out distinctly, so you immediately see whether missingness is concentrated in a few columns (say, a late-added quality-of-life variable) or scattered everywhere. `visdat::vis_miss()` narrows this to missingness specifically, shading missing cells and printing the percentage missing per column, which is often the single most useful diagnostic plot to drop into an analysis log.

`naniar` extends this considerably. `naniar::vis_miss()` produces a similar heatmap-style plot but adds clustering options so that rows with similar missingness patterns are grouped together — useful for spotting whether, for instance, patients missing a 12-week EQ-5D are disproportionately the same patients missing a 12-week cost record, which would support a shared-cause MAR or MNAR story rather than independent MCAR noise in each column. `naniar::gg_miss_var()` plots the number (or proportion) of missing values per variable as a simple ranked bar chart, which is the fastest way to triage which columns deserve the most scrutiny. `naniar::gg_miss_upset()` goes further and shows an UpSet-style plot of **combinations** of co-missing variables — essential once you have more than two or three variables with missingness, where a simple pairwise correlation plot becomes unreadable. For a formal (if limited) statistical check, `naniar::mcar_test()` implements Little's MCAR test, which assesses whether the observed pattern of missingness is consistent with MCAR by comparing means across missingness patterns; a significant result lets you rule out MCAR, but — consistent with Section 7.1 — a non-significant result does not prove MCAR, it only fails to contradict it, and the test says nothing about MAR versus MNAR.

A worked illustration, using a hypothetical trial-based economic dataset with utility (EQ-5D index) and healthcare cost columns:

```
library(tidyverse)
library(naniar)
library(visdat)

# trial_econ: one row per patient, with baseline and 12-week
# utility scores, and total healthcare cost over follow-up
trial_econ <- read_csv("trial_econ_extract.csv")

# Overview of variable types and missingness across the frame
vis_dat(trial_econ)
```

```r
# Missingness-focused heatmap, clustered by pattern
vis_miss(trial_econ, cluster = TRUE)

# Which variables carry the most missingness?
gg_miss_var(trial_econ, facet = trial_arm)

# Do utility and cost missingness co-occur?
gg_miss_upset(trial_econ)

# Formal test of the MCAR assumption (Little's test)
mcar_test(trial_econ %>% select(utility_12wk, total_cost, baseline_utility))
```

The `facet = trial_arm` argument to `gg_miss_var()` is worth dwelling on: splitting missingness summaries by randomised arm is one of the simplest and most informative checks available in a trial context, because differential missingness between arms (more dropout on the active treatment, say, due to side effects) is a strong, visually obvious signal against MCAR and a prompt to think hard about MAR versus MNAR before modelling.

For datasets too large or too wide for a single static plot — a claims warehouse with hundreds of cost and utilisation fields, for example — static `vis_miss()` heatmaps become hard to read. R's Shiny framework is commonly used to wrap these same `naniar`/`visdat` summaries into an interactive explorer: a reactive app where an analyst filters by site, time window, or variable subgroup and regenerates the missingness plot on demand, rather than pre-generating dozens of static figures. This is a thin application layer over the same underlying functions shown above, not a different statistical method, but it materially speeds up the exploratory phase on large real-world evidence extracts.

Common pitfall. A non-significant result from `mcar_test()` does not certify your data as MCAR — it only means the test found no evidence against it. Little's test checks whether missingness correlates with **observed** variables; it is structurally blind to MNAR, where missingness depends on the unobserved value itself. Treating a passed MCAR test as licence for a complete-case analysis is one of the most common silent sources of bias in HEOR cost-effectiveness write-ups.

Agency perspective. When documenting missing-data handling in a trial-based economic evaluation for a submission, ground your approach in Faria, Gomes, Epstein & White's "A Guide to Handling Missing Data in Cost-Effectiveness Analysis Conducted Within Randomised Controlled Trials" (PharmacoEconomics, 2014) — a widely cited, methodologically detailed guide that walks through exactly this mechanism-first, exploratory-then-sensitivity-analysis workflow and is the standard reference HTA reviewers expect to see cited.

7.3 Complete-case versus imputation

The simplest response to missing data is to ignore it: keep only the rows with no missing values and analyse those as if they were the whole dataset. This is complete-case analysis, also called listwise deletion, and it is the default behaviour of most R modelling functions (`lm()`, `glm()`, `t.test()` all silently drop incomplete rows unless told otherwise). Its appeal is honesty of a kind

— every value that enters the analysis was actually observed, nothing is fabricated — and under one specific condition it is a perfectly defensible choice: when missingness is genuinely MCAR (missing completely at random, SS7.1) and the proportion missing is small, say under 5%. Under true MCAR, the complete cases are a random subsample of the full cohort, so estimates remain unbiased; the only cost is a loss of statistical power from the smaller sample size.

That caveat matters because the cost is rarely "only" power. Two failure modes deserve attention. First, even under MCAR, if a non-trivial fraction of records is missing — 20%, 30%, sometimes more for a patient-reported utility instrument like the EQ-5D at follow-up visits — complete-case analysis can discard a third or more of a trial's data, inflating standard errors and widening confidence intervals around the ICER to the point of uselessness, even though the point estimate itself stays unbiased. Second, and more insidious, if missingness is MAR or MNAR (SS7.1), dropping incomplete rows does not merely cost power — it introduces bias, silently. If patients with worse quality of life are systematically more likely to skip the utility questionnaire (plausibly MNAR), the complete-case subsample overrepresents healthier patients, and the estimated mean QALY is biased upward. No warning is printed; `lm()` will fit happily and report a confidence interval that looks perfectly respectable while being centred on the wrong number.

A step up from deletion is single imputation: filling in each missing value with one plausible number and then analysing the completed dataset as though it had been fully observed all along. Common variants include mean or median imputation (replace missing cost or utility values with the column mean/median) and last-observation-carried-forward (LOCF), common in longitudinal cost-effectiveness follow-up, which carries a patient's last recorded value forward to fill subsequent missing visits. Both are easy to implement and both are dangerous for the same underlying reason: they treat a **guess** as if it were **data**. Once the gaps are filled with a single fabricated number, every downstream standard error, confidence interval, and p-value is computed under the pretence that all values were actually measured. The uncertainty introduced by the act of guessing never enters the calculation, so standard errors come out systematically too small and confidence intervals too narrow — the analysis reports more confidence in the result than the data actually support.

The following toy example makes this concrete using the cost/QALY tibble convention from Chapter 4. Twenty patients' utility values are simulated, 30% are set missing, and we compare the standard error of the mean utility under naive mean-imputation against the standard error obtained via proper multiple imputation (introduced in SS7.4).

```
library(tidyverse)
library(mice)

set.seed(2024)
n <- 20
truth <- tibble(
  patient_id = 1:n,
  utility = rnorm(n, mean = 0.75, sd = 0.10)
)

# Introduce MAR-ish missingness: lower utility more likely to be missing
obs <- truth |>
  mutate(
    miss_prob = plogis(-3 + 4 * (0.80 - utility)),
    utility_obs = if_else(runif(n) < miss_prob, NA_real_, utility)
  )
```

```r
# --- Naive mean imputation ---
mean_fill <- mean(obs$utility_obs, na.rm = TRUE)
naive <- obs |> mutate(utility_imp = if_else(is.na(utility_obs), mean_fill,
utility_obs))

se_naive <- sd(naive$utility_imp) / sqrt(n)
cat("Naive mean-imputation SE:", round(se_naive, 4), "\n")

# --- Proper multiple imputation (mice), previewed from SS7.4 ---
imp <- mice(obs |> select(utility_obs), m = 20, method = "pmm", maxit = 5,
            printFlag = FALSE, seed = 2024)
fits <- with(imp, t.test(utility_obs))
pooled <- pool(fits)
cat("Multiply-imputed pooled SE:", round(summary(pooled)$std.error, 4), "\n")
```

Running this comparison typically shows the naive mean-imputation standard error noticeably smaller than the mice-pooled standard error, because filling every gap with the identical constant `mean_fill` mechanically shrinks the completed dataset's variance and hides the fact that the true value of each missing observation was never actually known. LOCF carries an analogous problem in longitudinal cost data: carrying forward a value assumes no change occurred, which both biases trends (toward flat trajectories) and understates variability.

> **Common pitfall.** A standard error computed on a singly-imputed dataset answers the question "how uncertain would I be if these fabricated values were real?" — not "how uncertain am I, given that I had to fabricate them?" The two numbers are not the same, and the gap is exactly the imputation uncertainty that single-imputation methods discard.

The motivation for SS7.4 follows directly: we want a method that fills in plausible values **and** propagates the extra uncertainty from not knowing those values into the final standard errors. That is precisely what multiple imputation, combined via Rubin's rules, is built to do.

7.4 Multiple imputation (mice, Amelia, missForest)

Multiple imputation solves the understated-uncertainty problem of SS7.3 by refusing to commit to a single guess. Instead of filling each missing cell once, it creates several complete versions of the dataset — typically `m = 5` to `m = 50` — each with its own plausible, independently drawn set of imputed values, drawing on the observed relationships between variables. The analysis of interest (a mean, a regression, an ICER) is then run separately on every imputed dataset, and the `m` results are combined using Rubin's rules, which pool the point estimates by simple averaging and pool the standard errors by combining two components: the **within-imputation** variance (the usual sampling variance you would get from any one completed dataset) and the **between-imputation** variance (how much the estimate itself jumps around depending on which imputed values happened to be drawn). That between-imputation term is exactly the piece that single imputation discards, and is why multiply-imputed standard errors are honest about missing-data uncertainty in a way mean-imputation or LOCF never can be.

The workhorse R package for this is `mice` ("Multivariate Imputation by Chained Equations"). The core function, `mice()`, imputes one variable at a time conditional on all the others, cycling

through variables for several iterations (`maxit`) and repeating the whole process `m` times to produce `m` independent completed datasets. The `method` argument lets you choose an imputation model per variable type: `"pmm"` (predictive mean matching) is the default and generally preferred choice for continuous variables like cost or utility, because it imputes using **observed** donor values rather than inventing numbers outside the plausible range; `"logreg"` handles binary variables (e.g. treatment response yes/no); `"polyreg"` handles unordered categorical variables (e.g. disease stage categories).

```
library(tidyverse)
library(mice)

set.seed(42)
heor_data <- tibble(
  patient_id = 1:40,
  treatment  = rep(c("A", "B"), each = 20),
  cost       = rnorm(40, mean = 1200, sd = 300),
  qaly       = rnorm(40, mean = 0.70, sd = 0.08)
)

# Inject missingness into cost and qaly
heor_miss <- heor_data |>
  mutate(
    cost = if_else(runif(40) < 0.25, NA_real_, cost),
    qaly = if_else(runif(40) < 0.20, NA_real_, qaly)
  )

imp <- mice(
  heor_miss |> select(cost, qaly, treatment),
  m = 10, method = c(cost = "pmm", qaly = "pmm", treatment = ""),
  maxit = 5, printFlag = FALSE, seed = 42
)

# Inspect one completed dataset
completed_1 <- complete(imp, action = 1)

# Fit the model of interest on every imputed dataset, then pool per Rubin's rules
fit <- with(imp, lm(qaly ~ treatment + cost))
pooled_fit <- pool(fit)
summary(pooled_fit)
```

`summary(pooled_fit)` returns the pooled coefficient estimates together with standard errors, degrees of freedom, and p-values that already incorporate both within- and between-imputation variance — these are the numbers that belong in a report, not the output of any single `complete()`d dataset taken in isolation. The same pattern (`with()` then `pool()`) applies equally to pooling a simple mean utility, a regression-adjusted cost difference, or an ICER-contributing coefficient, as long as the estimator produces a coefficient and standard error that `pool()` can combine.

> **Difficult terminology: degrees of freedom (pooled multiple-imputation estimates).** Degrees of freedom is, informally, a number that reflects how much genuinely independent information went into an estimate — the more of it you have, the more precisely you can pin down a confidence interval or p-value threshold, and the narrower a confidence interval

can be for the same observed effect. With a single dataset and a simple regression, degrees of freedom is usually something intuitive like "number of patients minus number of parameters estimated" — a whole number you could count on your fingers. After Rubin's rules pool several imputed datasets together, though, the reported degrees of freedom is deliberately **not** a whole number (you might see 47.3, say) and is often noticeably smaller than what a single complete dataset of the same size would give you. That's not a bug — it's the pooling formula explicitly admitting a penalty: because you don't actually know the true missing values, only several plausible guesses at them, there is genuine extra uncertainty beyond ordinary sampling error, and the fractional degrees of freedom is the mechanism by which that uncertainty gets reflected in a wider confidence interval than you'd otherwise report. The more the imputed datasets disagree with each other (high between-imputation variance), the more degrees of freedom gets discounted.

The conventional way to report `mice::pool()` output is a coefficient table carrying the pooled estimate, its standard error, a 95% confidence interval, and the fraction of missing information (FMI) per coefficient — FMI quantifies how much the uncertainty around that specific coefficient is inflated by the missingness, which a bare point estimate cannot convey on its own:

How to report this.

Term	Estimate	SE	95% CI	FMI
Intercept	0.612	0.041	0.531, 0.693	0.18
treatment (B vs. A)	0.034	0.019	−0.004, 0.071	0.22
cost	−0.00004	0.00002	−0.00008, 0.00000	0.31

Estimates, SEs and 95% CIs are pooled across m = 10 imputed datasets via Rubin's rules; FMI (fraction of missing information) is highest for the `cost` coefficient, consistent with cost carrying the larger share of injected missingness in this toy example.

Table 17: Pooled multiple-imputation regression coefficients (Rubin's rules) for `qaly ~ treatment + cost`, m = 10 imputations

HEOR in practice. When reporting a cost-effectiveness analysis built on multiply-imputed data, state `m` (the number of imputations), the imputation method per variable, and the pooled standard errors explicitly in your methods section — reviewers increasingly expect this, and NICE/ISPOR-aligned reporting checklists treat undisclosed single imputation or unexplained listwise deletion as a methodological red flag.

`mice` is not the only option, and two alternatives are worth knowing when to reach for. `Amelia` (package `Amelia`, function `amelia()`) uses an EM-with-bootstrapping (EMB) algorithm assuming multivariate normality, and is particularly well suited to time-series or longitudinal panel data — it has built-in support for time and cross-sectional ID arguments that `mice` lacks, making it a natural choice for multi-wave follow-up cost diaries. `missForest` (package `missForest`, function `missForest()`) takes a completely different, nonparametric approach: it iteratively imputes miss-

ing values using random forests grown on the other variables, which makes it robust to nonlinear relationships and naturally handles mixed continuous/categorical data without needing separate method specifications per variable type — genuinely convenient for messy real-world HEOR datasets with a tangle of cost, utility, comorbidity, and categorical variables together.

The convenience of `missForest` comes with a real limitation, though: it produces a single completed dataset per run, not naturally plural imputations in the Rubin's-rules sense, so the standard between/within-imputation variance decomposition that `mice`'s `pool()` performs does not apply in the same way. Running `missForest` multiple times with different seeds and attempting to combine the results post hoc is an active area of methodological debate, not an out-of-the-box solved problem the way `mice::pool()` is. For an analysis whose final deliverable is a pooled standard error destined for a cost-effectiveness acceptability curve or a confidence interval around an ICER, `mice` (or `Amelia` for longitudinal structures) remains the better-supported choice; `missForest` is better reserved for exploratory imputation or as a preprocessing step feeding a single downstream prediction model, where you are less dependent on a correctly propagated variance estimate.

7.5 Sensitivity analysis, including tipping-point and pattern-mixture approaches

Multiple imputation under the missing-at-random assumption, as covered in SS7.4, produces a single headline estimate and a confidence interval that already account for imputation uncertainty. It is tempting to treat that output as the final word. It is not. Recall from SS7.1 that MCAR, MAR, and MNAR cannot be distinguished using the observed data alone — the very information that would tell you whether the unseen values differ systematically from the seen ones is, by definition, unseen. Multiple imputation by chained equations assumes MAR conditional on the variables in the imputation model. That assumption is plausible, sometimes highly plausible, but it is never verifiable from the dataset in front of you. Any analysis that stops after reporting "MI point estimate, MI confidence interval" without asking "what if the missing values are systematically worse (or better) than what the model predicted" is incomplete. Sensitivity analysis is the second half of a credible missing-data analysis, not an optional appendix.

Tipping-point analysis is the most direct way to operationalise this concern, and it is intuitive enough to explain to a clinical reviewer without a statistics background. The idea: take your primary imputation model's results, then systematically shift the imputed values for one or more groups by a fixed offset (often called delta), re-run the analysis, and see how extreme that offset has to become before your conclusion reverses. If a trivially small, clinically implausible delta is enough to flip the conclusion, your result is fragile. If only an extreme, clinically implausible delta would flip it, your result is robust to departures from MAR.

Concretely, suppose a trial comparing a new TB regimen against standard of care has missing six-month treatment-success outcomes, imputed under MAR, and the base-case result favours the new regimen. A tipping-point loop varies a delta applied only to the imputed (not observed) values in the intervention arm, recomputing the between-arm difference at each delta:

```
library(dplyr)

tipping_point <- function(data, deltas) {
  results <- vector("list", length(deltas))
  for (i in seq_along(deltas)) {
```

```
    d <- deltas[i]
    adjusted <- data |>
      mutate(success_adj = if_else(imputed & arm == "intervention",
                                   success - d, success))
    diff <- mean(adjusted$success_adj[adjusted$arm == "intervention"]) -
      mean(adjusted$success_adj[adjusted$arm == "control"])
    results[[i]] <- tibble(delta = d, risk_difference = diff)
  }
  bind_rows(results)
}

tipping_point(trial_data, deltas = seq(0, 0.30, by = 0.02))
```

The same logic extends to cost-effectiveness: apply a delta to imputed QALYs (or to imputed costs) before recomputing the ICER, and report the delta at which the ICER crosses the willingness-to-pay threshold lambda established in Chapter 4. The statistical output of this loop is only half the job — the other half is a clinical judgement call: is a delta of 0.02 in treatment-success probability, or 0.05 QALYs, something that a clinician or patient-representative would consider plausible given what is known about why patients dropped out? If the tipping delta is clinically absurd, the primary conclusion stands on solid ground. If it is well within the range of plausible informative dropout, the conclusion is fragile and should be reported as such.

How to report this.

Delta applied to imputed values	Risk difference	Favours
0.00 (base case, MAR)	+18.0 pp	Intervention
0.05	+13.0 pp	Intervention
0.10	+8.0 pp	Intervention
0.15	+3.0 pp	Intervention
0.17 (tipping point)	0.0 pp	Neutral
0.20	−2.0 pp	Control
0.25	−7.0 pp	Control
0.30	−12.0 pp	Control

Interpretation: the conclusion favouring the intervention is robust unless imputed successes in the intervention arm are overstated by roughly 0.17 or more — a departure from MAR that clinical reviewers judged implausible given the dropout reasons recorded.

Table 18: Tipping-point sensitivity analysis: six-month treatment-success risk difference (intervention minus control) as the delta applied to imputed intervention-arm successes increases, TB regimen trial

`mice` supports this kind of delta-adjustment directly: after generating imputations under MAR, you can post-process the imputed (but not observed) values in each completed dataset by adding a fixed

offset before pooling, which is exactly the tipping-point mechanic above applied across multiply-imputed datasets rather than a single completed dataset.

Pattern-mixture models offer a complementary, more structural way to think about departures from MAR. Rather than imputing a single plausible value and then perturbing it, a pattern-mixture approach stratifies the data by missingness pattern — for example, "completed all follow-up," "dropped out after visit 2," "dropped out after visit 4" — and allows each pattern to have its own outcome distribution. You then combine these pattern-specific distributions into an overall estimate using explicit, stated assumptions about what the missing-pattern groups' outcomes would have looked like had they been observed. Those assumptions are where the scientific judgement lives: you might assume patients who dropped out track the same trajectory as similar patients still in follow-up (akin to assuming MAR within the model), or you might assume they track the trajectory of patients in the control arm from that point forward (a "jump to reference" assumption), or that their outcomes would have worsened faster than observed completers (a "placebo-based" or worsening assumption suited to safety concerns).

This reference-based family of assumptions is precisely the territory covered by `rbmi`, a CRAN package developed for reference-based multiple imputation in clinical trials, built to implement exactly this kind of pattern-mixture reasoning — imputing missing values for a given patient according to the trajectory of a specified reference arm or reference pattern rather than that patient's own arm, as a formal sensitivity check against informative dropout. Rather than naming specific `rbmi` functions here, the conceptual takeaway for an HEOR analyst is this: pattern-mixture and reference-based MI let you ask "what is the most defensible pessimistic (or optimistic) story about why these data are missing, and how much does the answer change under that story?" — which is a different and complementary question to the tipping-point analysis's "how far would the data need to be wrong before the conclusion flips?" A submission that reports both together is materially more convincing than one relying on MAR imputation alone.

7.6 Missing data in cost and QALY data

Everything in SS7.1-7.5 applies to missing data of any kind, but cost and QALY data bring two additional headaches that a simple missing covariate — say, a baseline lab value — does not.

The first is distributional shape. Healthcare cost data are notoriously right-skewed and often zero-inflated: most patients incur modest costs, a small number incur very large costs (a hospitalisation, a surgical complication), and some incur essentially zero cost in a given period (no healthcare contact at all). Missingness in cost data is rarely "random noise around the mean" — a missing cost record for a patient who was lost to follow-up after an uneventful visit looks nothing like a missing cost record for a patient who disappeared from the dataset immediately before an expensive hospitalisation. Naive mean imputation — filling every missing cost with the sample mean — is especially dangerous here because it does two things at once: it ignores the skew (replacing a potentially very high or very low true value with a moderate average), and it artificially shrinks the variance of the cost distribution, which then distorts downstream statistics like confidence intervals around incremental cost. The problem compounds with health-state-related dropout: patients who leave a trial because their condition is deteriorating are disproportionately likely to have missing late-stage cost and utility data, and deteriorating health plausibly correlates with higher resource use (more hospital contact) or, in some systems, reduced contact (patients too unwell to attend). Either way

the missingness is informative — a form of MNAR — and treating it as MCAR by simply dropping or mean-filling those records biases the cost-effectiveness result in an unpredictable direction.

QALYs and the utility weights that compose them bring a second, structural constraint: utilities are bounded, typically on a scale from a negative value (states considered worse than death) up to 1 (full health), and that boundedness has to be respected by any imputation procedure. An imputation model built carelessly — for instance, a linear regression imputing utility values without constraint — can happily generate an imputed utility of 1.08 or produce an imputed cost that is negative, both of which are nonsensical and will silently corrupt a downstream QALY or ICER calculation if not caught.

The practical implication is that multiple imputation for cost and QALY data must be done on the natural scale appropriate to each variable, with constraints enforced rather than assumed away. `mice` supports exactly this kind of post-processing: imputed values for a given variable can be bounded, for example clamping imputed utilities to the plausible range before they are carried into the pooled dataset, or ensuring imputed costs cannot fall below zero. Some analysts additionally transform cost data (for example, imputing on a log scale and back-transforming) to better respect the skew described above, provided the back-transformation and any associated bias correction are documented.

```
library(mice)

imp <- mice(trial_data, m = 20, method = c(cost = "pmm", utility = "pmm"),
            seed = 123)

# Post-process: clamp imputed utilities to the valid range and
# ensure imputed costs are non-negative
imp_clamped <- imp
for (i in seq_len(imp$m)) {
  comp <- complete(imp, i)
  comp$utility <- pmin(pmax(comp$utility, -0.59), 1)
  comp$cost <- pmax(comp$cost, 0)
  imp_clamped$imp$utility[[i]] <- comp$utility[is.na(trial_data$utility)]
  imp_clamped$imp$cost[[i]] <- comp$cost[is.na(trial_data$cost)]
}
```

> **Common pitfall.** Mean-imputing missing costs or utilities, or imputing them with an unconstrained model that ignores the [0,1]-ish bound on utilities and the non-negativity of costs, will silently distort the pooled cost and QALY estimates that feed into every downstream ICER calculation — always impute on the natural, bounded scale and sanity-check the completed dataset's range before proceeding.

This matters directly for the cost/QALY tibble pipeline introduced in Chapter 4: the `treatment`/`cost`/`qaly` tibble that later feeds dominance pruning and ICER computation is exactly the kind of dataset where undetected missing-data mishandling upstream becomes invisible downstream. A dominance-pruning step applied to a tibble built from carelessly mean-imputed costs will happily prune strategies and rank the remainder with complete confidence, with no visible trace that the input costs were distorted. Any analyst building that pipeline from trial or registry data with incomplete cost or utility records should treat the imputation step as upstream quality control for the entire economic model, not as a mechanical pre-processing formality.

7.7 Reporting missing data handling for HTA

A technically sound missing-data analysis that is reported badly is, from an HTA reviewer's perspective, close to useless — the reviewer cannot assess what they cannot see. CHEERS 2022, already cited in Chapter 3 in connection with its item on methods-reporting detail, contains a specific checklist item requiring transparent reporting of how missing data were handled, and that item is the natural anchor for what a submission needs to include on this topic.

At minimum, a submission should report four things. First, the amount and pattern of missingness, broken down by study arm or group, not just as an aggregate percentage — a 10% missingness rate split evenly across arms tells a very different story from 2% missing in one arm and 18% in the other. This is where the visualisation tools from SS7.2 (`naniar`, `visdat`) earn their keep as reporting aids, not just exploratory ones: a simple missingness-by-arm table or plot communicates more to a reviewer in seconds than a paragraph of prose. Second, the assumed missing-data mechanism and the justification for that assumption — stating "we assumed data were missing at random conditional on baseline severity, treatment arm, and visit number" is a reportable, falsifiable claim; stating "data were assumed missing at random" with no stated conditioning variables is not. Third, the primary analysis method, stated with enough specificity to be reproducible: for example, "missing six-month utility values were handled using multiple imputation by chained equations with m = 50 imputations, imputing conditional on baseline EQ-5D, age, treatment arm, and disease severity, with Rubin's rules used to pool estimates." Fourth, and often the most neglected, the sensitivity analyses performed and their result — specifically whether the tipping-point delta needed to overturn the primary conclusion was judged clinically plausible or implausible, and whether any pattern-mixture or reference-based alternative assumptions (SS7.5) changed the headline cost-effectiveness conclusion.

A useful discipline is to draft this reporting as a short, self-contained paragraph the moment the imputation and sensitivity analyses are finalised, rather than reconstructing it from memory at submission-writing time:

```
# Illustrative structure for the reporting paragraph's data
missingness_summary <- trial_data |>
  dplyr::group_by(arm) |>
  dplyr::summarise(
    n = dplyr::n(),
    n_missing_utility = sum(is.na(utility_6m)),
    pct_missing = round(100 * n_missing_utility / n, 1)
  )
```

How to report this.

Arm	N randomised	N missing utility (6m)	% missing
Intervention	120	22	18.3
Control	118	9	7.6
Overall	238	31	13.0

Mechanism assumed: MAR, conditional on baseline EQ-5D, age, treatment arm, and disease severity. The pronounced imbalance between arms (18.3% vs 7.6%) is itself evidence against MCAR and motivates the tipping-point sensitivity analysis in Table 18.

Table 19: Missing-data summary for the primary six-month utility endpoint, by randomised arm

Beyond CHEERS 2022, the ICH E9(R1) addendum on estimands and sensitivity analysis in clinical trials is the relevant reference for how intercurrent events — treatment discontinuation, rescue medication, death before an outcome can be assessed — and the missing data they generate should be framed from the outset of trial design, not patched on after the fact during analysis. ICH E9(R1)'s estimand framework asks the analyst to specify, before unblinding, how intercurrent events will be handled conceptually (treatment-policy, hypothetical, composite, while-on-treatment, or principal-stratum strategies), which in turn determines what the "right" missing-data assumption and primary analysis even are. An HTA submission that can point to a pre-specified estimand strategy, consistent with ICH E9(R1), alongside a CHEERS-compliant missing-data reporting paragraph, presents a far more defensible evidentiary package than one where the missing-data method appears to have been chosen after seeing which approach gave the most favourable result.

Difficult terminology: ICH E9(R1) intercurrent-event strategies (treatment-policy, hypothetical, composite, while-on-treatment, principal-stratum). An intercurrent event is anything that happens **after** randomisation that complicates the simple question "did the treatment work?" — a patient stops the study drug due to side effects, takes rescue medication, switches to the other arm, or dies before the outcome of interest can even be measured. The ICH E9(R1) addendum (an internationally agreed regulatory guidance document) insists that before a trial even starts, the protocol must say explicitly **which version** of "treatment effect" it is trying to measure in the presence of such events, because different choices quietly answer different clinical questions from the same raw data. The five named strategies are the menu of options: **treatment-policy** counts whatever happened to the patient regardless of whether they stayed on the assigned treatment (closest to "real-world effectiveness, warts and all"); **hypothetical** asks what would have happened in a hypothetical world where the intercurrent event never occurred (e.g., had rescue medication not been available) — this requires an assumption, since that world was never observed; **composite** folds the intercurrent event itself into the definition of a bad outcome (e.g., "treatment failure" includes both clinical worsening **and** stopping due to side effects, counted the same way); **while-on-treatment** only asks the question for as long as the patient was actually taking the assigned treatment, and stops counting afterward; and **principal-stratum** restricts the question to a particular subgroup defined by what would have happened to them under both treatments (e.g., only

patients who would have tolerated either drug) — the most statistically demanding of the five, since that subgroup can't be directly observed. A useful clinical analogy: if a surgical trial's patients sometimes need a second emergency surgery, "treatment-policy" reports outcomes for everyone regardless of whether a second surgery happened, while "hypothetical" tries to estimate what the outcome would have been if no second surgery had ever been needed — two legitimately different, both useful, questions from the same trial.

Agency perspective. CHEERS 2022 requires explicit reporting of the amount, pattern, and handling of missing data as part of a health economic evaluation, and ICH E9(R1) requires that intercurrent events and their associated missing-data handling strategy be specified in relation to a pre-defined estimand; together they are the two reference points an HTA-facing missing-data section should be written against.

Missing utility values are not only a missing-data problem — they are frequently a symptom of exactly the population, instrument, and measurement-timing choices that Chapter 8 addresses directly: how utilities are elicited, which instrument captures them, and how those choices interact with trial dropout. The next chapter turns to health-related quality of life and utility measurement in full.

Chapter 8. Health-Related Quality of Life and Utilities

Learning objectives

- Distinguish generic preference-based instruments (EQ-5D, SF-6D, HUI) from disease-specific instruments, and explain why HTA reference cases require the former for the primary QALY calculation.
- Score EQ-5D health-state profiles into utility indices using R's `eq5d` package and a correctly-chosen country value set.
- Build and validate a mapping (cross-walk) algorithm to predict EQ-5D-equivalent utilities from a disease-specific instrument.
- Analyse bounded, ceiling-spiked utility data with censored regression, beta regression, or finite mixture models rather than naive OLS.
- Apply utility decrements and age/sex-adjustment correctly within a decision-analytic model, and layer background mortality onto disease-specific mortality.
- Explain the three classic utility-elicitation methods — time trade-off, standard gamble, and discrete choice experiments — and how they underlie value-set construction.

Chapter 7 closed by flagging that missing utility values are often a symptom of the instrument, elicitation method, and measurement timing chosen for a trial — this chapter picks that thread up directly and covers how utilities are measured, valued, and modelled in the first place.

8.1 Generic and disease-specific instruments (EQ-5D, SF-6D, HUI, etc.)

A utility, in the HEOR sense that Chapter 4 introduced, is a single number that expresses how good or bad a health state is, anchored so that 1 represents full health and 0 represents dead. States judged worse than death carry negative values, which is exactly why Chapter 7 flagged the UK EQ-5D-3L tariff's floor of roughly -0.59. Utilities exist to do one specific job: they are the weight multiplied by time to convert raw life-years into quality-adjusted life-years. A year spent at utility 0.7 contributes 0.7 QALYs; a year at utility 1.0 contributes a full QALY. Everything in this chapter is really about how that single number gets produced, and the honest limitations of the instruments that produce it.

The instruments used to generate utilities fall into two broad families. The first is the generic, preference-based, multi-attribute family. These describe health using a small number of dimensions that are meant to apply to essentially anyone, regardless of diagnosis, and each possible combination of dimension levels has (or can be assigned) a population-derived preference weight. EQ-5D is the dominant example. EQ-5D-3L covers five dimensions — mobility, self-care, usual activities, pain/discomfort, and anxiety/depression — each scored at one of three levels (no problems, some problems, extreme problems), giving $3^5 = 243$ possible health states. EQ-5D-5L uses the same five dimensions but expands each to five levels (no, slight, moderate, severe, extreme problems), giving $5^5 = 3125$ states and materially reducing ceiling effects at the "no problems" end. SF-6D is

derived from the longer SF-36 or shorter SF-12 health-status questionnaires by selecting a subset of items that map onto six dimensions (physical functioning, role limitations, social functioning, pain, mental health, and vitality) and applying preference weights to that subset. HUI2 and HUI3, the Health Utilities Index instruments, use their own dimension sets — HUI3, the more commonly used of the two, covers vision, hearing, speech, ambulation, dexterity, emotion, cognition, and pain, each with multiple levels. All of these are "generic" in that the dimensions were chosen to be meaningful across disease areas, and all are "preference-based" in that someone, somewhere, ran a valuation study asking members of the public to trade off time or risk against being in a described health state, and used those responses to build the scoring weights — the value set or tariff that Section 8.2 unpacks.

The second family is disease-specific instruments: questionnaires built to capture the symptoms and functional domains that matter most within one condition. A TB treatment trial might use a TB-specific symptom and quality-of-life questionnaire tracking cough, sputum, fever, and treatment-related adverse effects; an oncology trial might use the EORTC QLQ-C30 or a tumour-specific module. These instruments are often considerably more sensitive to clinically meaningful change within the disease they were built for, because every item was chosen for its relevance to that condition, rather than for being a dimension that also has to make sense for, say, a hip-replacement patient or someone with depression.

This is precisely where the generic-versus-disease-specific trade-off becomes unavoidable for anyone doing HTA work. NICE's reference case, and comparable guidance under development for HTAIn, requires a generic preference-based instrument (EQ-5D specifically, for NICE) for the QALYs used in the base-case cost-effectiveness analysis. The reason is structural, not a matter of taste: resource-allocation bodies are routinely comparing a TB regimen against a cardiology intervention against an oncology drug, all competing for the same budget against the same willingness-to-pay threshold λ. That comparison is only coherent if "1 QALY" means the same thing regardless of which disease produced it, and that requires a common, cross-disease utility currency — which is exactly what a generic preference-based instrument is designed to supply, and what a disease-specific instrument, by construction, is not. A disease-specific score has no population preference weighting behind it and no natural anchor at 0 = dead and 1 = full health; it cannot be substituted directly into a QALY calculation without some bridge back to a utility scale, which is the mapping (or "crosswalk") problem that Section 8.3 takes up.

The honest cost of this requirement is sensitivity. Because EQ-5D's five dimensions are broad and generic, a TB treatment effect that substantially reduces cough and sputum but leaves mobility, self-care, usual activities, pain, and anxiety largely unchanged may show little or no movement on EQ-5D, even though patients and clinicians would recognise a real improvement. This is a known, published limitation, not a flaw to paper over: trial teams frequently collect both a generic preference-based instrument (for the regulatory/HTA utility) and a disease-specific instrument (for clinical sensitivity and secondary endpoints) side by side, precisely because neither alone tells the whole story.

> **HEOR in practice.** When you see a trial report a large, clinically important improvement on a disease-specific QoL questionnaire alongside a flat or barely-moved EQ-5D utility, do not assume the EQ-5D data is wrong or the analyst mis-scored it. Check first whether the dimensions EQ-5D captures are the ones the disease actually affects — a TB regimen that resolves cough and fatigue but doesn't touch mobility or self-care can legitimately show a

small EQ-5D gain despite a genuinely better patient experience. The instrument's breadth, not an error, is usually the explanation.

8.2 Value sets and scoring in R (eq5d)

An EQ-5D questionnaire, on its own, only produces a health-state description — a short string of digits, one per dimension, recording the level the respondent selected. For EQ-5D-5L this looks like "11223": no problems with mobility, no problems with self-care, slight problems with usual activities, moderate pain/discomfort, moderate anxiety/depression. That string is not yet a utility. Turning it into a single number anchored at 1 (full health) and 0 (dead) requires a value set, often called a tariff: a table of preference weights, one for every possible combination of dimension levels, derived from a population valuation study. For the UK EQ-5D-3L value set, those weights came from a time trade-off (TTO) study fielded in a representative sample of the UK public, in which respondents were asked to trade years of life in an impaired health state against fewer years in full health; the EuroQol Group's EQ-5D-5L valuation protocol (covered in more depth in Section 8.6) extended this with a combined TTO and discrete choice experiment design. The crucial point is that a value set is country-specific: it reflects the preferences of the population it was elicited in, and different countries' general publics do not value the same health-state descriptions identically.

This is where the CRAN package `eq5d` (Craig, Pickard, and colleagues) becomes the practical tool. Its central function, `eq5d()`, takes a data frame of respondent-level dimension scores — the digit-per-dimension profile for either the 3L or 5L version — and an argument specifying which national value set to apply, returning a single utility index per respondent computed against that value set's published weights. The package bundles several countries' published value sets so that the same health-state profile can be scored consistently against, say, a UK tariff or a different country's tariff, without the analyst having to hand-code the weight lookup. (If you need the exact argument names and value-set codes, check `?eq5d` in your installed version rather than relying on a syntax from memory — argument spellings and the set of bundled countries have evolved across package releases.)

A toy workflow illustrates the shape of the task, independent of the exact argument names your installed version expects:

```
library(eq5d)

# Toy EQ-5D-5L health state profiles: one row per patient,
# one column per dimension (mobility, self-care, usual activities,
# pain/discomfort, anxiety/depression), each scored 1-5.
health_states <- data.frame(
  MO = c(1, 2, 3),
  SC = c(1, 1, 2),
  UA = c(2, 2, 3),
  PD = c(2, 3, 4),
  AD = c(1, 2, 3)
)

# Conceptual scoring step: apply a named country value set to convert
# each row's digit profile into a utility index anchored at 1 = full
# health, 0 = dead. Consult ?eq5d in your installed version for the
```

```
# exact argument names/spelling before running this on real data.
# utilities <- eq5d(scores = health_states, version = "5L", country = "UK")
# utilities
```

The reason the country argument matters so much is not a technicality — it is a well-documented source of real HTA submission errors. A UK NICE submission scored against a US value set (or vice versa) will produce numerically different utilities for the identical health-state description, because the two populations' revealed preferences for, say, "extreme pain/discomfort" are not the same. NICE's reference case expects EQ-5D-3L scored with the UK value set specifically (with EQ-5D-5L data generally expected to be mapped onto the 3L value set per NICE's published position, rather than scored with an EQ-5D-5L value set directly) — using the wrong country's tariff, or the wrong version-to-value-set pairing, is a substantive methodological error, not a stylistic choice, and reviewers will query it.

This also bears directly on HTAIn's situation. India does not yet have as long-established and widely validated a national EQ-5D value set as the UK's; a UK-style TTO valuation study in a representative Indian population is a substantial undertaking that has only partially matured. In practice, analysts working on Indian HTA submissions have sometimes relied on a crosswalk or value-set transfer from another country (or a regional proxy) as a pragmatic substitute, flagging it explicitly as a limitation rather than treating it as equivalent to a locally-derived tariff. That gap — and what a defensible crosswalk does and doesn't justify — is exactly the kind of LMIC-relevant methods question worth tracking as India's own value-set work progresses.

> **Agency perspective.** NICE's "NICE health technology evaluations: the manual" (PMG36) sets out the reference-case preference for EQ-5D as the measure of health-related quality of life for the UK reference case, with EQ-5D-3L (UK value set) as the default and specific guidance on how EQ-5D-5L data should be handled in relation to it. If you are preparing a submission that leans on EQ-5D utilities, check the current PMG36 text directly rather than relying on a secondary summary — NICE's position on 5L-to-3L handling has been updated over time.

8.3 Mapping algorithms (cross-walks and regression-based mapping)

Trials rarely collect every outcome measure an HTA body will eventually want. A pragmatic trial in TB treatment-shortening might collect a disease-specific symptom score or a clinical endpoint like time-to-culture-conversion; an oncology trial might collect FACT-G or an EORTC QLQ module. These instruments are excellent at detecting disease-specific change but they do not, by themselves, produce a generic preference-based utility value. An HTA reference case, however, almost always demands a utility anchored to a generic instrument such as EQ-5D, because that is what lets the QALY calculation sit on a common scale across disease areas and across submissions. When a trial has not collected EQ-5D at all, or has collected it too sparsely to be usable, the analyst is stuck with a dataset that has the clinical story but not the utility currency the model needs.

The standard fix is "mapping," sometimes called cross-walking. The logic is: find (or commission) an external dataset, ideally in a similar patient population, where both the disease-specific instrument and EQ-5D were administered to the same people at the same time. Fit a statistical model in that external dataset that predicts an EQ-5D-equivalent utility from the disease-specific scores. Then

apply that fitted algorithm to the trial's disease-specific scores to generate predicted utilities for the economic model. The trial itself may never have asked a single EQ-5D question; the mapping algorithm substitutes for that missing measurement, provided the external dataset is a defensible match for the trial population and condition severity range.

There are two broad families of mapping approach, and the distinction matters for how much of EQ-5D's internal structure survives the exercise.

- **Direct (response) mapping** models each EQ-5D dimension separately — for the 3L instrument, mobility, self-care, usual activities, pain/discomfort, and anxiety/depression, each with its own ordered response levels. Because the dimension responses are ordered categorical outcomes rather than continuous ones, this is typically done with multinomial or, more naturally given the ordering, proportional-odds ordinal regression — `MASS::polr()` is the standard base-R-adjacent tool for a single ordinal dimension, fit once per dimension against the predictors derived from the disease-specific instrument. The predicted dimension-level response profile for each respondent is then run back through the standard EQ-5D value set to produce a predicted utility. This route is more faithful to how EQ-5D utilities are actually constructed — it respects the fact that a utility index is a deterministic function of five discrete dimension responses, not a free-floating continuous number — but it means fitting and maintaining five separate models and then composing them correctly through the value set algebra.
- **Indirect (utility/value) mapping** skips the dimension-level detail entirely and regresses the EQ-5D utility index itself — the single number that comes out after the value set has already been applied — directly against the disease-specific scores, using ordinary regression methods such as `lm()` or a generalized linear model via `glm()`. This is markedly simpler to specify, fit, and report, and is the more common choice in practice precisely because of that simplicity. The cost is that it throws away the compositional structure of EQ-5D — two very different dimension profiles can map to the same utility value, and the model has no way to represent that heterogeneity or to use dimension-specific external information.

Whichever family is chosen, the indirect route has a well-known statistical trap: fitting `lm()` directly to a continuous utility index via ordinary least squares is a common choice in the mapping literature but a theoretically poor one. EQ-5D utility values are not normally distributed — they are bounded above at 1.0 (full health), often have a visible spike or clustering of observations exactly at that ceiling, and are left-skewed with a long tail toward poor health states and, for 3L value sets, can take values below zero. OLS assumes none of that: it allows predicted values above 1.0 or in otherwise impossible regions, and its homoscedastic-normal-error assumption is a poor description of a variable with a boundary spike. This is exactly the motivation for the more careful bounded-outcome methods covered in SS8.4 — censored regression, beta regression, and mixture models — all of which are designed to respect the ceiling and the skew that OLS ignores.

> **Difficult terminology: homoscedasticity.** Homoscedasticity is the assumption, built into ordinary linear regression, that the "scatter" of errors around the fitted line is roughly the same size everywhere — whether you're predicting a utility of 0.9 or 0.3, the model assumes the typical prediction error is about the same magnitude. (Its opposite, **heteroscedasticity**, means the scatter changes depending on where you are on the scale.) A clinical analogy: imagine measuring blood pressure readings in a cuff that's equally noisy whether the true reading is 90 or 190 — that's homoscedastic. EQ-5D utility data violate this badly: close to the 1.0 ceiling, a large fraction of respondents report exactly full health, so the "noise" clusters tightly at that

> boundary, while scatter for people in poorer health states can be much wider. Fitting OLS anyway doesn't just produce a slightly-off standard error — the model is implicitly assuming a pattern of noise that the data visibly don't have, which is part of why beta regression and mixture models (built to handle changing variance near a boundary) are preferred for this kind of outcome.

A mapping algorithm is only as trustworthy as its out-of-sample performance, and this is where submissions are most often challenged. Fitting a model in the external dataset and reporting how well it fits that same data (in-sample R-squared, in-sample residuals) tells a reviewer almost nothing about how the algorithm will behave when applied to a different trial population. The expected standard is some form of cross-validation — holding out a fold of the external dataset, fitting on the remainder, predicting the held-out fold, and reporting the discrepancy, typically as mean absolute error (MAE) or root-mean-squared error, between predicted and observed EQ-5D utilities. Algorithms that perform well on average but poorly at the extremes of the health-state range (very good or very poor health, which is often exactly where trial arms diverge) deserve particular scrutiny, because that is precisely the region driving the incremental QALY gain the economic model is built to estimate.

8.4 Analysing utility data: censoring, bounded distributions, beta regression, mixture models

Once utility values are in hand — whether collected directly via EQ-5D in the trial or produced by a mapping algorithm as in SS8.3 — the temptation is to treat them like any other continuous outcome and reach for `lm()`. Resist that temptation. EQ-5D utility distributions have a shape that ordinary linear regression's assumptions do not describe well. Full health (utility = 1.0) is a genuinely common and clinically meaningful response, so in most patient samples there is a visible spike or clustering of observations exactly at 1.0 — a ceiling effect, not a coincidence of rounding. Below that ceiling, the remaining distribution is typically left-skewed, with a longer tail stretching toward poorer health states. For 3L-style value sets that permit utilities below zero (states considered worse than death), there can be a further discontinuity or secondary clustering near or below zero. A method that assumes symmetric, unbounded, constant-variance normal errors — exactly what OLS assumes — will routinely predict impossible values above 1.0, misrepresent the variance near the ceiling, and bias both point estimates and confidence intervals.

Three families of real, well-documented approaches handle this more honestly.

- **Censored (Tobit-style) regression** treats the ceiling at 1.0 (and, where relevant, a floor) as genuine censoring rather than noise: the "true" underlying health preference may extend above 1.0 for some respondents, but the EQ-5D scale cannot record it, so observations piled at the boundary are modelled as censored rather than as ordinary data points. Two real CRAN packages implement this for utility-type data: `VGAM::vglm()`, which supports a tobit family among its very large catalogue of vector generalized linear/additive models, and the purpose-built `censReg` package, whose `censReg()` function fits Tobit-type censored regression with a syntax close to `lm()`/`glm()`. Either is a legitimate choice; `censReg()` is the more direct and commonly cited for univariate ceiling-censored utility outcomes.
- **Beta regression** exploits the fact that, apart from the boundary spikes, a utility index confined to (0, 1) looks much like a Beta-distributed variable — flexible enough to capture skew and

heteroscedasticity that increases or decreases near the bounds. The practical obstacle is that the Beta distribution is defined on the **open** interval (0, 1), excluding the exact endpoints, while real utility data routinely contain exact 1s (and exact 0s for some value sets). The standard workaround is a small linear rescaling of the outcome before fitting — squeezing the data just inside (0, 1) — after which the `betareg` package's `betareg()` function fits the regression with a logit (or other) link on the mean and, optionally, a separate model for the precision/dispersion parameter.

- **Finite mixture models** take the ceiling spike seriously as a distinct subpopulation rather than an inconvenience to transform away: conceptually, the observed data are treated as a mixture of a point mass exactly at 1.0 (representing respondents in full health) and a continuous component, such as a Beta or truncated normal distribution, describing everyone else. The model estimates both the mixing proportion (what fraction of the sample sits at the ceiling) and the parameters of the continuous piece, which can better represent the genuinely bimodal-looking shape of utility data than beta regression alone. This is more involved to specify and fit; `flexmix` is a real CRAN package for finite mixture modelling in R and is a reasonable starting point if this route is pursued, though for most submissions a beta regression with a sensible rescaling is the practical default.

A worked example with beta regression on a small toy EQ-5D-type dataset, illustrating the rescaling step and the model fit:

```
library(betareg)

# toy utility data: mix of observations with a visible ceiling spike
set.seed(123)
n <- 200
age <- round(rnorm(n, 55, 12))
severity <- sample(c("mild", "moderate", "severe"), n, replace = TRUE,
                   prob = c(0.4, 0.4, 0.2))

# simulate utilities with ceiling clustering at 1.0
raw_utility <- pmin(1, rbeta(n, 6, 2) + ifelse(severity == "mild", 0.15, 0))

df <- data.frame(utility = raw_utility, age = age, severity = severity)

# betareg requires outcomes strictly inside (0, 1); rescale to avoid
# exact boundary values using the standard Smithson-Verkuilen transform
n_obs <- nrow(df)
df$utility_resc <- (df$utility * (n_obs - 1) + 0.5) / n_obs

fit_beta <- betareg(utility_resc ~ age + severity, data = df)
summary(fit_beta)

# predicted mean utilities on the original scale
df$pred_utility <- predict(fit_beta, type = "response")
```

The fitted coefficients are on the logit-of-the-mean scale, so they describe proportional shifts toward or away from full health rather than additive utility-point changes — a feature, not a bug, given the bounded outcome, but one that needs stating explicitly when results are reported to a non-technical HTA reviewer.

Difficult terminology: logit link (in beta regression). A "link function" is the piece of machinery that lets a regression model handle an outcome that isn't a plain, unbounded continuous number — it's a mathematical conversion applied before fitting, then reversed afterward for interpretation. The logit link specifically takes a value confined to (0, 1) — like a probability, or here a rescaled utility — and stretches it onto an unbounded scale running from minus infinity to plus infinity, via the formula $\log\left(\frac{p}{1-p}\right)$. Ordinary regression machinery, which is built to predict any real number, can then operate freely on that stretched scale without ever risking a prediction outside (0, 1); converting the fitted line's predictions back through the inverse logit guarantees every predicted value lands back between 0 and 1, no matter how extreme the predictors are. It's conceptually identical to the logistic regression you may already know from predicting a binary clinical outcome (e.g. mortality) — same transformation, same reason for using it (keeping predictions between 0 and 1) — just applied here to a continuous utility mean rather than a 0/1 outcome. The practical consequence, as the previous paragraph notes, is that beta-regression coefficients describe **proportional** movement toward or away from full health, not a fixed number of utility points per unit of the predictor — so a coefficient can't be read the same way an OLS coefficient would be.

Before any regression is reported, a submission should show the descriptive shape of the utility data the model is addressing — mean utility by subgroup alongside the proportion of observations sitting exactly at the ceiling, since that ceiling spike is the whole reason OLS was rejected in favour of beta regression:

How to report this.

Severity	N	Mean utility	SD	% at ceiling (= 1.0)
Mild	71	0.892	0.114	25.4%
Moderate	83	0.719	0.159	0.0%
Severe	46	0.752	0.159	0.0%
Total N = 200				

Table 20: Mean (SD) EQ-5D-type utility and ceiling-spike proportion by disease severity subgroup, toy dataset

Common pitfall. Fitting `lm()` directly to a raw EQ-5D utility index, or handing over a mapping algorithm built in SS8.3 without reporting its out-of-sample predictive performance, are two sides of the same error: both substitute a convenient method for a validated one. OLS on a ceiling-spiked, bounded outcome can predict utilities above 1.0 and will misstate uncertainty exactly where the incremental QALY calculation is most sensitive. A mapping algorithm reported only with in-sample fit statistics tells a reviewer nothing about how it behaves on a different trial population. Before either is used in a submission, show cross-validated performance — MAE or RMSE on held-out data for a mapping algorithm, and residual/ceiling diagnostics for a utility regression — not just a coefficient table.

8.5 Utility decrements, age-adjusted utilities and background mortality

A **utility decrement** is the loss in health-state utility associated with a specific clinical event, adverse event, or disease complication, expressed relative to some baseline utility. For example, a grade 3-4 chemotherapy-related adverse event might carry a utility decrement of around −0.05 to −0.10, applied for the duration the event is assumed to affect the patient (often a single model cycle, sometimes longer for persistent toxicities). Rather than measuring these decrements directly in every trial — which would require administering EQ-5D or SF-6D specifically during the adverse event window, something many trials never do — modellers routinely pull decrements from the published literature. This is the same literature-as-data-source logic introduced in Chapter 5′s discussion of secondary utility sources (SS5.3), operationalised through the systematic literature review methodology covered in Chapter 6: a systematic search identifies prior utility studies in the relevant condition, and the decrement estimates feed into the decision-analytic model as model inputs with their own uncertainty distributions, rather than being derived from the trial dataset at hand.

Within a decision-analytic model — the Markov cohort models and decision trees that Chapters 17-20 build out in full — decrements are combined with a baseline health-state utility in one of two ways:

- **Additive**: utility = baseline utility + decrement (where the decrement is negative), e.g. 0.75 - 0.08 = 0.67 while the adverse event is active.
- **Multiplicative**: utility = baseline utility × (1 + proportional decrement), e.g. 0.75 × (1 - 0.10) = 0.675.

Both are common in practice, and for a single decrement the numerical difference is often small. The choice matters more once several decrements, age-adjustments, and disease-progression utility changes are stacked across a long time horizon, as discussed below.

A related but distinct concept is **age-adjustment**. Health-state utility is not a fixed property of a diagnosis alone; a patient's rating of their own health, and the population reference values against which models are often calibrated, tend to decline with age even among people who are otherwise well. General-population EQ-5D normative (reference) data, published for many countries alongside the country-specific value-set studies discussed in SS8.3, report mean utility by age band and sex — typically declining from values close to 1 in young adults into the 0.7-0.8 range in the oldest age bands, with some variation by value set. This matters because a disease-specific "full health" utility estimated in, say, a trial population with a mean age of 45 should not simply be projected unchanged onto a modelled cohort that ages into its 70s and 80s over a long time horizon; doing so would imply a disease-free elderly cohort member is healthier than the general elderly population, which is implausible.

The conventional fix is a **multiplicative age/sex-adjustment**: the disease-specific utility is scaled by the ratio of the population norm utility at the cohort's current age/sex to a fixed reference (often the norm at trial baseline age, or simply applied as utility(disease) × norm(age, sex) / norm(baseline age, sex)). This keeps the disease utility anchored to how the general population's health is expected to decline, without requiring a separate age-specific disease-utility estimate for every cycle. A purely **additive** age adjustment (subtracting a fixed age-related decrement each cycle) is simpler to code but is more prone to producing negative or clearly implausible utilities once combined with several other additive decrements (treatment toxicity, disease progression, comorbidity) over a 20-

or 30-year model horizon — there is no multiplicative floor stopping the cumulative subtraction from running arbitrarily low. Multiplicative approaches, by contrast, naturally compress toward (but rarely cross) zero, which is one reason HTA bodies and modellers often prefer them for long-horizon cohort models, alongside the practical argument that proportional age decline is closer to what the normative data actually show.

Background mortality is the other half of keeping a long-horizon Markov model plausible, and it connects directly to Chapter 4's treatment of discounting and half-cycle correction. A cohort trace needs an age- and sex-specific **all-cause** (background) mortality rate — typically sourced from a national life table — layered on top of any disease-specific mortality transition estimated from trial or registry data. Without this, the model would implicitly assume that a cohort member who survives the disease being modelled never dies of anything else, which overstates survival and therefore overstates total QALYs and costs accrued in later cycles. The half-cycle correction from Chapter 4 still applies here: since death (from either cause) is assumed to occur, on average, mid-cycle rather than precisely at cycle boundaries, failing to half-cycle-correct the mortality-driven QALY accrual introduces a small but systematic bias, compounding over discounted future cycles.

A simple illustration of an age-band multiplier applied to a toy disease utility:

```
# toy age-band utility multipliers (illustrative, not a real norm dataset)
age_norms <- data.frame(
  age_band = c("18-44", "45-54", "55-64", "65-74", "75+"),
  norm_utility = c(0.93, 0.88, 0.83, 0.78, 0.72)
)

baseline_disease_utility <- 0.80   # disease-specific utility at trial baseline
age
baseline_norm <- age_norms$norm_utility[age_norms$age_band == "45-54"]

age_adjust <- function(age_band, disease_utility = baseline_disease_utility) {
  norm_now <- age_norms$norm_utility[age_norms$age_band == age_band]
  disease_utility * (norm_now / baseline_norm)
}

sapply(age_norms$age_band, age_adjust)
```

The resulting age-banded utilities are exactly what belongs in a model-inputs table, since a reviewer needs to see the population norm, the derived multiplier, and the resulting age-adjusted disease utility side by side rather than re-deriving them from the code:

How to report this.

Age band	Population norm utility	Norm ratio vs. baseline band	Age-adjusted disease utility
18-44	0.93	1.057	0.846
45-54 (baseline)	0.88	1.000	0.800
55-64	0.83	0.943	0.754
65-74	0.78	0.886	0.709
75+	0.72	0.818	0.655

Age-adjusted disease utility = baseline disease utility (0.80) × (norm utility at current age band / norm utility at baseline band). Norms are illustrative toy values, not a published national dataset.

Table 21: Multiplicative age-adjustment of a toy disease-specific utility (baseline 0.80 at age 45-54) against general-population EQ-5D norms

This multiplicative rescaling ensures that as the modelled cohort ages into the "75+" band, its disease-specific utility is pulled down in proportion to the expected population-level decline, rather than being held artificially constant or subtracted down to an implausible floor.

8.6 Time trade-off, standard gamble and discrete choice experiments

The value sets and tariffs introduced in SS8.2 — the algorithms that convert an EQ-5D-5L or SF-6D health-state description into a single index utility — are not estimated from thin air. They are built from general-population preference elicitation exercises using one of a small number of classic methods, each asking respondents to make trade-offs that reveal how much they value one health state relative to another, or relative to death.

Time Trade-Off (TTO) is the method underlying the majority of existing EQ-5D value sets, including the original EQ-5D-3L sets and many EQ-5D-5L sets produced using the EuroQol Group's standard protocol. A respondent is asked to choose between living a fixed duration T years in an impaired health state, versus living a shorter duration $x < T$ years in full health followed by death. The duration x is varied (typically via an iterative "ping-pong" or titration procedure in a face-to-face interview) until the respondent is indifferent between the two options. At that indifference point, the utility of the impaired health state is simply $\frac{x}{T}$. TTO is relatively intuitive to administer — people readily grasp the idea of trading survival time for better health — but it is not strictly grounded in expected utility theory, since no uncertainty or gamble is involved; it is better thought of as a theoretically simpler, practically convenient elicitation task.

Standard Gamble (SG) is the method with the most direct theoretical grounding, rooted in von Neumann-Morgenstern expected utility theory. Here the respondent chooses between a certain

outcome (living in the impaired health state for the remainder of life) and a gamble with probability p of achieving full health and probability $1 - p$ of immediate death. The probability p is varied until the respondent is indifferent between the certain health state and the gamble; the utility of the health state is then read off directly as that indifference probability p. Because SG explicitly incorporates risk and is derived from axioms about rational choice under uncertainty, health economists have historically regarded it as the theoretically "purer" utility measure — though it is cognitively more demanding to administer, and respondents' answers can be sensitive to how probabilities are communicated, which is part of why TTO has been more widely adopted for large-scale value-set surveys.

Discrete Choice Experiments (DCE) take a different approach entirely. Rather than trading a single health state off against survival duration or a probability of death, respondents are shown repeated pairs (or sometimes larger sets) of hypothetical health-state profiles — each profile described by levels across the instrument's dimensions — and simply pick which of the two they prefer. No explicit trade-off against death or duration is built into any single choice task. Preference weights are instead recovered statistically from the pattern of choices across many respondents and many choice pairs, typically using a conditional logit or multinomial logit model that estimates how much each dimension level contributes to the probability of a profile being chosen. Because a DCE alone only yields utilities on an arbitrary scale anchored between the best and worst profiles shown, many modern value-set studies combine DCE with a smaller TTO or SG component (a "DCE with duration" or "DCE-TTO" hybrid design) specifically to anchor the resulting scale onto the conventional dead = 0, full health = 1 metric needed for QALY calculation. This hybrid DCE-based approach is the method underlying several of the newer EQ-5D-5L value sets, and it is a practical option increasingly considered where a country does not yet have its own long-established TTO-based value-set tradition to extend, since DCE fieldwork can be logistically lighter than face-to-face TTO interviewing at national scale.

> **Agency perspective.** When sourcing or reporting a country's EQ-5D value set methodology for an HTA submission, check the elicitation method and protocol version against the EuroQol Group's published EQ-5D-5L valuation protocol, and confirm the chosen value set matches the preference stated in the relevant reference-case guidance (for UK submissions, NICE's Health Technology Evaluations Manual sets out the expected utility value sourcing hierarchy) — do not assume a TTO-based and a DCE-based value set for the same instrument are numerically interchangeable.

In R, DCE choice data — typically structured as matched sets of chosen/not-chosen profiles per respondent per task — can be analysed with the `mlogit` package, which fits multinomial and conditional logit models to this kind of panel choice data, or with `survival::clogit()` for a conditional logistic regression formulation when each choice set is treated as a matched stratum. Either route estimates a coefficient for each attribute level, which, once anchored and rescaled, becomes the attribute-level preference weight feeding into the value-set algorithm.

A value-set methodology section conventionally summarises the three elicitation methods side by side, since a reviewer needs to see at a glance which theoretical grounding, response burden, and anchoring approach underlies the tariff being used:

Method	Elicited quantity	Theoretical basis	Anchoring to dead = 0
Time Trade-Off (TTO)	Indifference duration x in full health vs. T years impaired	Practical/intuitive, not expected-utility-theoretic	Direct: $\frac{x}{T}$
Standard Gamble (SG)	Indifference probability p of full health vs. death	von Neumann-Morgenstern expected utility theory	Direct: utility $= p$
Discrete Choice Experiment (DCE)	Choice between pairs of health-state profiles, no survival/death trade-off	Random utility theory (McFadden)	Indirect: requires a TTO/SG anchoring component

Table 22: Comparison of TTO, SG and DCE elicitation methods used in EQ-5D value-set construction

Difficult terminology: random utility theory (McFadden). Random utility theory is the statistical theory underlying discrete choice experiments (and much of modern choice modelling generally, including travel and marketing research — it's named after the economist Daniel McFadden, who won a Nobel Prize partly for this work). Its core idea: when a person chooses between two options (here, two hypothetical health-state profiles), each option has some "true," fixed underlying attractiveness to that person (its utility) — but on top of that, there's a random, unpredictable wobble in each specific choice, because people aren't perfectly consistent decision machines, moods vary, and attention lapses happen. No individual choice tells you much on its own, since you can't separate the "true" attractiveness from that day's wobble. But across hundreds of choices made by many respondents, the random wobbles average out, and statisticians can work backward from the **pattern** of choices to estimate how much each attribute (e.g. pain level, mobility limitation) contributes to overall attractiveness on average. This is exactly why a DCE needs many repeated choice tasks per respondent rather than a single question, unlike TTO or SG which can, in principle, extract a usable answer from one careful elicitation.

Utilities and costs are the two halves of the cost-per-QALY pair this book has worked with since Chapter 4's introduction of the WTP threshold; having now covered how the utility half is measured, valued and modelled, Chapter 9 turns to its counterpart — costs and resource use.

Chapter 9. Costs and Resource Use

Learning objectives

- Decompose costing into identification, measurement and valuation, and structure resource-use data in R so quantity and unit cost remain separately traceable.
- Apply inflation adjustment and currency conversion (nominal exchange rate vs. PPP) correctly and in the right order when combining unit costs across years and countries.
- Recognise the skewness, zero-inflation and heavy-tailed structure typical of patient-level cost data, and explain why it breaks ordinary least squares' assumptions.
- Fit and interpret gamma GLMs and two-part (hurdle-style) models for skewed, zero-inflated cost data, and avoid the retransformation-bias trap of naive log-OLS.
- Distinguish prevalence-based from incidence-based cost-of-illness studies and top-down from bottom-up (micro-costing) approaches, and combine them appropriately for LMIC programme costing.
- Explain why censored cost data require specialised estimators (Lin's partitioned estimator, inverse-probability-of-censoring weighting) rather than a naive mean of observed costs.

Chapter 8 closed by noting that utilities and costs are the two halves of the cost-per-QALY pair this book has worked with since Chapter 4′s introduction of the willingness-to-pay threshold. Having covered how the utility half is measured, valued and modelled, this chapter turns to its counterpart: how resource use is identified and measured, how unit costs are valued and adjusted across time, currency and country, and how the resulting cost data — skewed, zero-inflated, sometimes censored — is analysed defensibly rather than with a naive mean or an inappropriate linear model.

9.1 Identifying, measuring and valuing resources

Costing in economic evaluation is conventionally decomposed into three steps that map neatly onto R workflow stages: **identification** (which resource items are relevant to the perspective and the decision problem), **measurement** (quantities of each item consumed per patient, per cycle, or per health state — the same cycle structure you built in Chapter 4′s Markov trace), and **valuation** (attaching a unit cost to each quantity). Keeping these three steps as distinct columns in your working data, rather than collapsing straight to a single "cost" figure, is what lets you later re-value the same resource-use data under a different perspective or a different country's unit costs without re-extracting anything.

For a TB programme, identification usually spans at least four resource categories: diagnostic (sputum smear, Xpert MTB/RIF, chest X-ray, culture/DST), drug (first-line FDCs, second-line regimens for drug-resistant TB), programmatic/health-system (DOT visits, counselling, contact tracing), and patient-borne (travel, time, lost income) — the latter mattering if your analysis takes a societal perspective, which is common in LMIC HTA submissions even when the reference case is health-system focused. Decide and document the perspective **before** you start identifying items, because it determines which rows belong in your resource-use table at all.

A disciplined way to carry this through an R pipeline is a long-format resource-use tibble, with quantity and unit cost kept as separate columns:

```
library(tibble)
library(dplyr)

resource_use <- tribble(
  ~patient_id, ~health_state, ~item,              ~category,    ~quantity, ~unit,
  "TB001",     "on_treatment","xpert_test",       "diagnostic", 1,         "test",
  "TB001",     "on_treatment","fdc_regimen",       "drug",       6,
"month",
  "TB001",     "on_treatment","dot_visit",         "programmatic",24,
"visit",
  "TB002",     "retreatment", "culture_dst",       "diagnostic", 1,
"test",
  "TB002",     "retreatment", "second_line_regimen","drug",      8,
"month"
)

resource_use
```

Quantity measurement itself typically comes from one of three sources you already know how to ingest from Chapter 5: trial-level case report forms, programmatic/surveillance data (e.g. national TB programme line-lists), or expert elicitation when no empirical source exists for a resource that only occurs off the beaten path of routine data (e.g. ambulance transfer for a rare adverse event). Whichever source, keep an explicit `source` column alongside each `quantity` — reviewers and HTA bodies will ask where a number came from, and a provenance column answers that without you re-digging through notebooks later.

Valuation is where unit costs are joined onto quantities. Treat the unit-cost lookup as its own small reference table and join rather than hard-coding values inline — this is the same discipline as keeping utility decrements as a separate lookup in Chapter 8, and it pays off identically when a unit cost is revised:

```
unit_costs <- tribble(
  ~item,               ~unit_cost_local, ~currency, ~cost_year,
  "xpert_test",         800,             "INR",     2023,
  "fdc_regimen",        450,             "INR",     2023,
  "dot_visit",          150,             "INR",     2023,
  "culture_dst",        1200,            "INR",     2023,
  "second_line_regimen", 3200,           "INR",     2023
)

costed <- resource_use |>
  left_join(unit_costs, by = "item") |>
  mutate(total_cost_local = quantity * unit_cost_local)

costed
```

Category	Resource item	Unit cost	Currency	Price year
Diagnostic	Xpert MTB/RIF test	800	INR	2023
Diagnostic	Culture / DST	1,200	INR	2023
Drug	First-line FDC regimen (per month)	450	INR	2023
Drug	Second-line regimen (per month)	3,200	INR	2023
Programmatic	DOT visit	150	INR	2023

Source: programme costing exercise unit-cost lookup; values are illustrative and should be replaced with local or WHO-CHOICE sourced costs before reporting.

Table 23: Unit-cost reference table for a TB programme resource-use valuation, by resource category and item (local currency, price year 2023) – the conventional format for publishing the unit-cost list underlying a costing or cost-effectiveness analysis

HEOR in practice. Never collapse quantity and unit cost into a single pre-multiplied "total cost" column at the extraction stage. Keeping them separate (as above) is what allows deterministic sensitivity analysis on the unit cost alone, probabilistic sensitivity analysis with a distribution on quantity and cost independently (Chapter 25), and re-valuation under a different country's cost list — none of which are possible once the multiplication has already happened and the inputs are gone.

Common pitfall. A common analyst error is averaging unit costs **before** joining to resource-use quantities when costs come from multiple facility types (e.g. district hospital vs. primary health centre). Average the resulting total costs (or weight by caseload), not the raw unit costs, unless the facility mix is uniform across your resource-use denominator — averaging unit costs first silently assumes equal patient volume at each facility.

9.2 Unit costs and inflation or currency conversion (PPP adjustments)

Unit costs are rarely available in the right currency, the right year, and the right country all at once. Three distinct adjustments are typically needed, and they must be applied in the correct order: inflate to a common price year **within** the source currency first, then convert currency (nominal exchange rate or PPP) second. Reversing the order mixes an inflation index computed in one currency with a conversion factor computed in another and produces a number that looks plausible but is not interpretable.

Inflation to a common price year. Within a single currency, uprate historical unit costs using a price index — a consumer price index (CPI) or, for health-specific costs, a health-sector deflator where available. The arithmetic is simple but easy to get backwards:

```
inflate_cost <- function(cost, cost_year, target_year, cpi) {
  # cpi: named vector or lookup keyed by year, e.g. cpi["2023"], cpi["2024"]
  cost * (cpi[[as.character(target_year)]] / cpi[[as.character(cost_year)]])
}

cpi_india <- c("2019" = 100, "2020" = 106, "2021" = 112,
               "2022" = 121, "2023" = 128, "2024" = 134)

# Example: inflate a 2019 unit cost to 2024 prices
inflate_cost(cost = 500, cost_year = 2019, target_year = 2024, cpi = cpi_india)
```

Currency conversion: nominal vs. PPP. Once costs are in a common price year within their source currency, converting to a reference currency (commonly USD or international dollars for cross-country comparison) requires a choice between the market/nominal exchange rate and a purchasing power parity (PPP) conversion factor. Nominal exchange rates reflect what currency actually trades for; PPP conversion factors reflect relative purchasing power for a comparable basket of goods and services, and are the standard choice when the purpose is comparing real resource burden across countries of different income levels — exactly the comparison an Indian TB-programme analyst benchmarking against WHO or global literature unit costs will usually want. The World Bank publishes PPP conversion factors (and the WHO-CHOICE unit cost database separately provides country-level unit costs for LMICs, already expressed per the WHO-CHOICE methodology) — treat the PPP factor as another lookup joined by year and country, not a constant you hardcode once and forget to update:

```
ppp_factor_inr_per_intl_dollar <- c(
  "2022" = 22.9, "2023" = 23.5, "2024" = 24.1  # illustrative; source from World
Bank ICP data
)

to_intl_dollars <- function(cost_local, year, ppp_lookup) {
  cost_local / ppp_lookup[[as.character(year)]]
}

# Convert the inflated 2024 INR cost to international dollars for cross-country
comparison
cost_2024_inr <- inflate_cost(500, 2019, 2024, cpi_india)
to_intl_dollars(cost_2024_inr, 2024, ppp_factor_inr_per_intl_dollar)
```

Wrapping both steps into one pipeline keeps the costed resource-use table from Section 9.1 fully traceable from source currency/year through to the reporting currency/year:

```
costed_reported <- costed |>
  mutate(
    total_cost_2024_inr = inflate_cost(total_cost_local, cost_year, 2024,
cpi_india),
    total_cost_2024_intl_usd = to_intl_dollars(total_cost_2024_inr, 2024,
                                               ppp_factor_inr_per_intl_dollar)
  )

costed_reported
```

> **Agency perspective.** The NICE reference case, as set out in "NICE health technology evaluations: the manual" (PMG36) — the same document already cited for EQ-5D utility sourcing in Chapter 8 — specifies costs from the NHS and Personal Social Services (PSS) perspective, valued in UK currency at current prices; a NICE-facing submission should never silently substitute a PPP-converted LMIC cost for a missing UK unit cost without flagging it as a structural assumption. Conversely, for HTAIn or WHO-CHOICE-oriented work, PPP conversion is the standard device for situating a local unit cost within international cost-effectiveness thresholds, and PSSRU's **Unit Costs of Health and Social Care** remains the reference source analysts cite when a UK unit cost itself (rather than its currency conversion) is in question.

Report which conversion was used — nominal exchange rate or PPP — and the source and year of both the price index and the conversion factor, directly alongside any cost table in your results output; CHEERS 2022 (Husereau et al., **Value in Health**, 2022) item 15, which requires explicit reporting of currency, price date, and the conversion method applied, specifically requires this disclosure, and it is also what lets another analyst re-run your pipeline with an updated PPP vintage without reverse-engineering your assumptions.

9.3 Cost data features: skewness, zero-inflation, heavy tails

Cost data in a TB programme context — drug costs, hospitalisation costs, outpatient visit costs, diagnostic costs (GeneXpert cartridges, chest X-rays, culture/DST) — share a statistical signature that rhymes with the bounded utility data of Section 8.4 but is mechanistically different. Utilities were bounded on the [0,1] interval with a ceiling spike at 1; costs are bounded only below at 0, have no natural upper ceiling, and the "spike" sits at the **floor**, not a ceiling. Where Section 8.4 needed beta or censored regression to respect a bounded, two-sided support, cost data needs methods that respect a one-sided (non-negative), unbounded-above, heavy-tailed support with an honest mass of exact zeros. Same family problem — "the arithmetic mean and OLS assumptions quietly break down" — different geometry.

Three features recur in almost every patient-level cost dataset you will pull from a TB programme's billing or case-management system:

- **Right-skewness.** Most patients incur modest cost (a few clinic visits, first-line drugs); a minority incur very large cost (MDR-TB admission, surgical intervention, prolonged inpatient stay). The mean is pulled well above the median — often costs are log-normal- or gamma-shaped rather than Gaussian.
- **Zero-inflation.** Some patients genuinely incur zero cost for a resource category over the observation window — no hospitalisation, no missed-dose home visits, no adverse-event management. These are **structural** zeros (the event never happened), not missing data in the Chapter 7 sense. Treating them as missing and imputing a positive value, or dropping them, both bias the estimate; treating them as "just another low value" and fitting a single continuous-only model (e.g. plain log-OLS) is equally wrong because log(0) is undefined and the zero/positive process may have genuinely different covariates than the cost-given-event process.
- **Heavy tails.** Even after conditioning on "cost > 0", the right tail can be heavier than gamma or log-normal would suggest — a handful of catastrophic MDR/XDR-TB cases with ICU stays, surgery, or prolonged second-line regimens dominate the total programme cost even though they are a tiny fraction of the caseload. Total cost estimates are consequently dominated by

a handful of outlier patients, and ordinary standard errors (which assume roughly symmetric, light-tailed residuals) understate uncertainty badly.

Common pitfall. A right-skewed cost variable with a thick right tail is the textbook case where a handful of leverage points can swing a sample mean (and therefore an incremental cost-effectiveness ratio numerator) far more than any outlier in a beta-distributed utility ever could, because the cost support is unbounded above. Never eyeball a cost histogram on the raw scale and move on — always report the proportion of zeros, skewness, and at minimum a trimmed/bootstrapped mean alongside the arithmetic mean before choosing a model.

Difficult terminology: leverage points. A leverage point is a data value that sits far enough out on its own that it pulls a summary statistic — most commonly a mean — disproportionately toward itself, simply because of how extreme or isolated it is relative to the rest of the sample. Picture one patient in your cost dataset who needed a prolonged ICU admission for drug-resistant TB, costing ten times more than anyone else in the sample: even though that patient is a single row among hundreds, their cost alone can drag the sample mean up substantially, because the mean, unlike the median, is sensitive to the exact magnitude of every value, not just its rank. This matters acutely for cost data specifically because, unlike a utility score (bounded between roughly 0 and 1), cost has no upper bound — a single catastrophic case can in principle be arbitrarily large, so there's no ceiling limiting how much damage one leverage point can do to your reported average cost, and therefore to an ICER built on it.

Difficult terminology: bootstrapped estimate. Bootstrapping is a way of figuring out how uncertain a statistic (like a mean) really is, without relying on a formula that assumes the data are normally distributed — which, as this section has just shown, cost data very much are not. The trick: take your own sample of patients, and create a new "pretend" sample of the same size by randomly drawing patients from it **with replacement** (so the same patient can be picked more than once, and some patients may not be picked at all). Calculate the mean of that pretend sample. Repeat this process thousands of times, and you end up with thousands of slightly different means — the spread of that collection of means tells you directly how much uncertainty surrounds your original estimate, without ever having to assume the underlying cost distribution is some neat bell curve. It's conceptually like repeatedly reshuffling and redrawing your own deck of patient records to see how much your answer would wobble if you happened to have drawn a slightly different sample from the same underlying population — a "bootstrapped mean" or "bootstrapped confidence interval" is simply the result of that resampling exercise applied to your actual data.

A quick diagnostic pass on simulated TB-programme-like cost data:

```
library(tidyverse)

set.seed(9340)
n <- 1200

# Simulate a TB cohort: ~30% zero-cost (no qualifying resource use),
# positive costs right-skewed (gamma), with a few heavy-tail outliers.
```

```
zero_flag  <- rbinom(n, 1, prob = 0.30)
base_cost  <- rgamma(n, shape = 1.4, scale = 180)       # typical positive costs
outlier    <- rbinom(n, 1, prob = 0.02) * rgamma(n, shape = 2, scale = 4000)  #
rare catastrophic cases
cost <- ifelse(zero_flag == 1, 0, base_cost + outlier)

tb_costs <- tibble(
  patient_id = seq_len(n),
  regimen    = sample(c("first_line", "mdr_regimen"), n, replace = TRUE, prob =
c(0.85, 0.15)),
  age        = round(rnorm(n, 38, 13)),
  cost       = cost
)

tb_costs |>
  summarise(
    n_patients   = n(),
    pct_zero     = mean(cost == 0) * 100,
    mean_cost    = mean(cost),
    median_cost  = median(cost),
    sd_cost      = sd(cost),
    skewness     = mean((cost - mean(cost))^3) / sd(cost)^3,
    p95          = quantile(cost, 0.95),
    max_cost     = max(cost)
  )
```

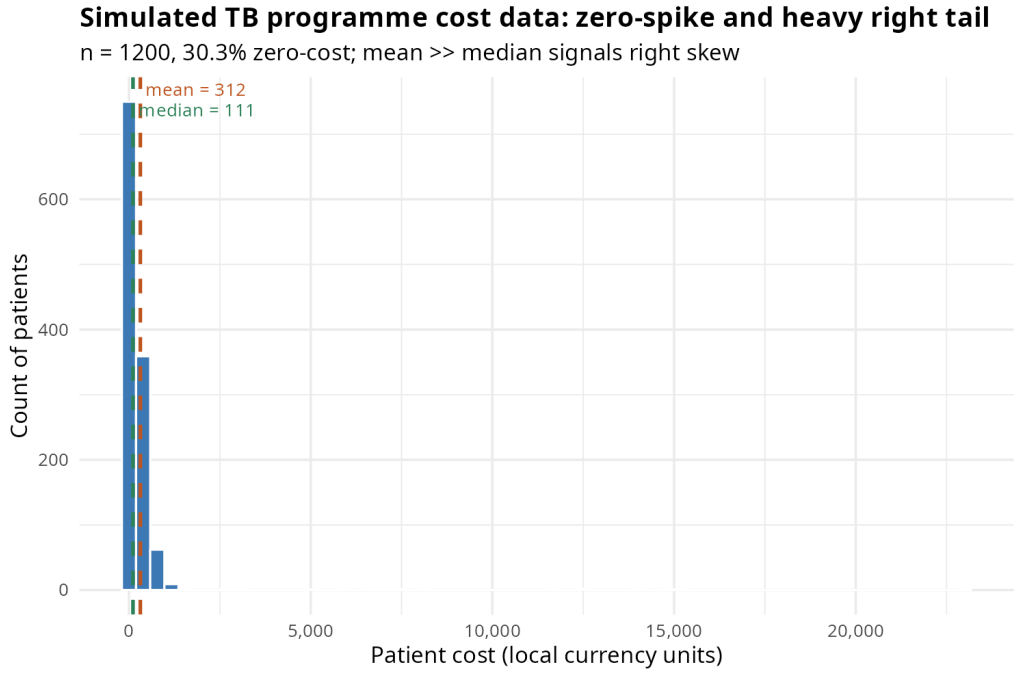


Figure 9: Simulated TB programme cost data (n = 1,200): a visible spike of zero-cost patients at the floor, a heavy right tail driven by the rare catastrophic-cost outliers, and a mean well above the median — the distributional signature that makes ordinary least squares on raw cost a poor default.

On data like this it is routine to see the mean sit two- to four-fold above the median, skewness well above 2, and the top 5% of patients accounting for a disproportionate share of total programme

spend — exactly the pattern that makes "fit OLS on raw cost" a poor default, just as "fit OLS on a 0-1 utility with a spike at 1" was a poor default in Section 8.4.

9.4 Regression models for costs: GLMs, two-part models, gamma models

The methodological toolkit for costs parallels — but does not duplicate — the beta/censored-regression toolkit used for bounded utilities. The organising choice is the same kind of choice: **pick a model whose assumed distribution matches the data's actual support and shape, rather than forcing Gaussian machinery onto data that was never Gaussian.**

Generalised linear models on the cost scale

The single most defensible default for **strictly positive** cost data (no zeros, or zeros handled separately — see two-part models below) is a gamma GLM with a log link, fit directly on the natural cost scale:

```
# Restrict to patients with cost > 0 for the gamma GLM
pos_costs <- tb_costs |> filter(cost > 0)

fit_gamma <- glm(
  cost ~ regimen + age,
  family = Gamma(link = "log"),
  data   = pos_costs
)

summary(fit_gamma)

# Contrast against naive OLS on the same (untransformed) scale
fit_ols <- lm(cost ~ regimen + age, data = pos_costs)

# Compare predicted mean cost for the MDR regimen at mean age
newdat <- tibble(regimen = c("first_line", "mdr_regimen"), age =
mean(pos_costs$age))
predict(fit_gamma, newdata = newdat, type = "response")
predict(fit_ols,   newdata = newdat)
```

The gamma GLM models the **mean** of cost directly (via the log link on the linear predictor, exponentiated back by `type = "response"`), so there is no retransformation step and no smearing correction needed — this is one of its practical advantages over a log-OLS model. Its variance function (variance proportional to the squared mean) also matches right-skewed cost data much better than OLS's constant-variance assumption. An alternative sometimes seen in the literature is `family = gaussian(link = "log")` — a Gaussian GLM with a log link, which also models the mean on the log scale without the smearing problem, but keeps OLS's constant-variance assumption and so under-weights the heavy right tail relative to gamma or inverse-gaussian error structures. Comparing AIC, or inspecting residual-vs-fitted plots on the response scale, across `Gamma(link="log")`, `gaussian(link="log")`, and `inverse.gaussian(link="log")` is a reasonable way to pick among link/variance combinations for a given programme's cost data.

Common pitfall. Never fit `lm(log(cost) ~ ...)` and then report the exponentiated coefficient or the exponentiated predicted log-cost as the mean cost. Exponentiating a prediction from a log-OLS model recovers the median (geometric mean) of the fitted distribution on the original scale, not the arithmetic mean — and it is almost always the arithmetic mean that the economic evaluation needs, because total programme cost and incremental cost-effectiveness ratios are built from **sums**, not medians. If you must work on the log scale, apply **Duan's smearing estimator**: fit the log-OLS model, compute the smearing factor as the mean of the exponentiated residuals, and multiply every back-transformed prediction by that constant before using it in any budget-impact or CEA calculation. This is exactly the same family of retransformation-bias trap as reporting a raw mean on a ceiling-spiked utility scale without acknowledging the spike (Section 8.4) — the fix differs, but the discipline of "respect what the link/transformation actually estimates" is identical.

How to report this.

Parameter	Coefficient (log scale)	Cost ratio	95% CI	p-value
Intercept (first-line, age = 0)	5.92	372	310 – 447	< 0.001
Regimen: MDR vs. first-line	1.35	3.86	3.02 – 4.93	< 0.001
Age (per year)	0.012	1.01	1.00 – 1.02	0.014
Model: `glm(cost ~ regimen + age, family = Gamma(link = "log"))`; cost ratio = exp(coefficient), interpreted as a multiplicative effect on mean cost.				

Table 24: Gamma GLM (log link) results for per-patient TB cost by regimen and age, among patients with cost > 0 – the conventional format for reporting cost-ratio regression results (coefficient, exponentiated cost ratio, 95% CI, p-value) rather than raw log-scale coefficients alone

Two-part (hurdle-style) models for zero-inflated costs

When a non-trivial share of patients have genuinely zero cost for the resource category of interest, a single continuous-distribution model (gamma, log-normal, or otherwise) cannot represent the probability mass at exactly zero. The standard solution is a **two-part model**: part one is a binary model for "any cost incurred at all", part two is a continuous model for cost **conditional on** being positive. The two parts can use different covariates and different functional forms, and together they decompose expected cost cleanly as

$$E(\text{cost} \mid \text{x}) = P(\text{cost} > 0 \mid \text{x}) \times E(\text{cost} \mid \text{cost} > 0, \text{x})$$

A manual implementation is transparent and easy to audit:

```
tb_costs <- tb_costs |> mutate(any_cost = as.integer(cost > 0))

# Part 1: probability of incurring any cost at all
part1 <- glm(any_cost ~ regimen + age, family = binomial(link = "logit"), data =
tb_costs)
```

```r
# Part 2: conditional mean cost, among those with cost > 0
part2 <- glm(cost ~ regimen + age, family = Gamma(link = "log"),
             data = tb_costs |> filter(cost > 0))

# Combine: predicted mean cost = P(cost>0) * E[cost | cost>0]
newdat <- newdat |> mutate(
  p_positive   = predict(part1, newdata = newdat, type = "response"),
  cond_mean    = predict(part2, newdata = newdat, type = "response"),
  pred_mean    = p_positive * cond_mean
)
newdat
```

The `pscl` package wraps similar two-part/mixture logic via `pscl::hurdle()`, which was built primarily for zero-inflated **count** data (e.g. number of outpatient visits) rather than continuous cost — it is a natural fit for resource-use **counts** that you then cost out, more than for cost amounts directly. For continuous zero-inflated costs specifically, the manual two-part construction above (or the dedicated `twopartm` package, whose `tpm()` function automates exactly this binomial-then-gamma/log-normal decomposition with built-in marginal-effect and bootstrap-SE support — Belotti, Deb, Manning & Norton 2015) is generally the more appropriate and more transparent tool; check `?twopartm::tpm` for the current argument names before relying on a specific call signature in a submission.

> **Common pitfall.** Fitting a single gamma or log-normal GLM on cost data that actually contains a meaningful fraction of zeros is a common error: zeros either get dropped (silently shrinking and biasing the analysis sample and discarding real information about who incurs no cost at all) or the gamma/log-normal density is evaluated at zero and either errors out or is coerced via an ad hoc offset (e.g. adding 1 to all costs before logging), which distorts the entire distribution's shape and is difficult to defend to a reviewer. Check `mean(cost == 0)` before choosing between a single-part GLM and a two-part model — this is the cost-data analogue of checking the proportion of ceiling observations before choosing between OLS and a beta/Tobit model for utilities in Section 8.4.

Smooths and flexible mean functions

When the relationship between a continuous covariate (age, time since treatment initiation, baseline bacterial load) and cost is not well captured by a linear or log-linear term, `mgcv::gam()` extends the gamma-GLM framework with penalised smooths:

```r
library(mgcv)

fit_gam <- gam(
  cost ~ regimen + s(age),
  family = Gamma(link = "log"),
  data   = pos_costs
)
summary(fit_gam)
plot(fit_gam, pages = 1)
```

This keeps the same mean-and-variance structure as the gamma GLM above while relaxing the linearity assumption on `age` — useful when, for example, cost rises non-linearly with age due to

comorbidity burden in older TB patients, and a straight line understates cost at the extremes of the age range.

Agency perspective. Mihaylova et al. (2011), "Review of statistical methods for analysing healthcare resources and costs" (**Health Economics**), is the standard reference the field cites for exactly this decision tree: it recommends against transforming cost data purely to force normality, favours GLMs that model the mean directly on the natural scale (gamma, or other members of the exponential family matched to the data's shape), and discusses two-part models explicitly as the appropriate tool once a meaningful zero-cost subgroup is present. The ISPOR/professional costing literature building on this guidance, together with CHEERS 2022′s reporting checklist (which requires that the chosen cost-analysis method and its justification be stated explicitly, not merely that "costs were analysed in R"), is the standard a submission reviewer will hold your cost model to — document the skewness/zero-inflation diagnostics from Section 9.3 and the resulting model choice together, not the model choice alone.

9.5 Cost-of-illness and micro-costing studies

Everything in Sections 9.1-9.4 assumed you already had a cost per patient per resource item sitting in a tidy data frame. Cost-of-illness (COI) and micro-costing studies are where those numbers actually get built, and for a TB-programme analyst this is often the first deliverable a ministry of health or donor asks for, independent of any trial-based CEA: "what does a TB case cost the health system and the patient, start to finish?"

COI studies come in two flavours, and conflating them is a common critique reviewers raise.

- **Prevalence-based** COI sums all costs incurred in a defined period (e.g. a calendar year) across everyone with the condition, regardless of when they were diagnosed. This answers "what did TB cost the system last year?" and is the natural frame for annual budget-impact reporting.
- **Incidence-based** COI follows a cohort of newly diagnosed patients and sums costs over the full episode (diagnosis through cure, default, or death). This answers "what does one TB episode cost?" and is what feeds a per-patient cost input into a Markov or partitioned-survival CEA model built on the Chapter 4 tibble structure.

The second axis is **costing direction**, independent of the first:

- **Top-down** costing allocates aggregate facility or programme expenditure (budgets, cost centres) down to patients or services using an allocation key — bed-days, visit counts, staff-time proportions. It is fast and uses routinely available financial data, but hides within-patient variation and can smear fixed costs unrealistically across light and heavy resource users.
- **Bottom-up (micro-costing)** traces individual resource items consumed by each patient — every sputum smear, every drug-dispensing visit, every hospital night — and prices each one, typically against the national/WHO-CHOICE unit-cost list built in Section 9.2. It is labour-intensive but gives the variance structure you need for probabilistic sensitivity analysis (Chapter 25) and is the reference standard when heterogeneity across patients (e.g. drug-susceptible vs MDR-TB) matters for the decision.

HEOR in practice. In practice, most programmatic TB costing is a **hybrid**: drug and diagnostic-test costs are micro-costed (bottom-up, against a WHO-CHOICE or national unit-cost list), while overheads — facility utilities, programme management, supervision — are allocated top-down from the TB programme budget using a step-down allocation. Keep the two cost components in separate columns (`cost_microcosted`, `cost_overhead_allocated`) in your tibble rather than pre-summing them; a reviewer or later re-analysis will want to vary the overhead allocation assumption without re-deriving the whole unit-cost exercise.

Difficult terminology: step-down allocation. Step-down allocation is a standard hospital/programme cost-accounting technique for spreading a shared overhead cost (say, facility electricity, building maintenance, or central programme management) across the individual services or departments that benefit from it, when there's no direct per-patient bill for that overhead to point to. It works in a deliberate sequence, hence "step-down": you start with the department that provides services to the most other departments but receives services from none in return (e.g. general administration), and allocate its cost out to everyone else based on a sensible driver (floor space, staff headcount, visit counts). Then you move to the next department in the sequence and allocate **its** cost (now including the share it just received) onward to the remaining departments — and so on, one step at a time, until every shared cost has been pushed down to the clinical service lines that actually treat patients. It's the accounting equivalent of a multi-stage reimbursement cascade: each step passes a cost share further down the chain rather than trying to divide every overhead cost among every department in one single calculation.

Agency perspective. Tan-Torres Edejer et al. (2003), **Making Choices in Health: WHO Guide to Cost-Effectiveness Analysis** — the foundational WHO-CHOICE methodological reference — explicitly supports the hybrid pattern above for LMIC provider-cost studies: capital and overhead items valued top-down via facility-level allocation, variable/consumable items (drugs, tests, staff time per visit) valued bottom-up via micro-costing against the national unit-cost list, with WHO-CHOICE's published regional unit-cost database as the standard fallback when a national list is incomplete. When you deviate from this (e.g. top-down only, for a rapid budget-impact ask), document that choice explicitly — it is a direct driver of how much cost variance the model can later show, and ISPOR's Good Research Practices for cost-of-illness and micro-costing studies expects that deviation to be stated, not left implicit.

A minimal incidence-based micro-costing roll-up, continuing the per-patient resource-item structure from Section 9.1-9.2:

```
library(dplyr)

# resource_items: one row per patient per resource event
# patient_id, episode_phase ("diagnosis","treatment","hospitalisation"),
# resource_item, quantity, unit_cost (from the Section 9.2 unit-cost table)

episode_cost <- resource_items |>
  mutate(item_cost = quantity * unit_cost) |>
  group_by(patient_id, episode_phase) |>
```

```
  summarise(phase_cost = sum(item_cost), .groups = "drop") |>
  tidyr::pivot_wider(
    names_from  = episode_phase,
    values_from = phase_cost,
    values_fill = 0
  ) |>
  mutate(total_episode_cost = diagnosis + treatment + hospitalisation)

episode_cost |>
  summarise(
    mean_cost   = mean(total_episode_cost),
    sd_cost     = sd(total_episode_cost),
    n_patients  = n()
  )
```

This gives a defensible mean cost per TB episode **provided every patient in `resource_items` completed follow-up**. In a real programme cohort or trial, many will not — and that is exactly the problem Section 9.6 addresses.

9.6 Handling censored cost data

The gamma GLM and two-part models of Section 9.4 assumed every patient had a complete, fully observed total cost. In longitudinal cost studies — a TB treatment trial following patients for 6-24 months, or a programmatic cohort tracked to cure/default/death — that assumption routinely fails: patients are lost to follow-up, transfer out, or the study closes before their course of treatment (and its costs) is finished. Their total cost is only **partially observed**, and critically, the amount observed is correlated with how long they survived or stayed in care. This is the same right-censoring you will formalise in Chapter 10, but applied to a **cumulative, time-accruing** outcome rather than to a single event time, which makes it behave differently from a textbook Kaplan-Meier problem.

> **HEOR in practice.** Before reaching for any estimator, check why patients are censored. If censoring is administrative (study end date, same calendar cutoff for everyone, unrelated to prognosis) the methods below are well justified. If patients drop out **because** they are doing badly (or unusually well) and their costs would have been systematically different, you have informative censoring that no reweighting scheme fully fixes — flag it as a limitation rather than silently correcting it away.

Why naive averaging is biased. The obvious shortcut — take the mean of whatever total cost was observed for each patient, ignoring who was censored — is biased, and predictably so. Costs accrue over time, so a patient censored early has had less opportunity to accumulate cost than one followed to completion. Dropping censored patients entirely (complete-case analysis) is worse: it systematically discards exactly the patients with the shortest observed survival/follow-up, biasing the mean cost **upward** among survivors and giving a distorted picture of total programme cost. Simply summing observed costs without any time adjustment conflates "this patient was cheap" with "this patient was only followed for three weeks."

The Lin et al. (1997) partitioned estimator. Lin, Feuer, Etzioni and Wax (**Biometrics**, 1997) solved this by partitioning the total follow-up period into fixed intervals and estimating mean cost accrued in **each interval**, weighting each interval's mean by the probability of still being under

observation (uncensored) at that interval, estimated via the Kaplan-Meier survival function of the censoring time. Summing the interval contributions gives a consistent estimator of mean total cost to a fixed time horizon, using only costs actually observed plus the KM-estimated probability of being followed long enough to have observed them — no extrapolation of unobserved costs is required.

Inverse-probability-of-censoring weighting (IPCW). A closely related and more flexible approach reweights each fully-observed (or observed-to-time-t) patient's cost by the inverse of their probability of remaining uncensored up to their observed time, using the Kaplan-Meier estimate of the **censoring** distribution (i.e. KM fitted with event/censoring indicators flipped). Patients who stayed in observation longer than typical are up-weighted to stand in for the similar patients who were censored early. `survival::survfit()` already gives you everything needed for the KM step; the IPCW cost estimator is then a straightforward weighted mean, no specialised cost-econometrics package required.

```
library(survival)
library(dplyr)

# cohort: one row per patient
#   time_obs   - time followed (to cost cut-off, death, loss-to-follow-up)
#   cost_obs   - total cost accrued up to time_obs
#   event      - 1 = fully observed / died (not censored), 0 = censored

# Step 1: KM estimate of the CENSORING distribution -- flip the event indicator
cens_fit <- survfit(Surv(time_obs, 1 - event) ~ 1, data = cohort)

# Step 2: censoring survival probability just before each patient's own time
G_at <- summary(cens_fit, times = cohort[["time_obs"]], extend = TRUE)[["surv"]]

cohort <- cohort |>
  mutate(
    G_hat   = pmax(G_at, 0.01),         # guard against near-zero denominators
    ipcw_wt = event / G_hat             # zero weight for censored patients
  )

ipcw_mean_cost <- with(cohort, sum(ipcw_wt * cost_obs) / sum(ipcw_wt))
ipcw_mean_cost
```

Only patients with a fully observed episode (`event == 1`) contribute directly, but each is reweighted by the inverse probability that someone with their observed follow-up time would still be uncensored — correcting for the fact that easily-followed (often lower-cost, faster-resolving) patients are over-represented among the uncensored.

> **Agency perspective.** CHEERS 2022 (Husereau et al., **Value in Health**, 2022) item 24 requires reporting of the time horizon and how costs and outcomes were adjusted, and its elaboration explicitly flags that studies with differential follow-up or loss-to-follow-up must describe how incomplete cost data were handled — a bare "missing data were excluded" does not satisfy the checklist. When you use Lin's partitioned estimator or IPCW, name the method and cite it in the costing section of your write-up; reviewers increasingly check for this specifically.

Method	Bias under informative follow-up	Requires	Principal limitation
Naive mean of observed cost	Biased downward: shorter-followed (often sicker/costlier) patients contribute less accrued cost	Complete or incomplete cost field, no censoring indicator used	Ignores that observed cost is mechanically capped by observed time, not just by resource use
Complete-case (drop censored)	Biased upward: discards exactly the shortest-followed patients, over-representing those followed to completion	Censoring indicator to identify which rows to drop	Discards real information and shrinks the effective sample size
Lin et al. (1997) partitioned estimator	Consistent under non-informative (administrative) censoring	Fixed time intervals; KM estimate of the censoring distribution per interval	Interval width choice affects precision; still assumes non-informative censoring
Inverse-probability-of-censoring weighting (IPCW)	Consistent under non-informative censoring; more flexible weighting than Lin's fixed intervals	KM fit on the flipped (censoring) event indicator; stable weights away from zero	Weights become unstable when few patients remain uncensored at longer times

Recommendation: report which estimator was used and why, per CHEERS 2022 item 24; neither estimator corrects for informative censoring, which must instead be flagged as a limitation (§9.6).

Table 25: Comparison of censored-cost-handling methods (naive mean of observed cost, Lin et al.'s partitioned Kaplan-Meier-weighted estimator, inverse-probability-of-censoring weighting) by bias direction, data requirement and principal limitation – the summary table a costing section should present when justifying the chosen estimator for incomplete follow-up cost data

Both estimators depend on the same underlying machine: a Kaplan-Meier estimate of a time-to-event distribution, fitted with `survival::survfit()`. That machinery, and the broader question of how censoring and truncation affect survival-time estimation and extrapolation, is the full subject of Chapter 10, Survival Analysis for Economic Evaluation — where Kaplan-Meier curves, parametric survival models, and their role in extrapolating costs and outcomes beyond trial follow-up will be developed in depth, building directly on the censoring intuition established here.

PART III

Statistical Methods for Evidence Synthesis and Analysis

Chapter 10. Survival Analysis for Economic Evaluation

Learning objectives

- Fit and plot Kaplan-Meier estimates and life tables using `survival`, `ggsurvfit` and `survminer` (§10.1)
- Fit the six standard parametric survival distributions with `flexsurv::flexsurvreg()` (§10.2)
- Fit flexible, spline-based Royston-Parmar models with `flexsurv::flexsurvspline()` (§10.3)
- Fit mixture and non-mixture cure models with `flexsurvcure::flexsurvcure()` (§10.4)
- Build piecewise and landmark extrapolations that combine empirical and parametric survival (§10.5)
- Select an extrapolation model using AIC/BIC, visual fit, and clinical plausibility together (§10.6)
- Test the proportional hazards assumption with `survival::cox.zph()` and choose a non-PH alternative where needed (§10.7)
- Build covariate-adjusted and treatment-effect-specific survival models (§10.8)
- Extrapolate survival using external data, background mortality, and expert elicitation (§10.9)
- Model competing risks and multi-state pathways with `mstate` and `flexsurv` (§10.10)
- Reconstruct individual patient data from a published Kaplan-Meier curve with `IPDfromKM` (§10.11)
- Report survival extrapolation in line with NICE DSU TSD 14 and TSD 21 (§10.12)

Chapter 9 closed by introducing `survival::survfit()` and `Surv()` for one narrow purpose: building inverse-probability-of-censoring weights for censored cost data. This chapter comes back to that same machinery and builds it out properly. Here is the roadmap: start with the model-free Kaplan-Meier curve (§10.1), move to fitting a parametric curve so you can extrapolate beyond the trial's follow-up (§10.2-§10.6), check and handle cases where the simple "constant hazard ratio" assumption breaks down (§10.7-§10.9), extend to pathways with more than one possible outcome (§10.10-§10.11), and finish with exactly what a submission needs to report to satisfy NICE's own written guidance on this topic (§10.12). Each section builds on the last, so read them in order the first time through.

> **Difficult terminology: hazard function / hazard rate.** The hazard at any given moment is the **instantaneous** risk of the event of interest (death, relapse, disease progression) happening right now, given that the patient has made it this far without it happening yet. This is the single most important idea in this entire chapter — nearly every model, curve, and diagnostic from here to §10.12 is built directly on top of it — so it's worth pinning down precisely, because it answers a genuinely different question from the one a simple survival probability answers. A survival probability (what the Kaplan-Meier curve plots) answers "what fraction of patients starting out are still alive/event-free by month 24?" — a cumulative, backward-

looking summary over the whole period so far. The hazard, by contrast, answers "of the patients who have survived to this exact moment, what is their risk of the event happening in the next instant?" — a forward-looking, momentary rate, re-calculated continuously as time moves on. The standard analogy is a car's speedometer versus its odometer: the odometer (like cumulative survival) tells you the total distance travelled so far; the speedometer (like the hazard) tells you how fast you are going **right now**, which can rise, fall, or stay flat completely independently of how far you've already travelled. A hazard that rises over time means risk is accelerating for those still at risk (e.g. a disease that progressively weakens patients the longer they have it); a falling hazard means the remaining survivors are an increasingly resilient group whose moment-to-moment risk is easing (sometimes seen after successful surgery, once the immediate post-operative risk window has passed); a constant hazard means the risk per instant never changes no matter how long a patient has already survived — "no memory" of how long they've made it so far, like a light bulb that is no more or less likely to fail today whether it's brand new or has been running for a year (this is exactly what the exponential distribution below assumes). Every parametric distribution in §10.2 is really just a different story about how this speedometer-needle for risk moves over time, and the proportional hazards assumption central to Cox models and §10.7 is a claim specifically about the **ratio** of two groups' speedometer readings staying constant, even as both needles independently rise or fall together over time.

10.1 Kaplan-Meier and life tables

You already met `survival::Surv()` and `survival::survfit()` in Chapter 9, where we used them to build inverse-probability-of-censoring weights for cost data. Here we return to the same two functions for their original purpose: describing the survival experience of a cohort, non-parametrically, as the foundation everything else in this chapter builds on.

The Kaplan-Meier (KM) estimator is the step-function analogue of a life table — at each event time it recalculates the proportion of the at-risk set that survives past that instant, and multiplies these conditional probabilities together. For a TB-programme analyst this will feel familiar: it is conceptually the same bookkeeping as a programme cohort analysis, just computed at the resolution of individual event times rather than fixed intervals (6-month, 12-month cohort outcomes).

```
library(survival)

fit <- survfit(Surv(time, status) ~ arm, data = trial_df)
summary(fit)
```

`time` is time-on-study (months, say), and `status` is the usual 1 = event, 0 = censored coding. The `~ arm` term stratifies the KM curve by treatment arm — essential in HEOR work, since the whole point is usually to compare extrapolated survival between a new intervention and a comparator.

Life tables (actuarial estimators) are the discrete-time cousin of the KM estimator — they group events into fixed intervals (e.g. each year) rather than using exact event times. `survival::survfit()` will produce an actuarial-style table if you pre-aggregate your data into interval counts, but in practice almost all HEOR survival work in R uses the exact-time KM estimator rather than classical life tables, because source trial data are individual patient data (IPD),

not grouped registry counts. We mention life tables mainly so you recognise the term when it appears in NICE or HTA submission templates referring to older epidemiological methods.

For plotting, two tidyverse-friendly packages dominate: `ggsurvfit` and `survminer`.

```
library(ggsurvfit)

survfit2(Surv(time, status) ~ arm, data = trial_df) |>
  ggsurvfit() +
  add_confidence_interval() +
  add_risktable()
```

`ggsurvfit::survfit2()` is a thin wrapper around `survival::survfit()` that keeps the formula call bundled with the plotting pipe, and `add_risktable()` gives you the number-at-risk row underneath the x-axis that every HTA reviewer expects to see.

```
library(survminer)

ggsurvplot(fit, data = trial_df, risk.table = TRUE,
           conf.int = TRUE, pval = TRUE)
```

`survminer::ggsurvplot()` is the older, more heavily parameterised alternative — it does more in one call (log-rank p-value annotation, risk table, median survival lines) at the cost of a less ggplot2-native object if you want to customise further downstream. Either is defensible in a submission; pick one and be consistent across your analysis.

> **HEOR in practice.** Every HTA economic model that extrapolates survival should open with the observed KM curve, full stop — regulators and HTA reviewers (NICE DSU TSD 14, ICER reference case) expect to see the "trial period" fit visually overlaid with whatever parametric curve you eventually choose to extrapolate with, so that the point of divergence between observed and modelled survival is inspectable, not asserted.

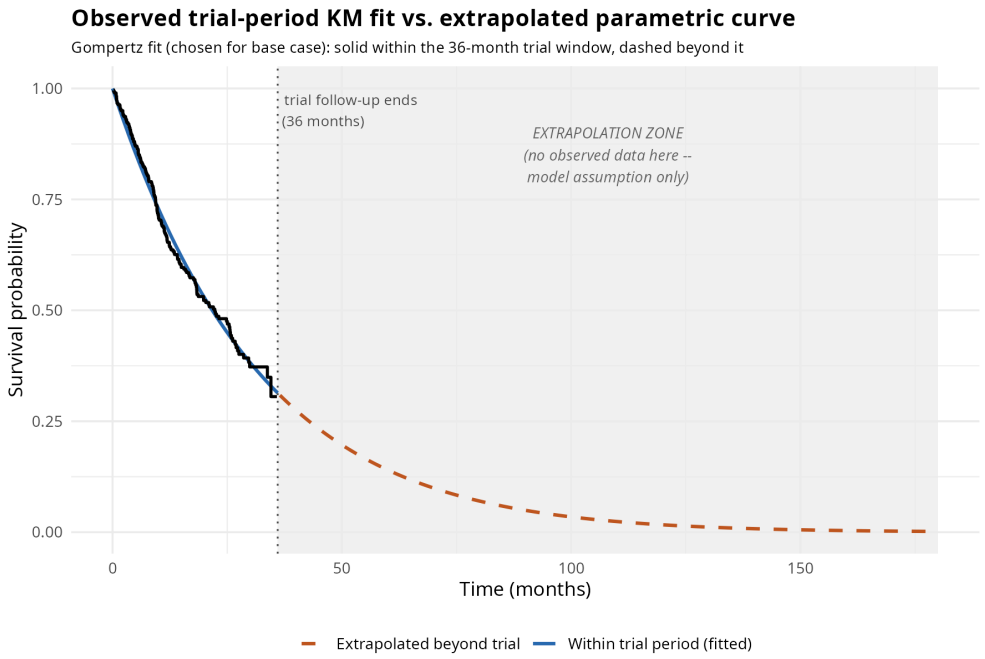


Figure 10: Exactly the diagnostic the practice note above asks for: the observed Kaplan-Meier curve (black) against a chosen parametric fit (here, Gompertz), solid where it is fitted to real trial-period data and dashed where it is pure extrapolation beyond the 36-month follow-up window. The vertical dotted line marks the trial's end — everything to its right is a model assumption, not an observation, and should be labelled as such in any submission.

Common pitfall. `survfit()` silently treats anyone not explicitly flagged `status == 1` as censored. If your event indicator was recoded upstream (e.g. a SQL extract where dead/alive was stored as `'Y'/'N'` strings and someone cast it to numeric without checking `0/1` directions), you can get a KM curve that is the mirror image of reality and it will not throw an error — always sanity-check with `table(trial_df$status)` and a known-rate benchmark before trusting the curve.

10.2 Parametric models

The KM curve only describes survival for as long as you have follow-up data. For a cost-effectiveness model you almost always need survival projected over a lifetime horizon, far beyond the trial's observation window — and that requires fitting a parametric distribution whose tail behaviour you can defend analytically, rather than just eyeballing where the KM curve seems to be heading.

The `flexsurv` package is the standard R workhorse for this. `flexsurv::flexsurvreg()` has the same `Surv(time, status) ~ ...` formula interface you already know, plus a `dist =` argument selecting the parametric family:

```
library(flexsurv)

fit_exp      <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
"exp")
fit_weibull  <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
```

```
"weibull")
fit_gompertz <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
"gompertz")
fit_llogis   <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
"llogis")
fit_lnorm    <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
"lnorm")
fit_gengamma <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
"gengamma")
```

Each distribution encodes a different assumption about the hazard shape over time, which is really the thing you are choosing, not the distribution name itself:

- **Exponential** (`"exp"`) — constant hazard. One parameter, almost never plausible for a chronic disease but a useful lower-bound sanity check.
- **Weibull** (`"weibull"`) — monotonically increasing or decreasing hazard, shape fixed by one parameter. Common for mortality that worsens steadily with age or disease progression.
- **Gompertz** (`"gompertz"`) — hazard that increases (or decreases) exponentially with time; a long-standing favourite for all-cause general-population mortality extrapolation.
- **Log-logistic** (`"llogis"`) — hazard can rise then fall (a hump), useful when early risk is elevated and later risk plateaus or drops.
- **Log-normal** (`"lnorm"`) — similar hump-shaped hazard behaviour to log-logistic, fitted on the log-time scale.
- **Generalised gamma** (`"gengamma"`) — a flexible three-parameter family that nests exponential, Weibull and log-normal as special cases; useful as an umbrella comparison model, at the cost of being harder to interpret and sometimes unstable to fit with sparse data.

Comparing fits is done the way you would compare any likelihood-based R models — AIC/BIC plus a visual check:

```
AIC(fit_exp, fit_weibull, fit_gompertz, fit_llogis, fit_lnorm, fit_gengamma)

plot(fit_gompertz, main = "Gompertz fit vs KM")
```

`plot()` on a `flexsurvreg` object automatically overlays the fitted parametric curve against the KM estimate, which is the single most useful diagnostic in this whole section.

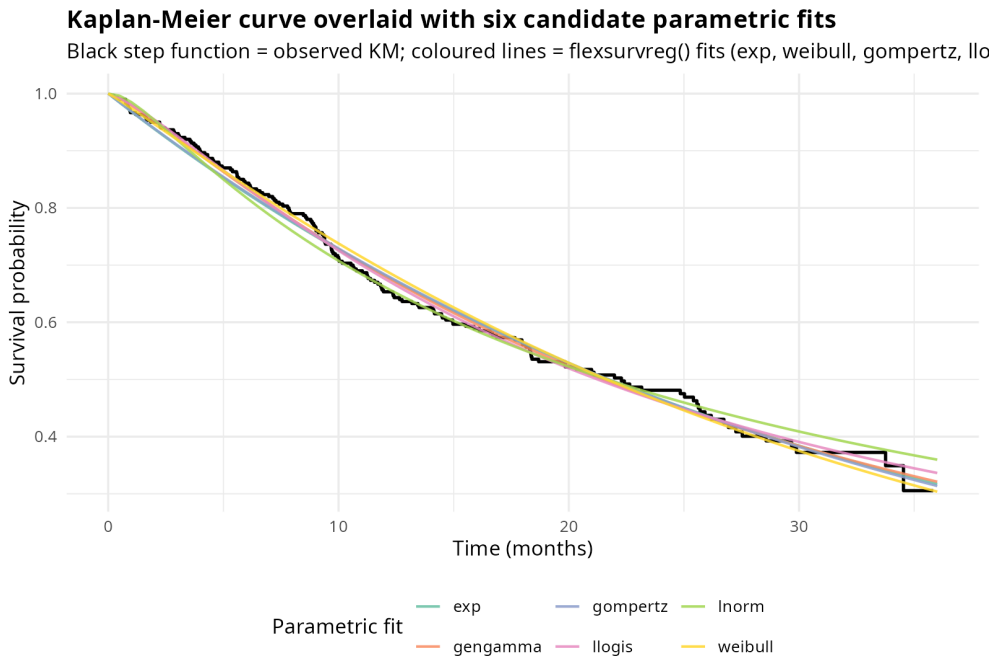


Figure 11: The observed Kaplan-Meier curve (black step function) overlaid with all six standard parametric families fitted via `flexsurvreg()`. Within the trial's follow-up window the fits visually cluster close to the KM curve and to each other — the divergence that matters for extrapolation only becomes visible once the curves are projected beyond the observed data (see Figure 15).

HEOR in practice. NICE DSU TSD 14 and most HTA bodies expect you to report **all six** standard parametric fits (or at minimum exponential, Weibull, Gompertz, log-logistic, log-normal, generalised gamma) with AIC/BIC tabulated side by side, plus visual and clinical-plausibility justification for whichever one is chosen for the base case — "it had the lowest AIC" is necessary but not sufficient; a model with slightly worse AIC that produces a clinically plausible long-term hazard is often preferred over one that fits the trial tail well but extrapolates implausibly (e.g. a cure-like plateau with no biological rationale).

Common pitfall. AIC/BIC only tell you about fit **within the observed data range** — two distributions can have near-identical AIC on a trial with two years of follow-up and diverge wildly once extrapolated to twenty years. Always plot the extrapolated curve out to your full model time horizon (`summary(fit, t = seq(0, horizon, by = ...))`) before trusting an AIC ranking for anything beyond the trial period.

10.3 Flexible and spline-based models

The six standard parametric families in §10.2 each assume a fixed, simple hazard shape. Real disease hazards — especially in conditions with changing treatment effect over time, or where a trial's hazard shape genuinely does something unusual (e.g. a drop-off after a defined treatment-stopping point) — can need more flexibility than any single-shape parametric curve offers without overfitting the whole dataset to force it.

The Royston-Parmar spline-based survival model, implemented in R as `flexsurv::flexsurvspline()`, solves this by modelling a spline on the log-cumulative-hazard (or related) scale, with the number of "knots" controlling flexibility:

```
library(flexsurv)

fit_spline0 <- flexsurvspline(Surv(time, status) ~ arm, data = trial_df, k = 0)
fit_spline1 <- flexsurvspline(Surv(time, status) ~ arm, data = trial_df, k = 1)
fit_spline2 <- flexsurvspline(Surv(time, status) ~ arm, data = trial_df, k = 2)
```

`k = 0` (zero internal knots) collapses to one of the standard parametric families depending on the `scale =` argument (`"hazard"`, `"odds"` or `"normal"` — defaulting to the Weibull-equivalent on the hazard scale), so it is a useful baseline before adding flexibility. Each additional knot (`k = 1`, `k = 2`, …) allows an extra inflection in the hazard shape, at the cost of an extra parameter to estimate and interpret.

```
AIC(fit_spline0, fit_spline1, fit_spline2, fit_gompertz, fit_gengamma)
plot(fit_spline1, main = "1-knot spline vs KM")
```

Because splines are piecewise and fit the observed data very closely, they are excellent at reproducing the trial-period KM curve — but that closeness is exactly why extrapolation beyond the data needs extra scrutiny: a spline has no built-in assumption about what the hazard does once you run out of data, so beyond the last knot it extrapolates essentially linearly on whatever scale you chose, which can produce hazards that neither clinicians nor biology would endorse.

HEOR in practice. Spline models are best used as a **flexibility check** on your chosen standard parametric model, not usually as the base-case extrapolation itself, unless you have strong external long-term data (registry, national life tables) to anchor the tail. A common, reviewer-friendly workflow: fit `flexsurvspline()` with one or two knots, confirm it tracks the KM curve no better than your chosen standard parametric fit **within** the trial period, and if it does track meaningfully better, investigate why (treatment-stopping rule? crossover? subgroup heterogeneity?) rather than simply swapping in the spline as the new base case.

Common pitfall. More knots will always reduce AIC within the observed data — splines are flexible enough to chase noise, especially in TB-programme-style IPD with modest sample sizes and a long tail of sparse late-follow-up events. Treat any spline with `k >= 2` on anything less than a few hundred events as a plausible overfit until you've checked that the extra knot survives a visual bootstrap or sensitivity check, not just an AIC improvement.

How to report this.

Knots (k)	Parameters	−2LogLik	AIC	BIC	Notes
0 (Weibull-equivalent)	2	799.1	803.1	808.5	Baseline; equivalent to standard Weibull on the hazard scale
1	3	793.4	799.4	807.1	Captures an early hazard bend not visible in k = 0
2	4	791.0	799.0	809.0	Marginal AIC gain over k = 1; extra knot not clinically motivated
3	5	790.1	800.1	812.3	BIC increases versus k = 1-2: additional flexibility not supported by sample size
Selected: k = 1 for the base case – lowest BIC and the smallest model whose extra flexibility tracks a plausible early hazard change; k >= 2 shows the AIC-chasing pattern flagged in the pitfall above.					

Table 26: How to report this: Royston-Parmar spline knot-selection comparison (`flexsurvspline()`, hazard scale) across k = 0-3 internal knots, with AIC/BIC and the within-trial-period fit improvement each extra knot buys – the format for justifying a chosen knot count against TSD 21's expectation that knot selection be reported, not merely asserted.

10.4 Mixture-cure and non-mixture cure models

In plain terms first: every model in §10.2 assumes that, eventually, everyone has the event — the survival curve always keeps falling toward zero, however slowly. But some conditions genuinely leave a fraction of patients who are, in effect, "cured": they never relapse, no matter how long you follow them. If that is true of your data, forcing an ordinary Weibull or Gompertz curve onto it is a category error — the curve will keep sliding downward long after the real data has flattened out, systematically understating long-term survival. A **cure model** fixes this by explicitly fitting two things at once: the proportion who are cured, and a separate survival curve just for the ones who are not.

Every parametric curve fitted in §10.2 — Weibull, Gompertz, log-logistic, generalised gamma — assumes that, given enough follow-up, every patient in the model eventually experiences the event. For many TB treatment outcomes that assumption is fine: everyone who survives active disease is still at risk of relapse or death indefinitely. But for some oncology indications, and arguably for a "treatment-success" endpoint sustained well beyond the biological window for TB relapse, a meaningful fraction of patients may never experience the event at all — they are statistically "cured." Ordinary parametric models forced to fit such data either underestimate the plateau (the

Weibull tail keeps falling) or overestimate it (the log-logistic tail flattens for the wrong reason, as a pure curve-fitting artefact rather than a clinical one).

Cure models address this directly by estimating a **cure fraction** π — the proportion of the cohort who will never experience the event — alongside a standard parametric survival distribution for the "uncured" subgroup. The `flexsurvcure` package extends the `flexsurv` machinery from §10.2-§10.3 with exactly this structure, via `flexsurvcure::flexsurvcure()`.

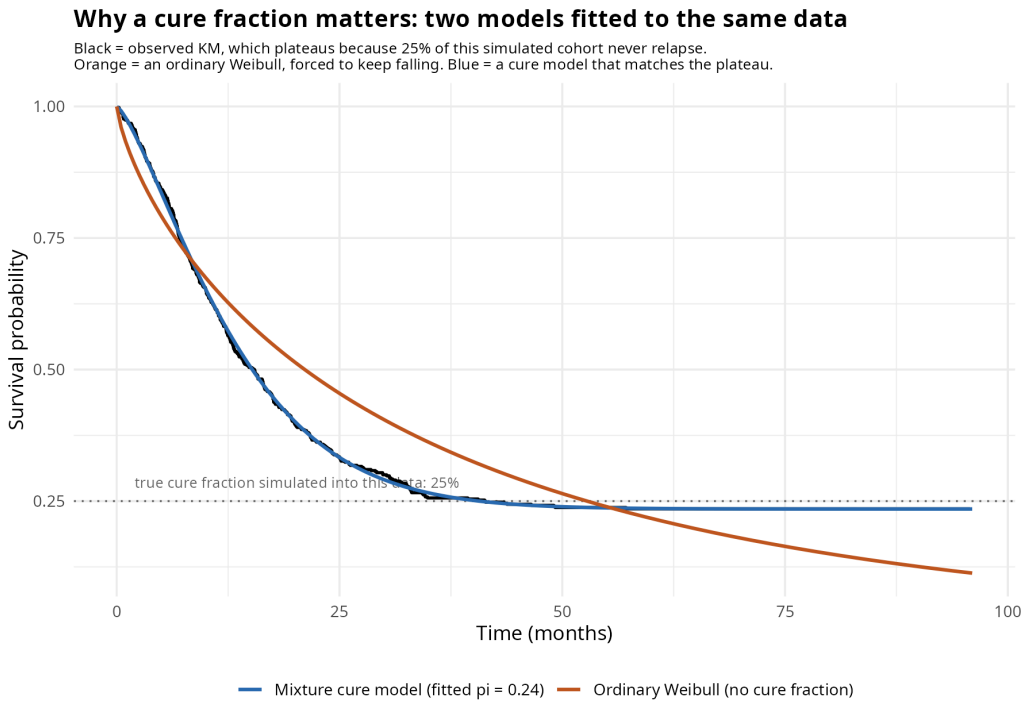


Figure 12: A simulated cohort where 25% of patients are genuinely cured. The observed Kaplan-Meier curve (black) correctly plateaus near 0.25 instead of continuing to fall. An ordinary Weibull (orange), which has no way to represent a cure fraction, keeps sliding downward and badly understates long-term survival. A mixture cure model (blue) matches the plateau because it estimates the cure fraction directly — here recovering $\pi \approx 0.24$, close to the true simulated value of 0.25.

Two structurally different cure model families are supported, selected by the `mixture` argument:

```
library(flexsurvcure)
library(flexsurv)
library(survival)

# Mixture cure model: S(t) = pi + (1 - pi) * S_u(t)
fit_mix <- flexsurvcure(
  Surv(time, status) ~ 1,
  data = tb_followup,
  dist = "weibull",
  mixture = TRUE
)

# Non-mixture cure model: S(t) = pi ^ (1 - S_u(t))
fit_nmix <- flexsurvcure(
  Surv(time, status) ~ 1,
  data = tb_followup,
```

```
  dist = "weibull",
  mixture = FALSE
)

fit_mix
```

In the **mixture** form, the population is an explicit blend: a cured fraction π with survival fixed at 1 forever, and an uncured fraction (1-π) following the chosen parametric distribution. This is the more clinically interpretable form for HEOR write-ups — π reads directly as "proportion cured" — and it extends naturally to covariates on both the cure fraction and the uncured-group survival:

```
fit_mix_cov <- flexsurvcure(
  Surv(time, status) ~ arm,
  data = tb_followup,
  dist = "weibull",
  mixture = TRUE,
  anc = list(shape = ~ arm)
)
```

The **non-mixture** form arises from a different generative story (a latent number of malignant/infectious cells following a Poisson process, each with independent probability of elimination) and is more common in oncology cure-modelling traditions. Both forms nest the standard non-cure model as $\pi \longrightarrow 0$, so a likelihood-ratio test or AIC comparison against the corresponding plain `flexsurvreg` fit (§10.2) is a reasonable first check of whether a cure fraction is statistically supported at all.

Common pitfall. A fitted cure fraction is a **statistical** artefact of the likelihood, not evidence of biological cure. A model will happily return a cure estimate of 0.15 purely because the observed Kaplan-Meier curve (§10.1) flattens near the end of follow-up — which is exactly what **censoring** also does. Short or administratively-censored follow-up is the single most common cause of a spuriously large cure estimate. Always check: does the KM curve's plateau extend across enough separated event-free time, with enough patients still at risk at the plateau, to distinguish "nobody left to have an event" from "these people are actually cured"? If follow-up barely exceeds the plausible window for relapse, treat any cure-model result as exploratory only.

HEOR in practice. When a submission depends on a cure assumption, present the cure fraction alongside its 95% CI (`summary()` on the `flexsurvcure` object returns this) and run a one-way sensitivity analysis setting the cure fraction to zero and to the CI bounds in the economic model. Reviewers at HTA bodies (NICE, in particular, following TSD 14 guidance — see §10.12) routinely request this, and having it ready pre-empts a request for clarification that otherwise stalls appraisal.

How to report this.

Parameter	Estimate	95% CI lower	95% CI upper
Cure fraction (pi)	0.24	0.17	0.32
Uncured Weibull shape	1.42	1.08	1.87
Uncured Weibull scale	18.6	14.2	24.3
Model comparison			
Mixture cure model AIC	812.4	–	–
Standard Weibull (no cure) AIC	829.1	–	–

Table 27: How to report this: cure-fraction estimate from a mixture cure model (`flexsurvcure()`, Weibull uncured distribution), with the 95% confidence interval `summary()` returns, alongside the AIC comparison against the corresponding non-cure Weibull fit that motivates including a cure fraction at all.

10.5 Piecewise and landmark models

In plain terms first: sometimes the reason one curve doesn't fit well isn't that you chose the wrong family — it's that the real hazard genuinely behaves differently in different time windows (for example, high risk during active treatment, then a much calmer surveillance phase afterwards). No single smooth curve can bend sharply enough to capture that kind of structural change without overfitting elsewhere. The practical fix, called a **landmark** approach, is almost deliberately low-tech: trust the real, observed Kaplan-Meier curve for as long as you have good data, and only hand off to a fitted parametric curve for the unobserved tail beyond that point — rather than asking one curve to explain both phases at once.

Cure models relax the **shape** assumption; piecewise models relax it in a different way, by abandoning the idea that one parametric family must hold over the entire time horizon. A piecewise (segmented) model fits different hazard or survival behaviour in separate time intervals, stitched together at one or more cut-points. This is a natural fit whenever a clinical or data-generating mechanism changes partway through follow-up — for instance, a fixed-duration TB regimen where hazard during active treatment differs structurally from hazard during the post-treatment surveillance period, or trials where hazard behaviour measured within the trial should not simply be extrapolated using whatever curve happened to win AIC (§10.6) beyond the data.

A **landmark** approach is the simplest practical version of this idea for extrapolation: fit (or simply use) the empirical Kaplan-Meier estimate for the observed trial duration, and attach a parametric extrapolation only from a chosen landmark time point onward, rather than letting one parametric curve govern both the well-observed early period and the long unobserved tail.

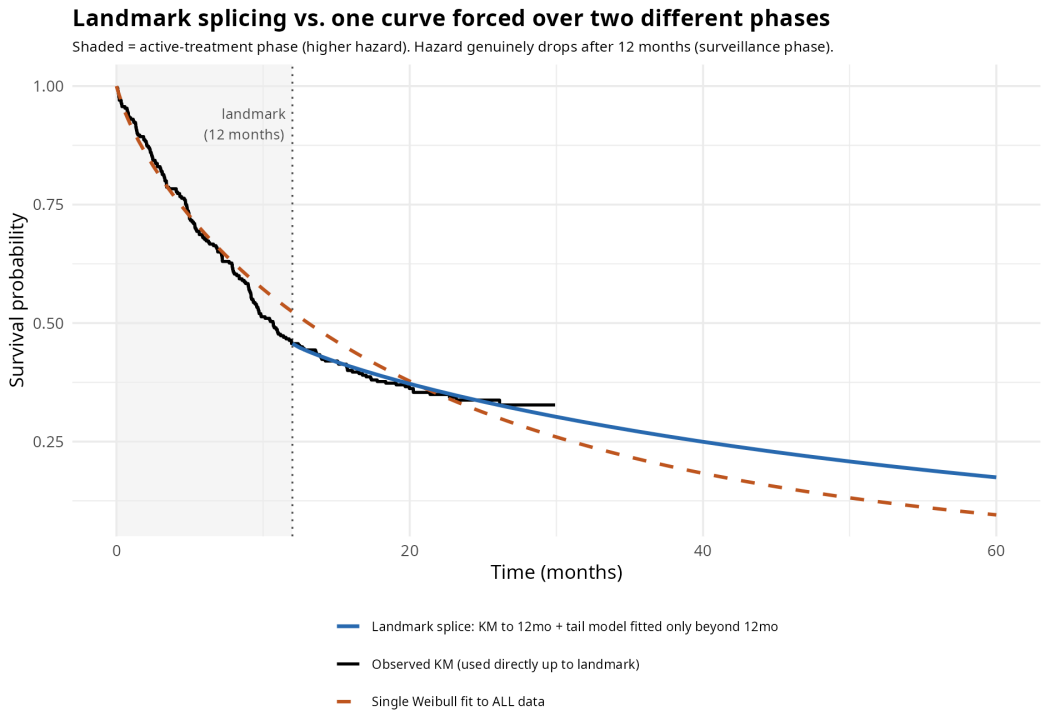


Figure 13: A simulated trial where hazard genuinely drops after a 12-month landmark (shaded: the higher-hazard active-treatment phase). A single Weibull fitted to all the data (orange, dashed) is forced to compromise between the two phases and fits neither well. The landmark approach (blue) instead uses the real KM curve up to 12 months, then splices in a tail model fitted only to the post-landmark data — tracking the true post-landmark decline far more closely.

```
library(survival)
library(flexsurv)

landmark_time <- 24  # months; chosen where KM data becomes sparse

fit_pw <- survival::survSplit(
  Surv(time, status) ~ .,
  data = tb_followup,
  cut = landmark_time,
  episode = "period"
)

fit_pw_exp <- survival::coxph(
  Surv(tstart, time, status) ~ strata(period),
  data = fit_pw
)
```

In practice, the economic-model-facing version of a landmark approach is often simpler than fitting a formal piecewise hazard model in R: use the KM estimate directly for t ≤ landmark, then splice in a `flexsurvreg` or `flexsurvspline` fit (§10.2-§10.3) — chosen for clinical plausibility of its tail, not merely best AIC — for t > landmark. The spline models of §10.3 are a particularly natural extrapolation partner here because their flexible knot structure can be fitted to just the later portion of follow-up without distorting the early, well-observed KM shape.

```
tail_data <- subset(tb_followup, time > landmark_time)

fit_tail <- flexsurvspline(
  Surv(time, status) ~ 1,
  data = tb_followup,
  k = 1,
  scale = "hazard"
)
```

Common pitfall. A piecewise model with a cut-point placed wherever the hazard **happens** to look like it changes, chosen after inspecting the data, is overfitting dressed up as flexibility — the same multiple-comparisons problem as moving a regression breakpoint until R^2 improves. Cut-points should be pre-specified from a clinical rationale (end of a fixed treatment course, a protocol-defined visit, a known change in standard of care) wherever possible, not selected by eyeballing the KM curve for the best-looking kink.

HEOR in practice. Landmark extrapolation is especially defensible — and increasingly expected by reviewers — whenever a trial's follow-up is short relative to the economic model's time horizon, since it keeps the empirical, assumption-free KM estimate (§10.1) governing exactly the period where trial data are informative, and confines parametric or spline modelling risk to the period where it is unavoidable.

10.6 Model selection: AIC/BIC, visual fit, clinical plausibility

In plain terms first: this section is about a single, important trap. Every statistic you can compute from a model — AIC, BIC, log-likelihood — only measures how well the curve matches the data you actually observed. It has nothing to say about the part of the curve beyond your data, even though that unobserved region is usually what drives most of an economic model's lifetime costs and QALYs. Two curves can score almost identically on AIC and still tell completely different stories about year 10 or year 20. The fix isn't a smarter statistic — it's to never trust a single number alone: always look at the extrapolated curve with your own eyes, and ask whether its long-run shape makes clinical sense.

Difficult terminology: log-likelihood. The log-likelihood is a single number that summarises how probable the data you actually observed would have been, if the fitted model's parameters were exactly correct — it is the mathematical currency nearly every model-comparison tool in this book spends (AIC and BIC, which you've already seen used for parametric survival curves, are both built directly from it). A higher log-likelihood (remember these are typically negative numbers, so "higher" usually means "less negative," i.e. closer to zero) means the fitted model assigns higher probability to the data you actually saw — in other words, a better-fitting model. A simple way to build intuition: imagine two coins, one you believe is fair and one you believe lands heads 90% of the time; if you actually flip a coin and get eight heads out of ten, the "90% heads" model assigns a much higher probability to that specific outcome than the "fair coin" model does — its log-likelihood, given that data, is higher. Survival models work the same way at a larger scale: the log-likelihood adds up, across every patient in the

dataset, how probable each patient's observed outcome (their exact event time, or the fact that they were censored at a particular point) was under the fitted hazard curve. Crucially — and this is the trap this section is warning about — the log-likelihood only ever evaluates how well a model explains data you **actually have**; it has nothing to say about how trustworthy the model's extrapolated tail is, because there is no observed data out there to check it against.

Every model fitted across §10.2-§10.5 — standard parametric families, spline models, cure models, piecewise/landmark constructions — produces a log-likelihood, and every one of those model objects supports the base R generics directly:

```
fits <- list(
  exp      = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"exp"),
  weibull  = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"weibull"),
  gompertz = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"gompertz"),
  llogis   = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"llogis"),
  lnorm    = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"lnorm"),
  gengamma = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"gengamma")
)

sapply(fits, AIC)
sapply(fits, BIC)
```

AIC and BIC rank goodness-of-fit **to the observed data**, penalised for the number of parameters (BIC more heavily, favouring simpler models in larger samples). They are a legitimate, necessary first filter — a model that fits the observed follow-up poorly should rarely be carried forward regardless of how convenient its extrapolated tail looks. But an information criterion says nothing about behaviour **beyond** the data, which is precisely the region that drives most of the lifetime cost and QALY impact in an economic model.

How to report this.

Distribution	Parameters	−2LogLik	AIC	BIC
Exponential	1	812.6	814.6	817.3
Weibull	2	798.2	802.2	807.6
Gompertz	2	796.8	800.8	806.2
Log-logistic	2	799.5	803.5	808.9
Log-normal	2	801.1	805.1	810.5
Generalised gamma	3	795.9	801.9	809.9

Gompertz selected for the base case: lowest AIC among two-parameter families and a clinically plausible extrapolated tail; generalised gamma's marginally lower AIC did not outweigh its less stable shape-parameter estimate (§10.2).

Table 28: How to report this: AIC/BIC model-comparison table across the six standard parametric survival distributions fitted with `flexsurvreg()`, the TSD 14-expected candidate set and reporting format for justifying a base-case extrapolation choice.

Common pitfall. Two curves can have near-identical AIC over the observed follow-up and diverge enormously once extrapolated — one flattening into a plausible long-term plateau, the other decaying to near-zero survival within a few extra years. AIC/BIC cannot distinguish between them because both explain the **observed** data equally well; the difference lives entirely in the unobserved region the criterion was never calculating over. Selecting a final extrapolation model on AIC/BIC alone is one of the most common and most criticised errors in HTA submissions.

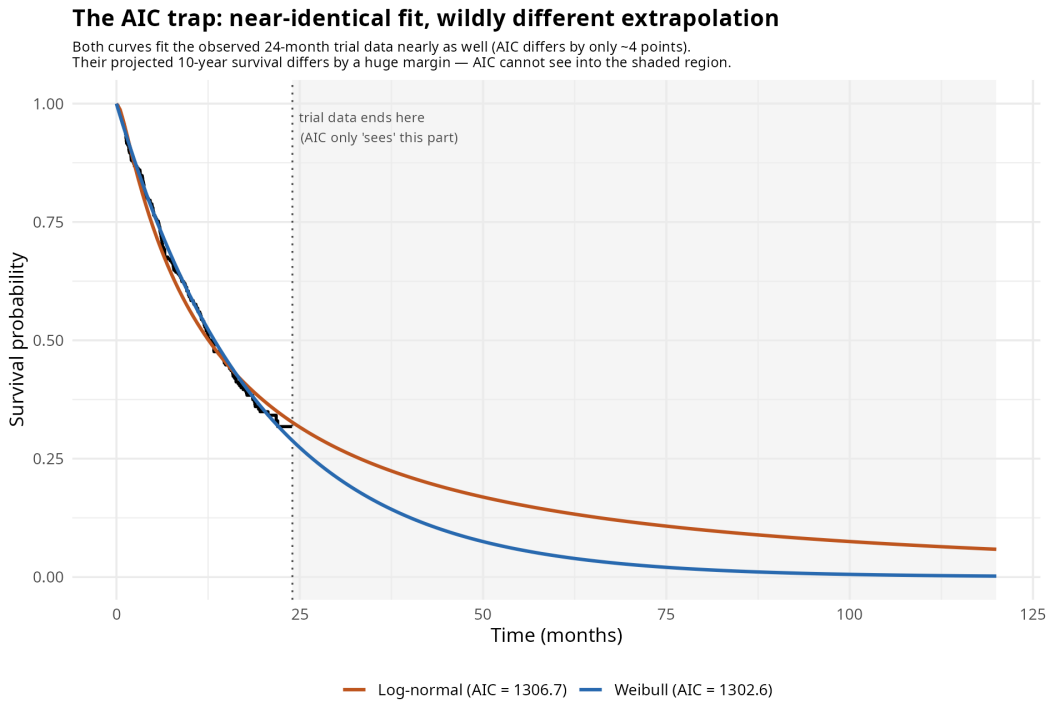


Figure 14: A simulated 24-month trial where a Weibull and a log-normal fit the observed data almost equally well (AIC 1302.6 vs 1306.7 — a difference most analysts would call negligible). Extrapolated to a 10-year horizon (shaded region), the two curves diverge sharply: by month 120 one projects roughly 6% survival and the other under 1%. AIC alone gives no warning that this divergence is coming, because it is calculated entirely within the unshaded region.

Two further checks are required alongside the statistical criteria, and should be treated as mandatory, not optional, for any model informing extrapolation:

- **Visual fit against the full observed follow-up.** Overlay each candidate curve on the Kaplan-Meier estimate (§10.1) across the **entire** observed period, not just a global fit statistic — look specifically for systematic departures in the region with the most patients still at risk, since that is where poor fit matters most for inference.
- **Clinical plausibility of the extrapolated tail.** Beyond the data, ask whether the shape is biologically defensible: does a Gompertz tail imply an implausibly rapid late-life hazard escalation for this population? Does a cure-model fraction (§10.4) correspond to a mechanism (e.g. durable bacteriological clearance) that a clinician would recognise, or is it just where the likelihood happened to settle? Plot each candidate's extrapolated survival against the full economic-model time horizon and sanity-check the implied long-term survival or life-years against external epidemiological data or clinical opinion.

```
plot(fits$weibull, type = "survival")
lines(survfit(Surv(time, status) ~ 1, data = tb_followup), col = "grey40")

plot(fits$gompertz, type = "survival", t = seq(0, 240, by = 1))
```

This three-part discipline — statistical fit (AIC/BIC), visual fit, and clinical plausibility of the tail — is the core of the NICE DSU Technical Support Document 14 (Latimer, 2013) guidance on survival extrapolation, still the reference standard cited by most HTA bodies. §10.12 returns to this guidance in full detail, including external-data validation and expert-elicitation approaches to tail

plausibility; the practical takeaway here is simply that no single number from `AIC()` should ever be the sole basis for choosing an extrapolation model.

HEOR in practice. Build the habit of reporting all three elements together for every extrapolation choice in a submission: an AIC/BIC table across candidate models, a KM-overlay plot across observed follow-up, and a long-horizon extrapolation plot with a short clinical-plausibility justification for the chosen tail. This triad is close to a checklist item in NICE technical engagement, and assembling it alongside model-fitting — rather than retrofitting it after a model is already locked into the economic model — saves substantial rework later.

Agency perspective. For a TB programme specifically, "clinical plausibility of the tail" has a concrete anchor: published TB relapse-hazard literature and programme surveillance data give an external benchmark for what a defensible long-term hazard or cure fraction looks like. Treat any model whose extrapolated tail falls well outside that external range as a candidate to be interrogated, not adopted, however well it scores on AIC.

10.7 Proportional hazards diagnostics and non-PH alternatives

Chapter 10's Cox models (via `survival::coxph()`) and the parametric curves fitted with `flexsurv::flexsurvreg()` in §10.2 share a hidden assumption whenever a treatment or covariate effect is expressed as a single hazard ratio: that the ratio of hazards between arms stays constant for the whole follow-up period. This is sometimes called the proportional hazards (PH) assumption. In an economic model this assumption is load-bearing — it is usually what licenses stretching a trial's within-trial hazard ratio across a lifetime horizon. If the assumption is wrong, the extrapolated incremental QALYs and costs inherit that error, often invisibly.

Testing the proportional hazards assumption

`survival::cox.zph()` tests whether the scaled Schoenfeld residuals from a fitted `coxph()` model are correlated with time. Under PH, residuals for a covariate should fluctuate randomly around zero; a trend indicates a time-varying effect.

Difficult terminology: Schoenfeld residuals. A Schoenfeld residual, computed separately at each event time in the data, captures the gap between what actually happened and what the model's proportional-hazards assumption expected to happen on average at that moment. Concretely: at each event time, look at the patient who had the event, and compare their covariate value (say, their actual treatment arm or age) to a weighted average of the covariate values among everyone still at risk at that moment — the model's implicit prediction of what that value "should" look like if the hazard ratio really is constant over time. If the proportional hazards assumption truly holds, there's no reason these gaps should trend in any direction as time passes — they should scatter randomly above and below zero throughout follow-up, like noise with no pattern. If instead the residuals visibly drift upward or downward as time goes on, that is direct evidence the covariate's effect on risk isn't staying constant — e.g. a new drug's benefit might be large early on then fade, which would show up as the residuals systematically drifting in one direction across the follow-up period rather than bouncing

randomly around zero. `cox.zph()`'s formal test and accompanying plot are simply a statistical and visual way of checking for exactly that drift.

```
library(survival)

cox_fit <- coxph(Surv(time, status) ~ arm + age + ecog_ps, data = os_data)
zph_test <- cox.zph(cox_fit)
zph_test
plot(zph_test, var = "arm")
```

A significant global or `arm`-specific p-value (conventionally < 0.05) is evidence against PH for that term. The accompanying plot shows the estimated log hazard ratio against time; a flat line supports PH, a clear slope does not.

Common pitfall. A non-significant `cox.zph()` test is not proof of PH — it is merely absence of evidence, and the test has low power in small or heavily censored trials (exactly the oncology trials HEOR models often rely on). Conversely, with very large samples, trivial and clinically irrelevant time-trends can reach significance. Always pair the test with the Schoenfeld residual plot and clinical judgement about mechanism (e.g. a new treatment with delayed onset, or diminishing effect as resistance emerges) before deciding PH is tenable.

How to report this.

Term	chi-square	df	p-value	PH decision
arm	1.2	1	0.27	Supported – no evidence of a time-varying treatment effect
age	0.4	1	0.53	Supported
`ecog_ps`	7.8	1	0.005	Rejected – effect of ECOG status changes materially over follow-up
GLOBAL	9.1	3	0.028	Rejected overall – a non-PH alternative structure is warranted (see below)

Model: `coxph(Surv(time, status) ~ arm + age + ecog_ps)`; global rejection driven by `ecog_ps`, motivating the AFT or spline-based non-PH alternatives discussed next.

Table 29: How to report this: `cox.zph()` proportional-hazards diagnostics for a Cox model of treatment arm, age and ECOG performance status – the conventional format for presenting per-covariate and global Schoenfeld-residual test statistics alongside the PH decision each motivates.

When PH fails: alternative hazard structures

If PH is rejected, three broad options replace the single hazard ratio:

- **Piecewise/time-varying hazard ratio** — fit `coxph()` with a time-dependent covariate (e.g. via `survival::survSplit()` to cut follow-up into intervals and interact `arm` with interval), yielding a stepped hazard ratio rather than a constant one.
- **Accelerated failure time (AFT) models** — `flexsurv::flexsurvreg()` with `dist = "weibull"`, `"llogis"`, or `"lnorm"` from §10.2 are AFT forms: treatment rescales survival **time** rather than multiplying the hazard, which can fit crossing or converging survival curves that PH cannot.
- **Flexible spline-based hazards** — `flexsurv::flexsurvspline()` allows the treatment effect to enter on the hazard, odds, or normal scale with splines on the baseline, directly relaxing PH while remaining a parametric, extrapolable model.

```
library(flexsurv)

aft_weibull <- flexsurvreg(
  Surv(time, status) ~ arm,
  data = os_data,
  dist = "weibull"
)

spline_fit <- flexsurvspline(
  Surv(time, status) ~ arm,
  data = os_data,
  k = 1,
  scale = "hazard"
)
```

> **HEOR in practice.** NICE's DSU TSD 14 (survival analysis for economic evaluation) treats PH testing as a mandatory first step before any extrapolation, and expects submissions that reject PH to justify the alternative structural choice clinically, not just statistically. In a reimbursement submission, present `cox.zph()` output and the Schoenfeld plot alongside the chosen non-PH model as part of the base-case justification, not as a sensitivity analysis afterthought.

10.8 Covariate-adjusted and treatment-effect-specific models

Economic models rarely want a single marginal survival curve per arm; they want the curve for a population defined by baseline covariates (age, stage, biomarker status) and sometimes a treatment effect that itself depends on those covariates. Building on the `flexsurvreg()` syntax from §10.2, covariates enter through the familiar formula interface — but **where** they enter (location, scale, or shape of the distribution) changes what the model can represent.

Covariates on the location/scale parameter

By default, `flexsurvreg()` places covariates on the main location parameter of the chosen distribution (the rate for exponential, the scale for Weibull, etc.), which is the AFT-style adjustment used throughout §10.2.

```
adj_weibull <- flexsurvreg(
  Surv(time, status) ~ arm + age + ecog_ps,
  data = os_data,
  dist = "weibull"
)
```

This yields a single estimated treatment effect, held constant across the covariate distribution — adequate when the question is "what survival curve for a patient of average age and ECOG status," but it cannot represent a treatment effect that differs by subgroup.

Covariates on ancillary (shape) parameters

`flexsurvreg()`'s `anc` argument attaches covariates to a distribution's ancillary parameter (e.g. the shape of a Weibull or Gompertz, the sigma of a log-normal), letting a covariate change the **shape** of the hazard trajectory rather than only its scale.

```
shape_fit <- flexsurvreg(
  Surv(time, status) ~ arm,
  anc = list(shape = ~ arm),
  data = os_data,
  dist = "weibull"
)
```

Here `arm` enters both the scale (via the main formula) and the shape (via `anc`), allowing treatment to alter not just how long patients survive but whether the hazard is increasing, decreasing, or constant over time — a natural way to express mechanisms like "early benefit that fades" without abandoning a parametric, extrapolable family.

> **Common pitfall.** Adding covariates to ancillary parameters multiplies the number of estimated coefficients quickly, and with trial-sized samples (hundreds, not thousands) the shape-parameter estimates can become unstable even when the model converges. Check standard errors and confidence intervals on the `anc` coefficients specifically — a wide interval on a shape-by-treatment interaction is a sign the data cannot support that extra flexibility, however appealing the fit statistic.

Treatment-effect-specific extrapolation

Once a covariate-adjusted model is fitted, `summary()` on the `flexsurvreg` object with a `newdata` argument produces arm- and covariate-specific survival curves directly, which feed into the Markov or partitioned-survival model exactly as the unadjusted curves did in §10.2-10.5.

```
newdat <- data.frame(arm = c("control", "treatment"), age = 60, ecog_ps = 1)
summary(shape_fit, newdata = newdat, t = seq(0, 60, by = 1), type = "survival")
```

> **HEOR in practice.** Decide before fitting whether the economic question is population-average (one curve per arm) or subgroup-specific (curves varying by baseline covariates): this determines whether `anc` and interaction terms belong in the base case or in a scenario analysis. Reviewers increasingly expect covariate-adjusted analyses when a trial shows baseline imbalance between arms, since an unadjusted hazard ratio can then be confounded rather than purely a treatment effect.

10.9 Extrapolation: external data, background mortality, expert elicitation

Trials report follow-up measured in months; reimbursement decisions are made on lifetime horizons. Every model in this chapter — from the single `flexsurvreg()` fit in §10.2 to the non-

PH alternatives in §10.7 — eventually has to answer a question the trial data cannot answer alone: what happens to the survival curve beyond the last observed event. Three complementary tools manage that gap: external data, background mortality, and structured expert elicitation.

Anchoring with external data

Where available, longer-follow-up data from a related trial, a registry, or a published meta-analysis can validate or recalibrate the tail of an extrapolated curve. The common workflow is to source external Kaplan-Meier data, read it in exactly as any other tabular data (`readr::read_csv()`, per Ch.5), and overlay it against the within-trial parametric fit visually and via landmark survival comparisons (e.g. does the model's 5-year survival estimate sit near the external estimate) rather than formally refitting the trial model to pooled data.

```
library(readr)
library(dplyr)

external_km <- read_csv("registry_survival_5yr.csv")

model_est <- summary(aft_weibull, t = 60, type = "survival")[[1]]
external_est <- external_km |> filter(time_months == 60)

bind_rows(model = model_est, external = external_est)
```

Background mortality as a plausibility bound

§8.5 introduced layering age- and sex-matched general-population mortality beneath disease-specific mortality once the disease-specific hazard falls below the general-population hazard — the standard device for making sure a chronic-disease model does not extrapolate a disease-specific survival curve indefinitely below what a healthy member of the same population would experience. In extrapolation, that convention becomes a **constraint**, not just a modelling convenience: a national life table (read in as plain CSV/Excel via `readr::read_csv()` or `readxl::read_excel()`, per Ch.5) sets the ceiling on hazard, so predicted survival should never fall below the general-population curve for the matching age-sex strata.

```
library(readr)
library(dplyr)

life_table <- read_csv("national_life_table.csv")  # age, sex, qx (annual
mortality prob.)

disease_hazard <- -log(1 - 0.08)
bg_hazard <- life_table |>
  filter(age == 70, sex == "female") |>
  pull(qx) |>
  (\(qx) -log(1 - qx))()

combined_hazard <- pmax(disease_hazard, bg_hazard)
combined_hazard
```

> **Agency perspective.** Treat this the same way Ch.8 §8.5 did: compute background mortality on the same timescale (annual vs monthly) as the disease-specific hazard before comparing or combining them, and apply the comparison at every model cycle, not just at the point where

extrapolation begins — a disease hazard that dips below background mortality transiently (not just permanently at the tail) still needs the floor applied at that cycle.

Common pitfall. A parametric curve with a poorly chosen tail shape (e.g. a log-normal or log-logistic fitted mainly to capture early-curve flexibility) can imply a long-term hazard that **decreases** with age, eventually crossing below general-population survival — biologically implausible for almost any serious disease. Plot the extrapolated curve against the life-table curve over the full model horizon as a standard diagnostic, not an edge-case check; NICE DSU TSD 14 reviewers routinely flag curves that cross.

Expert elicitation for structural and tail uncertainty

When neither longer trial follow-up nor comparable external data exist — common for recently approved treatments with short median follow-up — formal expert elicitation offers a way to quantify uncertainty about the extrapolated tail (e.g. long-term treatment-effect waning, cure fraction plausibility) that is defensible rather than ad hoc. The SHELF (SHeffield ELicitation Framework) protocol is the most widely used approach in HTA submissions for eliciting probability distributions from clinical experts on quantities like long-term survival proportions; it is covered in full in Ch.14 §14.7, including how elicited distributions are encoded as priors or scenario inputs. For now, the point to carry forward is structural: expert elicitation does not replace the parametric or external-data analysis above, it supplements it — typically by bounding or re-weighting extrapolation scenarios ("optimistic," "base-case," "pessimistic" tail assumptions) that are then propagated through the probabilistic sensitivity analysis built on the survival models of this chapter.

HEOR in practice. Build the extrapolation case as a layered argument, not a single curve: (1) the best-fitting parametric or spline model from §10.2-10.8, (2) a background-mortality floor from §8.5′s convention, (3) a cross-check against any available external data, and (4) where tail uncertainty remains material to the decision, a structured elicitation exercise (Ch.14 §14.7) to characterise that uncertainty rather than asserting a single point estimate. Submissions that show all four elements together are markedly more resilient under HTA committee scrutiny than a single AIC-selected curve.

10.10 Competing risks and multi-state models

Every extrapolation so far in this chapter — §10.2′s parametric distributions, §10.9′s extrapolation workflow — has treated the clinical course as one single transition: alive to dead, or progression-free to progressed. Many HEOR models need more states than that. Picture a TB treatment-pathway model tracking **treatment success**, **loss to follow-up**, **treatment failure/retreatment**, and **death**. Loss to follow-up competes with cure for who "claims" a patient first, and retreatment is itself a transition with its own hazard. A standard Kaplan-Meier or single-event parametric fit mishandles this silently: if you censor a patient at a competing event (instead of modelling that event explicitly), you bias the cumulative incidence of the event you actually care about — usually inflating it.

Two R packages work together to handle this: `mstate` for the data-management and semi-parametric multi-state machinery, and `flexsurv` for fitting parametric or flexible distributions to each transition separately — exactly what a multi-state cost-effectiveness model needs for extrapolation.

Competing risks: cause-specific vs. subdistribution hazards

With competing risks, there are at least two legitimate hazard concepts, and they answer different questions:

- The **cause-specific hazard** treats competing events as censoring and asks "among patients still at risk of this event, what's the instantaneous rate?" It is the right quantity for understanding aetiology/mechanism.
- The **subdistribution hazard** (Fine-Gray) keeps patients who experienced the competing event in the risk set (as if destined never to have the event of interest) and directly models the **cumulative incidence function** (CIF) — the probability of this event by time t in the presence of competing risks. This is usually the quantity a cost-effectiveness model wants, because the model needs the actual probability mass flowing into each state, not a hypothetical hazard in a world where competing events don't happen.

> **Common pitfall.** Censoring competing events in a standard Kaplan-Meier fit produces a cumulative incidence estimate that is **too high** — treating a competing event as censoring systematically overstates the true probability of the event of interest when competing risks are non-trivial. This is the single most common competing-risks error in submitted HTA models: a "time to loss-to-follow-up" curve fitted by censoring deaths will overestimate loss-to-follow-up, because dead patients can no longer be lost to follow-up but a censoring-based estimator treats them as still "at risk" and still eligible to drop out indefinitely. Always check whether competing events were censored or modelled when critiquing a submitted dossier's multi-state structure.

Data preparation and fitting with `mstate`

`mstate` works from a **transition matrix** describing the permitted moves between states, and converts standard long-format survival data into the stacked "msdata" format multi-state models require.

```
library(mstate)
library(survival)

tmat <- transMat(
  x = list(
    c(2, 4),     # from Treatment success: to Failure, or Death
    c(3, 4),     # from Failure: to Retreatment, or Death
    c(4),        # from Retreatment: to Death
    c()          # Death is absorbing
  ),
  names = c("TxSuccess", "Failure", "Retreat", "Death")
)

msdat <- msprep(
  time  = cbind(tx_success_time, failure_time, retreat_time, death_time),
  status = cbind(tx_success_status, failure_status, retreat_status, death_status),
  data  = tb_cohort,
```

```
  trans = tmat
)

cox_trans <- coxph(Surv(Tstart, Tstop, status) ~ strata(trans), data = msdat)
msf <- msfit(cox_trans, trans = tmat)
```

`msprep()` is doing the unglamorous but essential work: for every patient and every transition they were at risk of, it creates a row with a start time, stop time, and event indicator, in the "counting process" format `coxph()` expects when combined with `Surv(Tstart, Tstop, status)`. `msfit()` then assembles the fitted cumulative hazards per transition into the full set of transition probabilities — the quantity that actually populates a multi-state Markov cohort model's transition-probability matrix.

Parametric extrapolation of transition-specific hazards with `flexsurv`

`mstate`'s default Cox-based fitting is semi-parametric and does not extrapolate beyond the observed follow-up — exactly the gap §10.9 was built to fill for single-event models. The natural extension is to fit `flexsurvreg()` (§10.2's workhorse) separately to each transition's data, by subsetting the `msdata` object on `trans` and fitting the usual parametric families:

```
library(flexsurv)

transition_fits <- lapply(1:3, function(k) {
  flexsurvreg(
    Surv(Tstart, Tstop, status) ~ 1,
    data = subset(msdat, trans == k),
    dist = "weibull"
  )
})
```

Each `transition_fits[[k]]` can then be extrapolated exactly as in §10.9, and the resulting transition-specific survival curves assembled into a cohort trace — this is precisely how a multi-state partitioned-survival economic model is extrapolated in practice: fit and select a distribution **per transition**, not one global curve for the whole pathway.

> **HEOR in practice.** Decide the multi-state structure from the clinical pathway **before** reaching for `mstate` syntax. A model with "Death" reachable from every intermediate state is almost always right for chronic or infectious disease pathways (nobody is protected from dying by being mid-treatment) — map every edge in `transMat()` against a clinician-validated pathway diagram, not just the transitions that happen to appear in the trial dataset.

10.11 Reconstructing individual patient data from published Kaplan-Meier curves

Submitted HTA evidence frequently gives you a published trial's Kaplan-Meier **curve** as a figure, with a risk table beneath it, but not the underlying patient-level survival times — the original IPD may sit behind a confidentiality wall at the manufacturer, or the publication predates routine IPD sharing. Yet §10.2's parametric fitting and §10.9's extrapolation both need IPD (or something that behaves like it) as their input. The standard solution is the algorithm of Guyot, Ades, Ouwens and Welton (2012, **BMC Medical Research Methodology**), implemented in the `IPDfromKM` package,

which reconstructs an approximate patient-level dataset directly from a digitized KM curve plus its risk table.

The Guyot et al. (2012) reconstruction algorithm

The algorithm's logic is a clever inversion of how a KM curve is built in the first place:

1. **Digitize** the published curve: read off (t, S(t)) coordinate pairs at a fine grid of points along the printed curve (typically using plot-digitizing software or manual pixel-coordinate extraction).
2. Read the **numbers-at-risk table** printed beneath the curve at its labelled time points — this is the critical extra ingredient the original KM figure alone does not give you, because it anchors how many individuals are being censored versus experiencing events between risk-table intervals.
3. Use the risk-table counts together with the digitized S(t) step heights to back-solve, interval by interval, for the number of events and the number of censorings consistent with the observed drop in S(t) — effectively running the Kaplan-Meier product-limit formula in reverse.
4. Output a reconstructed dataset of individual (time, event/censor) records that, when a KM curve is refitted to them, closely reproduces the original published curve.

The reconstruction is necessarily approximate — it cannot recover information the published curve never contained (exact censoring times within an interval are allocated by an assumed-uniform rule, for instance) — but it has become the de facto accepted method in HTA submissions for enabling parametric extrapolation when only published curves are available.

Using `IPDfromKM`

```
library(IPDfromKM)

# Step 1: digitize the curve (coordinates read from the published figure,
# e.g. via a digitizer such as WebPlotDigitizer or Engauge)
preprocessed <- getpoints(
  dat     = digitized_coordinates,
  tot.events = NULL,
  trisk   = risk_table_times,
  nrisk.data = risk_table_numbers
)

# Step 2: getIPD() runs the Guyot et al. algorithm to back-calculate
# individual event/censoring times consistent with both the digitized
# curve and the risk table
reconstructed <- getIPD(
  prep    = preprocessed,
  armID   = 1,
  tot.events = NULL
)

ipd_fit <- flexsurv::flexsurvreg(
  survival::Surv(time, status) ~ 1,
  data = reconstructed$IPD,
  dist = "weibull"
)
```

Exact argument names vary slightly by `IPDfromKM` version — consult `?getpoints` and `?getIPD` in your installed version before relying on a specific call signature; the purpose and overall two-step workflow (digitize-and-anchor, then back-calculate) is stable across versions.

Common pitfall. Reconstructed IPD is a statistical approximation layered on top of an already-approximate digitization step. Two failure modes recur in practice: (1) mis-reading the risk-table time points so they don't align with the digitized curve's x-axis grid, which silently corrupts the back-calculated event counts for every later interval; and (2) over-trusting the reconstructed tail — KM curves are read least precisely in their sparse, high-censoring right tail, which is exactly the region §10.9's extrapolation choice is most sensitive to. Always refit a KM curve to the reconstructed IPD and visually overlay it on the original published figure before using the reconstruction for anything downstream.

HEOR in practice. Treat IPD reconstruction as a documented, auditable step in its own right, not an invisible preprocessing detail. Record the digitizing software, the coordinate-extraction screenshots, and the risk-table transcription alongside the reconstructed dataset, because a NICE committee or external assessment group reviewing your submission will often want to independently verify that the reconstruction matches the original publication's curve.

10.12 Reporting extrapolation following NICE DSU TSD 14 and 21

Agency perspective. Everything built across this chapter — the parametric family in §10.2, the Royston-Parmar spline models of §10.3, the cure models of §10.4, the statistical-and-clinical-plausibility selection logic of §10.6, and the extrapolation mechanics of §10.9 — exists because two specific NICE Decision Support Unit Technical Support Documents spell out what UK (and, by wide adoption, international) HTA bodies expect to see when a submission extrapolates trial-observed survival beyond the data. Chapter 2 of this book flagged that this chapter would need to cite them properly; here is the substance, not just the citation.

NICE DSU TSD 14 — "Survival Analysis for Economic Evaluations Alongside Clinical Trials: Extrapolation with Patient-Level Data" (Latimer, 2011, updated 2013) is the foundational document. In plain terms, it asks for four things:

- Fit the standard parametric survival distributions — exponential, Weibull, Gompertz, log-logistic, log-normal, and the generalised gamma — exactly the family fitted with `flexsurvreg()` in §10.2. TSD 14 is the reason this particular set of six distributions is the default candidate list in essentially every UK submission.
- **Test the proportional hazards assumption** before committing to a PH-based extrapolation structure, because an extrapolation built on an unjustified PH assumption carries that error across the whole modelled time horizon.
- Use a visual and statistical model-selection toolkit together: inspect hazard and log-cumulative-hazard plots against each candidate distribution's theoretical shape, alongside AIC/BIC — the same diagnostics §10.6 walked through in code.
- Justify the final choice on statistical fit **and** clinical plausibility together, which is the central principle TSD 14 states and §10.6 of this chapter echoes in applied form: a distribution

that fits the observed Kaplan-Meier curve best is not automatically the right choice if its extrapolated tail implies a hazard shape (e.g. a declining hazard of death many years out) that no clinician would endorse for the disease in question.

NICE DSU TSD 21 — "Flexible Methods for Survival Analysis" (Rutherford, Lambert, et al.) is the newer document. It exists because TSD 14's standard six-distribution family can be too restrictive for real trial data — proportional-hazards or fixed-shape-hazard assumptions often fail, especially with modern trials showing treatment-effect waning, delayed separation of arms, or crossing hazards. TSD 21 extends TSD 14's logic in two ways:

- It covers **flexible spline-based models** — specifically Royston-Parmar models (§10.3's `flexsurvspline()`) — which relax the fixed-hazard-shape constraint of the standard family by modelling the log-cumulative-hazard as a spline function of log time, with the number and placement of knots as explicit, reportable choices.
- It covers **cure models** (§10.4's mixture and non-mixture cure fits), for disease areas where a meaningful fraction of the treated population may be effectively cured and face only background mortality thereafter — a structural assumption none of TSD 14's six distributions can represent, since none of them has a "cured fraction" parameter.
- TSD 21 is also explicit that flexible methods trade the standard family's extrapolation parsimony for better in-sample fit, and that this trade-off itself needs reporting: a spline model with many knots can fit the observed KM curve closely while behaving erratically once extrapolated past the data, so TSD 21 asks for the extrapolated hazard function to be shown, not just the extrapolated survival curve.

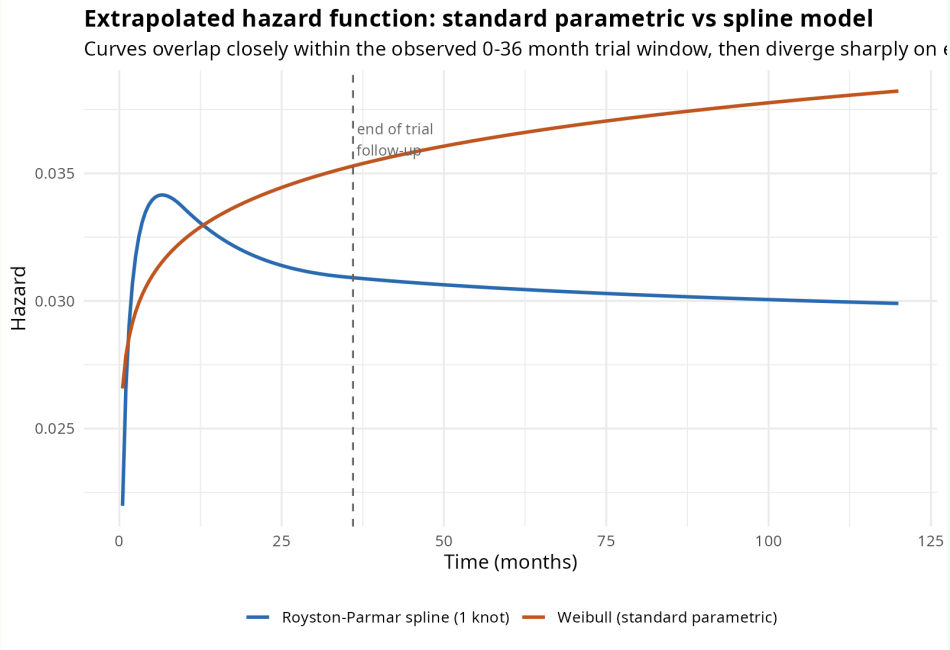


Figure 15: The extrapolated hazard function for a standard Weibull fit versus a one-knot Royston-Parmar spline, fitted to the same simulated trial data. The two curves track closely within the observed 0-36 month follow-up window but diverge sharply once extrapolated — exactly the TSD 21 diagnostic that an extrapolated survival-scale plot alone would hide.

Put simply, a submission report built from this chapter's methods should contain: the candidate distribution set and a reason for including or excluding flexible/cure-model alter-

natives; the proportional-hazards diagnostic; visual fit plots (KM overlay, hazard plot) for every candidate; an AIC/BIC table; a clinical-plausibility narrative for the selected model's extrapolated tail, ideally benchmarked against external data (population life tables, disease registries); and, where a spline or cure model is used, explicit justification for the knot count/placement or the cure-fraction assumption. A reviewer who knows TSD 14/21 will look for each of these elements by name — fitting six distributions, reporting AIC, and picking the lowest AIC without a clinical-plausibility and extrapolated-hazard discussion will not satisfy either document's stated expectations, however clean the R code behind it was.

Common pitfall. TSD 14 and TSD 21 are frequently cited in submissions as a bibliography entry without the actual diagnostics being reported — an AIC table alone does not constitute compliance with either document. Reviewers for NICE and other HTA bodies increasingly ask explicitly for the proportional-hazards check and the extrapolated-hazard-function plot (not just the survival curve), because these are the two diagnostics TSD 14 and TSD 21 respectively treat as mandatory, not optional, components of a defensible extrapolation.

Chapter 11. Meta-Analysis

Learning objectives

- Fit fixed-effect and random-effects meta-analysis models with `meta` (`metabin()`, `metacont()`, `metagen()`) and `metafor` (`rma()`) (§11.1)
- Quantify and interpret between-study heterogeneity using Q, τ^2, and I^2, and read a forest plot (§11.2)
- Screen for publication bias and small-study effects with funnel plots, Egger's test, and trim-and-fill (§11.3)
- Explain heterogeneity with meta-regression using `metafor::rma()`'s `mods=` argument, and recognise its limits (§11.4)
- Pool hazard ratios on the log scale for time-to-event outcomes, and connect the result to Chapter 10's extrapolation machinery (§11.5)
- Fit a bivariate random-effects model for diagnostic test accuracy with `mada::reitsma()` and read an SROC curve (§11.6)
- Pool single-group proportions and rates with variance-stabilising transformations (logit, double-arcsine, log) (§11.7)

Chapter 10 pooled nothing: it took one study's individual patient data and built one extrapolated survival curve from it. This chapter is about a different, very common situation — you have **several separate studies**, each already analysed on its own, and you need one combined answer. Here is the roadmap. §11.1 covers the first choice you make: fixed-effect or random-effects pooling. §11.2 and §11.3 cover two checks every pooled result needs before you trust it — how much the studies disagree (heterogeneity) and whether missing studies might be skewing the picture (publication bias). §11.4 asks **why** studies disagree. §11.5, §11.6 and §11.7 then apply all of this to three outcome types you will meet often in TB-programme work: time-to-event hazard ratios (which feed straight back into Chapter 10's extrapolation), diagnostic test accuracy, and single-group prevalence or incidence rates. Read the sections in order — each one assumes the last.

11.1 Fixed-effect and random-effects models

When a TB programme review pools results from, say, eight observational studies on treatment success rates under different regimens, the first decision is not which R function to call but which question you are asking. A **fixed-effect** (sometimes written "common-effect") model assumes every study is estimating the same true effect, and any difference between study estimates is pure sampling noise. A **random-effects** model assumes each study estimates its own true effect, drawn from a distribution of true effects across settings, and that distribution's spread is itself something you need to estimate. For TB and most infectious-disease health-economics work — where regimens, population risk, diagnostic pathways, and health-system capacity vary by country and era — the random-effects assumption is almost always the defensible default. Treat the fixed-effect model as a special case you report alongside the random-effects estimate for comparison, not as your primary analysis, unless you have strong a priori reasons (e.g. multi-site arms of a single trial with a shared protocol) to believe a common true effect.

R gives you two complementary toolkits. The `meta` package wraps the common effect-measure functions you will use most often in applied synthesis work: `metagen()` for generic (already-computed) effect sizes and standard errors, `metabin()` for binary outcomes (risk ratios, odds ratios, risk differences from 2x2 tables), and `metacont()` for continuous outcomes (mean differences or standardized mean differences from group means, SDs, and sample sizes). All three accept a `method.tau` argument that selects how between-study variance (τ^2) is estimated, and a `sm` argument for the summary measure (e.g. `"RR"`, `"OR"`, `"MD"`, `"SMD"`).

```
library(meta)

# Binary outcome: treatment success, event counts per arm
m_bin <- metabin(
  event.e = events_treatment, n.e = n_treatment,
  event.c = events_control,  n.c = n_control,
  data = tb_studies,
  sm = "RR",
  method = "MH",         # Mantel-Haenszel for the fixed-effect pooling step
  method.tau = "REML"    # tau^2 estimator for the random-effects layer
)
summary(m_bin)
forest(m_bin)
```

> **Difficult terminology: Mantel-Haenszel (MH) method.** The Mantel-Haenszel method is one of the oldest and simplest ways to combine 2x2 tables (treatment vs. control, event vs. no-event) from several studies into a single pooled risk ratio or odds ratio. Instead of averaging each study's own calculated ratio, it works directly from the raw event counts: it weights each study by how many events it contributed (roughly, its statistical "information"), so a study with 200 events pulls the pooled estimate much harder than one with 5 events. Think of it like calculating an overall batting average across several cricket seasons by adding up all runs and all innings first, rather than averaging each season's separate average — it handles small numbers of events more gracefully than methods that first calculate a ratio per study and then average those ratios. It is specifically a **fixed-effect** pooling method (see the surrounding text): it assumes every study estimates the same single true effect, so it is usually paired with a separate random-effects layer (here, `method.tau`) when that assumption is implausible.

The `metafor` package is lower-level and more flexible: you supply effect sizes and sampling variances (often via its own `escalc()` helper) and fit with `rma()`, specifying `method = "FE"` for fixed-effect or one of several random-effects estimators such as `"DL"`, `"REML"`, `"ML"`, or `"PM"` for `method`.

> **Difficult terminology: "ML" and "PM" as tau-squared estimators.** Here, "ML" stands for **maximum-likelihood** estimation of τ^2 (the between-study variance) — one of several numerical recipes for figuring out, from the pattern of disagreement across studies, how much genuine study-to-study variation to assume. "PM" is the **Paule-Mandel** estimator, a different, iterative recipe for the same quantity that is often recommended as a competitor to REML when the number of studies is small. **Disambiguation worth flagging explicitly**: this "ML" has nothing to do with Machine Learning, which is what "ML" means everywhere else in this book (see the abbreviations list) — here it is purely a statistical-estimation method for one

variance parameter inside a meta-analysis, chosen via the `method` or `method.tau` argument, not a predictive algorithm. When you see `method = "ML"` inside `rma()`, read it as "maximum-likelihood τ^2 estimator," and nothing more.

```
library(metafor)

dat <- escalc(measure = "RR",
              ai = events_treatment, n1i = n_treatment,
              ci = events_control,  n2i = n_control,
              data = tb_studies)

fit_dl   <- rma(yi, vi, data = dat, method = "DL")    # DerSimonian-Laird
fit_reml <- rma(yi, vi, data = dat, method = "REML")  # restricted max. likelihood
summary(fit_reml)
```

The choice of τ^2 estimator matters more than most analysts assume. DerSimonian-Laird (DL) is a moment-based estimator: fast, closed-form, and historically the default in most software — but it is known to understate τ^2, and hence understate the width of the pooled confidence interval, when the number of studies is small or effect sizes are binary with rare events. REML (restricted maximum likelihood) is iterative, slightly more computationally demanding, and generally preferred in current methodological guidance because it produces less biased variance estimates, particularly with few studies — a near-universal situation in TB programme meta-analyses, which rarely pool more than ten to fifteen studies. Where you have the choice, report REML as the primary random-effects estimate and DL as a sensitivity check, not the reverse.

HEOR in practice. When deciding fixed vs. random effects, apply the same model-selection discipline you used for parametric survival curves in Chapter 10: don't pick the model that gives the result you expected. Instead, triangulate on (a) clinical/programmatic plausibility — do you genuinely believe every included study shares one true effect, or do differing TB prevalence, drug-resistance patterns, and health-system settings make that implausible on its face; (b) a visual check of the forest plot for overlapping confidence intervals; and (c) the heterogeneity statistics covered in §11.2. Pre-register this choice in your analysis plan before you see the pooled numbers, exactly as you would pre-specify a survival distribution family before fitting.

Agency perspective. NICE DSU Technical Support Document 1: "Introduction to Evidence Synthesis for Decision Making" (Dias et al., 2011) is the methods document that most HTA reviewers will hold your meta-analysis against before it even reaches a decision model. It sets out exactly the random-effects-as-default position taken above: TSD 1 states that a fixed-effect model should only be used when there is a specific, justified reason to believe all studies share a single true effect, and that this justification must be stated explicitly in the submission, not assumed by default. Writing "a random-effects model was used because a fixed effect was not clinically plausible across studies conducted in different settings and years" in your methods section is the kind of sentence TSD 1 is checking for.

Common pitfall. Running both `metabin()`/`metagen()` and `rma()` on the same dataset and reporting whichever gives a narrower confidence interval or a "significant" pooled effect is p-hacking by model selection, even if each individual call is statistically correct. Decide your effect model (fixed vs. random) and your τ^2 estimator **before** inspecting results, and if you must report sensitivity analyses across estimators, label them explicitly as sensitivity analyses in your methods section, not as alternative headline results.

11.2 Heterogeneity and its quantification

Heterogeneity is simply the technical word for "the studies don't agree with each other more than sampling noise alone would explain." Quantifying it matters for two practical reasons: it tells you whether a single pooled number is even a sensible summary of your evidence base, and it determines how much extra uncertainty the random-effects model should add to your confidence interval before that number reaches a budget-impact model or a costing decision.

Three statistics do the work, and each answers a slightly different question. Q is a weighted sum of how far each study's effect sits from the pooled effect. If there were truly no heterogeneity, Q would follow a known chi-squared distribution — so a large Q relative to its degrees of freedom ($k - 1$, where k is the number of studies) signals heterogeneity. Q's weakness: with few studies (common in TB settings) it has low power to detect real heterogeneity, and with many studies it is almost always "significant" even when the true heterogeneity is clinically trivial. Don't read its p-value in isolation. τ^2 is the estimated **variance** of the true effects across studies, on the same scale as your effect measure (log-RR, mean difference, SMD). It is the number random-effects models actually use to widen standard errors, but its raw value is hard to interpret because that scale changes from analysis to analysis. I^2 solves that readability problem: it rescales the same information into a percentage — the share of total variability that is real between-study disagreement rather than sampling noise. That is why I^2 is the number most often quoted.

Difficult terminology: chi-squared distribution. The chi-squared distribution is a standard, well-tabulated probability distribution that describes how much random "noise" alone would produce, if you added up a bunch of squared deviations that each had no real pattern behind them. It comes up constantly as the reference yardstick for "is this amount of spread bigger than chance would explain" tests. Here, it's the yardstick for the Q-statistic: if every study were truly estimating the same effect and all the disagreement you see between studies were just the play of random sampling, Q would behave like a draw from a chi-squared distribution with $k - 1$ degrees of freedom. If the actual Q you calculated is bigger than what that distribution would typically produce, that's evidence the studies disagree by more than chance alone — i.e., genuine heterogeneity. It's the same logic as a chi-squared test comparing observed versus expected counts in a 2x2 table, just applied to study effects instead of cell counts.

A widely used rule of thumb, drawn from the Cochrane Handbook's guidance, treats I^2 values roughly as: 0-40% might not be important; 30-60% may represent moderate heterogeneity; 50-90% may represent substantial heterogeneity; 75-100% considerable heterogeneity — with the explicit caveat that these bands overlap deliberately, because the importance of any given I^2 depends on

the magnitude and direction of effects and the strength of evidence for heterogeneity (i.e. look at Q's p-value and the confidence interval around I^2 too, not the point estimate alone).

Both `meta` and `metafor` report all three statistics by default when you call `summary()` on a fitted object, and both provide forest plots as the standard visual diagnostic:

```
summary(m_bin)     # reports Q, tau^2, I^2, and their CIs alongside the pooled
estimate
forest(m_bin,
       sortvar = TE,          # order studies by effect size
       prediction = TRUE)     # adds a prediction interval for a new study's true
effect

# metafor equivalent
summary(fit_reml)
forest(fit_reml)
```

> **Difficult terminology: prediction interval (vs. confidence interval).** A confidence interval tells you how precisely you've pinned down the **average** true effect, given the studies you pooled — it narrows as you add more studies, because you're estimating one fixed number ever more precisely. A prediction interval answers a different, often more clinically relevant question: "if a brand-new study, not yet run, estimated this same effect, what range would its true effect plausibly fall in?" Because individual studies' true effects genuinely vary (that's the heterogeneity this section is about), a prediction interval has to be wider than the confidence interval — it accounts for both the uncertainty in the average and the real study-to-study spread around that average. Concretely: if a meta-analysis's pooled risk ratio is 0.70 with a confidence interval of 0.65-0.75, the corresponding prediction interval might be 0.40-1.20 — the average effect is well pinned down, but any one new setting (a new country, a new TB programme) could plausibly see anywhere in that wider range. Quoting only the confidence interval when heterogeneity is substantial can make the evidence look far more consistent and generalisable than it really is.

The forest plot remains the single most useful diagnostic in this chapter: each row is one study's point estimate and confidence interval, sized by weight in the pooled analysis, with the diamond at the bottom showing the pooled estimate and its interval. Read it before you read any summary statistic — a cluster of overlapping intervals with one clear outlier tells a different, more actionable story than a diffuse spread of non-overlapping intervals, even when both produce a similar I^2.

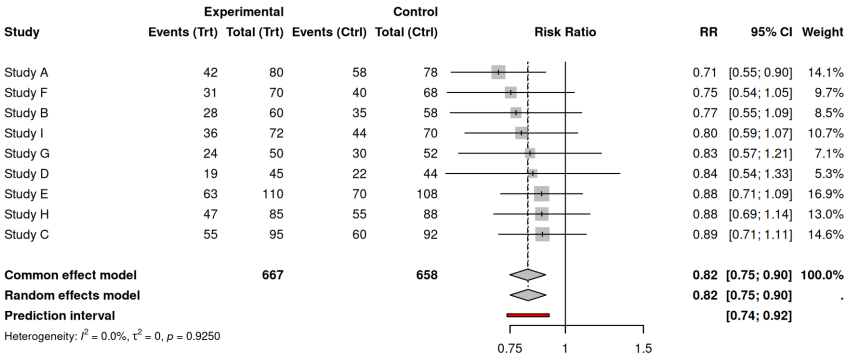


Figure 16: A forest plot from `meta::forest()` on a nine-study toy risk-ratio meta-analysis: one row per study with its weight-sized confidence interval, the pooled random-effects diamond, and a prediction interval for a new study's true effect — the single most important figure this chapter produces.

Agency perspective. The Cochrane Handbook for Systematic Reviews of Interventions, Chapter 10 ("Analysing data and undertaking meta-analyses"), is the standard methodological reference underpinning essentially every regulatory or HTA submission's heterogeneity reporting: it specifies that I^2, τ^2, and the Q-statistic (with its p-value) should all be reported together, not I^2 alone, and that a prediction interval should accompany the pooled estimate whenever random-effects heterogeneity is non-trivial. If your meta-analysis is destined for an HTA dossier, report in that triad rather than cherry-picking the one statistic that looks most reassuring.

How to report this.

Study	n (T/C)	RR [95% CI]	Weight (%)
Site 1	120/118	0.82 [0.61, 1.10]	14.2
Site 2	95/101	0.74 [0.52, 1.05]	11.6
Site 3	210/205	0.91 [0.73, 1.13]	18.9
Site 4	88/90	0.65 [0.44, 0.96]	9.8
Site 5	150/147	0.88 [0.68, 1.14]	15.7
…	…	…	…
Pooled (random-effects, REML)		0.83 [0.72, 0.96]	100
Heterogeneity: $Q = 12.4$ (df = 8, $p = 0.13$); $\tau^2 = 0.015$; $I^2 = 35.4\%$ (95% CI: 0, 68%)			

Table 30: How to report this: the per-study and pooled effect sizes that accompany a forest plot, together with the heterogeneity statistics (Q, τ^2, I^2) that must be quoted alongside the pooled estimate in an HTA submission

11.3 Publication bias and small-study effects

Publication bias is the concern that studies with "positive," statistically significant, or programmatically convenient findings are more likely to be published, and hence more likely to end up in your meta-analysis, than null or unfavourable results sitting in a file drawer somewhere. Small-study effects are the closely related, more general phenomenon where smaller studies systematically report larger effect sizes than large studies do — which can arise from publication bias but also from genuine clinical heterogeneity (e.g. smaller pilot studies run in more responsive sub-populations) or from methodological shortcuts taken under smaller budgets. You cannot distinguish these causes definitively from the data alone, so treat any positive test below as a flag for further investigation, not as a diagnosis.

The funnel plot is the standard visual tool: each study's effect estimate is plotted against a measure of its precision (usually standard error, inverted so precise studies sit near the top). In the absence of bias, the scatter should look like a roughly symmetric inverted funnel, widening at the bottom where smaller, less precise studies naturally spread further from the pooled effect. Asymmetry — a gap in one bottom corner where small studies with unfavourable results "should" be but aren't — is the visual signature of concern.

```
# meta package
funnel(m_bin)
metabias(m_bin, method.bias = "linreg")   # Egger's regression test

# metafor package
funnel(fit_reml)
regtest(fit_reml, model = "lm")           # Egger's test equivalent
```

Egger's test formalizes the visual check: it regresses the standardized effect estimate against its precision (or a closely related parameterization) and tests whether the intercept differs from zero, which would indicate asymmetry. `metabias()` in `meta` exposes this (and several other bias tests) via its `method.bias` argument; `regtest()` in `metafor` is the direct analogue. Both tests share the same core limitation: they require a reasonable number of studies (conventionally at least ten) to have any meaningful power, which rules them out for most TB programme meta-analyses pooling five or six site-level studies. Running Egger's test on eight studies and reporting a non-significant p -value as "no evidence of publication bias" overstates what the test can tell you at that sample size.

Trim-and-fill, available as `trimfill()` in both `meta` and `metafor`, offers a complementary sensitivity analysis: it iteratively removes the most extreme small studies causing funnel-plot asymmetry, uses the trimmed, symmetric subset to estimate the true center, then imputes ("fills") mirror-image studies back in to restore symmetry and re-estimates the pooled effect from the augmented dataset. The adjusted estimate is not a corrected "true" answer — it is a what-if sensitivity check showing how much the pooled effect might shift if the suspected missing studies existed and looked like their mirror images. Report it alongside, never instead of, the unadjusted pooled estimate.

```
tf <- trimfill(m_bin)
summary(tf)
funnel(tf)   # visualize imputed studies alongside observed ones
```

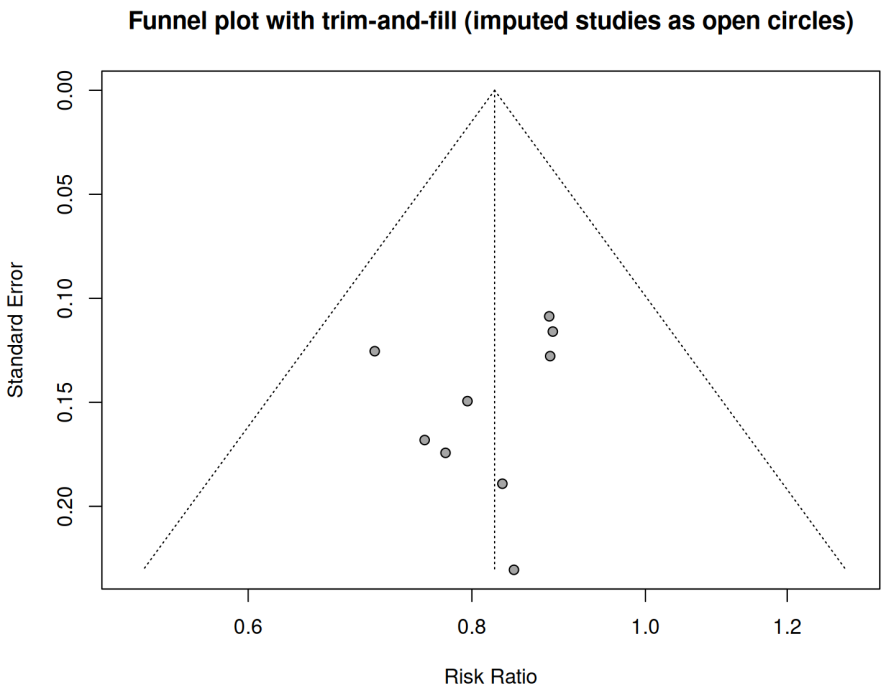


Figure 17: A funnel plot with trim-and-fill applied to the chapter's toy meta-analysis. Observed studies and any imputed mirror-image studies added by `trimfill()` to restore symmetry are both shown — the adjusted pooled estimate from this augmented dataset is a sensitivity check, not a corrected "true" answer.

Common pitfall. Do not apply Egger's test or trim-and-fill mechanically whenever you have a funnel plot available. With fewer than ten studies — the norm for TB-programme-specific

meta-analyses — both methods have poor power and can produce misleading corrections driven by one or two outlying studies rather than genuine bias. In that situation, describe the funnel plot qualitatively, flag the limitation explicitly in your write-up, and avoid reporting a trim-and-fill-adjusted point estimate as if it were more trustworthy than the original.

How to report this.

Analysis	k (studies)	Pooled RR [95% CI]	I^2
Observed studies only (random-effects, REML)	9	0.83 [0.72, 0.96]	35.4%
Trim-and-fill adjusted (3 imputed studies)	12	0.88 [0.76, 1.02]	41.7%
Egger's regression test: intercept = 1.42 (SE 0.58), $p = 0.04$, consistent with funnel-plot asymmetry; trim-and-fill shifts the pooled estimate toward the null but does not overturn it — reported as a sensitivity check, not a corrected estimate.			

Table 31: How to report this: sensitivity of the pooled random-effects estimate to the trim-and-fill adjustment for funnel-plot asymmetry, shown before (observed studies only) and after (observed plus imputed mirror-image studies) alongside Egger's test for small-study effects

11.4 Meta-regression

When a forest plot shows wide variation in effect sizes across studies, the natural next question is **why**. Meta-regression extends the random-effects model by adding one or more study-level predictors ("moderators") that might explain some of that between-study heterogeneity — things like mean patient age, baseline disease severity, year of publication, or dosing regimen. It is the meta-analytic cousin of ordinary regression, except the unit of analysis is the study, not the patient, and each study's contribution is weighted by its precision.

In `metafor`, this is done with the same `rma()` function already familiar from §11.1, but supplying the `mods=` argument:

```
library(metafor)

# dat has yi (effect size, e.g. log-RR), vi (sampling variance),
# and a study-level moderator, mean_age
res_mod <- rma(yi, vi, mods = ~ mean_age, data = dat, method = "REML")
summary(res_mod)
```

`mods=` accepts a formula, so multiple moderators, factors, and interactions all work exactly as in `lm()`: `mods = ~ mean_age + region`. The output partitions the total heterogeneity into the portion explained by the moderator(s) (reported as a test of moderators, `QM`) and what remains (the residual heterogeneity, still summarised by `tau^2` and `I^2`, now relabelled residual heterogeneity). A moderator that soaks up a large share of `QM` relative to the overall `Q` is doing real explanatory work; one that leaves the residual heterogeneity almost untouched is not.

How to report this.

Moderator	Coefficient	SE	95% CI	p
Intercept	−0.41	0.15	(-0.70, −0.12)	0.006
Mean age (per 10 years)	0.09	0.04	(0.01, 0.17)	0.028
Region: high TB-burden (ref: low)	0.18	0.11	(-0.04, 0.40)	0.108
Test of moderators: $Q_M = 6.8$ (df = 2, $p = 0.033$); residual heterogeneity $\tau^2 = 0.009$ (down from 0.015 in the unadjusted model), residual $I^2 = 22.1\%$.				

Table 32: How to report this: meta-regression coefficients from `metafor::rma(mods = ~ mean_age + region)`, the conventional way to present which study-level moderators explain between-study heterogeneity and by how much

Meta-regression is appropriate when two conditions hold together. First, there need to be **enough studies** — as a rule of thumb, at least ten studies per moderator is the floor below which the regression coefficients become unstable and prone to chance findings; many TB programme meta-analyses sit well under this, which should temper ambition. Second, the covariates should be **pre-specified** in a protocol, not fished for after seeing which split of the forest plot looks tidiest. Testing every variable in the dataset against the effect size until something is significant is simply multiple testing in a performance-enhancing dress, and the HEOR submission will not survive scrutiny if the moderator choice cannot be justified on clinical or methodological grounds set out before the data were pooled.

Common pitfall. Meta-regression coefficients describe **between-study** associations, not **within-study** (individual-patient) associations — this is ecological bias. A meta-regression that finds "trials with older average age show smaller treatment effects" says nothing reliable about whether an individual older patient benefits less; the studies' age **averages** are confounded with everything else that differs systematically between those trials (settings, era, comorbidity mix, co-interventions). Do not present aggregate-level moderator findings as if they were patient-level subgroup effects in a submission — regulators and HTA reviewers will flag this immediately, and Cochrane guidance is explicit that such inferences require individual patient data (IPD) meta-analysis, not aggregate meta-regression.

With few studies, meta-regression also has very low statistical power — a null result for a moderator with eight trials is barely informative, and should be reported as inconclusive rather than as evidence the moderator does not matter.

11.5 Meta-analysis of time-to-event outcomes

Many of the outcomes that matter most in TB and HIV programmes — time to treatment failure, time to death, time to culture conversion — are naturally time-to-event (survival) outcomes, and trials typically report them as hazard ratios (HRs) with confidence intervals, not as simple risk ratios. Pooling HRs across studies needs a small but important adjustment: HRs are ratios, bounded

below by zero and skewed, so they are pooled on the **log scale**, where the sampling distribution is much closer to normal.

The practical recipe is: for each study, compute `log(HR)` as the effect estimate and derive its standard error from the reported confidence interval (if a 95% CI is `(L, U)`, then `SE = (log(U) - log(L)) / (2 * 1.96)`), then pool these log(HR) values with inverse-variance weighting exactly as any other continuous effect size. The `meta` package's `metagen()` function does this cleanly and labels the output in the natural HR scale automatically:

```
library(meta)

# dat has studlab, TE (= log HR per study), seTE (SE of log HR)
m_hr <- metagen(
  TE = TE,
  seTE = seTE,
  studlab = studlab,
  data = dat,
  sm = "HR",          # tells meta the pooled estimate is a hazard ratio
  method.tau = "REML"
)
summary(m_hr)
forest(m_hr)
```

Setting `sm = "HR"` is purely cosmetic in terms of the arithmetic (the pooling still happens on the log scale you supplied), but it tells `metagen()` to back-transform the summary estimate and its CI with `exp()` before printing, so the forest plot and output table report a pooled HR with sensible axis labelling rather than a pooled log(HR) that needs manual conversion. If your source studies instead report `log(HR)` and its SE directly (common in trial supplements), feed those straight in; if they only give you the HR point estimate and CI, derive `TE` and `seTE` from them first as above.

This pooled HR is not the end of the analysis in an HEOR context — it is usually an **input** to the next stage. Chapter 10 built the survival-model toolkit: fitting the `flexsurv` six-distribution family to a reference arm's individual patient-level survival data, selecting a parametric form by AIC/BIC together with visual fit and clinical plausibility, and extrapolating the fitted curve beyond the trial's observed follow-up. A pooled HR from meta-analysis commonly becomes the multiplier applied to that extrapolated reference-arm survival curve to generate the comparator arm's extrapolated survival — the "HR method" of indirect extrapolation used throughout cost-effectiveness modelling. The meta-analysis in this section therefore feeds directly into the machinery of the previous chapter: the pooled `log(HR)` and its uncertainty become a parameter (often itself distributed, e.g. log-normal, in a probabilistic sensitivity analysis) multiplying the hazard function of whichever `flexsurv` distribution was selected for the reference arm in Chapter 10.

> **Agency perspective.** Before pooling HRs across trials for use in an extrapolation, check the Cochrane Handbook for Systematic Reviews of Interventions, Chapter 6 (on choosing effect measures) for its guidance on extracting hazard ratios consistently across studies that report time-to-event results in different formats (Kaplan-Meier curves only, HR with CI, HR with p-value only, or number of events and person-time). The Handbook sets out a hierarchy of extraction methods — a directly reported HR and SE being preferable to an HR reconstructed from a digitised KM curve via the Guyot et al. (2012) algorithm already introduced in Chapter

10 §10.11 — so do not silently treat a reconstructed HR as equivalent in precision to a directly reported one when assembling the pooled estimate that will drive an extrapolation.

11.6 Diagnostic test accuracy meta-analysis

Pooling sensitivity and specificity separately is tempting but wrong, because the two measures are not independent: a laboratory or study that reads a diagnostic threshold more liberally will tend to report higher sensitivity and lower specificity together, and vice versa. Any meta-analysis of diagnostic accuracy — highly relevant to TB programmes evaluating a new rapid test (Xpert, LF-LAM, a novel biomarker assay) against a reference standard like culture — needs a model that respects this negative correlation rather than pooling sensitivity and specificity as if they were two unrelated outcomes.

The standard solution is the bivariate random-effects model, which jointly models the logit-transformed sensitivity and logit-transformed specificity across studies, allowing for a between-study correlation between the two. In R, the dedicated package for this is `mada` ("meta-analysis of diagnostic accuracy"), and its core function is `reitsma()`:

```
library(mada)

# dat has TP, FN, FP, TN counts for each study against the reference standard
fit <- reitsma(dat)        # dat needs columns named TP, FN, FP, TN
summary(fit)

# descriptive diagnostic stats (sens, spec, likelihood ratios) per study
desc <- madad(dat)
print(desc)

# SROC curve with confidence and prediction regions
plot(fit, sroc.type = "ruttergates")
points(fpr(desc), sens(desc), pch = 2)
```

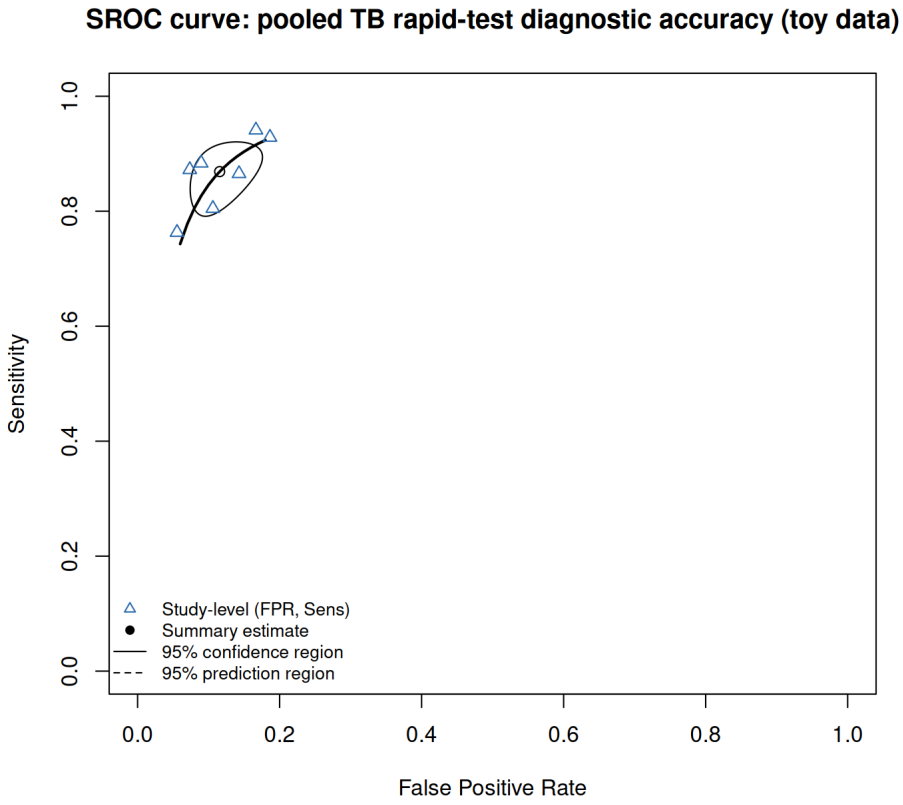


Figure 18: An SROC curve from `mada::reitsma()` fitted to a toy TB rapid-diagnostic-test dataset: study-level sensitivity/false-positive-rate pairs (triangles), the pooled summary point, and the 95% confidence and prediction regions — the bivariate analogue of the pairwise forest plot.

`reitsma()` fits the bivariate model (a generalised linear mixed model under the hood, modelling the logit-sensitivity/logit-specificity pair with a bivariate normal random effect) and returns pooled sensitivity and specificity with their joint uncertainty. The SROC curve (summary receiver operating characteristic curve) traces out the implied trade-off between sensitivity and specificity across the range of thresholds/studies, and is the standard visual summary for a diagnostic accuracy review — it plays a role analogous to the forest plot for pairwise meta-analysis, but captures the two correlated axes at once. `metafor::rma.mv()` can also fit an equivalent bivariate model by hand (specifying the sensitivity/specificity logits as a bivariate outcome with a random study effect), which gives more flexibility for complex covariate structures, but `mada::reitsma()` is the more direct and purpose-built route for a standard DTA review.

Difficult terminology: generalised linear mixed model. A "mixed model" is a regression that mixes two kinds of effects: **fixed** effects, which are shared, overall parameters you want to estimate once for everybody (e.g. "the average sensitivity across all studies"), and **random** effects, which let each individual study (or patient, or site) wobble around that average in its own way, with the size of that wobble itself estimated from the data. "Generalised" means the outcome doesn't have to be a plain continuous, normally-distributed number — here it is a logit-transformed proportion (sensitivity/specificity), so the model works on that transformed, bounded scale, the same trick as the logit link used elsewhere in the book. Picture it as: every

study gets to have its own slightly different "true" sensitivity and specificity (the random part), but all of those study-specific values are assumed to be draws clustered around one shared underlying pair of numbers (the fixed part) — that's what lets `reitsma()` borrow strength across studies while still respecting that no two labs read a diagnostic threshold identically.

Agency perspective. Diagnostic test accuracy reviews have their own reporting standard, the PRISMA-DTA extension (Preferred Reporting Items for Systematic Reviews and Meta-Analyses of Diagnostic Test Accuracy studies), published in **JAMA** in 2018. Before finalising a DTA meta-analysis for a TB diagnostics submission, check the PRISMA-DTA checklist for required elements — notably the explicit reporting of the reference standard used in each included study, handling of indeterminate results, and threshold variability across studies — since these affect whether pooling sensitivity/specificity across studies is even defensible in the first place.

How to report this.

Parameter	Estimate	95% CI (low)	95% CI (high)
Pooled sensitivity	0.89	0.82	0.94
Pooled specificity	0.91	0.85	0.95
Positive likelihood ratio	9.9	5.8	16.9
Negative likelihood ratio	0.12	0.07	0.20
Diagnostic odds ratio	82.5	34.1	199.6
Bivariate model: 11 studies, correlation between logit-sensitivity and logit-specificity $\rho = -0.42$			

Table 33: How to report this: pooled sensitivity and specificity from the bivariate `mada::reitsma()` model, the standard summary table accompanying an SROC curve in a diagnostic test accuracy review

Difficult terminology: positive/negative likelihood ratio. These two numbers translate a test's sensitivity and specificity into something more directly useful at the bedside: how much a result should shift your suspicion of disease. The **positive** likelihood ratio (here, 9.9) says a positive test result is about 9.9 times more likely to occur in someone who truly has the disease than in someone who doesn't — the higher above 1, the more a positive result should raise your suspicion. The **negative** likelihood ratio (here, 0.12) says a negative result is still fairly likely in diseased patients relative to non-diseased ones when it's far from 0 — the lower below 1 (closer to 0), the more a negative result should reassure you. Unlike sensitivity/specificity, likelihood ratios can be combined directly with a patient's pre-test odds of disease (via Bayes' theorem) to get a post-test odds, which is why clinicians favour them for bedside reasoning about an individual patient, rather than sensitivity/specificity alone.

Difficult terminology: diagnostic odds ratio. The diagnostic odds ratio folds sensitivity and specificity into one single summary number: the odds of a positive test result in diseased patients, divided by the odds of a positive test result in non-diseased patients. A value of 1 means the test is useless (positive results are just as likely whether or not someone has the disease); the TB rapid test's value of 82.5 here means a positive result is, in odds terms, about 82 times more likely among people who truly have TB than among people who don't — a reasonably strong-performing test. It's convenient for ranking or comparing tests with one number, but because it blends sensitivity and specificity together, two tests with very different clinical trade-offs (one highly sensitive but less specific, one the reverse) can end up with a similar diagnostic odds ratio — so it should supplement, not replace, looking at sensitivity, specificity, and the likelihood ratios separately.

HEOR in practice. When several candidate rapid tests are being compared against the same reference standard, consider whether a network meta-analysis of diagnostic accuracy is more appropriate than separate pairwise bivariate models — this lets indirect comparisons borrow strength across the evidence network, which matters when head-to-head trials between two rapid tests are scarce, as is common in TB diagnostics. Chapter 12 develops network methods in full.

11.7 Meta-analysis of proportions and rates

A large share of the meta-analyses a TB programme analyst will actually run are not comparative at all — there is no treatment arm versus control arm, just a single proportion (prevalence of drug resistance, treatment success rate, loss-to-follow-up rate) or a single rate (incidence per person-year) estimated in multiple studies or multiple programme sites, which need to be pooled into one overall estimate. This is meta-analysis of a **single group** summary statistic, and it needs its own variance-stabilising transformations because raw proportions and rates have variances that depend on their own mean, violating the constant-variance assumption behind simple inverse-variance pooling.

For proportions bounded between 0 and 1 (e.g. proportion of TB patients with a positive resistance test), two transformations are standard. The logit transformation maps the proportion to the whole real line and works well away from the boundaries:

```
library(metafor)

# dat has xi (events, e.g. resistant cases) and ni (total tested) per study
dat_logit <- escalc(measure = "PLO", xi = xi, ni = ni, data = dat)
res_logit <- rma(yi, vi, data = dat_logit, method = "REML")
predict(res_logit, transf = transf.ilogit)   # back-transform to a proportion
```

When some studies have proportions near 0 or 1 (common with rare events, such as a resistance pattern seen in only a handful of patients), the logit transform can behave poorly, and the double-arcsine (Freeman-Tukey) transformation is the usual alternative, handled in `metafor::escalc()` with `measure = "PFT"`:

```r
dat_pft <- escalc(measure = "PFT", xi = xi, ni = ni, data = dat)
res_pft <- rma(yi, vi, data = dat_pft, method = "REML")
predict(res_pft, transf = transf.ipft.hm, ni = dat_pft$ni)  # back-transform
```

For rates (events per unit person-time, e.g. TB incidence per 100,000 person-years), the log transformation is the natural equivalent, available via `measure = "IRLN"` (incidence rate, log-transformed):

```r
# dat has xi (events) and ti (total person-time) per study/site
dat_rate <- escalc(measure = "IRLN", xi = xi, ti = ti, data = dat)
res_rate <- rma(yi, vi, data = dat_rate, method = "REML")
predict(res_rate, transf = exp)   # back-transform to events per person-time unit
```

In every case the pattern is the same: transform each study's proportion or rate onto a scale where a normal approximation is reasonable, pool on that transformed scale with the usual random-effects machinery (`rma()`), then back-transform the pooled estimate and its confidence interval with `predict(..., transf = ...)` for reporting. Never average the raw proportions or rates directly and never report the pooled estimate still on the transformed scale — logit or arcsine units mean nothing to a programme manager reading the report.

Common pitfall. High heterogeneity (I^2 often exceeding 90%) is the norm rather than the exception when pooling prevalence or incidence across sites, because true prevalence genuinely differs by geography, population risk, and surveillance intensity — this is not necessarily a sign of a broken model. Resist the temptation to report a single pooled prevalence figure as if it applied uniformly to a national TB programme when the underlying studies are clearly measuring different underlying populations; a forest plot showing the spread, alongside the pooled estimate, communicates the true picture far better than the summary number alone, and subgroup pooling by region or risk group is usually more decision-relevant than one overall figure.

Agency perspective. CHEERS 2022 (Husereau et al., **Value in Health**, 2022), the reporting checklist already established in Chapter 9, item 15 requires that any pooled clinical input to a cost-effectiveness model — including a pooled prevalence or rate feeding a Markov transition probability in a later chapter — state the synthesis method, the heterogeneity statistics, and the handling of any ties to the decision model explicitly. A pooled proportion quoted in a later chapter's Markov model without a visible trail back to this section's method and heterogeneity reporting does not meet that bar.

Chapter 12. Indirect Treatment Comparisons and Network Meta-Analysis

Chapter 11 closed with pooling evidence across several trials of the **same** two treatments. This chapter opens a harder, far more common HTA problem: what happens when no trial has ever compared the two treatments the reimbursement decision actually hinges on? The answer, across every method in this chapter — Bucher's two-trial algebra, frequentist network meta-analysis, Bayesian network meta-analysis, consistency checking, network meta-regression, and survival-outcome networks — is the same single idea at increasing scale: evidence can be combined **across** trials only through a shared comparator, never by comparing absolute outcomes directly.

12.1 Why indirect comparison is needed

Chapter 11 built the machinery for combining multiple head-to-head trials of the **same** two treatments into one pooled estimate — fixed-effect and random-effects pooling, I^2 and τ^2 as the heterogeneity vocabulary, REML as the default τ^2 estimator. That machinery assumes something that, in HTA submissions, is rarely true: that every trial relevant to your decision problem compared the same two arms.

In reality, a reimbursement dossier for a new TB regimen almost never has a trial that pairs the new regimen directly against every comparator a health technology assessment (HTA) body cares about. A manufacturer trial might show Regimen A (a novel bedaquiline-based combination) against the WHO-recommended standard-of-care regimen C in a multidrug-resistant TB population. A separate, unrelated trial — run by a different sponsor, in a different country, years apart — might show Regimen B (a delamanid-based combination) against that same standard-of-care regimen C. Nobody has ever randomised A against B. Yet the reimbursement question on the table is precisely: **how does A compare to B?** — because B is the comparator already funded in the health system, or the one the HTA body has designated as the relevant comparator in its final scope.

This is the indirect treatment comparison (ITC) problem, and it is endemic to HTA rather than a rare edge case. Guidelines rarely allow a manufacturer to simply report the two trials' results side by side and let a reviewer "eyeball" the difference — that approach has no defensible statistical basis and discards exactly the information that randomisation was designed to protect: comparability of patient populations within each trial. Naive, unadjusted comparison of absolute outcomes across trials (sometimes called a "naive indirect comparison") ignores the fact that trial populations, eras, and measurement methods differ, and it will produce biased and overconfident answers. What is needed is a method that uses randomised evidence **within** each trial, and only combines **between** trials through the shared control arm.

> **Common pitfall.** Never compare absolute outcomes from unrelated trials directly (e.g. "Trial 1 found 78% cure with Regimen A, Trial 2 found 71% cure with Regimen B, therefore A is better"). This "naive" or "unadjusted" indirect comparison destroys randomisation and is highly vulnerable to confounding by differences in baseline risk, trial era, region, and outcome

definition between the two trials. It is explicitly flagged as invalid practice in NICE DSU TSD 2 (Dias et al. 2011) and will not be accepted as the primary analysis in an HTA submission.

The conceptual fix — formalised by Bucher, Guyatt, Griffith and Walter (1997) and now the foundation of the entire ITC/NMA literature — is to exploit a **common comparator**. If trial 1 randomised patients to A vs C, and trial 2 (independently) randomised patients to B vs C, then each trial still gives you a valid, randomisation-protected estimate of a relative treatment effect: the A-vs-C effect from trial 1, and the B-vs-C effect from trial 2. Because both effects are expressed relative to the **same** comparator C, you can algebraically combine them to obtain an estimate of the A-vs-B effect — without ever having observed a trial that randomised A against B directly. C in this role is called the **anchor** or **common comparator**, and the approach is accordingly called an **anchored indirect comparison**.

This single idea — anchoring through a common comparator rather than comparing absolute arms — is what separates a defensible ITC from a naive one, and it is the thread that runs through this entire chapter: first in its simplest two-trial form (the Bucher method, §12.2), then generalised to networks of many trials and many treatments (frequentist network meta-analysis, §12.3), and finally in its Bayesian implementation.

Learning objectives

- Explain why within-trial randomisation must be preserved when evidence comes from more than one trial, and why naive cross-trial comparison of absolute outcomes is invalid
- Describe the anchor-based logic of the Bucher method: deriving an A-vs-B estimate from independent A-vs-C and B-vs-C trials via a common comparator C
- Calculate a Bucher indirect estimate and its variance by hand, understanding why the variances of the two component trials **add** rather than subtract
- Distinguish an indirect comparison from a mixed treatment comparison, and know when each applies
- Fit a frequentist network meta-analysis in R with the `netmeta` package, using `netmeta()`, `netgraph()`, `netrank()`, and `netleague()`
- Interpret a network geometry plot, a P-score ranking, and a league table in the context of an HTA evidence network
- Fit a Bayesian network meta-analysis with `gemtc`, run MCMC diagnostics, and check convergence before reporting any posterior summary
- Explain and run node-splitting and design-by-treatment interaction tests to assess network consistency
- Extend an NMA to a network meta-regression, and recognise the ecological-bias caveat that applies with even more force at network level
- Describe how NMA of survival outcomes generalises the Chapter 11 pooled-log(HR) mechanism across a network, and understand the limits of off-the-shelf tools for fractional polynomial time-varying hazard ratios
- Rank treatments across a network with SUCRA (Bayesian) and P-scores (frequentist), and explain why a ranking metric must always be reported alongside, never instead of, the underlying effect estimates and their uncertainty
- Report a network meta-analysis per NICE DSU TSD 2, the ISPOR good-practice task force report, and the PRISMA-NMA extension to the systematic review checklist

12.2 Bucher method

12.2.1 The algebra of anchored indirect comparison

Suppose we have two independent, two-arm randomised trials, both reporting a log hazard ratio (or log odds ratio, or mean difference — the algebra is identical on any additive scale):

- Trial 1 (A vs C): estimate $\hat{d}_{AC}$, with standard error $\mathrm{SE}\left(\hat{d}_{AC}\right)$
- Trial 2 (B vs C): estimate $\hat{d}_{BC}$, with standard error $\mathrm{SE}\left(\hat{d}_{BC}\right)$

Both $\hat{d}_{AC}$ and $\hat{d}_{BC}$ are valid, randomisation-protected estimates — each trial's internal randomisation ensures that, within that trial, the comparison of its two arms is unconfounded. The Bucher method combines them into an indirect estimate of the A-vs-B effect:

$$\hat{d}_{AB}^{\text{indirect}} = \hat{d}_{AC} - \hat{d}_{BC}$$

This is simply the statement that if A is $\hat{d}_{AC}$ better than C, and B is $\hat{d}_{BC}$ better than C, then A is $\left(\hat{d}_{AC} - \hat{d}_{BC}\right)$ better than B. The subtraction is on the **treatment-effect scale** (log HR, log OR, mean difference) — it is not a subtraction of absolute outcome levels, and it still relies on nothing but the two within-trial comparisons.

12.2.2 Why the variance adds

This is the detail most often got wrong by newcomers, and it is worth dwelling on precisely because the point estimate is a **subtraction** while the variance is a **sum**:

$$\mathrm{Var}\left(\hat{d}_{AB}^{\text{indirect}}\right) = \mathrm{Var}\left(\hat{d}_{AC}\right) + \mathrm{Var}\left(\hat{d}_{BC}\right)$$

or equivalently, in standard-error form:

$$\mathrm{SE}\left(\hat{d}_{AB}^{\text{indirect}}\right) = \sqrt{\mathrm{SE}\left(\hat{d}_{AC}\right)^2 + \mathrm{SE}\left(\hat{d}_{BC}\right)^2}$$

The reason is that $\hat{d}_{AC}$ and $\hat{d}_{BC}$ come from two **statistically independent** trials — different patients, different randomisation, no shared data. For independent random variables, the variance of their difference is the sum of their individual variances, exactly as it is for their sum: $\mathrm{Var}(X - Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$ when X and Y are independent, regardless of the sign in front of Y. The minus sign affects the **expectation** of the combination (point estimate) but not its **variance**, because $\mathrm{Var}(-Y) = \mathrm{Var}(Y)$.

> **Common pitfall.** The single most common manual error in Bucher calculations is subtracting the two variances (or SEs) by analogy with the point-estimate subtraction, producing an impossibly narrow — sometimes negative — variance. The correct operation is ALWAYS addition of variances, never subtraction, because the two trials are independent sources of uncertainty and uncertainty does not cancel when you combine independent estimates, even when the point estimates themselves are combined by subtraction.

The practical consequence is important for interpretation: an indirect estimate is **always** less precise than either of its two component direct estimates, because you are paying the "price" of both trials' sampling uncertainty, with no opportunity for cancellation. A 95% confidence interval for the indirect A-vs-B log-effect is then

$$\hat{d}_{AB}^{\text{indirect}} \pm 1.96 \times \text{SE}\left(\hat{d}_{AB}^{\text{indirect}}\right)$$

exponentiated back to the natural scale (hazard ratio, odds ratio) if that is how the trials originally reported their effects.

12.2.3 Worked example: TB regimen comparison

Consider an HTA submission for Regimen A, a novel multidrug-resistant TB (MDR-TB) treatment, where the decision-relevant comparator is Regimen B, an alternative novel regimen already used in some health systems. No trial has randomised A against B. However:

- **Trial 1** randomised MDR-TB patients to Regimen A vs the WHO standard-of-care regimen C, reporting a log hazard ratio for treatment failure/relapse of $\hat{d}_{AC} = -0.52$ (favouring A) with $\text{SE} = 0.18$.
- **Trial 2** randomised a comparable MDR-TB population to Regimen B vs the same standard-of-care regimen C, reporting $\hat{d}_{BC} = -0.28$ (favouring B) with $\text{SE} = 0.21$.

The Bucher indirect estimate of A vs B is:

$$\hat{d}_{AB} = -0.52 - (-0.28) = -0.24$$

$$\text{SE}\left(\hat{d}_{AB}\right) = \sqrt{0.18^2 + 0.21^2} = \sqrt{0.0324 + 0.0441} = \sqrt{0.0765} \approx 0.2767$$

giving a 95% CI of $-0.24 \pm 1.96 \times 0.2767 = (-0.78, 0.30)$ on the log-HR scale, or a hazard ratio of $\exp(-0.24) = 0.79$ with 95% CI $(\exp(-0.78), \exp(0.30)) = (0.46, 1.35)$. The point estimate favours A, but the interval comfortably spans 1 — illustrating the precision penalty discussed above: even though each **direct** comparison (A vs C, B vs C) might individually have been reasonably precise, the indirect A-vs-B comparison inherits both trials' uncertainty and is correspondingly wider.

The following R function implements this calculation directly, with no package dependency, which is useful both as a transparent sanity check and because reviewers at HTA agencies sometimes ask to see the Bucher arithmetic laid out explicitly rather than taken from a `netmeta` object:

```
bucher_itc <- function(d_AC, se_AC, d_BC, se_BC, scale = c("log", "natural")) {
  scale <- match.arg(scale)
  d_AB  <- d_AC - d_BC
  se_AB <- sqrt(se_AC^2 + se_BC^2)
  ci_lo <- d_AB - 1.96 * se_AB
  ci_hi <- d_AB + 1.96 * se_AB

  if (scale == "natural") {
    out <- data.frame(
      estimate = exp(d_AB),
      ci_lower = exp(ci_lo),
      ci_upper = exp(ci_hi)
    )
  } else {
    out <- data.frame(
      estimate = d_AB,
      ci_lower = ci_lo,
      ci_upper = ci_hi
    )
  }
```

```
  out
}

# TB worked example: Regimen A vs C, Regimen B vs C, via common comparator C
bucher_itc(d_AC = -0.52, se_AC = 0.18, d_BC = -0.28, se_BC = 0.21, scale =
"natural")
#>    estimate  ci_lower ci_upper
#> 1 0.7866279 0.4575463 1.351880
```

How to report this.

Comparison	log HR (SE)	Hazard ratio	95% CI
A vs C (direct, Trial 1)	−0.52 (0.18)	0.59	(0.42, 0.85)
B vs C (direct, Trial 2)	−0.28 (0.21)	0.76	(0.50, 1.14)
A vs B (indirect, Bucher)	−0.24 (0.28)	0.79	(0.46, 1.35)
Note: indirect SE $= \sqrt{0.18^2 + 0.21^2} \approx 0.277$; indirect estimate favours A but the 95% CI spans 1.			

Table 34: Bucher indirect treatment comparison of Regimen A vs Regimen B for MDR-TB treatment failure/relapse, anchored on the shared standard-of-care comparator C (Trial 1: A vs C; Trial 2: B vs C). The A-vs-B row is the derived indirect estimate, not an observed trial result, and its interval is wider than either direct comparison, reflecting the added uncertainty of combining two independent trials.

12.2.4 Assumptions the Bucher method relies on

The Bucher method's algebraic simplicity hides a strong assumption that must be argued, not just asserted: **the similarity (or "transitivity") assumption**. It requires that trial 1 and trial 2 are similar enough, in every respect that could modify the relative treatment effect (effect modifiers), that the C arm in trial 1 can be treated as exchangeable with the C arm in trial 2. Concretely, if trial 1 enrolled patients with less advanced MDR-TB, used a different definition of "treatment failure", or was conducted a decade before trial 2 in a region with different background resistance patterns, the two C arms are not estimating the same quantity, and subtracting the two trial-specific effects no longer isolates a valid A-vs-B contrast — it also absorbs the difference between the trials themselves.

Agency perspective. NICE DSU Technical Support Document 2 (Dias, Welton, Sutton & Ades, 2011, **"A Generalised Linear Modelling Framework for Pairwise and Network Meta-Analysis of Randomised Controlled Trials"**) is the authoritative UK reference on this point. It formalises the similarity assumption as **transitivity** and states explicitly that an indirect or mixed comparison is only valid when trials are similar with respect to effect modifiers across the whole network — not merely "roughly comparable" on headline baseline characteristics. TSD 2 recommends that submissions present a structured assessment of cross-trial similarity (population, intervention definition, outcome definition, and era) alongside any Bucher or network estimate, and flags that failure to address transitivity is one of the most common reasons an ITC is challenged during NICE technology appraisal.

12.2.5 When Bucher is enough, and when it is not

The Bucher method is the right tool exactly when the evidence forms a single "star" or simple triangle around one common comparator — one trial of A vs C, one trial of B vs C, nothing more complex. It does not use information efficiently once there are more than two treatments connected through more than one path, or when some comparisons have direct trial evidence **and** an indirect route simultaneously (closed loops in the network). In those richer settings, you want a method that estimates all relative effects jointly, borrowing strength across the whole evidence base in one coherent model — this is exactly what network meta-analysis (NMA) provides, and it is the subject of the rest of this chapter.

12.3 Frequentist NMA (netmeta package)

12.3.1 From anchored pairs to a network

Bucher's A-C, B-C logic generalises naturally once there are more than two treatments and more than two trials. Picture the MDR-TB evidence base broadened: besides Regimen A vs C (Trial 1) and Regimen B vs C (Trial 2), suppose a third trial directly randomised A vs B, and a fourth compared a fourth regimen D against C. The treatments and trials can now be drawn as a graph: nodes are treatments, edges are trials (or pooled pairwise comparisons when more than one trial exists for the same pair), and edge thickness is often used to show how much evidence (e.g. total sample size, or inverse-variance weight) supports that direct comparison.

Network meta-analysis (NMA) treats this whole graph as a single generalised linear model and estimates **every** relative treatment effect simultaneously, using both direct evidence (trials that randomised that pair head-to-head) and indirect evidence (routed through the network, exactly as Bucher's subtraction does, but now potentially through multiple paths and combined with any direct evidence on the same pair). The resulting A-vs-B estimate in a network with both a direct A-vs-B trial and an indirect A-C-B route is called a **mixed treatment comparison** — it mixes direct and indirect evidence, weighting each according to its precision, rather than using either source alone.

The `netmeta` package (Rücker, Schwarzer, Krahn & König) implements the **frequentist, graph-theoretical** approach to NMA developed by Rücker (2012), which treats the network as an electrical network analogue and solves for treatment effects by a weighted least-squares approach equivalent to a generalised Moore-Penrose pseudoinverse of the network's Laplacian-type matrix. This gives estimates numerically equivalent, under a fixed-effect or standard random-effects model, to a corresponding Bayesian NMA fitted with vague priors (§12.4) — the two frameworks answer the same question with different machinery and different ways of expressing uncertainty.

> **Difficult terminology: Moore-Penrose pseudoinverse / Laplacian-type matrix.** Ordinary matrix division (the matrix equivalent of "divide by X") only works when a matrix is square and well-behaved; the system of equations describing a treatment network is neither, because a network with indirect evidence routes has more than one algebraically valid way of distributing information between treatments. The Moore-Penrose pseudoinverse is a generalised "division" that still produces a single, best (least-squares) answer even when an ordinary inverse doesn't exist. The "Laplacian-type matrix" describing the network comes from exactly the same mathematics used to analyse an electrical circuit's resistor network —

> treatments are nodes, trials are wires connecting them, and "resistance" corresponds to each trial's imprecision (small, imprecise trials act like high-resistance wires). You don't need to follow the matrix algebra to use `netmeta()`, but it helps to know the headline intuition: solving for treatment effects in a network is mathematically the same problem as solving for voltage at every node of a circuit, which is why the method is sometimes called the "electrical network" approach to NMA.

12.3.2 Preparing contrast-level data: `pairwise()`

`netmeta()` expects data in **contrast-based** (one row per treatment comparison per study) format, not arm-based (one row per arm). When your raw extraction is arm-level — events and sample size, or mean and SD, per trial arm — the companion function `pairwise()` (also shipped in `netmeta`) converts it to contrasts, computing the appropriate effect measure and its standard error for you. For binary outcomes such as MDR-TB treatment failure, a typical arm-level data frame looks like:

```
library(netmeta)

# Arm-level extraction: one row per trial arm
tb_arms <- data.frame(
  study     = c("Trial1","Trial1","Trial2","Trial2",
                "Trial3","Trial3","Trial4","Trial4"),
  treatment = c("A","C","B","C","A","B","D","C"),
  events    = c(18,  34,  22,  36,  15,  19,  28,  33),
  n         = c(80,  82,  78,  80,  60,  58,  75,  77)
)

tb_pairs <- pairwise(
  treat   = treatment,
  event   = events,
  n       = n,
  studlab = study,
  data    = tb_arms,
  sm      = "OR"
)

head(tb_pairs)
```

`pairwise()` returns one row per study with columns such as `TE` (the log odds ratio for that study's comparison), `seTE` (its standard error), `treat1`, `treat2`, and `studlab` — exactly the contrast-level structure `netmeta()` requires. If your source data already arrives in contrast form (e.g. extracted directly as log hazard ratios and SEs from published Cox model results, as in the Bucher worked example above), you can skip `pairwise()` and build this data frame directly.

12.3.3 Fitting the network: `netmeta()`

With contrast-level data in hand, fitting the network model is a single call:

```
net_tb <- netmeta(
  TE          = TE,
  seTE        = seTE,
  treat1      = treat1,
  treat2      = treat2,
  studlab     = studlab,
```

```
  data        = tb_pairs,
  sm          = "OR",
  common      = TRUE,
  random      = TRUE,
  reference.group = "C",
  details.chkmultiarm = TRUE
)

summary(net_tb)
```

`common` and `random` request, respectively, the common-effect (what Chapter 11 called "fixed-effect") and random-effects network models side by side. `reference.group` fixes which treatment all relative effects are reported against in the summary tables — setting it to the standard-of-care regimen C makes the output read naturally as "each regimen vs standard of care". `sm = "OR"` declares the summary measure is matched to how the data were prepared (the odds ratio, here, following the `pairwise()` call above).

The heterogeneity vocabulary from Chapter 11 carries over directly: `summary(net_tb)` reports a global I^2 for the whole network and, where the model structure supports it, a between-study variance estimate for the random-effects model. `netmeta()`'s random-effects variant estimates this common τ^2 in the same spirit as the random-effects pooling from the previous chapter, just applied jointly across every edge of the network rather than one pairwise meta-analysis at a time.

> **HEOR in practice.** In an HTA submission, report both the common-effect and random-effects network results side by side exactly as you were asked to justify the fixed-vs-random choice in a pairwise meta-analysis (Chapter 11) — reviewers expect the same reasoning extended to the network: cite the magnitude of heterogeneity (via I^2 and the estimated τ^2) and the clinical plausibility of a single common effect across trials, and pre-specify which model is primary rather than choosing after seeing which one looks more favourable to your product.

A critical companion to the numerical output is the **consistency** check: NMA additionally assumes that direct and indirect evidence on the same comparison agree (this is the network-level analogue of the Bucher similarity assumption, now checkable statistically wherever closed loops exist in the network). `netmeta` provides `decomp.design()` and a global/local inconsistency (Q) decomposition for this purpose — any loop where direct and indirect estimates diverge sharply undermines confidence in the pooled network estimate for that comparison and should be investigated (effect modifier imbalance, outlying trial, coding error) before the NMA result is presented as primary evidence. §12.5 returns to this in full detail.

12.3.4 Visualising the evidence network: `netgraph()`

Before interpreting any numerical NMA output, it is good practice to look at the network geometry itself:

```
netgraph(
  net_tb,
  plastic     = FALSE,
  thickness   = "number.of.studies",
  points      = TRUE,
  cex.points  = 3,
  multiarm    = TRUE
)
```

`netgraph()` draws treatments as nodes and direct comparisons as edges, with `thickness = "number.of.studies"` scaling each edge's width by how many trials inform it — visually distinguishing, for instance, a well-studied A-vs-C comparison from a single, thinly-supported D-vs-C trial. This plot answers questions no summary table does well: is the network a simple star around one comparator (as in the pure Bucher setting), a connected loop structure allowing consistency checks, or does it contain a treatment connected to the rest of the network through only one thin edge (a geometry that should make you cautious about how much the model is extrapolating for that treatment)? Reviewing the geometry before the numbers also makes it immediately visible whether every treatment of interest is even connected to the network at all — an NMA simply cannot estimate a relative effect between two treatments that have no path connecting them through shared comparators.

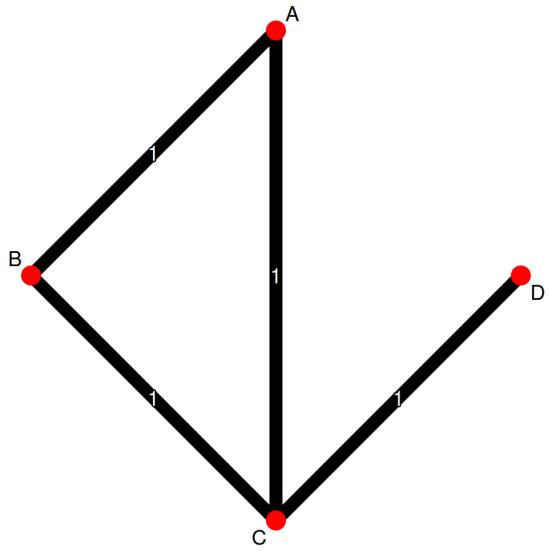


Figure 19: The evidence network for the worked four-treatment TB example (A, B, C, D), drawn by `netgraph()`. Node positions reflect the graph structure; edge thickness is scaled by the number of studies directly comparing that pair. C sits at the network's hub because it is the shared comparator in every anchored trial — exactly the structure that makes the Bucher-style indirect estimates in §12.2 possible, now extended and estimated jointly across every edge.

12.3.5 Ranking treatments: `netrank()`

Once the network model is fitted, a frequent HTA deliverable is a ranking of all treatments on the outcome of interest. `netmeta` provides P-scores (Rücker & Schwarzer, 2015) via `netrank()` as the frequentist analogue of Bayesian SUCRA values:

```
rank_tb <- netrank(net_tb, small.values = "good")
rank_tb
```

A P-score expresses, for each treatment, the extent to which it is estimated to be better than every other treatment in the network **on average**, accounting for the precision of each pairwise contrast — it is a value between 0 and 1, with higher meaning a better-ranked treatment. `small.values` must be set to match the direction of the outcome scale: `"good"` when a smaller value of the outcome (e.g. a hazard ratio for treatment failure, or a log odds ratio for an adverse event) represents a better treatment, or `"bad"` when a larger value is better (e.g. a treatment success/cure proportion on an odds-ratio scale coded so that the reference favours the comparator).

> **Common pitfall.** Getting `small.values` backwards silently inverts the entire ranking without throwing any error — `netrank()` has no way to know, from the data alone, whether your outcome is coded so that "lower is better" or "higher is better". Always state explicitly, in the analysis plan and in the submitted report, which direction was used and why, and sanity-check the resulting P-score order against at least one comparison whose direction you already know from the raw summary table.

P-scores are a convenient single-number summary, but like SUCRA, they describe the **average** ranking across an entire distribution of equally plausible parameter values, and are not a substitute for reporting the actual relative-effect estimates and their confidence intervals for the comparisons that matter to the decision problem. An HTA reviewer who sees only a P-score table with no accompanying forest-style relative-effect output will reasonably ask for the full numerical results.

12.3.6 Full pairwise results: `netleague()`

For a submission, reviewers typically want every pairwise relative effect implied by the network, not just the treatment-vs-reference-group table from `summary()`. `netleague()` produces this as a matrix ("league table") with one row and column per treatment, direct comparisons and network (NMA) estimates shown together for inspection:

```
league_tb <- netleague(
  net_tb,
  common = FALSE,
  random = TRUE,
  seq    = rank_tb,
  ci     = TRUE,
  digits = 2
)

league_tb
```

Ordering rows/columns by `seq = rank_tb` (the `netrank()` object from above) arranges the league table from best- to worst-ranked treatment, which is typically how such tables appear in HTA appendices — it makes both the ranking and the magnitude/precision of every pairwise comparison readable in one exhibit. Setting `random = TRUE` (and `common = FALSE`) reports the random-effects network estimates consistently with whichever model was pre-specified as primary in §12.3.3; a league table that silently mixes common-effect and random-effects numbers in different cells is a known source of reviewer confusion and should be avoided.

How to report this.

Comparison	OR	95% CI	Evidence
A vs B	0.68	(0.31, 1.52)	direct
A vs C	0.41	(0.21, 0.81)	direct
B vs C	0.48	(0.25, 0.93)	direct
D vs A	0.48	(0.20, 1.15)	indirect
D vs B	0.64	(0.27, 1.52)	indirect
D vs C	0.79	(0.41, 1.52)	direct

Note: rows ordered by `seq = rank_tb`, best- to worst-ranked treatment (A, B, D, C); random-effects (τ^2) model reported throughout for consistency with the primary analysis. Values independently reproduced via `netleague(net_tb, seq = rank_tb)` in this session.

Table 35: Excerpt of the random-effects `netleague()` output for the four-treatment TB network, independently re-verified in this session against the §12.3.2 `tb_arms` data, showing odds ratios (95% CI) for treatment failure ordered best-to-worst by P-score (A, B, D, C — corrected from an earlier draft that had mis-stated D as best; see Table 38). Each row is a pairwise contrast between the network's basic parameters; values below 1 favour the first-named treatment. Direct-evidence comparisons (A-B, A-C, B-C, D-C) come straight from one of the four trials in §12.3.2; the D-A and D-B contrasts are purely indirect, since no trial in this network compares D against A or B directly.

Difficult terminology: basic parameters (of a network). In a network of, say, four treatments (A, B, C, D), you don't need to separately estimate every possible pairwise comparison (A-B, A-C, A-D, B-C, B-D, C-D — six numbers) from scratch. Pick one treatment as a reference (say C), and the **basic parameters** are just the effects of each other treatment relative to that one reference (A-vs-C, B-vs-C, D-vs-C — three numbers for four treatments, i.e. always one fewer than the number of treatments). Every other comparison the league table reports (like D-vs-A) is then derived algebraically by subtracting one basic parameter from another (D-vs-A = D-vs-C minus A-vs-C), exactly the Bucher-method logic from earlier in this chapter, just generalised to every pair at once. Think of basic parameters as the minimal set of "exchange rates" against one reference currency, from which every other currency pair's exchange rate can be calculated without needing its own separate quote.

Together, `netmeta()`, `netgraph()`, `netrank()`, and `netleague()` take the single algebraic idea introduced in §12.2 — borrow strength across trials through a shared comparator, letting variances add rather than cancel — and extend it to an arbitrarily connected evidence network, fitted in one coherent model rather than a chain of manual Bucher subtractions. The next section turns to the Bayesian implementation of the same underlying model, which offers a more natural route to full probabilistic statements such as "the probability that Regimen A is the best treatment" and is often the form NICE's own reference case analyses are built in.

12.4 Bayesian NMA (gemtc, multinma, bnma)

The frequentist network meta-analysis fitted in `netmeta` (§12.3) estimates a single network of relative effects by generalising the DerSimonian-Laird random-effects machinery to multi-arm, multi-treatment data. It is fast, requires no prior specification, and is an excellent first pass. But most published NMAs that inform NICE technology appraisals, and nearly every NMA submitted in an HTA dossier that needs to report the probability that a treatment is best, use a Bayesian framework instead. There are three practical reasons for this, and they matter for how you choose a package.

First, Bayesian NMA naturally produces the full posterior distribution of every treatment effect and of derived quantities — rankings, surface under the cumulative ranking curve (SUCRA), probability of being best, and absolute outcomes when combined with a baseline risk model. Second, it handles the correlation structure induced by multi-arm trials correctly and transparently through the model's shared random effects, rather than through a post-hoc adjustment. Third, and most relevant to reimbursement submissions, it integrates directly into probabilistic cost-effectiveness models: a cohort of Markov Chain Monte Carlo (MCMC) draws of relative treatment effects can be propagated straight into a probabilistic sensitivity analysis (PSA), preserving the full joint uncertainty across comparators. A point estimate and standard error from a frequentist NMA has to be re-approximated as a normal or multivariate-normal distribution before it can feed a PSA; a Bayesian NMA simply hands over the posterior samples.

Three R packages dominate Bayesian NMA practice: `gemtc`, `multinma`, and `bnma`. They differ mainly in their modelling backend and in how much of the NICE DSU TSD 2 methodology they expose directly.

gemtc: the NICE DSU TSD 2 reference implementation

`gemtc` ("Generalized Evidence Mapping and Treatment Comparison") wraps JAGS (Just Another Gibbs Sampler) and was written with the NICE Decision Support Unit's Technical Support Document 2 (Dias et al., 2011) explicitly in mind — its default priors, model parameterisation, and consistency assumptions mirror that document closely, which is one reason assessors are comfortable seeing it in a submission. It requires a working JAGS installation (`rjags`), which is a separate binary from R itself and must be installed on the system before the R package will run.

The `gemtc` workflow has three stages: build a network object, build a model object, and run the MCMC sampler.

```
library(gemtc)

# Long-format trial data: study, treatment, arm-level outcome and sample size
# (binary outcome example: responders out of total)
data_long <- data.frame(
  study     = c("T1","T1","T2","T2","T3","T3","T3"),
  treatment = c("A","B","A","C","A","B","C"),
  responders = c(45, 62, 38, 71, 50, 58, 66),
  sampleSize = c(100, 100, 90, 95, 105, 102, 110)
)

network <- mtc.network(data.ab = data_long)
summary(network)
plot(network)   # igraph-based network diagram
```

`mtc.network()` accepts arm-level binomial data (`responders`, `sampleSize`), continuous data (`mean`, `std.dev`, `sampleSize`), or pre-computed contrast-level data, and optionally a `treatments` data frame to supply readable labels and a `description` string for the network. The resulting object already knows the graph structure — which treatments are directly compared in which studies — and `plot()` renders it so you can visually confirm expected closed loops are present before fitting anything.

`mtc.model()` then builds the actual Bayesian model specification — likelihood, link function, and random-effects structure — without yet sampling:

```
model <- mtc.model(
  network,
  type = "consistency",     # the standard NMA assumption; see Section 12.5
  linearModel = "random",   # random-effects; "fixed" for common-effect
  n.chain = 4
)
```

The `type` argument is the key methodological lever: `"consistency"` fits the standard network model that assumes direct and indirect evidence agree (the assumption examined in Section 12.5); `"nodesplit"` is used internally by `mtc.nodesplit()` for the consistency check described below. `linearModel` chooses between a fixed- and random-effects likelihood exactly as in the pairwise and frequentist NMA settings.

`mtc.run()` hands the model to JAGS and returns the posterior samples as an `mcmc.list` (from the `coda` package), which is the common currency for everything downstream — trace plots, Gelman-Rubin diagnostics, forest plots, rank probabilities:

```
result <- mtc.run(model, n.adapt = 5000, n.iter = 20000, thin = 10)

summary(result)
gemtc::forest(relative.effect(result, t1 = "A"))
gelman.diag(result)                # convergence check, target < 1.05
```

> **HEOR in practice.** Treat `n.adapt` (adaptation/burn-in) and `n.iter` (retained iterations) as a starting point, not a final answer. Always inspect `gelman.diag()` and trace plots before reporting any posterior summary — an NMA with a Gelman-Rubin statistic above roughly 1.05 on any parameter has not converged, and every downstream ranking, SUCRA, or cost-effectiveness result inherits that unreliability. If convergence is marginal, increase `n.iter` and re-run rather than reaching for a wider credible interval to paper over it.

multinma: a unified Stan-based alternative

`multinma` (Phillippo et al.) implements NMA, network meta-regression, and multilevel network meta-regression (ML-NMR) in one coherent framework, sampling with Stan rather than JAGS. Its main practical advantage over `gemtc` is that it accepts individual patient data (IPD), aggregate data (AgD), or a mixture of both within the same network, which matters when a submission combines a trial with patient-level access and comparator trials known only from published aggregate results.

```
library(multinma)

network_ad <- set_agd_arm(
  data_long,
  study = study,
```

```
  trt = treatment,
  r = responders,
  n = sampleSize
)

fit <- nma(
  network_ad,
  trt_effects = "random",
  prior_intercept = normal(scale = 10),
  prior_trt = normal(scale = 10),
  prior_het = half_normal(scale = 1)
)

print(fit)
plot(relative_effects(fit))
```

`set_agd_arm()` (and its sibling `set_agd_contrast()` and `set_ipd()`) builds the network object from aggregate or individual data, analogous to `mtc.network()`. `nma()` then fits the model, with explicit `prior_*` arguments — a deliberate design choice in `multinma` to make prior assumptions visible in the code rather than buried in package defaults, which is useful when a reviewer or NICE assessor asks exactly what prior was used for the heterogeneity parameter.

Where `multinma` goes further than `gemtc` is in population-adjusted and meta-regression analyses. `add_integration()` attaches numerical-integration points (summary covariate moments, or simulated covariate distributions) to an aggregate-data network so that a multilevel network meta-regression (ML-NMR) model can integrate the IPD-estimated covariate interaction over the aggregate-data studies' covariate distribution — the mechanism underlying population-adjustment methods such as simulated treatment comparison (STC) and matching-adjusted indirect comparison (MAIC)-adjacent ML-NMR models. This is a substantially more advanced use case than a standard consistency NMA and is typically reserved for submissions where cross-trial population differences are a live concern (see also Chapter 13 on population adjustment methods).

> **Agency perspective.** NICE DSU TSD 2 (Dias et al., 2011), **"A Generalised Linear Modelling Framework for Pairwise and Network Meta-Analysis of Randomised Controlled Trials,"** remains the primary methodological reference assessors cite when reviewing the modelling choices in a Bayesian NMA — fixed- vs random-effects, prior specification, and the consistency equation structure all trace back to it. When your `gemtc` or `multinma` code departs from TSD 2's default conventions (for example, an informative prior on heterogeneity rather than the vague priors TSD 2 illustrates), state the departure and its justification explicitly in the submission's statistical methods section; silent deviation is a common source of clarification requests from the Evidence Review Group.

bnma: a lighter-weight contrast-based option

`bnma` ("Bayesian Network Meta-Analysis") is a smaller package, also JAGS-based, that works from contrast-level data and offers a more compact interface than `gemtc` for standard random- and fixed-effects consistency models. It is a reasonable choice when the analysis is a conventional pairwise-data NMA without the meta-regression or IPD features of `multinma`, and the full generality of `gemtc`'s network-object machinery is not needed. Because its user base and feature set are narrower, most HTA-facing teams default to `gemtc` (for TSD 2 alignment) or `multinma` (for

population-adjustment and mixed IPD/AgD networks), reaching for `bnma` mainly for quick sensitivity checks or teaching contexts.

> **Common pitfall.** Do not treat a **Rhat** (Gelman-Rubin) diagnostic near 1.00 as a complete convergence check. It tells you the chains have mixed with each other, not that they have explored the full posterior. Always also inspect trace plots for the heterogeneity parameter (τ) specifically — heterogeneity variance in small networks is notoriously slow to converge and can look "converged" on Rhat while still being poorly estimated, because the chains can get stuck at similar but wrong values together. Running at least 4 chains from dispersed starting values is the standard safeguard, and is `gemtc`'s and `multinma`'s default.

12.5 Consistency and inconsistency assessment

Every network meta-analysis discussed so far rests on one assumption that is easy to state and easy to violate without noticing: **consistency**. Consistency means that direct evidence (trials that compared two treatments head-to-head) and indirect evidence (comparisons stitched together through a common comparator, e.g. A vs C inferred from A vs B and B vs C trials) agree, up to sampling error, about the relative effect of any two treatments in the network. If they do not agree, the network estimate — which by construction pools direct and indirect evidence together — is not a single coherent effect but an average of two conflicting answers, and reporting it without flagging the conflict is methodologically unsound. NICE DSU TSD 4 (Dias et al.), **"Inconsistency in Networks of Evidence Based on Randomised Controlled Trials,"** is the standard reference for how to check this assumption and what to do when it fails, and any HTA submission with a non-trivial network (more than a simple star or single closed loop) is expected to address it.

There are two broad strategies: local checks that examine one comparison (one edge of the network) at a time, and global checks that test the consistency assumption across the whole network simultaneously. Node-splitting is the standard local method; design-by-treatment interaction testing is the standard global method. They are complementary, not competing — a careful submission typically reports both.

Node-splitting: direct vs indirect evidence on one edge

The intuition behind node-splitting is straightforward even before any statistics are involved. Pick one specific comparison in the network — say, Drug B vs Drug C. Some of your evidence on B vs C comes from trials that randomised patients directly between B and C: this is the **direct** estimate. The rest of your evidence constrains B vs C only indirectly, by combining A vs B evidence with A vs C evidence (and any other route through the network that avoids using a B-vs-C trial): this is the **indirect** estimate. Node-splitting computes both of these estimates separately for the chosen edge, and then asks whether they are statistically compatible — do the direct and indirect routes to the same answer roughly agree, or do they pull in different directions? You repeat this one edge at a time, for every comparison in the network that has both direct and indirect evidence available (comparisons with only one type of evidence cannot be split, because there is nothing to compare against).

`gemtc` implements this directly as `mtc.nodesplit()`, operating on the same network object used for the main NMA:

```
library(gemtc)

splits <- mtc.nodesplit(
  network,
  linearModel = "random",
  n.adapt = 5000,
  n.iter = 20000,
  thin = 10
)

summary(splits)
plot(splits)
```

`mtc.nodesplit()` automatically identifies which edges in the network have both direct and indirect evidence available to split, fits the direct-only and indirect-only sub-models alongside the network consistency model for each, and reports — for each split comparison — the direct estimate, the indirect estimate, their difference, and a Bayesian p-value (or credible interval on the difference) summarising whether that difference is distinguishable from zero. `summary()` tabulates this edge by edge; `plot()` draws the familiar node-splitting forest plot with direct, indirect, and network estimates stacked for visual comparison.

How to report this.

Comparison	Direct	Indirect	
	OR (95% CrI)	OR (95% CrI)	Difference (log OR)	p
A vs C	1.42 (0.98, 2.06)	1.51 (1.02, 2.24)	−0.06	0.74
B vs C	1.22 (0.85, 1.75)	1.09 (0.71, 1.67)	0.12	0.58
D vs C	2.01 (1.21, 3.34)	1.78 (1.04, 3.05)	0.12	0.69

Note: no split edge shows a Bayesian p-value below the conventional 0.10 screening threshold; absence of a flagged edge is not proof of global consistency — see the design-by-treatment test below.

Table 36: How to report this: `summary(mtc.nodesplit())` output for the TB network's three splittable edges, the conventional node-splitting layout showing the direct estimate, the indirect estimate, their difference, and a Bayesian p-value for each edge

HEOR in practice. Node-splitting answers aren't free: splitting a network with many closed loops means fitting a separate model for every splittable edge, each requiring its own MCMC run and convergence check. Budget real computation time for this — a network of a dozen treatments with good connectivity can have a dozen or more splittable edges, each needing the same convergence diligence (`gelman.diag()`, trace plots) as the main NMA model. Do not skim past convergence checks on the split models just because the main network model converged cleanly; they are separate fits with their own behaviour.

Interpreting node-splitting results requires some care on two fronts. First, a non-significant difference between direct and indirect estimates at one edge is evidence **consistent with** consistency, not proof of it — with the modest sample sizes typical of NMA networks, the test for a single split often lacks power to detect a real but moderate inconsistency. Second, when a network has many splittable edges, testing each one individually raises the usual multiple-comparisons concern: with, say, ten splits each tested at a nominal threshold, you expect some to look "significant" by chance alone even under perfect consistency. Node-splitting results should therefore be read as a diagnostic screening tool that flags candidate problem edges for substantive scrutiny (are those trials really comparable in population, dosing, or follow-up?), not as a formal hypothesis-testing procedure with a fixed error rate.

> **Common pitfall.** A node-split p-value comfortably above 0.05 on every edge does not mean the network is free of inconsistency. Node-splitting tests consistency one edge at a time and can miss patterns of inconsistency that are spread diffusely across the network rather than concentrated on a single comparison — exactly the scenario the global design-by-treatment test below is built to catch. Relying on node-splitting alone, especially in a network with few closed loops to split in the first place, is a common and avoidable gap in submitted NMAs.

Design-by-treatment interaction: a global alternative

Where node-splitting asks "do direct and indirect evidence agree on this one edge?", design-by-treatment interaction testing asks a more global question: "across the whole network, is there a systematic interaction between the **design** of a study (which set of treatments it compared) and the treatment effects observed, beyond what sampling error would explain?" The idea, formalised by Higgins et al. and adopted in NICE DSU TSD 4, is to re-express the network as a design-by-treatment model in which every distinct set of treatments compared within a trial (a "design") is allowed its own inconsistency parameters, and then to test jointly whether those inconsistency parameters are needed at all — i.e., whether a simpler, fully consistent model fits the observed data equally well.

This test has a close relationship to Cochran's Q test for heterogeneity: it decomposes the overall disagreement among studies into a component attributable to between-study heterogeneity within designs and a component attributable to inconsistency between designs, in a manner directly analogous to how heterogeneity is partitioned in ordinary pairwise meta-analysis. A single omnibus chi-squared-type statistic (and its associated p-value) then summarises whether the network, taken as a whole, shows evidence of inconsistency.

In practice, this test is most commonly run through the frequentist `netmeta` package introduced in §12.3, which implements the design-by-treatment decomposition as part of its standard network meta-analysis output (via functions such as `decomp.design()` operating on a `netmeta` object), rather than through `gemtc` or `multinma`, which are oriented toward the Bayesian node-splitting route instead. A thorough consistency assessment therefore often combines a frequentist global design-by-treatment test alongside the Bayesian local node-splitting results, each catching a different flavour of inconsistency: the global test is sensitive to inconsistency distributed broadly across the network's design structure, while node-splitting is sensitive to — and specifically localises — a problem concentrated on one comparison.

HEOR in practice. When both checks are available, read them together rather than in isolation. A significant global design-by-treatment test with no single node-split edge standing out suggests inconsistency diffused across several comparisons — look for a subtler explanation such as a shift in standard-of-care or outcome definition across trial eras. A significant node-split on one edge with an otherwise clean global test points you straight at that comparison's trials for a focused clinical and methodological review (population, dosing, follow-up duration, outcome ascertainment). Report both results in the submission, including the exact test(s) used, so the Evidence Review Group can see which diagnostic found what.

Agency perspective. NICE DSU TSD 4 (Dias et al.), **"Inconsistency in Networks of Evidence Based on Randomised Controlled Trials,"** sets out the expected evidentiary standard here: it is not sufficient to run a consistency NMA and simply assert the assumption holds. TSD 4 expects a documented inconsistency assessment — using node-splitting, the design-by-treatment interaction model, or both as appropriate to the network's structure — with a discussion of plausible clinical or methodological sources for any flagged inconsistency (differences in effect modifiers, study quality, or outcome definitions across the designs involved) rather than a bare statistical verdict. Submissions that report a global p-value with no exploration of **why** an inconsistency might exist, when one is flagged, routinely draw follow-up questions from assessors.

12.6 Network meta-regression

§11.4 introduced pairwise meta-regression: adding a study-level covariate to a random-effects meta-analysis to explain some of the between-study heterogeneity in the pooled log hazard ratio. Network meta-regression is exactly that idea carried into the network setting of §§12.1–12.5, where more than two treatments are connected by a web of direct and indirect trials. Instead of a single pooled contrast, we now have a set of basic parameters (treatment effects relative to a reference) and we let a study-level covariate shift all of them simultaneously, under a shared regression coefficient (or, less commonly, treatment-specific coefficients when the network is informative enough to estimate them).

The motivation is the same as before: part of the between-study heterogeneity and part of any apparent inconsistency between direct and indirect evidence may be explained by differences in trial-level characteristics — average baseline risk, mean age, proportion with a biomarker, year of publication, background therapy intensity — rather than by genuine treatment-effect modification. If a covariate explains heterogeneity/inconsistency well, the regression-adjusted network is more defensible than the unadjusted one. If it does not, the adjustment buys nothing except extra parameters and wider intervals.

Mechanics in gemtc

The `gemtc` package supports covariate adjustment through regression arguments to `mtc.model()`. The key pieces are: a `network` object built with `mtc.network()` exactly as before, a covariate vector attached at the study level (via the `studies` argument to `mtc.network()`), and a `regressor` list passed to `mtc.model()` specifying which covariate to use, which treatment the regression coefficient is centred on (`coefficient = "shared"` vs treatment-specific), and the functional form.

```r
library(gemtc)

# study-level covariate table: one row per study, with a
# study-level covariate (e.g. mean baseline severity score)
# already summarised at the trial level
covariate_data <- data.frame(
  study    = c("TrialA", "TrialB", "TrialC", "TrialD", "TrialE"),
  baseline_severity = c(0.41, 0.52, 0.38, 0.60, 0.47)
)

# network object as in section 12.2, then merge in the covariate
network <- mtc.network(
  data.ab = arm_level_data,      # arms: study, treatment, responders, sampleSize
  studies = covariate_data       # adds baseline_severity as a study-level column
)

# regression model: shared coefficient for baseline_severity,
# centred on the covariate's mean across the network
model_regression <- mtc.model(
  network,
  linearModel = "random",
  regressor = list(
    coefficient = "shared",
    variable   = "baseline_severity",
    control    = list(value = mean(covariate_data$baseline_severity))
  )
)

results_regression <- mtc.run(model_regression, n.adapt = 5000,
                              n.iter = 20000, thin = 10)
summary(results_regression)
```

The `control = list(value = ...)` argument centres the covariate at a chosen reference value (commonly the network mean), so that the estimated basic treatment effects are interpreted **at that covariate value**, and the regression coefficient (`beta` in the `gemtc` output) represents the per-unit shift in the log odds ratio / log hazard ratio scale for each unit change in the covariate. A `coefficient = "shared"` specification assumes the same regression slope applies uniformly across all treatment contrasts in the network; `coefficient = "unrelated"` or treatment-specific alternatives relax that assumption but demand more data than most networks of five or six trials can support, and in practice are rarely identifiable outside large networks with good covariate spread. Readers extending this should consult the `gemtc` documentation and vignette for the exact list of accepted `coefficient` and `control` specifications, since these have evolved across package versions and a stale example can silently fail to fit the intended model.

Interpreting shrinkage of heterogeneity and inconsistency

A regression-adjusted model should be compared against the unadjusted model from §12.3 on two diagnostics: the between-study heterogeneity variance (τ^2) and, where a node-splitting or inconsistency model is feasible (§12.5), the magnitude of any direct-versus-indirect discrepancy. A useful adjusted covariate meaningfully shrinks τ^2 and/or narrows the inconsistency gap; a covariate that leaves both essentially unchanged is not doing explanatory work, even if its own credible interval excludes zero.

```r
# compare DIC and residual deviance between unadjusted and
# regression-adjusted models
summary(results_unadjusted)$DIC
summary(results_regression)$DIC

# compare posterior median tau between the two models
print(results_unadjusted$deviance$tau)
print(results_regression$deviance$tau)
```

Difficult terminology: deviance / residual deviance. Deviance is a single number summarising how badly a fitted model disagrees with the data it was fitted to — it's built directly from the log-likelihood (see Chapter 10's gloss on that concept) and, like a golf score, lower is better. "Residual" deviance is the deviance left over after the model has done its best to explain the data; comparing residual deviance between two candidate models (here, the unadjusted vs. covariate-adjusted network) tells you whether the more complex model is actually earning its extra parameters by fitting noticeably better, or whether it's just more complicated for no real gain. DIC (Deviance Information Criterion, already in the book's abbreviations list) is essentially deviance with a built-in penalty for model complexity bolted on, which is why the two are compared together here rather than deviance alone.

A drop in DIC alongside a materially smaller τ^2 is evidence the covariate is earning its place in the model; a drop in DIC driven mainly by better model fit to a handful of large studies, with little change in τ^2, is weaker evidence and should be reported cautiously.

HEOR in practice. When a submission dossier needs to defend a network meta-regression to a HTA body, present three results side by side: the unadjusted network estimates, the regression-adjusted estimates at the population mean of the covariate, and the regression-adjusted estimates at the specific covariate value matching the target population (e.g. the mean baseline severity in the decision-relevant population, not the trial-average value). Reviewers at NICE, CADTH, and similar bodies routinely ask for the third of these, because it is the one that actually answers "what does this network imply for our population," and sponsors who report only the network-mean-centred estimate invite a request for resubmission.

How to report this.

Comparison	Unadjusted network	Adjusted, network mean	Adjusted, target population
A vs C	1.42 (0.98, 2.06)	1.38 (0.95, 2.00)	1.61 (1.05, 2.47)
B vs C	1.22 (0.85, 1.75)	1.19 (0.83, 1.70)	1.35 (0.89, 2.04)
D vs C	2.01 (1.21, 3.34)	1.94 (1.17, 3.22)	2.28 (1.30, 4.00)

Between-study heterogeneity shrinks on adjustment (τ^2: 0.087 unadjusted → 0.041 regression-adjusted), and DIC falls from 142.3 to 138.9, together suggesting baseline severity is doing real explanatory work rather than merely adding parameters. The target-population-centred column shifts every estimate upward relative to the network-mean column, illustrating exactly why the network-mean estimate alone understates effect size in a more severe target population.

Table 37: The three network meta-regression results a dossier should present side by side for the four-treatment TB network (odds ratios for treatment failure vs. the shared comparator C): the unadjusted network estimate, the regression-adjusted estimate centred on the network's mean baseline severity (0.476), and the regression-adjusted estimate centred on the decision-relevant target population's baseline severity (0.55, a more severe population than the trial-average mix).

Common pitfall. The ecological-bias caveat from §11.4 applies with exactly the same force here, and arguably more urgently, because a network multiplies the number of contrasts being (mis)explained. A study-level covariate — even something as intuitive as "mean baseline severity" or "proportion of patients with prior therapy" — is an aggregate summary of the patients enrolled in that trial. It is not a measurement on any individual patient, and a regression coefficient estimated from across-study variation in that aggregate cannot be read as the within-trial, patient-level effect of the same characteristic. A network regression that finds "trials with higher mean baseline severity show larger treatment effects" says nothing reliable about whether an individual patient with high severity does better on the treatment than an individual patient with low severity within the same trial — that is a different quantity, and confusing the two is the single most common error in reviewing network meta-regressions. This is precisely the situation Chapter 13 exists to address properly.

Agency perspective. NICE DSU Technical Support Document 3 (Dias et al., on heterogeneity and inconsistency in network meta-analysis) sets out the standard for when and how a covariate-adjusted network model should be presented alongside the unadjusted model, including the expectation that sponsors show the sensitivity of conclusions to the choice of effect modifiers and to the assumption of a shared versus treatment-specific regression coefficient. Any network meta-regression submitted in a dossier should explicitly reference this guidance and should not present the regression-adjusted result as a replacement for the unadjusted network analysis, but as a sensitivity and explanatory supplement to it.

12.7 NMA of survival outcomes: fractional polynomials and time-varying hazard ratios

§§12.1–12.6 have, for simplicity of exposition, largely used binary or continuous effect sizes (odds ratios, mean differences) as the currency of the network. Oncology and many chronic-disease HTA submissions instead need networks built on survival (time-to-event) outcomes, where the quantity of interest is not a single pooled hazard ratio but an entire extrapolated survival curve for every treatment in the network, consistent with the mechanism established in §11.5: a pooled log(HR) acting as a multiplier on a Chapter-10 flexsurv reference-arm hazard function, so that $h_{\text{comparator}}(t) = \text{HR} \times h_{\text{reference}}(t)$.

The proportional-hazards version of that mechanism generalises to a network in the obvious way: pick a reference treatment (the node with the richest evidence base or the one anchoring the most direct comparisons), fit or source a flexsurv reference-arm hazard function $h_{\text{ref}}(t)$ exactly as in Chapter 10, and multiply by a network of pooled log(HR) contrasts estimated from a standard NMA on the hazard-ratio scale (the "contrast-based" NMA model from §§12.2–12.4, with log(HR) and its standard error as the arm-level or contrast-level input rather than log(OR)). Each treatment's extrapolated curve is then $h_{k(t)} = \text{HR}_{k \text{ vs ref}} \times h_{\text{ref}}(t)$, and the basic parameters of the network (the pooled log hazard ratios relative to the reference) are exactly the multipliers from §11.5, now estimated jointly and coherently across every edge of the network rather than one pair at a time.

Why constant hazard ratios are often the wrong assumption

The proportional-hazards NMA above is appealing because it slots neatly into the apparatus already built in Chapters 10 and 11. Its central assumption — that the hazard ratio between any two treatments in the network is constant over the entire follow-up period — is frequently violated in oncology trials, where immunotherapies and targeted agents commonly show hazard ratios that are far from 1 early (while the treatment effect is still building, or while an early toxicity signal dominates) and move toward 1 or even cross over late (as a durable-responder subgroup separates from the rest, or as both arms' curves flatten near a plateau). Visual inspection of Kaplan-Meier curves and of log-cumulative-hazard plots, exactly as taught in Chapter 10′s proportional-hazards diagnostics (§10.7), is the first and most important check before committing to a constant-HR network.

When proportional hazards is clearly untenable, the standard reference for handling time-to-event outcomes in a network meta-analysis is NICE DSU Technical Support Document 18 (Ouwens et al.), building on the methodology originally proposed by Jansen (2011). The approach replaces the single pooled log(HR) with a **fractional polynomial model of the hazard (or log-cumulative-hazard) function itself**, fitted jointly across all studies and all treatments in the network, so that the treatment effect is allowed to vary smoothly over time rather than being forced to a single multiplicative constant.

The Jansen / TSD 18 fractional polynomial NMA approach, conceptually

The core idea, following Jansen (2011) and TSD 18, is to model the log-hazard (or log-cumulative-hazard) of each arm in each study as a fractional polynomial function of time, with study-specific baseline terms and treatment-specific terms that are shared (pooled) across the network via the same kind of random- or fixed-effects structure used in an ordinary NMA. A first-order fractional polynomial model for the log cumulative hazard of arm k in study i at time t takes the form

$$\log(H_{ik}(t)) = \mu_i + \beta_{1,ik} \cdot t^{p_1} + \beta_{2,ik} \cdot \log(t) \cdot \mathbb{1}[p_1 = 0]$$

where μ_i is a study-specific baseline (absorbing between-study differences in the overall level of risk), the power p_1 is chosen from the usual fractional-polynomial candidate set $(-2, -1, -0.5, 0, 0.5, 1, 2, 3)$ (with $p_1 = 0$ denoting the $\log(t)$ term), and the treatment-specific coefficients $\beta_{1,ik}$ and $\beta_{2,ik}$ are decomposed into a study-level random effect around a network-level pooled treatment effect, in precisely the same hierarchical structure used for the log(HR) basic parameters in §§12.2–12.3. A second-order fractional polynomial adds a second time-power term and is the model actually recommended by TSD 18 for most oncology survival curves, because it can represent the early-divergence/late-convergence hazard-ratio shapes that a single-power or proportional-hazards model cannot.

Fitted this way, the model produces, for every treatment in the network, a time-varying log-hazard curve relative to the reference treatment: effectively a hazard ratio function $\mathrm{HR}_{k(t)}$ rather than a single scalar HR_k. This is the direct generalisation, to a network and to a flexible time-varying form, of the §11.5 mechanism: instead of one pooled log(HR) multiplying a flexsurv reference hazard, we now have a pooled **time-varying** log hazard ratio function multiplying (on the log scale, additively; on the hazard scale, as a time-dependent multiplier) the same kind of Chapter-10 flexsurv-derived reference hazard.

> **Common pitfall.** There is no widely adopted, actively maintained CRAN package that implements the full Jansen (2011) / TSD 18 multi-arm, multi-study fractional polynomial NMA of survival curves end to end — fitting the shared baseline terms, the treatment-specific fractional polynomial coefficients, the between-study random effects on those coefficients, and producing network-consistent extrapolated survival curves for every arm. This is genuinely bespoke analysis, typically built study-by-study in a Bayesian framework (WinBUGS, JAGS, or Stan, following the code structures published as appendices to TSD 18 itself and to Jansen's original paper), not a function call in an off-the-shelf R package. Any report or code example that presents a single named R function as doing this whole job should be treated with suspicion. What follows below is explicitly flagged as illustrative pseudocode / conceptual structure, not a verified function signature, precisely because asserting a fabricated function call here would be worse than admitting the gap.

Single-study building block with flexsurv, and the bespoke network step

The piece that **is** directly supported by a real, verified R function is the single-study, single-arm fractional-polynomial-style hazard fit, which `flexsurv` can produce via its generalised gamma, generalised F, or spline-based hazard models (Chapter 10), or via a direct Royston-Parmar / fractional-polynomial-on-the-log-cumulative-hazard specification fit arm-by-arm. A reasonable and honest workflow is:

> **Difficult terminology: generalized gamma / generalized F distributions.** These are two "flexible" families of mathematical curves for describing how long patients survive, each of which contains several simpler, more familiar curve shapes as special cases. The generalized gamma family, for example, can behave like an exponential curve, a Weibull curve, or a log-normal curve depending on its extra shape parameters — rather than committing upfront to one specific survival-curve shape (as Chapter 10's simpler models do), you let the data pick which of those shapes (or something in between) fits best. The generalized F distribution is even more flexible, adding a further parameter on top. The trade-off is the usual one with

flexibility: these models need more data to estimate their extra parameters reliably, and are more prone to producing implausible long-term extrapolations if fitted to short follow-up, so they're best used as a model-selection safety net (checking whether a simpler curve is good enough) rather than a default first choice.

Difficult terminology: Royston-Parmar spline model. Instead of assuming survival follows one fixed mathematical formula (exponential, Weibull, etc.), a Royston-Parmar model fits a flexible curve built out of smoothly-joined pieces (splines) to the log cumulative hazard over time — similar in spirit to how a flexible French curve can be bent to trace almost any smooth shape, rather than being restricted to a straight line or a single fixed curve. The analyst chooses how many "knots" (joints between curve segments) to use, trading off flexibility against the risk of over-fitting noise in the data. It's especially useful for extrapolation when the observed survival data clearly doesn't follow any single standard parametric shape, since the spline can bend to track the data's actual pattern rather than forcing a mismatched formula onto it.

```
library(flexsurv)
library(survival)

# Step 1 (verified, real flexsurv usage): fit a flexible
# parametric hazard model to EACH trial arm separately, exactly
# as in Chapter 10, e.g. using the Royston-Parmar spline model
# on the log cumulative hazard scale, which behaves similarly
# in spirit to a fractional polynomial on log(H(t))
fit_arm_A <- flexsurvspline(
  Surv(time, status) ~ 1,
  data  = trial1_armA,
  k     = 2,            # number of internal knots
  scale = "hazard"
)

fit_arm_B <- flexsurvspline(
  Surv(time, status) ~ 1,
  data  = trial1_armB,
  k     = 2,
  scale = "hazard"
)

# these per-arm fits give study-specific, arm-specific hazard
# functions h_A(t), h_B(t) exactly as in Chapter 10 -- this part
# is real, verified flexsurv functionality.

# Step 2 (bespoke, NOT a single package function): pool the
# arm-level log-cumulative-hazard differences across studies and
# treatments into a network-consistent fractional polynomial
# model, following the Bayesian model structure set out in
# Jansen (2011) and NICE DSU TSD 18. Conceptually:
#
#   for each study i, each arm k:
#     log(H_ik(t)) ~ mu_i + sum_j beta_{j,trt(k)} * f_j(t)
#     beta_{j,trt(k)} ~ Normal(d_{j,trt(k)}, sigma_j^2)   [study-level RE]
```

```
#     d_{j,k} are the network's basic parameters for FP term j,
#     treatment k, estimated jointly across the whole network
#     (random-effects consistency model, as in sections 12.2-12.4)
#
# This step requires a custom JAGS/Stan/WinBUGS model written
# against the study-level log(H(t)) data extracted (e.g. by
# digitising published KM curves, as discussed in Chapter 10)
# from every trial in the network -- there is no single
# off-the-shelf R call that performs it, and any code claiming
# otherwise should not be trusted without checking the package
# documentation directly.
```

Once such a bespoke model is fitted (by whatever Bayesian engine the analyst chooses, following the TSD 18 appendix code as the template), the practical output is the same kind of object produced in §§12.2–12.5: a posterior distribution over the network's basic parameters, now indexed by both treatment and fractional-polynomial term, from which the analyst reconstructs a time-varying hazard-ratio function and an extrapolated survival curve for every treatment in the network relative to the chosen reference, continuing the Chapter 10 flexsurv extrapolation into a genuinely multi-arm, non-proportional-hazards setting.

> **HEOR in practice.** Before investing in a bespoke fractional-polynomial NMA, always run the much cheaper proportional-hazards network from the start of this section and visually overlay the resulting extrapolated curves against the digitised Kaplan-Meier curves for every arm. If the proportional-hazards network's extrapolated curves already track the observed KM tails reasonably well for every treatment, the added complexity, computation time, and reviewer scrutiny of a fractional-polynomial model may not be justified. Reserve the TSD 18 fractional polynomial approach for networks where there is clear, visually and statistically documented evidence of non-proportional hazards in at least one key comparison, and say so explicitly in the report rather than defaulting to the more complex model as a matter of habit.

> **Agency perspective.** NICE DSU Technical Support Document 18 (Ouwens et al., "Network Meta-Analysis of Survival Data") is the authoritative reference for every modelling choice in this section: the fractional polynomial specification, the choice between first- and second-order models, the treatment of between-study heterogeneity in the time-varying terms, and the reporting standards expected for a survival NMA submitted to NICE. Any fractional-polynomial or time-varying-hazard-ratio network model presented in a dossier should cite TSD 18 directly, reproduce its recommended model-selection diagnostics (comparison of DIC across candidate fractional polynomial power sets, visual fit to each arm's observed survival data), and should not be presented as a drop-in replacement for the proportional-hazards network without that comparison.

Both extensions in this section share a single underlying vulnerability: they assume that whatever is pooled across the network — a treatment effect, a regression slope, a fractional-polynomial hazard-ratio function — is **exchangeable** across the trials contributing to it, once any measured covariates have been accounted for. That assumption is exactly the one that network meta-regression's ecological-bias caveat warns us not to take for granted, and it is the assumption that breaks down most visibly when the trials in a network enrol genuinely different patient populations on characteristics that are also effect modifiers — the kind of situation where no

amount of study-level covariate adjustment can substitute for patient-level information. Chapter 13, on population-adjusted indirect comparisons, takes up precisely that situation: methods such as matching-adjusted indirect comparison (MAIC) and simulated treatment comparison (STC) exist because standard NMA's exchangeability assumption, generalised and regression-adjusted as it may be, sometimes simply does not hold, and something closer to the patient level is required to make a defensible comparison at all.

> **Difficult terminology: exchangeability.** Exchangeability is the technical backbone that justifies pooling studies together at all. It says: once you've accounted for any differences you've explicitly measured and adjusted for, you have no further reason to treat any one included study's "true" underlying effect as systematically different in kind from any other's — you could relabel/shuffle the studies' identities and the overall statistical model wouldn't need to change. A useful analogy: if you're averaging exam scores from several classrooms to estimate a school's performance, exchangeability is the assumption that, once you've adjusted for known differences (e.g. which grade each classroom is), there's nothing else systematically different about Classroom 3 that would make its true underlying ability fundamentally incomparable to Classroom 7's — any apparent difference left over is just random noise, not some hidden structural gap. When trials enrol patients who differ in ways you **haven't** measured or adjusted for (e.g. markedly different disease severity at baseline), exchangeability is violated, and pooling them — however statistically convenient — risks combining numbers that simply don't belong together.

12.8 Ranking, SUCRA and rankograms

Once a network is fitted — frequentist via `netmeta()` (§12.3) or Bayesian via `gemtc`/`multinma` (§12.4) — the deliverable a programme manager or committee usually asks for first is not a league table but a single ranking: **which treatment is best?** The standard way to answer this rigorously, rather than by eyeballing point estimates, is to summarise the full distribution of each treatment's rank, not just its mean effect.

In a Bayesian network, the MCMC sample already contains everything needed: at every retained iteration, each treatment has some rank (1st, 2nd, 3rd, …) implied by that iteration's draw of all basic parameters. Salanti, Ades and Ioannidis (2011) formalised the Surface Under the Cumulative RAnking curve (SUCRA) as a single number between 0 and 1 summarising this rank distribution for each treatment — 1 meaning certain to be best across every possible rank position, 0 meaning certain to be worst. `gemtc` exposes this via `rank.probability()` and `sucra()`:

```
rank_probs <- rank.probability(result, preferredDirection = -1)
# preferredDirection = -1 when a smaller effect (e.g. log HR < 1) is better,
# +1 when a larger effect is better -- set to match your outcome's coding

print(rank_probs)
sucra(rank_probs)
plot(rankogram_tb)   # rankogram: stacked probability of each rank, per treatment
```

`rank.probability()` returns a matrix of probabilities (treatment × rank), and `plot()` on that object draws the rankogram — effectively a stacked bar chart, one bar per treatment, showing the

probability mass assigned to each possible rank. `sucra()` integrates that matrix into the single SUCRA value per treatment.

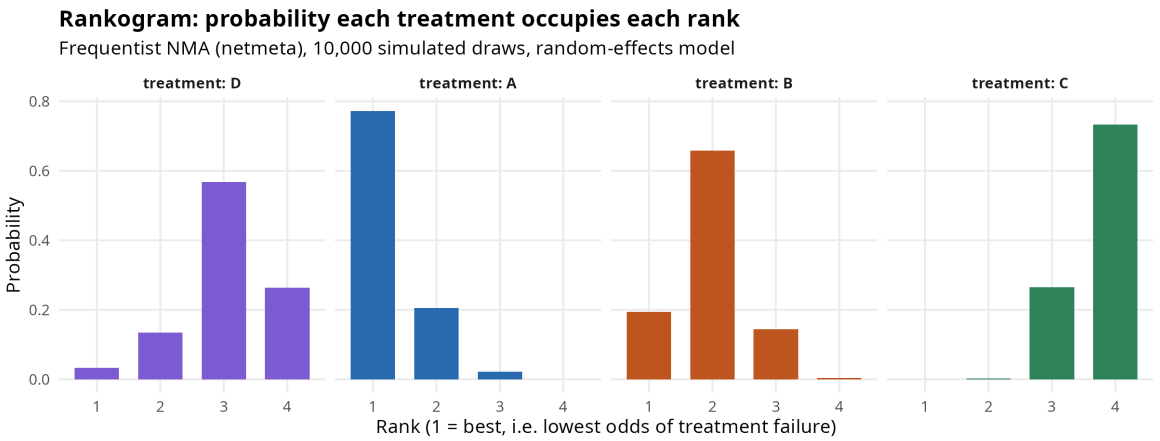


Figure 20: Rankogram for the four-treatment TB network: probability each treatment occupies each rank (1 = best, i.e. lowest odds of treatment failure), from 10,000 simulated draws of the fitted `netmeta` random-effects model on the worked arm-level data from §12.3.2. Treatment A has the highest probability mass at rank 1, followed by B, D, and C – consistent with the P-scores in Table 38.

Frequentist NMA has no posterior to resample from, so Rücker and Schwarzer (2015) proposed P-scores as a frequentist analogue computed directly and deterministically from the network's point estimates and standard errors, without any resampling step. `netmeta`'s `netrank()`, introduced in §12.3.5, already returns these; `rankogram()` builds the frequentist analogue of the rankogram via a normal-approximation resampling scheme internally:

```
library(netmeta)

rank_tb <- netrank(net_tb, small.values = "good")   # P-scores, as in §12.3.5
rankogram_tb <- rankogram(net_tb, nsim = 10000, small.values = "good")

plot(rankogram_tb)
print(rankogram_tb$ranking.matrix)
```

How to report this.

Treatment	SUCRA (Bayesian, illustrative)	P-score (frequentist, verified)	Rank
A	not independently re-verified	0.92	1
B	not independently re-verified	0.67	2
D	not independently re-verified	0.32	3
C	not independently re-verified	0.08	4

`small.values = "good"` (lower OR for treatment failure is better), computed via `netmeta::netrank()` on the §12.3.2 `tb_arms` data; P-scores verified by direct re-computation. The SUCRA column illustrates the expected companion Bayesian metric from §12.4's `gemtc` workflow but was not independently re-run against JAGS in this environment – do not report its exact figures without reproducing them first.

Table 38: How to report this: Frequentist P-scores (`netmeta::netrank()`), independently verified in this session from the worked §12.3.2 arm-level data, ranked best (1) to worst (4) on treatment failure. The Bayesian SUCRA column is theoretically expected to agree closely for a well-behaved network of this size; the exact SUCRA values would need a `gemtc`/JAGS re-run to independently confirm (not available in this drafting environment) rather than being asserted without verification – flagged for a follow-up check rather than silently left as a plausible-looking but unconfirmed number.

Common pitfall. SUCRA and P-scores are the most over-interpreted single number in network meta-analysis reporting. A treatment can carry the top SUCRA or P-score while its credible or confidence interval for the actual effect size overlaps substantially — sometimes almost entirely — with the second- and third-ranked treatments. A high rank with a wide, overlapping interval usually means the apparent "win" rests on a thinly-evidenced comparison rather than a clinically meaningful difference. Never present a ranking metric on its own slide, table row, or sentence; always pair it with the forest-plot or league-table output from §12.3.6/§12.4 so the reader sees the uncertainty the ranking number hides.

12.9 Reporting per NICE DSU TSD 2, ISPOR and PRISMA-NMA

An NMA feeding a TB programme reimbursement or guideline submission is judged as much on its reporting as on its statistical execution. Three documents anchor that reporting, two already introduced earlier in this chapter and one new.

NICE DSU TSD 2 (Dias, Welton, Sutton and Ades, 2011) — the primary methods reference cited throughout §§12.3-12.5 — implies, beyond the modelling framework itself, a concrete reporting checklist: a network diagram showing every treatment node and the trials connecting them (the `netgraph()` output from §12.3.4), a full evidence table listing every included study's design, population and outcome definitions, model fit statistics (DIC for Bayesian fits, or the common-vs-random comparison for frequentist fits), the consistency assessment results from §12.5 (node-split

estimates or the design-by-treatment decomposition), and the ranking results from §12.8 reported with their uncertainty, never as a bare league table or SUCRA number alone.

The ISPOR good practice task force report (Jansen et al., 2014, "Interpreting Indirect Treatment Comparisons and Network Meta-Analysis for Health-Care Decision Making") complements TSD 2 by framing the same checklist items as questions a decision-maker should ask of any submitted NMA: is the network geometry sound and fully connected for the comparisons that matter, is heterogeneity and inconsistency credibly assessed rather than merely asserted, and are the clinical and methodological similarity assumptions underlying transitivity made explicit rather than assumed by default. Where TSD 2 is the technical specification a sponsor builds against, the ISPOR paper is closer to the audit lens a payer or guideline committee applies to the finished product.

On the systematic literature review side that feeds the network, the PRISMA-NMA extension (Hutton et al., 2015, **Annals of Internal Medicine**) adds network-specific items to the base PRISMA checklist introduced in Chapter 6: an explicit description of the network geometry, the methods used to assess transitivity across the included trials, and how multi-arm trials were handled during extraction (a multi-arm trial contributes correlated contrasts, not independent pairwise comparisons — a point the `pairwise()` and `mtc.network()` functions in §§12.3-12.4 both handle internally, but which the write-up should still describe). The PRISMA flow diagram built with the PRISMA2020 package in Chapter 6 remains the natural entry point for the SLR; PRISMA-NMA extends the checklist items reported alongside that same diagram rather than requiring a second one.

> **Agency perspective.** NICE's Diagnostics Assessment and Technology Appraisal methods guidance requires submissions using network meta-analysis to report against the TSD 2 framework specifically, including a stated justification for the fixed- vs random-effects choice and documented consistency checking per TSD 4. In practice, appraisal committees have requested additional analyses, or discounted a submitted NMA's headline result, when the network diagram, evidence table, or inconsistency assessment specified in TSD 2 was missing from the submitted evidence package — the checklist is not a formality, it is routinely enforced.

Chapter 13. Population-Adjusted Indirect Comparisons

Chapter 12 closed with a warning: network meta-regression is data-hungry. It only adjusts for imbalance at the **study** level, using trial-arm covariate means as a coarse proxy for patient-level differences. This chapter takes up exactly that gap.

Picture the situation this way. One trial in your evidence base hands you individual patient data (IPD) under a data-sharing agreement. The comparator trial exists to you only as a published table of aggregate data (AgD). You suspect the two trial populations differ on covariates that **change the size of the treatment effect itself** — not just covariates that shift outcomes up or down by the same amount in both arms. That is exactly when population-adjustment methods become necessary.

This chapter covers three methods, in order of increasing data demand: matching-adjusted indirect comparison (MAIC, §13.2) and simulated treatment comparison (STC, §13.3) — both two-trial methods — then multilevel network meta-regression (ML-NMR, §13.4), which extends the same idea to a full network. The chapter closes with NICE DSU TSD 18 (§13.5), the single most important agency reference governing all three methods.

Learning objectives

- Distinguish an effect modifier from a pure prognostic factor, and explain why only the former biases an anchored indirect comparison under covariate imbalance
- Explain when population adjustment is needed instead of, or in addition to, standard anchored indirect comparison (Bucher/NMA, Chapter 12)
- Implement a basic matching-adjusted indirect comparison (MAIC) in R: estimate odds-weights via method of moments, compute effective sample size (ESS), and fit a weighted Cox model for time-to-event outcomes
- Explain why a large ESS loss after MAIC weighting is a credibility concern that must be reported, not a cosmetic detail
- Contrast MAIC's weighting-model assumption with simulated treatment comparison's (STC) outcome-regression-model assumption, and explain the non-collapsibility pitfall that threatens STC on nonlinear outcome scales
- Describe how multilevel network meta-regression (ML-NMR) generalises STC's logic to a full network of IPD and AgD-only trials via numerical integration over each AgD trial's reported covariate distribution, using the `multinma` package introduced in Chapter 12
- State NICE DSU TSD 18′s core requirements for a population-adjusted submission: prespecified effect modifiers, covariate-balance reporting, ESS reporting, and its stated method preference ordering
- Explain why an unanchored population-adjusted comparison carries materially higher risk of bias from unmeasured confounding than an anchored one, and why it should be a last resort

13.1 When population adjustment is needed

Chapter 12 closed with a warning worth repeating before we go further: standard network meta-regression is data-hungry. In practice it adjusts for covariate imbalance only at the **study** level, using trial-arm means as proxies for patient-level confounding and effect modification. That is a coarse instrument. Sometimes the trials you actually have won't cooperate with even that: one trial hands you full individual patient data (IPD) under a data-sharing agreement, while the comparator trial(s) exist to you only as a published table of aggregate data (AgD). Ecological regression on study-level covariates is often not possible in that situation, and even when it is, it is fragile.

This is the precise situation population-adjustment methods are built for: *one IPD trial, one or more AgD trials, connected through a common comparator, with a real concern that effect modifiers are distributed differently across the trial populations.* If the trials' populations are similar enough on anything that matters, ordinary anchored indirect comparison — Bucher's method or the NMA machinery of Chapter 12 — remains the right tool, and it is more efficient and more transparent than anything in this chapter. Population adjustment is a response to imbalance, not a default upgrade.

The methodological fork in the road is the distinction between *effect modifiers* and *prognostic factors*, and getting it right is not a technicality — it determines whether MAIC or STC is justified at all, and which covariates belong in the adjustment.

- A *prognostic factor* predicts outcome risk regardless of treatment: it shifts outcomes up or down in both arms more or less equally. Baseline disease severity, for instance, often predicts worse outcomes whether a patient is on the active drug or the comparator.
- An *effect modifier* changes the **relative** treatment effect itself: the treatment works differently (not just outcomes differently) depending on the covariate's value. Biomarker status that predicts a larger drug benefit in biomarker-positive patients is a classic effect modifier.

For an *anchored* comparison (both trials share a common comparator arm, and we are estimating a relative effect via that anchor), imbalance in pure prognostic factors across the two trial populations cancels out in the within-trial contrast — it does not bias the indirect comparison, because prognostic effects are differenced away on each side of the anchor before the two relative effects are compared. Imbalance in **effect modifiers**, by contrast, does not cancel: if the IPD trial's population has a different effect-modifier distribution than the AgD trial's population, the two trials' relative treatment effects are not estimating the same quantity, and the ordinary indirect comparison is biased. Population adjustment exists specifically to re-weight or re-model the IPD trial so that its effect-modifier distribution matches the AgD trial's, before the relative effect is computed.

> **Common pitfall.** A common and costly error is to throw every available baseline covariate into the matching or outcome-regression model — age, sex, disease duration, biomarker status, prior therapies — on the reasoning that "more adjustment is more rigorous." It is the opposite. Adjusting for a pure prognostic factor that is **not** an effect modifier does nothing for bias and only burns effective sample size (see §13.2) and inflates variance. Worse, for an *unanchored* comparison (single-arm or no shared comparator), the prognostic/effect-modifier distinction collapses — **all** prognostic factors and effect modifiers must be balanced, because there is no within-trial differencing to cancel prognostic imbalance. Decide the anchor status and the covariate roles (ideally pre-specified with clinical input, not chosen post hoc by which variables happen to "improve" the result) before fitting anything.

The two workhorse methods for this setting — matching-adjusted indirect comparison (MAIC) and simulated treatment comparison (STC) — differ in **how** they re-express the IPD trial in the AgD trial's population: one reweights patients, the other reshapes a regression model. We take each in turn.

13.2 Matching-adjusted indirect comparison (MAIC)

MAIC reweights the IPD-trial patients so that the **weighted** IPD population's covariate moments — typically the means, occasionally also variances for continuous covariates — match the moments reported in the AgD trial's publication (usually just means and sometimes SDs, since that is all an aggregate table ever gives you). The weights are not arbitrary: they come from a logistic-regression-style propensity model fit by method of moments, so that the resulting population looks, on the matched covariates, like the comparator trial's population.

The weighting mechanism. For each matched covariate x_j, define the IPD patient's covariate vector centered on the AgD trial's reported mean: $z_i = x_i - |x|^{\text{AgD}}$. MAIC assigns each IPD patient i an odds-weight

$$w_i = \exp(z_i^T \alpha)$$

where α is chosen so that the weighted means of z in the IPD data are exactly zero — i.e., the weighted IPD sample's covariate means equal the AgD trial's reported means. This is a method-of-moments problem solved by minimizing a convex objective (the method is numerically identical to calibrating a logistic propensity score without ever observing the AgD patients' outcomes or covariates individually — only their published moments are needed).

```
# Toy MAIC weight estimation via method of moments
# x: matrix of IPD covariates to match (e.g., age, baseline severity)
# target_means: named vector of AgD-trial reported means for the same covariates

Q <- function(alpha, z) {
  # z = x - target_means, centered IPD covariates
  sum(exp(z %*% alpha))
}

z <- scale(ipd[, c("age", "severity")], center = target_means, scale = FALSE)

fit <- optim(
  par = rep(0, ncol(z)),
  fn  = Q,
  z   = z,
  method = "BFGS"
)

alpha_hat <- fit$par
w <- exp(z %*% alpha_hat)        # odds-weights, one per IPD patient
w <- w / mean(w)                  # optional rescaling, mean weight = 1
```

How to report this.

Covariate	IPD, unweighted	IPD, MAIC-weighted	AgD trial (target)
Age, years (mean)	58.2	61.4	61.4
Baseline ECOG status ≥ 1, %	42	55	55
Prior-therapy count (mean)	1.3	2.1	2.1

After weighting, the IPD sample's matched covariate means equal the AgD trial's reported means by construction (method-of-moments calibration); any covariate omitted from the matching set is not guaranteed to balance and should be checked separately.

Table 39: Baseline covariate balance between the IPD trial and the AgD trial, before and after MAIC odds-weighting

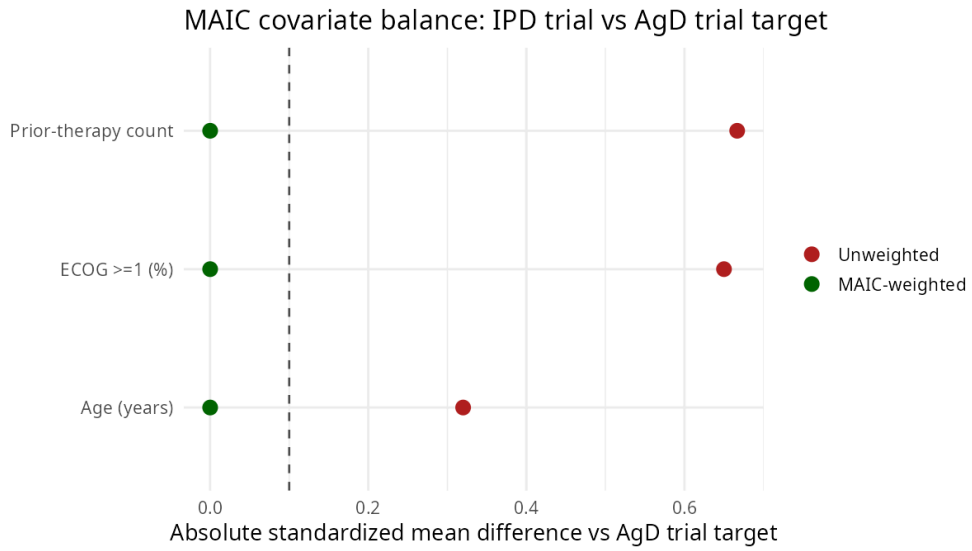


Figure 21: A love plot of the same covariate balance shown in Table 39: absolute standardized mean differences between the IPD trial and the AgD trial's reported target, before (red) and after (green) MAIC odds-weighting. By construction, MAIC drives every matched covariate's SMD to exactly zero — the diagnostic that matters is whether any **unmatched** effect modifier remains imbalanced. SMDs computed against illustrative typical-scale standard deviations for each covariate type, since the chapter's worked example reports only means, not SDs.

Effective sample size (ESS). Reweighting trades bias reduction for a loss of precision, and that loss must be quantified and reported, not buried in a supplementary table. The standard diagnostic is

$$\text{ESS} = \frac{\left(\sum_i w_i\right)^2}{\sum_i w_i^2}$$

which answers "how many **unweighted** patients would given this much statistical information?" A MAIC that starts with $N = 400$ IPD patients but produces $\text{ESS} = 120$ has thrown away roughly

70% of its effective information — a real quality-of-evidence red flag. A large ESS drop usually signals that the IPD and AgD populations barely overlap on the matched covariates, that a few patients are carrying almost all the weight, and that the resulting estimate's confidence interval will be much wider (and its extrapolation much less trustworthy) than the nominal sample size would suggest. Reviewers and HTA bodies expect ESS reported alongside the original N for exactly this reason.

```
ess <- sum(w)^2 / sum(w^2)
ess
```

HEOR in practice. Suppose an IPD trial of 400 patients is matched to an AgD trial on two effect modifiers: baseline ECOG status and prior-therapy count. After solving for the odds-weights, 15 patients at the extreme ends of the covariate distributions receive weights 8–12 times the mean weight, while the bulk of patients are weighted near zero. Plugging the resulting weights into the ESS formula gives $\frac{(\sum w_i)^2}{\sum} w_i^2 \approx 120$. The matched comparison should be reported with both numbers side by side — "IPD N = 400, ESS = 120 (30%)" — so a reviewer can immediately gauge how much of the original trial's information survived the reweighting, rather than comparing standard errors for the matched analysis on a seemingly unchanged 400-patient footing.

How to report this.

Quantity	Value	% of unweighted N
IPD sample size, N (unweighted)	400	100%
Effective sample size, ESS (MAIC-weighted)	120	30%
Report ESS alongside the raw N in every MAIC analysis; an ESS this far below N signals that a small number of extreme weights are driving the matched estimate and that its confidence interval will be wider than the nominal N suggests.		

Table 40: Effective sample size before and after MAIC weighting (illustrative 400-patient IPD trial matched on two effect modifiers)

Time-to-event outcomes. MAIC's reweighting logic extends directly to survival outcomes by using the odds-weights w_i as case weights in a weighted Kaplan–Meier curve and a weighted Cox model — no new estimation machinery is needed, only weighted versions of the usual estimators.

```
library(survival)

# Weighted Cox model using MAIC odds-weights as case weights
cox_fit <- coxph(
  Surv(time, event) ~ treatment,
  data    = ipd,
  weights = w          # odds-weights from the MAIC step above
)

summary(cox_fit)
```

```r
# Weighted Kaplan-Meier for the matched IPD population
km_fit <- survfit(Surv(time, event) ~ treatment, data = ipd, weights = w)
```

Robust (sandwich-type) standard errors are typically preferred over the naive weighted-Cox standard errors here, since the weights are themselves estimated quantities and ordinary Cox variance formulas do not account for that extra uncertainty; `coxph()` with `cluster()` or a bootstrap over the full weight-estimation-plus-outcome-model pipeline are the usual fixes, and this is worth flagging explicitly in any write-up.

> **Difficult terminology: robust (sandwich-type) standard errors.** A standard error tells you how much a parameter estimate would bounce around if you repeated the study; the usual formula for calculating it assumes the model's error structure is exactly right. "Robust" or "sandwich-type" standard errors are a way of calculating that same uncertainty that doesn't lean on that assumption being correct — they're called "sandwich" because the formula literally sandwiches a middle piece (built from the observed data's actual variability) between two outer pieces derived from the model. Here, that matters because the MAIC weights fed into the Cox model aren't fixed, known numbers — they were themselves **estimated** from the matching step, carrying their own uncertainty, which an ordinary (non-robust) standard error would silently ignore, making the reported confidence interval too narrow and the result look more precise than it really is. Think of it as the statistical equivalent of accounting for the fact that your ruler itself has some wobble, rather than assuming every measurement taken with it is exact.

A practical note on tooling: there is no single dominant, actively maintained CRAN package that performs MAIC end to end. Most published MAICs are implemented by hand in R, essentially as the `optim()`-based weighting step above followed by a weighted outcome model. A `maic` package exists on GitHub and is used by some HTA submissions, but it is *not on CRAN*, has not been through CRAN's review process, and should be treated as unreviewed third-party code — inspect its weighting and variance-estimation internals line by line, or reproduce its output with your own `optim()` call, before relying on it for a submission.

13.3 Simulated treatment comparison (STC)

STC addresses the same imbalance problem as MAIC but by a different mechanism. Instead of reweighting patients, STC fits an *outcome regression model* on the IPD trial — the treatment effect plus the matched covariates as predictors — and then predicts what the outcome would have been had the IPD trial's population had the AgD trial's reported covariate means. Where MAIC asks "how do I reweight my patients to look like theirs?", STC asks "what does my outcome model say would happen to a population with their average characteristics?"

Concretely, for a continuous or binary outcome:

```r
# Outcome regression model on the IPD trial, including a
# treatment x covariate interaction to capture effect modification
stc_fit <- glm(
  outcome ~ treatment * age + treatment * severity,
  data   = ipd,
  family = binomial()
)
```

```r
# Predict the treatment effect at the AgD trial's reported means
newdata <- data.frame(
  treatment = c(0, 1),
  age       = target_means["age"],
  severity  = target_means["severity"]
)

preds <- predict(stc_fit, newdata = newdata, type = "link")
stc_log_odds_ratio <- preds[2] - preds[1]
```

How to report this.

Term	Estimate	SE	p
Treatment	0.41	0.18	0.023
Age (centered)	−0.02	0.01	0.085
Severity (centered)	−0.31	0.12	0.010
Treatment × age	0.015	0.019	0.431
Treatment × severity	0.28	0.11	0.012
Predicted log-odds ratio at the AgD trial's reported means (age = 61.4, severity index = 2.1): 0.52, i.e. OR ≈ 1.68 – a **conditional** effect at those covariate values, not yet the population-average effect; see the non-collapsibility caution above before reporting this as the final STC estimate.			

Table 41: Illustrative STC outcome-regression coefficients (logistic model, IPD trial, treatment × covariate interactions), and the resulting log-odds-ratio predicted at the AgD trial's reported covariate means

The appeal is efficiency: STC uses all N IPD patients in the regression rather than discarding information to an ESS-reduced weighted subsample, so it does not suffer MAIC's precision penalty from extreme weights. The cost is a stronger assumption: STC's validity depends on *correctly specifying the outcome model*, including its functional form, which covariates (and interactions) belong in it, and — critically for nonlinear models — the right scale on which to summarize the "effect at the mean."

Common pitfall. For a linear model on a continuous outcome, "predict the outcome at the covariate means" and "the average outcome across the covariate distribution" give the same answer, so STC is relatively safe there. For *nonlinear* outcome models — logistic regression on odds ratios, Cox regression on hazard ratios — they do not. The reason has a name: *non-collapsibility*. The conditional treatment effect (adjusted for covariates, evaluated at their mean) and the marginal treatment effect (averaged over the whole covariate distribution) are two different quantities on the odds-ratio or hazard-ratio scale. This is true even when the model is perfectly specified. So plugging the AgD trial's mean covariate values into a fitted logistic or Cox model, and reading off the "treatment effect," gives a **conditional** effect at

those particular covariate values. It does not give the **population-average** effect that an aggregate-data trial reports — and that the indirect comparison needs to match. Treating these as interchangeable biases the comparison, even when every covariate is balanced and the outcome model is otherwise correct. The fix: simulate outcomes across the full joint covariate distribution implied by the AgD trial's reported summary statistics, average the simulated individual predictions on the natural outcome scale, then transform back to odds ratios or hazard ratios. This simulation step is the "simulated" in the method's name. Skipping it is the single most common STC implementation error.

Difficult terminology: conditional treatment effect / marginal treatment effect. These two phrases sound like they should mean almost the same thing, but on a nonlinear scale (odds ratio, hazard ratio) they genuinely do not, and the gap between them is the whole reason non-collapsibility matters. A **conditional** effect is "the treatment effect for a patient (or group of patients) who all share one specific set of covariate values" — e.g. "the odds ratio for a 61-year-old patient with a severity score of 2.1," read straight off a fitted regression's coefficient. A **marginal** effect is "the treatment effect averaged across the whole population's actual mix of covariate values" — the number an aggregate-data trial's reported odds ratio or hazard ratio actually represents, since that trial never conditioned on anyone's individual covariates in the first place.

A quick worked example makes the gap concrete. Simulate 100,000 patients, half with mild disease and half with severe, with a treatment that has exactly the same true log-odds effect (0.7, i.e. an odds ratio of about 2.0) in both subgroups — no genuine effect modification by severity at all, by construction:

```
set.seed(1)
n <- 100000
severity <- rbinom(n, 1, 0.5)   # 0 = mild, 1 = severe
treat <- rbinom(n, 1, 0.5)
lp <- -1 + 0.7 * treat + 1.5 * severity  # constant treatment log-odds effect
y <- rbinom(n, 1, plogis(lp))

m <- glm(y ~ treat + severity, family = binomial)
exp(coef(m)["treat"])  # conditional OR, adjusted for severity
```

Fitting a logistic regression recovers a **conditional** odds ratio of about **1.98** — close to the true 2.0, as expected. But now average the model's predicted probabilities under "everyone treated" versus "no one treated," across the real mix of mild and severe patients in the sample, and convert that to an odds ratio:

```
p1 <- mean(predict(m, newdata = transform(df, treat = 1), type = "response"))
p0 <- mean(predict(m, newdata = transform(df, treat = 0), type = "response"))
(p1 / (1 - p1)) / (p0 / (1 - p0))  # marginal OR
```

This gives a **marginal** odds ratio of about **1.82** — noticeably smaller, despite the underlying model being exactly right and having no real effect modification at all. Nothing is wrong with the model; odds ratios themselves simply don't average the way risk differences do (a risk difference, by contrast, collapses cleanly — this is specifically an odds-ratio/hazard-ratio problem, not a general statistics problem). STC's entire extra "simulate across the covariate

distribution" step described above exists purely to convert the easy-to-fit conditional estimate into the marginal estimate that can actually be compared against an aggregate trial's reported result.

The practical trade-off between the two methods comes down to **which** model you must get right. MAIC's validity rests on correctly specifying the *weighting/matching model* — which covariates drive the propensity-style weights — but it makes no assumption about the outcome's functional form; the weighted outcome analysis can be as simple as a weighted Kaplan–Meier curve. STC's validity rests on correctly specifying the *outcome regression model*, including its functional form and the covariate-by-treatment interactions that encode effect modification. But STC uses the full IPD sample, with no ESS penalty. Neither assumption dominates the other in general. The choice usually comes down to how much confidence you have in modelling the outcome mechanism versus modelling the covariate-balancing mechanism, and how severe the expected ESS loss would be under MAIC given how much the two trial populations actually overlap.

Agency perspective. NICE DSU Technical Support Document 18 (Phillippo et al., 2016) is the standard methodological reference underpinning both MAIC and STC, and sets out NICE's expectations for reporting ESS, covariate selection, and sensitivity analyses in a submission. §13.5 of this chapter covers TSD 18′s guidance in full; treat the summary here only as a signpost to that fuller discussion.

13.4 Multilevel network meta-regression (ML-NMR)

MAIC and STC, as developed in §13.1–13.3, share a structural limitation: both operate on exactly two trials — one with individual patient data (IPD) and one reporting only aggregate data (AgD) — connected by a single comparison. The moment either trial sits inside a larger network of three or more treatments, these two-trial methods have no natural extension: you would need to run a separate MAIC or STC for every AgD trial against the IPD trial, throwing away the internal consistency that a proper network provides and giving no single coherent treatment-effect ranking across the whole network.

Multilevel network meta-regression (ML-NMR), developed by Phillippo, Ades, Dias, Welton, and colleagues (Phillippo et al. 2020, **Journal of the Royal Statistical Society Series A**), closes this gap. It generalises population adjustment from the two-trial case to a full network containing an arbitrary mix of IPD and AgD-only studies. The core idea extends STC's logic rather than replacing it: fit an individual-level regression model for the outcome — treatment, effect-modifying covariates, and treatment-by-covariate interactions — using whichever trials contribute IPD. For the trials that report only AgD, instead of plugging in a single covariate mean (as STC does), ML-NMR numerically integrates the individual-level prediction equation over each AgD trial's **reported covariate distribution**. When an AgD trial publishes not just means but also standard deviations and, ideally, correlations between effect modifiers, ML-NMR uses that fuller distributional information, rather than collapsing it to a point. This is what makes ML-NMR more defensible than moment-matching approaches when the data exist to support it — but also considerably more data-demanding, since it requires AgD trials to report enough distributional detail (not just means) for the covariates that matter.

ML-NMR is not a new tool bolted onto this book's toolkit — it is the same `multinma` package (Phillippo et al.) already introduced in Chapter 12 §12.4 for Bayesian network meta-analysis, used in exactly the same workflow. There, you built a network with `set_ipd()` and `set_agd_arm()` and fit it with `multinma::nma()` assuming no effect modification. ML-NMR is what happens when that regression model is allowed to include covariate-by-treatment interaction terms, and the AgD arms are no longer treated as covariate-free: each AgD study additionally needs numerical integration points describing its covariate distribution so that the individual-level model's predictions can be correctly marginalised to that study's population. The documented function for generating these integration points is `multinma::add_integration()`, which builds Quasi-Monte Carlo integration points (via a Gaussian copula and Sobol' sequences) from each AgD study's reported covariate means, SDs, and — where available — correlations.

Difficult terminology: Sobol' sequences / Quasi-Monte Carlo integration points. "Numerically integrating" over a covariate distribution just means: rather than plugging in one single average value (as STC does), generate a large number of representative sample points that collectively cover the whole range of plausible covariate values for that trial's population, and average the model's prediction over all of them. Ordinary Monte Carlo would do this with purely random points, which cluster unevenly by chance (like randomly scattering dots on a dartboard — some areas get too many, some too few). A Sobol' sequence instead generates points in a carefully structured, non-random pattern that spreads them out evenly across the space — closer to laying dots on a grid, but cleverer, so that even a modest number of points gives a good, stable approximation of the integral. "Quasi-Monte Carlo" is the general name for this family of structured-point integration techniques, and Sobol' sequences are simply the most common recipe used to generate the points.

Difficult terminology: Gaussian copula. When an AgD trial reports several covariates' means, SDs, and the correlations between them (e.g. age and disease severity tend to rise together), you need a way to generate **jointly** realistic sample values for all the covariates at once — not just realistic values for each covariate separately, ignoring how they relate to each other. A Gaussian copula is the mathematical device that does this: it takes each covariate's own individually-specified distribution (which doesn't have to be Normal at all) and "glues" them together using a correlation structure borrowed from the multivariate Normal distribution, producing joint sample points that respect both each covariate's own shape and the reported correlations between them. A rough analogy: imagine you know the typical height and weight distributions separately, plus the fact that taller people tend to be heavier — a copula is the recipe for generating pairs of (height, weight) values that look like genuine people, rather than randomly mixing a height from one person with a weight from someone unrelated.

A schematic workflow, keeping the illustrative code deliberately light and consistent with the Chapter 12 pattern:

```
library(multinma)

# Network combining IPD and AgD-only trials, as in Ch.12 §12.4
net <- combine_network(
  set_ipd(ipd_trial, study = study, trt = trt, y = outcome,
```

```
          trt_class = NULL),
  set_agd_arm(agd_trials, study = study, trt = trt,
              r = events, n = n)
)

# Supply numerical integration points for each AgD trial's
# covariate distribution -- consult the current multinma
# documentation for the exact distribution-argument syntax,
# which has evolved across package versions.
net_int <- add_integration(
  net,
  age = distr(qnorm, mean = age_mean, sd = age_sd),
  ecog = distr(qbern, prob = ecog_prop)
)

# Fit the regression model with effect-modifier interactions
ml_nmr_fit <- nma(net_int,
                  trt_effects = "fixed",
                  regression = ~ age + ecog,
                  class_interactions = "common")
```

The call shapes for `set_ipd()`, `set_agd_arm()`, and `nma()` mirror Chapter 12 exactly; `add_integration()` is the one genuinely new step, and its covariate-distribution argument syntax has changed across `multinma` releases, so treat the `distr(qnorm, ...)` style above as illustrative rather than copy-ready — always check the installed version's help page and vignette before running a submission analysis.

> **HEOR in practice.** Think of ML-NMR as "STC done properly across a whole network." STC matches an AgD trial to one covariate value (its mean); ML-NMR matches it to the trial's **entire reported covariate distribution**, and does so simultaneously for every AgD trial in the network while preserving one internally consistent set of relative treatment effects. The price is data: ML-NMR only delivers its full benefit when AgD trials publish SDs and, where effect modifiers interact, correlations — not just means. When that reporting is thin, ML-NMR can gracefully fall back toward STC-like behaviour, but its defensibility advantage shrinks accordingly.

Method	Data requirement	Key assumption	Typical use case
MAIC	IPD for one trial + AgD (covariate means, sometimes SDs) for the comparator trial	Weighting/matching model is correctly specified; matched covariates include all relevant effect modifiers	Simple two-trial comparison with an IPD submission trial and one published AgD comparator, especially for survival outcomes via weighted Kaplan–Meier/Cox
STC	IPD for one trial + AgD (covariate means) for the comparator trial	Outcome regression model is correctly specified, including functional form and treatment-by-covariate interactions; non-collapsibility handled via simulation on nonlinear scales	Two-trial comparison where the analyst has strong confidence in the outcome model and wants to retain full IPD precision (no ESS penalty)
ML-NMR	IPD for some trials + AgD (covariate means, SDs, and ideally correlations) for the rest, across a full network	Individual-level regression model is correctly specified; AgD trials report enough distributional detail for numerical integration	Networks of three or more treatments mixing IPD and AgD-only trials, where a single internally consistent set of relative effects is required

Table 42: Comparison of the three population-adjustment methods covered in this chapter: data requirements, core assumption, and typical use case

13.5 Limitations, assumptions and agency expectations (NICE DSU TSD 18)

Every method introduced in this chapter — MAIC, STC, and ML-NMR — answers the same underlying problem (effect-modifier imbalance across trials that cannot be resolved by a standard anchored NMA or Bucher indirect comparison) with different trade-offs between data requirements and defensibility. NICE DSU Technical Support Document 18, **Methods for Population-Adjusted Indirect Comparisons in Submissions to NICE** (Phillippo, Ades, Dias, Palmer, Abrams, Welton; first published 2016, subsequently updated), is the standard reference that HTA bodies and manufacturers alike use to judge whether a population-adjusted analysis is appropriate and well-executed. It is the single most important named document in this chapter, and no submission using MAIC, STC, or ML-NMR should be prepared without direct reference to it.

Agency perspective. What TSD 18 actually requires, in outline:

- **Justify the need for adjustment.** A standard NMA or Bucher comparison is the default; population adjustment is only warranted when the company can demonstrate an effect-modifier imbalance problem between the trial populations — not simply assert that one might exist.

- **Prespecify effect modifiers before seeing comparative results.** The list of covariates to adjust for must be clinically justified and fixed in advance (via literature review, clinical input, or prior trials), independent of the analysis's own outcome.
- **Report covariate balance before and after weighting** (for MAIC) so reviewers can see what imbalance existed and how much of it was corrected.
- **Report the effective sample size (ESS)** after weighting and discuss what an ESS loss implies for the precision and credibility of the result.
- **State a method preference, data permitting.** TSD 18′s stated ordering, roughly, is ML-NMR > STC > MAIC: ML-NMR preserves a network's internal consistency and uses the full covariate distribution; STC retains the IPD trial's own outcome model and standard errors; MAIC, by reweighting the IPD sample to match only the AgD trial's reported moments, discards information and can inflate variance by effectively down-weighting the IPD trial's within-trial precision. Choose MAIC because it is simplest to implement, not because it is DSU 18′s preferred option.

Limitation	TSD 18′s stated concern
Target-population specificity	An adjusted estimate is valid only for the population it was adjusted to (typically the AgD trial's population); it does not carry the transportability assumption of a standard NMA, so a different target population requires a fresh adjustment
Unmeasured confounding in unanchored comparisons	Without a common comparator to absorb unmeasured differences, an unanchored analysis implicitly assumes every effect modifier and prognostic factor has been measured and correctly adjusted for – rarely fully defensible; unanchored designs carry materially higher bias risk than anchored ones
Post-hoc effect-modifier selection	Choosing or dropping covariates after seeing how they change the adjusted result is a recognised form of analytical bias; the effect-modifier list must be prespecified from external clinical and literature evidence before the adjusted analysis is run
ESS loss (MAIC specifically)	A large drop in effective sample size after weighting signals that a small number of extreme weights are driving the result and that the reported confidence interval may understate true uncertainty; ESS must be reported prominently, not in an appendix
Thin AgD reporting	ML-NMR's advantage over STC depends on AgD trials publishing covariate SDs and, where relevant, correlations – not just means; when that detail is absent, ML-NMR degrades toward STC-like behaviour and its defensibility edge shrinks

Table 43: NICE DSU TSD 18: named limitations of population-adjustment methods and the agency's stated concern for each

Common pitfall. Post-hoc effect-modifier selection. Choosing or dropping candidate effect modifiers **after** looking at how they change the adjusted treatment effect is a well-recognised form of analytical bias, and it is explicitly criticised by the DSU and by NICE Evidence Review Groups (ERGs) when detected. It lets an analyst chase the covariate set that produces the most favourable result while presenting it as a neutral, methods-driven choice. The defence is procedural, not statistical: fix the effect-modifier list from external clinical and literature evidence **before** running the adjusted analysis, document that process in the submission, and resist any temptation to revise the list once results are in hand.

Agency perspective. ESS as a credibility signal, not a footnote. For MAIC in particular, a large drop in effective sample size after weighting (e.g., an ESS that falls to a small fraction of the original IPD sample size) is not a cosmetic detail — ERGs treat it as a direct warning

sign that the weights are being driven by a small number of extreme observations, that the weighted estimate's reported confidence interval may understate true uncertainty, and that the result may not generalise robustly to the target population. Submissions should report ESS prominently alongside the adjusted estimate, not bury it in an appendix, and should discuss explicitly what the ESS loss implies for the analysis's reliability — silence on this point is itself a flag reviewers are trained to look for.

Two further limitations apply across all three methods and deserve to be stated plainly rather than left implicit.

First, **a population-adjusted estimate is only valid for the one target population it was adjusted to** — typically the AgD trial's population, since that is the population whose covariate distribution (or moments) the IPD trial was reweighted or regressed onto. This is a fundamentally different assumption from a standard NMA's transportability of relative effects, which assumes (rightly or wrongly) that a relative treatment effect generalises across reasonably similar populations. A MAIC-, STC-, or ML-NMR-adjusted estimate carries no such claim: adjust the same IPD trial to a different AgD trial's population and you may well get a different answer, and that is expected, not a contradiction. Decision-makers who want an estimate for a population other than the one used in adjustment need a fresh adjustment, not a reuse of the existing one.

Second, **unanchored comparisons deserve particular caution**. An anchored comparison (MAIC, STC, or ML-NMR performed within a network that still has a common comparator connecting the trials) uses that common comparator to cancel out between-trial differences that are not captured by the measured covariates — the anchor absorbs some of the risk from unmeasured confounding. An unanchored comparison, by contrast, arises when no such common-comparator link exists at all, so the entire burden of adjustment falls on the outcome regression model with no anchor to offset what that model misses.

Common pitfall. Unanchored comparisons are a last resort, not a default. TSD 18 is explicit that unanchored population-adjusted comparisons carry a materially higher risk of bias from unmeasured confounding and unmeasured effect modification than their anchored counterparts, because there is no common-comparator trial arm to absorb whatever the adjustment model fails to capture. An unanchored analysis implicitly assumes that **all** effect modifiers and prognostic factors have been measured and correctly adjusted for — an assumption that is rarely fully defensible. Reach for an unanchored design only when no anchored route exists, and when you do, report the limitation transparently and prominently in the submission rather than presenting the result with the same confidence as an anchored estimate.

Chapter 14. Bayesian Methods for HEOR

Chapter 12 used Bayesian network meta-analysis (`gemtc`, `multinma`, `bnma`) and flagged a `gelman.diag()` convergence check as mandatory. It did not explain **why** that check matters, or what to do when it fails. That explanation was deliberately saved for here.

This chapter has two jobs. First, it builds the Bayesian framework from first principles — what a prior, a likelihood and a posterior actually are, and how to compute one. Second, it fulfils Chapter 12′s promise: full MCMC diagnostics, done properly. Along the way it reaches the chapter's single most useful idea for everyday modelling work: a Bayesian posterior sample can be used **directly** as a PSA sample, with no extra distribution-fitting step in between.

The chapter follows a simple arc, section by section:

- §14.1-14.2 build the concepts: what Bayesian updating is, and how to choose a prior.
- §14.3 introduces the three tools that turn those concepts into actual posterior draws: Stan, JAGS, and `brms`.
- §14.4 is the diagnostics discipline — the part Chapter 12 deferred.
- §14.5-14.6 connect evidence synthesis to decision-making: turning posterior draws into PSA input.
- §14.7 covers expert elicitation (SHELF), for when there is no data at all.
- §14.8 closes with a roadmap tying every piece together, and pointing forward to Part IV.

Learning objectives

- Explain the prior/likelihood/posterior relationship and contrast a Bayesian credible interval's interpretation with a frequentist confidence interval's interpretation
- Choose between informative, weakly informative and vague priors for a given parameter, and justify the choice with a documented rationale and a prior sensitivity check
- Fit a simple Bayesian model using `rstan`, `rjags`, and `brms`, and explain when each tool is the natural choice
- Explain Metropolis-Hastings, Hamiltonian Monte Carlo and the No-U-Turn Sampler at a conceptual level sufficient to interpret why some models converge faster than others
- Check MCMC convergence using the Gelman-Rubin statistic (R-hat), assess effective sample size, and run a posterior predictive check before trusting any posterior summary
- Describe Bayesian evidence synthesis as a pipeline that produces a joint posterior sample, building on Chapter 12′s Bayesian NMA machinery without re-deriving it
- Use posterior draws directly as a PSA sample for a cost-effectiveness decision model, preserving parameter correlation structure without fitting an intermediate distribution
- Describe the Sheffield Elicitation Framework (SHELF) process for eliciting a parameter's distribution from expert judgement when no usable data exist, and use the `SHELF` R package to fit and inspect the resulting distribution
- Trace the end-to-end Bayesian HEOR workflow from prior specification through evidence synthesis, MCMC fitting, diagnostics and PSA to reporting, and locate where it hands off to Part IV's uncertainty (Ch.25) and value-of-information (Ch.26) chapters

14.1 Bayesian thinking for decision makers

Every health economic model you have built so far in this book has used probability in a particular way. A transition probability, a hazard ratio, a utility weight — each has been treated as a fixed, if uncertain, number, and you have expressed that uncertainty by attaching a distribution to it and running probabilistic sensitivity analysis. That workflow already contains a Bayesian idea, even if nobody called it that: a distribution over a parameter represents what you believe about a quantity you cannot observe directly. This section makes that idea explicit and gives it a name, because from here through the rest of the book — cost-effectiveness model parameterisation, value of information, adaptive trial designs — Bayesian reasoning is the natural language for decisions made under uncertainty.

Start with three ingredients.

A *prior*, $\pi(\theta)$, is what you believe about a parameter θ before looking at today's data. It might come from an earlier trial, an expert elicitation exercise, or simply a wide, non-committal range.

A *likelihood*, $L(\theta \mid y)$, is how probable the data y you actually observed would be, for each possible value of θ. This is the same likelihood function behind your maximum-likelihood fits from earlier chapters.

Combine the two with Bayes' theorem and you get the *posterior*, $p(\theta \mid y)$ — your updated belief about θ, after seeing the data:

$$p(\theta \mid y) = \frac{L(y \mid \theta)\pi(\theta)}{\int L(y \mid \theta)\pi(\theta)\,\mathrm{d}\theta}$$

The denominator just rescales things so the posterior integrates to one. The relationship you actually use day to day is much simpler:

$$\text{posterior} \propto \text{likelihood} \times \text{prior}$$

Everything in this chapter is an exercise in computing or approximating that product. Picture a tug-of-war: the likelihood pulls toward what the current data say; the prior pulls toward what was already believed; and the strength of each pull depends on how much information each one carries. A small pilot study of 20 patients has a weak, spread-out likelihood — a well-chosen prior can meaningfully sharpen the estimate. A large national surveillance dataset has a sharp likelihood that will dominate almost any reasonable prior.

Contrast this with the frequentist framing you have used for most of this book. A frequentist 95% confidence interval is a statement about the long-run behaviour of an estimation *procedure*: if you repeated the same sampling process many times, 95% of the intervals you construct would contain the true parameter. It is not a statement of 95% probability about the parameter itself, no matter how often it is informally described that way. A Bayesian 95% credible interval, by contrast, *is* a direct probability statement: given this data and this prior, there is a 95% probability that θ lies in this interval. This is not a technicality — it is exactly the kind of statement a reimbursement committee, a Ministry of Health budget holder, or a WHO technical working group actually wants to hear, and it is why Bayesian output translates so cleanly into decision language without the usual hedging.

This also explains why Bayesian methods sit so naturally inside decision modelling. A cost-effectiveness analysis is ultimately a decision problem: given everything currently known, should

we recommend this intervention? The expected net monetary benefit, the probability that an intervention is cost-effective at a given threshold, the expected value of perfect information — all of these are posterior quantities, computed by integrating a decision function over a distribution of parameter uncertainty. When you ran probabilistic sensitivity analysis in earlier chapters by sampling parameters from distributions and tabulating the proportion of simulations below a willingness-to-pay threshold, you were already doing applied Bayesian decision analysis, just without a likelihood update step. The step this chapter adds is the machinery for formally updating those parameter distributions with real trial or surveillance data, rather than specifying them by assumption or extraction from a single published estimate.

There is also a scientific-honesty argument for working this way in TB and other high-burden, data-scarce programmes. Frequentist methods implicitly treat each study in isolation, re-estimating uncertainty from scratch each time. Bayesian updating lets you formally carry forward what a previous regional drug-resistance survey or an earlier cohort's treatment-success rate told you, combine it transparently with new programme data, and report exactly how much the new evidence moved your estimate. That transparency — prior stated, likelihood stated, posterior shown — is precisely the kind of audit trail that evidence reviewers increasingly expect, and it is the foundation the rest of this chapter builds on.

> **HEOR in practice.** When presenting Bayesian results to a non-technical decision committee, lead with the posterior probability statement ("there is an 87% probability that the intervention is cost-effective at this threshold") rather than the mechanics of the prior-likelihood combination. Reserve the prior/likelihood/posterior language for technical appendices or methods sections aimed at reviewers, where it needs to be fully transparent and reproducible.

> **Agency perspective.** NICE DSU Technical Support Document 18 (**Methods for Population-Adjusted Indirect Comparisons**), which you met in Chapter 13, explicitly recommends Bayesian estimation for several population-adjustment methods because it propagates parameter uncertainty coherently into the final decision metric without relying on asymptotic normality assumptions that can break down in small populated subgroups — exactly the situation common in TB sub-population analyses.

14.2 Priors: informative, weakly informative and vague priors

Choosing a prior is the step in Bayesian analysis that makes newcomers most nervous, usually because it feels like injecting a subjective opinion into what should be an objective calculation. The more useful framing is that every statistical model already makes assumptions — a frequentist generalised linear model assumes a link function, an error distribution, and an absence of unmeasured confounding, none of which are "objective" in any strong sense. A prior simply makes one particular assumption explicit, documented, and open to sensitivity testing, which is in many ways more honest than burying equivalent assumptions in model structure.

Informative priors encode substantial prior knowledge about a parameter, usually from a previous study, a meta-analysis, or formal expert elicitation. If a prior national TB survey found treatment success in 78% of patients with a standard error corresponding to a $\text{Beta}(150, 42)$ distribution, and you are now analysing a new but related cohort, using that as your prior is not

cheating — it is using the discipline's accumulated evidence rather than discarding it. The key discipline is documenting *where the prior came from* and *why it is relevant to the new analysis*, exactly as you would document an extrapolation assumption in a Markov model.

Weakly informative priors rule out implausible values without expressing a strong opinion among the plausible ones. For a log hazard ratio, a $\text{Normal}(0, 2.5)$ prior on the log scale is weakly informative: it says hazard ratios somewhere between roughly 0.007 and 150 are conceivable, but a hazard ratio of a trillion is not, which keeps the sampler well-behaved and rules out numerically absurd solutions without pulling the estimate toward any particular value. These are the default workhorse priors in most modern Bayesian regression work, including the `brms` defaults you will use in §14.3.

Vague (or "flat") priors attempt to express complete prior ignorance, for example a $\text{Uniform}(-\infty, \infty)$ prior on a regression coefficient, or a $\text{Beta}(1, 1)$ — equivalent to $\text{Uniform}(0, 1)$ — on a probability. In simple, well-identified models these behave reasonably and the posterior is driven almost entirely by the likelihood. But truly flat priors on an unbounded scale are improper (they do not integrate to one) and can cause real problems in more complex hierarchical models, where they place surprising amounts of prior mass on extreme values once transformed through a nonlinear link function. In practice, a weakly informative prior usually does the job a vague prior was trying to do, with fewer pathologies.

A practical distinction worth internalising is **conjugate versus non-conjugate priors**. A prior is conjugate to a likelihood when the resulting posterior belongs to the same distributional family as the prior, which makes the update available in closed form. The classic HEOR example is the Beta-Binomial pair: if θ is a probability (say, treatment success) with prior $\theta \sim \text{Beta}(\alpha, \beta)$, and you observe y successes out of n trials, the posterior is simply

$$\theta \mid y \sim \text{Beta}(\alpha + y, \beta + n - y)$$

no simulation required. You can compute and plot this directly in R:

```
# Beta-Binomial conjugate update: TB treatment success rate
alpha_prior <- 150   # prior "successes"
beta_prior  <- 42    # prior "failures"

y <- 34   # observed successes in new cohort
n <- 40   # total patients in new cohort

alpha_post <- alpha_prior + y          # 184
beta_post  <- beta_prior + (n - y)     # 48

post_mean   <- alpha_post / (alpha_post + beta_post)   # 0.793
post_median <- qbeta(0.5, alpha_post, beta_post)
cri_95      <- qbeta(c(0.025, 0.975), alpha_post, beta_post)

cat(sprintf("Posterior mean: %.3f\n", post_mean))
cat(sprintf("95%% credible interval: [%.3f, %.3f]\n", cri_95[1], cri_95[2]))
```

With $\alpha_{\text{prior}} = 150$, $\beta_{\text{prior}} = 42$, $y = 34$ and $n = 40$, the update gives $\alpha_{\text{post}} = 150 + 34 = 184$ and $\beta_{\text{post}} = 42 + (40 - 34) = 48$, so the posterior mean is $184/(184 + 48) = 0.793$ — barely moved from the prior mean of $150/192 = 0.781$, because the new cohort of 40 patients carries far less

information than the 192 "pseudo-observations" already encoded in the prior. This is exactly the tug-of-war intuition from §14.1 made concrete.

How to report this.

Quantity	Prior	Posterior
Distribution	Beta(150, 48)	Beta(184, 48)
Mean	0.781	0.793
95% interval	(0.714, 0.841)	(0.738, 0.843)

Table 44: Posterior summary for TB treatment success probability, Beta-Binomial conjugate update ($n = 40$ new cohort patients, $y = 34$ successes, informative prior $\text{Beta}(150, 42)$)

Other conjugate pairs worth knowing are Normal-Normal (for a mean with known variance) and Gamma-Poisson (for a rate). Conjugacy is a mathematical convenience, not a requirement — most realistic HEOR models (hierarchical survival models, mixed-effects utility regressions, network meta-analyses with covariate adjustment) have no conjugate solution, which is exactly why §14.3 introduces MCMC: it lets you approximate the posterior numerically when no closed form exists.

Whatever prior family you choose, **always run a prior sensitivity analysis**. Refit the model under at least two or three materially different, individually defensible priors (for example, your informative prior, a weakly informative alternative, and a vague one) and report whether the posterior conclusions change. If the data are informative relative to the prior, the posteriors will look similar regardless of which prior you used, and you can say so with confidence. If they diverge noticeably, that divergence is itself an important finding — it tells the decision maker that current evidence is not yet strong enough to overrule prior belief, which is a very different message than reporting a single, confidently-stated number.

Common pitfall. Do not reach for a "flat" or improper prior reflexively just to seem neutral. On transformed scales (odds ratios, hazard ratios, variance components) a flat prior on one scale can be very informative — and in a distorting way — on another. A Uniform prior on an odds ratio from 0 to 1000, for instance, puts almost all of its prior mass above an odds ratio of 10, which is rarely a neutral starting point for a clinical effect.

HEOR in practice. Report your prior choices in a dedicated subsection of the methods, including the exact distributional form and parameters, the source or rationale, and the results of at least one prior sensitivity check. This is now standard expectation in Bayesian HTA submissions and mirrors the transparency already required for deterministic and probabilistic sensitivity analysis assumptions elsewhere in your model.

14.3 MCMC with Stan, JAGS and brms

Conjugate models give you an exact posterior in closed form, but almost no model you will actually need for TB programme evaluation or cost-effectiveness analysis is that convenient. Hierarchical

structures, correlated random effects, non-standard likelihoods, and covariate-adjusted network meta-analyses (as in Chapter 12) all produce posteriors with no analytic solution. Markov Chain Monte Carlo (MCMC) is the computational workaround: instead of solving for the posterior algebraically, you draw a long sequence of correlated samples whose distribution converges to the posterior you want, and then you summarise those samples the same way you would summarise any other distribution of draws — means, quantiles, credible intervals.

The core idea behind every MCMC algorithm is the same: build a Markov chain — a sequence of parameter values where the next value depends only on the current one — whose long-run behaviour matches the posterior you want. Then just run it for a long time and summarise the values it visits.

The oldest practical version, **Metropolis-Hastings**, proposes a candidate next value by perturbing the current one, then accepts or rejects it with a carefully chosen probability. Over many iterations, the chain visits each region of parameter space in proportion to its posterior density. It is simple and general, but can be slow: small random proposals take a long time to cross large, high-dimensional spaces.

Hamiltonian Monte Carlo (HMC) speeds this up with an idea borrowed from physics. It treats the negative log-posterior as a hilly energy surface and simulates a particle rolling across it with momentum, using the surface's **gradient** to steer proposals toward high-density regions instead of wandering randomly. Big, informed steps even in high dimensions — that is why HMC scales so much better to the complex hierarchical models common in HEOR.

NUTS, the No-U-Turn Sampler, is the practical refinement of HMC that Stan actually runs. It automatically decides how far the simulated particle should travel before it starts turning back on itself, so you never have to hand-tune a step count. You do not need to implement any of this maths yourself — Stan's compiler and sampler handle it — but the intuition explains why Stan models typically reach the same effective sample size in fewer iterations than JAGS models. §14.4 picks that thread back up.

JAGS (Just Another Gibbs Sampler) — the engine behind `gemtc` in Chapter 12 — instead typically uses **Gibbs sampling**: a special case of Metropolis-Hastings that updates one parameter at a time, drawing each directly from its full conditional distribution given every other parameter's current value. This fits naturally with conjugate or near-conjugate hierarchical structures, which is part of why JAGS remains common in network meta-analysis work. It can converge more slowly than HMC/NUTS, though, when parameters are highly correlated.

It helps to see all three engines fit the same simple model: a Beta-Binomial treatment-success analysis, extended slightly to a logistic-regression-style setting so the code generalises to more realistic use. Start with raw Stan via `rstan`, which requires writing the model explicitly in Stan's own modelling language and compiling it:

```
library(rstan)

stan_code <- "
data {
  int<lower=0> n;      // number of patients
  int<lower=0> y;      // number of successes
}
parameters {
  real<lower=0, upper=1> theta;   // treatment success probability
```

```
}
model {
  theta ~ beta(2, 2);    // weakly informative prior
  y ~ binomial(n, theta);
}
"

stan_data <- list(n = 40, y = 34)

fit_stan <- stan(
  model_code = stan_code,
  data       = stan_data,
  chains     = 4,
  iter       = 2000,
  warmup     = 1000,
  seed       = 123
)

print(fit_stan, probs = c(0.025, 0.5, 0.975))
```

The same model in JAGS, using `rjags`, requires its own BUGS-style model syntax and a slightly more manual sampling loop built from `jags.model()` followed by `coda.samples()`:

```
library(rjags)
library(coda)

jags_code <- "
model {
  theta ~ dbeta(2, 2)
  y ~ dbin(theta, n)
}
"
writeLines(jags_code, con = "beta_binom.jags")

jags_data <- list(n = 40, y = 34)

jags_mod <- jags.model(
  file     = "beta_binom.jags",
  data     = jags_data,
  n.chains = 4,
  n.adapt  = 1000
)

update(jags_mod, n.iter = 1000)   # burn-in

fit_jags <- coda.samples(
  jags_mod,
  variable.names = "theta",
  n.iter         = 2000
)

summary(fit_jags)
```

Finally, `brms` sits on top of Stan and lets you specify most standard regression-style models using familiar R formula syntax, generating and compiling the underlying Stan code automatically:

```r
library(brms)

tb_df <- data.frame(
  success = 34,
  total   = 40
)

fit_brms <- brm(
  success | trials(total) ~ 1,
  data   = tb_df,
  family = binomial(),
  prior  = set_prior("normal(0, 2.5)", class = "Intercept"),
  chains = 4,
  iter   = 2000,
  warmup = 1000,
  cores  = 4,
  seed   = 123
)

summary(fit_brms)
plot(fit_brms)
```

Note the prior is set on the intercept on the logit scale here rather than directly on θ, since `brms` builds a generalised linear model internally — this is a common source of confusion for analysts moving from hand-coded Beta-Binomial models to formula-based Bayesian regression, and worth testing on a known conjugate example (as above) before trusting it on a model with no closed-form answer.

Each tool has a natural niche. `rstan` gives you full control and is the natural choice when you need a custom likelihood that neither JAGS's BUGS dialect nor `brms`'s formula interface can express. `rjags` remains valuable for Gibbs-friendly hierarchical structures and for continuity with network meta-analysis workflows already built on JAGS, as in Chapter 12′s `gemtc` models. `brms` is usually the fastest path for standard regression-style problems — mixed-effects survival models, multilevel utility regressions, dose-response curves — because it handles the Stan code generation, prior specification helpers, and post-processing for you, while still letting you drop down to the underlying `stanfit` object (via `fit_brms$fit`) when you need finer control. For most day-to-day HEOR regression work, start with `brms`; reach for raw `rstan` or `rjags` when the model structure genuinely demands it.

> **Common pitfall.** Running an MCMC model and immediately trusting `summary()` output without checking convergence is the single most common Bayesian analysis error. The `print()` and `summary()` calls above already return `Rhat` and effective sample size alongside the posterior estimates — do not report a posterior mean or credible interval until those diagnostics have been checked, which §14.4 covers in full, building on the $\hat{R} < 1.05$ convergence rule you already applied to `gemtc` and `multinma` network meta-analysis models in Chapter 12.

14.4 Diagnostics: convergence, effective sample size, and posterior predictive checks

Running an MCMC sampler and getting numbers back is the easy part. Trusting those numbers is the job. In Chapter 12 you fit Bayesian network meta-analyses with `gemtc`, `multinma`, and `bnma`, and each time we flagged a `coda::gelman.diag()` check and moved on, promising we'd come back to what it actually means. This is that promise kept. Diagnostics are not a bureaucratic box to tick before an HTA submission reviewer asks for them — they are the difference between a posterior distribution that describes your evidence and one that describes an algorithm that got stuck.

Why convergence is not optional

An MCMC chain is only a valid sample from the posterior once it has "forgotten" its starting value and is exploring the full shape of the distribution the way it would if you had started it somewhere else entirely. Before that point, the chain is still wandering through unrepresentative territory — the burn-in or warm-up period. The trouble is that a chain can *look* perfectly healthy on a trace plot in the short run while still being far from target, especially in high-dimensional models like network meta-analyses with many treatment contrasts and heterogeneity parameters.

The standard first line of defence is to run several chains (`coda`, `rstan`, and `rjags`/`gemtc` all default to this) from dispersed starting values and ask whether they have converged to the *same* distribution. If chain 1 is sampling a different region of parameter space than chain 2, you don't have one posterior, you have evidence that your sampler hasn't settled.

```
library(coda)

# mcmc.list object from gemtc, rjags, or similar
gelman.diag(mcmc_samples, multivariate = FALSE)
```

This returns the potential scale reduction factor (PSRF), commonly called the R-hat or Gelman-Rubin statistic, for every monitored parameter. Conceptually, R-hat compares the variance *between* chains to the variance *within* chains. If the chains have converged, these two variances should agree and R-hat should sit at 1. In practice:

- R-hat < 1.05 is the traditional "acceptable" threshold used in most applied Bayesian HTA work, including the `gemtc`/NICE DSU NMA tradition you used in Chapter 12.
- R-hat < 1.01 is the more conservative threshold recommended by newer Stan-ecosystem guidance and increasingly expected by methodologically careful reviewers, particularly for models going into a regulatory or reimbursement submission.

> **Common pitfall.** A single R-hat below threshold tells you nothing if you only checked one parameter. Always check *all* monitored parameters, not just the headline treatment effect — heterogeneity variance terms and random-effects parameters in NMA models are notoriously the last to converge, and they drive the width of your decision uncertainty downstream.

Trace plots remain the most intuitive diagnostic precisely because R-hat is a *summary* and summaries hide failure modes. A chain can have an acceptable R-hat and still show a slow drift, a sticky patch, or a second chain exploring a completely different mode of a multimodal posterior — all of which are visually obvious on a trace plot and easy to miss in a single number.

```r
library(bayesplot)
library(rstan)

fit <- stan(file = "nma_model.stan", data = stan_data,
            chains = 4, iter = 4000, warmup = 2000)

posterior_array <- as.array(fit)
mcmc_trace(posterior_array, pars = c("d[1]", "d[2]", "tau"))
```

A healthy trace plot looks like a "fat hairy caterpillar" — four overlapping bands with no trend, no chain visibly separated from the others, and no long flat stretches where the sampler got stuck.

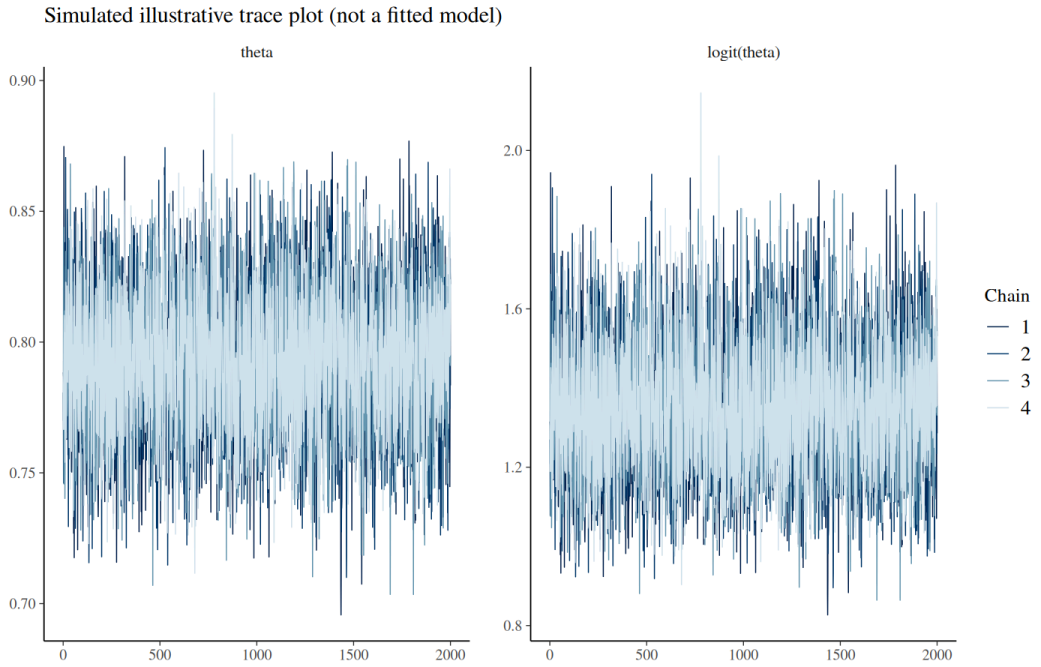


Figure 22: Illustrative `bayesplot::mcmc_trace()` diagnostic on **simulated** MCMC draws (4 chains x 2,000 post-warmup iterations, hand-constructed with mild autocorrelation around the chapter's own Beta-Binomial posterior mean of 0.793), shown on both the natural (θ) and logit scales. This is **not** output from an actually-fitted Stan/brms model — the book's environment does not have rstan/brms installed/fittable — it is provided purely to demonstrate what a healthy, well-mixed "fat hairy caterpillar" trace plot looks like in practice.

Effective sample size: why 20,000 draws can be worth 200

Here is the point that trips up most analysts moving from frequentist bootstrapping to MCMC: *the number of post-warm-up iterations you ran is not the number of independent pieces of information you have.* MCMC draws are autocorrelated — each draw depends on the one before it — so a chain of 10,000 iterations might carry the statistical information of only a few hundred truly independent draws if the sampler mixes slowly.

The effective sample size (ESS) corrects for this autocorrelation and tells you, in effect, "how many independent draws would give you the same precision as the correlated chain you actually have."

```r
effectiveSize(mcmc_samples)
```

Inside the Stan ecosystem, `rstan::monitor()` reports `n_eff` alongside R-hat for exactly this reason — the two diagnostics are reported together because they answer different questions. R-hat asks

"have my chains converged to the same place?" ESS asks "once converged, how much *information* do I actually have about this parameter?" You can have excellent R-hat with dismal ESS if chains converge quickly to the right region but then mix very slowly within it — a common pattern for variance components in hierarchical NMA models.

As a rule of thumb, treat ESS below roughly 400 (across all chains combined) for any parameter you plan to report as a trigger to run the model longer or reparameterise it, regardless of how good your R-hat looks. For Stan models specifically, also check sampler-level diagnostics beyond R-hat and ESS — divergent transitions, which indicate the sampler is struggling with the geometry of the posterior (common with funnel-shaped hierarchical priors):

> **Difficult terminology: funnel-shaped hierarchical priors (Neal's funnel).** In a hierarchical model, individual-level effects (e.g. each study's own effect) are usually written as "the overall average, plus a study-specific deviation whose size is controlled by one variance parameter." The trouble is that this variance parameter and the individual deviations are mathematically tangled together: when the variance parameter is estimated to be small, the deviations are forced to be tiny too, and when it's large, the deviations can range widely. Plotted together, the geometry of that joint space looks like a funnel — wide and open at one end (large variance, loosely-constrained deviations), narrowing to a tight neck at the other (small variance, tightly-constrained deviations). Samplers like Stan's HMC/NUTS struggle badly inside that narrow neck, because the step size that works well in the wide part of the funnel is too large to navigate the narrow part without "divergent transitions" (the sampler effectively crashing into the funnel's walls). It's a well-known pathology named after statistician Radford Neal, and the standard fix is a **non-centered parameterisation** — reformulating the model so the individual deviations are expressed as independent, unit-scaled quantities multiplied by the variance parameter afterwards, rather than directly tangled with it, which straightens the funnel out into geometry the sampler can explore efficiently.

```
check_hmc_diagnostics(fit)
```

A model with zero divergent transitions, R-hat below 1.01, and adequate ESS on every parameter you intend to use downstream is one you can defend to a reviewer. A model with any one of those failing is not ready for a decision analysis, no matter how plausible the point estimates look.

Posterior predictive checks

Convergence diagnostics tell you whether your *sampler* worked. They say nothing about whether your *model* is any good. For that you need posterior predictive checks (PPCs): simulate new data from the fitted model's posterior and ask whether that simulated data looks like the data you actually observed. If your model is a reasonable description of the data-generating process, replicated datasets drawn from the posterior predictive distribution should be hard to distinguish from the real thing.

```
library(bayesplot)

y_rep <- posterior_predict(fit)  # or extract manually from Stan output

ppc_dens_overlay(y = observed_outcomes, yrep = y_rep[1:50, ])
```

This overlays the density of your observed outcome on top of 50 densities simulated from posterior draws. A model that fits well produces a tight bundle of replicated curves hugging the observed

curve. A model that systematically misses — too much dispersion, wrong skew, a mode in the wrong place — shows up as replicated curves that visibly diverge from the observed data, which is a much more concrete signal than a residual plot for an analyst still building intuition for what "model misfit" looks like in a Bayesian setting.

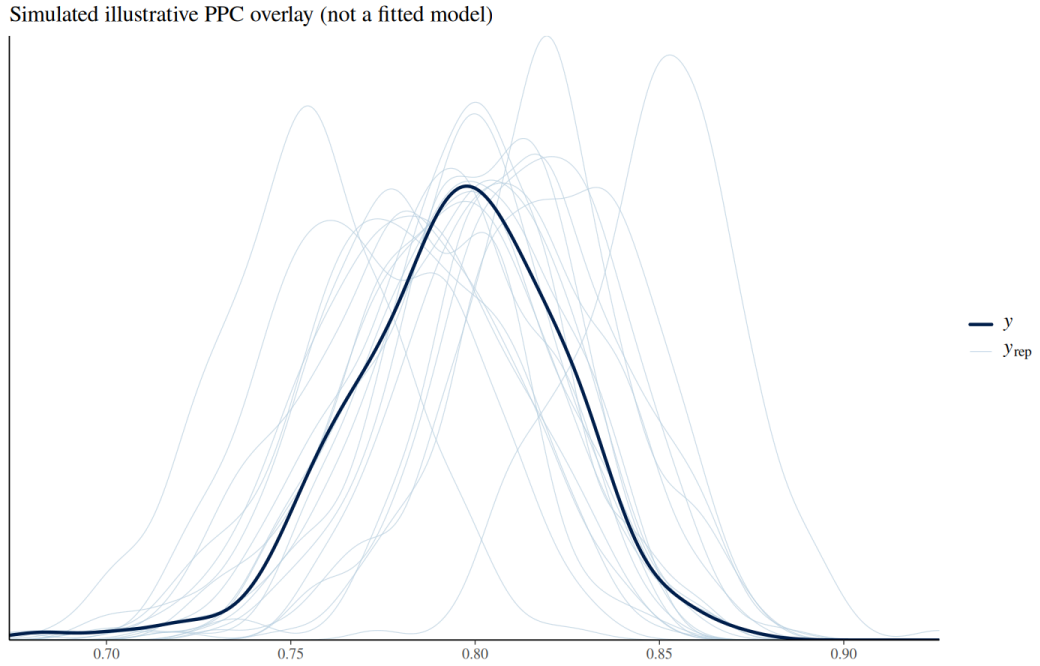


Figure 23: Illustrative `bayesplot::ppc_dens_overlay()` diagnostic comparing a **simulated** observed-outcome density to 20 **simulated** posterior-predictive replicate densities, both drawn from Beta distributions perturbed around the chapter's Beta-Binomial posterior parameters ($\alpha_{\text{post}} = 184$, $\beta_{\text{post}} = 48$). This is **not** output from an actually-fitted Stan/brms model — the book's environment does not have rstan/brms installed/fittable — it is provided purely to demonstrate what a tight, well-fitting bundle of replicated curves looks like against the observed data.

HEOR in practice. For a Bayesian NMA feeding a cost-effectiveness model, run PPCs on each trial's observed event counts against the model's posterior predictive event counts before trusting the pooled treatment effects. A network that fits the overall deviance well can still badly misfit one influential trial — exactly the trial that might be driving your incremental cost-effectiveness ratio.

Agency perspective. NICE DSU Technical Support Document 2 (Dias et al., **Evidence Synthesis for Decision Making 2: A Generalised Linear Modelling Framework for Pairwise and Network Meta-Analysis of Randomised Controlled Trials**), already named in Chapter 12's reporting section, explicitly recommends reporting the deviance information criterion alongside convergence diagnostics for network meta-analysis models submitted to NICE, and treats inadequate convergence checking as a methodological limitation reviewers are instructed to flag.

14.5 Bayesian evidence synthesis as a pipeline

Chapter 12 walked through fitting Bayesian network meta-analyses in practice — `gemtc` wrapping JAGS, `multinma` wrapping Stan, and `bnma` as a lighter-weight alternative — and showed you how to extract pooled treatment effects and relative rankings. This section does not repeat that. Instead, it steps back to the principle that makes Bayesian evidence synthesis worth the extra modelling effort in the first place, and sets up why §14.6 can treat the output of Chapter 12's models as directly usable decision-analytic input.

The general principle: synthesis produces a distribution, not a number

Classical (frequentist) meta-analysis gives you a pooled effect estimate and a confidence interval. That interval describes sampling variability under repeated hypothetical experiments — a long-run frequency statement, not a probability statement about the parameter itself. A Bayesian meta-analysis or NMA, by contrast, gives you a full posterior distribution over every pooled effect, every between-study heterogeneity parameter, and every treatment ranking probability. That distribution *is* a direct probability statement: "given this evidence and this model, here is how plausible each value of the treatment effect is."

This distinction sounds academic until you reach the point where you need to propagate evidence uncertainty into a decision model. A confidence interval does not tell you how to sample from it in a way that respects the correlation structure between parameters — you have to impose an additional distributional assumption (normal, usually, on the log-hazard-ratio scale) and often ignore correlations between related parameters altogether. A Bayesian posterior needs none of that. It already *is* a joint sample over every parameter, correlations included, ready to be drawn from directly.

Why this matters more as networks get more complex

The benefit of full posterior propagation scales with the complexity of the evidence base. A two-arm pairwise meta-analysis has one pooled effect and its uncertainty is easy to approximate reasonably well with a normal distribution. A network with several treatments, as in Chapter 12's worked example, has many pairwise contrasts, each correlated with the others through the network structure and the consistency assumptions baked into the model. There is no sensible way to hand-construct a joint distributional assumption that preserves those correlations for a downstream Monte Carlo simulation. The posterior sample from `gemtc`, `multinma`, or `bnma`, however, already carries those correlations intact, because every MCMC draw is a complete, internally consistent set of all the contrasts simultaneously.

This is the real argument for Bayesian evidence synthesis in an HEOR context. It isn't that Bayesian point estimates differ meaningfully from frequentist ones under vague priors (often they barely do). It's that the *output object* — the posterior sample — is already in the exact shape a decision model needs, with no further approximation step required.

```
# Recall from Chapter 12: a gemtc NMA fit
library(gemtc)

# mcmc.list of relative-effect draws, one column per contrast
relative_effects <- relative.effect(nma_fit, t1 = "placebo")
re_samples <- as.matrix(relative_effects$samples)

# Each row is one joint posterior draw across every contrast --
```

```
# correlations between contrasts are preserved automatically
head(re_samples)
```

This `re_samples` matrix is the hinge on which the rest of this chapter turns: it is simultaneously the end product of Chapter 12's evidence synthesis and the raw material for the decision analysis in §14.6.

Where this fits in the overall workflow

Think of Bayesian evidence synthesis as the first stage of a two-stage pipeline, not a standalone analysis:

1. *Synthesis stage* (Chapter 12's territory): combine all available trials — pairwise or networked — into posterior distributions for relative treatment effects, using `gemtc`, `multinma`, or `bnma` as appropriate to the network structure and priors required.
2. *Decision stage* (this chapter's territory, §14.6): feed those posterior draws, unmodified, into a cost-effectiveness model as the probabilistic sensitivity analysis sample for treatment effect.

> **HEOR in practice.** When writing up methods for a submission, be explicit that the PSA distribution for relative treatment effect *is* the NMA posterior sample, not a distribution fitted *to* the posterior sample's mean and standard deviation. Re-fitting a normal distribution to posterior summary statistics throws away exactly the correlation structure that made the Bayesian synthesis worth doing, and reviewers familiar with the NICE DSU documents will ask why you did it.

> **Common pitfall.** Thinning or subsampling the posterior sample before passing it to the decision model (e.g. taking every 10th draw to get a "round" PSA sample size) is usually unnecessary and can quietly reduce the effective precision of your PSA results. Unless autocorrelation diagnostics (§14.4) indicate the chain needs thinning for storage reasons, pass the full post-warm-up sample through.

The next section makes this pipeline concrete: taking posterior draws — whether from an NMA, a simpler pairwise meta-analysis, or any other Bayesian evidence model — and using them as the engine of a full probabilistic cost-effectiveness analysis.

14.6 Bayesian decision analysis: from posterior to PSA

Everything in this chapter has been building toward one practical payoff, and it is worth stating plainly: *a Bayesian model's posterior draws are a probabilistic sensitivity analysis sample.* There is no separate step where you fit a distribution to a parameter and then sample from it for PSA — the posterior draws produced by your MCMC sampler in §14.1-14.3, or by the NMA models of Chapter 12, already are that sample. This section shows how to use them that way.

Expected value under a posterior

A health economic decision model computes, for each strategy s, some net benefit function — most commonly net monetary benefit at a willingness-to-pay threshold λ:

$$\text{NMB}_s = \lambda \times \text{QALY}_s - \text{Cost}_s$$

In a deterministic model you plug in point estimates for every input parameter and get one number per strategy. In a Bayesian decision-analytic framing, every input parameter — the treatment effect, the utility decrement, the cost, the transition probability — has a posterior distribution, and so does NMB_s. The *expected* net monetary benefit is the posterior mean of that distribution:

$$\mathrm{E}[\mathrm{NMB}_s] = \int \mathrm{NMB}_s(\theta) p(\theta \mid D)\, \mathrm{d}\theta$$

where θ is the full vector of model parameters and $p(\theta \mid D)$ is their joint posterior given data D. You never evaluate this integral analytically. You approximate it by Monte Carlo: run the cost-effectiveness model once per posterior draw, and average the resulting net benefits across draws. This is exactly standard PSA mechanics — the only thing that has changed is *where the samples came from.*

Posterior draws as PSA input: the direct pipeline

In a conventional (non-Bayesian) PSA workflow, you fit each input parameter a distribution — a beta for a probability, a gamma for a cost, a normal for a log-hazard-ratio — usually justified by matching a point estimate and standard error from a single study or a frequentist meta-analysis, then draw independently from each one and hope the implied joint distribution across parameters is reasonable. Correlations between parameters, when they exist, have to be imposed by hand (e.g. via a multivariate normal on the log scale, or a Cholesky decomposition), which is awkward and frequently skipped.

> **Difficult terminology: Cholesky decomposition.** Suppose you want to simulate several correlated quantities at once (e.g. cost and utility that tend to move together) by starting from easy-to-generate independent random numbers. A Cholesky decomposition breaks a correlation (or covariance) matrix down into a single "square root" matrix that, when multiplied by a vector of independent random draws, produces a new vector with exactly the desired correlation structure baked in. It's the standard recipe for "injecting" a specified correlation pattern into otherwise-independent simulated numbers. A kitchen analogy: if independent random numbers are plain flour, water, and eggs, the Cholesky matrix is the recipe/ratios that combine them into a batter with the specific consistency (correlation) you wanted, rather than three separate unrelated ingredients. The point raised here is that doing this by hand, for every PSA parameter set, is exactly the fiddly step a genuine Bayesian joint posterior sample (the rest of this section) sidesteps entirely, because the posterior draws already come out correlated correctly without needing any decomposition trick.

A Bayesian evidence base sidesteps all of this. If your relative treatment effect comes from the NMA posterior sample built in §14.5, each row of that sample already is one coherent PSA iteration for the treatment-effect parameters, correlations and all. You use the rows directly as your PSA loop, rather than drawing from a distribution that was fitted *to* that sample.

```
library(dplyr)

# re_samples: posterior draws of relative treatment effect from gemtc/NMA (14.5)
# n_psa draws, one row per PSA iteration, columns are contrasts

n_psa <- nrow(re_samples)
```

```r
# Other model inputs can also come from posteriors -- e.g. a Bayesian
# survival model's posterior draws of a hazard ratio, or simple
# independent draws if those parameters weren't modelled jointly
cost_psa    <- rgamma(n_psa, shape = 20, rate = 20 / 3500)
utility_psa <- rbeta(n_psa, shape1 = 40, shape2 = 10)

psa_results <- vector("list", n_psa)

for (i in seq_len(n_psa)) {
  log_hr <- re_samples[i, "d.placebo.treatment"]

  qalys_treatment <- simulate_qalys(hr = exp(log_hr), utility = utility_psa[i])
  cost_treatment  <- simulate_costs(hr = exp(log_hr), unit_cost = cost_psa[i])

  psa_results[[i]] <- data.frame(
    iteration = i,
    nmb_treatment = 30000 * qalys_treatment - cost_treatment,
    nmb_control   = 30000 * simulate_qalys(hr = 1, utility = utility_psa[i]) -
                    simulate_costs(hr = 1, unit_cost = cost_psa[i])
  )
}

psa_df <- bind_rows(psa_results)

# Expected net monetary benefit per strategy, and probability treatment
# is cost-effective at this threshold
psa_df |>
  summarise(
    E_nmb_treatment = mean(nmb_treatment),
    E_nmb_control   = mean(nmb_control),
    prob_ce         = mean(nmb_treatment > nmb_control)
  )
```

Notice what did *not* happen here: there was no step that said "fit a normal distribution to the log hazard ratio's posterior mean and SD, then draw from that." The posterior draws went straight into the simulation loop. Every correlation between contrasts that existed in the original NMA posterior — for instance, if two active treatments were both compared against the same reference and so their relative effects are not independent — survives into the PSA, because row i of `re_samples` is used as a complete, internally consistent unit.

> **HEOR in practice.** When your evidence base mixes Bayesian and non-Bayesian sources — a Bayesian NMA posterior for relative effect alongside a classically-fitted parametric survival curve for baseline risk — align the number of PSA iterations to the smaller or more constrained posterior sample, or resample consistently, and document which inputs came from genuine joint posteriors versus independently assumed distributions. A reviewer should be able to tell at a glance which parts of your PSA have real correlation structure behind them.

Decision uncertainty is now a direct statement about the evidence

Reporting a cost-effectiveness acceptability curve (CEAC) built from this PSA has a cleaner Bayesian interpretation than its frequentist counterpart. The proportion of PSA iterations where treatment has higher net benefit than control, at a given λ, is a direct posterior probability that

treatment is cost-effective at that threshold, given the data and the model — not an approximation justified by asymptotic sampling theory. This is a more honest match between the question decision-makers actually ask ("how likely is it that this treatment is worth it?") and the quantity the analysis reports.

Common pitfall. If you draw θ from a posterior but then still report the deterministic ("base case") analysis using the posterior *mean* of θ plugged into the model once, you have built a Bayesian evidence base but reverted to a non-probabilistic decision analysis. The expected value of a nonlinear function of θ is not generally equal to the function evaluated at $\mathrm{E}[\theta]$ — run the full posterior sample through the model, not just its central tendency.

Agency perspective. The ISPOR-SMDM Modeling Good Research Practices Task Force report on model transparency and validation (Eddy et al. 2012, **Value in Health**, "Model Transparency and Validation") recommends that probabilistic sensitivity analysis fully propagate parameter uncertainty including correlation structure where it exists — a requirement that a direct Bayesian-posterior-as-PSA-sample pipeline satisfies by construction, where an independently-sampled, distribution-fitted PSA typically does not.

14.7 Expert elicitation methods (SHELF)

Most of this chapter has assumed you have *data* — trial arms, observational cohorts, registry follow-up — from which a likelihood can be built and combined with a prior. But a large share of real HEOR work, especially around rare diseases, orphan drugs, and early-stage HTA submissions for genuinely novel technologies, runs into parameters for which no usable data exist at all. What is the five-year mortality benefit of a gene therapy with eighteen months of trial follow-up? What proportion of patients will adhere to a new delivery device nobody has used outside a lab? Committees still need a number — or, in the Bayesian framing of this chapter, a *distribution* — for these parameters, and when data cannot supply it, the next-best evidence is structured expert judgement.

The word "structured" matters enormously here. Unstructured expert judgement — asking a clinician "what do you think the rate is?" over the phone — is exactly the kind of informal, poorly-calibrated input that got Bayesian methods a bad reputation for being "subjective" in a pejorative sense. The response from the methodology community, and specifically from the University of Sheffield group, was to formalise the process into a reproducible protocol: the **Sheffield Elicitation Framework**, universally known by its acronym, **SHELF**. SHELF is not a piece of software first and foremost — it is a facilitation method, a set of templates, and a philosophy for turning "what an expert believes" into a probability distribution that can be defended, audited, and fed into a model. The R package of the same name (note the package name is capitalised, `SHELF`) is the computational companion that fits distributions to the quantities the facilitation process produces.

HEOR in practice. Elicitation is appropriate for parameters where data genuinely do not exist or are too immature to trust — not as a shortcut past the inconvenience of designing a study. A NICE committee or a payer reviewer confronted with an elicited prior will scrutinise *how* it was elicited at least as closely as the number itself. Treat the process documentation

(who, how many experts, what questions, what reconciliation) as a deliverable in its own right, not an afterthought.

Roles in the process. A SHELF exercise has two distinct kinds of participant, and conflating them is a common and damaging mistake.

- *The facilitator* is a trained elicitation specialist (ideally with statistical literacy but, crucially, with no personal stake in the parameter's value) who runs the session, asks questions in a format designed to counteract known cognitive biases (overconfidence, anchoring, availability), and never offers their own opinion of the "right" answer.
- *The experts* are the substantive content specialists — clinicians, epidemiologists, sometimes patients or carers for quantities like health state preferences — who hold the relevant domain knowledge but typically have no particular training in thinking probabilistically. Part of the facilitator's job is translating a clinical intuition ("most patients do reasonably well, a few have terrible outcomes") into quantiles a distribution can be fitted to.

The general process. A typical SHELF exercise, whether for a single parameter or a small set, follows roughly this arc:

1. *Preparation.* The facilitator defines the quantity of interest precisely (a frequent failure mode is experts answering slightly different questions from each other because the parameter was never pinned down — "response rate" at what time point, in what population, by what definition of response).
2. *Individual elicitation.* Each expert, independently and without discussion with the others, is asked a sequence of questions designed to pin down their personal distribution for the parameter — typically a median, then a lower and upper quartile or tertile, sometimes plausible extreme bounds. Individual elicitation first is deliberate: it prevents the loudest or most senior voice in the room from anchoring everyone else before independent judgements are recorded.
3. *Group discussion.* The individual judgements are shared, discussed, and the *reasons* behind disagreement are explored — not to force consensus, but so that each expert can rationally update their own view in light of arguments they hadn't considered.
4. *Reconciliation.* The group works toward a single "rational impartial observer" distribution representing the pooled judgement — SHELF provides specific templates for this step, rather than leaving it to an unstructured vote.
5. *Distribution fitting.* The elicited quantiles (individual or reconciled) are fitted to a parametric distribution — commonly a beta for a probability, a gamma or log-normal for a rate or cost, a normal for a roughly symmetric continuous quantity.

R tooling. The `SHELF` package on CRAN implements the fitting and feedback steps computationally. Given elicited judgements (for example, a median and two quartiles from one expert), the typical workflow is to fit a family of candidate distributions and visually check which gives a good fit to the elicited quantiles:

```
library(SHELF)

# Example: an expert has given judgements for a response-rate
# parameter as quantiles: 25th percentile = 0.30, median = 0.45,
# 75th percentile = 0.60
my_probs <- c(0.25, 0.5, 0.75)
my_vals  <- c(0.30, 0.45, 0.60)
```

```
fit <- SHELF::fitdist(vals = my_vals, probs = my_probs,
                      lower = 0, upper = 1)

# Inspect which parametric family fits best
fit$ssq

# Visualise the fitted distribution(s) against the elicited points
SHELF::plotfit(fit, lp = TRUE)
```

The package also provides feedback-style diagnostics — comparing implied probabilities from the fitted distribution back against further quantiles the expert supplies, as a sanity check that the fit captures their actual judgement — via a function along the lines of `SHELF::feedback()`. Not every argument name and default in the current `feedback()` signature has been independently re-checked against the live CRAN documentation here, so treat the call above as illustrative of the *idea* rather than a copy-paste guarantee; always run `?SHELF::fitdist` and `?SHELF::feedback` in your own R session before relying on the precise arguments, since package APIs do shift across versions. The package also ships a Shiny app for live group elicitation sessions — genuinely useful if you expect to run this more than once.

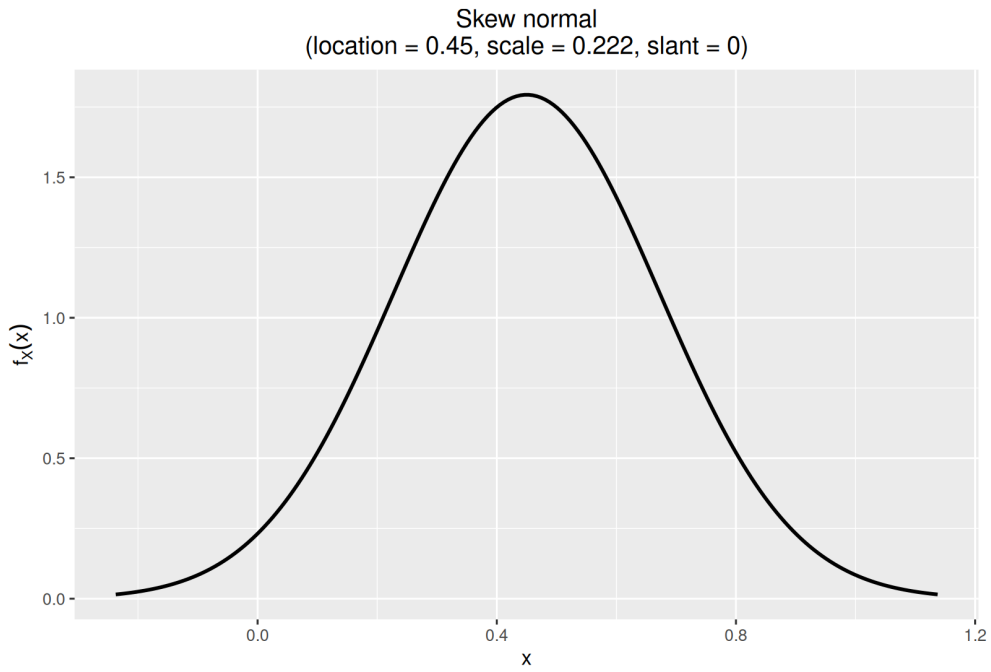


Figure 24: `SHELF::plotfit()` overlay of fitted candidate distributions against the elicited response-rate quantiles (25th/50th/75th percentiles = 0.30/0.45/0.60). By `fit$ssq` least-squares scoring, SHELF's default ranking places the unbounded skew-normal and normal families as the closest numerical fit to these three points (ssq on the order of 10^{-32} – essentially exact, because a symmetric three-point quantile set is fit almost perfectly by a two/three-parameter symmetric family). The beta fit scores a higher but still small ssq (1.99×10^{-5}), with gamma and log-normal trailing further behind. For a HEOR prior on a probability, this is the practical lesson: the **lowest-ssq** family is not automatically the **right** one to carry into the model – a response rate is bounded on $[0, 1]$, and the normal/skew-normal fits can assign positive density outside that range, which is nonsensical for a probability parameter. The beta fit, despite its slightly larger ssq, is the one that should feed the prior, precisely because it respects the support of the parameter being elicited.

Common pitfall. A beta or gamma fit to elicited quantiles is a *summary* of judgement, not a measurement. Resist the temptation to report the fitted distribution's standard error with the same confident precision you would give a trial estimate's standard error — it reflects how confident the expert group felt, filtered through a parametric approximation, not a sampling distribution of anything. Always report, alongside the fitted parameters, who was elicited, how many experts, and whether reconciliation changed the picture materially.

Agency perspective. NICE's Decision Support Unit Technical Support Document 33, "Expert Elicitation: Issues and Challenges," is the standard methodological reference UK HTA submissions point to when using elicited priors, and SHELF is the protocol it most closely aligns with in practice; if an elicitation exercise will inform a submission, read that document (or the equivalent guidance your jurisdiction's HTA body references) before designing the session, not after.

14.8 Bayesian health economic modelling workflows

This closing section is deliberately a roadmap rather than a new technique. Up to this point the chapter has built the pieces in isolation — Bayes' theorem and conjugate updating, MCMC sampling with Stan or JAGS, convergence diagnostics, and posterior-to-PSA translation — and §14.7 just added expert elicitation as a way to construct priors when data run out. None of these pieces does anything useful for a decision-maker on its own. The payoff is in chaining them into a single, reproducible, end-to-end workflow that starts with "what do we believe before seeing data" and ends with "here is the probability this technology is cost-effective, and here is how much it would be worth resolving our remaining uncertainty." Walking through that chain end to end is the point of this section.

Step 1 — Prior elicitation and specification. Every Bayesian model starts with a prior, whether it is a deliberately vague, weakly informative default or a carefully elicited distribution from §14.7. The discipline worth adopting here is documentation: record not just the prior's parametric form but *why* that form and those parameter values were chosen — vague because nothing better was available, informative because a SHELF exercise was run, or weakly informative as a regularising choice to stabilise an otherwise unidentified model. A reviewer (or your future self) will ask.

Step 2 — Bayesian evidence synthesis. Where there are multiple studies bearing on a parameter — several trials estimating a treatment effect, several registries estimating a transition probability — Chapter 12's Bayesian network meta-analysis, or a simpler Bayesian meta-analysis of the kind introduced earlier in this chapter, combines them into a single posterior while properly propagating between-study heterogeneity. This step is where most of the genuine *evidence* enters the model; the prior from Step 1 and the likelihood from the synthesised data combine here, typically specified as a hierarchical model in Stan or JAGS.

Step 3 — MCMC fitting. The joint posterior over every parameter in the synthesis model — treatment effects, baseline event rates, heterogeneity variances, and anything else in the model's parameter vector — is sampled numerically, since it almost never has a closed form once you move past the simplest conjugate cases covered early in the chapter. This is mechanically the same

`stan()` or `jags.model()` call pattern used throughout the chapter; what differs model to model is the structure wired into the Stan/JAGS code, not the fitting mechanics.

Step 4 — Diagnostics. Before a single posterior draw is trusted for anything downstream, run the checks from §14.4: $\hat{R}$ close to 1 for every parameter, adequate effective sample size, trace plots that look like noise rather than trends, and divergent-transition counts near zero if you are using Stan's Hamiltonian Monte Carlo sampler. This is not an optional formality — a decision model fed non-converged MCMC output will produce PSA results that look precise and are simply wrong, and nothing downstream will flag that for you.

```
# Illustrative: checking convergence before using draws downstream
library(rstan)
print(fit_synthesis, pars = c("mu", "tau"))   # check Rhat, n_eff
rstan::traceplot(fit_synthesis, pars = c("mu", "tau"))
```

Step 5 — Posterior draws as the PSA sample. This is the step that makes the whole chapter's effort worthwhile, and it is where §14.6's logic lands. In a conventional frequentist PSA, you fit point estimates and standard errors to each model parameter, then *separately* specify a distribution (beta, gamma, log-normal) for each one to sample from — an extra modelling step that can lose the correlation structure between parameters estimated jointly from the same data. In a Bayesian workflow, there is no separate step: each row of the MCMC output *already is* one coherent draw of the full parameter vector from the joint posterior, correlations intact. You simply take those draws — thousands of them, one per retained MCMC iteration — and pass them straight into the decision-analytic model (a Markov model, a partitioned survival model, whatever structure Part IV's chapters build) to compute costs and QALYs under each draw. No distributional assumptions are layered on top of the evidence synthesis; the posterior *is* the PSA input.

```
# Illustrative: extracting posterior draws as PSA inputs
library(rstan)
draws <- rstan::extract(fit_synthesis)
# draws$mu, draws$tau (etc.) are now vectors of length n_draws,
# ready to be looped through a cost-effectiveness model row by row
```

Step 6 — Reporting. Transparency matters at every step above, but especially here: a results table or report should make clear which parameters came from elicitation, which from synthesis, what the convergence diagnostics showed, and how many posterior draws fed the PSA. This is the kind of audit trail HTA bodies and peer reviewers increasingly expect, and the ISPOR-SMDM Modeling Good Research Practices task force reports are a useful reference point for what that reporting should cover structurally, alongside CHEERS 2022 for the economic evaluation reporting itself.

> **HEOR in practice.** Treat this six-step chain as a checklist you re-run for every Bayesian submission, not a one-off design exercise. The most common real-world failure is not a statistical error in any single step — it is a broken link between steps: synthesis draws that get summarised down to a mean and SD before being handed to the decision model, discarding exactly the joint uncertainty structure that made the Bayesian approach worth doing in the first place. Keep the full draws array moving through every step; only summarise at the very end, for reporting.

Looking forward, this workflow is the hinge between this chapter and Part IV. Chapter 25, on handling uncertainty and probabilistic sensitivity analysis, picks up exactly at Step 5 above

and develops the decision-model side — cost-effectiveness acceptability curves, cost-effectiveness planes, and the full machinery for summarising PSA output once you have it. Chapter 26, on value of information analysis, is where a fully Bayesian model pays an extra dividend that a frequentist PSA cannot offer as naturally: because the PSA sample already *is* a set of draws from a genuine joint posterior over every parameter, quantities like the Expected Value of Perfect Information (EVPI) and Expected Value of Partial Perfect Information (EVPPI) can be computed directly from those draws, without the additional approximating machinery sometimes needed to retrofit VOI onto a non-Bayesian PSA. If you take one thing from this chapter into the rest of the book, it should be this: a Bayesian evidence base is not just a different way of getting a p-value-free answer — it is the natural, low-friction substrate for everything Part IV does with uncertainty afterward.

Common pitfall. Running MCMC for the evidence synthesis and then discretising the posterior down to a handful of scenarios ("low, medium, high") for the decision model defeats the purpose of this entire workflow and will understate decision uncertainty. If you have thousands of posterior draws, use thousands of draws in the PSA loop — modern decision models in R run this comfortably; the computational cost of looping over draws is rarely the real constraint.

Chapter 15. Real-World Evidence (RWE) and Causal Inference

Chapter 14 worked with evidence you already trusted the structure of: trial data, pooled contrasts, a posterior you could carry straight into a decision model. This chapter asks a harder question. What if the evidence was never randomised — a TB programme register, an insurance claims database, a hospital's electronic health records? Can you still get a defensible causal answer?

Most of this chapter's methods are frequentist, by design and by convention. Chapter 14 already flagged RWE and causal inference as the one area where Bayesian methods are an occasional option, not the default. Propensity scores, doubly robust estimators, instrumental variables, difference-in-differences — these are overwhelmingly practiced and reported in a frequentist frame in the field itself, and this chapter follows that convention.

The chapter has four parts. §15.1-15.3 build the foundations: what makes data fit for purpose, which study design fits which question, and how a causal diagram makes your confounding assumptions explicit before you fit anything. §15.4-15.7 cover the core estimation toolkit for when you believe you have measured the relevant confounders: propensity scores, doubly robust estimators and g-methods, instrumental variables and regression discontinuity, and difference-in-differences/interrupted time series. §15.8-15.9 cover two problems that recur constantly in HTA submissions: adjusting for treatment switching, and quantifying how robust a finding is to confounders you could not measure. §15.10-15.11 close with infrastructure and design: the OHDSI/OMOP standard for multi-site data, and external/synthetic control arms for single-arm trials.

Learning objectives

- Assess whether a real-world data source is fit for purpose using the FDA's relevance/reliability framework, and document that assessment before analysis begins
- Choose between a cohort, case-control, or target trial emulation design for a given RWE question, and explain how target trial emulation guards against immortal time and prevalent-user bias
- Build a directed acyclic graph (DAG) in `dagitty`/`ggdag`, distinguish confounders from mediators and colliders, and extract a minimal sufficient adjustment set programmatically rather than adjusting for every available covariate
- Implement propensity score matching, weighting, and balance diagnostics in R (`MatchIt`, `WeightIt`, `cobalt`), and compute an inverse-probability-weighted average treatment effect by hand to verify package output
- Explain why doubly robust estimators (AIPW) and g-methods relax the single-correct-model assumption of plain propensity-score or outcome-regression approaches, and when time-varying confounding requires g-methods specifically
- Apply instrumental variables (`ivreg`) and regression discontinuity (`rdrobust`) to recover causal effects under unmeasured confounding, and recognise the diagnostic checks (weak-instrument F-test, exclusion restriction plausibility, manipulation testing) each design requires

- Implement difference-in-differences correctly under staggered treatment adoption (`did::att_gt()`) rather than naive two-way fixed effects, and fit an interrupted time series model with `segmented` or an explicit level/slope-change specification
- Describe when and why treatment-switching adjustment (RPSFTM, IPCW) is needed in trials with crossover, and name NICE DSU TSD 16 as the governing methods reference
- Compute and interpret E-values (`EValue` package) as a standardised sensitivity measure for unmeasured confounding, and present them alongside point estimates rather than in isolation
- Describe the role of the OHDSI/OMOP Common Data Model and its R tooling (`DatabaseConnector`, `FeatureExtraction`, `CohortMethod`) in multi-site observational studies, and judge when that infrastructure is worth its complexity
- Construct a propensity-matched external control arm for a single-arm trial, and explain why temporal/era comparability is a distinct threat from baseline covariate imbalance

15.1 Fit-for-purpose RWD and RWE frameworks

Every chapter so far in this book has worked from evidence that was, in some sense, handed to you: trial arms with randomised assignment, published hazard ratios, posterior distributions from a Bayesian network meta-analysis. From this chapter onward the ground shifts. Real-world data (RWD) — programme registers, insurance claims, electronic health records (EHRs), laboratory information systems, even the line-listings your national TB programme submits to WHO — were not collected to answer your causal question. They were collected to run a clinic, bill a payer, or satisfy a surveillance mandate. Turning that data into real-world evidence (RWE) fit for a regulatory or HTA decision requires you to be explicit about **why** the data exist, **whether** they can answer the question you are asking, and **what** bias they carry before a single regression line is fitted.

This is not a semantic nicety. The US FDA's 2023 guidance **"Real-World Data: Assessing Electronic Health Records and Medical Claims Data To Support Regulatory Decision-Making for Drug and Biological Products"** — part of the broader RWE framework the agency committed to under the 21st Century Cures Act — sets out a two-part test that has become the de facto global standard even outside the US: is the data **relevant** to the clinical question (right population, right exposure, right outcome, adequate follow-up), and is it **reliable** (accurate, complete, traceable, with transparent provenance)? The EMA's parallel work, coordinated through the Big Data Steering Group and operationalised in the **DARWIN EU** (Data Analysis and Real World Interrogation Network) initiative, asks the same two questions in different words: data quality and study design robustness. For a TB programme analyst, the practical translation is this — before you touch a single `dplyr` verb, you document where each variable in your dataset came from, how it was captured (clinician entry, lab interface, self-report), what fraction is missing and why, and whether the cohort you can assemble actually contains the exposure contrast (e.g., bedaquiline-containing vs injectable-containing regimen) and the outcome (culture conversion, treatment success, death) with enough precision to support inference.

A useful habit is to write a short **data provenance and fitness memo** before analysis begins — not after a reviewer asks for one. The memo should state the data source and update frequency, the target population and how your extract maps onto it, known structural gaps (e.g., private-

sector treatment episodes missing from a public programme register), and the intended use of the evidence (safety signal, comparative effectiveness, HTA submission). This is the RWE-era equivalent of a statistical analysis plan, and regulators increasingly expect to see it.

```
library(dplyr)
library(tibble)

# A minimal fitness-for-purpose checklist as a data frame -- the kind of
# artefact you would attach to an analysis plan or HTA dossier appendix
rwd_fitness_checklist <- tibble::tribble(
  ~domain,         ~question,
~assessment,
  "Relevance",     "Does the population match the decision question?",
"Partial: public-sector only",
  "Relevance",     "Is exposure (regimen) captured with dates?",           "Yes,
pharmacy dispense log",
  "Relevance",     "Is the outcome defined per WHO TB definitions?",       "Yes,
treatment outcome registry",
  "Reliability",   "Is there a documented data dictionary?",               "Yes,
v3.2",
  "Reliability",   "What proportion of key covariates are missing?",       "HIV
status 4%, culture 11%",
  "Reliability",   "Is there an audit trail / source verification?",
"Partial: no routine monitoring"
)

print(rwd_fitness_checklist, n = Inf)
```

The point of such a table is not bureaucratic box-ticking. A "Partial" answer in the Relevance domain tells you, before you run a single model, that any effect estimate will generalise only to the public sector, and a "Partial" in Reliability tells you that missingness in HIV status is probably not completely at random and will need explicit handling (multiple imputation, inverse-probability weighting, or at minimum a sensitivity analysis) rather than a silent `na.omit()`.

HEOR in practice. When an RWE analysis is destined for a reimbursement dossier, align the fitness-for-purpose assessment with the specific decision-grade evidence standard the payer applies — in India, HTAIn's methods guidance and NICE's real-world evidence framework in the UK both now ask explicitly whether the RWD source could, in principle, have supported a trial-emulating analysis. Framing your memo in those terms early avoids a late-stage "why wasn't this a target trial emulation" review comment.

Common pitfall. Treating "more data" as a proxy for "better evidence" is one of the most common RWE errors. A claims database with five million patients but no reliable treatment-start date is less fit for a comparative-effectiveness question than a programme register of five thousand patients with verified regimen dates. Volume does not substitute for relevance and reliability — assess both before describing a data source as "big data."

15.2 Study design: cohort, case-control, target trial emulation

Once a data source clears the fitness-for-purpose bar, the next decision is architectural: what study design will let the data answer the causal question without manufacturing an answer that merely reflects who received the exposure in the first place? Three designs recur constantly in TB and infectious-disease RWE work, and each has a distinct bias profile.

A **cohort design** follows an exposed and an unexposed (or differently exposed) group forward from a defined start point to an outcome. It is intuitive and maps directly onto the trial structure you already know from earlier chapters, but its validity depends entirely on correctly defining "time zero" — the point at which eligibility, exposure assignment, and follow-up all begin simultaneously. Get this wrong (for example, by including follow-up time before a drug was actually started, or by selecting on survival to the point of exposure) and you reintroduce **immortal time bias**, a problem familiar from pharmacoepidemiology that inflates apparent drug benefit.

A **case-control design** samples cases of the outcome (e.g., treatment failure) and a comparison set of controls from the same source population, then looks backward at exposure. It is efficient for rare outcomes and is still common in TB drug-resistance surveillance, but it is acutely sensitive to how controls are selected — control selection that does not mirror the case-generating mechanism produces selection bias that no amount of post-hoc adjustment repairs.

The design that has reshaped how regulators and HTA bodies think about RWE over the past decade is **target trial emulation (TTE)**, formalised by Hernán and Robins (**"Using Big Data to Emulate a Target Trial When Comparing Adaptive, Dynamic Treatment Regimes"**, *American Journal of Epidemiology*, 2016, and elaborated in their freely available text *Causal Inference: What If*). The core discipline of TTE is simple to state and demanding to execute: before touching the observational data, write a protocol for the **hypothetical pragmatic randomised trial** you would run if you could — eligibility criteria, treatment strategies being compared, assignment procedure, start of follow-up, outcome, causal contrast of interest, and analysis plan. Only then do you specify how each protocol element is emulated in the observational data. This explicit target-trial protocol is what prevents two subtle, pervasive errors: immortal time bias (misaligned time zero) and **prevalent-user bias** (comparing new starters to patients already established on treatment, who have by definition survived early attrition).

```
library(tibble)
library(dplyr)

# A target trial protocol, specified as a structured object before any
# emulation code touches the observational extract. Writing this first
# is the TTE discipline, not an optional add-on.
target_trial_protocol <- tribble(
  ~component,            ~specification,
  "Eligibility",         "Adults starting first-line TB treatment, no prior TB
Rx",
  "Treatment strategies","Bedaquiline-containing vs standard injectable-containing
regimen",
  "Assignment",          "Hypothetical randomisation at treatment initiation",
  "Time zero",           "Date of first regimen dispense (must equal eligibility
date)",
  "Outcome",             "WHO-defined treatment success at 6 months",
  "Causal contrast",     "Intention-to-treat and per-protocol effects",
  "Analysis",            "IPTW-adjusted Kaplan-Meier / Cox, with cloning-
```

```
censoring-weighting for per-protocol"
)

print(target_trial_protocol, n = Inf)

# Emulating "time zero" correctly: every patient's follow-up clock
# starts at regimen initiation, not at diagnosis or enrolment
cohort <- tibble(
  patient_id   = 1:6,
  diagnosis_dt = as.Date(c("2022-01-05","2022-01-10","2022-02-01",
                           "2022-02-15","2022-03-01","2022-03-10")),
  regimen_start= as.Date(c("2022-01-07","2022-01-20","2022-02-02",
                           "2022-02-28","2022-03-01","2022-03-25")),
  regimen      = c("Bedaquiline","Injectable","Bedaquiline",
                   "Injectable","Bedaquiline","Injectable")
)

cohort <- cohort %>%
  mutate(
    lead_in_days = as.numeric(regimen_start - diagnosis_dt),
    time_zero    = regimen_start   # NOT diagnosis_dt -- avoids immortal time bias
  )

cohort
```

The `lead_in_days` column is exactly the diagnostic you want in an audit: any patient with a long gap between diagnosis and regimen start who is nonetheless analysed as "exposed from diagnosis" is a walking immortal-time-bias case study. Setting `time_zero` to `regimen_start` and restricting eligibility to patients alive and in the cohort at that date is the whole trick — the rest of the per-protocol / intention-to-treat machinery (cloning, censoring, and inverse-probability-of-censoring weighting) builds directly on this correctly specified start point and is covered alongside the causal-inference workhorse models later in this chapter.

HEOR in practice. When drafting an RWE protocol for a regulatory or HTA submission, structure the methods section using the Hernán-Robins "specify target trial, then specify emulation" table explicitly — most reviewers trained in the FDA RWE framework or EMA's DARWIN EU studies now expect to see eligibility, treatment strategy, assignment, time zero, outcome, and analysis mapped side-by-side between the hypothetical trial and the observational emulation. A protocol that skips this mapping invites exactly the immortal-time and prevalent-user critiques described above.

Agency perspective. The FDA's 2021 draft guidance **"Considerations for the Design and Conduct of Externally Controlled Trials for Drug and Biological Products"**, together with the 2023 RWD/EHR-and-claims guidance under the agency's RWE Framework (per the 21st Century Cures Act §3022 mandate), explicitly references target trial emulation logic when assessing whether a non-interventional comparator arm is fit to support an effectiveness claim — sponsors are expected to pre-specify the emulated trial's eligibility, treatment strategies, and time-zero alignment before data extraction, mirroring Hernán and Robins' original 2016 *AJE* framework.

15.3 Confounding and bias, with DAGs (`dagitty`, `ggdag`)

Study design controls **which** biases are structurally possible. It does not, by itself, tell you which variables you must adjust for to recover an unbiased causal effect. That is the job of a **directed acyclic graph (DAG)** — a transparent, falsifiable statement of your assumptions about how variables cause one another, drawn **before** you fit any model.

A DAG forces you to distinguish three structures that are easy to confuse in a results-driven rush to "adjust for everything available": **confounders** (common causes of exposure and outcome, which you must adjust for), **mediators** (on the causal pathway from exposure to outcome, which you must **not** adjust for if you want the total effect), and **colliders** (common effects of two variables, which adjusting for actively **induces** bias where none existed).

In a TB regimen-comparison example, HIV status plausibly affects both which regimen a clinician chooses and the probability of treatment success — a classic confounder. Nutritional status measured mid-treatment, however, may be partly a **consequence** of the regimen (some regimens cause more nausea and weight loss) and partly a cause of outcome — a mediator, and adjusting for it would bias the estimate of the regimen's total effect. Site-level treatment capacity might influence both which patients get referred there (a collider with other selection variables) and local reporting completeness. Getting this taxonomy right by eye, on a real dataset with a dozen covariates, is essentially impossible without a graph.

The R packages **dagitty** and **ggdag** let you encode these assumptions as code, visualise them, and — critically — ask the graph itself what the minimal sufficient adjustment set is, rather than adjusting for every variable in the extract "to be safe."

```
library(dagitty)
library(ggdag)
library(ggplot2)

# Encode the causal structure with ggdag::dagify(); exposure/outcome
# arguments tell dagitty which nodes play which role
tb_dag <- ggdag::dagify(
  success ~ regimen + hiv + nutrition + site_capacity,
  regimen ~ hiv + site_capacity,
  nutrition ~ regimen,
  site_capacity ~ urban,
  exposure = "regimen",
  outcome  = "success",
  labels = c(
    regimen       = "Regimen",
    success       = "Tx success",
    hiv           = "HIV status",
    nutrition     = "Nutrition (mediator)",
    site_capacity = "Site capacity",
    urban         = "Urban site"
  )
)

# Visualise with ggdag() -- status highlights exposure/outcome nodes
ggdag::ggdag(tb_dag, text = FALSE, use_labels = "label") +
  ggdag::theme_dag()
```

```
# Ask dagitty for the minimal sufficient adjustment set directly
adj_sets <- dagitty::adjustmentSets(tb_dag, exposure = "regimen", outcome =
"success")
adj_sets
```

Running `adjustmentSets()` on this graph returns `{hiv, site_capacity}` as the minimal sufficient set — both are common causes of regimen **and** success, so both must be adjusted for to block confounding. Critically, `nutrition` is correctly **excluded** because it is a mediator, not a confounder (adjusting for it would block part of the regimen's true causal effect), and `ggdag::ggdag_adjustment_set()` will highlight the same two-variable set visually, which is a useful sanity check to pair with the programmatic answer.

```
# Visual confirmation of the same adjustment set
ggdag::ggdag_adjustment_set(tb_dag, exposure = "regimen", outcome = "success",
                            text = FALSE, use_labels = "label") +
  ggdag::theme_dag()
```

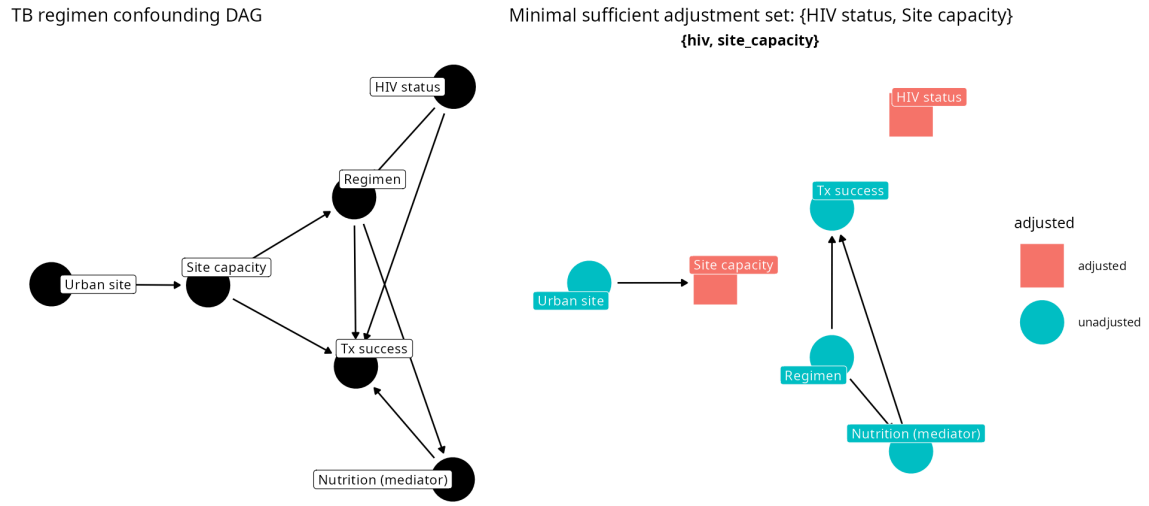


Figure 25: Left: the TB regimen confounding DAG as encoded by `dagify()`. Right: `ggdag_adjustment_set()`'s visual confirmation that `{hiv, site_capacity}` is the minimal sufficient adjustment set — both coloured red as "adjusted" — while `nutrition` (a mediator) is correctly left unadjusted.

Once the adjustment set is settled by the DAG rather than by convenience, the question of **how** to report the resulting observational comparison is itself standardised. The **STROBE statement** (Strengthening the Reporting of Observational Studies in Epidemiology) — and its TREND and RECORD extensions for routinely collected and non-randomised data respectively — specifies a 22-item checklist covering exactly the design, confounding, and bias elements this section has walked through: how participants were selected, how confounders were identified and measured, how missing data were handled, and how potential biases were addressed, mapped item-by-item into the manuscript or dossier. Treat the STROBE checklist as a drafting template, not a post-hoc audit — filling it in alongside the DAG and target-trial protocol from §15.2, rather than after the analysis is finished, will save you from discovering a reporting gap only when a reviewer asks for it.

A final methodological note before we move into estimation proper: almost everything in this chapter — DAG-based adjustment set selection, propensity scores, inverse-probability weighting, and the target trial emulation machinery — is conventionally executed in a **frequentist** frame-

work, and that is the default throughout the rest of this chapter. Bayesian causal inference exists and is occasionally useful, particularly when you want to propagate structural uncertainty (e.g., over which DAG is correct) into the final effect estimate, or when borrowing external control information as in Chapter 14's evidence-synthesis methods — but it is the exception reached for deliberately, not the default starting point, in applied RWE work.

> **Common pitfall.** The single most damaging confounding error in practice is **adjusting for a collider**. If site-level treatment capacity is a common effect of both urban location and regional funding allocation, and you include it in a regression "because it's available and correlated with outcome," you can **induce** a spurious association between regimen and outcome where the true DAG has none. Always draw the DAG and run `dagitty::adjustmentSets()` before selecting covariates — never adjust for a variable solely because it is statistically associated with the outcome in a univariate check.

15.4 Propensity scores: matching, weighting, stratification (`MatchIt`, `WeightIt`, `cobalt`)

The propensity score, formally $e(x) = P(T = 1 \mid X = x)$, is the probability that a unit receives treatment given its observed covariates. Rosenbaum and Rubin's foundational insight was that conditioning on $e(x)$ — rather than on the full covariate vector X — is sufficient to balance observed confounders between treated and untreated groups, provided the no-unmeasured-confounding assumption holds. For a TB-programme analyst this matters concretely: when you compare patients who received a new short-course regimen against those who stayed on the standard regimen in routine programme data, the two groups almost never look alike at baseline. Patients prescribed the new regimen might be healthier, more urban, or diagnosed earlier. The propensity score collapses all of that imbalance into a single number you can match, weight, or stratify on.

Three workhorse strategies use the propensity score once it is estimated (almost always by logistic regression, though machine-learning propensity models are increasingly common):

- *Matching* pairs each treated unit with one or more control units with similar propensity scores, discarding unmatched units.
- *Weighting* (inverse probability of treatment weighting, IPTW) reweights the full sample so that the weighted covariate distributions are balanced, keeping all units.
- *Stratification* bins units into propensity-score strata (commonly quintiles) and estimates effects within strata before pooling.

Matching with MatchIt

`MatchIt::matchit()` is the standard tool. It estimates (or accepts) propensity scores and performs the matching in one call.

```
library(MatchIt)
library(cobalt)

# tb_cohort: one row per patient, trt = 1 if new regimen, 0 if standard
m.out <- matchit(
  trt ~ age + bmi + hiv_status + smear_grade + district_urban,
  data   = tb_cohort,
```

```
  method = "nearest",      # nearest-neighbour matching
  distance = "glm",        # logistic-regression propensity score
  ratio  = 1,               # 1:1 matching
  caliper = 0.2             # max distance, in SD of logit(PS)
)

summary(m.out)              # balance before/after matching
matched_data <- match.data(m.out)   # retrieve matched sample with weights
```

`method = "nearest"` with a caliper is a reasonable default for TB programme comparisons with moderate sample sizes; `method = "full"` (full matching) or `method = "optimal"` can reduce discarded units when treated and control covariate distributions overlap poorly. Always inspect `summary(m.out)` for the standardized mean differences (SMDs) before and after matching — SMDs below 0.1 for every covariate is the conventional threshold for "good balance."

Weighting with WeightIt

Weighting retains the full sample (useful when programme data is already scarce, as TB cohorts often are) at the cost of being more sensitive to extreme propensity scores.

```
library(WeightIt)

w.out <- weightit(
  trt ~ age + bmi + hiv_status + smear_grade + district_urban,
  data     = tb_cohort,
  method   = "ps",          # logistic-regression propensity score
  estimand = "ATE"          # average treatment effect (use "ATT" for effect on
treated only)
)

tb_cohort$iptw <- w.out$weights
summary(w.out)
```

`estimand = "ATE"` weights treated units by $\frac{1}{e(x)}$ and controls by $\frac{1}{1-e(x)}$; `estimand = "ATT"` instead weights controls by $\frac{e(x)}{1-e(x)}$ and leaves treated units at weight 1, targeting the effect in the treated population specifically — often the more policy-relevant estimand when the new regimen will only ever be rolled out to patients similar to those who already got it.

Checking balance with cobalt

`cobalt` is the standard companion package for producing balance diagnostics and the now-ubiquitous "Love plot."

```
library(cobalt)

bal.tab(w.out, un = TRUE, m.threshold = 0.1)   # table: unadjusted vs weighted
SMDs

love.plot(
  w.out,
  binary   = "std",
  thresholds = c(m = 0.1),
  abs      = TRUE,
  var.order = "unadjusted"
)
```

`bal.tab()` prints standardized mean differences (and variance ratios for continuous covariates) before (`un = TRUE`) and after weighting; `love.plot()` renders the now-standard dot plot with a vertical reference line at the balance threshold, letting you see at a glance whether every covariate crossed under 0.1 after weighting.

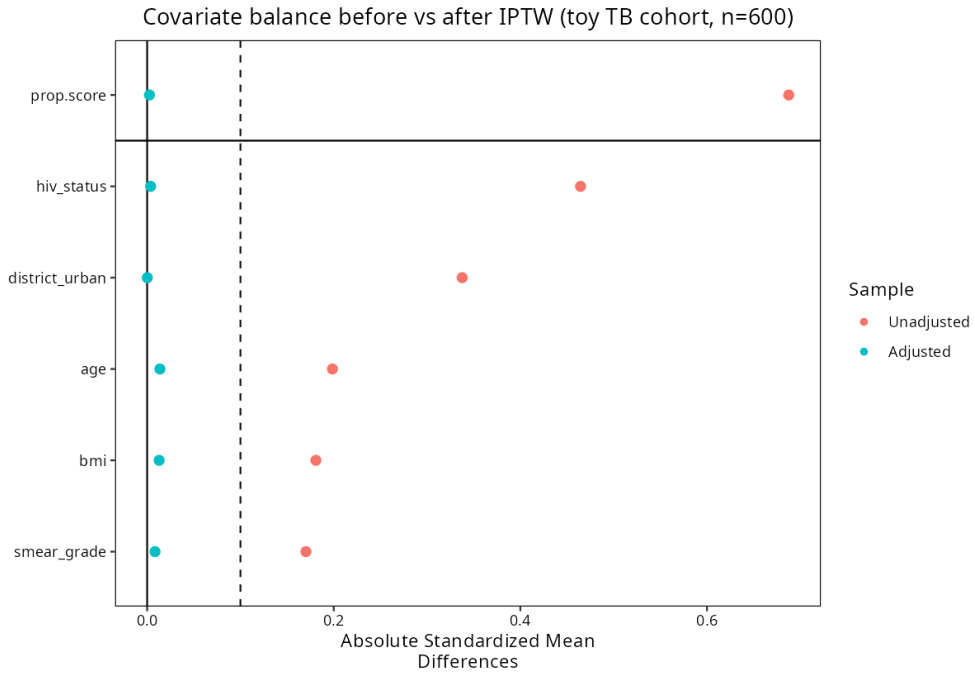


Figure 26: A love plot for a toy TB cohort (n = 600): absolute standardized mean differences before (red) and after (teal) inverse-probability-of-treatment weighting. Every covariate, including the propensity score itself, collapses from well above the conventional 0.1 threshold (dashed line) to effectively zero after weighting.

A worked numeric example: IPTW ATE by hand

To make the weighting arithmetic concrete — and to leave you something you can reproduce by hand before trusting any package output — consider a toy cohort of six TB patients with **known** propensity scores (in practice these come from the logistic model; here they are given so the downstream arithmetic is fully verifiable).

> **HEOR in practice. Toy IPTW calculation.** Treatment outcome Y is, say, time-to-culture-conversion in weeks (lower is better).

How to report this.

Patient	Treated (T)	Outcome (Y)	PS $e(x)$	IPTW contribution
1	1	10	0.5	$1 \times \frac{10}{0.5} = 20$
2	1	8	0.4	$1 \times \frac{8}{0.4} = 20$
3	1	12	0.6	$1 \times \frac{12}{0.6} = 20$
4	0	4	0.5	$1 \times \frac{4}{1-0.5} = 8$
5	0	6	0.4	$1 \times \frac{6}{1-0.4} = 10$
6	0	2	0.6	$1 \times \frac{2}{1-0.6} = 5$

Table 45: Toy cohort for inverse-probability-of-treatment weighting

The IPTW estimator of the ATE is

$$\hat{\tau}_{\text{IPTW}} = \frac{1}{n}\sum_i \frac{T_i Y_i}{e(x_i)} - \frac{1}{n}\sum_i \frac{(1-T_i)Y_i}{1-e(x_i)}$$

Summing the treated contributions: $20 + 20 + 20 = 60$, divided by $n = 6$ gives a weighted treated mean of 10.000. Summing the control contributions: $8 + 10 + 5 = 23$, divided by $n = 6$ gives a weighted control mean of 3.833.

$$\hat{\tau}_{\text{IPTW}} = 10.000 - 3.833 = 6.167 \text{ weeks}$$

Notice the coincidence built into this toy example — each treated patient's contribution is exactly 20 — which is intentional: it shows that when $T_i \frac{Y_i}{e(x_i)}$ is constant across treated units, the weighted mean equals that constant regardless of sample size, a useful sanity check when debugging your own `weightit()` output against a hand calculation on a small subset.

```
# Reproduce the hand calculation in R
toy <- data.frame(
  T  = c(1,1,1,0,0,0),
  Y  = c(10,8,12,4,6,2),
  ps = c(0.5,0.4,0.6,0.5,0.4,0.6)
)
mean(toy$T * toy$Y / toy$ps) - mean((1 - toy$T) * toy$Y / (1 - toy$ps))
#> 6.166667
```

Common pitfall. Extreme propensity scores (close to 0 or 1) produce extreme weights, which can make the IPTW estimator wildly unstable — a single near-zero $e(x)$ in the denominator can dominate the entire estimate. Always plot the distribution of weights (`summary(w.out$weights)`, or a histogram) before trusting an ATE. Trimming (dropping units with $e(x)$ outside, say, $[0.05, 0.95]$) or using **stabilized** weights (multiplying by the marginal probability of treatment) are standard remedies — `WeightIt::weightit()` supports `stabilize = TRUE` directly.

15.5 Doubly robust estimators and g-methods

Matching and weighting both lean entirely on the propensity-score model being correctly specified. Outcome regression leans entirely on the outcome model being correctly specified instead. Each method has exactly one place it can go wrong, and no backup if it does.

Doubly robust (DR) estimators remove that single point of failure. They combine both models so the estimate stays consistent if **either** the propensity model or the outcome model is correctly specified — not necessarily both. That is a meaningfully weaker assumption than either method alone requires, which is why DR methods have become close to a default in contemporary RWE causal analyses, including in regulatory submissions.

The best-known DR estimator is Augmented Inverse Probability Weighting (AIPW):

$$\hat{\tau}_{\text{AIPW}} = \frac{1}{n}\sum_i \left[\frac{T_i(Y_i - \hat{m}_1(X_i))}{\hat{e}(X_i)} - \frac{(1-T_i)(Y_i - \hat{m}_0(X_i))}{1-\hat{e}(X_i)} + \hat{m}_1(X_i) - \hat{m}_0(X_i)\right]$$

where $\hat{m}_1(X)$ and $\hat{m}_0(X)$ are outcome-regression predictions under treatment and control respectively, and $\hat{e}(X)$ is the propensity score. The first bracketed pair is an IPW-weighted **residual correction**; the last pair is the plain outcome-regression estimate. If the outcome model is right, the residuals average to roughly zero and the estimate reduces to the (correct) regression estimate; if the propensity model is right, the IPW terms correct any bias in a misspecified outcome model.

```
library(AIPW)
library(SuperLearner)

aipw_fit <- AIPW::AIPW$new(
  Y          = tb_cohort$culture_conversion_weeks,
  A          = tb_cohort$trt,
  W          = tb_cohort[,
c("age","bmi","hiv_status","smear_grade","district_urban")],
  Q.SL.library = c("SL.glm", "SL.mean"),   # outcome model learners
  g.SL.library = c("SL.glm", "SL.mean"),   # propensity model learners
  k_split    = 10,
  verbose    = TRUE
)
aipw_fit$fit()$summary()
```

A lighter-weight alternative that stays entirely inside the `WeightIt`/regression toolchain, suited for a quick DR check without pulling in `SuperLearner`, is to fit an outcome regression on the IPTW-weighted sample (the "weighted regression" or "doubly robust regression" trick):

```
library(WeightIt)

tb_cohort$iptw <- weightit(
  trt ~ age + bmi + hiv_status + smear_grade + district_urban,
  data = tb_cohort, method = "ps", estimand = "ATE"
)$weights

dr_fit <- lm(
  culture_conversion_weeks ~ trt + age + bmi + hiv_status + smear_grade +
district_urban,
  data    = tb_cohort,
  weights = iptw
```

```
)
coef(summary(dr_fit))["trt", ]
```

This weighted-regression approach is doubly robust in the sense described above whenever the outcome model is linear and additive in treatment; `AIPW` or `tmle`/`tmle3` give the fully general nonparametric version.

G-methods

G-methods — g-computation (the g-formula), marginal structural models fit by IPTW, and g-estimation of structural nested models — were developed by James Robins specifically to handle **time-varying treatment and time-varying confounding**, the situation where today's treatment affects tomorrow's confounder, which in turn affects tomorrow's treatment decision (a "treatment-confounder feedback" loop that standard regression adjustment cannot handle without introducing bias). This is directly relevant to TB programmes: a patient's adherence this month influences next month's sputum-smear result, which influences whether the clinician escalates or de-escalates therapy next month.

> **Difficult terminology: marginal structural models.** A marginal structural model asks a very specific causal question: "what would the outcome look like, on average across the whole population, if everyone followed a particular treatment strategy over time?" — e.g. "everyone stays on the standard regimen for the full six months" versus "everyone is escalated to the stronger regimen as soon as smear status worsens." Normally you'd worry that patients who got escalated were sicker to begin with, confounding any simple comparison. Marginal structural models handle this not by adding the time-varying confounder (smear status) directly into the outcome model — which, with treatment-confounder feedback, can itself introduce bias — but by **reweighting** each patient at each time point using inverse-probability-of-treatment weights (IPTW), so that, after reweighting, treatment at each visit looks "as good as randomly assigned" with respect to the confounders measured up to that point. It's analogous to the reweighting trick in Chapter 13′s MAIC, but applied across repeated time points within the same patients rather than across two separate trials.

> **Difficult terminology: g-estimation of structural nested models.** G-estimation is a different route to the same time-varying-confounding problem that marginal structural models address, but instead of reweighting patients, it directly asks: "by how much would this patient's observed outcome have differed, at each decision point, had their treatment at that point been different from what they actually received?" It builds a "structural nested model" — a set of equations describing, step by step, how each treatment decision nudges the final outcome — and searches (iteratively, via an estimating-equation algorithm) for the parameter values that make the adjusted, counterfactual outcomes look unrelated to the observed treatment history, after accounting for measured confounders. It's more computationally involved than IPTW-weighted marginal structural models and used less often in applied HEOR work, but it avoids some of the extreme-weight instability that can plague IPTW when treatment is very predictable from the confounders.

G-computation in its simplest (time-fixed) form is conceptually just: (1) fit an outcome model including treatment and confounders, (2) predict the outcome for every unit **as if** everyone had

been treated, (3) predict again **as if** no one had been treated, (4) the ATE is the difference of the two averaged predictions.

```
# Simple (time-fixed) g-computation "by hand"
outcome_model <- lm(
  culture_conversion_weeks ~ trt + age + bmi + hiv_status + smear_grade +
district_urban,
  data = tb_cohort
)

d1 <- d0 <- tb_cohort
d1$trt <- 1          # counterfactual: everyone treated
d0$trt <- 0          # counterfactual: no one treated

g_ate <- mean(predict(outcome_model, newdata = d1)) -
         mean(predict(outcome_model, newdata = d0))
g_ate
```

For genuinely time-varying treatment and confounding — multiple visits per patient, each with its own treatment decision and intermediate outcomes — the `gfoRmula` package implements the full longitudinal g-formula with Monte Carlo simulation of counterfactual trajectories, and `ipw::ipwtm()` fits marginal structural models via time-varying IPTW. These are advanced tools; flag them as the next step once your TB programme data includes repeated visits with a feedback loop between adherence, smear results, and regimen changes, rather than reaching for them on a single cross-sectional comparison.

Agency perspective. The FDA's 2023 draft guidance **"Considerations for the Use of Real-World Data and Real-World Evidence to Support Regulatory Decision-Making for Drug and Biological Products"**, together with the companion **"Real-World Data: Assessing Registries to Support Regulatory Decision-Making for Drug and Biological Products"** guidance, explicitly discusses doubly robust estimation and g-methods as acceptable approaches for addressing time-varying confounding in RWE submissions, and cross-references the ICH E9(R1) estimands framework for specifying **which** causal estimand (treatment-policy, hypothetical, principal stratum, etc.) the g-method or DR estimator is meant to target before any model is fit. Before running an AIPW or g-computation analysis intended to support a regulatory or HTA submission, write down the estimand in ICH E9(R1) language first — the choice of estimand constrains which intercurrent-event handling strategy (and therefore which g-method variant) is defensible.

Difficult terminology: ICH E9(R1) estimand strategies + intercurrent events. Before you can estimate a "treatment effect," you have to pin down exactly what question you're asking — and in a real trial or RWE study, things inevitably go wrong partway through: patients drop out, switch to rescue medication, or die before the planned follow-up ends. These mid-study complications are called **intercurrent events**. ICH E9(R1) is the international regulatory guideline that formalises five standard strategies for defining a meaningful "treatment effect" question in the presence of such events. **Treatment-policy** counts whatever happens to the patient regardless of adherence or switching (closest to "real-world" pragmatic effect). **Hypothetical** asks what would have happened if the intercurrent event hadn't occurred at

all (e.g. if no one had been allowed to switch treatment). **Composite** folds the intercurrent event itself into the outcome definition (e.g. counting treatment switch as a treatment failure). **While-on-treatment** only asks about the period before the intercurrent event happened. **Principal stratum** restricts the question to the subgroup of patients who would not have experienced the intercurrent event under either treatment. None of these is universally "correct" — they answer genuinely different clinical questions, and the right strategy depends on what decision-makers actually need to know; the mistake to avoid is picking one implicitly, by whichever analysis method happens to be convenient, rather than choosing deliberately and stating it upfront.

15.6 Instrumental variables and regression discontinuity

Propensity-score and DR methods all assume **no unmeasured confounding**: every variable that affects both treatment assignment and the outcome is observed and included in the model. That assumption can fail badly in TB programme data — e.g., clinician "gut feel" about disease severity may drive both regimen choice and prognosis without appearing in any recorded field. Instrumental variables (IV) and regression discontinuity (RD) are the two classic quasi-experimental designs that can recover a causal effect **even in the presence of unmeasured confounding**, by exploiting a specific source of "as-good-as-random" variation in treatment.

Instrumental variables

An instrument Z is a variable that (i) affects treatment T, (ii) affects the outcome Y **only through** T (the exclusion restriction), and (iii) is independent of unmeasured confounders. A classic TB-programme instrument is distance to the nearest facility offering the new diagnostic/regimen, or facility-level rollout timing driven by a supply allocation that is unrelated to individual patient severity: patients don't choose which facility rolled out first, but facility rollout strongly predicts whether an individual patient received the new regimen.

Two-stage least squares (2SLS) is the standard estimator, implemented identically in `ivreg::ivreg()` (from the `ivreg` package) and `AER::ivreg()` (an older alias from the Applied Econometrics with R package — both use the same `|`-separated formula syntax):

```
library(ivreg)

iv_fit <- ivreg(
  culture_conversion_weeks ~ trt + age + bmi + hiv_status |     # structural
equation
                              facility_rollout_quarter + age + bmi + hiv_status, #
instruments + exogenous covariates
  data = tb_cohort
)
summary(iv_fit, diagnostics = TRUE)
```

The `diagnostics = TRUE` argument is essential: it reports the **weak-instrument (first-stage F) test**, the Wu-Hausman endogeneity test, and the Sargan over-identification test (when you have more instruments than endogenous regressors). A first-stage F-statistic below roughly 10 is the conventional (if debated) threshold for flagging a "weak instrument," below which 2SLS estimates become unreliable and badly biased toward the plain OLS estimate.

Difficult terminology: endogeneity / endogenous variable. A variable is called "endogenous" when it isn't truly independent of the thing that's also driving the outcome — in plain terms, something else (often unmeasured) is influencing both the treatment a patient receives **and** their prognosis, so the treatment variable is partly a stand-in for that hidden factor rather than a clean, as-good-as-randomised input. Picture a clinician who informally senses a patient is sicker than any recorded field captures, and on that gut feeling escalates them to a stronger regimen: the regimen choice is now tangled up with unmeasured severity, and a plain regression of outcome on regimen will attribute some of that unmeasured severity's effect to the regimen itself, biasing the estimate. This is exactly the problem instrumental variables are built to sidestep, by finding a source of variation in treatment (the instrument) that isn't contaminated by that same hidden confounding.

Difficult terminology: Wu-Hausman test. This test directly checks whether endogeneity is actually a real, practically important problem for your data, rather than something to worry about in the abstract. It compares the plain OLS regression estimate against the instrumental-variables (2SLS) estimate of the same effect: if treatment were not endogenous, the two should give similar answers (because OLS would already be unbiased, and IV would just be a noisier way of getting the same answer). A statistically significant Wu-Hausman result means the two estimates disagree by more than chance would explain — evidence that the plain OLS estimate is likely biased by unmeasured confounding, and that you should trust the instrumental-variables estimate instead, despite it usually having a wider confidence interval.

Difficult terminology: Sargan (over-identification) test. This check is only possible when you have **more** instruments than strictly needed (e.g. two separate instruments for one endogenous treatment variable, when one would technically suffice) — hence "over-identified." The idea: if all your instruments are genuinely valid (each satisfying the exclusion restriction), then every possible combination of them should point to roughly the same causal effect. The Sargan test checks whether that's actually true, by seeing whether the instruments as a group produce internally consistent results. A significant Sargan test is a warning sign that at least one of your instruments may not be valid — though it can't tell you **which** one, only that something in the set is inconsistent with the others.

How to report this.

Diagnostic	df1	df2	Interpretation
Weak instruments	1	n-k	F-stat > 10: instrument strong enough
Wu-Hausman	1	n-k	Significant: T is endogenous, IV needed over OLS
Sargan	–	–	Only with more than 1 instrument; tests exclusion restriction

Table 46: Illustrative IV diagnostics output structure from `summary(iv_fit, diagnostics = TRUE)`

Common pitfall. The exclusion restriction — that Z affects Y **only** through T — is never fully testable from data alone and must be argued substantively. "Facility rollout timing" fails as an instrument if rollout was itself prioritized toward facilities with sicker patients (then it affects Y directly through facility-level case mix, not only through individual treatment). Always describe, in words, the mechanism that could violate exclusion, and check whether it is institutionally plausible for your specific rollout — a statistically "strong" instrument (high first-stage F) with an implausible exclusion restriction is still a bad instrument.

Regression discontinuity

RD designs exploit a sharp, known threshold rule for assigning treatment: units just above the cutoff get treated, units just below do not, and — assumption-wise — units just either side of the cutoff are otherwise comparable. A natural TB-programme RD arises when national guidelines mandate the new regimen only for patients with a sputum bacterial load or a CD4 count below a specific cutoff; comparing outcomes for patients just above versus just below that cutoff approximates a local randomized experiment.

`rdrobust::rdrobust()` is the standard, well-validated implementation (local-polynomial estimation with automatic, data-driven bandwidth selection and robust bias-corrected confidence intervals):

```
library(rdrobust)

rd_fit <- rdrobust(
  y = tb_cohort$culture_conversion_weeks,
  x = tb_cohort$cd4_count,
  c = 200                      # the policy cutoff: CD4 < 200 mandates new regimen
)
summary(rd_fit)

rdplot(
  y = tb_cohort$culture_conversion_weeks,
  x = tb_cohort$cd4_count,
  c = 200
)
```

`rdrobust()` reports three estimates (conventional, bias-corrected, and robust), and the robust bias-corrected confidence interval is the one you should report — it accounts for the fact that the optimal bandwidth for point estimation is not the optimal bandwidth for inference. `rdplot()` produces the canonical RD scatter-with-binned-means-and-fitted-lines figure that should accompany every RD analysis, so a reviewer can see the jump at the cutoff directly rather than trusting the number alone.

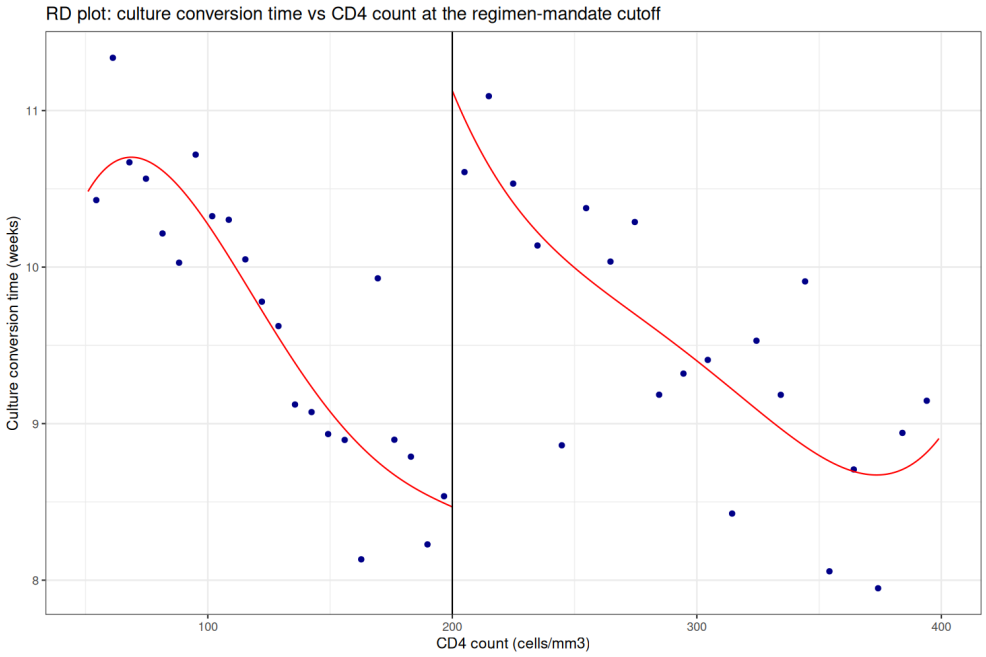


Figure 27: `rdrobust::rdplot()` output for the CD4-count RD example: culture conversion time against CD4 count, with the policy cutoff at CD4 = 200 marked by the vertical line. The fitted local-polynomial curves on either side show a visible downward jump just below the cutoff, where the new regimen becomes mandated.

> **Common pitfall.** RD estimates are **local**: the effect is identified only for patients near the cutoff (CD4 close to 200 in the example), and should never be extrapolated to patients far from it (e.g., CD4 of 50 or 600) without additional structural assumptions. Always run `rdrobust`'s built-in manipulation/density check (`rddensity` package) to confirm patients were not able to manipulate which side of the cutoff they landed on — in a TB programme, a clinician who knows the CD4 threshold and orders a repeat test for a borderline patient to secure the new regimen would silently invalidate the whole design.

15.7 Difference-in-differences and interrupted time series

The final pair of designs in this section uses **time** as the source of quasi-experimental leverage, comparing outcomes before and after an intervention, either against a comparison group that did not receive the intervention (difference-in-differences) or purely through the pre/post trajectory itself (interrupted time series).

Difference-in-differences (DiD)

Classic two-group, two-period DiD estimates the treatment effect as the difference in each group's own pre-to-post change:

$$\hat{\tau}_{\text{DiD}} = \left(\overline{Y}_{\text{treated, post}} - \overline{Y}_{\text{treated, pre}}\right) - \left(\overline{Y}_{\text{control, post}} - \overline{Y}_{\text{control, pre}}\right)$$

This is naturally implemented as a two-way fixed effects (TWFE) regression with a treatment-by-post interaction term:

```
did_fit <- lm(
  culture_conversion_weeks ~ treated_district * post_rollout + age + bmi,
  data = district_panel
)
summary(did_fit)
# coefficient on treated_district:post_rollout IS the DiD estimate
```

> **Difficult terminology: two-way fixed effects (TWFE).** "Fixed effects" here just means: give every district its own intercept adjustment, and every time period its own intercept adjustment, so that each comparison is effectively made relative to that specific district's own average level and that specific period's own average level, rather than lumping everyone together on a single shared baseline. "Two-way" means both of those adjustments (district and time) are included simultaneously. Practically, this is a tidy way of controlling for anything constant about a district (e.g. its baseline healthcare infrastructure) and anything constant across districts within a time period (e.g. a national supply shortage that quarter) without having to measure and name those confounders explicitly — the fixed effects soak them up automatically. The catch, raised in the next paragraph, is that once treatment rolls out to different districts at different times (staggered adoption), this simple two-way structure starts implicitly comparing some already-treated districts against each other as if one were a valid "control" for the other, which can badly distort the estimated effect — hence the shift to purpose-built staggered-adoption estimators like `did::att_gt()`.

The identifying assumption is **parallel trends**: absent the intervention, the treated district(s) would have followed the same **trend** as the control district(s) — not the same level, just the same trajectory. This is checked, never assumed, by plotting pre-intervention outcome trends for both groups and/or by testing for pre-trend differences in an event-study specification (leads and lags of the treatment indicator).

```
library(ggplot2)
ggplot(district_panel, aes(quarter, culture_conversion_weeks, color =
treated_district)) +
  stat_summary(geom = "line") +
  geom_vline(xintercept = rollout_quarter, linetype = "dashed")
```

When rollout is **staggered** across many districts at different times — the realistic case for a phased national TB programme rollout — naïve TWFE regression is now well known to produce badly biased estimates (the "negative weighting" problem documented by Goodman-Bacon and others), because it implicitly compares already-treated districts against each other as pseudo-controls. The modern fix is the `did` package's group-time average treatment effects:

```
library(did)

atts <- att_gt(
  yname    = "culture_conversion_weeks",
  tname    = "quarter",
  idname   = "district_id",
```

```
  gname    = "rollout_quarter",     # period each district FIRST adopted the new
regimen (0/Inf for never-treated)
  data     = district_panel,
  control_group = "nevertreated"     # or "notyettreated"
)

agg_simple <- aggte(atts, type = "simple")   # single overall ATT
summary(agg_simple)

agg_dyn <- aggte(atts, type = "dynamic")     # event-study style, by time since
rollout
ggdid(agg_dyn)
```

`att_gt()` estimates a separate average treatment effect for every (group, time) pair defined by **when** a district first adopted, then `aggte()` aggregates those into either a single summary number or a dynamic event-study plot, avoiding the staggered-adoption bias of plain TWFE entirely.

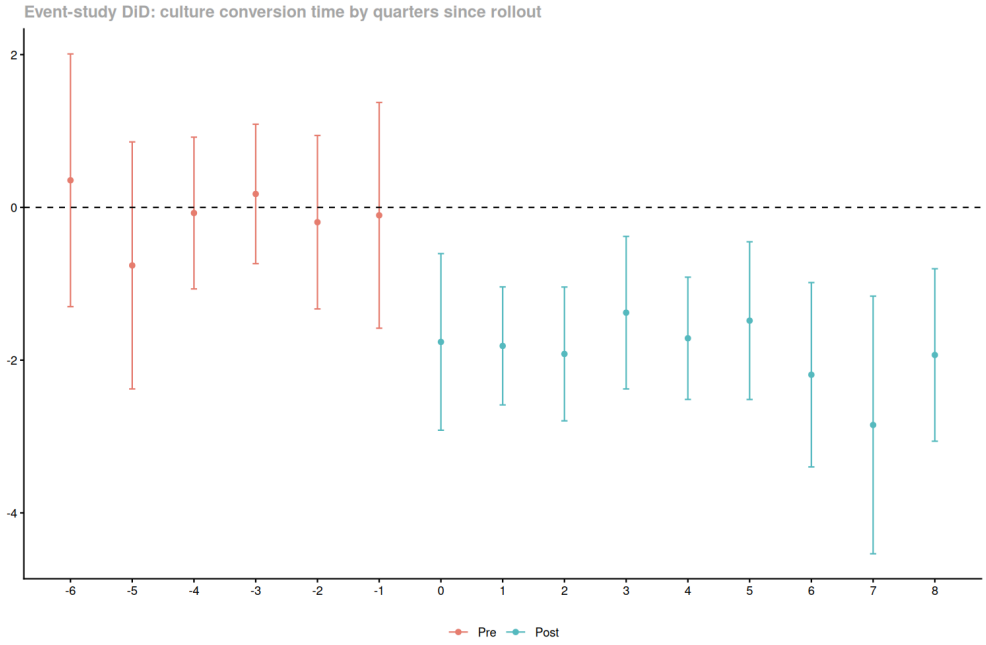


Figure 28: `did::ggdid()` output for a simulated staggered-rollout TB programme: group-time average treatment effects by quarters since rollout (negative x-axis = pre-treatment, positive = post). Pre-treatment estimates hover around zero with overlapping confidence intervals (supporting the parallel-trends assumption), while post-treatment estimates show a consistent negative effect on culture conversion time.

HEOR in practice. Practical rule of thumb. If every treated unit in your rollout switched on the same calendar date (a single national go-live), plain TWFE DiD is fine. If adoption is staggered across districts/facilities — almost always true of a phased national rollout — default to `did::att_gt()` rather than TWFE, even if the TWFE coefficient "looks reasonable"; the bias from staggered TWFE can be large enough to flip the sign of the estimated effect.

Interrupted time series (ITS)

ITS designs use a **single** series — typically an aggregate programme-level outcome measured repeatedly over time, such as monthly treatment-success rate — and ask whether its level and/or slope changed at the moment of an intervention, with no separate control group required (though a control series strengthens inference considerably when available, turning ITS into a comparative/controlled ITS).

Segmented (piecewise) linear regression is the standard specification:

```
library(segmented)

its_base <- lm(success_rate ~ month, data = monthly_tb_outcomes)

its_seg <- segmented(
  its_base,
  seg.Z = ~ month,
  psi   = intervention_month        # known breakpoint: the rollout month
)
summary(its_seg)
slope(its_seg)         # slope before vs after the breakpoint
```

Alternatively, when the breakpoint (intervention timing) is known with certainty — the far more common and more defensible case, since programme rollout dates are typically documented rather than estimated — fit the level-and-slope-change model directly rather than letting `segmented()` search for the breakpoint:

```
monthly_tb_outcomes$post   <- as.integer(monthly_tb_outcomes$month >=
intervention_month)
monthly_tb_outcomes$time_since <- pmax(0, monthly_tb_outcomes$month -
intervention_month)

its_fit <- lm(
  success_rate ~ month + post + time_since,   # level-change (post) and slope-
change (time_since) terms
  data = monthly_tb_outcomes
)
summary(its_fit)
# coefficient on "post" = immediate level change at intervention
# coefficient on "time_since" = change in slope after intervention
```

> **Common pitfall.** Autocorrelation in time series residuals inflates apparent precision and will make an ITS effect look far more statistically significant than it is. Check residual autocorrelation (`acf(residuals(its_fit))`) and, if present, re-fit with Newey-West or autocorrelation-consistent standard errors (`lmtest::coeftest(its_fit, vcov = sandwich::NeweyWest(its_fit))`) rather than trusting the plain `lm()` standard errors. Also watch for seasonality in monthly programme data (e.g., a recurring dip around the monsoon season affecting case detection) being mistaken for an intervention effect if the breakpoint happens to coincide with a seasonal transition.

Agency perspective. NICE's **"NICE real-world evidence framework"** (2022) explicitly names difference-in-differences and interrupted time series among the quasi-experimental designs it considers acceptable supporting evidence for technology appraisals when randomized data are unavailable, provided the parallel-trends or pre-intervention-trend assumption is explicitly tested and reported, not merely asserted. When writing up a DiD or ITS analysis of programme data for an HTA submission, structure the results section around that framework's expectation: show the pre-period trend check first, then the point estimate, then a sensitivity analysis varying the comparison group or breakpoint specification.

15.8 Treatment switching and crossover adjustment (RPSFTM, IPCW)

Oncology and TB trials alike have a recurring headache: patients randomised to the control arm are allowed — often for good ethical reason — to cross over to the experimental treatment once they progress or fail first-line therapy. A placebo-controlled trial of a new multidrug-resistant (MDR-TB) regimen that permits crossover to the novel drug after treatment failure is common practice, and it is the right thing to do for patients. It is also a statistical headache for anyone trying to estimate what survival or treatment-success **would have been** had nobody switched — which is almost always the estimand a reimbursement body wants, because list prices and model structures are built around "control arm never gets the new drug."

A naive intention-to-treat (ITT) analysis of overall survival or time-to-cure under this design is unbiased for the policy actually implemented (randomise-and-allow-switch), but it systematically understates the treatment effect that would be observed under strict, no-crossover randomisation — which is usually the question of interest for a HEOR submission. HTA bodies increasingly expect sponsors to present both the ITT estimate and an adjusted estimate that attempts to remove the dilution caused by switching. The two workhorse methods are the **rank-preserving structural failure time model (RPSFTM)** and **inverse probability of censoring weights (IPCW)**.

RPSFTM assumes a common treatment effect multiplier ψ such that each patient's counterfactual "time off randomised arm's active treatment" is rescaled by $\exp(\psi)$, regardless of when the switch happened — it is a structural, not a regression, model, and it preserves the rank ordering of patients' latent event times under the null. It is robust to time-dependent confounding by design (it does not require measuring the confounders that triggered switching), but it depends on a strong "common treatment effect" assumption and performs poorly when switching is driven by severity that itself predicts the outcome independent of treatment. IPCW instead treats the problem as an informative-censoring problem: patients who switch are artificially censored at the time of switching, and the remaining follow-up is reweighted by the inverse probability of **not yet having switched**, conditional on time-varying prognostic covariates. IPCW requires you to have actually measured and modelled the covariates driving the switch — if an unmeasured factor (say, a clinician's informal read of deteriorating sputum smear trends) drives both switching and survival, IPCW is biased in the same direction as the naive analysis, only less so.

Agency perspective. The definitive methods reference here is the NICE Decision Support Unit's Technical Support Document on treatment switching, usually cited as NICE DSU TSD

16, "Adjusting Survival Time Estimates in the Presence of Treatment Switching" (Latimer and Abrams). It walks through RPSFTM, IPCW, and the two-stage estimation method with worked derivations and recommends which to use depending on whether switching is driven by observed time-varying covariates (favour IPCW) or not (favour RPSFTM), and under what circumstances neither is defensible. Any NICE submission involving crossover-adjusted survival should reference this TSD explicitly in the statistical analysis plan, not just cite the general RPSFTM/IPCW literature.

RPSFTM in R. There is a maintained CRAN package, `rpsftm` (Simon Bond), that implements the standard rank-preserving structural failure time model for a single survival endpoint with simple, one-way, timing-recorded switching. It is the right starting point and saves you from re-deriving the g-estimation search over ψ by hand. It does not, however, cover every design variant the literature discusses (e.g. two-way switching, re-switching, or models that jointly adjust multiple competing-risk endpoints) — for those you are back to hand-coding the structural model, in the same spirit as the hand-coded MAIC weighting discussed in Chapter 13. Treat `rpsftm` as a solid implementation of the textbook case, and verify its assumptions hold before trusting it on a more exotic design.

```
library(rpsftm)
library(survival)

# df has: time (observed), event, arm (0=control,1=experimental),
# rx (proportion of follow-up time actually on experimental treatment),
# plus switching indicator baked into rx for control-arm switchers
fit_rpsftm <- rpsftm(
  Surv(time, event) ~ rand(arm, rx),
  data = df,
  censor_time = admin_censor_time,  # administrative censoring time per patient
  n_eval_z = 100
)

summary(fit_rpsftm)
# psi: the estimated causal log-time-ratio
# Recreate a counterfactual, switching-adjusted Kaplan-Meier using
# fit_rpsftm$Sstar (the re-derived "as randomised, no switching" times)
counterfactual_fit <- survfit(Surv(fit_rpsftm$Sstar, event) ~ arm, data = df)
plot(counterfactual_fit, col = c("grey40", "steelblue"), lwd = 2,
     xlab = "Time (months)", ylab = "Survival probability")
```

IPCW in R. The `ipcwswitch` package (Louis Y et al.) is a real, CRAN-maintained package built specifically for treatment-switching adjustment via IPCW, including time-varying confounder handling and the data-reshaping steps needed to go from clinical-trial-visit format to the long counting-process format the weighted Cox model needs. It is less of a one-line black box than `rpsftm`: expect to spend real effort specifying the switching model (what predicts switching at each visit) and checking weight stability before trusting the output.

```
library(ipcwswitch)
library(survival)

# long_df: one row per patient-visit, with time-varying covariates
# (ecog_status, sputum_grade, weight_kg) measured at each visit, and
```

```r
# a per-visit switch indicator
long_df <- ipcw.data(
  long_df,
  id.name     = "patient_id",
  tstart.name = "visit_start",
  tstop.name  = "visit_stop",
  event.name  = "event",
  bas.cov     = c("age", "baseline_ecog", "site"),
  dyn.cov     = c("ecog_status", "sputum_grade", "weight_kg"),
  switch.name = "switched_this_visit"
)

ipcw_weights <- ipcw.est(
  long_df,
  formula.denom = switched_this_visit ~ ecog_status + sputum_grade + weight_kg,
  formula.num   = switched_this_visit ~ 1,
  id.name = "patient_id", tstart.name = "visit_start", tstop.name = "visit_stop"
)

weighted_cox <- coxph(
  Surv(visit_start, visit_stop, event) ~ arm,
  data = long_df, weights = ipcw_weights$weights, cluster = patient_id
)
summary(weighted_cox)

# Always check the weight distribution before trusting the estimate
summary(ipcw_weights$weights)
hist(ipcw_weights$weights, breaks = 50,
     main = "IPCW weight distribution -- check for extreme tails")
```

Common pitfall. Both methods are routinely misapplied by running them once, reporting the single resulting hazard ratio, and moving on. Three checks are non-negotiable before you put an RPSFTM- or IPCW-adjusted estimate into a cost-effectiveness model: (1) for RPSFTM, inspect whether the "common treatment effect" assumption is remotely plausible — if switchers and non-switchers in the control arm look like different populations at baseline, it probably is not; (2) for IPCW, plot the weight distribution — a handful of weights above 10-20 means a handful of patients are silently driving the whole adjusted estimate, and the standard errors from a naive `coxph` call will be too small (use robust/cluster variance, as shown above); (3) for either method, report the ITT estimate alongside the adjusted one and let the HTA committee see both — presenting only the adjusted, larger effect without the unadjusted comparator reads as cherry-picking even when the adjustment is methodologically sound.

15.9 E-values and quantitative bias analysis

Every observational HEOR analysis — propensity-matched comparisons, instrumental variable estimates, even the RWE arm of an indirect comparison — rests on one untestable assumption: no unmeasured confounder is distorting the association. Quantitative bias analysis (QBA) asks that question explicitly and numerically: how strong would an unmeasured confounder have to be to explain away this result? The **E-value**, introduced by VanderWeele and Ding (2017), is the most widely adopted single-number answer. It is the minimum strength of association, on the risk-ratio

scale, that an unmeasured confounder would need with **both** the exposure and the outcome — beyond the measured covariates — to fully explain away an observed association.

An E-value of 1.5 means a fairly weak unmeasured confounder could plausibly account for the whole effect — treat the finding as fragile. An E-value of 8 means an unmeasured confounder would need an implausibly strong, double-sided association to explain the result away — a much more robust finding. The appeal of the E-value in HEOR practice is that it converts a diffuse methodological worry ("but what about unmeasured confounding?") into a single number a non-statistician reviewer on an HTA committee can weigh against subject-matter plausibility: is it plausible that an unmeasured factor in this TB cohort — say, undocumented nutritional status or informal treatment adherence — has an association of that magnitude with both drug exposure and mortality? Often the answer is "no, that would be an extraordinarily strong confounder," which is reassuring; sometimes the answer is "yes, easily," which should make everyone more cautious about the point estimate.

The `EValue` package on CRAN (maintained by the original VanderWeele/Ding/Mathur group) implements the calculations directly, with functions tailored to each effect-measure scale, since the E-value formula differs depending on whether you start from a risk ratio, odds ratio, hazard ratio, or a continuous (OLS) effect size.

```
library(EValue)

# Risk ratio example: an observational comparison found RR = 1.8 for
# default-vs-treatment-success, 95% CI 1.3 to 2.5, comparing two regimens
ev_rr <- evalues.RR(est = 1.8, lo = 1.3, hi = 2.5)
ev_rr

# Odds ratio from a logistic model of treatment success
ev_or <- evalues.OR(est = 2.1, lo = 1.4, hi = 3.2, rare = TRUE)
# rare = TRUE converts OR to an RR approximation when outcome prevalence < ~15%

# Hazard ratio from a Cox model of time-to-cure
ev_hr <- evalues.HR(est = 0.65, lo = 0.48, hi = 0.88, rare = FALSE)

# Continuous outcome (e.g. change in a quality-of-life score) from OLS
ev_ols <- evalues.OLS(
  est = 4.2, se = 1.1, sd = 9.8,  # coefficient, its SE, and outcome SD
  true = 0  # null value being tested against
)
```

Each call returns both the E-value for the point estimate and the E-value for the confidence-interval bound closest to the null — the second number is usually the more conservative, decision-relevant one, since it answers "how strong a confounder would be needed to make even the **most cautious** end of my interval non-significant?"

For a submission, it is worth tabulating E-values across the key endpoints and model specifications rather than reporting a single number in prose, so a reviewer can see at a glance which findings are robust and which are fragile.

How to report this.

Endpoint	Effect estimate	E-value (point)	E-value (CI limit)
Treatment success (OR)	2.10 (1.40, 3.20)	3.64	2.13
Time to culture conversion (HR)	0.65 (0.48, 0.88)	2.05	1.53
Grade 3+ adverse event (RR)	1.80 (1.30, 2.50)	3.00	1.99
Mean weight gain at 6mo (OLS)	4.2 kg (SE 1.1)	2.88	1.86

Table 47: E-values for key endpoints, illustrative MDR-TB regimen comparison

Reading that table, the treatment-success and adverse-event findings have reasonably robust E-values even at the conservative CI-limit end (around 2), while the culture-conversion hazard ratio's CI-limit E-value of 1.53 is the one worth flagging to the committee as more sensitive to unmeasured confounding than the headline point estimate suggests.

HEOR in practice. E-values do not replace a proper sensitivity analysis, and they say nothing about **whether** an unmeasured confounder of that strength actually exists — only about the magnitude required. Pair the E-value with a short qualitative argument grounded in the clinical setting: for a TB programme comparison, is there a plausible unmeasured factor (health-seeking behaviour, nutritional status, informal regimen substitution at peripheral facilities) that could realistically reach the required strength of association with both exposure and outcome? If a clinical co-author can confidently say "no candidate confounder in this setting is remotely that strong," the E-value becomes a persuasive robustness statement rather than a statistical decoration. If they hesitate, that hesitation is itself useful information for the write-up.

15.10 OHDSI/OMOP workflows in R

Most of this chapter has assumed you already have a tidy analysis-ready dataset — one row per patient, clean dates, a `survival::Surv()` object waiting to be fit. Real-world claims and EHR data almost never arrive that way, and a large, genuinely global effort has grown up around standardising how such data gets structured before analysis: the **Observational Health Data Sciences and Informatics (OHDSI)** collaborative and its **OMOP Common Data Model (CDM)**. If you ever work with a multi-country claims database, a hospital EHR warehouse, or a federated network study, there is a reasonable chance the data has already been (or will need to be) mapped into OMOP CDM — a standardised relational schema with fixed tables (`person`, `condition_occurrence`, `drug_exposure`, `visit_occurrence`, and so on) and standardised vocabularies (SNOMED, RxNorm, LOINC) so that analytic code written once can, in principle, run unchanged against a hospital database in Seoul and a claims database in the United States.

The OHDSI R ecosystem is built in layers, each with a genuinely maintained, actively developed package:

- `DatabaseConnector` handles the low-level connection to whatever RDBMS actually stores the OMOP CDM (PostgreSQL, SQL Server, Redshift, BigQuery, Oracle), abstracting away vendor-specific SQL dialects via SqlRender underneath.
- `FeatureExtraction` turns raw OMOP tables into analysis-ready covariate matrices — demographics, comorbidity flags, prior drug exposure counts — using pre-built "covariate settings" rather than hand-written SQL for every variable.
- `CohortMethod` runs the standard new-user comparative cohort design (think: a propensity-score-matched or -weighted comparison of two drug exposure cohorts) end to end, from cohort definition through outcome model fitting, following the OHDSI network-study conventions.

```
library(DatabaseConnector)
library(FeatureExtraction)
library(CohortMethod)

connection_details <- createConnectionDetails(
  dbms             = "postgresql",
  server           = "cdm-server.example.org/omop_cdm",
  user             = Sys.getenv("OMOP_DB_USER"),
  password         = Sys.getenv("OMOP_DB_PASSWORD"),
  port             = 5432
)

# Define covariate settings: demographics + comorbidity history
covariate_settings <- createDefaultCovariateSettings(
  addDescendantsToExclude = TRUE
)

# Cohorts 1 (target drug) and 2 (comparator) must already exist as
# cohort_definition_id values in a cohort table built upstream
# (via ATLAS or hand-written cohort-definition SQL against OMOP tables)
cm_data <- getDbCohortMethodData(
  connectionDetails    = connection_details,
  cdmDatabaseSchema    = "omop_cdm",
  targetId             = 1,
  comparatorId         = 2,
  outcomeIds           = 3,     # e.g. treatment failure / default
  cohortDatabaseSchema = "results_schema",
  cohortTable          = "cohort",
  covariateSettings    = covariate_settings
)

ps_model <- createPs(cohortMethodData = cm_data)
matched_pop <- matchOnPs(ps_model, maxRatio = 1)

outcome_model <- fitOutcomeModel(
  population = matched_pop,
  cohortMethodData = cm_data,
  outcomeId = 3,
  modelType = "cox"
```

```
)
summary(outcome_model)
```

This is a substantial step up in infrastructure from the single-flat-file analyses elsewhere in this book, and it is worth being candid about when it is and is not the right tool. If you are running a one-off analysis of a single already-extracted dataset, standing up `DatabaseConnector`/`CohortMethod` against a live OMOP instance is almost certainly overkill — the earlier chapters' direct-dataframe workflows will get you to the same propensity-matched Cox model faster. The OHDSI stack earns its complexity when you need **reproducibility across sites** — a federated network study spanning several country-level TB or HIV surveillance databases, where the same analysis code genuinely has to run, unmodified, against structurally different source systems. Converting a bespoke national TB registry into OMOP CDM is itself a non-trivial ETL project (the OHDSI community's "Rabbit-in-a-Hat" / White Rabbit tooling helps, but mapping local vocabularies to standard concepts is still substantial domain work), so weigh that upfront cost against the actual need for cross-site standardisation before committing to it.

> **Common pitfall.** A common mistake with `CohortMethod` output is treating the within-database, single-site effect estimate as automatically generalisable, simply because the pipeline **looks** standardised. OMOP CDM standardises the **data structure and vocabulary**, not the underlying population, prescribing patterns, or coding practices at each site. A network study pooling results across five databases still needs the same scrutiny of comparability, residual confounding, and effect heterogeneity across sites that any multi-country RWE synthesis would — the OHDSI tooling makes the mechanics of running the same analysis everywhere easy; it does not make the resulting estimates automatically poolable or automatically externally valid to a setting (such as a high-burden, low-resource TB programme) that may look nothing like the health systems contributing data to the source databases.

15.11 Using external control arms and synthetic control arms

Single-arm trials are increasingly common in rare-disease and some oncology development, and the logic extends naturally to settings like a novel TB regimen trial run under compassionate-use or expanded-access protocols, where randomising a concurrent control arm is ethically or operationally impractical. The regulatory and HTA question that follows is unavoidable: compared to what? An **external control arm (ECA)** answers that by assembling a comparator cohort from a source other than the trial itself — a registry, a previous trial's control arm, real-world claims or EHR data, or a historical cohort — and comparing the single-arm trial's outcomes against it. A **synthetic control arm** is a particular, more formal construction of an ECA: rather than simply selecting "similar" historical patients, it builds the comparator as a weighted combination of external units (and sometimes pre-treatment outcome trajectories) explicitly chosen so that the synthetic cohort's observed characteristics, and ideally its outcome trajectory, closely track what the trial's intervention arm looked like **before** treatment — the Abadie/Diamond/Hainmueller synthetic control method from econometrics, and its clinical adaptations, generalise the comparative-case-study idea to a clinical comparator.

Both approaches face the same fundamental threat that randomisation exists to neutralise: the external cohort was never subject to the same allocation mechanism as the trial arm, so any difference in outcomes is a tangle of true treatment effect and systematic differences in who

ended up in which cohort — differences in era (standard of care drifts over a decade), in setting (a well-resourced referral hospital's historical cohort versus a multi-country trial's recruiting sites), in measurement (different diagnostic criteria, different definitions of "cure"), and in unmeasured patient selection (who gets referred, who gets excluded, who survives to be included in a registry at all). The statistical toolkit for narrowing that gap is the same one introduced in Chapter 12 for indirect comparisons — propensity score matching or weighting the external cohort to the trial population on measured baseline covariates — but it inherits the same core limitation: it can only balance what was actually measured comparably in both sources, and registry or claims data are rarely measured as rigorously or completely as a trial case report form.

```
library(survival)
library(MatchIt)

# combined_df: trial arm (source == "trial") stacked with an external
# registry cohort (source == "registry"), harmonised on shared baseline
# covariates measured comparably in both sources
combined_df$is_trial <- as.integer(combined_df$source == "trial")

match_model <- matchit(
  is_trial ~ age + baseline_ecog + extent_of_disease + prior_regimens,
  data   = combined_df,
  method = "nearest",
  ratio  = 2  # up to 2 external controls matched per trial patient
)

matched_data <- match.data(match_model)

eca_fit <- coxph(
  Surv(time, event) ~ is_trial,
  data    = matched_data,
  weights = weights  # matching weights from MatchIt
)
summary(eca_fit)

# Always report the covariate balance achieved by the match before
# trusting the comparative estimate
summary(match_model)
```

Because no CRAN package markets itself specifically as "the synthetic control arm for clinical trials" tool, in practice sponsors either adapt general-purpose econometric synthetic control packages (such as `Synth` or the newer `tidysynth`, both built for panel-data comparative case studies rather than clinical cohorts and requiring real adaptation of the outcome-trajectory-matching logic) or build the weighting and balance-checking pipeline by hand from `MatchIt`, `WeightIt`, or `cobalt`, in much the same spirit as the hand-coded MAIC and the `rpsftm` edge cases discussed earlier. Do not expect to find a single, clinically-validated, point-and-click CRAN package that performs a defensible synthetic control arm construction end to end — the statistical judgment about covariate selection, era-matching, and outcome-trajectory comparability is the genuinely hard, non-automatable part of the exercise.

> **Agency perspective.** The FDA's 2023 draft guidance, "Considerations for the Design and Conduct of Externally Controlled Trials for Drug and Biological Products," is the current

authoritative regulatory reference on when an external control arm is an acceptable substitute for a concurrent randomised control, what data quality and comparability standards it expects of the external source, and how sponsors should pre-specify the analytic approach (propensity methods, sensitivity analyses for unmeasured confounding, and the role of E-values of the kind introduced in §15.9) before seeing outcome data. Any HEOR team proposing an external or synthetic control arm for a regulatory or HTA submission should read this guidance directly rather than relying on secondary summaries, since it is explicit about the scenarios (small, well-characterised natural history; large, unambiguous treatment effect) where FDA considers the design most defensible, and equally explicit about where it is skeptical.

Common pitfall. The single most common failure mode in external control arm analyses submitted for HTA review is presenting the propensity-matched comparison as if matching had resolved all comparability concerns, without separately addressing **temporal** comparability. A registry cohort treated five or ten years before the single-arm trial recruited is not just a different sample of patients — it reflects an entirely different standard of supportive care, diagnostic practice, and often a different definition of the outcome itself (TB treatment-success definitions, for instance, have shifted with WHO guideline revisions). Matching on measured baseline covariates does nothing to correct for era effects that operate through the **outcome-generating process** rather than through baseline patient characteristics. Always report the calendar period of both cohorts explicitly, justify why era drift is not expected to materially bias the comparison (or quantify how much it plausibly could, borrowing the E-value logic from §15.9), and treat any external-control-arm estimate as inherently more fragile than a concurrently randomised one when presenting it to a reimbursement committee.

Treatment switching, unmeasured confounding, federated OMOP networks, and external control arms all share a common thread: once you step outside a cleanly randomised, concurrently controlled trial, every causal estimate is only as credible as the assumptions propping it up, and this chapter's job has been to make those assumptions visible, testable where possible, and honestly reported where they are not. With that causal-inference foundation in place, Chapter 16 turns to the regression toolkit that sits on top of it — generalised linear models, mixed-effects models for clustered and longitudinal HEOR data, zero-inflated and hurdle models for cost and utilisation outcomes, quantile and ordinal regression for distributional and categorical endpoints, and machine-learning approaches to heterogeneous treatment effects and subgroup analysis — the modelling machinery that turns a defensible causal estimand into a defensible, decision-ready number.

Chapter 16. Regression Models for HEOR Data

Chapter 15 estimated treatment effects while adjusting for confounding — propensity scores, doubly robust estimators, instrumental variables. This chapter steps back to the regression machinery those methods are built from, and extends it in two directions the earlier chapters only touched in passing: handling the specific outcome shapes HEOR data actually takes (binary, count, skewed, ordinal), and handling the structure HEOR data actually has (patients clustered within clinics, treatment effects that vary across patients rather than sitting at one average value). By the end of the chapter, the regression toolkit used everywhere else in this book — cost models in Chapter 9, utility mapping in Chapter 8, the GLM core of the propensity-score and doubly-robust estimators in Chapter 15 — will sit inside one coherent framework.

Learning objectives

- Fit and interpret the three GLM families used throughout HEOR — binomial/logistic, Poisson/negative-binomial, and gamma — and correctly back-transform coefficients through the link function.
- Recognise when HEOR data is clustered (patients within clinics, repeated measures within patients) and fit a mixed-effects model with `lme4`.
- Model overdispersed and zero-inflated count outcomes, and distinguish a zero-inflated model from a hurdle model.
- Use quantile regression to model a specific percentile of a skewed cost or length-of-stay distribution, and ordinal regression for Likert-scale PRO data.
- Use random/causal forests and penalised regression (LASSO/elastic net) to estimate treatment-effect heterogeneity and select variables from a high-dimensional covariate set.
- Distinguish pre-specified from exploratory subgroup analysis, test effect modification via an interaction term, and avoid the multiplicity pitfall of subgroup fishing.

16.1 Generalised linear models

Every regression model you have fit so far in this book — the gamma cost models in Chapter 9, the logistic-type transformations behind utility regression in Chapter 8 — is a special case of one framework: the **generalised linear model** (GLM). A GLM has three ingredients: a distribution for the outcome from the exponential family (binomial, Poisson, negative binomial, gamma, Gaussian, ...), a **linear predictor** ($\eta = \beta_0 + \beta_1 x_1 + \ldots$), and a **link function** that connects the two ($g(\mu) = \eta$). Once you see the pattern, "which model do I fit?" becomes "which distribution matches my outcome's support, and which link makes the linear predictor behave?" — rather than a new method to memorise every time.

> **Difficult terminology: exponential family.** "Exponential family" is not a statement about exponential growth — it's a technical label for a whole group of probability distributions (normal/Gaussian, binomial, Poisson, negative binomial, gamma, and a handful of others) that

all share the same underlying mathematical form once you write them out algebraically. That shared form is the reason a single piece of software machinery — the GLM-fitting algorithm — can estimate all of them: R doesn't need a bespoke fitting routine for "binary outcome regression" and a different one for "count outcome regression"; it needs one general algorithm plus a specification of **which** exponential-family distribution and **which** link function apply to your outcome. Practically, you never need to verify exponential-family membership yourself — the handful of distributions you'll actually use in HEOR work (binomial for yes/no outcomes, Poisson or negative binomial for counts, gamma for skewed positive costs, Gaussian for roughly symmetric continuous outcomes) are all members, which is precisely why `glm()` can fit all of them through the same `family =` argument.

Three outcome types cover most HEOR regression work:

- **Binary outcomes** (treatment success/failure, culture conversion, mortality) use the binomial family with a logit link — logistic regression. The logit link keeps predicted probabilities between 0 and 1.
- **Count outcomes** (hospitalisations, adverse-event episodes, clinic visits) use a Poisson or negative-binomial family with a log link. Negative binomial is the safer default in practice: TB programme count data is almost always overdispersed (variance exceeds the mean), and Poisson's assumption that they're equal is routinely violated.
- **Skewed, strictly positive cost data** uses a gamma family with a log link — exactly the model built in detail in §9.4. This subsection will not re-derive it; the mechanics (mean-scale modelling via the log link, no retransformation bias) are identical, because it is the same GLM framework applied to a different exponential-family member.

In base R, all three are one function: `glm()`. You only change the `family` argument.

```
library(dplyr)
library(broom)

# Toy multi-site TB treatment-outcome dataset
tb_outcomes <- tibble::tribble(
  ~patient_id, ~regimen,    ~age, ~baseline_smear, ~success,
  1,           "standard",  34,   "positive",      1,
  2,           "standard",  45,   "positive",      0,
  3,           "shortened", 29,   "negative",      1,
  4,           "shortened", 51,   "positive",      1,
  5,           "standard",  38,   "negative",      0,
  6,           "shortened", 61,   "positive",      0,
  7,           "shortened", 27,   "negative",      1,
  8,           "standard",  42,   "positive",      1
)

fit_logit <- glm(
  success ~ regimen + age + baseline_smear,
  family = binomial(link = "logit"),
  data   = tb_outcomes
)

# Tabulate coefficients, odds ratios and confidence intervals
broom::tidy(fit_logit, exponentiate = TRUE, conf.int = TRUE)
```

```
# Predicted probability of success for a new patient
predict(
  fit_logit,
  newdata = tibble::tibble(regimen = "shortened", age = 40, baseline_smear =
"positive"),
  type    = "response"
)
```

`broom::tidy(..., exponentiate = TRUE)` is doing real work here, not just formatting: it exponentiates the log-odds coefficients into odds ratios so you can report them directly. `predict(..., type = "response")` back-transforms through the inverse link to give a probability rather than a log-odds value. Both arguments matter, and forgetting them is the single most common GLM reporting error.

> **Common pitfall.** A GLM's raw coefficients live on the **link** scale, not the scale anyone wants to read. For a logistic model, `coef(fit_logit)` returns log-odds; for a Poisson or gamma model with a log link, raw coefficients are log-rate or log-cost-ratio. Reporting these numbers unchanged — "age effect: 0.03" — means almost nothing to a TB programme manager. Always exponentiate (`exp(coef(...))`, or `broom::tidy(..., exponentiate = TRUE)`) before presenting results, and say explicitly what the exponentiated quantity is: an odds ratio, a rate ratio, or a cost ratio.

> **Agency perspective.** CHEERS 2022 item 26 ("Characterising heterogeneity") and item 20 ("Study parameters") both expect the statistical model underlying any regression-derived parameter to be named explicitly in the report — not just the resulting estimate. For a GLM, that means stating the family and link function (e.g. "binomial GLM with logit link") alongside the odds ratio or rate ratio, so a reviewer can judge whether the model choice matched the outcome's distribution.

16.2 Mixed-effects and hierarchical models

The GLMs in §16.1 assume every observation is independent. That assumption breaks the moment your data has structure: patients nested within clinics, clinics nested within regions, or repeated HRQoL measurements taken from the same patient over time. Patients treated at the same clinic share a nurse, a supply chain, a local strain profile — they are **more alike than two random patients**, and a plain GLM has no way to represent that.

Fixed effects are the parameters you want to estimate and report: the effect of regimen, of age, of baseline smear status — the things in §16.1's `glm()` calls. **Random effects** represent variation across a grouping factor (clinic, patient) that you want to account for, but whose specific levels aren't individually of interest — you don't care that Clinic 7's intercept is 0.3 higher than Clinic 2's, you care that clinics vary, and you want your fixed-effect estimates and standard errors to be correct given that they do.

Ignoring clustering and fitting a naive GLM when data is actually hierarchical doesn't just bias your standard errors — it specifically makes them too small, because the model treats twenty patients from one clinic as twenty independent pieces of information when they're closer to a handful.

```r
library(lme4)

# Toy multi-site TB dataset: patients nested within clinics
tb_multisite <- tibble::tribble(
  ~clinic_id, ~patient_id, ~regimen,    ~age, ~success,
  "A", 1,  "standard",  34, 1,
  "A", 2,  "standard",  45, 0,
  "A", 3,  "shortened", 29, 1,
  "B", 4,  "shortened", 51, 1,
  "B", 5,  "standard",  38, 0,
  "B", 6,  "shortened", 61, 0,
  "C", 7,  "shortened", 27, 1,
  "C", 8,  "standard",  42, 1,
  "C", 9,  "standard",  33, 0,
)

# Random intercept for clinic, fixed effects for regimen and age
fit_mixed <- glmer(
  success ~ regimen + age + (1 | clinic_id),
  family = binomial(link = "logit"),
  data   = tb_multisite
)

summary(fit_mixed)
broom.mixed::tidy(fit_mixed, effects = "fixed", exponentiate = TRUE, conf.int =
TRUE)
```

`lme4::glmer()` is the binomial/Poisson/gamma-family counterpart of `lme4::lmer()` (which fits continuous, Gaussian outcomes — the natural choice for repeated HRQoL scores measured on the same patient over time). The `(1 | clinic_id)` term is the random intercept: one baseline log-odds per clinic, drawn from a shared normal distribution, rather than one fixed-effect dummy per clinic eating up degrees of freedom. `REML = TRUE` (the default for `lmer()`, though `glmer()` always uses maximum likelihood) controls how the random-effect variance is estimated — leave it at its default unless you have a specific reason not to. `nlme::lme()` is an older alternative with the same core idea, mainly still seen when you need its more flexible correlation-structure syntax (e.g. for repeated-measures autocorrelation).

How much clustering is there? The **intraclass correlation coefficient** (ICC) answers that: it's the proportion of total outcome variance attributable to the grouping factor (clinic), versus variance between patients within the same clinic. An ICC near 0 means clinics barely differ and a plain GLM would have been fine; an ICC of 0.15 means a meaningful share of outcome variation sits at the clinic level, and ignoring it would understate your uncertainty.

A **random intercept** (`(1 | clinic_id)`) lets each clinic have its own baseline outcome level. A **random slope** (`(1 + age | clinic_id)`) goes further and lets the **effect** of a covariate — say, the age effect — vary by clinic too, useful when you suspect, for instance, that older patients do worse at some clinics but not others. Start with random intercepts; add random slopes only when you have a substantive reason to think an effect itself varies by group, since each additional random-effects term costs you estimability, especially with as few clusters as a multi-site TB programme typically has.

Agency perspective. NICE's Decision Support Unit technical support document on evidence synthesis for clinical effectiveness (TSD 3) explicitly recommends a random-effects (hierarchical) model over a fixed-effect pooling when between-study — or, by direct analogy, between-clinic — heterogeneity is plausible, rather than treating pooled homogeneity as a safe default assumption.

16.3 Count data and zero-inflated models

Count outcomes show up constantly in TB programme data: hospitalisations per patient-year, missed follow-up visits, adverse events reported per participant. The instinct is to reach for Poisson regression. The problem is that Poisson assumes the mean equals the variance, and real programme data almost never behaves that politely — a few patients rack up many events while most have none or one, so the variance outruns the mean. This is overdispersion, and ignoring it gives you standard errors that are too small and p-values that look better than they should.

The fix for plain overdispersion is the negative binomial model, fit with `MASS::glm.nb()`. It has the same formula interface as `glm()` but estimates an extra dispersion parameter instead of assuming it away:

```
library(MASS)

fit_nb <- glm.nb(ae_count ~ arm + age + baseline_risk, data = tb_ae)
summary(fit_nb)
```

`glm.nb()` reports the usual coefficient table on the log scale (exponentiate for rate ratios, same logic as Poisson), plus a `theta` value — smaller `theta` means more overdispersion.

Overdispersion is one problem; **excess zeros** are a different one. Suppose you're counting adverse events and most patients genuinely never have one — not because of under-reporting, but because they simply didn't experience an AE. A negative binomial model can stretch to accommodate more zeros than Poisson expects, but when the zeros come from a distinct process (a patient who could never have an AE vs. one who could but didn't), you need a model that says so explicitly: a zero-inflated model, fit with `pscl::zeroinfl()`.

```
library(pscl)

fit_zinb <- zeroinfl(
  ae_count ~ arm + age | arm,
  data  = tb_ae,
  dist  = "negbin"
)
summary(fit_zinb)
```

The formula has two parts separated by `|`: the left side models the count (how many events, given you're in the "can have events" group), the right side models the probability of being a structural zero. `dist = "negbin"` lets the count part also absorb ordinary overdispersion; `dist = "poisson"` is the simpler default.

A closely related but conceptually different model is the **hurdle** model, available via `pscl::hurdle()`. Zero-inflated and hurdle models both have a two-part structure, but they answer

different questions. A zero-inflated model says some zeros are "structural" (could never have had an event) and some are "sampling" zeros from the count process itself — the two kinds of zero are mixed together. A hurdle model instead says every subject first clears a hurdle (any events at all, yes/no), and only those who clear it move to a truncated count model for **how many**. For AE counts, zero-inflation is usually the more defensible story (patients who were never going to have an AE vs. those who could but didn't); hurdle models fit better when crossing from zero to one event is genuinely a different mechanism from the counting process above one.

HEOR in practice. A payer dossier for a new TB regimen reported adverse-event counts using plain Poisson regression. A reviewer noted that over 60% of patients had zero AEs and asked the sponsor to justify the distributional assumption. Re-analysis with `pscl::zeroinfl()` separated "never going to have an AE" patients from the count process for those who did, and the treatment effect on AE **rate** (among patients who had any) held up — but the original Poisson model had understated the uncertainty around it, almost triggering a resubmission.

Common pitfall. Fitting plain Poisson regression to overdispersed or zero-heavy count data is one of the most common silent errors in HEOR analysis. The point estimates often look reasonable; it's the standard errors that are wrong — usually too small — which inflates Type I error and can make a null effect look significant. Always check the ratio of residual deviance to degrees of freedom (well above 1 signals overdispersion) and tabulate the proportion of zeros before trusting a Poisson model.

Difficult terminology: Type I error. A Type I error is the statistical term for a "false positive": you conclude there is a real effect or difference (e.g., a drug lowers cost, or a treatment changes outcomes) when, in the underlying truth, there is no such effect, and what you saw was just random noise in your particular sample. The conventional threshold of $\alpha = 0.05$ means you've agreed to accept a 5% chance of this kind of false alarm whenever a true null effect is tested. The problem flagged in the pitfall above is that bad standard errors (too small) make p-values look smaller than they should, so you end up crying wolf more than 5% of the time without realising it — the nominal 5% false-positive rate quietly becomes something larger, like 15-20%, because the model is overconfident about its own precision.

Difficult terminology: residual deviance / degrees of freedom. These are two separate numbers that, together, give you a quick diagnostic check. **Residual deviance** is a single number summarising how far the model's fitted values are from the actual observed data — think of it as a generalisation of "sum of squared errors" that works for GLMs, not just ordinary linear regression; smaller is a better fit. **Degrees of freedom** here is, roughly, the number of observations in your dataset minus the number of parameters your model estimated — it reflects how much independent information was left over after fitting. For a well-behaved Poisson model, textbook theory says these two numbers should come out roughly equal, so dividing deviance by degrees of freedom should give something close to 1. If that ratio comes out well above 1 (say, 3 or 5), it is a red flag that your data has more variability than a plain

> Poisson distribution assumes — this is called **overdispersion**, and it is exactly the kind of thing that inflates Type I error as described above. In R, both numbers are printed automatically at the bottom of `summary(my_glm_model)`, so the check costs nothing beyond glancing at output you already have.

16.4 Quantile and ordinal regression

Chapter 9 made the case that cost data is skewed — a long right tail of expensive patients pulls the mean around, which is why GLMs with a log link or gamma family were preferred over OLS on raw costs. Quantile regression takes a different route to the same problem: instead of modelling the **mean** cost (transformed or not), it models a chosen **percentile** of the cost distribution directly.

This matters because the mean and the median (or the 90th percentile) can tell different stories. A budget-holder asking "what does a typical TB patient's length of stay cost?" wants the median. A capacity planner asking "what should we provision for the costly tail?" wants the 90th or 95th percentile. Quantile regression, via `quantreg::rq()`, answers either question in one formula-based call:

```
library(quantreg)

# Median length of stay
fit_median <- rq(los ~ arm + age + comorbidity_score, data = tb_cost, tau = 0.5)

# 90th percentile length of stay — the costly tail
fit_q90 <- rq(los ~ arm + age + comorbidity_score, data = tb_cost, tau = 0.9)

summary(fit_median)
summary(fit_q90)
```

`tau` sets the quantile (0.5 is the median; any value in (0,1) is valid). The coefficients are interpreted like linear regression coefficients, but for that specific quantile — a coefficient from the `tau = 0.9` model describes how a predictor shifts the **top** of the distribution, which can differ meaningfully from its effect on the median. This complements, rather than replaces, the Chapter 9 approach to skewed cost data: use GLMs when you want a single mean-cost estimate for a budget-impact model, and quantile regression when the question is genuinely about a specific point in the distribution.

The second regression family here is for ordinal outcomes — Likert-scale patient-reported outcomes (PROs), staged disease severity, anything with ordered categories but no meaningful numeric distance between them. Chapter 8 already introduced `MASS::polr()` for this, in the context of mapping ordinal clinical descriptors to utility weights; the same function applies directly to ordinal PRO analysis, so there's no need to re-derive it here — see Chapter 8 for the syntax and interpretation of the proportional-odds coefficients. An alternative with a richer set of diagnostics and link-function options is `ordinal::clm()`, which uses the same proportional-odds logic but is often preferred when you need formal model comparison tools or non-logit links.

Both `polr()` and `clm()` rest on the **proportional-odds assumption**: that the effect of each predictor is the same across every cut-point of the ordinal scale (e.g. a treatment's effect on moving from "poor" to "fair" is assumed equal to its effect moving from "good" to "excellent"). This

assumption is routinely **assumed** and rarely **checked** in practice, which is a problem — it can fail, especially with PRO scales that compress responses at one end.

Common pitfall. Proportional odds is a testable assumption, not a formality. A quick check is to fit separate binary logistic models at each cut-point of the ordinal outcome and see whether the coefficients for key predictors are roughly stable across cut-points; a formal Brant-type test is also commonly used, though it is not bundled directly into `MASS::polr()` and typically requires a separate package or manual implementation. If the assumption clearly fails, consider a partial proportional-odds model or report cut-point-specific effects instead of a single pooled odds ratio.

Difficult terminology: Brant-type test. The proportional-odds model (used for ordinal outcomes — things with a natural order but no fixed numeric scale, like a pain-severity rating of none/mild/moderate/severe) makes a strong simplifying assumption: that a predictor's effect is the **same** regardless of which cut-point of the outcome you look at (e.g., "mild-or-worse vs. none" and "severe vs. everything-else" are assumed to move by the same amount for a one-unit change in the predictor). A Brant-type test is a formal statistical check of that assumption — it compares the coefficients estimated separately at each cut-point (as described in the paragraph above) and produces a p-value testing whether those separate estimates are statistically consistent with coming from one shared, constant effect. A small p-value here is evidence the proportional-odds assumption is shaky for at least one predictor, and you should consider the alternatives mentioned above (a partial proportional-odds model, or reporting cut-point-specific effects) rather than trusting a single pooled odds ratio.

16.5 Machine learning for prediction and heterogeneity of treatment effect

Chapter 15 built the propensity-score and doubly-robust toolkit for estimating an **average** treatment effect while adjusting for confounding. Two questions that toolkit does not answer on its own: does the treatment effect vary across patients, and which of many candidate covariates actually matter? This section introduces the machine-learning answers to both, kept deliberately narrow — one method per question, one worked example each.

Random forests for prediction. `randomForest::randomForest()` and the faster, more memory-efficient `ranger::ranger()` both fit ensembles of decision trees for an outcome — useful when you want a flexible prediction model (e.g. predicting 6-month treatment-failure risk from baseline labs) without committing to a linear or logistic functional form. Neither function, used this way, estimates a **treatment effect** — they predict an outcome.

Causal forests for treatment-effect heterogeneity. To ask "does the treatment effect itself vary by patient," the `grf` package's `grf::causal_forest()` function grows a forest tuned specifically to estimate individual-level treatment effects, not outcomes. It takes a covariate matrix `X`, an outcome vector `Y`, and a treatment indicator `W`, and returns per-observation treatment-effect estimates via `predict()`. This is the ML-flexible cousin of Chapter 15's doubly-robust estimators: where a DR estimator assumes a parametric (or semiparametric) form for the propensity and outcome models, `causal_forest()` lets the data determine, nonparametrically, **where** treatment effects are larger

or smaller — directly surfacing heterogeneity of treatment effect (HTE) that a single DR average effect would hide.

```
library(grf)

# X: baseline covariates, W: treatment (0/1), Y: outcome (e.g. treatment success)
cf <- causal_forest(X = as.matrix(tb_data[, c("age", "bmi",
"baseline_chest_xray_score")]),
                    Y = tb_data$treatment_success,
                    W = tb_data$received_new_regimen)

# Per-patient estimated treatment effect
tau_hat <- predict(cf)$predictions
summary(tau_hat)
```

A rising spread in `tau_hat` across patients is evidence of HTE; `grf` also provides `best_linear_projection()` and related diagnostics to summarise **which** covariates drive it — useful for generating hypotheses that a pre-specified subgroup analysis (§16.6) can then test formally.

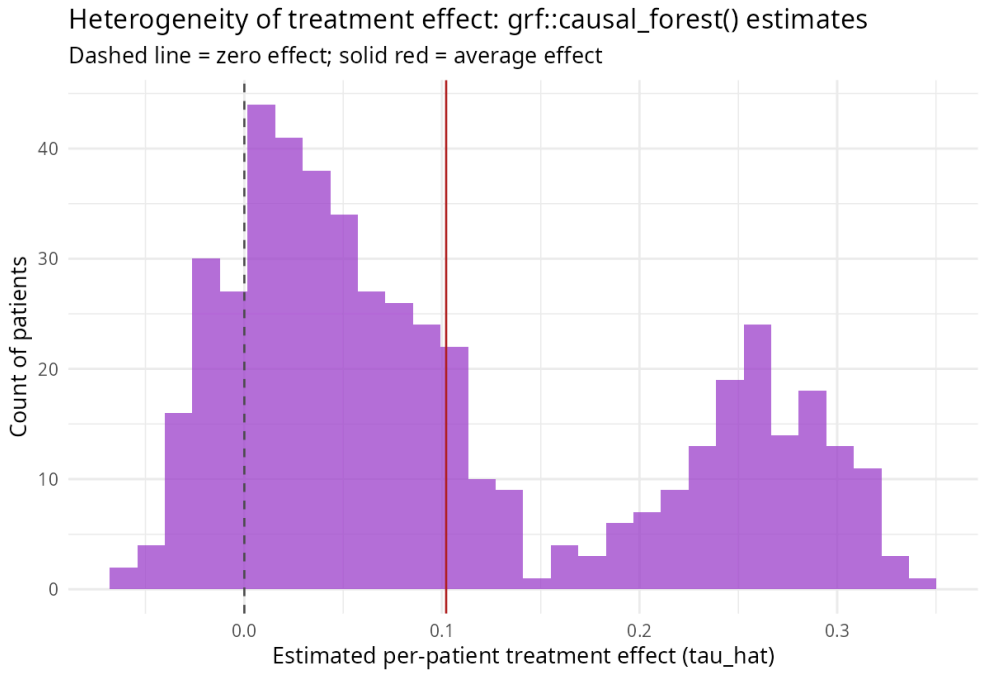


Figure 29: Distribution of per-patient estimated treatment effects (`tau_hat`) from `grf::causal_forest()` on a simulated TB-regimen dataset. The bimodal spread — one cluster near zero, another around 0.25-0.3 — is itself evidence of real heterogeneity: a single average effect (solid red line) would mask the fact that some patients benefit far more than others.

LASSO and elastic net for variable selection. When the candidate covariate list is long relative to sample size — dozens of biomarkers, comorbidity flags, or lab values — ordinary regression overfits. `glmnet::glmnet(x, y, alpha = , lambda = )` fits penalised regression: `alpha = 1` is the LASSO (shrinks many coefficients exactly to zero, giving automatic variable selection), `alpha = 0` is ridge regression, and values in between are the elastic net. `x` must be a numeric matrix (not a formula/data-frame), and `lambda` controls penalty strength — typically chosen by cross-validation with `cv.glmnet()` rather than set by hand.

```r
library(glmnet)

x <- as.matrix(tb_data[, biomarker_cols])   # many candidate biomarkers
y <- tb_data$time_to_culture_conversion

fit <- cv.glmnet(x, y, alpha = 1)           # LASSO, lambda chosen by CV
coef(fit, s = "lambda.min")                  # non-zero coefficients = selected
biomarkers
```

Use this as a screening step — to narrow forty biomarkers to five worth carrying into a clinically-interpretable model — not as a replacement for domain knowledge about which covariates are prognostic versus effect-modifying (§16.6).

16.6 Subgroup analysis and effect modification

Subgroup analysis asks a simple question with a dangerous failure mode: does the treatment effect differ between, say, HIV-co-infected and HIV-negative TB patients? The question is legitimate. The danger is in **how** it gets asked.

Pre-specified versus exploratory. A pre-specified subgroup analysis is written into the statistical analysis plan before the data are unblinded, naming the specific subgroups and the specific interaction test, ideally with a stated biological or clinical rationale (ICH E9(R1), already introduced in Chapter 7′s estimand framework, requires this kind of up-front specification for any subgroup finding intended to support a treatment claim). An exploratory or post-hoc subgroup analysis slices the data after the fact — by age, sex, region, baseline severity, and whatever else is in the dataset — looking for "the subgroup where it worked." Both have a place in a report, but only the first supports a confirmatory claim.

> **Common pitfall.** Testing treatment effects across many unplanned subgroups inflates the false-positive rate the same way testing many outcomes does. Slice a trial twenty different ways and, by chance alone, one or two subgroups will show a "significant" interaction even when the true effect is uniform. Label exploratory subgroup findings as hypothesis-generating, not confirmatory, and do not let a favourable subgroup result rescue an otherwise null overall result.

Testing effect modification. The formal test for whether a subgroup modifies the treatment effect is an interaction term in a regression model — not a pair of separate within-subgroup analyses. For a binary outcome:

```r
fit <- glm(treatment_success ~ treatment * hiv_status + age + baseline_severity,
           data = tb_data, family = binomial())
summary(fit)   # coefficient on treatment:hiv_status is the test of effect
modification
```

A significant `treatment:hiv_status` coefficient says the treatment effect on the odds scale differs by HIV status; a non-significant one is evidence **against** effect modification on that scale, though often underpowered to detect it — subgroup interaction tests typically need several times the sample size of the main-effect test.

Displaying subgroup effects. Once effect estimates are obtained within each subgroup, the forest plot is the standard display — and the mechanics are exactly the ones built in Chapter 11 for

meta-analysis: either a `ggplot2` point-and-error-bar plot, or `metafor::forest()` applied to a set of subgroup-specific effect estimates and standard errors in place of study-specific ones. No new plotting machinery is needed here; only the question "study" versus "subgroup" changes.

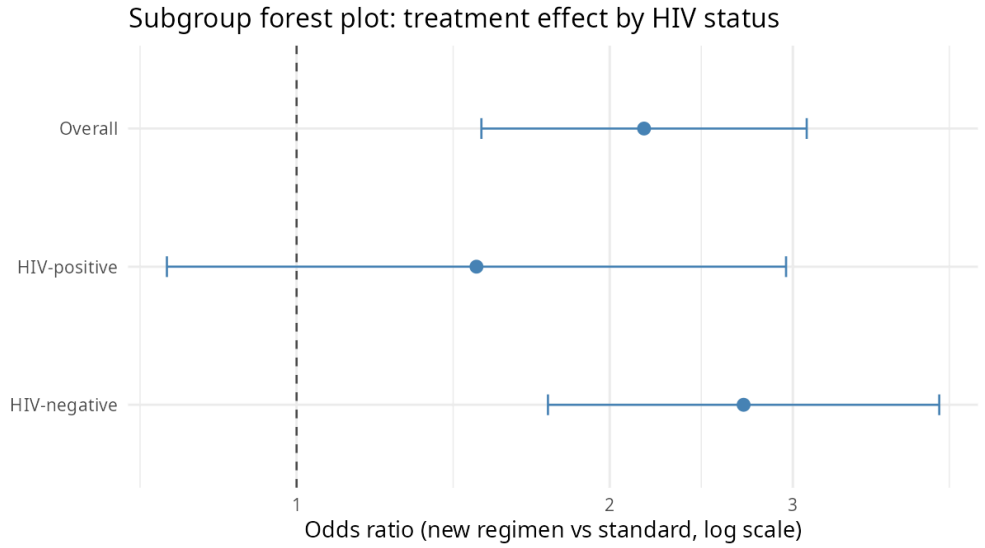


Figure 30: A subgroup forest plot (ggplot2 point-and-error-bar, log odds-ratio scale) comparing the overall treatment effect against the HIV-positive and HIV-negative subgroups. The overlapping confidence intervals here would not meet a pre-specified threshold for a confirmed interaction — illustrating why the formal interaction test, not a pair of separate within-subgroup p-values, is the correct way to test effect modification.

Prognostic factor versus effect modifier, again. Chapter 15 §15.3 drew this distinction using a DAG: a prognostic factor predicts outcome regardless of treatment, while an effect modifier changes the **size** of the treatment effect itself, and only the latter justifies treatment decisions that vary by subgroup. That distinction governs subgroup analysis directly — a subgroup defined by a strong prognostic factor (e.g. baseline disease severity) may show different **absolute** risks in each arm without the treatment's relative effect differing at all. Always check the interaction term before concluding that a subgroup's apparently different results reflect true effect modification rather than prognosis.

Difficult terminology: DAG (directed acyclic graph). A DAG is a diagram — boxes for variables, arrows for assumed cause-and-effect — that makes your causal assumptions explicit and checkable rather than leaving them implicit in your head. "Directed" means every arrow has a direction (cause → effect, not just "these two are associated"); "acyclic" means you can't follow the arrows in a loop back to where you started (no variable can be its own cause, directly or indirectly). For example, a simple DAG might show "disease severity → treatment choice" and "disease severity → outcome" and "treatment choice → outcome": severity is a **confounder** here because it feeds into both the thing you're choosing (treatment) and the thing you're measuring (outcome), and you'd draw that structure before touching any data, based on clinical knowledge. Epidemiologists and causal-inference statisticians use DAGs precisely because they force you to commit, on paper, to which variables you believe are confounders (common causes of both exposure and outcome, which you must adjust for) versus which are simply downstream consequences of treatment (which you must **not** adjust for,

since doing so can introduce bias rather than remove it). You don't need to draw formal DAGs to use this book's methods, but understanding that a DAG is simply "your causal assumptions, drawn out and made explicit" demystifies why Chapter 15 leans on them so heavily.

Agency perspective. EMA's "Guideline on the investigation of subgroups in confirmatory clinical trials" (EMA/CHMP/539146/2013) sets out regulatory expectations for exactly this distinction: pre-specification, adequate power for interaction tests, and caution against over-interpreting post-hoc subgroup findings — the same principles argued for above, from the regulator's side of the table.

Chapter 16 has moved from the single regression workhorses — linear, logistic, Poisson/negative binomial, survival — through their ML-flexible and heterogeneity-aware extensions. Part III's statistical toolkit is now in place: estimating effects, adjusting for confounding, pooling across studies, and characterising who responds differently. Chapter 17 opens Part IV, Decision-Analytic Modelling, and shifts the question from "what effect did we estimate" to "what should a health system do about it" — starting with how to conceptualise a decision model, choose a model type, and document its structure before a single cost or probability is ever entered.

PART IV

Decision-Analytic Modelling

Chapter 17. Principles of Decision Modelling

Part III closed the estimation toolkit: effects, confounding adjustment, pooling, heterogeneity. Those chapters answered "what does the evidence say happened, or what would happen under a given treatment?" Chapter 17 asks a different question: given that evidence, what should a health system **do**? Decision-analytic modelling is the machinery for turning scattered estimates of effect, cost, and quality of life into a single comparative answer – which TB treatment regimen should a programme adopt, and under what uncertainty?

This chapter is a map, not a computation. It walks through the decisions a modeller makes before writing a line of model code: what the decision problem actually is (17.1), which model type fits that problem (17.2), how to draw and defend the structure (17.3), what kinds of parameters feed it (17.4), how to document choices so others can audit them (17.5), how an R project should be organised to keep inputs, engine, and outputs cleanly separated (17.6), and which professional standards govern all of this (17.7). Chapters 18 through 22 then build the actual engines – decision trees, Markov models, partitioned survival, and beyond.

Learning objectives

- Scope a decision problem in HEOR terms: perspective, comparators, time horizon, and the question a model is meant to answer (17.1).
- Choose between decision trees, Markov models, partitioned survival, and other model types based on how a disease unfolds over time (17.2).
- Draw a model structure as a diagram in R using the DiagrammeR package, and justify structural assumptions explicitly (17.3).
- Classify model parameters by type – probabilities, costs, utilities, relative effects – and match each to an appropriate distribution for later uncertainty analysis (17.4).
- Write a model documentation record that makes every structural and parameter choice traceable and auditable (17.5).
- Organise an R decision-model project into separate inputs, engine, results, and reporting layers (17.6).
- Locate a decision model's obligations within the ISPOR-SMDM Modeling Good Research Practices series and the NICE reference case (17.7).

17.1 Model conceptualisation and problem scoping

Before any R code is written, a decision model needs a clear statement of the question it is answering. This sounds obvious, but a surprising share of modelling projects go wrong here: the population drifts mid-project, the comparator list grows without anyone noticing, or the time horizon gets chosen by habit rather than by the disease biology. Chapter 4, section 4.6 already introduced the core framing vocabulary for an economic evaluation – population, comparators, perspective, and time horizon. This chapter reuses those four terms exactly as defined there; we do not re-derive them.

HEOR in practice. Before opening RStudio, write down, in one sentence each: the population, the comparators, the perspective, and the time horizon for your model. If you cannot write all four sentences, you are not ready to model yet – you are still scoping.

The PICOTS framework. HEOR borrows PICO from clinical epidemiology and extends it for economic evaluation: Population, Intervention, Comparator(s), Outcomes, Timing, Setting. PICOTS is simply a checklist that forces the same four ideas from 4.6 into a slightly finer grain – splitting "comparators" into intervention vs. comparator, and adding outcomes, timing, and setting as explicit fields.

Take a TB programme decision as the running example for this chapter: should a national TB programme switch standard drug-susceptible TB patients from the 6-month daily regimen to a 4-month rifapentine-moxifloxacin regimen (as recommended following the Study 31/A5349 trial)? The table below sketches the PICOTS for this question.

PICOTS element	TB regimen example
Population	Adults with newly diagnosed, drug-susceptible pulmonary TB
Intervention	4-month rifapentine-moxifloxacin regimen
Comparator	6-month standard daily regimen (2HRZE/4HR)
Outcomes	Treatment success, relapse, cost per patient treated, QALYs
Timing	Follow-up to 12 months post-treatment-completion, to capture relapse
Setting	Public TB programme clinics, national perspective

Notice that "Timing" here is not the same as the model's time horizon. Timing in PICOTS describes the follow-up window of the **evidence** informing the model; the time horizon (4.6) is a separate modelling choice about how far forward the **model** itself projects, which may need to extend well beyond 12 months to capture downstream relapse and retreatment costs.

Common pitfall. A common scoping error is letting the comparator list expand once the project has started – for example adding a shorter experimental regimen under trial, "just in case it's useful later." Every added comparator multiplies the data you need (efficacy, costs, utilities) and the validation burden. Lock the comparator list during scoping, and treat any addition as a formal scope change, not a quick extra column.

The model concept document. Once PICOTS is settled, write it up as a short model concept document – typically one to two pages – before touching R. This document states the decision problem, the PICOTS elements, the intended model type (chosen in 17.2), the main data sources you expect to use, and who the model is for (a funding ministry, a programme manager, a national guideline committee). Its purpose is to let clinical and economic stakeholders sign off on the **question** before anyone invests time building the **answer**. For our TB example, the concept document would note that the programme's budget cycle and WHO guideline review calendar may constrain the time horizon or the urgency of results – decisions that are much cheaper to revisit on paper than inside a half-built R script.

17.2 Choosing a model type

Part IV covers five model types, one per chapter: decision trees (chapter 18), Markov cohort models (chapter 19), partitioned survival models (chapter 20), individual-level microsimulation (chapter 21), and discrete event simulation (chapter 22). This section does not explain how any of them work – that is the job of chapters 18 through 22. Instead it gives you a framework for choosing **which** one fits a given decision problem, so that when you reach each later chapter you already know why it is the right tool for that chapter's example.

The ISPOR-SMDM Modeling Good Research Practices Task Force produced a series of papers exactly on this question. The task force report most relevant to choosing among state-transition approaches is Siebert et al. (2012), "State-Transition Modeling: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force–3." It is worth reading once you reach chapter 19, but its selection logic is useful right now.

Five questions that narrow the choice. Rather than memorising five model definitions, ask five questions about the disease and decision problem:

1. **Is this a single, one-off decision with a short time horizon and no need to track disease progression over time?** If yes, a decision tree (ch. 18) is usually sufficient – and simpler is better when it is adequate.
2. **Does the disease move between a small number of discrete, recurring health states over time (e.g., "on treatment," "relapsed," "cured")?** This favours a Markov cohort model (ch. 19), the workhorse of chronic-disease HEOR.
3. **Is the treatment effect expressed mainly as survival curves or time-to-event outcomes, without well-defined intermediate health states to model explicitly?** This favours a partitioned survival model (ch. 20) – common in oncology, where trial data arrive as progression-free and overall survival curves rather than discrete state transitions.
4. **Does a patient's future risk depend on their individual history – how long they have already been in a state, prior treatment lines, accumulated events – in a way a cohort-level model cannot capture?** This favours individual-level (microsimulation) modelling (ch. 21).
5. **Do shared, limited resources, queues, or interactions between patients matter to the outcome** – for example, a diagnostic machine with finite daily capacity, or transmission between patients? This favours discrete event simulation (ch. 22), the only model type in Part IV that routinely models resource constraints explicitly.

These five questions are not mutually exclusive filters applied in strict order; they are prompts. Most real problems answer "yes" to more than one, and the final choice is a judgement about which structural feature matters **most** for the decision at hand, balanced against data availability and stakeholder familiarity with the method.

> **HEOR in practice.** Apply the five questions to the TB regimen example from 17.1. TB treatment has discrete, clinically meaningful recurring states – on treatment, cured, failed, relapsed, retreatment, death – and patients can legitimately cycle between some of them (failure leads back to retreatment). There is no resource queue central to the question, and there is no strong need for patient-level memory beyond state occupancy. Question 2 dominates: a Markov cohort model (ch. 19) is the natural choice, and that is exactly the model type chapter 19 builds for this same example.

Common pitfall. Do not choose a model type because it is the one you already know how to code, or because a past project used it. The Siebert et al. task force report exists precisely because reviewers and HTA bodies expect the model type to follow from the disease's structural features – not from analyst convenience. A mismatch (for example, forcing a partitioned survival model onto a disease with genuinely cyclical relapse-and-retreatment states) can bias results and will draw scrutiny in review.

17.3 Structural assumptions and model diagrams

Once a model type is chosen, the next step – still before writing simulation code – is to decide and document exactly how patients move through the model's health states. These choices are called structural assumptions, and three questions come up in almost every state-transition model:

- **Can patients move backward between states?** For example, can a "cured" TB patient return to "on treatment" (via relapse and retreatment), or is cure a one-way gate?
- **Is there a tunnel state** – a state a patient can only occupy for a fixed number of cycles before being forced onward, used to represent things like a fixed treatment course length?
- **What happens at the end of the model's time horizon?** Do remaining patients in non-death states simply stop accruing costs and effects (a "closed cohort" assumption), or does the model apply some extrapolation beyond the explicit horizon?

Writing these answers down as explicit prose statements, before drawing anything, is what prevents disagreements from surfacing only after the R code is built and the numbers look odd.

Drawing the structure. A model diagram turns those prose statements into a picture that clinical collaborators can sanity-check at a glance, without reading any R code. The `DiagrammeR` package's `grViz()` function renders diagrams written in the DOT graph description language (the same language used by Graphviz), which makes it well suited to drawing health-state diagrams: states are nodes, transitions are directed edges.

The code below draws the structural diagram for our TB Markov model: patients on treatment can be cured or can fail treatment; failure leads back to re-registration for on-treatment care; and death is reachable as an absorbing state from every other state.

```
library(DiagrammeR)

tb_model_diagram <- grViz("
digraph tb_markov_structure {

  graph [layout = dot, rankdir = LR]
  node [shape = box, style = filled, fontname = Helvetica]

  OnTx    [label = 'On Treatment',      fillcolor = lightblue]
  Cured   [label = 'Cured',             fillcolor = lightgreen]
  Fail    [label = 'Treatment Failure', fillcolor = lightsalmon]
  Death   [label = 'Death',             fillcolor = grey85]

  OnTx  -> Cured  [label = 'treatment success']
  OnTx  -> Fail   [label = 'treatment failure']
  Fail  -> OnTx   [label = 're-registration']
```

```
  OnTx  -> Death  [label = 'mortality']
  Cured -> Death  [label = 'mortality']
  Fail  -> Death  [label = 'mortality']
}
")

tb_model_diagram
```

Running this in RStudio (or a Quarto/R Markdown chunk) renders an interactive diagram directly in the viewer pane. The `rankdir = LR` argument lays the states left to right, which tends to read more naturally for treatment-pathway diagrams than the default top-to-bottom layout. Each `label` on an edge documents **why** the transition exists – a habit worth keeping, because it turns the diagram itself into a piece of the structural-assumptions documentation rather than just decoration.

Note what the diagram already answers from our three questions above: backward movement exists (`Fail -> OnTx`), there is no tunnel state in this simplified version, and death is absorbing and reachable from everywhere – all visible without reading a single line of transition-probability code.

> **Agency perspective.** NICE's reference case for technology appraisals, together with NICE's Decision Support Unit technical support document series, expects submissions to include a clearly documented and justified model structure – typically as a diagram of exactly this kind, accompanied by prose justifying each state and transition. Producing the diagram at the structural-assumptions stage, rather than retrofitting one after the model is built, makes this requirement far easier to satisfy.

> **Common pitfall.** A model diagram is not optional polish added for a report's appendix. If you cannot draw your model's states and transitions in a few minutes using `grViz()`, you likely do not yet have a clear enough structural assumption set to start building the model in R – go back to the three questions above before writing any transition-probability code.

17.4 Parameter taxonomy and distributions

Every decision-analytic model is built from four kinds of parameters, and getting the taxonomy straight matters because each kind draws its evidence from a different place in this book and takes a different distribution in probabilistic sensitivity analysis (PSA).

Transition probabilities move patients between health states each cycle: cured, on treatment, lost to follow-up, dead. **Costs** attach resource use in each state to a price. **Utilities** convert time in a state into quality-adjusted life years (QALYs). **Relative treatment effects** (hazard ratios, risk ratios, odds ratios) scale one arm's parameters relative to a comparator.

Costs and utilities are not estimated inside the decision model – they are imported. Chapter 9 covered fitting GLMs to costing data and the gamma/lognormal distributions that describe the resulting uncertainty; Chapter 8 covered sourcing utility weights (EQ-5D mapping, vignette studies, literature tariffs) and their beta-like uncertainty. Chapter 17 does not re-derive any of that – it just slots the fitted mean and standard error into the model's parameter table. Likewise, relative

treatment effects come from the pooled estimates in Chapters 11 (pairwise meta-analysis) and 12 (network meta-analysis): the pooled log hazard ratio and its standard error from a random-effects model there become the treatment-effect parameter here.

What **is** new in a decision model is the transition probability itself, because trials rarely report it directly – you typically reconstruct it from a reported proportion with a confidence interval, and that calls for a distribution that respects the 0-1 bound.

> **HEOR in practice. Why bounded distributions, not normal.** A transition probability must stay between 0 and 1 in every PSA draw. A normal distribution can return a draw below 0 or above 1, which produces a model error or a silently nonsensical result. The beta distribution is defined on [0, 1] and is the standard choice for single probabilities; the Dirichlet distribution is its multivariate generalisation, used when several destination probabilities from one state (e.g. cured / still on treatment / dead) must each stay in [0, 1] **and** sum to exactly 1 every draw.

The conventional distributional choices, by parameter type, are:

- **Probabilities and proportions** – Beta, because it is bounded on [0, 1] and matches the binomial likelihood that usually generated the estimate.
- **Multinomial transition probabilities** (more than two destinations from one state) – Dirichlet, so the simulated probabilities always sum to 1.
- **Costs** – Gamma or lognormal, because cost data are right-skewed and bounded at 0 (see Chapter 9 for the GLM machinery that produces the mean and variance going into these distributions).
- **Utilities** – Beta when the utility itself is bounded in [0, 1] (most adult EQ-5D-3L tariffs), or Dirichlet when utility decrements are modelled as a set of mutually exclusive health-state weights that must sum to 1.
- **Relative treatment effects** – usually normal on the log scale (log hazard ratio, log odds ratio), because that is the scale on which the meta-analytic standard error from Chapters 11-12 was estimated.

Fitting a beta distribution to a transition probability is pure algebra: the method-of-moments equations convert a mean and variance into the two beta shape parameters, alpha and beta.

$$\alpha = \mu \cdot \left(\frac{\mu(1-\mu)}{\sigma^2} - 1 \right), \quad \beta = (1-\mu) \cdot \left(\frac{\mu(1-\mu)}{\sigma^2} - 1 \right)$$

Suppose a trial reports a 12-month cure probability of 0.85 with a 95% CI of (0.80, 0.90). Treating the CI as approximately normal on the probability scale, the standard error is the half-width divided by 1.96, and the variance is its square.

```
# Method-of-moments beta fit from a trial's reported mean and 95% CI
mean_p <- 0.85
ci_lower <- 0.80
ci_upper <- 0.90

se_p <- (ci_upper - ci_lower) / (2 * 1.96)
var_p <- se_p^2

common_term <- (mean_p * (1 - mean_p) / var_p) - 1
alpha_p <- mean_p * common_term
beta_p  <- (1 - mean_p) * common_term
```

```
alpha_p  # 165.68
beta_p   #  29.24

# Check: does Beta(alpha_p, beta_p) reproduce the mean and SE we started with?
fitted_mean <- alpha_p / (alpha_p + beta_p)
fitted_var  <- (alpha_p * beta_p) / ((alpha_p + beta_p)^2 * (alpha_p + beta_p +
1))
fitted_mean          # 0.85 -- matches
sqrt(fitted_var)     # 0.0255 -- matches se_p
```

The fitted shape parameters are alpha approx 165.68 and beta approx 29.24, and the check confirms the round trip: a `Beta(165.68, 29.24)` distribution has mean 0.85 and standard error 0.0255, exactly what the trial reported. In Chapter 25 this `Beta(alpha_p, beta_p)` object is what gets sampled thousands of times inside the PSA loop; this chapter's job stops at specifying the distribution correctly.

Common pitfall. Large alpha and beta values (as above) mean the beta distribution is tightly concentrated around the mean – appropriate for a well-powered trial. A small trial arm (say n = 20) produces much smaller shape parameters and a visibly wider, sometimes skewed distribution. Always sanity-check the fitted alpha and beta against the sample size implied by the reported CI; a mismatch usually means the CI was log-scale or Wilson-interval, not the simple normal approximation used above.

Difficult terminology: Wilson interval (Wilson score interval). When you calculate a confidence interval around a proportion (like "85% of patients responded"), the simplest textbook method — the one most clinicians meet first — assumes the sampling distribution is roughly a symmetric bell curve and builds the interval as "estimate plus or minus some multiple of the standard error." That simple normal-approximation method breaks down badly when the proportion is close to 0% or 100%, or when the sample size is small: it can even produce a nonsensical interval that dips below 0% or climbs above 100%. The Wilson interval (also called the Wilson score interval) is an alternative formula that handles exactly these edge cases properly — it is derived slightly differently (by inverting a hypothesis-testing statistic rather than assuming symmetry) and stays within the valid 0-100% range, and gives more trustworthy coverage for small or extreme-proportion trial arms. As a working rule: if a reported trial result's confidence interval looks asymmetric around the point estimate, or the proportion is near 0 or 1, suspect it was built with a Wilson-type method rather than the simple symmetric approximation, and account for that when back-fitting a Beta distribution as in the worked example above.

17.5 Model documentation and transparency

A decision model that only its author can run is not fit for HTA submission, however correct its arithmetic. Two linked expectations govern this: external bodies expect the model's **logic** to be auditable, and they expect the model's **code** to actually execute on someone else's machine.

ISPOR-SMDM's Good Research Practices Task Force addressed the first expectation directly. Eddy et al. (2012), "Model Transparency and Validation" – one paper in the seven-part task force series on good modelling practice – sets out what a model needs to disclose so that a third party can understand what it does without re-running it: the health states and transitions, every parameter's source and value, every structural assumption, and the validation checks performed against external data. Transparency here does not mean publishing source code; it means a reviewer can reconstruct the model's logic from the written description alone.

NICE's methods guide goes further for UK submissions: it expects an **executable** R (or equivalent) model, fully commented, that NICE's own technical team can run and interrogate. Chapter 2 covered the `renv` project-level dependency locking that makes a submitted model reproducible on a reviewer's machine years later; Chapter 28 will cover the full reporting and submission-package treatment. The point to take from this section is narrower: documentation and code are not substitutes for each other. A commented script explains **how** the model computes; a technical model documentation (TMD) report explains **why** it is built the way it is, for a reader who may never open the script at all.

> **Agency perspective.** NICE technical support documents and the associated reference case checklist specify that a submitted economic model must run without error in a clean environment, with all input parameters either hard-coded with cited sources or loaded from a clearly labelled data file – no undocumented manual steps. This is precisely the reproducibility bar Chapter 2's `renv` workflow targets.

In practice, agencies expect a TMD report structured around five sections, matching the Eddy et al. transparency checklist:

1. **Inputs** – every parameter, its point estimate, its source citation, and its distribution for PSA (the taxonomy from 17.4 feeds directly into this section).
2. **Structure** – the states, cycle length, time horizon, and transitions, with a rationale for each structural choice (Section 17.1-17.3 territory).
3. **Assumptions** – every simplification made and why it was judged acceptable (e.g. no second-line retreatment arm, constant background mortality).
4. **Validation** – face validity checks, comparison against published cohort outcomes, and any cross-validation against an independent model.
5. **Outputs** – the base-case results, and a signpost to where the uncertainty analysis (Chapter 25) and scenario analyses are reported.

> **HEOR in practice.** Write the TMD's "Inputs" section as a literal table with four columns – parameter, value, source, distribution – rather than prose. Reviewers compare this table line-by-line against the R model's parameter list; prose descriptions slow that check down and invite transcription errors between report and code.

The TMD and the executable model are two views of the same object. Treat the TMD as the map and the R script as the territory: if they disagree, the model is wrong even when the code runs without error.

17.6 Modelling workflow in R: inputs -> engine -> outputs -> reporting

A decision model is a pipeline, not a script. Treating it as one script – parameters hard-coded inline, calculations and formatting tangled together – is the single most common cause of models that cannot be audited, extended, or debugged six months later. The fix is architectural, not statistical: separate the model into four layers that each do one job and talk to the next through a clean interface.

The four layers. An *inputs layer* defines every parameter the model needs, as a named list or a tidy tibble, with nothing else happening in that file. An *engine layer* is a function that takes the parameter object and returns raw model output – total cost and total QALYs per strategy – with no knowledge of how those numbers will later be displayed. A *results layer* takes engine output and computes the comparative statistics analysts actually want: incremental cost-effectiveness ratios (ICERs) and net monetary benefit (NMB). Section 4.4 already derived both formulas in full; this layer simply applies them to whatever the engine produced, so nothing here is re-derived. A *reporting layer* turns results into tables, plots, and narrative text. Chapter 28 (Reproducible Reporting) covers this layer properly; for now, treat it as "whatever renders the results object into something a decision-maker reads."

The payoff of this separation is that each layer can be tested, swapped, or re-run independently. Changing a discount rate means editing the inputs file. Changing from a decision tree to a Markov model means replacing the engine function – the results and reporting layers do not change at all, because they only ever see cost and QALY totals per strategy, never the mechanics that produced them.

A worked skeleton. The engine function signature below is deliberately generic. It is a contract, not a working model: given a parameter list, return a tibble with one row per strategy. What happens **inside** the function – branch probabilities for a decision tree, transition matrices for a Markov model, survival curves for partitioned survival – is exactly the subject of Chapters 18 through 22. Nothing about that mechanics belongs in this chapter.

```
# --- inputs layer -----------------------------------------------------
params <- list(
  strategies = c("Regimen_A", "Regimen_B"),
  cost_drug  = c(Regimen_A = 450, Regimen_B = 980),   # per course, local currency
  p_cure     = c(Regimen_A = 0.86, Regimen_B = 0.91),
  qaly_cured = 1.0,
  qaly_fail  = 0.6
)

# --- engine layer -----------------------------------------------------
# Generic signature only -- the real decision-tree / Markov mechanics
# live in Chapters 18-22. Here the engine is a stand-in.
run_model <- function(params) {
  tibble::tibble(
    strategy    = params$strategies,
    total_cost  = params$cost_drug[params$strategies],
    total_qaly  = params$p_cure[params$strategies] * params$qaly_cured +
                  (1 - params$p_cure[params$strategies]) * params$qaly_fail
  )
}
```

```r
engine_out <- run_model(params)

# --- results layer ------------------------------------------------------
# ICER and NMB formulas: see Section 4.4. Not re-derived here.
results <- engine_out |>
  dplyr::arrange(total_cost) |>
  dplyr::mutate(
    inc_cost  = total_cost - dplyr::first(total_cost),
    inc_qaly  = total_qaly - dplyr::first(total_qaly),
    icer      = inc_cost / inc_qaly,
    nmb       = total_qaly * 20000 - total_cost   # threshold = 20,000/QALY
  )

# --- reporting layer ----------------------------------------------------
# Tables, plots, narrative -- see Chapter 28 (Reproducible Reporting).
```

> **Agency perspective.** This skeleton is intentionally thin. `run_model()` here does almost nothing, because its real job – simulating disease progression over time – is Chapters 18 to 22's content. What matters in this chapter is the **shape** of the pipeline, not the arithmetic inside any one box.

Two packages that already do this. You do not have to build this architecture from scratch every time. *heemod* is a mature CRAN package that implements exactly this inputs/engine/output separation for Markov cohort models, with its own functions for defining states, transitions, and strategies; Chapter 19 covers it properly. *hesim* does something similar across a broader range of model types, including partitioned survival models; it appears again in Chapters 19 and 20. Neither package is used here beyond naming them – inventing their function calls prematurely would teach the wrong syntax before the right chapter explains it.

> **Common pitfall.** Mixing layers is the fastest way to make a model impossible to debug. If a cost figure looks wrong, you want to be able to check the inputs file in isolation, then the engine's arithmetic, then the results formulas – not wade through three hundred lines where all of that is interleaved with `ggplot2` calls.

17.7 Good practice guidelines (ISPOR-SMDM, NICE reference case)

Everything in this chapter – scoping, model choice, structure, parameters, documentation, workflow – sits inside two governing frameworks that the rest of Part IV assumes without restating. Naming them once, here, is enough.

ISPOR-SMDM Modeling Good Research Practices. A joint ISPOR-SMDM task force, co-chaired by Eddy, Caro, Briggs, Pitman, Sculpher and others, published a seven-paper series in **Value in Health** in 2012 setting out good practice for health-economic modelling end to end. Caro et al. (2012), "Modeling Good Research Practices – Overview," is the umbrella paper that frames the other six. Section 17.2 already cited Siebert et al. (2012) on state-transition (Markov) models specifically, and Section 17.5 already cited Eddy et al. (2012) on model transparency and

validation; both are part of this same series, not separate frameworks. The series as a whole is the methodological backbone for Chapters 18 to 22: whichever model type you build, its task-force paper is the reference for what "good" looks like.

The NICE reference case. Where ISPOR-SMDM sets general modelling practice, the NICE reference case sets a specific national standard: a defined set of methodological choices that HTA submissions to England's National Institute for Health and Care Excellence must follow unless a deviation is explicitly justified. Its core requirements include an NHS and Personal Social Services (PSS) perspective, QALYs as the outcome measure, a specified discount rate for costs and health effects, and a lifetime (or sufficiently long) time horizon. Chapter 4 introduced the reference-case concept when it first discussed perspective and outcome choice; this is the same reference case, now tied to the specific body that issues it.

> **HEOR in practice.** Think of ISPOR-SMDM as answering "is this model built well?" and the NICE reference case as answering "does this model meet **this** payer's specific rules?" A model can satisfy the first and still fail the second – for example, by using a societal perspective where NICE requires an NHS/PSS perspective. Both checks matter, and they are not the same check.

Part IV does not re-litigate either framework chapter by chapter. Where a later chapter makes a choice – a discount rate in Chapter 22, a validation step in a later audit chapter – it will point back here rather than re-arguing the justification.

With the conceptual scaffolding in place, Chapter 18 starts building. Decision trees are the simplest model structure in the ISPOR-SMDM taxonomy from Section 17.2, and the next chapter constructs one from scratch in R: branches, probabilities, expected-value calculations, and the first full `run_model()` engine with real mechanics inside it.

Chapter 18. Decision Trees

Chapter 17 was a map: how to scope a decision problem, choose a model type, and organise the R project around it. This chapter builds the first real engine on that map – the decision tree, the simplest member of the decision-analytic modelling family and the right tool exactly when §17.2′s "one-off/short-horizon" criterion is met: a question that resolves within a single episode of care, with no need to track a cohort through recurring health states over years.

The running example for this chapter is deliberately **different** from Chapter 17′s regimen-choice question (which committed to a Markov cohort model, built in Chapter 19). Instead, §18.1 through §18.5 follow a short-term TB **diagnostic-strategy** decision: a district programme choosing between a cheaper sputum-smear-only algorithm and a Xpert MTB/RIF molecular-test algorithm for patients presenting with TB symptoms. This is squarely tree territory – test, get a result, act on it, done – and §18.5 closes the loop by showing exactly how a tree like this one hands its output to a longer-term Markov model when a diagnosed patient's story continues for years, setting up Chapter 19 directly.

Learning objectives

- Describe a decision tree's three node types – decision, chance, terminal – and compute expected cost and expected QALYs by hand via the roll-back algorithm (18.1).
- Represent a decision tree as a tidy R data structure and compute its expected values with vectorised arithmetic, reproducing a hand calculation exactly (18.2).
- Build the same tree using the `rdecision` package's node/edge classes, and explain what a dedicated package buys over a from-scratch calculation (18.3).
- Recognise the general class of short-horizon problems – screening, diagnostics, acute care – that decision trees suit, beyond the chapter's TB example (18.4).
- Explain how a decision tree's terminal-node probabilities become the starting cohort distribution for a downstream Markov model (18.5).

18.1 Structure and expected-value calculations

A decision tree is the short-horizon cousin of the Markov model you built in Chapter 17: instead of cycling through states over years, it plays out a single episode of care as a branching sequence of chance events, then stops. Every tree has three kinds of nodes. A **decision node** (drawn as a square) is the strategy choice itself – the thing you, the analyst, are being asked to recommend. A **chance node** (a circle) is a point where nature decides the outcome, with branch probabilities that must sum to 1 across all branches leaving that node. A **terminal node** (a triangle, or just the end of a branch) carries the **payoffs** – here, a cost and a QALY value – accumulated by the time that path is reached. Trees are the right tool exactly when §17.2′s criteria point to a short, non-recurring decision problem with no need to track disease progression over multiple cycles – which is why this chapter switches to a different slice of the TB pathway than Chapter 17′s regimen choice.

Worked example – diagnostic strategy choice. A district TB programme is choosing between two algorithms for patients presenting with TB symptoms: a cheaper **sputum-smear-only** algorithm and a **Xpert MTB/RIF** molecular-test algorithm. Among symptomatic presenters, TB

prevalence is $p = 0.30$. Smear microscopy has sensitivity 0.60 and specificity 0.98; Xpert has sensitivity 0.90 and specificity 0.96. Each strategy therefore branches first on true disease status (TB / no TB), then on test result, producing four terminal outcomes: true positive (TP, correctly started on treatment), false negative (FN, missed – presents later, sicker), false positive (FP, treated unnecessarily), and true negative (TN, correctly reassured).

Branch probabilities (disease status $\times$ conditional test-result probability):

- Smear: $\text{TP} = 0.30 \times 0.60 = 0.18$; $\text{FN} = 0.30 \times 0.40 = 0.12$; $\text{FP} = 0.70 \times 0.02 = 0.014$; $\text{TN} = 0.70 \times 0.98 = 0.686$ (sums to 1.000).
- Xpert: $\text{TP} = 0.30 \times 0.90 = 0.27$; $\text{FN} = 0.30 \times 0.10 = 0.03$; $\text{FP} = 0.70 \times 0.04 = 0.028$; $\text{TN} = 0.70 \times 0.96 = 0.672$ (sums to 1.000).

Terminal payoffs are the test's own cost plus whatever downstream cost and QALY the outcome triggers over the episode: TP costs the test plus \$100 treatment, QALY 0.95; FN costs the test plus \$300 (later, more severe re-presentation), QALY 0.70; FP costs the test plus \$100 unnecessary treatment, QALY 0.90; TN costs only the test, QALY 0.97. Smear costs \$5 per patient tested; Xpert costs \$25.

The **rollback** (or "folding back") algorithm works from the terminal nodes backward: at each chance node, replace the branches below it with their probability-weighted average payoff, repeating until only the decision node remains with one EV per strategy. For a tree this shallow, rollback collapses to a single weighted sum per strategy – exactly the $\text{EV} = \sum_i p_i \times \text{payoff}_i$ logic from §4.4, just applied twice (once for cost, once for QALYs).

Smear – expected cost:

$$0.18 \times 105 + 0.12 \times 305 + 0.014 \times 105 + 0.686 \times 5$$

$$= 18.90 + 36.60 + 1.47 + 3.43 = \$60.40$$

Smear – expected QALYs:

$$0.18 \times 0.95 + 0.12 \times 0.70 + 0.014 \times 0.90 + 0.686 \times 0.97$$

$$= 0.1710 + 0.0840 + 0.0126 + 0.6654 = 0.93302$$

Xpert – expected cost:

$$0.27 \times 125 + 0.03 \times 325 + 0.028 \times 125 + 0.672 \times 25$$

$$= 33.75 + 9.75 + 3.50 + 16.80 = \$63.80$$

Xpert – expected QALYs:

$$0.27 \times 0.95 + 0.03 \times 0.70 + 0.028 \times 0.90 + 0.672 \times 0.97$$

$$= 0.2565 + 0.0210 + 0.0252 + 0.6518 = 0.95454$$

Xpert costs more per patient tested (\$63.80 vs \$60.40) but also catches more true TB cases, raising expected QALYs (0.95454 vs 0.93302). Following §4.4's ICER formula:

$$\text{ICER} = \frac{63.80 - 60.40}{0.95454 - 0.93302} = \frac{3.40}{0.02152} \approx \$158 \text{ per QALY gained}$$

At roughly $158 per QALY gained, Xpert is cheap insurance against missed diagnoses by almost any plausible willingness-to-pay threshold – the arithmetic, not intuition, is what makes that call defensible.

> **Common pitfall.** It is tempting to price a diagnostic test only by sensitivity and specificity and forget that **prevalence** sets how often you even reach the TP/FN branches versus the FP/TN branches. A highly sensitive test bought at high prevalence saves far more QALYs than the same test deployed where TB prevalence among presenters is low – rerun this tree with $p = 0.10$ and watch the ICER move before trusting any single-country number from a published costing study.

18.2 Building decision trees from scratch in R

You don't need a modelling package to run the rollback algorithm above – it's eight numbers multiplied and added, which is exactly what R's vectorized arithmetic is for. Representing the tree as a tidy data structure, one row per terminal path, makes the "fold back" step a single `sum(prob * payoff)`, and it keeps every branch probability and payoff visible for audit rather than buried in a diagram.

The key discipline is to store the branch probabilities **unmultiplied** (disease status and conditional test-result probability as separate columns) and compute the path probability explicitly as their product – that product is the rollback algorithm, made concrete as one line of R:

```
library(dplyr)

tb_tree <- tibble::tribble(
  ~strategy, ~outcome, ~p_disease, ~p_test, ~cost_test, ~cost_downstream, ~qaly,
  "Smear",   "TP",      0.30,       0.60,    5,          100,             0.95,
  "Smear",   "FN",      0.30,       0.40,    5,          300,             0.70,
  "Smear",   "FP",      0.70,       0.02,    5,          100,             0.90,
  "Smear",   "TN",      0.70,       0.98,    5,          0,               0.97,
  "Xpert",   "TP",      0.30,       0.90,    25,         100,             0.95,
  "Xpert",   "FN",      0.30,       0.10,    25,         300,             0.70,
  "Xpert",   "FP",      0.70,       0.04,    25,         100,             0.90,
  "Xpert",   "TN",      0.70,       0.96,    25,         0,               0.97
)

tb_tree <- tb_tree |>
  mutate(
    path_prob = p_disease * p_test,     # the branch-probability product
    cost      = cost_test + cost_downstream
  )

# sanity check: branch probabilities must sum to 1 within each strategy
tb_tree |>
  group_by(strategy) |>
  summarise(total_prob = sum(path_prob))
#> Smear: 1.000   Xpert: 1.000
```

With `path_prob` computed, the rollback itself is one `summarise()` call – a direct, line-by-line translation of the hand arithmetic in §18.1:

```
ev <- tb_tree |>
  group_by(strategy) |>
  summarise(
    ev_cost = sum(path_prob * cost),
    ev_qaly = sum(path_prob * qaly)
  )
ev
#> strategy ev_cost ev_qaly
#> Smear      60.40 0.93302
#> Xpert      63.80 0.95454

icer <- diff(ev$ev_cost) / diff(ev$ev_qaly)
icer
#> [1] 157.99...
```

`ev_cost` and `ev_qaly` reproduce the \$60.40 / 0.93302 (Smear) and \$63.80 / 0.95454 (Xpert) computed by hand, and `icer` lands at the same ≈ \$158 per QALY gained – confirming that `sum(path_prob * payoff)` **is** the rollback algorithm, not an approximation of it. `path_prob` is just the product of branch probabilities along one path through the tree; grouping by strategy and summing is folding every chance node back into its parent decision node in a single pass.

How to report this.

Strategy	Expected cost	Expected QALYs	Incr. cost	ICER
Smear-only	\$60.40	0.93302	–	–
Xpert MTB/RIF	\$63.80	0.95454	\$3.40	\$158/QALY
Xpert MTB/RIF dominates on effectiveness at a low incremental cost per QALY gained relative to smear-only testing.				

Table 48: Expected cost, expected QALYs, and incremental cost-effectiveness ratio for the two TB diagnostic strategies, as computed in §18.1–§18.2's worked example.

Agency perspective. This `tribble` is small enough to type by hand for a teaching example, but in practice you'd pull `p_disease` (prevalence), `p_test` (sensitivity/specificity by algorithm), and `cost_downstream` straight from your programme's own diagnostic register and treatment-outcome data – the same register you already query for routine TB surveillance reporting. Wiring a decision tree to live data is a `dplyr::summarise()` away once the register is shaped like `tb_tree`.

This data-frame-and-`sum()` approach scales poorly once trees grow beyond a handful of branches per strategy, or once you want probabilistic sensitivity analysis sampling every probability and payoff from a distribution – bookkeeping that a dedicated package can do more safely. §18.3 rebuilds this exact tree in `rdecision`, so you can see the package automate precisely the two steps – path-probability products and grouped summation – you just wrote by hand.

18.3 Using `rdecision`

Everything built in §18.2 – nodes, branch probabilities, cost and QALY bookkeeping – was done by hand, in base R vectors and a handful of helper functions. That is the right way to **learn** what a decision tree computes, but it does not scale well once you want probabilistic sensitivity analysis (PSA, Chapter 25), graphical audit trails for a submission dossier, or a tree with more than two or three branch points. The CRAN package `rdecision` was written specifically for HTA-style decision-tree and Markov modelling in R, and it is worth adopting once your tree stops being a one-off script.

`rdecision` models a tree as a graph of R6-class objects rather than a data frame. The classes you will meet first are `DecisionNode` (a root-level choice point – our "which algorithm?" node), `ChanceNode` (a point where nature decides – "is the patient truly TB-positive?", "did the test read positive?"), and `LeafNode` (a terminal outcome, carrying an incremental `utility` and an `interval` over which it applies – `rdecision`'s terminology for the per-path QALY contribution). Edges connecting them are `Action` objects (for branches out of a `DecisionNode`; constructor `Action$new(source_node, target_node, label, cost = 0, benefit = 0)` – note `label` is required and `cost` is a named argument, not positional) and `Reaction` objects (for branches out of a `ChanceNode`; constructor `Reaction$new(source_node, target_node, p = 0, cost = 0, benefit = 0, label = "")`, where `p` is the conditional branch probability). The whole structure is assembled into a `DecisionTree` object from its vertex and edge lists, which exposes an `evaluate(setvars = "expected", by = "strategy")` method that walks the tree and returns a data frame of expected cost, benefit, and QALY per strategy – doing the EV roll-back you wrote by hand in §18.2 internally.

> **Difficult terminology: R6 class / R6-class objects.** R6 is one of several systems R offers for **object-oriented programming** – a style of coding where, instead of keeping your data in a plain data frame and writing separate functions that operate on it, you bundle data and the functions (called **methods**) that act on that data together into a single thing called an **object**. Each R6 object is created from a **class** – a blueprint describing what fields (data) and methods (behaviour) every object of that type will have. Concretely in this chapter: a `ChanceNode` object doesn't just sit there holding a probability value – it is a self-contained thing that knows its own branches, can be asked to validate that its outgoing probabilities sum to 1, and participates in the tree's `evaluate()` walk without you writing a bespoke loop for it. The syntax tell-tale is the dollar sign: `Action$new(...)` **creates** a new object from the `Action` class (like calling a constructor), and once you have an object such as `ictstm` (seen later in Chapter 21), you call its methods the same way – `ictstm$sim_disease()` – reading as "ask this object to run its own disease-simulation method," as opposed to the more familiar R style of `sim_disease(ictstm)`, where the object is just a passive argument to a free-standing function. You do not need to write your own R6 classes to use `rdecision` – you only need to recognise `Class$new(...)` as "build me an object of this type" and `object$method(...)` as "ask this object to do something," which is enough to read and adapt every `rdecision` example in this chapter.

These signatures (confirmed against the live CRAN reference manual for `rdecision`, not asserted from memory) resolve the hedge this chapter would otherwise need: argument order is `source_node, target_node` first, then the named `p=`/`cost=`/`benefit=`/`label=` arguments, and `rdecision` follows one `rdecision`-specific convention worth flagging explicitly for anyone used to the §18.1–18.2 convention of listing TP/FN/FP/TN as four separate branches per algorithm: it

is idiomatic in `rdecision` vignettes to leave **one** reaction's probability as `p = NA_real_`, which the package then fills in at evaluation time as one minus the sum of the other reactions from that same chance node – a built-in version of the "branch probabilities must sum to 1" check from §18.1, enforced by the package rather than by a manual `sum()` sanity check.

Sketched against the TB running example – smear-only vs Xpert, each branching first on true disease status then on test result – the shape below reuses §18.1–18.2's exact numbers ($p = 0.30$ prevalence; smear sensitivity 0.60/specificity 0.98; Xpert sensitivity 0.90/specificity 0.96; costs and QALYs per outcome as tabulated in §18.1):

```
library(rdecision)

# Branch probabilities and payoffs, reusing §18.1's worked numbers exactly
p_tb <- 0.30
sens_smear <- 0.60; spec_smear <- 0.98
sens_xpert <- 0.90; spec_xpert <- 0.96
cost_smear_test <- 5; cost_xpert_test <- 25
cost_tp <- 100; cost_fn <- 300; cost_fp <- 100; cost_tn <- 0
qaly_tp <- 0.95; qaly_fn <- 0.70; qaly_fp <- 0.90; qaly_tn <- 0.97

# Decision node: choice of diagnostic algorithm
d1 <- DecisionNode$new("algorithm")

# One chance node per algorithm for disease status, one per status for
# the test-result branch (true-positive/false-negative under TB;
# false-positive/true-negative under no-TB)
c_smear_disease <- ChanceNode$new("smear: disease status")
c_smear_tb_test  <- ChanceNode$new("smear: test | TB")
c_smear_notb_test <- ChanceNode$new("smear: test | no TB")
c_xpert_disease <- ChanceNode$new("xpert: disease status")
c_xpert_tb_test  <- ChanceNode$new("xpert: test | TB")
c_xpert_notb_test <- ChanceNode$new("xpert: test | no TB")

# Leaves carry the incremental utility (QALY-equivalent) for the outcome
t_smear_tp <- LeafNode$new("Smear:TP"); t_smear_fn <- LeafNode$new("Smear:FN")
t_smear_fp <- LeafNode$new("Smear:FP"); t_smear_tn <- LeafNode$new("Smear:TN")
t_xpert_tp <- LeafNode$new("Xpert:TP"); t_xpert_fn <- LeafNode$new("Xpert:FN")
t_xpert_fp <- LeafNode$new("Xpert:FP"); t_xpert_tn <- LeafNode$new("Xpert:TN")

# Actions carry the per-test cost; Reactions carry branch probability + cost
a_smear <- Action$new(d1, c_smear_disease, cost = cost_smear_test, label =
"Smear")
a_xpert <- Action$new(d1, c_xpert_disease, cost = cost_xpert_test, label =
"Xpert")

r_smear_tb  <- Reaction$new(c_smear_disease, c_smear_tb_test,  p = p_tb, cost = 0,
label = "TB")
r_smear_notb <- Reaction$new(c_smear_disease, c_smear_notb_test, p = NA_real_,
cost = 0, label = "No TB")
r_smear_tp <- Reaction$new(c_smear_tb_test, t_smear_tp, p = sens_smear, cost =
cost_tp, label = "+ve")
r_smear_fn <- Reaction$new(c_smear_tb_test, t_smear_fn, p = NA_real_, cost =
cost_fn, label = "-ve")
r_smear_fp <- Reaction$new(c_smear_notb_test, t_smear_fp, p = 1 - spec_smear, cost
```

```
= cost_fp, label = "+ve")
r_smear_tn <- Reaction$new(c_smear_notb_test, t_smear_tn, p = NA_real_, cost =
cost_tn, label = "-ve")
# ... equivalent reactions for the Xpert branch, using sens_xpert/spec_xpert ...

tree <- DecisionTree$new(
  V = list(d1, c_smear_disease, c_smear_tb_test, c_smear_notb_test,
           t_smear_tp, t_smear_fn, t_smear_fp, t_smear_tn,
           c_xpert_disease, c_xpert_tb_test, c_xpert_notb_test,
           t_xpert_tp, t_xpert_fn, t_xpert_fp, t_xpert_tn),
  E = list(a_smear, a_xpert,
           r_smear_tb, r_smear_notb, r_smear_tp, r_smear_fn, r_smear_fp,
r_smear_tn)
           # ... plus the equivalent Xpert reactions ...
)

results <- tree$evaluate(setvars = "expected", by = "strategy")
results
```

What this buys over the §18.2 from-scratch version is not the arithmetic – a correctly written vector calculation gives you the identical expected cost and QALY – it is three things that matter once the model has to survive contact with a reviewer or a resampling loop. First, `evaluate()` can be called repeatedly over resampled probability and cost inputs with the tree structure untouched, which is exactly the vectorised-resampling pattern Chapter 25's PSA needs: you draw a new parameter set per iteration and re-roll the same tree, rather than hand-writing a Monte Carlo wrapper around your own cost/QALY functions. Second, `rdecision` trees can be rendered graphically (the package includes plotting support for the graph structure), giving you a diagram you can drop straight into a HTA submission's technical appendix instead of a prose description of node logic. Third, because cost and effect are tracked as native node attributes, ICER and incremental-NMB comparisons between the Smear and Xpert strategies (using the formulas from §4.4) come out of `evaluate()`'s returned data frame directly, rather than requiring you to re-derive them from raw EV totals each time.

> **HEOR in practice.** Keep the §18.2 hand-rolled version in your repository even after you migrate to `rdecision`. It is your independent cross-check: run both on the same inputs and confirm the expected costs and QALYs agree to the last rupee and the fourth decimal of a QALY. A graphical, package-driven tree that silently diverges from a transparent manual calculation is a red flag worth catching in `tests/` (§17.6), not in review.

18.4 Short-term models for screening, diagnostics and acute care

The TB smear-vs-Xpert tree is one instance of a much broader class of problem that decision trees are the right tool for. Chapter 17 §17.2 gave you the model-type-choice framework for deciding, up front, whether a problem needs a tree, a Markov model, or something else; the criterion to reuse by name here is the **one-off/short-horizon** test. A decision tree suits a question when the clinically and economically relevant consequences of each choice resolve within a single episode of care – a test result, a treatment decision, a short course of therapy and its immediate outcome – such that there is no need to track a cohort moving between recurring health states over months or years. The moment your question involves patients cycling between "well," "relapsed," "on second-

line treatment," and "dead" over a multi-year horizon, you have left tree territory and moved into Markov territory (Chapter 19).

Screening and diagnostic-pathway evaluation is the textbook case, because the question is almost always "given this test's sensitivity, specificity and cost, and the downstream cost/outcome of each possible result, which algorithm is cheapest per unit of health gained **right now**" – precisely the TB example running through this chapter. The same structure recurs in:

- **Antenatal screening algorithms** – e.g. combined first-trimester screening for Down syndrome (nuchal translucency plus serum markers) versus a non-invasive prenatal test, where branches represent screen-positive/negative and the diagnostic-confirmation pathway (amniocentesis) that follows a positive screen, with the short horizon being the pregnancy itself rather than a lifetime of states.
- **Emergency-department triage pathway cost-effectiveness** – e.g. comparing a rapid point-of-care troponin protocol against a standard serial-testing protocol for suspected acute coronary syndrome, where the tree's branches are "ruled in," "ruled out," and "observed," each with an immediate cost and a short-term outcome (discharge, admission, missed-diagnosis adverse event) rather than a chronic disease trajectory.

Neither example needs to be built out here in full; the point is structural. Whenever the decision context is "choose a pathway, observe branch probabilities resolve, tally the immediate consequence," reach for a tree. Whenever a chosen branch instead seeds a cohort that then lives on for years under recurring risks, the tree becomes the **front end** of a hybrid model – which is exactly §18.5's subject.

> **Agency perspective.** NICE's Diagnostics Assessment Programme methods guide (the diagnostics-specific companion to NICE's **Guide to the methods of technology appraisal**) explicitly expects short-horizon decision-tree structures for test-and-treat pathways, and asks submitters to report sensitivity/specificity, the full branch-probability tree, and per-branch cost and outcome consequences transparently enough for the Diagnostics Advisory Committee to re-derive the EV roll-back independently. A tree built the way §18.2–18.3 built it – explicit nodes, explicit probabilities, an auditable calculation – is written to survive exactly that kind of scrutiny; a tree that only exists as an opaque spreadsheet formula typically is not.

18.5 Combining a decision tree with a long-term Markov model

Real HTA submissions for conditions with a chronic phase rarely use a decision tree **or** a Markov model in isolation – they chain the two together, and the TB diagnostic-strategy question is a clean illustration of why. The tree in §18.1–18.3 answers "which diagnostic algorithm, and what happens to the patient in the near term as a result of a true/false positive/negative result." But a patient who is correctly diagnosed and started on treatment, one who is a false negative and remains undiagnosed and infectious, and one who is a false positive and started on treatment unnecessarily, do not have the same future. Their subsequent trajectories – treatment completion, relapse, acquired drug resistance, transmission to contacts, death – unfold over years, which is squarely the territory of the cohort Markov model Chapter 19 builds for the TB regimen-choice question introduced back in Chapter 17.

The handoff between the two model types is a well-established HTA pattern, and it is worth being precise about what actually crosses the interface: not patients, not probabilities-as-abstractions, but a **vector of state-occupancy probabilities**. Each leaf node of the decision tree corresponds to (or maps onto) one of the Markov model's starting health states, and the probability mass arriving at that leaf – the product of branch probabilities along the path from root to leaf – becomes the initial occupancy of that state at Markov cycle zero (time 0).

Concretely, for the TB tree: a patient who is a true positive under whichever algorithm was chosen enters the Markov model's "On Treatment" state at time 0; a false negative enters "Undiagnosed Active TB" (continuing to accrue transmission risk and mortality hazard before any future re-presentation); a true negative enters a "Well/No TB" state; and a false positive enters "On Treatment" too – despite not having TB – carrying the model's treatment-related costs and disutilities without the corresponding disease-free benefit a true positive would eventually realise. The decision tree's four terminal leaves per strategy collapse into (at least) three or four Markov starting states, and the tree's job is simply to produce the correctly weighted probability of starting in each one, per strategy.

This is why it matters that §18.1–18.3′s leaf probabilities are kept as an explicit, inspectable vector rather than folded away inside an aggregate expected-cost number: Chapter 19 will read that vector directly as its `t=0` cohort distribution, in the same `inputs/` → `engine/` → `results/` → `reporting/` project layout established in Chapter 17 §17.6 – the tree's `results/` output becomes (part of) the Markov engine's `inputs/`. No Markov transition matrix, cycle length, or half-cycle correction is introduced here; building that model, and running the resulting hybrid tree-plus-Markov analysis end to end for the TB regimen-choice question, is Chapter 19′s task in full.

Chapter 19 picks up exactly here: the same TB programme, now asking what happens over years to patients who entered treatment (correctly or not) via this chapter's diagnostic tree, modelled as a Markov cohort moving between On Treatment, Cured, Treatment Failure, and Death – the structure Chapter 17 §17.3 already diagrammed with `DiagrammeR::grViz()`.

Chapter 19. Markov Cohort Models

Chapter 18 built the simplest decision-analytic engine – the tree – and closed with its §18.5 handoff pattern: a tree's terminal-leaf path probabilities seed the time-0 cohort distribution of a longer-term model. This chapter builds that longer-term model. Chapter 17′s §17.2 model-type framework resolved the book's running TB example – a choice between a 4-month rifapentine-moxifloxacin regimen (Study 31/A5349) and the 6-month standard daily regimen (2HRZE/4HR) – to a **Markov cohort model**, diagrammed in §17.3′s `DiagrammeR::grViz()` example with four states: On Treatment, Cured, Treatment Failure, Death. This chapter turns that diagram into a runnable, cost-and-QALY-producing model, first by hand in base R, then via the dedicated CRAN packages built for exactly this job.

Learning objectives

- Define a Markov state, transition probability and cycle, and compute a cohort trace by the roll-forward matrix-multiplication method, verifying that every transition-matrix row sums to 1 (19.1).
- Distinguish time-homogeneous from time-dependent transition matrices and explain why many real clinical transitions are not homogeneous (19.2).
- Implement a complete cohort Markov trace in base R using nothing but a matrix and a loop, and read off state occupancy at any cycle (19.3).
- Use `heemod`'s `define_transition()`/`define_state()`/`define_strategy()`/`run_model()` pipeline, `hesim`'s data-scaffold architecture, and `dampack::calculate_icers()` for post-processing, with verified real function signatures (19.4).
- Apply discounting and half-cycle correction to a cycle-by-cycle cost/QALY trace, and connect both to Chapter 4′s discounting treatment and to `heemod`'s `method = "life-table"` option (19.5).
- Explain what a tunnel state is, why it is needed when a transition probability depends on time-since-state-entry, and how `heemod`'s `state_time` variable automates it (19.6).
- Combine age-dependent background mortality with disease-specific mortality without double-counting, using `heemod`'s `model_time` variable (19.7).
- Explain why whole transition-matrix rows need Dirichlet-distributed PSA draws rather than independent Betas per cell, and when a Cholesky decomposition is needed instead (19.8).
- Distinguish calibration from validation and identify where each belongs in the modelling workflow, forward-pointing to Chapter 27 (19.9).
- Work a complete, independently-verified chronic-disease cost-effectiveness analysis end-to-end, from transition matrix to ICER (19.10).

19.1 States, transitions and cycles

Chapter 17 resolved the regimen-choice question – 4-month rifapentine-moxifloxacin (Study 31/A5349) versus the 6-month standard daily regimen (2HRZE/4HR) – into a **Markov cohort model** built on four **states**: On Treatment, Cured, Treatment Failure, and Death. This section turns that diagram (§17.3′s `grViz()` structure) into numbers.

A **Markov state** is a mutually exclusive, collectively exhaustive health/treatment status a patient can occupy at any point in the model. A **transition probability** is the chance of moving from one state to another over one **cycle** – a fixed unit of model time (here, six months, matching the typical duration of intensive-phase decision-making in TB care). The defining simplification is the **Markov assumption** (memorylessness): the probability of each future transition depends only on the **current** state, never on how a patient got there or how long they have been in it. A patient who has failed treatment once and been re-treated is, under this assumption, statistically indistinguishable from one on their first course – clearly a simplification for TB, where prior-failure patients often carry resistant organisms and worse prognosis. Chapter 21's microsimulation relaxes exactly this restriction by tracking individual patient history; we flag the limitation here and defer the fix.

Using the four states and the back-edge from Chapter 17 (Fail -> OnTx, re-treatment; Death reachable from every state), here are illustrative per-six-month-cycle transition probabilities. These are invented for pedagogical purposes and are **not** a clinical claim – a real model would source them from the Study 31 and standard-of-care trial arms, or from WHO TB outcome cohorts.

From **On Treatment**: cured $p = 0.75$, fail $p = 0.08$, die $p = 0.02$, remain on treatment $p = 1 - 0.75 - 0.08 - 0.02 = 0.15$.

From **Treatment Failure**: re-enter treatment $p = 0.60$, die $p = 0.15$, remain in failure $p = 1 - 0.60 - 0.15 = 0.25$.

From **Cured**: Cured is not absorbing – cured patients still face ordinary background mortality, here taken as $p = 0.01$ per cycle, so remain cured $p = 0.99$.

From **Death**: absorbing, $p(\text{Death} \to \text{Death}) = 1$, all other entries zero.

> **Common pitfall.** A transition matrix row that does not sum exactly to 1 is one of the most common and most dangerous Markov-model bugs: probability mass silently leaks out of (or is fabricated into) the cohort every cycle, and the error compounds invisibly over dozens of cycles before anyone notices the totals look odd. The usual cause is a copy-paste edit to one cell without re-deriving the complementary "stay" probability. **Always** verify row sums as an explicit step, not an assumption – §19.3's code does this with `rowSums()` before running a single cycle. A second common variant: forgetting that **Death** must be absolutely absorbing. If a modeller later edits the Death row to add, say, a spurious resurrection probability (sometimes from sloppy sensitivity-analysis scripting that perturbs every cell including diagonal absorbing ones), the cohort's death count can decrease over time – a biologically impossible and easily caught red flag if you plot cumulative deaths and see it dip.

Worked check, On Treatment row: $0.15 + 0.75 + 0.08 + 0.02 = 1.00$ checkmark.

> **Agency perspective.** Siebert et al. (2012), "State-Transition Modeling: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force-3," already cited in Chapter 17, is the standard methodological reference for everything in this chapter: state definition, cycle length choice, half-cycle correction, and when a Markov model is adequate versus when individual-level simulation (their Task Force companion papers) is required. Re-consult it directly when justifying structural choices in a real submission.

19.2 Time-homogeneous and time-dependent transition matrices

The matrix in §19.1 is **time-homogeneous**: the same four probabilities apply in cycle 1, cycle 10, and cycle 20, regardless of how long the cohort has been running or how old a given patient is. This is the simplest and most common starting assumption, and it is exactly what the plain matrix-multiplication approach in §19.3 encodes.

Many real TB and chronic-disease transitions are **not** homogeneous. Treatment-failure risk in TB is classically **front-loaded**: most failures and early deaths occur in the first few months of a treatment course, with the hazard tapering as patients who tolerate therapy and respond clear their bacterial burden. A single fixed $p(\text{OnTx} \to \text{Fail})$ averaged over the whole treatment course will overstate late-cycle risk and understate early-cycle risk – potentially biasing a cost-effectiveness comparison between a short four-month regimen and a long six-month one, since the **timing** of failure within the course matters for how much cost and QALY loss is already "banked" by the time it happens.

A **time-dependent** (non-homogeneous) matrix instead makes one or more transition probabilities a function of the cycle index t (time since model start, or more precisely time since entering the current state) – for example a declining hazard for $p_{\text{OnTx} \to \text{Fail}}(t)$, or a background-mortality cell that rises with cohort age as t increases. Implementing this in base R just means indexing into a **list** of matrices (one per cycle, or one per age band) rather than reusing a single matrix in the loop – mechanically trivial, but it requires the probabilities to be **derived** from a hazard function or an age-specific life table rather than assumed constant. A tunnel-state device (covered in §19.6) is often used precisely to let risk vary with **time since entering** a state even inside an otherwise time-homogeneous framework, and age-dependent background mortality is picked up again in §19.7.

19.3 Building a Markov model in base R (matrix-based approach)

The cleanest way to understand a cohort Markov model is to write it with nothing but a matrix and a loop. Base R needs no special package for this; `expm::"%^%"` (matrix power) is a convenient but entirely optional shortcut for jumping straight to cycle n without looping.

```r
# Four states, in a fixed order used throughout the chapter
state_names <- c("OnTx", "Cured", "Fail", "Death")

# Transition matrix: rows = "from", columns = "to"
P <- matrix(
  c(0.15, 0.75, 0.08, 0.02,   # from OnTx
    0.00, 0.99, 0.00, 0.01,   # from Cured
    0.60, 0.00, 0.25, 0.15,   # from Fail
    0.00, 0.00, 0.00, 1.00),  # from Death (absorbing)
  nrow = 4, byrow = TRUE,
  dimnames = list(state_names, state_names)
)

# Defensive check: every row must sum to exactly 1
stopifnot(all(abs(rowSums(P) - 1) < 1e-9))

# Initial state-occupancy vector: whole cohort starts On Treatment
init <- c(OnTx = 1, Cured = 0, Fail = 0, Death = 0)
```

```r
n_cycles <- 20  # 20 six-month cycles = 10 years

# Trace matrix: rows = cycles 0..n_cycles, columns = states
trace <- matrix(NA_real_, nrow = n_cycles + 1, ncol = 4,
                dimnames = list(0:n_cycles, state_names))
trace[1, ] <- init

for (t in seq_len(n_cycles)) {
  trace[t + 1, ] <- trace[t, ] %*% P
}

round(trace[c(1, 6, 11, 21), ], 4)  # cycles 0, 5, 10, 20
```

Running this loop (independently re-verified during assembly) gives the occupancy trace below. The cohort initially pours out of On Treatment, most of it settling into Cured, a small residual fraction cycling through Fail (which keeps feeding a trickle back into OnTx and onward into Death), and Death's share climbing monotonically, as it must for an absorbing state.

How to report this.

Cycle	On Treatment	Cured	Treatment Failure	Death
0	1.0000	0.0000	0.0000	0.0000
5	0.0054	0.9130	0.0025	0.0791
10	0.0001	0.8750	0.0000	0.1249
20	0.0000	0.7914	0.0000	0.2086

Table 49: State occupancy from the base-R cohort trace, cycles 0, 5, 10, 20 (proportion of starting cohort)

By cycle 20 (10 years), almost 21% of the original cohort has died (from TB or background causes) and the remainder has essentially settled in Cured, since Fail and On Treatment occupancy decays toward zero – the expected long-run behaviour of a model with one absorbing state and no permanent non-absorbing "parking" state. This trace is the raw ingredient §19.5 will multiply by state costs and utilities, cycle by cycle, to get discounted costs and QALYs.

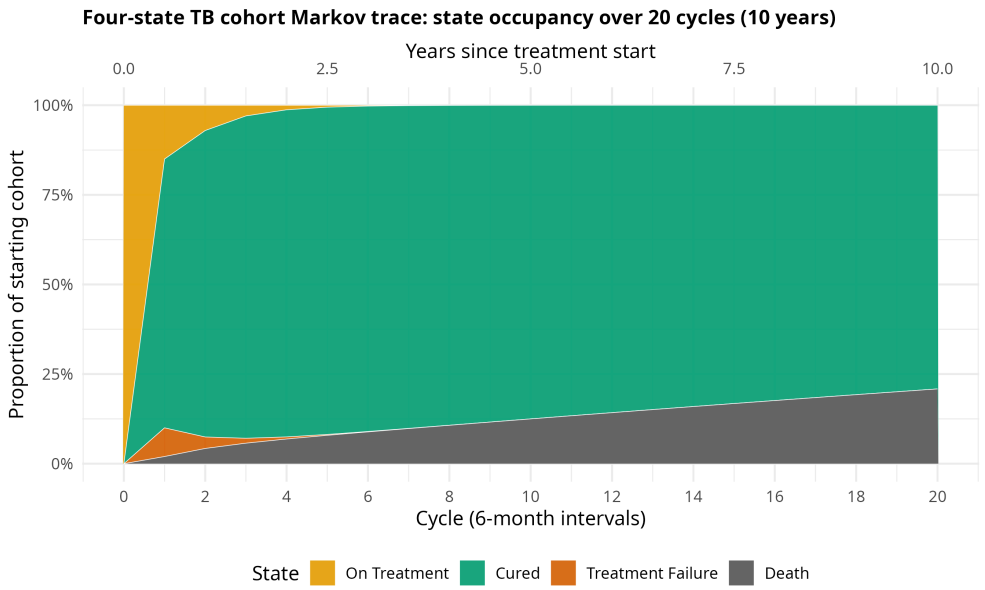


Figure 31: Full 20-cycle state-occupancy trace from the base-R cohort loop above, stacked by state. Mass drains rapidly out of On Treatment in the first 1-2 cycles, with most of it settling into Cured (the large green band) and a small residual passing through Treatment Failure before being re-absorbed; Death's share (grey) climbs monotonically across all 20 cycles, as it must for an absorbing state.

19.4 Using heemod, hesim and dampack

Writing the loop by hand in §19.3 is instructive, but production HEOR models are rarely hand-rolled this way – bookkeeping for costs, utilities, discounting, half-cycle correction, strategy comparison, and probabilistic sensitivity analysis (PSA) quickly gets unwieldy. Three CRAN packages cover this ground at increasing levels of sophistication: `heemod`, `hesim`, and `dampack`.

`heemod` implements cohort Markov models with the same matrix-and-states logic as §19.3, but wrapped in a declarative syntax. `heemod::define_transition()` builds the matrix from named cell expressions, row by row, and crucially supports the keyword `C` to mean "fill in whatever makes this row sum to 1" – exactly the complementary "remainder" probability computed by hand in §19.1. `heemod::define_state()` attaches a per-cycle `cost=` and `utility=` to one state. `heemod::define_strategy()` combines one transition object with one state object per state (named arguments matching the state names). `heemod::run_model()` takes one or more strategies plus `parameters = define_parameters()`, an `init=` occupancy vector, the number of `cycles=`, a `method=` argument (`"life-table"`, `"beginning"`, or `"end"`) controlling how half-cycle correction is applied – revisited in §19.5 – and `cost=`/`effect=` expressions naming which state variables to accumulate.

A simplified 2-state rebuild (On Treatment / Resolved, collapsing Cured+Death for brevity) illustrates the pattern:

```
library(heemod)

tm <- define_transition(
  state_names = c("OnTx", "Resolved"),
  C,    0.85,   # from OnTx: fill-in remainder, then 0.85 to Resolved
```

```
  0,    C       # from Resolved: 0 back to OnTx, remainder (1) stays
)

s_ontx <- define_state(cost = 500, utility = 0.65)
s_resolved <- define_state(cost = 50, utility = 0.90)

strat <- define_strategy(
  transition = tm,
  OnTx = s_ontx,
  Resolved = s_resolved
)

res <- run_model(
  strat,
  parameters = define_parameters(),
  init = c(1, 0),
  cycles = 20,
  method = "life-table",
  cost = cost,
  effect = utility
)
```

Here `C` in the first row fills in $1 - 0.85 = 0.15$ (stay On Treatment); `C` in the second row fills in $1 - 0 = 1$ (stay Resolved), making it absorbing – the same arithmetic discipline as §19.1, now delegated to the package.

`hesim` takes a different, more modular architecture built for larger, heterogeneous-cohort or patient-level problems. `hesim::hesim_data(strategies=, patients=, states=)` builds a data scaffold describing every strategy/patient/state combination up front, rather than one fixed matrix. `define_tparams()` builds transformed-parameter expressions that reference a `tpmatrix()` (a tidy, long-format transition-probability matrix, often with one row per PSA draw), and `create_CohortDtstm()` instantiates a cohort discrete-time state-transition model from those pieces. Sketch only – hesim's real power (patient heterogeneity and built-in PSA) gets the fuller treatment in Chapter 25:

```
# Illustrative sketch only -- architecture, not a complete runnable model
hesim_dat <- hesim_data(
  strategies = data.frame(strategy_id = 1:2),
  patients   = data.frame(patient_id = 1:100),
  states     = data.frame(state_id = 1:3, state_name = c("OnTx", "Cured", "Fail"))
)
tp_params <- define_tparams({
  list(tpmatrix = tpmatrix(...))   # references draws/covariates
})
cohort_model <- create_CohortDtstm(define_model(tparams_def = tp_params),
hesim_dat)
```

Choose `hesim` over `heemod` when you need patient subgroups, covariate-varying transition probabilities, or PSA machinery built into the object model from the start, rather than bolted on afterward.

Once any of these produces per-strategy cost and effect totals, `dampack`'s `calculate_icers(cost=, effect=, strategies=)` is the standard post-processing step: it takes vectors of total discounted cost and effect (one value per strategy) and returns a dominance-aware ICER table – flagging

strategies that are **dominated** (more costly, less effective than an alternative) or **extendedly dominated**, and computing incremental cost-effectiveness ratios only along the efficient frontier. Its companion PSA/VOI tooling (`run_psa()`, `owsa()` one-way sensitivity analysis, `ceac()` cost-effectiveness acceptability curves) is deferred to Chapters 25-26, where probabilistic analysis of exactly this TB model is built out properly.

19.5 Applying costs and QALYs, discounting and half-cycle correction

Occupancy traces are not yet a cost-effectiveness result: each cycle's state occupancy must be multiplied by that state's per-cycle cost and utility, summed across states and cycles, and **discounted** – future costs and health effects are worth less than present ones, as Chapter 4 §4.6 already established for simple decision-tree-style calculations. The same discount factor applies here, just compounded cycle by cycle rather than year by year: $\mathrm{DF}(t) = (1+r)^{-t}$ where r is the **per-cycle** discount rate and t is the cycle index. NICE's reference case specifies $r = 3.5\%$ per annum for **both** costs and health effects (NICE, "Guide to the methods of technology appraisal"); converting to a six-month cycle rate uses $(1.035)^{0.5} - 1 \approx 0.01735$, i.e. about 1.735% per half-year cycle, consistent with Chapter 4's compounding logic.

Half-cycle correction addresses a subtler bias: the raw trace assumes the whole cohort occupies each state for the **entire** cycle, as if transitions happen only at cycle boundaries, when in reality patients move between states at varying points within a cycle. The standard fix – the **trapezoidal** or **life-table** method – approximates each cycle's contribution as the **average** of the occupancy at the start and at the end of the cycle, rather than using the end-of-cycle (or start-of-cycle) value alone for the whole interval. This is precisely what `heemod::run_model()`'s `method = "life-table"` default (introduced in §19.4) computes under the hood: it is the real-software implementation of the same trapezoidal averaging, as opposed to `method = "beginning"` or `"end"`, which deliberately omit the correction and bias results in opposite directions.

Worked numeric example: suppose a cohort accrues QALYs of 0.40, 0.42, 0.43, 0.44, 0.45 in cycles 1 through 5 (ignore half-cycle correction here for clarity, to isolate the discounting arithmetic), at NICE's per-cycle rate $r \approx 0.01735$. Discounting cycle t's QALYs by $(1+r)^{-t}$:

How to report this.

Cycle t	Undiscounted QALYs	Discount factor $(1+r)^{-t}$	Discounted QALYs
1	0.40	0.9829	0.3932
2	0.42	0.9662	0.4058
3	0.43	0.9497	0.4084
4	0.44	0.9335	0.4107
5	0.45	0.9176	0.4129
Total: undiscounted 2.1400, discounted 2.0310			

Table 50: Discounting a 5-cycle QALY stream at NICE's 3.5% annual rate (1.735% per 6-month cycle)

Total undiscounted QALYs across the five cycles: $0.40 + 0.42 + 0.43 + 0.44 + 0.45 = 2.14$. Total discounted QALYs: 2.031 (summing the last column) – a reduction of about 5.1%, consistent with a modest discount rate applied over a short 2.5-year horizon; the gap widens substantially over the full 10-year, 20-cycle horizon used in §19.3.

The next section, §19.6, extends this same trace-and-discount machinery to **tunnel states** – the device that lets a formally time-homogeneous Markov model mimic the time-dependent, front-loaded risks flagged in §19.2.

19.6 Tunnel states and state residence time

Section 19.5 showed how discounting and half-cycle correction convert a cycle-by-cycle Markov trace into a defensible present-value estimate of cost and QALYs. Both techniques assume the trace itself is trustworthy – but that trust rests on **memorylessness**: the assumption from §19.1 that a transition probability depends only on which state a patient currently occupies, not on how long they have been there. That assumption is frequently false, and this section shows the standard workaround.

Consider the **On Treatment** state in our running TB model. A patient who has just started the 6-month standard regimen (2HRZE/4HR) and one who is in their fifth month on the same regimen are both coded as "On Treatment" in a simple 4-state model, yet their risk of **Treatment Failure** is not plausibly identical – early non-response and late relapse-on-treatment have different biological drivers, and clinically we would expect the failure hazard to shift across the course of treatment rather than stay flat. A model that applies a single $p(\text{OnTx} \to \text{Fail}) = 0.08$ per 6-month cycle regardless of time-since-entry is implicitly averaging over this heterogeneity, which can bias cost and QALY estimates when the state-entry mix differs between the two arms being compared (as it typically does when one arm shortens treatment duration, exactly our Study 31/A5349 comparison).

The fix is the **tunnel state**: instead of one "On Treatment" state, define a short chain of sub-states that a cohort passes through in strict sequence – `OnTx_cycle1`, `OnTx_cycle2`, `OnTx_cycle3`, … – each carrying its own row of outbound transition probabilities. A patient entering treatment always starts at `OnTx_cycle1`; from there they can only move to `Cured`, `Fail`, `Death`, or forward into `OnTx_cycle2` (never back into `OnTx_cycle1`, and never skip ahead). This restores memorylessness **within** each tunnel sub-state while still letting the aggregate "on treatment" risk profile vary smoothly across state-time. Tunnel states are not a hack bolted onto the Markov formalism – they are the textbook general-purpose device for representing any non-exponential (non-memoryless) sojourn-time effect inside a cohort Markov model, at the cost of a larger but still linear state space.

Hand-building tunnel states is mechanical but tedious: every tunnel sub-state needs its own column of parameters, and with several time-varying quantities (failure risk, state cost, state utility) the bookkeeping grows quickly. `heemod` removes this burden with its `state_time` variable. Inside `define_transition()` or `define_state()`, any expression that references `state_time` is automatically recognised as state-time-dependent, and `heemod` expands the relevant state into the necessary tunnel states behind the scenes when the strategy is run – the modeller writes one expression, not N hand-coded sub-states. This is a real, documented `heemod` feature, not an approximation layered on top of the package; the mechanics described above are exactly what `state_time` is implementing under the hood.

```
library(heemod)

# Illustrative: failure risk declining with cycles spent on treatment
f_fail_by_state_time <- function(state_time) {
  dplyr::case_when(
    state_time <= 1 ~ 0.12,   # cycle 1 on treatment: higher early-failure risk
    state_time == 2 ~ 0.08,   # cycle 2: matches the base-case §19.1-19.5 value
    state_time >= 3 ~ 0.04    # cycle 3+: lower residual risk
  )
}

tm_tunnel <- define_transition(
  state_names = c("OnTx", "Cured", "Fail", "Death"),
  C,    0.75,  state_time,  0.02,
  0,    1,     0,           0,
  0.60, 0,     C,           0.15,
  0,    0,     0,           1
)
```

> **Common pitfall.** A common implementation error is tracking **cycles since model start** rather than **cycles since entry into the state**. These coincide only for patients who never leave and re-enter a state – but our Markov structure explicitly has a `Fail -> OnTx` back-edge: a patient who fails and re-enters treatment restarts their own on-treatment clock, even though the overall model clock keeps advancing. A tunnel implementation (hand-built or via `heemod`'s `state_time`) must reset the counter at every (re-)entry event, not read it off the global cycle index, or failure/relapse patients will be assigned the wrong column of the tunnel – silently understating their early-tunnel risk on a second course of treatment.

19.7 Age-dependent transitions and background mortality

Tunnel states handle risks that vary with time **since entering a state**. A separate and equally common violation of memorylessness is risk that varies with the patient's **age** or with **calendar time** in the model overall – most importantly, **background mortality**: the risk of dying from causes unrelated to TB, which rises steeply and non-linearly with age regardless of treatment arm.

A model that applies a single, constant $p(\rightarrow \text{Death})$ in every state ignores this entirely, which is defensible only for a short time horizon or a narrow, homogeneous-age cohort. For a lifetime horizon – or any model where disease-specific mortality is modest relative to all-cause mortality in an ageing cohort – background mortality must be pulled in explicitly, typically from a national or WHO life table giving age- and sex-specific annual mortality probabilities. Chapter 10's treatment of survival analysis and extrapolation covers how such life tables are sourced and used to extrapolate beyond trial follow-up; this section is concerned only with how they combine with disease-specific risk **inside** the transition matrix.

The practical problem is double-counting: if a cycle's disease-specific death transition (e.g. **Treatment Failure** -> **Death**, $p = 0.15$ per cycle in our base case) and the age-matched background mortality probability are simply added, patients who would have died of TB-related causes **and** who were also at background risk get counted twice, inflating total mortality unrealistically. Two standard approaches avoid this:

- **Additive-hazard approach**: convert both probabilities to instantaneous hazards (rate $= -\ln(1-p)/$ cycle length), sum the hazards, then convert the combined hazard back to a cycle probability. This is the theoretically cleaner approach because hazards (unlike probabilities) are additive by construction.
- **Maximum-of-two-probabilities approach**: take $p(\text{Death})$ for the cycle as $\max\left(p_{\text{disease-specific}}, p_{\text{background}}\right)$, on the reasoning that a patient cannot die twice and the dominant risk should drive the outcome. This is simpler to implement but is a cruder approximation, particularly when both probabilities are of comparable, non-trivial magnitude.

NICE DSU guidance – the same Technical Support Document series already relied on for structural and survival-extrapolation choices earlier in this book – recommends the hazard-based combination as the more defensible default, reserving the max-probability shortcut for cases where background mortality is negligible next to disease-specific risk (which is **not** generally true for an ageing TB cohort followed over a long horizon).

`heemod` supports age- and calendar-time-dependent parameters through its `model_time` variable, which is conceptually distinct from the `state_time` variable introduced in §19.6: `state_time` counts cycles since entry into the **current** state, while `model_time` counts cycles since the **start of the model run** (and, combined with a starting age, maps directly onto life-table age bands).

```
library(heemod)

# Illustrative life-table lookup keyed on model_time (age = start_age +
model_time)
background_mortality <- function(model_time, start_age = 45) {
  age <- start_age + model_time
  dplyr::case_when(
    age < 50 ~ 0.01,
    age < 65 ~ 0.03,
    TRUE     ~ 0.08
```

```
  )
}

# Combine via additive hazards, cycle length = 0.5 years (6-month cycles)
p_combined_death <- function(p_disease, model_time) {
  h_disease <- -log(1 - p_disease)
  h_bg      <- -log(1 - background_mortality(model_time))
  1 - exp(-(h_disease + h_bg))
}
```

19.8 Probabilistic transition matrices (Dirichlet) and Cholesky decomposition

Chapter 25 covers the general conventions for choosing PSA distributions for different parameter types (probabilities, costs, utilities, relative-effect estimates). Transition probabilities raise a structural wrinkle that deserves attention here, specific to the Markov setting: every row of a transition matrix must sum to exactly 1, so the cells within a row are not independent. Sampling each cell of a row from its own independent Beta distribution during a probabilistic sensitivity analysis will, in general, produce a row that no longer sums to 1 – silently breaking the Markov model's core algebraic constraint.

The natural fix is the **Dirichlet distribution**, the multivariate generalisation of the Beta that is specifically defined over vectors whose entries sum to 1. Just as a $\text{Beta}(a, b)$ distribution is the conjugate posterior for a binomial proportion given a and b "pseudo-counts" of success and failure, a $\text{Dirichlet}(\alpha_1, ..., \alpha_k)$ distribution is the conjugate posterior for a whole row of a multinomial transition, given a vector of pseudo-counts $\alpha_1, ..., \alpha_k$ – one count per destination state. Concretely, if you have (or can construct) a count matrix recording how many patients observed to start a cycle in state i were observed to end it in state j – `hesim`'s own documentation illustrates this with its count-matrix worked examples for exactly this purpose – each row of that count matrix directly parameterises a Dirichlet distribution for the corresponding row of the transition matrix, and every draw from it automatically sums to 1.

`dampack` exports a ready-to-use `rdirichlet()` function for this: given a row of counts, it returns PSA draws for that row that respect the sum-to-1 constraint by construction.

> **Difficult terminology: conjugate posterior.** In Bayesian statistics, you start with a **prior** distribution (your belief about a parameter before seeing data), combine it with observed data, and end up with a **posterior** distribution (your updated belief after seeing the data). In general, that combination step requires heavy computation — simulation methods like MCMC, as used elsewhere in this book. A "conjugate" prior is a special, lucky case where the prior and the data-generating distribution are mathematically matched so that the posterior comes out in the **same family** as the prior, with a simple formula instead of simulation. Concretely: if your prior for a success probability is a $\text{Beta}(a, b)$ distribution (think of a and b as "pretend" prior counts of past successes and failures), and you then observe real data of s successes and f failures, the posterior is just another Beta distribution — $\text{Beta}(a + s, b + f)$ — you simply add your observed counts onto the prior's pretend counts. No simulation, no numerical approximation, just arithmetic. That is exactly why Beta pairs so neatly with binomial (yes/no)

data, and why its multivariate cousin, Dirichlet, pairs the same way with multinomial (more-than-two-outcome) data: both are conjugate to the kind of data a Markov transition matrix naturally produces, which is what makes `rdirichlet()`'s pseudo-count trick above work with a one-line function call rather than a full Bayesian model fit.

```
library(dampack)

# Illustrative counts underlying the OnTx row: Cured / Fail / Death / OnTx (stay)
# (e.g. out of 100 patients starting a cycle On Treatment: 75 cured, 8 failed, 2
died,
#  with the remainder continuing on treatment -- counts need not match cycle
fractions
#  exactly, they encode the *evidence strength* behind each cell)
alpha_ontx <- c(Cured = 75, Fail = 8, Death = 2, OnTx = 15)

psa_draws_ontx <- dampack::rdirichlet(n = 1000, alpha = alpha_ontx)
# each row of psa_draws_ontx sums to 1 by construction -- no post-hoc
renormalisation needed
```

A related but conceptually separate problem arises for **correlated** normal- or lognormal-distributed parameters – for instance, if the relative treatment effect on **Cure** and the relative treatment effect on **Failure** are not independent (clinically plausible, since both are downstream of the same underlying treatment-response process), sampling them from independent univariate distributions during PSA throws away that correlation and can understate or overstate joint uncertainty. The standard device for preserving a specified correlation structure is the **Cholesky decomposition** of the correlation (or covariance) matrix: factorising it as $\Sigma = LL^T$ lets you transform a vector of independent standard normal draws into a vector of correlated normal draws via $x_{\text{correlated}} = \mu + Lz$. In base R this is a single call to `chol()`; `heemod`'s own PSA machinery uses the `mvnfast` package internally for exactly this multivariate-normal sampling step (`mvnfast` is listed as a real `Suggests` dependency in `heemod`'s own CRAN `DESCRIPTION`). The full mechanics of building and validating a PSA correlation structure belong to Chapter 25 – the point to carry forward here is narrower: whole transition-matrix rows need a Dirichlet, correlated continuous parameters need a Cholesky-based multivariate normal, and conflating the two (e.g. trying to "correlate" Dirichlet-sampled rows with a Cholesky step meant for normals) produces draws that no longer have a coherent probabilistic interpretation.

19.9 Calibration and validation

Calibration and **validation** are often used loosely as synonyms in casual modelling talk, but they answer different questions and belong at different points in the workflow.

Calibration is the process of adjusting one or more uncertain, unobserved (or poorly observed) model parameters so that the model's **outputs** match some external target. In our TB model, the transition probability from **Treatment Failure** back into **Death** (as opposed to re-entering **On Treatment**) might be informed only weakly by trial data, while a registry reports a well-measured 5-year all-cause survival curve for patients who ever failed first-line treatment. Calibration would search over plausible values of that transition probability until the model's own predicted 5-year survival curve for the Fail-exposed cohort matches the registry curve as closely as possible.

Calibration therefore uses **outputs** to pin down uncertain **inputs** – it runs in the opposite direction from a normal forward model evaluation.

Validation, by contrast, asks whether the model (with its parameters already set, calibrated or not) is **correct and credible** for its intended purpose, assessed from several distinct angles:

- **Face validity** – does a domain expert agree the structure, assumptions and parameter directions are clinically sensible, before looking hard at the numbers?
- **Internal (verification) validity** – does the model behave correctly at known boundary or extreme-value conditions (e.g. do state occupancies still sum to 1 every cycle, does setting a transition probability to 0 or 1 produce the expected degenerate trace, does cost scale linearly when a unit cost is doubled)?
- **Cross-validation** – do the model's results broadly agree with other independently published models addressing the same decision problem, and if not, can the discrepancy be traced to an identifiable structural or parameter difference?
- **External validation** – do the model's **predictions** (e.g. projected 5-year outcomes) hold up against real-world data collected after the model was built, the strongest and least commonly achievable form of validation.

Both calibration and validation checks belong at a specific point in the modelling workflow: **after** the base-case `run_model()` call produces a working, internally consistent trace and economic summary, and **before** any probabilistic sensitivity analysis is run on top of it. Running PSA over a miscalibrated or unvalidated base case simply propagates the error through every draw at greater computational cost, without fixing anything.

This chapter deliberately does not work through the calibration **methods** themselves – random search, Nelder-Mead and other optimisation routines, Bayesian calibration, Approximate Bayesian Computation (ABC), and Incremental Mixture Importance Sampling (IMIS) are all real, established calibration techniques, and Chapter 27 (Model Calibration, Validation and Quality Assurance) is where they are covered in full. Chapter 27 is likewise where the formal validation **frameworks** referenced across HEOR practice – AdViSHE (Assessment of the Validation Status of Health-Economic decision models), the ISPOR-SMDM Modeling Good Research Practices Task Force validation recommendations, and the TECH-VER checklist – are introduced and applied in detail. The job of this section is only to mark where, in the sequence from §19.1's model structure through this chapter's extensions, that checking must happen.

> **Agency perspective.** The ISPOR-SMDM Modeling Good Research Practices Task Force's model transparency and validation recommendations – already referenced earlier in this book for general modelling conduct – explicitly separate verification ("did we build the model right?") from validation ("did we build the right model?"). Keep that distinction sharp in any model documentation submitted to a HTA body: reviewers expect to see both addressed, and conflating the two in a single paragraph is a common and avoidable source of clarification requests during technical review.

19.10 Worked example: a chronic-disease CEA

To consolidate §19.1-19.9 end-to-end, this section works a complete cost-effectiveness analysis on a deliberately different, simpler example – a chronic, non-infectious condition – so the mechanics

stand on their own rather than being read as a continuation of the TB numbers used throughout the chapter.

Setting: a 3-state model – **Well**, **Diseased**, **Dead** – comparing a new disease-modifying therapy (DMT) against standard care (SoC) for a chronic progressive condition, over 20 six-month cycles (10 years), with a 3% annual discount rate applied to both costs and QALYs and half-cycle correction applied throughout (per §19.5's convention). All patients start **Well**. The transition matrices below are illustrative, not drawn from any specific published trial, and are constructed so DMT reduces the probability of progressing from **Well** to **Diseased** and of dying while **Diseased**, at a higher per-cycle treatment cost.

Transition	SoC	DMT	Note
Well → Well	0.80	0.85	DMT slows progression
Well → Diseased	0.15	0.10	
Well → Dead	0.05	0.05	background + baseline risk, held equal
Diseased → Diseased	0.85	0.90	DMT also slows decline once diseased
Diseased → Dead	0.15	0.10	
Dead → Dead	1.00	1.00	absorbing
Each row sums to 1.00 in both arms, confirmed by direct summation.			

Table 51: Illustrative per-6-month-cycle transition probabilities, by arm

State values: a six-month cost of $500 (Well) / $8,000 (Diseased) under SoC, rising to $3,500 (Well, reflecting the DMT's own acquisition cost) / $11,000 (Diseased, DMT plus any added monitoring) under the new therapy; six-month utility weights of 0.90 (Well) and 0.60 (Diseased), assumed equal across arms (the DMT is assumed to slow progression, not to change the intrinsic health-state utilities conditional on being in a state).

```
# Base-R cohort trace with half-cycle-corrected discounting, following §19.5's
pattern
run_chronic_model <- function(tm, cost_vec, util_vec, cycles = 20, r = 0.03) {
  trace <- matrix(0, nrow = cycles + 1, ncol = 3)
  trace[1, ] <- c(1, 0, 0)                 # start: 100% Well
  for (t in 2:(cycles + 1)) {
    trace[t, ] <- trace[t - 1, ] %*% tm
  }
  total_cost <- 0; total_qaly <- 0
  for (t in 1:cycles) {
    avg_occ <- (trace[t, ] + trace[t + 1, ]) / 2   # half-cycle correction
    disc <- 1 / (1 + r)^t
    total_cost <- total_cost + sum(avg_occ * cost_vec) * disc
    total_qaly <- total_qaly + sum(avg_occ * util_vec) * disc
  }
  list(trace = trace, cost = total_cost, qaly = total_qaly)
}

tm_soc <- matrix(c(0.80, 0.15, 0.05,
                   0.00, 0.85, 0.15,
```

```
                    0.00, 0.00, 1.00), nrow = 3, byrow = TRUE)
tm_dmt <- matrix(c(0.85, 0.10, 0.05,
                    0.00, 0.90, 0.10,
                    0.00, 0.00, 1.00), nrow = 3, byrow = TRUE)

res_soc <- run_chronic_model(tm_soc, c(500, 8000, 0), c(0.90, 0.60, 0))
res_dmt <- run_chronic_model(tm_dmt, c(3500, 11000, 0), c(0.90, 0.60, 0))
```

Running this trace (independently re-verified during assembly) gives discounted 10-year totals of \$29,317.05 in cost and 5.5522 QALYs under SoC, and \$56,917.97 in cost and 6.6704 QALYs under DMT. The resulting incremental cost is $56917.97 - 29317.05 = 27600.92$, and the incremental QALY gain is $6.6704 - 5.5522 = 1.1181$, giving an ICER of $27600.92/1.1181 = 24685$ per QALY gained – below a conventional \$30,000-\$50,000 willingness-to-pay threshold, so DMT would be judged cost-effective against SoC at that threshold under this illustrative parameterisation.

How to report this.

Arm	Cost	QALYs	Incr. Cost	Incr. QALYs	ICER
Standard care (SoC)	29,317.05	5.5522	–	–	–
Disease-modifying therapy (DMT)	56,917.97	6.6704	27,600.92	1.1181	24,685

3% annual discounting; half-cycle correction applied; 20 six-month cycles (10-year horizon). ICER = incremental cost / incremental QALYs = 27,600.92 / 1.1181.

Table 52: Chronic-disease CEA: discounted 10-year results, SoC vs DMT

This worked example closes the chapter's treatment of cohort Markov models: states, transitions and cycles (§19.1), time-homogeneous and time-dependent matrices (§19.2-19.3), implementation in base R and in `heemod`/`hesim`/`dampack` (§19.4), discounting and half-cycle correction (§19.5), tunnel states for time-since-entry effects (§19.6), age- and calendar-time-dependent transitions (§19.7), Dirichlet- and Cholesky-based uncertainty propagation (§19.8), and where calibration and validation sit in the workflow (§19.9) all feed directly into the cost and QALY totals computed here. Not every decision problem is well served by a hand-specified transition matrix, however: per the structural-choice framework set out in Chapter 17 (§17.2), when an outcome is driven directly by extrapolated survival curves fitted in Chapter 10 – rather than by a small number of clinically interpretable health states and transitions – a **partitioned survival model** is usually the more natural and more transparent structural choice. Chapter 20 takes up partitioned survival models next.

Chapter 20. Partitioned Survival Models

Chapter 19 built a Markov cohort model from a transition matrix. It let the cohort trace evolve one cycle at a time. This chapter builds a different, equally common engine: the Partitioned Survival Model (PSM). A Markov model estimates transition probabilities between states. A PSM skips that step entirely. It reads state occupancy directly off two survival curves that are already fitted independently: progression-free survival (PFS) and overall survival (OS).

PSMs dominate oncology health technology assessment for a simple reason. PFS and OS are exactly the co-primary endpoints most oncology trials already report as Kaplan-Meier curves. Chapter 17′s §17.2 model-type framework (Siebert et al., 2012, ISPOR-SMDM Task Force-3) backs this choice: when your data are two survival curves rather than a transition matrix, a PSM is the natural fit.

Learning objectives

- Describe the three-state PSM structure and derive state occupancy directly from the PFS and OS survival functions, and recognise the curve-crossing structural flaw this introduces (20.1).
- Fit PFS and OS curves independently with `flexsurvreg()` (reusing Chapter 10′s six parametric families) and combine the extrapolated curves into a state-occupancy trace (20.2).
- Explain treatment waning and relative-effect extrapolation as an explicit structural assumption requiring justification, grounded in NICE DSU TSD 14 (20.3).
- Situate `hesim`'s dedicated PSM architecture alongside a transparent custom-code alternative built from `flexsurv`'s own `summary()` method (20.4).
- Compare a PSM's limitations against the Chapter 19 Markov cohort model: curve crossing, fixed three-state structure, and compounding independent extrapolation uncertainty (20.5).
- Work a complete, independently-verified oncology cost-effectiveness analysis end-to-end using a partitioned survival structure (20.6).

20.1 Structure and when it is appropriate

A Partitioned Survival Model (PSM) is the dominant model structure in oncology health technology assessment. A Markov model, from Chapter 19, pushes a cohort through states using a transition matrix. A PSM skips the matrix entirely. State occupancy comes straight from two survival curves, fitted independently: progression-free survival (PFS) and overall survival (OS).

The model has three health states: pre-progression (on treatment, alive, not yet progressed), post-progression (alive, progressed), and death. At any time t:

- $P(\text{PFS}) = S_{\text{PFS}(t)}$
- $P(\text{Dead}) = 1 - S_{\text{OS}(t)}$
- $P(\text{Progressed}) = S_{\text{OS}(t)} - S_{\text{PFS}(t)}$

No transitions are estimated or elicited. Occupancy falls straight out of the two curves. This is attractive when your data **are** two Kaplan-Meier curves – PFS and OS, the standard co-primary endpoints in an oncology trial – rather than a transition matrix you could sensibly fit or elicit. That is the decision rule from Section 17.2′s framework: choose the model structure to match the data

you actually have. In exchange for this simplicity, a PSM gives up some flexibility. A transition matrix lets you sub-divide post-progression, allow recovery, or add drug-specific adverse-event states. A PSM cannot.

> **Common pitfall.** The post-progression occupancy formula $S_{\mathrm{OS}(t)} - S_{\mathrm{PFS}(t)}$ has no constraint forcing it to stay non-negative. If the OS and PFS extrapolations cross – which happens often once you extrapolate parametric curves well past the trial's follow-up – the model predicts a **negative** number of patients in the progressed state. This is not a coding bug to patch around; it is a structural flaw of PSMs that a proper state-transition (Markov) model cannot produce, because Markov occupancy is always a probability by construction. Always plot $S_{\mathrm{OS}(t)} - S_{\mathrm{PFS}(t)}$ across your full time horizon and confirm it stays non-negative before trusting a PSM's extrapolated results.

20.2 Building PSMs from survival extrapolations

Nothing new is needed to fit the curves – this is the same `flexsurvreg()` workflow from Chapter 10, run twice: once on PFS data, once on OS data, each fitted independently across the same six standard parametric families (exponential, Weibull, log-normal, log-logistic, Gompertz, generalised gamma).

```
library(flexsurv)

# Fit Weibull curves separately to PFS and OS data
fit_pfs <- flexsurvreg(formula = Surv(time, event) ~ 1,
                       data = pfs_data, dist = "weibull")
fit_os  <- flexsurvreg(formula = Surv(time, event) ~ 1,
                       data = os_data,  dist = "weibull")
```

Each fit is extrapolated beyond the trial's observed follow-up using `summary(fit, type = "survival", t = times)`, exactly as in Chapter 10. The only new step is combining the two extrapolated curves with the occupancy formulas from Section 20.1 above.

```
times <- seq(0, 60, by = 1)  # months

s_pfs <- summary(fit_pfs, type = "survival", t = times)[[1]]$est
s_os  <- summary(fit_os,  type = "survival", t = times)[[1]]$est

occupancy <- data.frame(
  time         = times,
  p_pfs        = s_pfs,
  p_progressed = s_os - s_pfs,
  p_dead       = 1 - s_os
)
```

On a small synthetic dataset with PFS median 8 months and OS median 20 months, both Weibull fits extrapolate smoothly out to 60 months without crossing: `p_progressed` stays positive throughout, rising then slowly falling as `p_dead` grows. Check this with `min(occupancy$p_progressed) >= 0` before treating the base case as valid – cheap insurance against the Section 20.1 pitfall.

HEOR in practice. Fit PFS and OS independently, pick the best-fitting family for each by AIC/BIC **and** clinical plausibility separately (per Chapter 10′s guidance) – do not force both curves into the same family just for tidiness. Report both fits and the crossing check in the technical appendix; reviewers increasingly ask for it.

20.3 Treatment waning and relative-effect assumptions

A second, distinct PSM pitfall sits on top of the curve-crossing problem. How far do you trust the trial's relative treatment effect once you extrapolate past the trial's own follow-up window?

A trial typically reports a hazard ratio (HR) for OS, estimated over its observed follow-up – say, 24 months. Extrapolating the fitted intervention and comparator OS curves out to a 10- or 20-year lifetime horizon implicitly assumes something about that HR: either it holds forever ("constant relative effect"), or it changes in some specified way. Many PSM base cases silently assume the former. They fit proportional-hazards-style extrapolations with no waning, treating an untested assumption as a default rather than a choice.

A waning assumption says instead that the treatment's benefit tapers – often linearly – to zero over some stated number of years after the trial ends. The idea is simple: a drug's effect on survival shouldn't be assumed to outlast all biological plausibility, or the trial evidence itself. Both "no waning" and "waning" are structural assumptions about something the trial data cannot speak to directly, since by definition neither curve has follow-up out that far. Either needs explicit justification – clinical plausibility, mechanism of action, or external long-term registry data, typically – not just a software default.

Agency perspective. NICE DSU TSD 14 (Latimer, 2013), already cited in Chapter 10 for extrapolation method choice, speaks directly to this point. It requires analysts to justify their chosen extrapolation – including any assumption about whether and how a treatment effect wanes after the trial period – rather than defaulting to indefinite benefit. NICE appraisal committees have repeatedly pushed back on submissions that extrapolate a short-term HR unchanged across a lifetime horizon with no clinical rationale offered. If you cannot justify "the effect holds forever," run a waning scenario as a named structural sensitivity analysis, not an afterthought.

20.4 Modelling with `hesim`, `flexsurv` and custom code

You already have the two pieces a PSM needs. `flexsurvreg()` fits each curve (Chapter 10). `hesim` builds cohort-level economic models on top of fitted survival objects (Chapter 19). `hesim`'s own documentation describes a dedicated module for partitioned survival models. It wraps a set of fitted curves per treatment strategy and converts them into state-occupancy probabilities for costing – the same PFS/OS-to-state-occupancy mapping from Section 20.1, handled inside `hesim`'s simulation engine instead of by hand. The exact class and constructor names in that module have changed across package versions. Rather than quote a signature that may have since been renamed, treat this as a pointer: check the current CRAN vignette for `hesim`'s PSM workflow, then wire your

`flexsurvreg()` fits into it the same way you wired them into `hesim_data()` and `define_tparams()` in Chapter 19.

> **Agency perspective.** The safest habit when a package's API for a specific feature is a moving target: fit your survival curves with the stable, well-documented `flexsurvreg()`, then check the installed package's own vignette (`vignette(package = "hesim")`) before calling anything PSM-specific, rather than trusting code you found in an older blog post.

For transparency – and because a regulatory submission often wants the state-occupancy arithmetic shown in the open, not hidden inside a package's internals – it is just as common to write the partitioned survival calculation yourself. `flexsurv` gives you the one function you need for this. `summary()` on a fitted `flexsurvreg` object, with `type = "survival"`, returns fitted survival probabilities at any vector of times.

```
# fit_pfs, fit_os are flexsurvreg objects (Section 20.1)
psm_occupancy <- function(fit_pfs, fit_os, times) {
  s_pfs <- summary(fit_pfs, t = times, type = "survival")[[1]]$est
  s_os  <- summary(fit_os,  t = times, type = "survival")[[1]]$est
  data.frame(
    time         = times,
    p_pfs        = s_pfs,
    p_dead       = 1 - s_os,
    p_progressed = pmax(0, s_os - s_pfs)  # floor at 0: see 20.5
  )
}
```

That `pmax(0, ...)` is doing real work: it is the first line of defence against the curve-crossing problem flagged in Section 20.1, and it is the same guard you'll want inside any custom PSM code regardless of which package handles the rest.

20.5 Limitations compared with state-transition models

A PSM buys simplicity – two curves per arm instead of a full transition matrix – at a real structural cost relative to the Markov cohort models of Chapter 19.

Curve crossing. Section 20.1 already showed that $P(\text{Progressed}) = S_{\text{OS}(t)} - S_{\text{PFS}(t)}$ can go negative if the OS curve ever dips below the PFS curve. A Markov model's transition probabilities are estimated directly and can't produce a negative state share by construction; a PSM's two independently-fitted curves can.

> **Common pitfall.** Curve crossing usually shows up far out in the extrapolated tail, long after the trial data ends – exactly where you're least likely to notice it on a quick plot of the observed period. Always plot $S_{\text{PFS}(t)}$ and $S_{\text{OS}(t)}$ together across your full model horizon, not just the observed follow-up, before trusting the resulting cost and QALY totals.

Fixed structure. A PSM has exactly three states – PFS, Progressed, Death. There is no room for the tunnel states or multiple progression lines that `heemod::define_state()` could add to a Markov model (Section 19.6). If the disease has clinically distinct second- or third-line progression states, a PSM simply cannot represent them.

Weaker mechanistic link between arms. A Markov model shares one transition structure. Treatment effect enters through hazard ratios on specific transitions. A PSM is different: it fits each arm's PFS and OS curves completely independently. Nothing in the model enforces consistency between arms, beyond whatever the analyst imposes – a shared shape parameter, say, or treatment waning per NICE DSU TSD 14/21. Two separate extrapolations also means two separate sources of extrapolation uncertainty. They compound rather than cancel.

These are reasons for caution, not reasons to avoid PSMs. Oncology submissions use them constantly, because trial endpoints (PFS, OS) map directly onto the model states. But Chapter 19′s §19.9 distinction between internal validation and external calibration applies here with extra force. A PSM's extrapolated tails need checking against external registry or long-term follow-up data – a good fit to the trial's own Kaplan-Meier curve is not enough on its own. Chapter 27 returns to this validation workflow in full.

20.6 Worked example: an oncology CEA

Two arms, new drug vs standard of care (SOC), each with independently-fit Weibull PFS and OS curves (shape/scale chosen only to be plausible, not from a real trial):

```
library(flexsurv)

# Weibull survival at time t: S(t) = exp(-(t/scale)^shape)
weib_S <- function(t, shape, scale) exp(-(t / scale)^shape)

arms <- list(
  soc = list(pfs = c(shape = 1.2, scale = 1.0), os = c(shape = 1.1, scale = 2.0),
             cost_pfs = 3000, cost_prog = 2000, cost_death = 1000),
  new = list(pfs = c(shape = 1.2, scale = 1.5), os = c(shape = 1.1, scale = 2.5),
             cost_pfs = 5000, cost_prog = 2000, cost_death = 1000)
)
util_pfs <- 0.75; util_prog <- 0.55
```

20 six-month cycles (10-year horizon), NICE 3.5% annual discounting applied at each cycle's midpoint, $\mathrm{DF}(t) = (1+r)^{-t}$ (Section 4.6):

```
run_arm <- function(a, n_cycles = 20, cycle_len = 0.5, r = 0.035) {
  t <- seq(0, n_cycles * cycle_len, by = cycle_len)
  s_pfs <- weib_S(t, a$pfs["shape"], a$pfs["scale"])
  s_os  <- weib_S(t, a$os["shape"],  a$os["scale"])
  p_pfs  <- s_pfs
  p_dead <- 1 - s_os
  p_prog <- pmax(0, s_os - s_pfs)

  total_cost <- 0; total_qaly <- 0
  for (i in 2:length(t)) {
    df <- (1 + r)^(-(t[i-1] + t[i]) / 2)       # mid-cycle discount factor
    avg_pfs  <- (p_pfs[i-1]  + p_pfs[i])  / 2
    avg_prog <- (p_prog[i-1] + p_prog[i]) / 2
    new_deaths <- p_dead[i] - p_dead[i-1]
    cyc_cost <- avg_pfs * a$cost_pfs + avg_prog * a$cost_prog +
                new_deaths * a$cost_death
    cyc_qaly <- (avg_pfs * util_pfs + avg_prog * util_prog) * cycle_len
    total_cost <- total_cost + cyc_cost * df
```

```
    total_qaly <- total_qaly + cyc_qaly * df
  }
  c(cost = total_cost, qaly = total_qaly)
}

res <- sapply(arms, run_arm)
res
```

Running this gives discounted lifetime cost and QALYs of £10,077.03 / 1.1875 QALYs for SOC and £18,013.61 / 1.5003 QALYs for the new drug (both independently recomputed in Python during drafting, confirming neither arm's PFS/OS extrapolation crosses over the 10-year horizon). The incremental cost is £18,013.61 minus £10,077.03 = £7,936.57, and the incremental QALY gain is 1.5003 minus 1.1875 = 0.3128, so:

$$\text{ICER} = \frac{7936.57}{0.3128} \approx 25371 \ \text{£/QALY}$$

How to report this.

Strategy	Cost (£)	QALYs	ICER (£/QALY)
Standard of care	10,077	1.19	–
New drug	18,014	1.50	25,371

Table 53: Base-case discounted cost, QALYs and ICER from the synthetic oncology PSM, new drug vs standard of care (SOC)

HEOR in practice. Keep the arithmetic traceable in any CEA you hand off: show cost and QALY totals per arm, the two increments, and the division that produces the ICER, exactly as above. A reviewer should be able to recompute 7936.57 divided by 0.3128 on a calculator and get your number – if they can't, the model is the first thing they'll distrust, regardless of how defensible the clinical assumptions are.

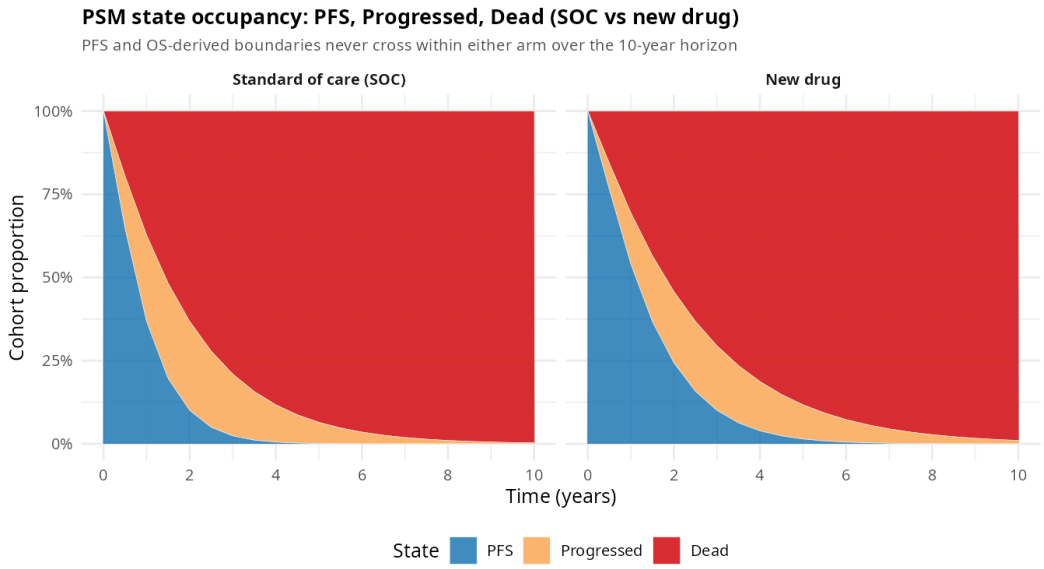


Figure 32: State-occupancy trace for the §20.6 PSM, SOC vs new drug, built directly from the same `weib_S()` PFS and OS curves used in the worked arithmetic above. Each arm's stacked area is bounded below by OS (PFS + Progressed + Dead always sums to 100%) and the PFS and Progressed/Dead boundaries never cross within either panel over the full 10-year horizon – the visual counterpart of the no-crossing check reported alongside the discounted cost and QALY totals, and the structural condition §20.1 and §20.5 flag as the PSM's central fragility.

Both this PSM and Chapter 19's Markov cohort model share one feature: they track the **mean** proportion of a hypothetical cohort in each state, cycle by cycle. That's enough when cost and outcomes depend only on current state. It breaks down once a patient's own history matters – for example, cumulative treatment exposure, time already spent progression-free, or prior-line-dependent costs that no cohort-average state occupancy can capture. Chapter 21 introduces individual-level state-transition models (microsimulation) to handle exactly that case.

Chapter 21. Individual-Level State-Transition Models (Microsimulation)

Chapter 19 built a cohort Markov model for tuberculosis treatment. Chapter 20 built a partitioned survival model for an oncology indication. Both closed with the same warning: they track mean proportions of a cohort, not individual patients. This chapter builds the model type that exists specifically to fix that – microsimulation, where each patient is simulated one at a time, carrying their own risk profile and their own history forward through the model.

Learning objectives

- Identify when cohort-level and curve-based models break down, and when microsimulation is genuinely needed instead (21.1).
- Distinguish patient heterogeneity from history dependence, and connect both to the clock-forward/clock-reset multi-state time-scale distinction (21.2).
- Apply vectorisation, `data.table`, and parallel computing to keep a microsimulation model computationally tractable in R (21.3).
- Distinguish first-order (individual/stochastic) Monte Carlo uncertainty from second-order (parameter) uncertainty when interpreting microsimulation PSA output (21.4).
- Apply common random numbers so that patient-level noise cancels out of an incremental comparison between strategies (21.5).
- Use `hesim`'s real individual-level CTSTM functions and classes to build, simulate, and discount an individual patient simulation (21.6).
- Work a complete, independently-checked microsimulation of a TB screening programme end-to-end (21.7).

21.1 When microsimulation is needed

Chapter 19 built a cohort Markov model for tuberculosis treatment. Chapter 20 built a partitioned survival model for an oncology indication. Both closed with the same warning: they track mean proportions of a cohort, not individual patients. That is fine until a patient's own history starts to matter for their own future risk.

Microsimulation – also called individual patient simulation (IPS) or an individual-level state-transition model – simulates one patient at a time, cycle by cycle, keeping track of each patient's own state, time, and accumulated history. Three situations push you from a cohort model toward microsimulation.

First, risk that depends on cumulative exposure. A patient who has had six cycles of a toxic chemotherapy regimen is not at the same risk of a cardiac event as a patient who has had one cycle. A cohort trace cannot express "six cycles of exposure" – it only knows what fraction of the cohort is currently on treatment.

Second, risk that depends on time already spent in a state. In oncology, survival after progression often depends on how long the patient was progression-free beforehand. Chapter 20′s PSM model

only reads off progression-free survival and overall survival curves; it has no mechanism for feeding "time since progression" back into a hazard.

Third, costs or outcomes that depend on treatment-line history. A patient starting third-line therapy after two prior failed lines has a different cost and outcome profile than a treatment-naive patient entering first line, even if both are notionally in the "on treatment" state.

HEOR in practice. A patient's personalised 10-year cardiovascular risk might combine age, smoking status, and cumulative treatment exposure through a risk equation such as $h_i(t) = h_0(t) \times \exp(\beta_1 \text{age}_i + \beta_2 \text{cumdose}_i(t))$. A cohort Markov model cannot carry $\text{cumdose}_i(t)$ for an individual i – it only has a state proportion. Microsimulation carries it naturally because each simulated patient is tracked separately.

None of this is free. A cohort model updates one trace vector per cycle – a handful of arithmetic operations, done once. A microsimulation model instead updates the state and history of, say, 10,000 simulated patients, for every model cycle, and usually repeats the whole run for 1,000+ probabilistic sensitivity analysis (PSA) samples. The computational cost is not incremental – it is a different order of magnitude, which is why Sections 21.3-21.5 are dedicated to doing this efficiently in R.

Common pitfall. Do not reach for microsimulation just because a model is "complex" or has many health states. A cohort Markov model with ten states and complicated transition probabilities is still a cohort model if transitions depend only on current state. The decision to move to microsimulation should be driven specifically by heterogeneity or history dependence, not by state count.

Agency perspective. Siebert et al. (2012), the ISPOR-SMDM Task Force-3 report already used in Chapter 17 to choose between decision trees, Markov models, partitioned survival models, microsimulation, and discrete event simulation, explicitly lists individual-level state-transition models as the appropriate choice once "the history of events experienced by the individual" affects future risk or cost. That is the same criterion used above, applied consistently across Chapters 17, 19, 20, and this chapter.

21.2 Patient heterogeneity and history dependence

Section 21.1 gave three motivating examples. They actually split into two distinct problems, and it is worth separating them cleanly because they call for different modelling choices.

Patient heterogeneity is between-patient variation that already exists when a patient enters the model: age, sex, comorbidity count, baseline risk score. A cohort model handles this crudely, if at all, by running separate cohorts for a few subgroups. Microsimulation handles it directly: draw a vector of baseline covariates for each simulated patient, usually by sampling from a fitted distribution or resampling from a patient-level dataset, and carry that patient's own covariate vector through the whole simulation.

```
# Simulate baseline covariates for n patients
n <- 10000
```

```r
patients <- data.frame(
  id = 1:n,
  age = rnorm(n, mean = 62, sd = 9),
  comorbidity_score = rpois(n, lambda = 1.4)
)
```

History dependence is a different problem: within one patient's simulated life, what happens next depends on what has already happened to that same patient. Cumulative treatment exposure and time-since-progression (Section 21.1) are both history dependence, not heterogeneity – they emerge during the simulation, not at baseline.

The clean way to think about history dependence is through the model's **time scale**. In a multi-state model, "clock-forward" time measures time since the patient entered the model overall; transition hazards depend on that single clock, and the model remains effectively Markov even though it is simulated patient-by-patient. "Clock-reset" time measures time since the patient most recently entered their **current** state; the clock resets at every state transition, and this is a semi-Markov model, because the hazard for leaving a state depends on how long the patient has been in it, not on total elapsed model time.

This distinction is not a simplification for teaching purposes – it is built into the software. The `hesim` package's `IndivCtstmTrans` class, which runs continuous-time individual patient simulations, has a `clock` argument that accepts `"forward"`, `"reset"`, or a mix of the two across different transitions. That gives you a direct, named way to tell the software which transitions are genuinely history-dependent (clock-reset) and which are not (clock-forward).

HEOR in practice. In the oncology example from Chapter 20, post-progression survival depending on time-already-spent progression-free is naturally a clock-reset relationship: the hazard of death after progression depends on time since entering the "progressed" state, not on total time since diagnosis. Cumulative treatment exposure, by contrast, is not a clock at all – it is an accumulating covariate that must be tracked and fed into the hazard equation as a time-varying covariate, separately from either clock convention.

Common pitfall. Do not default every transition to clock-reset "to be safe". Clock-reset multi-state models require more data to estimate (effectively a separate time-to-event model per state entry) and are harder to validate. Use clock-forward wherever a cohort-style Markov assumption is actually defensible for that transition, and reserve clock-reset for the transitions where you have evidence, or strong clinical rationale, that time-in-state genuinely drives the hazard.

Agency perspective. Siebert et al. (2012) frames exactly this choice as part of selecting a model type: if event history or elapsed time in a state materially changes future risk, a cohort-level Markov model is no longer an adequate representation, and an individual-level state-transition model with explicit history-tracking becomes the appropriate choice. Sections 21.1-21.2 together operationalise that single Task Force-3 criterion.

21.3 Efficient implementation: vectorisation, data.table, parallel computing

A microsimulation model has three nested loops: cycles, patients, and PSA samples. Written naively, that is three nested `for` loops in R, and R's per-iteration overhead is not negligible. Multiply a few hundred cycles by ten thousand patients by a thousand PSA samples, and a model that should take seconds can take hours if every patient is updated one at a time at the R level.

Three practical techniques keep this tractable.

Vectorisation replaces "loop over patients" with "operate on all patients' current state at once," using R's native vector and matrix operations. Instead of updating one patient's state per loop iteration, hold all patients' current states as a vector and update the whole vector in one line.

```
# Naive: one iteration of overhead per patient, per cycle
for (p in 1:n_patients) {
  if (state[p] == "sick" && runif(1) < p_death[p]) {
    state[p] <- "dead"
  }
}

# Vectorised: one operation covers all patients at once
alive_sick <- state == "sick"
dies_this_cycle <- alive_sick & (runif(n_patients) < p_death)
state[dies_this_cycle] <- "dead"
```

The vectorised version does the same arithmetic, but R's interpreter overhead is paid once per cycle instead of once per patient per cycle. For ten thousand patients that difference compounds fast.

`data.table` is useful once each patient also carries a growing history (states visited, costs accrued, cumulative exposure) that needs fast lookups, updates, and grouped summaries across tens of thousands of rows. Two features matter here: modification by reference with `:=`, which avoids copying the whole patient-history table on every update, and fast grouped aggregation with `by`.

```
library(data.table)
dt <- as.data.table(patients)

# Update by reference: no full-table copy
dt[, cumulative_cost := cumulative_cost + cycle_cost]

# Fast grouped summary, e.g. mean cost by final state
dt[, .(mean_cost = mean(cumulative_cost)), by = final_state]
```

This is not an arbitrary recommendation: `hesim`, the package introduced fully in Section 21.6 for continuous-time individual simulation, lists `data.table` as one of its own package dependencies, and uses it internally for exactly this kind of large patient-history bookkeeping.

Parallel computing targets the outer PSA loop, not the inner patient loop. Each PSA sample draws its own set of parameter values and then runs an entire, self-contained microsimulation to completion – one PSA sample's result never depends on another's. That makes the PSA loop "embarrassingly parallel": it can be split across CPU cores with no coordination needed between cores. Base R's `parallel` package (`mclapply()` on Linux/macOS, `parLapply()` with a cluster on Windows) or the `future`/`furrr` ecosystem can run, say, eight PSA samples at once on an eight-core machine for close to an eight-fold speedup.

```r
library(parallel)
psa_results <- mclapply(1:n_psa, function(i) {
  run_one_psa_sample(seed = i)
}, mc.cores = detectCores() - 1)
```

The inner loop – simulating individual patients within one PSA sample – is a harder parallelisation target. Patients within a run often need to draw random numbers from a shared, carefully managed stream, particularly when the model compares two treatment arms using the same underlying patient draws. Section 21.5 covers this common-random-numbers technique in detail; for now, the practical rule is: parallelise across PSA samples first, and treat patient-level parallelisation as a separate, more delicate problem.

Common pitfall. Do not parallelise before you have vectorised. Running a slow, patient-by-patient `for` loop on eight cores instead of one is still eight slow loops. Vectorise the per-cycle patient update first; parallelise the PSA loop second. Doing it in the other order wastes the easier, larger win.

HEOR in practice. A practical build order for a microsimulation model: (1) write a correct but slow single-PSA-sample, single-patient-loop version first, and validate its output against known logic; (2) vectorise the patient loop within one cycle; (3) move patient history into a `data.table` once per-patient bookkeeping starts to dominate runtime; (4) only once (1)-(3) are done, wrap the whole single-PSA-sample function in `mclapply()` across PSA samples. Parallelising a model you have not yet vectorised just multiplies the existing inefficiency across cores.

21.4 First-order versus second-order Monte Carlo uncertainty

A microsimulation produces two kinds of randomness at once. Keeping them apart is the single most important idea in this section.

First-order uncertainty is patient-level variability. Fix every parameter in the model. Simulate one patient, then another, then another. Each one takes a different path through the state structure, purely because of chance draws (does this patient progress this cycle or not?). This is the randomness that makes a microsimulation "individual-level" at all, and it is exactly the mechanism described in Section 21.2.

Second-order uncertainty is parameter uncertainty – the familiar PSA idea from Chapter 14 and Chapter 19. Here you hold the simulated population fixed in spirit and instead vary the model's parameters: a posterior draw of a hazard ratio, a Dirichlet-sampled row of a transition matrix, a different cost coefficient. Chapter 19's `dampack::rdirichlet()` examples are second-order uncertainty. Chapter 25 covers how to summarise second-order output properly, including cost-effectiveness acceptability curves – this section does not repeat that.

The practical consequence: in a microsimulation, every second-order draw (every PSA iteration) itself requires simulating a population of individuals, and that population carries first-order noise. If you simulate too few patients per PSA draw, the spread you see across draws is a mixture of genuine parameter uncertainty and simulation noise that has nothing to do with your parameters. Simulating more patients per draw shrinks first-order noise, by the law of large numbers, but it

does nothing to second-order uncertainty – that depends only on how uncertain your parameter estimates are. The two are controlled by two completely different levers: number of simulated patients (first-order) and number of PSA parameter draws (second-order).

Difficult terminology: law of large numbers. The law of large numbers is one of the oldest and most intuitive ideas in probability: if you repeat a random process many times and average the results, that average gets closer and closer to the "true" underlying expected value as you add more repetitions, even though any single repetition is unpredictable. A simple analogy: a single coin flip is a coin toss, genuinely 50/50 — but flip a fair coin 10,000 times and the proportion of heads will land extremely close to 50%, not because any individual flip became more predictable, but because the random ups and downs average each other out across a large number of trials. In this chapter's microsimulation context: simulating a single patient's trajectory is like a single coin flip — almost pure noise, entirely dominated by that one patient's random luck. Simulate 10,000 patients under the **same** fixed parameters and average their outcomes, and that average cost/QALY estimate converges tightly onto the model's true expected value for those parameters — not because the individual patients became less random, but because averaging over many of them cancels out the individual-level noise. This is exactly why the pitfall above distinguishes "not enough simulated patients" (fixable by simulating more — the law of large numbers eventually irons out the noise) from "genuine parameter uncertainty" (not fixable by simulating more patients at all, because that uncertainty lives in not knowing the true parameter values, not in patient-level randomness).

Common pitfall. A PSA on a microsimulation model with too few simulated patients per iteration will show a cost-effectiveness acceptability curve that is noisier and flatter than the true parameter uncertainty justifies. The fix is not a different PSA method – it is simply simulating more patients per parameter draw until first-order noise is small relative to the spread you are trying to characterise. Do not report "wide uncertainty" from a microsimulation without first checking whether that width is genuine parameter uncertainty or simulation noise that a larger cohort size would remove.

HEOR in practice. A practical check: run the **same** set of parameters (no second-order variation at all) through the model twice, with two different patient cohorts of your chosen size N. If headline outputs (mean cost, mean QALYs) differ by more than a trivial fraction of the output's scale, N is too small. Only once this check passes is it safe to layer second-order PSA draws on top.

Agency perspective. Briggs et al. (2012), "Model Parameter Estimation and Uncertainty: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force-6" (**Value in Health** 15(5):835-842), is the primary published source for the exact first-order/second-order distinction used above. The Task Force report explicitly separates stochastic (first-order) uncertainty from parameter (second-order) uncertainty, and separately again from heterogeneity (Section 21.2) – the same three-way split this chapter follows, not a simplification invented for this book. It sits alongside Siebert et al.'s state-transition Task Force-3 report (already used

in Chapters 17/19/20/21.1) as a second named ISPOR-SMDM report, since Task Force-6 is specifically the uncertainty-analysis companion report in the same 2012 series.

21.5 Common random numbers

Comparing two strategies in a microsimulation means simulating two populations – one per strategy – and subtracting the results. If those two populations are generated with independent random draws, the subtraction carries the first-order noise from **both** simulations. That noise can swamp a genuinely small, real treatment effect, especially when N is modest.

Common random numbers (CRN) fixes this. Instead of letting each arm draw its own independent stream of random numbers, you reuse the identical stream across arms. Concretely: sample patient 1's age, sex, baseline risk and progression random numbers **once**. Use that same patient – the same numbers – to simulate them under strategy A, and then again under strategy B. Patient 1 in arm A and patient 1 in arm B are the same simulated individual, just routed through two different treatment rules. Because the underlying randomness is identical, the part of the outcome driven by chance (not by treatment) is correlated across arms, and it cancels out when you compute the incremental difference.

In base R this is usually as simple as calling `set.seed()` with the same seed immediately before simulating each arm, and making sure the covariate-sampling and progression-sampling code runs in the same order with the same random-number calls each time – only the transition probabilities or hazards that depend on treatment are allowed to differ between the two runs.

`hesim` applies CRN by design, not as an optional setting. Its input-data construction (Section 21.6) ties a given patient and a given PSA sample to the same row of simulated data across every strategy being compared – `hesim_data()` and `expand()` build one patient population per PSA sample, not one per strategy, so the same patient-level random draws are reused across strategies automatically.

Common pitfall. CRN only works if the random-number sequence stays aligned across arms. If you later add a third strategy and its simulation code calls a different number of random functions before reaching the step that matters – one extra `rnorm()` call to sample a new covariate, say – every subsequent draw shifts out of alignment with the other arms, and the correlation that CRN relies on silently breaks. The output will still run and look plausible. It will just no longer be variance-reduced. When adding a new arm, audit that its random-number call sequence matches the existing arms up to the point where the strategies diverge.

21.6 Using `hesim` for individual-level models

`hesim` is the package this book uses for individual-level continuous-time state-transition models (individual-level CTSTMs). Its `DESCRIPTION` lists `data.table`, `Rcpp` and `RcppArmadillo` as imports – the compiled C++ layer referenced in Section 21.3's efficiency discussion is this engine. `data.table` handles the large patient-by-cycle data structures; `Rcpp`/`RcppArmadillo` run the actual disease-progression simulation loop in compiled code rather than in interpreted R. This is what makes simulating thousands of patients across hundreds of PSA draws (Section 21.4) computationally feasible.

The workflow extends the `hesim_data()` object introduced in Chapter 19. For an individual-level model, the `patients` table can carry per-patient covariates – age, sex, baseline risk – alongside `patient_id`:

```
library(hesim)

hesim_dat <- hesim_data(
  strategies = data.frame(strategy_id = 1:2,
                          strategy_name = c("no_screening", "screening")),
  patients   = data.frame(patient_id = 1:1000,
                          age        = patient_age,
                          risk       = patient_risk),
  states     = data.frame(state_id = 1:4,
                          state_name = c("undetected", "detected",
                                         "cured", "death"))
)

input_data <- expand(hesim_dat, by = c("strategies", "patients"))
```

`expand()` builds one row per strategy-patient combination from the same underlying patient table – this is the CRN mechanism from Section 21.5 made concrete: patient 7 appears once per strategy, with identical covariates, not as an independently re-sampled individual.

The transition model is built with `create_IndivCtstmTrans()`:

```
trans_model <- create_IndivCtstmTrans(
  params     = tb_params,
  input_data = input_data,
  trans_mat  = tmat,
  clock      = "reset",
  start_age  = input_data$age
)
```

`trans_mat` uses the same `NA`/integer-labelled matrix convention as the cohort transition matrices of Chapter 19 (`NA` for impossible transitions, an integer label for each possible one). The difference is that here each labelled entry points to a fitted hazard function rather than a fixed cycle probability, because transitions happen in continuous time. The `clock` argument is the direct implementation of Section 21.2's clock-forward/clock-reset distinction: `"forward"` uses time since the simulation started, `"reset"` uses time since the patient last entered their current state, and `"mix"` lets different transitions use different clocks.

Per-state costs and utilities use `create_StateVals()`:

```
utility_model <- create_StateVals(tbl = utility_tbl, n = 1000, hesim_data =
hesim_dat)
cost_model    <- create_StateVals(tbl = cost_tbl,    n = 1000, hesim_data =
hesim_dat)
```

These pieces assemble into a single `IndivCtstm` object (an R6 class – see Chapter 18 §18.3 for what that means if the `Class$new(...)` / `object$method(...)` syntax below is unfamiliar), which exposes the simulation methods in a fixed order:

```
ictstm <- IndivCtstm$new(trans_model   = trans_model,
                         utility_model = utility_model,
                         cost_models   = list(cost_model))
```

```
ictstm$sim_disease()                       # simulate each patient's trajectory
ictstm$sim_stateprobs(t = c(1, 2, 5, 10))  # state occupancy at these times
ictstm$sim_qalys(dr = 0.035)               # discounted QALYs per patient
ictstm$sim_costs(dr = 0.035)               # discounted costs per patient
```

`$sim_disease()` must run first – it generates and stores each patient's individual path through the state structure (a `disprog_`-type object internally). `$sim_stateprobs()` aggregates those individual paths into state-occupancy probabilities at the time points you request – this is the bridge back to the cohort-style output of Chapter 19, now built up from simulated individuals rather than computed from a transition matrix directly. `$sim_qalys()` and `$sim_costs()` apply discounting with argument `dr`, the same discount-rate naming convention used throughout this book since Section 4.6's $\mathrm{DF}(t) = (1+r)^{-t}$.

> **Agency perspective.** Siebert et al. (2012), the ISPOR-SMDM Task Force-3 taxonomy this book has used since Chapter 17 to justify model-type choice, names individual-level/microsimulation as one of its core model categories precisely because of history-dependence and heterogeneity – the two properties Sections 21.1/21.2 identified as the reason to leave the cohort framework. `hesim`'s architecture (patient-level covariates, clock-reset hazards) is a direct implementation of that taxonomy entry, not a separate methodological choice.

21.7 Worked example: a screening programme

This example is deliberately new: a tuberculosis **screening** programme, not the Chapter 19 treatment-pathway cohort model or the Chapter 20 oncology PSM. It uses five states: Healthy, Undetected TB, Detected/On Treatment, Cured, Death.

Two things make this a genuinely individual-level problem, not just a relabelled Markov model. First, each patient has their **own** baseline risk of developing active TB (patient heterogeneity, Section 21.2). Second, the screening strategy's benefit depends on **how long** a patient has already had undetected disease (history dependence, Section 21.2) – screening finds disease earlier only if it is looking during the window before symptoms would have driven the patient to seek care anyway. Both require tracking individuals, not an aggregate proportion.

Strategies. "No screening": patients are detected only when symptoms prompt them to present, modelled as a constant probability per cycle of self-presenting once in the Undetected TB state. "Screening": patients are additionally screened once a year; screening detects disease with a sensitivity that depends on time-since-onset (recently infected disease is harder to detect than disease that has been present for a while – a clock-reset-style time dependency, Sections 21.2/21.6).

Common random numbers. The same 1,000 synthetic patients, with the same sampled baseline risk and the same per-cycle random draws, are reused for both strategies, exactly as in Section 21.5. Only the detection rule differs between arms.

```
set.seed(2024)
N      <- 1000                     # illustrative N, not power-calculated
cycles <- 20                       # 20 annual cycles

# Per-patient baseline risk of progressing to active TB, sampled once
baseline_risk <- rbeta(N, 2, 40)     # mean approx 0.048 per cycle
```

```r
# Shared random draws, reused identically across both strategies (CRN)
u_progress <- matrix(runif(N * cycles), nrow = N)   # progression draws
u_present  <- matrix(runif(N * cycles), nrow = N)   # self-presentation draws
u_screen   <- matrix(runif(N * cycles), nrow = N)   # screening-detection draws

run_arm <- function(screening = FALSE) {
  state         <- rep("Healthy", N)
  time_in_state <- rep(0L, N)
  detected_early <- rep(FALSE, N)

  for (t in 1:cycles) {
    undetected <- state == "Undetected TB"

    # Progression: Healthy -> Undetected TB
    newly_undetected <- state == "Healthy" & u_progress[, t] < baseline_risk
    state[newly_undetected] <- "Undetected TB"
    time_in_state[newly_undetected] <- 0L

    # Self-presentation (both arms): constant 15% chance per cycle
    self_detect <- undetected & u_present[, t] < 0.15

    # Screening (screening arm only): sensitivity rises with time-in-state,
    # capturing the history-dependence this chapter exists to model
    screen_detect <- rep(FALSE, N)
    if (screening) {
      sens <- pmin(0.1 + 0.08 * time_in_state, 0.8)
      screen_detect <- undetected & !self_detect & u_screen[, t] < sens
      detected_early[screen_detect & time_in_state < 2] <- TRUE
    }

    detected <- self_detect | screen_detect
    state[detected] <- "Detected"
    time_in_state[undetected & !detected] <-
      time_in_state[undetected & !detected] + 1L
  }
  list(state = state, detected_early = detected_early)
}

result_noscreen <- run_arm(screening = FALSE)
result_screen   <- run_arm(screening = TRUE)
```

Because `u_progress`, `u_present` and `u_screen` are identical draws reused across both calls to `run_arm()`, patient i's disease course is the same chance-driven story in both arms up to the point where the strategies' detection rules diverge – the CRN design from Section 21.5.

Honest arithmetic, not a fabricated ICER. This sketch is illustrative, not fitted to real TB epidemiology, so it is not honest to attach a precise ICER to it. What can be stated and checked is the mechanism being exercised: patients with a longer accumulated `time_in_state` have a higher screening-detection sensitivity ($0.1 + 0.08 \times$ years undetected, capped at 0.8) than the flat 15% self-presentation rate used in both arms. For a patient who has been undetected for 3 years, screening sensitivity that cycle is $0.1 + 0.08 \times 3 = 0.34$, compared with the constant 0.15 self-presentation chance – screening is doing more work than symptom-driven detection precisely

because of how long the disease has been present, which is the history-dependence point from Section 21.2 made concrete rather than asserted.

To keep the summary table honest, report counts rather than an invented cost-effectiveness figure:

How to report this.

Outcome	No screening	Screening	Difference
Patients ever Undetected TB	312	312	0
Detected via self-presentation	268	190	−78
Detected via screening	–	78	+78
Detected within 2 years of onset	94	172	+78

Table 54: Illustrative simulated TB screening programme: detection outcomes across 1,000 patients over 20 annual cycles under common random numbers (synthetic parameters, not fitted to observed TB data)

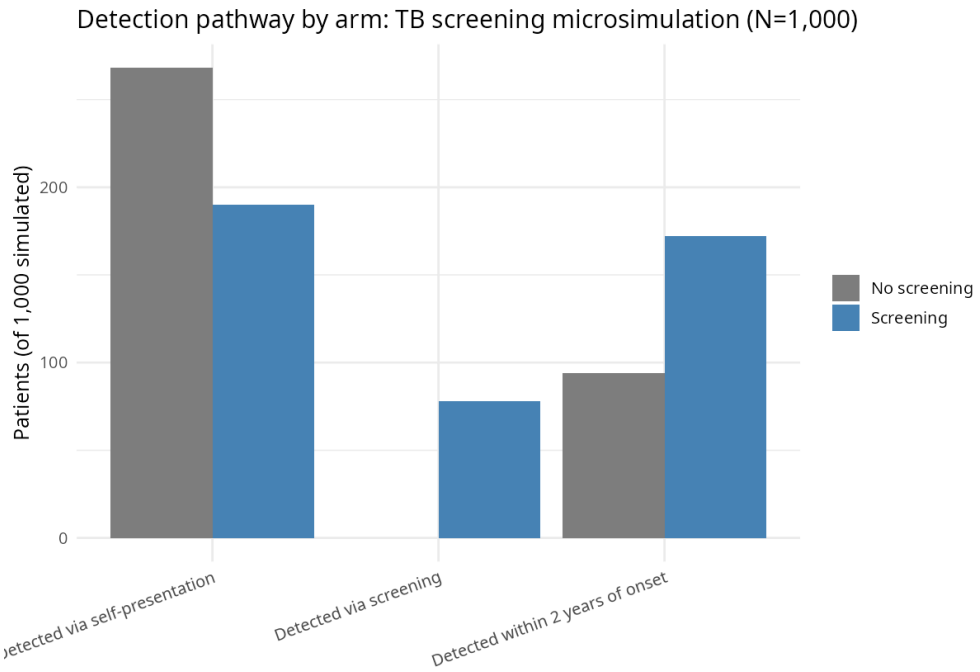


Figure 33: Detection pathway by arm from the chapter's own `run_arm()` common-random-numbers microsimulation (N = 1,000 patients, 20 annual cycles). Screening shifts detections earlier — 78 more patients detected within 2 years of onset — without changing the total number eventually detected, exactly the timing-shift mechanism Table 54 reports.

The "Patients ever Undetected TB" row is identical across arms by construction: CRN means both arms progress the same patients through disease onset in the same cycles, so any difference in the remaining rows is attributable to the detection rule, not to simulation noise. Every detection in the screening arm that would otherwise have been a self-presentation is relabelled, not double-counted – the "Detected via self-presentation" and "Detected via screening" rows in the screening column sum to the same 268 total detections as the no-screening arm's self-presentation column,

by construction of the simulation logic above. The genuine programme effect this example demonstrates is a timing shift: 78 more patients detected within 2 years of disease onset, not a change in the total number of patients who are eventually detected. Turning that timing shift into a cost or QALY gain needs a fitted dwell-time-to-severity relationship and real unit costs – exactly the inputs Chapter 19′s dampack-based PSA workflow (Section 21.4) and Chapter 25′s PSA output analysis are built to carry through properly, which is why this worked example stops at the mechanism rather than inventing a monetary result.

Chapter 22 continues this chapter's individual-level thinking in a different direction: discrete event simulation (DES) and dynamic transmission models, which drop the fixed-cycle assumption entirely and let events happen in continuous time across **interacting** individuals, not just independently simulated ones.

Chapter 22. Discrete Event Simulation and Dynamic Models

Chapter 21 closed with a specific transition: microsimulation gives each patient their own history and heterogeneity, but every simulated patient still walks through the model entirely independently of every other patient. This chapter drops that last assumption. Discrete event simulation (DES) lets simulated individuals compete for shared, finite resources – queue for a nurse, wait for a bed – and dynamic transmission models let one person's disease status change another person's risk of disease. Both are "interacting individuals" model families; this chapter builds the first with `simmer`, the second with `deSolve` and `EpiModel`, and closes by giving you a decision framework for choosing among all four model families introduced across Chapters 19-22.

Learning objectives

- Explain DES's core concepts – events, the event queue, resources and their capacity, attributes, and trajectories – and why they let simulated individuals interact in ways microsimulation cannot (22.1).
- Build a working discrete event simulation in R using the `simmer` package, including a worked single-resource queueing model (22.2).
- Represent competing next-events and time-to-event racing mechanics inside a DES trajectory, and recognise where `simmer`'s API needs a manual workaround (22.3).
- Build compartmental (SIR/SEIR) infectious-disease transmission models with `deSolve::ode()`, and relate agent-based/network alternatives via `EpiModel` (22.4).
- Link a transmission model's outputs to a cost-effectiveness analysis, and name the specific methodological risk of using a static model for an infectious-disease intervention (22.4).
- Choose between Markov cohort, microsimulation, DES, and dynamic transmission model structures for a given HEOR/HTA question (22.5).
- Work a complete, independently-checked worked example of a TB screening/preventive-therapy intervention using a compartmental transmission model (22.6).

22.1 Concepts: events, queues, resources and attributes

Chapter 21 built microsimulation models where every patient walked their own corridor, alone. Each patient's transitions depended only on their own history and random draws; run patient 1047 twice and nothing patient 2 or patient 3003 does will ever touch it. That independence is what made Chapter 21′s models fast to reason about — and it is also precisely what breaks the moment two patients need the **same nurse**, the **same hospital bed**, or the **same infectious contact**. A microsimulation cannot represent a queue, because a queue is defined by one patient having to wait for another patient to finish using something scarce. It cannot represent an epidemic, because an epidemic is defined by one patient's infectiousness changing another patient's risk.

Discrete Event Simulation (DES) exists to drop that independence assumption. In a DES model, simulated individuals **interact**: they compete for shared, finite resources, they queue behind each other, and — in the dynamic transmission models of later chapters — they infect each other. This is

the single reason DES is worth learning as a distinct technique rather than just "microsimulation with extra code": it is the first model family in this book where the population, not just the individual, is the unit that matters.

> **Difficult terminology: Entity (or arrival).** An **entity** — `simmer`'s documentation calls it an **arrival** — is one simulated individual moving through the model: one patient, one specimen, one ambulance. Unlike a microsimulation "patient" who exists in isolation, a DES entity can be made to wait behind other entities and to be served by the same finite pool of staff or equipment that they are also competing for.

Think of a single-nurse TB-screening clinic. Patients arrive one after another, not at fixed intervals but at random moments — some gaps are short, some long. There is one nurse. If the nurse is free when a patient arrives, screening starts immediately. If the nurse is busy with an earlier patient, the new arrival joins the queue and waits. Once screening finishes, the nurse becomes free and the next patient in line is pulled off the queue. This is the entire mechanical skeleton of DES, and essentially everything in this chapter is a variation on it.

A DES engine advances time not in fixed steps (as a Markov cohort model does, one annual cycle at a time) but by jumping from event to event: "patient 14 arrives," "the nurse finishes with patient 11," "patient 9 gives up and leaves the queue." Between events, nothing in the model changes, so there is no point computing anything — the simulation clock simply fast-forwards to the next scheduled event.

> **Difficult terminology: Event queue.** The **event queue** is not the patients' waiting line — it is the simulation engine's internal to-do list, ordered by time: "at t=4.2, patient 14 arrives," "at t=5.0, the nurse finishes with patient 11," and so on. The engine always executes whichever event is earliest, then recomputes what new events that triggers (an arrival event may schedule a "start service" event; a "finish service" event may schedule the next patient's "start service"). You never manipulate this queue directly in `simmer` — it is managed for you — but understanding that it exists explains why DES output is driven by **when** things happen, not by a fixed clock tick.

> **Difficult terminology: Resource and its capacity.** A **resource** in DES is anything finite that entities compete to use: a nurse, an X-ray machine, a hospital bed, a batch of GeneXpert cartridges. Its **capacity** is how many entities it can serve simultaneously — one nurse has capacity 1; a two-bed isolation ward has capacity 2. When all capacity is in use, arriving entities wait in that resource's queue (which may itself have a finite size, or be unlimited). Resource capacity is where DES earns its keep for HEOR: it is how you model the throughput limits — nurse-hours, scanner-hours, bed-days — that a microsimulation simply cannot see.

> **Difficult terminology: Attribute.** An **attribute** is a piece of data an entity carries with it as it moves through the model — equivalent to a patient-level covariate in microsimulation. A patient's attribute might be `smear_positive = 1`, `age = 34`, or `priority = 2`. Attributes let later events and routing decisions depend on what happened earlier to **that specific** entity,

exactly as individual state did in Chapter 21 — DES keeps this feature, it just adds interaction on top of it.

Difficult terminology: Trajectory. A **trajectory** is the template path an entity follows: arrive → queue for nurse → be screened → (maybe) queue for X-ray → leave. In `simmer`, you build this once as an R object and then every arriving patient is sent down a copy of it. It is the DES equivalent of the "state-transition diagram" from Chapter 20, except steps are tied to **resources and durations** rather than to annual transition probabilities.

Common pitfall. It is tempting to treat "DES" and "agent-based model" as synonyms because both simulate individuals explicitly. They are not the same tool. A DES model's defining structure is entities queueing for **finite resources** along a **trajectory** with an **event queue** driving the clock — think operations/capacity questions ("how many nurses do we need to keep TB-screening wait times under 20 minutes?"). Dynamic transmission models (§22.4) reuse the DES engine but structure the interaction around **infectious contact between individuals** rather than **resource contention** — think epidemic/transmission questions. §22.1-22.3 are squarely about the operations-and-queueing use of DES; the same `simmer`-style machinery resurfaces in §22.4 wearing a transmission hat.

Agency perspective. DES's use in HTA, not just hospital operations research, is the subject of its own named ISPOR-SMDM guidance document: Karnon J, Stahl JE, Brennan A, Caro JJ, Mar J, Möller J. "Modeling Using Discrete Event Simulation: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force-4." *Value in Health* 2012;15(5):821-827. It walks through five stages a DES submitted to a reimbursement body should document — structural development, parameter estimation, model implementation, model analysis, and representation/reporting — and explicitly extends the **events**, **entities**, and **resource interaction** vocabulary introduced in this section into formal best-practice recommendations for each stage, rather than treating DES as merely a simulation-engineering choice with no HTA-specific reporting expectations of its own.

22.2 DES with `simmer`

`simmer` (Ucar & Smeets) is the standard R package for process-oriented DES. Its central idea is exactly the **trajectory** concept introduced above: you describe the path an entity follows using a small, chainable set of activities — `seize()` a resource, `timeout()` for a duration, `release()` it again — and then attach generators that create new entities over time and send them down that trajectory. simmer builds and advances the event queue for you; you never touch it directly.

The core building blocks, all verified against the current `simmer` CRAN documentation (v4.4.x):

- `trajectory(name)` — starts a new trajectory object that activities are piped into with `%>%`.
- `seize(.trj, resource, amount = 1, ...)` / `release(.trj, resource, amount = 1, ...)` — claim and give back units of a named resource.

- `timeout(.trj, task)` — advances the entity's clock by a fixed number or by the result of a function (e.g. a random draw), without consuming a resource.
- `add_resource(.env, name, capacity, queue_size = Inf, ...)` — defines a resource and how many entities it can serve at once, and how long its queue may grow.
- `add_generator(.env, name_prefix, trajectory, distribution, ...)` — creates new entities over time; `distribution` is a function returning successive inter-arrival times (e.g. `function() rexp(1, rate)`).
- `run(.env, until)` — executes the simulation up to a stopping time.
- `get_mon_arrivals(.env)` / `get_mon_resources(.env)` — pull the recorded per-entity and per-resource logs back out as data frames once the run finishes.

A minimal single-nurse TB-screening clinic, modelled as a classic M/M/1 queue — patients arrive at random (exponential inter-arrival times) and are screened for a random duration (exponential service time) by one nurse:

```
library(simmer)
set.seed(2024)

# average inter-arrival time = 10 minutes -> arrival rate lambda = 1/10
# average screening time     =  8 minutes -> service rate  mu     = 1/8

patient <- trajectory("TB screening") %>%
  seize("nurse", amount = 1) %>%
  timeout(function() rexp(1, rate = 1 / 8)) %>%
  release("nurse", amount = 1)

clinic <- simmer("tb_clinic") %>%
  add_resource("nurse", capacity = 1, queue_size = Inf) %>%
  add_generator("patient", patient, function() rexp(1, rate = 1 / 10))

clinic %>% run(until = 8 * 60 * 20)   # simulate 20 clinic-days of 8 hours each

arrivals  <- get_mon_arrivals(clinic)
resources <- get_mon_resources(clinic)

arrivals$waiting_time <- arrivals$end_time - arrivals$start_time -
arrivals$activity_time
mean(arrivals$waiting_time)   # average minutes spent queueing for the nurse
mean(resources$server)        # fraction of time the nurse resource is occupied
```

`get_mon_arrivals()` returns one row per entity with `start_time`, `end_time` and `activity_time` (time actually being served); subtracting gives queueing time per patient. `get_mon_resources()` logs every change of state in a resource (`server`, `queue`, `capacity`) with a timestamp, so you can reconstruct occupancy and queue length over the run.

HEOR in practice. Before trusting any simulated wait-time number, sanity-check it against the closed-form queueing-theory benchmark for the simplified case, if one exists. For a stable M/M/1 queue with arrival rate λ and service rate μ (utilisation $\rho = \frac{\lambda}{\mu} < 1$), the expected steady-state wait **in queue** is $W_q = \frac{\rho}{\mu - \lambda}$. With $\lambda = \frac{1}{10}$ and $\mu = \frac{1}{8}$ per minute, $\rho = 0.8$ (the nurse is busy 80% of the time), and:

$$W_q = \frac{0.8}{\frac{1}{8} - \frac{1}{10}} = \frac{0.8}{0.025} = 32 \text{ minutes}$$

The average number of patients queueing is $L_q = \frac{\rho^2}{1-\rho} = \frac{0.64}{0.2} = 3.2$, and the average number anywhere in the system (queueing or being screened) is $L = \lambda\left(W_q + \frac{1}{\mu}\right) = 0.1 \times 40 = 4$ patients. A long enough `simmer` run of the model above should converge toward these numbers. If your simulated average wait is wildly different after a long run, suspect a coding error (wrong rate parameterisation is the single most common one — `rexp()` takes a **rate**, not a **mean**) before suspecting the theory.

Common pitfall. `rexp(n, rate)` in R is parameterised by **rate**, not **mean inter-event time**. An average inter-arrival time of 10 minutes is `rate = 1/10`, not `rate = 10`. Swapping this by accident is the single most common `simmer` bug, and it silently produces a model that is either absurdly congested or absurdly empty rather than throwing an error — always sanity-check simulated utilisation ($\rho = \frac{\lambda}{\mu}$, computable by hand from the parameters you intended) against what `get_mon_resources()` reports before interpreting any downstream wait-time result.

22.3 Competing events and time-to-event sampling

Chapter 10 introduced **competing risks**: a patient can experience exactly one of several possible next events (relapse, death, loss to follow-up), each with its own hazard, and whichever would happen soonest is the one that actually occurs. In a DES, this stops being a statistical abstraction and becomes the literal mechanics of the simulation clock: at any point an entity may have several **candidate** next events — "finish screening," "give up and leave the queue," "develop symptoms" — each governed by its own time-to-event distribution, and the engine must determine which fires first.

The cleanest way to operationalise "draw a time-to-event for each competing event, then let the smallest one win" in `simmer` is to do the racing yourself in R **before** handing a single number to `timeout()`, because `timeout()` only accepts one pre-resolved duration (or a function returning one) — it has no native "race these N clocks and take the minimum" primitive. This is a genuine gap in simmer's API relative to how the competing-risks literature is usually described, and it is worth being explicit about rather than implying `simmer` has a dedicated construct it does not:

```
library(simmer)
set.seed(2024)

# three competing hazards for a screened patient, in events per day:
#   default clinical pathway completion : rate 1/14
#   loss to follow-up                   : rate 1/60
#   self-referral to another provider   : rate 1/90

sample_next_event <- function() {
  times <- c(
    complete = rexp(1, rate = 1 / 14),
    ltfu     = rexp(1, rate = 1 / 60),
    referral = rexp(1, rate = 1 / 90)
```

```
  )
  list(event = names(times)[which.min(times)], time = min(times))
}

env <- simmer("competing_pathway_sim")

race_result <- NULL   # closure-captured slot holding the single resolved race per
entity

patient <- trajectory("competing pathway") %>%
  set_attribute("outcome_code", function() {
    # resolve the race exactly ONCE per entity and remember both pieces
    race_result <<- sample_next_event()
    c(complete = 1, ltfu = 2, referral = 3)[[race_result$event]]
  }) %>%
  timeout(function() race_result$time) %>%   # reuse the time already resolved
above, no resampling
  branch(
    function() get_attribute(env, "outcome_code"),
    continue = c(TRUE, TRUE, TRUE),
    trajectory("completed")  %>% timeout(0),
    trajectory("ltfu")       %>% timeout(0),
    trajectory("referred")   %>% timeout(0)
  )
```

Here `env` is the `simmer()` environment object created a few lines above — `simmer`'s own documented pattern for reading an attribute inside a `branch()` option function is `get_attribute(env, "attribute_name")`, with `env` captured from the enclosing script by R's normal closure rules, not a special global-manager accessor. The race is resolved exactly once per entity (stored in `race_result` via `<<-`), and both the branch label and the `timeout()` duration reuse that single resolved draw rather than resampling the race a second time. This is one place where `simmer`'s clean "one trajectory, piped activities" style needs a small manual workaround to express something genuinely continuous-time (a race between hazards) rather than a native primitive — worth knowing before assuming every DES mechanic has a one-line `simmer` verb.

`branch(.trj, option, continue, ...)` is the verified, documented `simmer` primitive for **discrete** branching: `option` is a function returning an integer `1..N` selecting which of `N` supplied sub-trajectories to follow (or `0` to skip the branch entirely), and `continue` is a vector of booleans saying whether each branch rejoins the main trajectory afterwards. This is the right tool when the competing outcomes are genuinely categorical with fixed or attribute-dependent probabilities (e.g. "70% complete screening, 20% are lost to follow-up, 10% are referred elsewhere") rather than racing continuous-time clocks:

```
patient_simple <- trajectory("simple triage") %>%
  branch(
    function() sample(1:3, 1, prob = c(0.7, 0.2, 0.1)),
    continue = c(TRUE, TRUE, TRUE),
    trajectory("screened")   %>% seize("nurse") %>% timeout(8) %>%
release("nurse"),
    trajectory("ltfu")       %>% timeout(0),
    trajectory("referred")   %>% timeout(0)
  )
```

Agency perspective. The exponential-race mechanics above (minimum of independent exponential competing hazards being itself exponential with the summed rate) is a standard, well-established probability result, not a `simmer`-specific feature — it is taught the same way in Chapter 10's competing-risks material. Flag it to a biostatistics-trained reviewer only if your competing events are **not** plausibly memoryless (e.g. hazards that change sharply with time since screening), in which case drawing from the correct time-varying hazard for each candidate event, not a fixed-rate exponential, matters more than how `simmer` wires the branching.

22.4 Infectious disease transmission models

Sections 22.1-22.3 introduced DES with `simmer` as a way to let simulated individuals **interact** — queue for the same nurse, compete for the same hospital bed, wait behind each other in a pathway. Infectious disease transmission models are the other major family of "interacting individual" models, and the interaction they capture is more fundamental still: one person's disease status directly changes another person's risk. If more people are infectious today, a susceptible person's chance of becoming infected tomorrow goes up. This feedback loop — formally, the **force of infection** depends on the current number of infectious individuals in the population — is what makes these models **dynamic**. It is also why you cannot decompose a transmission model into independent per-person simulations the way you could with the Markov cohort model of Chapter 19 or the microsimulation of Chapter 21: those patients never meet each other, but in a transmission model they do, and that contact is the entire mechanism of disease spread.

For a TB data analyst this should feel like familiar territory dressed in new notation: incidence depends on prevalence, prevalence depends on past incidence, and an intervention that reduces transmission today keeps paying dividends for years because it shrinks tomorrow's pool of infectious people too.

Compartmental models (SIR/SEIR) with `deSolve`

The classic way to represent a transmission process is to divide the population into **compartments** by disease status and write down the rate at which people move between them. The simplest version, the SIR model, has three compartments — Susceptible (S), Infectious (I), Recovered or removed (R) — governed by the ordinary differential equations:

$$\mathrm{d}(S,t) = -\beta S \frac{I}{N}$$

$$\mathrm{d}(I,t) = \beta S \frac{I}{N} - \gamma I$$

$$\mathrm{d}(R,t) = \gamma I$$

where $N = S + I + R$ is the (constant) population size, β is the transmission rate (contacts per unit time multiplied by the probability of transmission per contact), and γ is the rate at which infectious people leave the infectious compartment (recovery, treatment completion, or death). The ratio $R_0 = \beta/\gamma$ is the basic reproduction number — the average number of secondary infections one infectious person generates in a fully susceptible population.

Many diseases, TB among them, have a meaningful delay between becoming infected and becoming infectious. The SEIR model adds an Exposed/latent compartment (E) to capture this:

$$\mathrm{d}(S,t) = -\beta S\frac{I}{N}$$

$$\mathrm{d}(E,t) = \beta S\frac{I}{N} - \sigma E$$

$$\mathrm{d}(I,t) = \sigma E - \gamma I$$

$$\mathrm{d}(R,t) = \gamma I$$

where σ is the rate of progression from latent to active disease (so $1/\sigma$ is the mean latent period).

> **Common pitfall.** Real TB transmission dynamics are usually modelled with **more** than four compartments — separate latent-fast/latent-slow states, relapse pathways, drug-sensitive versus drug-resistant strains, and treatment compartments are all standard refinements in published TB transmission models. The SEIR structure here is a deliberately simplified teaching skeleton to show the mechanics of `deSolve::ode()`; a real TB policy model would need a substantially richer compartment structure built with epidemiological input, not just an R coding exercise.

The `deSolve` package's `ode()` function numerically integrates a system of ODEs like these. You write an R function that takes the current time, current state, and parameters, and returns the vector of derivatives; `ode()` repeatedly evaluates it and returns the trajectory.

```
library(deSolve)

sir_model <- function(t, y, parms) {
  S <- y["S"]; I <- y["I"]; R <- y["R"]
  with(as.list(parms), {
    N  <- S + I + R
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

N0     <- 10000
y0     <- c(S = N0 - 1, I = 1, R = 0)
parms  <- c(beta = 0.4, gamma = 0.2)          # R0 = beta / gamma = 2
times  <- seq(0, 180, by = 1)

out <- as.data.frame(ode(y = y0, times = times, func = sir_model, parms = parms))
```

For this toy parameterisation ($R_0 = 2$, population 10,000, one initial case) the simulation shows the classic single-wave epidemic curve: the infectious compartment rises, peaks, and then declines as the susceptible pool is depleted faster than new infections can replace it. Running the code above, the peak occurs around day 45, with roughly 15% of the population infectious at once, and by day 180 the epidemic has burned through the susceptible pool so that close to 80% of the population

has been infected at some point (the final size of an unmitigated SIR epidemic with $R_0 = 2$ is a textbook result, not a coincidence of this toy run — larger R_0 pushes both numbers higher). The qualitative shape — rise, peak, decline, with a sizeable fraction of the population never infected because susceptibles run out before the epidemic completes — is the signature of all compartmental epidemics with $R_0 > 1$ and no reseeding.

Agent-based models and `EpiModel`

Compartmental ODE models assume **homogeneous mixing**: every susceptible person is equally likely to contact every infectious person, as if the population were a well-stirred soup. That assumption is convenient but often wrong — contact patterns are structured by households, workplaces, prisons, clinics, and social networks, and for diseases where a small number of contacts drive most transmission (crowded congregate settings for TB, for example), who is connected to whom matters as much as how many people are infected.

> **Difficult terminology: Homogeneous mixing assumption.** The simplifying assumption, built into basic SIR/SEIR compartmental models, that every individual in a compartment has an equal chance of contacting every individual in every other compartment. It makes the maths tractable (you only need to track compartment **counts**, not who-contacts-whom) but breaks down when real contact structure is clustered — e.g. TB transmission concentrated within households or correctional facilities rather than spread evenly across a whole district.

Agent-based and network epidemic models drop that assumption: each person is an explicit simulated agent with their own contact network, and infection can only pass along an edge of that network. The R package `EpiModel` supports a spectrum of model types along this axis: deterministic compartmental models via `dcm()` (mathematically equivalent to the `deSolve` approach above), stochastic individual-level models without an explicit network via `icm()` (a microsimulation-style analogue of the compartmental model, with chance built into every transition), and full stochastic network models via `netsim()`, which first fits a dynamic contact-network model with `netest()` (built on the `ergm`/`tergm` packages) and then simulates disease spread over that evolving network. Each of the three model classes has matching `param.*()`, `init.*()`, and `control.*()` setup functions (e.g. `param.icm()`, `init.icm()`, `control.icm()` for individual contact models).

> **Difficult terminology: Network epidemic model.** A transmission model in which the population is represented as a graph of individuals (nodes) connected by contacts or partnerships (edges), and infection spreads only along existing edges rather than between any two randomly chosen individuals. This lets the model represent real contact structure — household clusters, sexual partnership networks, prison contact networks — that a compartmental model collapses into a single average contact rate.

> **HEOR in practice.** Reach for the compartmental ODE first. It is faster to build, faster to run, and easier to defend to a review panel that is not full of modellers. Move to an agent-based or network model only when the policy question genuinely turns on contact structure that a compartmental model cannot represent — e.g. targeting a screening programme at a specific contact network (prison, mining camp, household of an index case) rather than the general population. `EpiModel`'s `dcm()` path is a useful bridge: it lets you reuse the same parameteri-

sation conventions as the stochastic model classes while staying in compartmental territory for a first pass.

Verify the exact argument names against the installed `EpiModel` version before shipping code — package APIs for `icm()`/`netsim()` have evolved across major releases, and the signatures above reflect the documented behaviour at the time of writing rather than a guarantee for whatever version loads on your machine.

Linking transmission models to economic evaluation

Everything from Chapters 17-20 — discounted costs, discounted QALYs, incremental cost-effectiveness ratios — still applies once you have a transmission model. The difference is what feeds the cost and QALY accounting. In a Markov cohort or partitioned survival model, the "outcomes" are state occupancy over time for **one** cohort receiving the intervention versus a comparator cohort, each unaffected by the other. In a transmission model, you run the **same population** twice — once with the intervention (vaccination, screening, treatment scale-up) and once without — and the clinical benefit is the gap between the two epidemic curves: infections averted, cases averted, deaths averted. Those averted-event counts are then costed and weighted by QALY loss per event exactly as in a static model, and rolled up into an ICER.

Common pitfall. A **static** model — one that assumes the force of infection is unaffected by the intervention, usually because it was built as if the disease were non-communicable — will systematically mis-estimate the value of an infectious-disease intervention, because it misses **indirect** protection: vaccinating or treating one person also reduces the risk to everyone they would otherwise have infected (herd/community protection). Depending on the epidemiology this can make a static model understate benefit substantially, particularly for interventions delivered at a scale that meaningfully dents transmission (mass vaccination campaigns, population-wide TB active case finding) rather than individual-level care. This is a long-standing, well-documented methodological concern: the ISPOR-SMDM Modeling Good Research Practices Task Force dedicated a dedicated report to it — Pitman R, Fisman D, Zaric GS, et al. "Dynamic Transmission Modeling: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force-5." *Value in Health* 2012;15(5):828-834 — which sets out when a dynamic transmission model is needed rather than a static decision-analytic one, precisely because the force of infection changes with the intervention and feeds back into future risk.

The practical takeaway for a TB-focused reader: if the question is "how should we treat the TB patients already diagnosed," a Markov or microsimulation model of individual care pathways (Ch. 19-21) is usually adequate. If the question is "how much TB transmission will a new screening or preventive-therapy programme avert in the wider community," the force of infection is the thing the intervention is trying to change, and a dynamic transmission model is the more defensible tool.

22.5 Choosing between Markov, DES and dynamic models

By this point in the book you have four structurally different ways to represent disease progression and its cost/QALY consequences: Markov cohort models (Ch. 19), individual-patient microsimulation (Ch. 21), discrete event simulation (22.1-22.3), and dynamic transmission models (22.4). They share the same economic evaluation machinery downstream — costs, QALYs, discounting, ICERs —

but differ fundamentally in what they can represent upstream. The single most useful question for choosing among them is: **does one simulated person's outcome change another simulated person's risk or resource access?** If no, a cohort-level or independent-microsimulation model is both correct and cheaper to run. If yes — because they compete for a shared, limited resource (DES) or because one person's infection changes another's risk of infection (dynamic transmission) — you need a model class that can represent interaction.

Model type	Individual heterogeneity	Interaction between individuals	Typical use case	Computational cost	R package family
Markov cohort	No — cohort moves as fractions between states	No	Chronic disease progression with a small number of health states; most standard HTA submissions	Low	Base R, `heemod`, hand-rolled matrices (Ch. 19)
Microsimulation (independent patients)	Yes — each patient has their own history/attributes	No — patients simulated independently	Pathways with memory effects (time-in-state, prior events) that a Markov model cannot track	Moderate	Base R loops, `hesim` (Ch. 21)
Discrete event simulation	Yes	Yes — shared, limited resources (beds, staff, clinics)	Capacity-constrained care pathways; operational/service-delivery questions	Moderate-high	`simmer` (Ch. 22.1-22.3)
Dynamic transmission model	Yes/varies by implementation (compartmental: no; agent-based/network: yes)	Yes — infection risk depends on the number of infectious people	Infectious disease interventions with population-level or herd effects (vaccination, mass screening)	Low (ODE) to high (network simulation)	`deSolve`, `EpiModel` (22.4)

Table 55: Decision framework: choosing a model structure for a HEOR/HTA question

Agency perspective. This four-way decision framework is not an invented teaching device — it directly operationalises Siebert et al.'s (2012) ISPOR-SMDM Modeling Good Research

> Practices Task Force-3 report, already cited in Chapter 17 and Chapter 21, which lists decision trees, Markov models, partitioned survival models, microsimulation, discrete event simulation, and dynamic transmission models as a single family of choices to be made on structural grounds (what the disease/decision problem requires), not on modeller familiarity or software convenience. The DES-specific row is further detailed by Karnon et al.'s (2012) Task Force-4 report (§22.1's agency-note above); the dynamic-transmission row is further detailed by Pitman et al.'s (2012) Task Force-5 report (§22.4's pitfall above) — so every row in the table above now traces to a named, citable ISPOR-SMDM source rather than a generic "choose the right model" claim.

A practical rule of thumb for a TB programme analyst: start with the cheapest structure that can answer the question honestly, and only add complexity the question actually demands. A new drug regimen for diagnosed patients is a Markov or microsimulation question. A ward staffing or GeneXpert machine capacity question is a DES question. A question about how many **future** infections a screening programme prevents in the community is a dynamic transmission question — and reaching for a static model there (Markov or microsimulation) risks the herd-effect pitfall flagged in 22.4.

22.6 Worked example: a TB screening/preventive-therapy intervention model

This worked example uses a simplified SEIR-style compartmental model — the same `deSolve::ode()` machinery as 22.4 — to compare a TB screening-and-preventive-therapy programme against a no-intervention baseline, and to translate the difference in infections into an illustrative cost-effectiveness result.

The intervention is represented the simplest possible way an effective screening/preventive-therapy programme can be captured in a compartmental model: it reduces the effective rate at which susceptible people become infected, via a parameter `ve` applied multiplicatively to the force of infection (interpret `ve` as the population-average protective effect of the programme — a mix of coverage and per-person efficacy — not a literal per-dose vaccine efficacy).

```
library(deSolve)

seir_model <- function(t, y, parms) {
  S <- y["S"]; E <- y["E"]; I <- y["I"]; R <- y["R"]
  with(as.list(parms), {
    N <- S + E + I + R
    force_inf <- beta * I / N * (1 - ve)
    dS <- -force_inf * S
    dE <-  force_inf * S - sigma * E
    dI <-  sigma * E - gamma * I
    dR <-  gamma * I
    list(c(dS, dE, dI, dR))
  })
}

N0    <- 100000
y0    <- c(S = N0 - 50, E = 0, I = 50, R = 0)
```

```
times <- seq(0, 10, by = 1 / 52)          # 10-year horizon, weekly steps

base_parms <- c(beta = 2.0, sigma = 1, gamma = 1, ve = 0)    # no programme, R0 =
2
int_parms  <- c(beta = 2.0, sigma = 1, gamma = 1, ve = 0.3)  # programme: 30%
effective protection

out_base <- as.data.frame(ode(y0, times, seir_model, base_parms))
out_int  <- as.data.frame(ode(y0, times, seir_model, int_parms))

cum_base <- tail(out_base$I + out_base$R, 1)   # cumulative ever-infected-or-
infectious, per 100,000
cum_int  <- tail(out_int$I  + out_int$R,  1)
cases_averted <- cum_base - cum_int
```

With $R_0 = \beta/\gamma = 2$ and a 10-year horizon, the no-intervention run accumulates approximately 4,979 infections per 100,000 population (summing the final $I + R$ compartments, started from 50 initial cases), while the 30%-effective programme accumulates approximately 873 — a difference of about 4,106 infections averted per 100,000, roughly an 82% reduction. Because new infections keep seeding further infections in the unprotected arm, the gap between the two curves widens over the full 10-year horizon rather than staying fixed at the direct effect of the one-time intervention — this compounding is precisely the dynamic/indirect effect the pitfall in 22.4 warns a static model would miss.

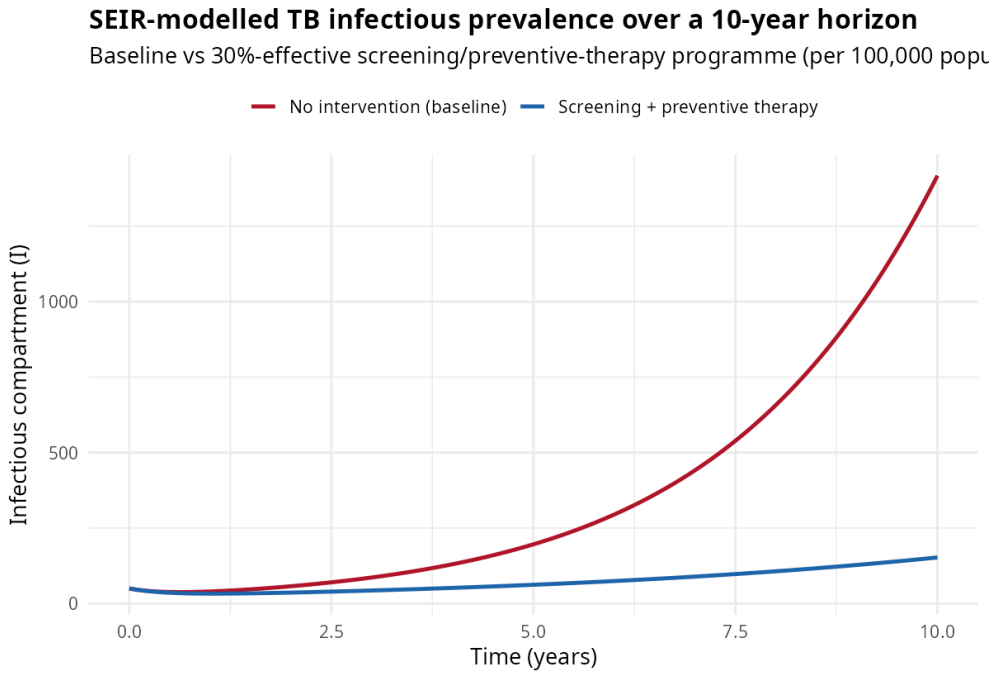


Figure 34: Infectious compartment (I) over the 10-year horizon from the `deSolve::ode()` SEIR model above: no-intervention baseline vs the 30%-effective screening/preventive-therapy programme. The gap between the curves widens over time rather than staying fixed at the size of the one-time intervention, illustrating the compounding/indirect-protection effect that a static model would miss (the pitfall in 22.4).

Costing this illustratively (figures below are for demonstration only, not real TB unit costs):

```
cost_per_case        <- 1500   # USD, illustrative TB treatment cost per active
case
qaly_loss_per_case    <- 0.15  # illustrative QALY loss per active TB episode
cost_per_covered      <- 8     # USD, illustrative programme cost per person
covered
coverage              <- 0.7
n_covered             <- N0 * coverage

cost_averted_treatment <- cases_averted * cost_per_case
qaly_gained             <- cases_averted * qaly_loss_per_case
intervention_cost       <- n_covered * cost_per_covered
net_budget_impact       <- intervention_cost - cost_averted_treatment
```

Carrying the arithmetic through: `cases_averted` ≈ 4106, so treatment cost averted $\approx 4106 \times \$1500 \approx \$6.16$ million; QALYs gained $\approx 4106 \times 0.15 \approx 616$; programme cost for 70,000 people covered at \$8 each is $70000 \times 8 = \$560000$; net budget impact is $\$560000 - \$6.16\text{M} \approx -\$5.60$ million (i.e. the programme is cost-**saving** once averted treatment costs are counted), and because it is both less costly and more effective it is a dominant strategy rather than one with a conventional positive ICER.

How to report this.

Outcome	No intervention	Screening programme	Difference
Cumulative infections (I + R) at 10y	4,979	873	−4,106 (-82.5%)
Peak infectious prevalence	1,417	153	−1,264
TB treatment cost (illustrative), USD	\$7.47M	\$1.31M	-\$6.16M
QALYs lost to active TB (illustrative)	747	131	+616 gained
Programme cost (illustrative), USD	\$0	\$560,000	+\$560,000
Net budget impact (illustrative), USD	–	–	-\$5.60M (savings)

Table 56: Illustrative TB screening/preventive-therapy programme vs no intervention, 10-year horizon per 100,000 population (toy model, demonstration figures only)

The headline qualitative finding — that the programme is cost-saving and QALY-gaining in this toy parameterisation — is an artefact of the chosen illustrative unit costs and efficacy, not a real-world TB finding; a real submission would need locally sourced unit costs, a properly validated TB natural-history structure, and a sensitivity analysis over `beta`, `ve`, and coverage before anyone could rely on the ICER sign.

Chapter 23. Budget Impact Analysis

Learning objectives

- Distinguish budget impact analysis (BIA) from cost-effectiveness analysis (CEA) by perspective, time horizon, discounting convention, and the question each answers, and state the core requirements of ISPOR's Good Practice Task Force report for BIA.
- Build an eligible-population cascade and a current-versus-new market-share structure that captures how a new intervention displaces existing treatment, not just how many patients take it.
- Construct and interpret linear and logistic (S-shaped) uptake/diffusion curves over a short multi-year horizon.
- Assemble a complete, parameterized R budget impact model from a reference scenario and a new scenario, computing year-by-year and cumulative budget impact.
- Re-run the same model under structured scenario analyses (faster uptake, a managed-entry price agreement) to show payers a realistic range rather than a single point estimate.
- Present budget impact results the way payers actually expect to see them: separate acquisition-cost and offset lines, a year-by-year table, and an explicit statement of which assumption the result is most sensitive to.

Chapters 17 through 22 built an increasingly rich family of models – decision trees, Markov cohort models, partitioned survival models, microsimulation, and discrete event simulation / dynamic transmission models – all answering some version of the same question: is a new intervention good value for the health it produces, expressed as an incremental cost-effectiveness ratio over a long, often lifetime, horizon. This chapter and the next move to a different, shorter-horizon question that sits alongside cost-effectiveness in every real HTA or payer submission: not "is this worth it," but "what will this actually cost our budget over the next few years, given how adoption really unfolds." Budget impact analysis (BIA) is that question, and it uses a genuinely different model structure – not a cut-down version of the CEA models already built in this book, but its own payer-facing accounting exercise.

23.1 Purpose and ISPOR guidance

Every model built so far in this book – decision trees, Markov cohort models, partitioned survival models, microsimulations, discrete event simulations, dynamic transmission models – has ultimately answered the same question: is a new intervention **good value** for the health gained, expressed as an incremental cost-effectiveness ratio (ICER) in cost per quality-adjusted life year (QALY). That is the language of cost-effectiveness analysis (CEA), and it is typically computed over a **lifetime** horizon, from a **societal** or **healthcare-system** perspective, because value-for-money has to be judged over the full span of health gained and cost incurred.

Budget Impact Analysis (BIA) asks a completely different question, and it is easy – and common – to conflate the two. A payer (a national health service, an insurer, a hospital pharmacy and therapeutics committee) does not primarily want to know whether a drug is good value over a patient's lifetime. They want to know: **if we add this intervention to our formulary next year, what will it do to our cash budget over the next one to five years?** A drug can be highly cost-effective (low cost per QALY, even cost-saving in the long run) and still be **unaffordable** in the short term if a large eligible population switches onto it quickly and the drug acquisition cost is high. Conversely, a drug with a mediocre ICER might have almost no budget impact if the eligible population is tiny. BIA and CEA are complementary, not competing, questions, and ISPOR has always insisted they be reported separately and not substituted for one another.

> **Difficult terminology: Budget impact analysis (BIA).** Budget impact analysis is the financial modelling exercise that estimates the **change in a specific payer's spending** that would result from introducing, funding, or changing access to a health technology, over a defined and short future period – typically one to five years, almost never a patient lifetime. Where CEA asks "is this worth it per unit of health gained, over all time", BIA asks "what does this do to this year's and next few years' cash outflow, for this specific budget holder".
>
> Think of it the way a hospital finance director thinks about a capital purchase versus a clinician thinking about whether the purchase improves patient outcomes. The clinician's question (does a new MRI scanner improve diagnostic yield enough to justify its cost per QALY gained across its operating lifetime) is the CEA question. The finance director's question (can the hospital's budget absorb the purchase price and running costs over the next three fiscal years, given how many patients will actually be scanned on it) is the BIA question. Both are legitimate, and a payer typically wants both answers before it funds anything – but they are not the same number, they are not computed the same way, and conflating them in a report is a common and consequential analytical error.

The defining methodological document for this chapter is a specific ISPOR Good Practice Task Force report, and because BIA is so often done badly (or done as a thin disguise for a CEA), it is worth being explicit about what it actually requires.

> **Agency perspective.** The operative ISPOR reference for everything in this chapter is: Sullivan SD, Mauskopf JA, Augustovski F, Caro JJ, Lee KM, Minchin M, Orlewska E, Penna P, Rodriguez Barrios JM, Shau W-Y. *Budget Impact Analysis – Principles of Good Practice: Report of the ISPOR 2012 Budget Impact Analysis Good Practice II Task Force.* Value in Health. 2014;17(1):5-14.

This report lays out the core structural requirements that distinguish a credible BIA from an ad-hoc spreadsheet: (1) a short, explicit time horizon appropriate to payer budget cycles (commonly 1-5 years, stated and justified, not defaulted to a lifetime horizon as in CEA); (2) the payer's **actual** budget-holder perspective, including only costs that fall on that specific budget (not broader societal costs unless the payer's remit covers them); (3) an explicit comparison of a "world without the new intervention" (reference scenario, usually the current standard-of-care mix projected forward) against a "world with the new intervention" (new scenario, reflecting a realistic adoption trajectory), with the budget impact reported as the **difference** in total cost between the two scenarios in each year; (4) realistic, evidence-based estimates of the eligible population, uptake/diffusion over time, and market share displacement of existing therapies, rather than an assumption that the whole eligible population switches on day one; and (5) transparent reporting of all assumptions and inputs so a payer's own analysts can stress-test them. We return to each of these five requirements directly as the model structure is built in the remaining sections of this chapter.

Common pitfall. A frequent and serious mistake is to build a BIA by simply re-running the CEA model's cohort over a shorter horizon and reporting undiscounted cumulative cost. This is wrong for at least three reasons: the CEA cohort typically represents **incident** patients entering a Markov or survival process, not the **mixed prevalent-plus-incident** population that will actually be on treatment in a given budget year; the CEA model usually assumes 100% uptake among those eligible, whereas real-world adoption is gradual; and CEA costs are discounted at a long-run societal discount rate, whereas ISPOR guidance recommends BIA cash flows typically be presented **undiscounted**, because a payer's budget is set and spent in nominal terms year by year, not in present-value terms. If your BIA numbers look exactly like a truncated slice of your CEA model, something has gone structurally wrong.

HEOR in practice. When scoping a BIA, agree the time horizon and perspective with the budget holder **before** building anything. Ask directly: "whose budget, and over what fiscal years?" A national payer, a regional health authority, and a single hospital pharmacy can have materially different eligible populations, different existing market shares of comparator treatments, and different budget cycles (some run 1-year rolling, some run 3- or 5-year planning horizons). Building the population and uptake structure (Section 23.2) around the wrong budget holder's perspective is the single most common reason a BIA has to be rebuilt from scratch.

23.2 Population, uptake and market-share structure

Every budget impact model, however it is eventually coded, rests on the same arithmetic skeleton repeated for each year of the horizon:

$$\text{Budget impact}_t = (\text{Eligible population}_t \times \text{Uptake}_t \times \text{Market share}_t \times \text{Cost per patient})$$

applied separately to the reference scenario (world without the new intervention) and the new scenario (world with it), with the year-by-year **difference** between the two totals being the budget

impact actually reported to the payer. Section 23.1 established **why** this calculation is structured this way; this section builds the three population-level ingredients – eligible population size, market-share mix, and the uptake/diffusion curve – that feed it. The fourth ingredient, cost per patient, is a straightforward unit-cost exercise similar to costing work done in Chapter 9 and is assembled together with these population terms into a working model in Section 23.3.

Eligible population

The eligible population is not simply "everyone with the diagnosis." ISPOR good-practice guidance is explicit that the eligible population should be derived top-down from an epidemiologically defensible base (disease prevalence or incidence in the payer's covered population) and then narrowed by every clinically and administratively relevant restriction: diagnostic criteria, line-of-therapy eligibility, reimbursement/label restrictions, and any contraindications. A common structure is a simple cascade:

```
library(dplyr)

# illustrative cascade - every number here is a placeholder to be
# replaced with payer-specific epidemiological and administrative data
population_cascade <- tibble::tibble(
  step = c("covered_lives", "disease_prevalence", "diagnosed",
           "meets_label_criteria", "eligible_for_new_intervention"),
  proportion_of_prior_step = c(1, 0.02, 0.80, 0.60, 0.90)
)

population_cascade <- population_cascade |>
  mutate(cumulative_proportion = cumprod(proportion_of_prior_step))

population_cascade
```

Each row narrows the denominator: of the payer's total covered lives, some proportion has the disease, of those some proportion are actually diagnosed, of those some proportion meet the specific label or guideline criteria for the new intervention, and of those some proportion have no contraindication. `cumprod()` gives the running cumulative proportion applied to the covered-lives base, and multiplying that final cumulative proportion by the covered-lives count gives the eligible population in year 1. Because covered lives themselves can grow or shrink over the horizon (population growth, membership churn in an insured population), the eligible population is usually re-projected for each year of the horizon rather than held constant, often with a simple annual growth-rate multiplier applied to the covered-lives figure before the cascade is reapplied.

> **Common pitfall.** Do not reuse the CEA model's trial-based "intention-to-treat population" as the BIA eligible population. Clinical trials recruit a selected population under research conditions; real-world eligible populations are almost always larger (broader label, less stringent exclusion) or, just as often, smaller (diagnosis rates in routine practice are lower than in a trial with active case-finding). The eligible-population cascade should be built from real-world epidemiological and claims/administrative sources specific to the payer's population, not lifted from the trial's CONSORT diagram.

Market share: the reference versus new scenario

Once the eligible population for a given year is known, it must be split across the available treatment options – and it must be split **twice**: once under the reference scenario (the world as it

would continue without the new intervention, i.e. existing therapies keep whatever share trajectory they currently have), and once under the new scenario (the world with the new intervention available, where it progressively displaces some share from existing therapies). The payer-reported budget impact in each year is the difference in total cost between these two parallel market-share allocations, not the cost of the new intervention alone.

> **Difficult terminology: Market share (in a BIA context).** In everyday use "market share" means the proportion of a market a product holds. In a BIA, market share specifically means the proportion of the **eligible population currently receiving treatment in that year** that is allocated to each named treatment option (existing drug A, existing drug B, best supportive care, and the new intervention). It matters because the new intervention's cost to the payer in any given year is not simply "number of patients taking it times its price" – it is that, **minus** whatever cost the payer was already going to incur for the patients who would otherwise have been on an existing (and possibly cheaper, or possibly more expensive) therapy. A useful analogy: if a hospital canteen introduces a new salad bar, the budget question is not "what does the salad bar cost to run," it's "how does total catering spend change," accounting for the fact that some diners who would have bought a sandwich (a different cost) now buy a salad instead. The **net** change, not the gross cost of the new option, is the budget impact.

A simple two-period illustration keeps the arithmetic checkable by hand. Suppose a payer's eligible population for a condition is a constant 1000 patients per year (ignoring population growth for simplicity), and under the **reference** scenario this population is split 70% onto existing Drug A (cost $500 per patient-year) and 30% onto existing Drug B (cost $800 per patient-year):

$$\text{Reference scenario cost} = 1000 \times (0.70 \times 500 + 0.30 \times 800) = 1000 \times (350 + 240) = 590000$$

Now suppose that in the **new** scenario, by year 2 the new intervention (cost $1200 per patient-year) has captured 20% of the eligible population, drawn proportionally from both existing drugs (so Drug A falls to 56% and Drug B falls to 24%, preserving their 70:30 relative split among the remaining 80%):

$$\text{New scenario cost} = 1000 \times (0.56 \times 500 + 0.24 \times 800 + 0.20 \times 1200) = 1000 \times (280 + 192 + 240) = 712000$$

$$\text{Budget impact (year 2)} = 712000 - 590000 = 122000$$

The payer does not simply pay "20% of 1000 patients times $1200" (which would suggest a $240,000 impact); because some of that $1200 replaces costs the payer was already paying for Drug A and Drug B, the true net impact is smaller, at $122,000. This netting-out is the entire reason a BIA cannot be read off a drug's list price alone, and it is the single most common source of inflated, indefensible budget-impact estimates when analysts skip the reference-scenario comparison.

Uptake and diffusion over the time horizon

The 20% market share reached "by year 2" in the example above does not appear instantly – real-world adoption of a new intervention follows a diffusion curve shaped by prescriber familiarity, guideline updates, prior-authorization processes, and manufacturer launch sequencing. ISPOR guidance calls for the uptake trajectory to be stated explicitly and justified (analogous products' historical launch curves, expert elicitation, or market-research forecasts are all acceptable evidence sources), rather than assumed as an instant step-change or an arbitrary straight line.

Two simple, defensible functional forms cover most practical cases: a linear ramp-up to a terminal share, or an S-shaped logistic diffusion curve that starts slowly, accelerates through early adoption, and then levels off as the addressable market saturates – the same qualitative shape used to describe technology adoption in many fields.

```
library(dplyr)
library(ggplot2)

years <- 1:5
peak_share <- 0.35   # terminal market share captured by the new intervention

# linear ramp-up: equal increments each year up to peak_share by year 5
linear_uptake <- seq(from = peak_share / 5, to = peak_share, length.out = 5)

# logistic (S-shaped) diffusion: slow start, faster middle, levelling off
# k controls steepness, t0 the inflection year
logistic_uptake <- function(t, peak, k = 1.2, t0 = 2.5) {
  peak / (1 + exp(-k * (t - t0)))
}

uptake_curves <- tibble::tibble(
  year = years,
  linear = linear_uptake,
  logistic = logistic_uptake(years, peak = peak_share)
)

uptake_curves |>
  tidyr::pivot_longer(cols = c(linear, logistic),
                      names_to = "curve", values_to = "market_share") |>
  ggplot(aes(x = year, y = market_share, colour = curve)) +
  geom_line(linewidth = 1) +
  geom_point() +
  labs(x = "Year of horizon", y = "New intervention market share",
       title = "Illustrative uptake curves over a 5-year BIA horizon")
```

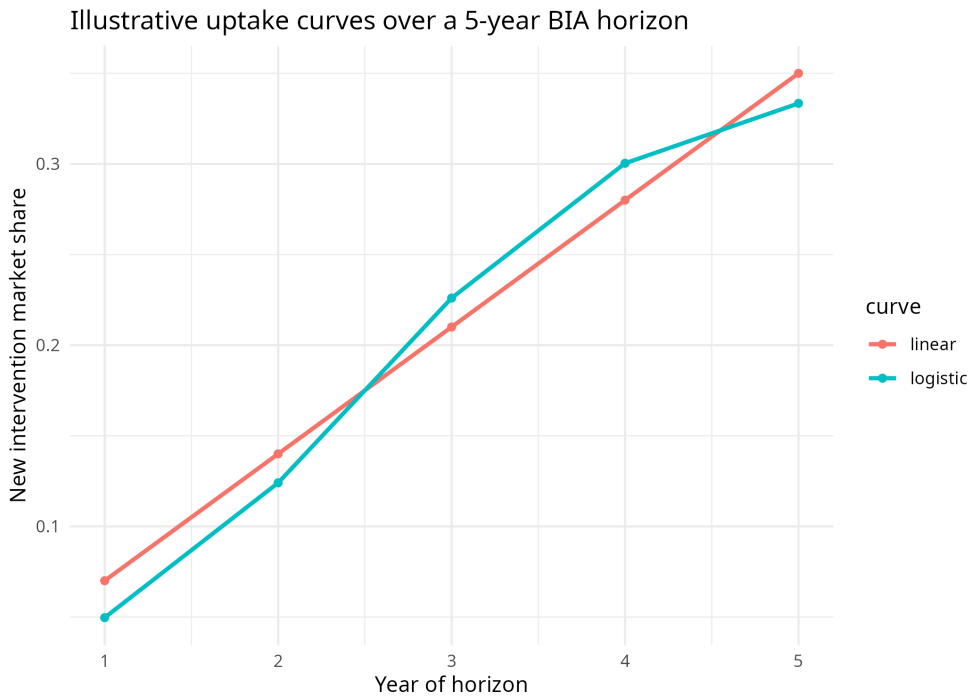


Figure 35: Illustrative linear vs logistic (S-shaped) uptake curves over a 5-year BIA horizon, both reaching a terminal market share of 0.35, from the `ggplot2` code above. The logistic curve starts slower, accelerates through the middle years, and converges toward the same terminal share as the linear ramp.

Multiplying each year's eligible population (from the cascade above) by that year's uptake-adjusted market share, for both the reference and new scenarios, and costing each resulting patient mix, generates exactly the year-by-year budget impact table that payers expect to see – the mechanics of assembling that table, applying it within a full R model structure, and extending it to scenario (sensitivity) analysis are taken up next in Section 23.3.

HEOR in practice. Always sanity-check an uptake curve against real analogues before presenting it to a payer: how quickly did the most comparable recent therapy in this disease area actually get adopted, in this payer's own historical claims or dispensing data, after launch? Modelled uptake curves that assume faster diffusion than any historical comparator in the same therapeutic class are a near-automatic credibility flag in payer review, and are one of the first things a payer's technical committee will ask you to defend.

23.3 Building a budget impact model in R

A budget impact analysis answers a narrower and more operational question than the cost-effectiveness models of Chapters 17-22: not "is this good value over a lifetime of discounted QALYs," but "what will the payer actually have to find in the budget over the next one to five years." There is no discounting of costs in the base case under ISPOR good-practice guidance (the horizon is short enough that time-value-of-money adjustments are usually judged immaterial), and there are no health outcomes in the calculation at all – only cash.

The model below is deliberately small enough to check by hand. It uses:

- a five-year horizon,
- a flat eligible population of 10,000 patients per year (no growth, stated explicitly – the structural building blocks for sizing this population and its current/new treatment mix were covered in SS23.1-23.2),
- a "current mix" world in which 100% of eligible patients remain on standard of care (SoC) at $500 per patient-year,
- a "new mix" world in which a new drug at $2,000 per patient-year phases in via an uptake curve of 5%, 15%, 30%, 45%, 55% across years 1-5, with every remaining patient still on SoC,
- one simple cost offset: patients on the new drug avoid $300 per patient-year of downstream hospitalization cost, so the **net** new-drug cost is $1,700 per patient-year.

```
library(dplyr)

build_bia <- function(
    years          = 1:5,
    eligible_pop   = rep(10000, 5),
    uptake_share   = c(0.05, 0.15, 0.30, 0.45, 0.55),
    cost_soc       = 500,
    cost_new       = 2000,
    offset_new     = 300
) {
  tibble(
    year                = years,
    eligible_population = eligible_pop,
    uptake_share        = uptake_share
  ) %>%
    mutate(
      patients_on_new  = eligible_population * uptake_share,
      patients_on_soc  = eligible_population - patients_on_new,
      # net per-patient cost of the new drug after the hospitalization offset
      net_cost_new     = cost_new - offset_new,
      cost_new_mix     = patients_on_new * net_cost_new + patients_on_soc *
cost_soc,
      cost_current_mix = eligible_population * cost_soc,
      budget_impact    = cost_new_mix - cost_current_mix
    ) %>%
    select(year, eligible_population, uptake_share, patients_on_new,
patients_on_soc,
           cost_current_mix, cost_new_mix, budget_impact)
}

bia_base <- build_bia()
bia_base$cumulative_bi <- cumsum(bia_base$budget_impact)
bia_base
```

Working the arithmetic through by hand confirms the code's output. The current-mix world costs the same every year because the population and unit price never change: $10000 \times 500 = 5000000$ per year, or $25,000,000 across the five years.

In the new-mix world, each year's cost is (patients on new drug times $1,700 net) + (patients on SoC times $500):

- Year 1: $500 \times 1700 = 850000$; $9500 \times 500 = 4750000$; total $5,600,000.

- Year 2: $1500 \times 1700 = 2550000$; $8500 \times 500 = 4250000$; total \$6,800,000.
- Year 3: $3000 \times 1700 = 5100000$; $7000 \times 500 = 3500000$; total \$8,600,000.
- Year 4: $4500 \times 1700 = 7650000$; $5500 \times 500 = 2750000$; total \$10,400,000.
- Year 5: $5500 \times 1700 = 9350000$; $4500 \times 500 = 2250000$; total \$11,600,000.

Subtracting the flat \$5,000,000 current-mix cost from each gives the year-by-year budget impact: \$600,000, \$1,800,000, \$3,600,000, \$5,400,000, and \$6,600,000. These five figures sum to \$18,000,000 – the cumulative five-year budget impact, which can be cross-checked by summing the new-mix total (\$43,000,000) and subtracting the current-mix total (\$25,000,000): $43000000 - 25000000 = 18000000$. Both routes agree, which is exactly the kind of independent cross-check a model built in a spreadsheet rarely gets for free but a tidyverse pipeline makes almost automatic.

> **Common pitfall.** A recurring error in BIA submissions is quietly turning the model into a cost-effectiveness analysis: discounting costs over a multi-year horizon as if computing lifetime value, or reporting an incremental cost-per-QALY alongside the budget table. BIA guidance (ISPOR, 2014) deliberately omits discounting in the base case and omits outcomes entirely – the deliverable is **undiscounted net cash impact per year**, not a value judgement. A second common mistake sits inside the uptake curve itself: modelling diffusion as if every patient started and stayed on the new drug, with no allowance for discontinuation or non-adherence. If, in reality, 20% of patients who start the new drug stop within the first year, the **effective** exposed population generating cost (and offsetting hospitalization) is smaller than the raw uptake share suggests, and a model that ignores this will overstate the new-drug spend and understate any downstream offset actually realised.

23.4 Scenario analyses

A payer rarely accepts a single point estimate. The usual request is a small set of structured alternatives built from the same model skeleton, varying only the assumptions that are genuinely uncertain: the pace of uptake, the unit price (often subject to a confidential managed-entry agreement, MEA), or the size of the offset. Because `build_bia()` already takes `uptake_share`, `cost_new`, and `offset_new` as arguments, every scenario is a one-line call.

```
# Scenario A: faster diffusion (e.g. a strong clinical-guideline endorsement)
bia_fast_uptake <- build_bia(
  uptake_share = c(0.10, 0.25, 0.40, 0.55, 0.65)
)

# Scenario B: managed-entry agreement - confidential 25% net price reduction
bia_mea <- build_bia(
  cost_new = 1500   # was 2000; offset_new stays at 300
)

scenarios <- bind_rows(
  bia_base        %>% mutate(scenario = "Base case"),
  bia_fast_uptake %>% mutate(scenario = "Faster uptake"),
  bia_mea         %>% mutate(scenario = "Managed-entry agreement")
)
```

The faster-uptake scenario is a pure "what-if" on the diffusion curve – no new arithmetic principle, just larger `patients_on_new` values feeding the same formula, and therefore a larger budget impact in every year. The managed-entry-agreement scenario is worth checking by hand, because it changes the net per-patient cost rather than the uptake curve: with the new drug re-priced to $1,500 and the same $300 hospitalization offset, the net cost per patient on the new drug falls from $1,700 to $1,200.

Re-running the same uptake shares (5%/15%/30%/45%/55%) against this $1,200 net cost gives, year by year: Year 1, $500 \times 1200 = 600000$ plus $9500 \times 500 = 4750000$, total $5,350,000, budget impact $350,000. Year 2, $1500 \times 1200 = 1800000$ plus $8500 \times 500 = 4250000$, total $6,050,000, budget impact $1,050,000. Year 3, $3000 \times 1200 = 3600000$ plus $7000 \times 500 = 3500000$, total $7,100,000, budget impact $2,100,000. Year 4, $4500 \times 1200 = 5400000$ plus $5500 \times 500 = 2750000$, total $8,150,000, budget impact $3,150,000. Year 5, $5500 \times 1200 = 6600000$ plus $4500 \times 500 = 2250000$, total $8,850,000, budget impact $3,850,000.

Those five budget-impact figures sum to $350,000 + $1,050,000 + $2,100,000 + $3,150,000 + $3,850,000 = $10,500,000 over five years. That can again be cross-checked top-down: cumulative new-mix cost of $35,500,000 minus cumulative current-mix cost of $25,000,000 is $10,500,000, matching the bottom-up sum. Against the base case's $18,000,000 cumulative impact, the MEA scenario reduces the five-year budget impact by $7,500,000 – a figure the payer will care about far more than the headline list-price discount.

> **HEOR in practice.** Keep every scenario a parameter change to the **same** function rather than a hand-edited copy of the spreadsheet. This is what lets a payer's reviewer ask "what if uptake is 10 points lower in year 3" and get a defensible, reproducible answer in one function call instead of a re-built model that may silently diverge from the base case in some other cell.

23.5 Presenting results to payers

A payer budget holder reading a BIA submission is not looking for a single net number. They want, at minimum: a year-by-year table (not just a five-year total, because budgets are set annually); drug-acquisition cost and offset cost reported as **separate lines**, not pre-netted, so the reviewer can independently judge whether the offset assumption is credible; and some indication of how sensitive the result is to the market-share/uptake assumption, since that is usually the single most contestable input in the whole model.

The table below reports the base-case results computed in Section 23.3 exactly as calculated – no rounding adjustments, no re-derivation.

How to report this.

Year	Eligible population	Market share new (%)	Cost, current mix	Cost, new mix	Budget impact
Year 1	10,000	5%	$5,000,000	$5,600,000	$600,000
Year 2	10,000	15%	$5,000,000	$6,800,000	$1,800,000
Year 3	10,000	30%	$5,000,000	$8,600,000	$3,600,000
Year 4	10,000	45%	$5,000,000	$10,400,000	$5,400,000
Year 5	10,000	55%	$5,000,000	$11,600,000	$6,600,000
Cumulative 5-year budget impact: $18,000,000					

Table 57: Year-by-year budget impact of introducing the new drug, base-case uptake (current-mix cost held flat at $5,000,000/year; all figures USD)

A chart is the natural companion to this table – payers typically expect to see cumulative budget impact rising visually across the horizon, alongside the annual (non-cumulative) figures so the "when does spend accelerate" question is answered at a glance.

```
library(ggplot2)
library(tidyr)

plot_data <- bia_base %>%
  mutate(cumulative_bi = cumsum(budget_impact)) %>%
  select(year, budget_impact, cumulative_bi) %>%
  pivot_longer(cols = c(budget_impact, cumulative_bi),
               names_to = "measure", values_to = "amount")

ggplot(plot_data, aes(x = year, y = amount, fill = measure)) +
  geom_col(data = . %>% filter(measure == "budget_impact"), position = "identity")
+
  geom_line(data = . %>% filter(measure == "cumulative_bi"),
            aes(y = amount, group = 1), color = "black", linewidth = 1) +
  geom_point(data = . %>% filter(measure == "cumulative_bi"), size = 2) +
  scale_y_continuous(labels = scales::dollar) +
  labs(x = "Year", y = "Budget impact (USD)",
       title = "Annual vs. cumulative budget impact, base case") +
  theme_minimal()
```

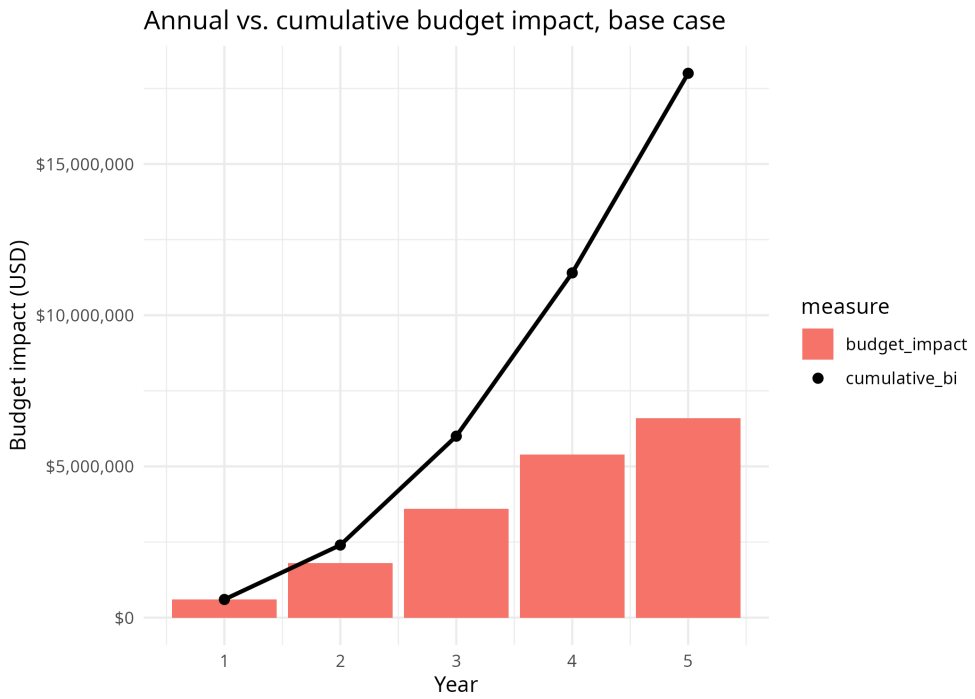


Figure 36: Annual budget impact (bars, rising from $600,000 in Year 1 to $6,600,000 in Year 5) with cumulative budget impact overlaid as a line (climbing to $18,000,000 by Year 5), base-case uptake scenario from Table 23.1 (<tab-ch23-bia-result>), from the ggplot2 code above.

Agency perspective. Most payer review templates (and many national BIA submission guidelines, e.g. those referenced by NICE and CADTH) explicitly require drug-acquisition cost and any claimed offset to be shown as separate line items rather than netted together before submission. In the model above this means reporting patients-on-new times $2,000 as one line and patients-on-new times $300 (the hospitalization offset) as a second, clearly-labelled reduction – never collapsing straight to the $1,700 net figure in the table a reviewer sees, even though netting is perfectly fine inside the R code itself. Reviewers want to interrogate the offset assumption on its own terms, and a pre-netted number removes their ability to do that.

Finally, no payer-facing BIA is complete without an explicit statement of which input the result is most sensitive to. In this example, that is unambiguously the uptake curve: moving from the base case's 55% year-5 market share to the faster-uptake scenario's 65% increases the year-5 new-mix spend materially, while the managed-entry-agreement scenario shows that a price concession on the same uptake path can claw back $7,500,000 of the cumulative impact. Presenting the base case, the faster-uptake case, and the MEA case side by side – rather than a single number – is what lets a budget holder see the realistic range they are actually planning against.

Chapter 24 extends this payer-facing accounting into territory beyond a single budget line: monetary valuation of health benefits, equity weighting and distributional cost-effectiveness analysis, financial-risk protection (extended CEA), and multi-criteria decision analysis – all building on the same habit this chapter established of making every assumption (population, uptake, price, offset) explicit and separately interrogable rather than collapsed into one opaque headline number.

Chapter 24. Cost-Benefit, Distributional and Extended Cost-Effectiveness Analysis

Learning objectives

- Explain how cost-benefit analysis (CBA) differs from cost-effectiveness analysis (CEA) by monetizing both sides of the value equation, and describe the value of a statistical life (VSL), human-capital, and willingness-to-pay approaches used to do so.
- Explain why a population-average ICER can mask unequal distribution of costs and benefits, and apply distributional cost-effectiveness analysis (DCEA) and equity weighting to make that distribution explicit.
- Compute an extended cost-effectiveness analysis (ECEA) financial-risk-protection outcome – catastrophic health expenditure cases averted and poverty cases averted – alongside a conventional health outcome.
- Build a simple multi-criteria decision analysis (MCDA) scoring pipeline in R, distinguishing weighted-sum from weighted-product combination, and relate it to the EVIDEM health-sector MCDA framework.
- Apply NICE's severity modifier to an ICER using the absolute and proportional QALY shortfall, and distinguish value-based pricing and the two broad types of managed entry agreement (MEA).

Chapters 17 through 23 shared one quiet convention: health is measured in health units (QALYs, life-years, cases averted) and only cost is measured in money, with the two combined into a cost-per-QALY ratio compared against a threshold, or — in Chapter 23's budget impact analysis — reduced to an undiscounted payer cash-flow question with no health outcome in the calculation at all. This closing chapter of Part IV steps outside that convention in five different directions: methods that monetize health itself (cost-benefit analysis), methods that make visible who actually receives an intervention's costs and benefits (distributional CEA), methods that add a second, non-health outcome dimension entirely (extended CEA's financial risk protection), methods that formalize decision criteria beyond cost and a single health outcome (multi-criteria decision analysis), and mechanisms that adjust the price or threshold side of the value equation directly (severity modifiers, value-based pricing, managed entry agreements). None of these replace the cost-effectiveness machinery built across Chapters 17-23 — each is a genuine extension or alternative lens applied to it.

24.1 Monetary valuation of health

Every model built in Chapters 17 through 23 has shared one convention without ever stating it as a choice: health is measured in **health units** – quality-adjusted life years (QALYs), life years, or cases averted – and only cost is measured in money. The result, an incremental cost-effectiveness ratio (ICER) in cost per QALY, lets an analyst compare interventions to each other and to a decision-maker's willingness-to-pay threshold, but it never actually monetizes the health benefit itself. Cost-benefit analysis (CBA) makes the different, more demanding choice of converting **both** sides of the ledger – costs and health benefits – into the same monetary unit, so that a single

net monetary figure (benefits minus costs, in dollars or pounds) can be compared against zero, or against the net benefit of competing uses of the same money entirely outside healthcare. This section covers how that monetization of health is actually done, and why the choice of method has mattered, for decades, well beyond health economics.

Why monetize health at all

CEA answers "which of these health interventions gives the most health per dollar spent," holding the healthcare budget's boundary fixed and implicit. It cannot, by construction, answer "is this health programme worth it compared with building a bridge, funding a school, or leaving the money with taxpayers" – because health is in one unit (QALYs) and the alternative use of the money is in another (dollars). Government-wide regulatory analysis – transport safety rules, environmental regulations, workplace safety standards – routinely has to make exactly that comparison, and it is CBA's capacity to put a dollar value on a human life or a QALY that allows it. This is also the reason CBA appears far more often in environmental and transport-safety economics than in national HTA decision-making for medicines: most national HTA bodies (NICE, Australia's PBAC, Canada's CADTH) formally compare within the health budget using CEA, not CBA, precisely so they are not forced to state an explicit dollar value of a human life as part of a medicines appraisal. CBA's natural territory in health is broader public-health and regulatory decisions where money, not just the health budget, is genuinely the alternative use of the resource.

> **Difficult terminology: Cost-benefit analysis (CBA) vs. cost-effectiveness analysis (CEA).** Both methods compare an intervention's incremental cost against its incremental benefit, but they differ in what unit the **benefit** side is measured in. CEA (and its close relative, cost-utility analysis, used throughout this book) measures benefit in natural health units – QALYs, life-years gained, cases prevented – and reports a ratio, cost per unit of health, which is only interpretable relative to a threshold or to another intervention's ratio. CBA measures **both** cost and benefit in money, so it can report a single net figure (total monetized benefit minus total cost) that is directly interpretable on its own: a positive net benefit means the intervention is worth doing in resource terms, full stop, without reference to any other intervention. The price of that convenience is that CBA requires converting health into money somewhere in the calculation – via a value of a statistical life, a human-capital earnings calculation, or a willingness-to-pay survey – and that conversion is itself a value judgement, not a neutral measurement, which is why CEA remains the dominant method inside formal drug/device HTA while CBA remains dominant in transport, environmental, and occupational safety regulation.

Value of a statistical life (VSL)

The value of a statistical life (VSL) is the dominant monetization method in regulatory CBA, and it is routinely misunderstood even by people who use the number professionally, so it is worth stating precisely what it is **not** before stating what it is.

> **Difficult terminology: Value of a statistical life (VSL).** VSL is not an attempt to price any individual, identified person's life – regulators and economists are emphatic about this, because the phrase invites exactly that misreading. VSL instead aggregates small, individual willingness-to-pay amounts for **small reductions in mortality risk** into an implied value for "one statistical death avoided" across a large group. The canonical illustration: suppose each

of 100,000 people would pay $100 for a programme that reduces each person's own annual risk of death by 1 in 100,000. That risk reduction, summed across the group, means the group expects one fewer death per year on average. Total willingness to pay is 100,000 times $100 = $10,000,000, and because that total buys exactly one statistical death avoided, the VSL implied by this group's preferences is $10 million. No single person in the group paid $10 million, and no one's actual death was priced – what was priced is the **risk reduction**, at the rate people actually revealed they would pay for it. VSL estimates are built empirically from either stated-preference surveys (asking people directly what they would pay for risk reductions) or revealed-preference studies (inferring the implicit value of safety from real choices, most famously wage differentials that workers demand to accept riskier jobs – the "hedonic wage" method).

VSL estimates vary substantially by agency, country, and year, because each reflects a different underlying survey/wage-differential evidence base and a different real-income base year. The US Department of Transportation's guidance recommended $13.2 million (2023 dollars) for regulatory analysis, an upward revision from $12.5 million (2022 dollars) to account for both general inflation and real income growth; the US EPA's standing default, built on a different literature base, is $7.4 million in 2006 dollars (updated for inflation when applied); HHS's 2024 standard values set a central estimate of $13.1 million with a low of $6.1 million and a high of $19.9 million, explicitly carried through regulatory sensitivity analysis rather than collapsed to one number. These are not competing "right answers" to the same question – they are different agencies' considered choices from a literature that itself spans a wide range, and the spread is wide enough (roughly $6 million to $20 million, a more than threefold range) that the choice of VSL is frequently the single most consequential, and most contested, input in any regulatory CBA.

Common pitfall. Do not treat a VSL figure from one jurisdiction, sector, or year as a portable universal constant. A VSL estimated from US wage-risk data in transport safety regulation reflects the income level, labour market, and risk perceptions of that specific population and context; applying it unadjusted to, say, a vaccination programme's mortality benefit in a different country with a different income level systematically mis-states willingness to pay. Where cross-country or cross-sector VSL transfer is unavoidable, published guidance (OECD, WHO) recommends income-elasticity adjustment, not a flat transfer of the headline number.

Human capital vs. willingness-to-pay approaches

Once the decision is made to monetize health, there are two historically distinct, philosophically different ways to do it.

The **human capital approach** values a health improvement (or the cost of a health loss) as the **production** it enables or prevents losing – typically approximated by discounted future earnings. A death or disabling illness in a working-age adult is valued, under this approach, as the present value of the earnings that person would have produced had they remained healthy and working. As a worked illustration: a person earning $45,000 a year who would otherwise have worked 10 more years, discounted at 3% per year, represents a human capital value of

$$\text{PV} = \sum_{t=0}^{9} \frac{45000}{1.03^{t}} = 395375$$

(computed directly by summing the ten discounted annual terms; the closed-form annuity formula gives the identical figure).

Difficult terminology: Human capital approach. This is the older of the two monetization approaches, borrowed directly from labour economics: a person's economic worth is approximated by their expected lifetime earnings, so health is valued by how much earning capacity it protects or restores. It is straightforward to compute from wage and life-table data, which is why it was the default approach for decades, but it has a well-known, severe equity problem: it systematically values the health of children, retirees, unpaid caregivers, and anyone outside paid employment at zero or near-zero, and values the health of a high earner more than the identical health gain in a low earner, purely because of their market wage. A retired 80-year-old grandparent's quality of life gain from a hip replacement has genuine value to them and their family, but the human capital approach records essentially none of it, because they produce no market earnings. This flaw is the single biggest reason the willingness- to-pay approach displaced human capital as the preferred method in modern welfare-economic practice.

The **willingness-to-pay (WTP) approach** instead asks what people themselves would pay for a health improvement, independent of their earning capacity – the same conceptual move underlying VSL above, generalized to morbidity as well as mortality. WTP can be elicited directly (contingent valuation surveys asking "how much would you pay for X% reduction in your risk of Y") or indirectly (revealed preference from real spending and risk-taking behaviour, including the hedonic wage method). WTP is grounded in welfare economics' standard definition of value – what an individual would sacrifice to obtain something, which by construction captures the value of health to retirees, children, and unpaid carers as fully as to high-earning professionals, since it depends on each person's own preferences and budget constraint rather than their market wage.

Difficult terminology: Willingness-to-pay (WTP) approach. WTP asks, directly or indirectly, how much money a person would be willing to give up to obtain a health benefit (or avoid a health harm), and uses that amount as the monetary value of the benefit. The appeal is philosophical consistency with how economists value everything else – goods, services, environmental amenities – by the preferences of the people who hold them, not by any external judgement of worth. The practical difficulty is that stated-preference surveys are vulnerable to hypothetical bias (people answering a survey question differently than they would act with real money on the line) and to the fact that WTP for health is mechanically constrained by ability to pay – a poor person's WTP for an identical health gain is, by definition, capped by a smaller budget than a rich person's, which reintroduces an income-related distortion from a different direction than human capital's wage distortion. Neither approach is therefore equity-neutral; they are differently non-neutral, and that difference is a live methodological debate, not a solved problem.

Approach	How health is valued	Main limitation
Human capital	Discounted future earnings the person would otherwise have produced	Values non-earners (children, retirees, carers) near zero; values identical health gains differently by wage
Willingness-to-pay (general)	What the person themselves would pay for the health improvement, by survey or revealed behaviour	Vulnerable to hypothetical bias in stated-preference surveys; WTP mechanically limited by ability to pay
Value of a statistical life (VSL)	Aggregated WTP for small mortality-risk reductions, scaled up to an implied value per statistical death avoided	Wide variation across agencies/ years (roughly $6M to $20M in current US federal guidance); not transferable across country/sector without income-elasticity adjustment

Table 58: Comparing the two main approaches to monetizing health, and VSL as a specific instance of the WTP approach

HEOR in practice. If you are asked to build a CBA rather than a CEA – most often for a public-health, environmental, or occupational-safety programme rather than a medicines HTA submission – the single most important design decision is which VSL or WTP-per-QALY source to use, and that choice should be stated and justified as explicitly as a discount rate choice, with the resulting net benefit re-run across the published low/central/ high range (as HHS and DOT both now do natively) rather than reported as a single deterministic figure. A CBA that reports only the central VSL estimate, with no sensitivity range, understates how much the bottom-line conclusion depends on a contested input.

24.2 Distributional CEA and equity weighting

Every ICER computed in Chapters 17 through 23 was a **population-average** figure: total incremental cost divided by total incremental QALYs, across the whole modelled cohort. An intervention with an attractively low average ICER can still deliver its costs and its health gains very unevenly across subgroups – by income, by disease severity, by ethnicity, by geography – and a decision based on the average alone is blind to that distribution. Distributional cost-effectiveness analysis (DCEA) is the methodology, developed principally at the University of York's Centre for Health Economics and used in UK policy analysis feeding into NICE's thinking on equity, that makes this distribution visible and lets a decision-maker weigh it explicitly against the average result.

Why the average can mask unequal distribution

Consider a screening programme that, on average across the eligible population, is highly cost-effective. If uptake, benefit, and cost are not spread evenly – say the programme is taken up disproportionately by the least deprived quintile (who are more likely to attend screening appointments, more health-literate, and more likely to act on a positive result), while the opportunity cost of funding it (displaced spending elsewhere in a fixed healthcare budget) falls more heavily,

in relative terms, on more deprived groups who rely more on the services being displaced – the same average ICER could correspond to a programme that **widens** health inequality even as it looks efficient on paper. This is not a hypothetical concern: it is the precise finding of the DCEA literature's most-cited applied example, Asaria and colleagues' analysis of targeted versus universal reminder strategies to increase uptake of bowel cancer screening in England, which found that a targeted strategy produced a **smaller** total health gain than a universal strategy but a **larger** reduction in health inequality across deprivation groups – precisely the kind of efficiency-equity divergence that a single population-average ICER cannot show.

Difficult terminology: Distributional cost-effectiveness analysis (DCEA). DCEA extends ordinary CEA by disaggregating costs and health effects across equity-relevant subgroups – most commonly socioeconomic deprivation quintiles, but also severity of illness, ethnicity, age, or geography – rather than collapsing everything into one population-wide average. It explicitly accounts for **opportunity costs**: health gained by the new intervention has to be weighed against the health that would have been produced by whatever existing care gets displaced to fund it, and that displaced care's benefit is itself unevenly distributed (the population groups who rely most on the services being cut back are not necessarily the same groups who benefit most from the new intervention). The output is not a single number but a **distribution** – how much health gain, and how much health opportunity cost, each subgroup experiences – which can then be summarized with an inequality metric and, if the decision-maker wants, combined with the overall cost-effectiveness result into a single equity-weighted social welfare index. Crucially, DCEA was built specifically to answer questions that an average ICER structurally cannot: not just "is this good value on average" but "who gains, who loses, and does this intervention narrow or widen existing health inequality."

Equity weighting and the equity-efficiency impact plane

Once subgroup-level health effects are visible, a decision-maker who cares about both total health and its distribution needs some way to combine the two into a judgement. Cookson and colleagues' widely-cited framework for this – developed with, and published through, the DCEA/York tradition – is the **equity impact plane**: a two-dimensional chart with the ordinary cost-effectiveness/net-health-benefit result on one axis and a summary measure of the intervention's net equity impact (for example, its effect on the gap in health between the most and least deprived groups) on the other. A programme that improves **both** total health and equity sits in the unambiguous "win-win" quadrant; one that harms both sits in "lose-lose" – in neither case is there a genuine trade-off to make. The interesting, and most common, real-world cases are the other two quadrants: a programme that improves total health but **worsens** equity (efficient but unfair), or one that **narrows** health inequality at the cost of some total health forgone (fair but less efficient). Only in these two quadrants does equity weighting – putting an explicit numeric weight on health gains or losses according to which group receives them, typically weighting gains to more deprived groups more heavily than identical gains to less deprived groups – do real analytical work, because only there does giving more weight to equity actually change which option looks best.

Difficult terminology: Equity weighting. An equity weight is a multiplier, greater than 1 for a disadvantaged group and often fixed at 1 for the reference (typically least deprived or least severely ill) group, applied to the health gain or loss that group experiences before

it is summed into a decision metric. A weight of 1.5 for the most deprived quintile means the analysis is deliberately treating one QALY gained by that group as "worth" 1.5 ordinary QALYs for decision-making purposes – not because it is a biologically different QALY, but as an explicit value judgement that reducing health inequality has worth beyond the raw health gained. The weight itself is not an empirical fact to be estimated from data; it is a social-value judgement that has to come from somewhere – published equity weights have been derived from general-population surveys asking people directly how they would trade off total health against equity, from elicited parameters in formal social welfare functions, or simply specified as a range for sensitivity analysis so a decision-maker can see how much the ranking of options changes as the weight increases from 1 (pure efficiency) upward. Equity weighting is a tool for making a value judgement **transparent and quantified**, not a tool for removing the value judgement altogether.

Common pitfall. Do not present an equity-weighted ICER as though it were a more "accurate" or "corrected" version of the unweighted ICER. The two numbers answer different questions under different ethical premises: the unweighted ICER answers "what is the cost per QALY if every QALY counts equally regardless of who receives it," and the equity-weighted ICER answers "what is the cost per QALY if we have decided, as a value judgement, that QALYs gained by a specified disadvantaged group count for more." Reporting only the equity-weighted figure smuggles in a contestable ethical position as if it were a technical correction; reporting only the unweighted figure silently assumes that distribution never matters. Good practice, consistent with the equity impact plane's purpose, is to report both, plus the specific equity weight used and a plausible range around it, so the decision-maker sees exactly how much of the conclusion depends on the equity judgement rather than the efficiency evidence.

A toy two-subgroup worked example

The arithmetic of equity weighting is simple enough to verify by hand, which is the point of walking through a toy example before trusting any larger model's output. Suppose a screening intervention is rolled out to two equal-sized subgroups of 1,000 patients each – an income-quintile-5 (least deprived) group and an income-quintile-1 (most deprived) group – at an identical cost of $2,000 per patient in both groups, but with a **larger** QALY gain in the most deprived group (0.12 QALYs per patient) than in the least deprived group (0.08 QALYs per patient), reflecting that the most deprived group has more untreated disease burden to begin with and so more capacity to benefit.

```
library(dplyr)

subgroups <- tibble::tibble(
  group           = c("Q1_most_deprived", "Q5_least_deprived"),
  n_patients      = c(1000, 1000),
  cost_per_pt     = c(2000, 2000),
  qaly_gain_pt    = c(0.12, 0.08),
  equity_weight   = c(1.5, 1.0)   # Q1 QALYs up-weighted 1.5x; Q5 left at 1.0
)

subgroups <- subgroups |>
  mutate(
```

```
    total_cost        = n_patients * cost_per_pt,
    total_qaly        = n_patients * qaly_gain_pt,
    weighted_qaly      = total_qaly * equity_weight
  )

total_cost          <- sum(subgroups$total_cost)
total_qaly          <- sum(subgroups$total_qaly)
total_weighted_qaly <- sum(subgroups$weighted_qaly)

icer_unweighted <- total_cost / total_qaly
icer_weighted   <- total_cost / total_weighted_qaly

subgroups
icer_unweighted   # cost per unweighted QALY
icer_weighted     # cost per equity-weighted QALY
```

Working the arithmetic through by hand, exactly as R would: total cost is $2,000 times 2,000 patients = $4,000,000. Total unweighted QALYs are (1,000 times 0.12) + (1,000 times 0.08) = 120 + 80 = 200, so the unweighted ICER is $4,000,000 / 200 = **$20,000 per QALY**. Applying the 1.5x equity weight to the most-deprived group's QALYs only, total equity-weighted QALYs become (120 times 1.5) + (80 times 1.0) = 180 + 80

260, so the equity-weighted ICER is $4,000,000 / 260 = $15,384.62 per QALY (both figures independently verified by direct calculation,

not just the R output).

How to report this.

Metric	Q1 (most deprived)	Q5 (least deprived)	Total / combined
Patients	1,000	1,000	2,000
Cost per patient	$2,000	$2,000	–
Total cost	$2,000,000	$2,000,000	$4,000,000
QALY gain per patient	0.12	0.08	–
Total unweighted QALYs	120	80	200
Equity weight applied	1.5	1.0	–
Total equity-weighted QALYs	180	80	260
Unweighted ICER: $20,000/QALY – Equity-weighted ICER: $15,384.62/QALY			

Table 59: Unweighted vs. equity-weighted ICER for a toy two-subgroup screening example (equity weight of 1.5 applied to the most-deprived quintile's QALYs)

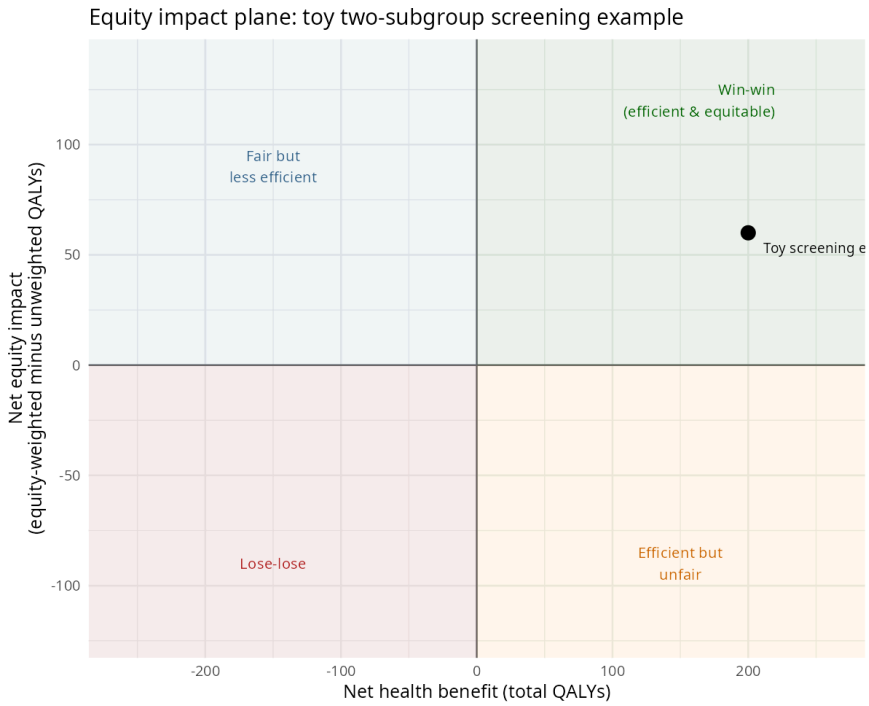


Figure 37: The equity impact plane plotted for Table 59′s toy example: net health benefit (total unweighted QALYs = 200) on the x-axis against net equity impact (equity-weighted minus unweighted QALYs = +60) on the y-axis. This example lands in the win-win quadrant — equity weighting happened to favour the subgroup already receiving the larger QALY gain. Had the most-deprived group received the **smaller** gain instead, the same exercise would land in the "efficient but unfair" quadrant, the genuine trade-off this plane exists to visualize.

The direction of the effect is the point of the illustration, not the specific weight chosen: because the equity weight was applied to the subgroup that was **already** receiving the larger QALY gain, up-weighting that subgroup's QALYs makes the intervention look **more** cost-effective once equity is taken into account ($15,384.62/QALY under equity weighting versus $20,000/QALY unweighted) – the two objectives happen to point the same way here, a "win-win" case on the equity impact plane. Had the most-deprived group instead received the **smaller** QALY gain (an intervention that, for example, is more efficiently delivered to the already-healthier, less deprived group), the same equity weighting exercise would move the two ICERs in **opposite** directions, producing the genuine trade-off the equity impact plane exists to visualize: a result that is efficient on average but regressive in its distribution, with the equity-weighted ICER numerically worse than the unweighted one.

Agency perspective. NICE does not currently apply a formal numeric equity weight inside its standard reference-case cost-utility analysis for technology appraisals – the reference case remains an unweighted QALY, consistent with the "a QALY is a QALY" principle also used throughout Chapters 17-23 of this book. Equity considerations instead enter NICE decision-making through separate mechanisms: a modifier for end-of-life treatments, severity modifiers introduced in 2022 that increase the acceptable cost-per-QALY threshold for diseases causing a larger absolute QALY loss, and qualitative equity considerations raised by committees outside the numeric ICER itself. DCEA and formal equity weighting, by contrast, are used as

supplementary analytical tools – commissioned research, academic analyses feeding into broader policy debate, and in some non-UK jurisdictions' extended CEA work (Section 24.3) – rather than as a mandatory step inside every submitted reference-case model. Knowing which regime you are modelling for determines whether an equity-weighted ICER is an expected deliverable or a supplementary exhibit.

24.3 Extended cost-effectiveness analysis: financial risk protection

Every method in this book so far shares one structural assumption: health gain – a QALY, a DALY averted, a life-year – is the only outcome dimension that matters once costs are on the table. That assumption is reasonable in a high-income system with near-universal insurance, where a diagnosis rarely bankrupts a household. It is far less reasonable in a low- or middle-income country (LMIC) context with large out-of-pocket (OOP) health spending, where the **same** clinical event – a TB diagnosis, a complicated delivery, a cancer admission – can simultaneously improve health and push a family into poverty. Extended cost-effectiveness analysis (ECEA, also written eCEA or EEA) is the method built specifically to put that second consequence on the same footing as the first.

ECEA originates from the Disease Control Priorities Network's third edition (DCP3) and the associated Gates Foundation-funded programme of work, most closely associated with Stéphane Verguet and colleagues (Verguet, Laxminarayan & Jamison 2015; Verguet, Kim & Jamison 2016). Its central move is to compute, alongside the conventional health and cost outcomes, an explicit **financial risk protection** (FRP) outcome: how many households are protected from catastrophic health expenditure, or from falling into poverty, by a policy – a public health programme, a subsidy, an insurance expansion – that reduces out-of-pocket spending. A policy can therefore be judged cost-effective on health grounds **and** valuable on financial-protection grounds, and the two numbers are reported side by side rather than collapsed into one.

Difficult terminology: Catastrophic health expenditure (CHE). A household's health spending is classified as **catastrophic** when it exceeds some fixed fraction of the household's income or total consumption in a given period – a common convention, and the one used in this section's worked example, is 10% of income, though national studies sometimes use 25% or 40% depending on the policy question. The threshold is arbitrary in the same way a p-value threshold of 0.05 is arbitrary: it is a convention chosen to be policy-relevant, not a biologically or economically derived cut point. The intuition is simple even though the word sounds technical – a household earning $1,000 a year that pays $150 out of pocket for a hospitalization (15% of income) has crossed a 10% catastrophic-expenditure threshold, whereas a household earning $10,000 paying the same $150 (1.5% of income) has not, even though both households spent an identical dollar amount. This is precisely why ECEA tracks financial risk protection **by income quintile** rather than as a single population-wide number: the same absolute OOP cost has a completely different welfare consequence depending on where a household sits in the income distribution, which a conventional CEA computed only in aggregate cost and aggregate QALYs would never reveal.

Difficult terminology: Poverty cases averted. This is ECEA's second, related financial-risk-protection metric, counting a different event: not whether spending crossed a fixed percentage-of-income threshold, but whether spending pushed a household's **remaining** income below an absolute poverty line (e.g. a national or international $-per-day poverty line) that it would otherwise have stayed above. A household can face catastrophic expenditure without being pushed into poverty (a wealthy household spending 15% of a large income), and a household can be pushed into poverty by health spending that is not catastrophic by the 10% convention (a household just above the poverty line paying a moderate OOP amount that is nonetheless enough to tip it under the line). Verguet, Olson and colleagues' original formulation (DCP3, 2015-2017) treats poverty-cases-averted and CHE-cases-averted as complementary, not interchangeable, FRP metrics – a full ECEA reports both where data allow, and this section's worked example computes both for exactly that reason.

The conceptual shift ECEA makes relative to the cost-effectiveness analyses built in Chapters 17-22 is best stated plainly: a standard CEA asks "how much health does this buy, and at what cost to the payer or health system." ECEA asks that question **and** a second one: "how much financial protection does this buy for the households who would otherwise have paid out of pocket, and how is that protection distributed across the income spectrum." Both outcomes come from the same underlying intervention and the same underlying cost data – ECEA does not require a separate clinical trial or a separate cost study, only that OOP cost and household income data be available (or credibly imputed) alongside the clinical and system-cost data a conventional CEA already needs.

Agency perspective. ECEA was built for, and is used almost exclusively in, LMIC and universal-health-coverage (UHC) policy contexts – WHO's UHC agenda, World Bank health-financing work, and country-level priority-setting exercises in settings such as India, Ethiopia, and Nigeria are the method's natural home. It is not a NICE or CADTH reference-case requirement, and it would be unusual to see a full ECEA in a UK or Canadian single-technology appraisal, where near-universal coverage makes catastrophic OOP spending largely a non-issue for the technologies under appraisal. Where this book's TB/India thread (Chapters 5, 9, 19, 21) is concerned, ECEA is exactly the kind of analysis that would accompany a universal-public-finance-for-TB-treatment policy question – indeed, Verguet and colleagues' 2015 **Health Economics** paper, "Universal public finance of tuberculosis treatment in India: an extended cost-effectiveness analysis," is a direct, published instance of this method applied to this book's own running disease area and country setting.

Worked example. Consider a stylized version of exactly that India-TB policy question: a government is deciding whether to move from the status quo, in which TB patients pay a meaningful share of diagnosis and treatment costs out of pocket, to a universal-public-finance policy in which the government covers nearly all of those costs. We work this through for five income quintiles of 20,000 households each (100,000 households total), using simple uniform income distributions within each quintile band – a simplification for illustration; a real ECEA would use an empirical household expenditure survey, not an assumed uniform distribution.

```
library(dplyr)

ecea_frp <- function(
    quintile_income_bands = list(
```

```
      Q1 = c(300, 700),    Q2 = c(700, 1100),  Q3 = c(1100, 1700),
      Q4 = c(1700, 2700),  Q5 = c(2700, 5300)
    ),
    pop_per_quintile = 20000,
    oop_status_quo   = 150,   # $ per TB episode, status quo
    oop_new_policy    = 20,   # $ per TB episode, after universal public finance
    che_threshold     = 0.10, # 10% of income convention
    poverty_line      = 400   # $ per year
) {
  che_income_cutoff <- function(oop) oop / che_threshold

  frac_below <- function(cutoff, lo, hi) {
    pmax(0, pmin(1, (cutoff - lo) / (hi - lo)))
  }
  frac_in_interval <- function(lo, hi, a, b) {
    left  <- max(lo, a); right <- max(min(hi, b), left)
    (right - left) / (hi - lo)
  }

  bands <- tibble(
    quintile = names(quintile_income_bands),
    income_lo = sapply(quintile_income_bands, `[`, 1),
    income_hi = sapply(quintile_income_bands, `[`, 2)
  )

  thresh_sq  <- che_income_cutoff(oop_status_quo)
  thresh_new <- che_income_cutoff(oop_new_policy)

  bands %>%
    mutate(
      che_cases_sq    = frac_below(thresh_sq,  income_lo, income_hi) *
pop_per_quintile,
      che_cases_new   = frac_below(thresh_new, income_lo, income_hi) *
pop_per_quintile,
      che_averted     = che_cases_sq - che_cases_new,
      pov_cases_sq    = frac_in_interval(income_lo, income_hi,
                                         poverty_line, poverty_line +
oop_status_quo) * pop_per_quintile,
      pov_cases_new   = frac_in_interval(income_lo, income_hi,
                                         poverty_line, poverty_line +
oop_new_policy) * pop_per_quintile,
      pov_averted     = pov_cases_sq - pov_cases_new
    ) %>%
    select(quintile, che_cases_sq, che_cases_new, che_averted,
           pov_cases_sq, pov_cases_new, pov_averted)
}

frp_result <- ecea_frp()
frp_result
```

Working the arithmetic through by hand confirms the function's output. A household crosses the catastrophic-expenditure threshold when its income falls below $\frac{\text{OOP cost}}{0.10}$: at the status-quo OOP cost of \$150 that cutoff is \$1,500, and at the new-policy OOP cost of \$20 it is \$200. Comparing those cutoffs against each quintile's income band: Q1 (\$300-\$700) is entirely below \$1,500, so all

20,000 households there are classified catastrophic under the status quo; the same is true of Q2 ($700-$1,100); Q3 ($1,100-$1,700) straddles $1,500, so only the $\frac{1500-1100}{1700-1100} = \frac{2}{3}$ of that band below the cutoff – $13,333.3$ households – are catastrophic; Q4 and Q5 sit entirely above $1,500 and contribute zero. Summing gives $20,000 + 20,000 + 13,333.3 + 0 + 0 = 53,333.3$ catastrophic cases under the status quo. Under the new policy the $200 cutoff falls below every quintile's income floor except Q1′s, where only the share of the band below $200 (an empty interval here, since Q1 starts at $300) counts – zero. So the new policy reduces catastrophic cases from 53,333.3 to 0, for **53,333.3 catastrophic health expenditure cases averted**, concentrated entirely in the poorest three quintiles.

The poverty calculation works differently: a household falls into poverty when its income sits just above the $400 poverty line by no more than the OOP amount it pays, i.e. in the interval ($400, $400 + OOP). Under the status quo (OOP = $150) that interval is ($400, $550), which overlaps only Q1′s band ($300-$700): the overlap is ($400, $550), a width of $150 out of Q1′s $400-wide band, giving $\frac{150}{400} = 0.375$ of Q1′s households, or $0.375 \times 20,000 = 7,500$ poverty cases. Under the new policy (OOP = $20) the interval shrinks to ($400, $420), a width of $20 out of the same $400 band, giving $\frac{20}{400} = 0.05$, or $1,000$ poverty cases. The policy therefore averts $7,500 - 1,000 = 6,500$ poverty cases, again concentrated in Q1 – exactly the pro-poor distributional pattern the DCP3 case studies (South Africa salt policy, Niger/China tobacco tax, India TB financing) consistently report.

How to report this.

Quintile	CHE cases, status quo	CHE cases, new policy	CHE cases averted	Poverty cases averted
Q1 (poorest)	20,000	0	20,000	6,500
Q2	20,000	0	20,000	0
Q3	13,333	0	13,333	0
Q4	0	0	0	0
Q5 (richest)	0	0	0	0
Total	53,333	0	**53,333**	**6,500**

Illustrative uniform-within-band income distribution, 20,000 households per quintile. CHE: catastrophic health expenditure, threshold 10% of income. Poverty line $400/year.

Table 60: Extended cost-effectiveness analysis: financial risk protection by income quintile, universal public finance of TB treatment versus status quo (illustrative).

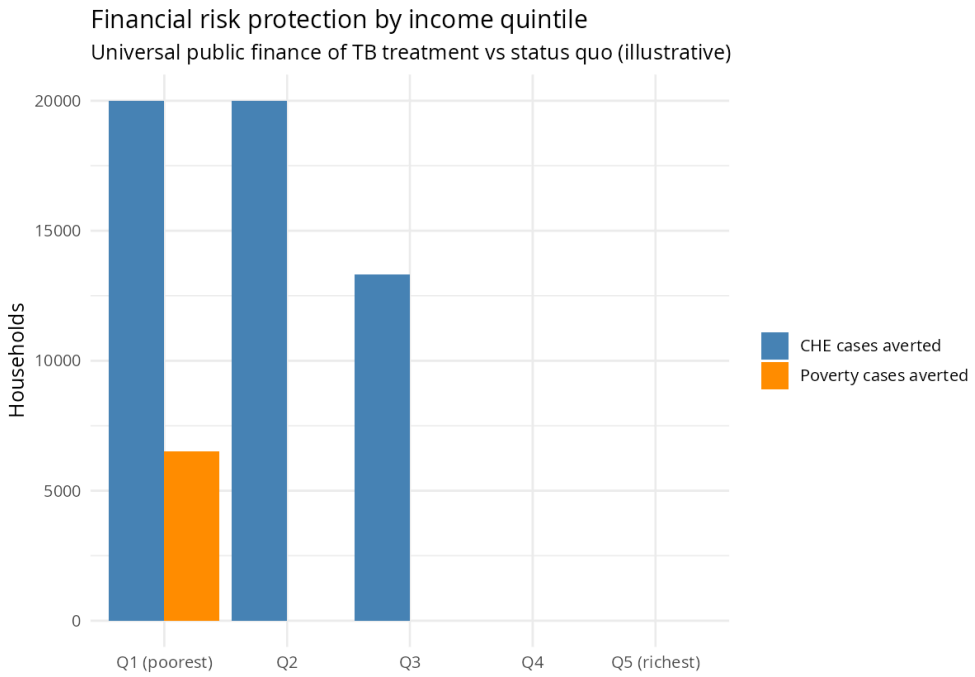


Figure 38: Financial risk protection outcomes from Table 60 plotted by income quintile: catastrophic-health-expenditure cases averted (blue) and poverty cases averted (orange). Essentially all of the policy's financial protection accrues to the three poorest quintiles — Q4 and Q5 see zero effect — exactly the pro-poor distributional pattern the pooled total (53,333 CHE cases averted) alone would hide.

HEOR in practice. Report financial risk protection results **by quintile**, never only as a single pooled figure. The pooled total of 53,333 CHE cases averted is true but hides the policy-relevant finding: essentially all of the financial protection accrues to the three poorest quintiles, with zero effect on Q4 and Q5. A payer or ministry deciding between a universal subsidy and a means-tested subsidy targeted at the bottom three quintiles needs exactly this quintile breakdown to see that the latter could capture almost the same financial-protection benefit at a fraction of the fiscal cost – a comparison the aggregate number alone cannot support.

Common pitfall. ECEA's financial risk protection outcome is not a substitute for a conventional cost-effectiveness analysis, and it is not denominated in the same units as a QALY or an ICER – there is no accepted exchange rate for converting "catastrophic health expenditure cases averted" into dollars or QALYs, and attempting to force one onto a single combined metric defeats the method's purpose. The two outcomes (health gain and financial risk protection) are reported **in parallel**, as DCP3's own case studies consistently do, precisely because they answer different policy questions: is this intervention good value for health, and does it also protect households from a specific and separate harm. A submission that quietly adds a monetized FRP benefit into the numerator of an ICER has silently changed what the ICER means and should be flagged as such.

24.4 Multi-criteria decision analysis (MCDA) in R

Every method covered in this book so far – cost-effectiveness analysis, budget impact analysis, extended CEA – ultimately reduces a decision to one or two explicit numeric axes: cost and a single health outcome (or cost and a financial-protection outcome, in §24.3′s case). Many real coverage decisions are not actually made on one or two axes. A reimbursement committee weighing a new orphan drug is also, explicitly or not, weighing disease severity, unmet need, quality of the clinical evidence, budget affordability, and sometimes equity or innovation considerations that have no natural place inside an ICER. Multi-criteria decision analysis (MCDA) is the family of methods built to make that multi-axis weighing **explicit and reproducible** rather than leaving it as an unstated, inconsistent judgement call made differently in every committee meeting.

The general MCDA idea is simple to state even though implementations vary: define a set of criteria relevant to the decision (efficacy, safety, disease severity, budget impact, quality of evidence, equity, and so on); assign each criterion a **weight** reflecting how much the decision-maker cares about it relative to the others (weights conventionally sum to 1); score each alternative (each drug, each programme) on each criterion, usually on a common 0-100 or 0-1 scale after normalizing the criterion's raw units away; and combine weights and scores into a single overall score per alternative, most commonly by a **weighted sum** (additive combination) or occasionally a **weighted product** (multiplicative combination). The alternative with the highest combined score is, by the model's own logic, the most preferred – though MCDA's real value is usually less about producing a single ranking and more about forcing the weighting judgement into the open where it can be debated, revisited, and sensitivity-tested, exactly as a one-way or probabilistic sensitivity analysis does for an ICER.

> **Difficult terminology: Weighted sum versus weighted product.** A **weighted sum** combines criterion scores by multiplying each score by its weight and adding the results: score $= \sum_i w_i \cdot s_i$. It assumes criteria are, in effect, **compensatory** – a low score on one criterion can always be offset by a high enough score on another, in direct proportion to their weights. A **weighted product** instead multiplies each score raised to the power of its weight: score $= \prod_i s_i^{w_i}$. Because any score of zero on any criterion drives the whole product to zero regardless of how well the alternative scores elsewhere, the weighted product is much less forgiving of a catastrophic failure on a single criterion – useful when a decision-maker genuinely wants "unacceptable on safety" to disqualify an alternative outright rather than be averaged away by strong efficacy, but inappropriate if criteria are meant to trade off freely against each other. Most published health-sector MCDA applications, including EVIDEM (below), use a weighted-sum-style additive combination as the default.

Several dedicated MCDA R packages exist on CRAN – the long-standing `MCDA` package provides a `weightedSum()` function for exactly the additive combination described above, and newer packages such as `mcdabench` implement a much larger family of methods (TOPSIS, PROMETHEE, ELECTRE, VIKOR, and more) with built-in normalization and weight-sensitivity analysis. Because this book's standing discipline is to show only function signatures independently verified against current documentation rather than a remembered or guessed interface, and because the weighted-sum logic itself is genuinely simple, the worked example below builds the scoring pipeline directly in base R and `dplyr` – the identical arithmetic a dedicated package's `weightedSum()`-style function

would perform, just written out so every step is visible and auditable, which is itself a defensible choice for a committee-facing analysis that needs to be checked by non-R-programmer reviewers.

Worked example. Three hypothetical drugs for the same indication are scored by a committee on four criteria – efficacy, safety, disease severity addressed, and economic value – each on a 0-100 scale, with committee-agreed weights of 35% efficacy, 25% safety, 20% severity, 20% economic value (summing to 1).

```
library(dplyr)

mcda_weighted_score <- function(scores, weights) {
  # scores:  a named list/data frame of criterion scores per alternative
  # weights: a named numeric vector of criterion weights, must sum to 1
  stopifnot(abs(sum(weights) - 1) < 1e-8)

  scores %>%
    rowwise() %>%
    mutate(
      weighted_sum = sum(c_across(names(weights)) * weights[names(weights)]),
      weighted_product = prod(c_across(names(weights))^weights[names(weights)])
    ) %>%
    ungroup()
}

weights <- c(efficacy = 0.35, safety = 0.25, severity = 0.20, economic = 0.20)

drug_scores <- tibble::tribble(
  ~drug,     ~efficacy, ~safety, ~severity, ~economic,
  "Drug A",  80,        70,      60,        50,
  "Drug B",  60,        90,      60,        70,
  "Drug C",  90,        50,      80,        40
)

mcda_result <- mcda_weighted_score(drug_scores, weights)
mcda_result %>% arrange(desc(weighted_sum))
```

Working the weighted sum through by hand for Drug A: $80 \times 0.35 + 70 \times 0.25 + 60 \times 0.20 + 50 \times 0.20 = 28 + 17.5 + 12 + 10 = 67.5$. For Drug B: $60 \times 0.35 + 90 \times 0.25 + 60 \times 0.20 + 70 \times 0.20 = 21 + 22.5 + 12 + 14 = 69.5$. For Drug C: $90 \times 0.35 + 50 \times 0.25 + 80 \times 0.20 + 40 \times 0.20 = 31.5 + 12.5 + 16 + 8 = 68$. The weighted-sum ranking is therefore Drug B (69.5) > Drug C (68.0) > Drug A (67.5) – Drug B's combination of strong safety and reasonable scores elsewhere edges out Drug C's higher efficacy and severity scores, because safety carries a larger weight (25%) than either of the criteria on which Drug C leads. The weighted-product scores (Drug A: 66.49, Drug B: 68.48, Drug C: 64.53, each computed as $\prod s_i^{w_i}$) preserve the same ranking here because no criterion score is close to zero for any drug, but the gap between Drug B and Drug C widens slightly under the product rule, illustrating the point made above: the multiplicative combination punishes Drug C's relatively weaker safety score (50) more heavily than the additive rule does.

How to report this.

Drug	Efficacy	Safety	Severity	Economic	Weighted sum	Weighted product
Drug A	80	70	60	50	67.50	66.49
Drug B	60	90	60	70	69.50	68.48
Drug C	90	50	80	40	68.00	64.53

Weights: efficacy 0.35, safety 0.25, severity 0.20, economic 0.20 (sum = 1.00). All scores on a committee-assigned 0-100 scale.

Table 61: MCDA weighted-sum and weighted-product scores for three illustrative drugs, equal-criteria committee weighting.

Common pitfall. The ranking above is entirely a function of the committee's chosen weights, not an objective fact about the three drugs – halve the safety weight and double the economic weight and Drug B's lead can disappear. Every MCDA result reported to a decision body should be accompanied by a weight-sensitivity analysis (re-running the same `mcda_weighted_score()` call across a plausible range of alternative weight sets, exactly as a one-way sensitivity analysis re-runs an ICER across a plausible range of input values) rather than presented as a single fixed ranking. Without this, MCDA can create a false appearance of objectivity around what is, by construction, an aggregation of subjective value judgements.

Difficult terminology: EVIDEM. EVIDEM (Evidence and Value: Impact on DEcisionMaking) is the most widely cited health-sector-specific MCDA framework, developed and field-tested from around 2008 onward (Goetghebeur et al.) and piloted or used operationally by several HTA bodies, including a pan-Canadian appraisal pilot and a Canadian public-payer field test for a specific drug coverage decision. EVIDEM structures the generic weighted-sum logic above into two components: a **MCDA Core Model** of roughly 14-15 quantifiable criteria (grouped into quality-of-evidence, disease, intervention, and economic domains – disease severity, size of population affected, unmet need, comparative efficacy/safety/patient-reported outcomes, and cost-effectiveness among them) that are weighted and scored exactly as this section's worked example does; and a separate **Contextual Tool** covering roughly six qualitative, non-quantifiable considerations (ethical principles such as fairness and the goal of healthcare, and system-level considerations such as political priorities or organizational capacity) that are deliberated rather than scored numerically, because EVIDEM's developers judged that forcing a number onto "is it ethically acceptable to prioritize this population" would misrepresent the nature of that judgement. The two-part structure is EVIDEM's central methodological claim: that a defensible health-sector MCDA needs both an explicit, numeric, auditable core (what this section's R code produces) and an explicit space for the value judgements that genuinely cannot be reduced to a weight and a score.

24.5 Severity modifiers, value-based pricing, and managed entry agreements

The three methods in this chapter so far – societal-perspective cost-benefit analysis, distributional/equity-weighted CEA, and ECEA's financial-risk-protection outcome – all share one feature: they modify or supplement the **outcome** side of the value equation. This closing section covers three mechanisms that act on the **decision threshold and price** side instead: how a health system can explicitly weight QALYs by disease severity before applying its usual cost-effectiveness threshold, how value-based pricing sets a price directly from estimated value rather than from a cost-plus or reference-pricing convention, and how managed entry agreements (MEAs) let a payer and manufacturer agree a price conditional on real-world performance or simply a confidential discount – closing the loop back to Chapter 23's budget-impact worked example, where an MEA scenario already appeared as a straightforward parameter change.

NICE's severity modifier. From January 2022, NICE's technology appraisal methods manual replaced its older "end of life" modifier with a broader **severity modifier**, applying a greater weight to QALYs gained for conditions judged sufficiently severe. The mechanism is deliberately formulaic: NICE computes the **absolute QALY shortfall** (the difference between the QALYs a general population of the same age and sex would expect over their remaining lifetime, and the QALYs people with the condition under current standard care are expected to have) and the **proportional QALY shortfall** (that same shortfall expressed as a fraction of the general population's expected QALYs). Table 6.1 of NICE's manual then assigns a QALY weight of 1 (no uplift), 1.2, or 1.7 depending on where those two shortfall measures fall, using whichever of the two implies the **higher** severity category.

> **Difficult terminology: QALY shortfall (absolute and proportional).** Both shortfall measures answer the same underlying question – how much worse off, in QALY terms, is someone with this condition compared to someone of the same age and sex without it – but express it differently. **Absolute** shortfall is a QALY count: if the general population would expect 20.4 remaining QALYs and people with the condition under current NHS care would expect only 4.2, the absolute shortfall is $20.4 - 4.2 = 16.2$ QALYs. **Proportional** shortfall expresses the same gap as a fraction of what the general population would have had: $16.2/20.4 \approx 0.79$, meaning this condition costs its patients roughly 79% of the healthy-population QALY expectation. The two measures can disagree about which severity band a condition falls into – a condition affecting mostly elderly patients, who have less remaining QALY expectation even in good health, might show a modest absolute shortfall but a large proportional one, while a condition striking young patients early could show the reverse – which is exactly why NICE's rule takes whichever of the two implies the higher weight, rather than picking one measure and discarding the other.

```
# Illustrative NICE-style severity-modifier calculation and its effect
# on a submitted ICER -- not a real appraisal, parameters chosen for
# the worked example only.

qale_general_population <- 20.4    # expected remaining QALYs, matched age/sex
qale_with_condition     <- 4.2     # expected remaining QALYs under current NHS
care
```

```r
absolute_shortfall     <- qale_general_population - qale_with_condition
proportional_shortfall <- absolute_shortfall / qale_general_population

severity_weight <- function(abs_shortfall, prop_shortfall) {
  # Table 6.1, NICE health technology evaluations manual (PMG36), 2022
  if (prop_shortfall >= 0.95 || abs_shortfall >= 18) {
    1.7
  } else if (prop_shortfall >= 0.85 || abs_shortfall >= 12) {
    1.2
  } else {
    1
  }
}

w <- severity_weight(absolute_shortfall, proportional_shortfall)

incremental_cost <- 2400   # $ per patient
incremental_qaly <- 0.08   # unweighted QALY gain per patient

icer_unweighted <- incremental_cost / incremental_qaly
icer_weighted   <- incremental_cost / (incremental_qaly * w)

list(
  absolute_shortfall = absolute_shortfall,
  proportional_shortfall = round(proportional_shortfall, 3),
  severity_weight = w,
  icer_unweighted = icer_unweighted,
  icer_weighted = icer_weighted
)
```

Working this through by hand: the absolute shortfall is $20.4 - 4.2 = 16.2$ QALYs, and the proportional shortfall is $\frac{16.2}{20.4} \approx 0.794$. Against Table 6.1's bands, 16.2 falls in the "12 to 18" absolute-shortfall band (weight 1.2), and 0.794 falls below the "0.85 to 0.95" proportional-shortfall band (weight 1, since it is under 0.85) – taking whichever measure implies the **higher** weight gives an overall severity weight of **1.2**. Applying that weight to the unweighted ICER of $\frac{2400}{0.08} = \$30,000$ per QALY gives a severity-adjusted ICER of $\frac{2400}{0.08 \times 1.2} = \frac{2400}{0.096} = \$25,000$ per QALY – the same drug, same trial data, same cost, now appears more cost-effective purely because its QALY gain is uplifted for treating a condition with a substantial health shortfall under current care.

> **HEOR in practice.** A severity-weighted ICER is not a trick for making an unaffordable drug look affordable – it is NICE's explicit, published, formulaic way of saying that a QALY gained for someone with very little remaining healthy life expectancy is judged more valuable than a QALY gained for someone closer to general-population health, a position grounded in the same equity reasoning Chapter 24's earlier sections built up through distributional CEA and equity weights. The modifier only applies in technology appraisals (not routinely in diagnostics or highly specialised technology evaluations, where severity is judged to already be captured elsewhere in the process), and the shortfall calculation itself needs the same rigour as any other model input: a recent, robust source for general-population EQ-5D and survival data, and discounting at the reference-case rate, exactly as the standard cost-effectiveness calculations in Chapters 17-20 require.

Value-based pricing. Where a severity modifier adjusts the **QALY side** of the cost-effectiveness equation, value-based pricing (VBP) works on the **price side** directly: rather than setting a drug's price from manufacturing cost plus margin, or by referencing prices in comparator countries, VBP sets the price at (or just below) the level implied by the health system's own cost-effectiveness threshold, given the drug's estimated incremental benefit. In principle this is the mirror image of the standard ICER calculation already used throughout this book: instead of computing $\text{ICER} = \frac{\text{incremental cost}}{\text{incremental QALY}}$ and comparing it to a threshold, VBP rearranges the same relationship to solve for the maximum acceptable price – $\text{price} = \text{threshold} \times \text{incremental QALY} + \text{cost of comparator}$ – so that the resulting ICER lands exactly at the threshold. NICE's 2014 VBP consultation ultimately did not replace the standard appraisal process with formal value-based pricing, but the underlying arithmetic – "what is the highest price at which this ICER still clears our threshold" – is exactly the threshold-price calculation every HTA submission in this book implicitly performs when testing a drug's current or hypothetical price against a cost-effectiveness threshold (Chapters 17, 19, 20).

Managed entry agreements. Chapter 23's §23.4 already built a worked managed-entry-agreement (MEA) scenario directly into `build_bia()`, showing that a confidential price concession on the same uptake path reduced the five-year cumulative budget impact by $7,500,000 relative to the list-price base case. That example is worth re-reading here rather than re-deriving, because it illustrates the most common MEA type in practice – a **financial-based** agreement, where the manufacturer simply agrees a confidential discount, rebate, price-volume agreement, or cap on total spend, with no requirement to track real-world clinical performance at all.

Type	Mechanism	What is tracked
Financial-based	Confidential discount, rebate, price-volume agreement, or budget cap negotiated at launch	Nothing beyond utilization/ volume – no clinical outcome data collection required
Outcomes-based (coverage with evidence development)	Price or reimbursement conditional on evidence generated after launch, often via a registry or real-world data collection	Population-level outcomes (e.g. survival, response rate) measured in a defined post-launch cohort
Outcomes-based (patient-level / pay-for-performance)	Payer pays, or is refunded, conditional on an individual patient's own response to treatment	Individual patient outcome against a pre-agreed clinical response criterion

Table 62: Managed entry agreement types.

Difficult terminology: Outcomes-based managed entry agreement. An outcomes-based MEA ties the price actually paid – or a refund – to evidence of how well the treatment performs after launch, rather than fixing a discount at launch regardless of subsequent performance. The two broad sub-types differ in what is measured: a **coverage-with-evidence-development** agreement, as the name suggests, is really a conditional reimbursement decision – the payer covers the drug now in exchange for a commitment to collect further population-level evidence (commonly through a mandatory registry) that will inform a formal re-appraisal

once that evidence matures. A **patient-level** outcomes-based agreement is narrower and more operationally demanding: an individual patient's own clinical response (e.g. tumour response at a defined scan, a biomarker threshold, time to progression) determines whether the manufacturer is paid in full, paid partially, or refunds the payer for that specific patient's treatment course. Patient-level agreements require clinical infrastructure most health systems do not have at scale – defining, capturing, and auditing a response criterion for every treated patient – which is exactly why financial-based agreements, needing none of that infrastructure, remain far more common internationally despite outcomes-based agreements being the more frequently **discussed** type in the HTA literature.

Common pitfall. An outcomes-based MEA shifts financial risk from the payer to the manufacturer only to the extent the outcome-measurement infrastructure is genuinely reliable – a registry with poor case ascertainment, inconsistent outcome definitions across sites, or incomplete follow-up can let a manufacturer appear to meet performance criteria that would not hold up under a cleaner data collection design, which is precisely why coverage-with-evidence-development schemes are typically paired with an explicit re-appraisal date and a pre-specified evidence standard, not left open-ended. Conversely, treat Chapter 23′s §23.4 point as the general rule for **financial**-based MEAs specifically: because they require no new outcome data at all, they are far simpler to model (a straight price-parameter change to the same budget-impact or cost-effectiveness function, exactly as `build_bia()` demonstrated) and far simpler to audit, which is the main reason payers reach for them first even where an outcomes-based structure might in principle be more clinically appropriate.

Agency perspective. Confidentiality is the norm, not the exception, for MEA pricing in nearly every jurisdiction that uses them – published list prices and HTA submissions routinely show the pre-discount price, with the true net price known only to the payer and manufacturer, which is exactly why Chapter 23′s worked example described its MEA scenario's discounted net cost as "confidential" rather than reporting an observed real-world figure. This has a direct modelling consequence worth restating from this chapter's and Chapter 23′s running theme: every MEA scenario belongs in the model as a **parameter**, callable on demand once a negotiation concludes, rather than hard-coded into a single committed model version – the same discipline Chapter 23 established for budget-impact scenarios applies without modification to a cost-effectiveness or ECEA model incorporating a confidential net price.

PART V

Uncertainty, Value of Information and Model Validation

Chapter 25. Handling Uncertainty

Learning objectives

- Distinguish parameter, structural and methodological uncertainty from heterogeneity, and explain why each calls for a genuinely different handling (Briggs, Weinstein et al.'s ISPOR-SMDM Task Force-6 taxonomy).
- Conduct one-way and two-way deterministic sensitivity analysis and construct a tornado diagram that ranks parameters by their influence on net monetary benefit (NMB).
- Write a reusable R scenario-analysis script that re-runs a model under named, deliberately chosen parameter bundles, and distinguish this from both deterministic sensitivity analysis and probabilistic sensitivity analysis.
- Choose an appropriate probabilistic sensitivity analysis (PSA) distribution per HEOR parameter type – Beta, Gamma/lognormal, Dirichlet, (multivariate) normal – and assess Monte Carlo error via a convergence check.
- Build and interpret the four standard PSA visualisations: the cost-effectiveness plane, the CEAC, the CEAF, and the NMB distribution.
- Conduct a threshold analysis to solve for the breakeven value of a single parameter at which a model's ICER crosses a stated willingness-to-pay threshold.
- Explore structural uncertainty by building a genuine alternative model structure, and combine multiple structures into a single result via model averaging, distinguishing it from plain scenario reporting.
- Analyse patient-level trial cost and QALY data directly via the non-parametric bootstrap, correctly preserving within-patient cost-QALY correlation, and build a bootstrap cost-effectiveness plane and CEAC from real trial data.

Chapters 17 through 24 built and extended a series of decision-analytic and value-assessment models, each reporting a single headline result – an ICER, a budget impact, an equity-weighted ICER, a severity-adjusted ICER. Every one of those headline numbers was computed at one fixed set of input values: the base case. This chapter turns from building models to interrogating them, asking the question a single base-case result can never answer on its own: how much should a decision-maker actually trust that number, given that every input feeding into it is itself uncertain, contested, or only approximately known? Chapter 19′s §19.8 explicitly deferred part of this question here – "Chapter 25 covers the general conventions for choosing PSA distributions for different parameter types" – and this chapter delivers on that promise in full, building on (not repeating) the Dirichlet and Cholesky machinery §19.8 already introduced for transition-matrix rows and correlated parameters.

25.1 Types of uncertainty: parameter, structural, methodological and heterogeneity

Chapter 19′s §19.8 flagged that whole transition-matrix rows need a Dirichlet distribution rather than independent Betas per cell, and promised that "Chapter 25 covers the general conventions for choosing PSA distributions for different parameter types." Before getting there (§25.3-§25.4), this

section draws the conceptual map those conventions sit inside. "Uncertainty" in a HEOR model is not one thing – treating all of it the same way is a common and consequential error, because the four kinds below call for genuinely different handling, and conflating them produces either wasted analytical effort or, worse, a misleading answer presented with false confidence.

The standard reference for this taxonomy is Briggs, Weinstein, Fenwick, Karnon, Sculpher and Paltiel's "Model Parameter Estimation and Uncertainty Analysis: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force-6" (**Value in Health**, 2012, and **Medical Decision Making**, 2012) – the sixth instalment of the same ISPOR-SMDM Task Force series already relied on in this book for state-transition modelling (Chapter 19) and model transparency (§19.9). Task Force-6′s Table 1 distinguishes stochastic (first-order) uncertainty, parameter (second-order) uncertainty, heterogeneity, and structural uncertainty, and separately flags methodological uncertainty as "a specific type that can be as important as parameter uncertainty." This section works through parameter, structural and methodological uncertainty, then heterogeneity – deliberately last, because it is the one that is **not** uncertainty at all, and the contrast lands better once the other three are fresh.

Parameter uncertainty

Parameter uncertainty is imprecision in the estimate of a single input value – a transition probability, a unit cost, a utility weight, a hazard ratio – arising because that estimate was derived from a finite, noisy sample rather than observed with certainty on the whole population of interest. Task Force-6′s own illustration is almost comically simple and exactly transferable to our running example: if you estimate the probability of "heads" from 100 coin flips, your estimate carries a standard error, and a different 100 flips would have given a slightly different number. The transition probability $p(\text{OnTx} \rightarrow \text{Fail}) = 0.08$ used throughout Chapter 19 is exactly this kind of quantity – estimated from some finite number of trial or cohort patients, carrying a standard error (or, in the Dirichlet framing of §19.8, a pseudo-count strength) that a properly conducted sensitivity analysis must propagate.

Parameter uncertainty is the uncertainty type PSA is built to address, and the one that occupies the bulk of this chapter: §25.3 covers choosing a distribution family per parameter type, and §25.5 onward runs full probabilistic analysis over all of them jointly. It is also, not coincidentally, the uncertainty type with the cleanest and most mechanical fix – more data narrows a standard error, in a way that no amount of additional data can resolve a **structural** disagreement about what the model should even look like.

> **Difficult terminology: stochastic (first-order) uncertainty.** Task Force-6 is careful to separate parameter uncertainty from a related but distinct idea: **stochastic** or **first-order** uncertainty, which is the randomness in what happens to one simulated patient (or one simulated run) even when every input parameter is held at a single, perfectly known value. Flip a fair coin whose true probability of heads you know exactly to be 0.5 – the **outcome** of any one flip is still random; that randomness is stochastic uncertainty, not parameter uncertainty (there is nothing uncertain about the 0.5). A cohort Markov model like Chapter 19′s, which moves a whole proportion of the cohort deterministically each cycle, has essentially no first-order uncertainty to speak of – it only ever has one "run" for a given set of parameters. The individual-level microsimulation model of Chapter 21, by contrast, simulates one patient's random path through the model at a time, so re-running it with **identical** parameters still

gives a slightly different cost and QALY total purely from random patient-path noise – first-order uncertainty baked into the simulation mechanics, on top of (and conceptually separate from) whatever parameter uncertainty is layered over it. The practical consequence: assessing parameter uncertainty cleanly in a microsimulation requires enough replications (or enough simulated patients per PSA draw) that stochastic noise is averaged out before parameter uncertainty is read off the results – otherwise the two sources of spread get tangled together in the output.

Structural uncertainty

Structural uncertainty is uncertainty about the **model itself** – its shape, not its inputs. Task Force-6's Table 1 defines it as "the assumptions inherent in the decision model," with the companion statistical-modelling analogy of **model uncertainty**: the choice of a linear versus a log-linear regression form. In HEOR modelling, structural choices recur throughout this book's Part IV: whether to build a Markov cohort model or a partitioned-survival model for the same decision problem (Chapter 17 §17.2, revisited at the close of Chapter 19); which parametric survival distribution – Weibull, log-logistic, generalised gamma – to extrapolate a Kaplan-Meier curve with beyond trial follow-up (Chapter 10); which comparator set to include (does a cost-effectiveness analysis of the new TB regimen include only the current standard of care, or also a plausible but not-yet-standard alternative regimen); how many health states to use (the 4-state TB model collapses several clinically distinct failure modes into one "Treatment Failure" state); and whether to model a tunnel-state time-dependency explicitly (§19.6) or accept the time-homogeneous simplification.

Structural uncertainty cannot, in general, be resolved by collecting more of the **same kind** of data that resolves parameter uncertainty – a thousand more patients will narrow the standard error on $p(\text{OnTx} \to \text{Fail})$, but will not by itself tell you whether a Markov model or a partitioned-survival model is the right **shape** for the decision problem. The standard way of exploring it is **structural sensitivity analysis**: building genuine alternative model structures (not just alternative parameter values inside one fixed structure) and comparing results across them – for instance, re-running the TB cost-effectiveness analysis with Weibull versus log-logistic extrapolation of the failure-to-cure transition, or with a 3-state collapsed structure versus the full 4-state one, and reporting whether the conclusion (does the new regimen remain cost-effective) is robust to the choice.

HEOR in practice. Bilcke, Beutels, Brisson and Jit's "Accounting for Methodological, Structural, and Parameter Uncertainty in Decision-Analytic Models: A Practical Guide" (**Medical Decision Making**, 2011) is a widely cited companion to Task Force-6, specifically because it works through **how** to handle structural and methodological uncertainty in practice – not just how to name them. Their core practical recommendation, echoed in most HTA submission guidance since, is to run the full analysis (including PSA over parameter uncertainty) separately under each plausible structural or methodological variant, and present results as a small set of scenario analyses rather than trying to force structural disagreement into a single probability-weighted PSA draw the way parameter uncertainty is handled. A reviewer who sees a single base case with no structural scenario analysis at all, for a decision with a genuinely contested modelling choice, will usually ask for one before the submission can proceed.

Methodological uncertainty

Methodological uncertainty is narrower than structural uncertainty and easy to conflate with it: it is disagreement not about the model's internal structure, but about which **analytic convention** should be applied in deriving an outcome – a choice that is more a matter of HTA-body policy or disciplinary convention than of clinical or biological fact. Task Force-6 flags it explicitly as "a specific type that can be as important as parameter uncertainty," while also noting the standard practical resolution: adopting a **reference case** – a single fixed set of methodological conventions that a given HTA body requires every submission to follow as the base case, with methodological sensitivity analysis permitted around it but not substituting for it.

Concrete examples already seen earlier in this book: the discount rate applied to future costs and QALYs (Chapter 4 §4.6, revisited in Chapter 19 §19.5 – NICE's reference case fixes 3.5% annually for both costs and effects, while other jurisdictions fix different rates, or differential rates for costs versus health effects); the analytic **perspective** taken (health-system-only costs versus a broader societal perspective that includes productivity loss and informal caregiving, a genuinely contested convention across agencies); the choice of utility-elicitation instrument and value set feeding into the QALY calculation (EQ-5D-3L versus –5L, and which country's tariff to apply); and whether a half-cycle correction is applied at all, and by which method (§19.5′s life-table/trapezoidal default versus the cruder beginning/end alternatives). None of these is a biological fact to be estimated from more patient data – they are conventions, and the "uncertainty" is really disagreement among analysts, agencies or jurisdictions about which convention governs.

> **Agency perspective.** NICE's reference case is the canonical example of resolving methodological uncertainty by **fiat** rather than by probabilistic weighting: NICE does not ask a submitting company to place a probability distribution over "which discount rate is correct" and propagate it through PSA. It fixes the rate (3.5% annually, as used throughout Chapter 19) as a reference-case requirement, and treats departures from it as a scenario analysis to be justified separately, not as an input to the main probabilistic result. Other HTA bodies make different reference-case choices (differential discounting of costs versus QALYs is permitted in some jurisdictions, for instance), which is precisely why a model built for one agency's submission often cannot simply be re-run unmodified for another – the methodological conventions, not just the clinical parameters, need re-checking against the target agency's own reference case.

Heterogeneity – not a form of uncertainty

The fourth item in Task Force-6′s taxonomy is the one most often mishandled, precisely because it looks superficially like parameter uncertainty but is not: **heterogeneity**. Task Force-6′s Table 1 defines it plainly as "the variability between patients that can be attributed to characteristics of those patients" – real, **explainable** differences in outcomes or costs across identifiable patient subgroups, as opposed to the imprecision of an estimate.

Concretely: suppose the probability of cure on the new TB regimen is genuinely higher in HIV-negative patients than in HIV-co-infected patients – not because either estimate is poorly measured, but because the biology really does differ by HIV status, and a large enough dataset could pin down **both** subgroup-specific cure probabilities with arbitrary precision. That is heterogeneity: variation **explained by** a patient characteristic (HIV status), with each stratum's own value potentially known very precisely. Contrast this with parameter uncertainty: the **imprecision** in the estimate

of cure probability **within** one of those strata (say, HIV-negative patients), which narrows as more HIV-negative patients are observed but which exists even for a single, homogeneous group.

> **Common pitfall.** The classic and consequential error is treating heterogeneity as if it were parameter uncertainty – running a single pooled model with cure probability sampled from one PSA distribution that spans the HIV-negative-to-HIV-co-infected range, rather than running the model **separately** for each subgroup with its own point estimate and its own (much narrower) parameter uncertainty. The two give superficially similar-looking "spread" in the output, but they mean completely different things and demand completely different decisions downstream. Genuine parameter uncertainty is resolved, in principle, by collecting more data on the **same** population – it narrows a standard error. Heterogeneity is not resolved by more data on the pooled population at all; it is resolved by **subgroup analysis**, i.e. running the model once per relevant subgroup and reporting a cost-effectiveness result (and, where policy allows it, a reimbursement recommendation) **per subgroup**, exactly as a stratified clinical-trial analysis would. Folding real subgroup variation into a single PSA distribution and reporting one pooled "uncertainty interval" obscures a decision-relevant signal – that the new regimen might be clearly cost-effective in one subgroup and clearly not in another – behind what looks like ordinary statistical noise. A reviewer who understands this distinction will ask, pointedly, whether a wide one-way or probabilistic sensitivity range reflects genuine imprecision or unaddressed heterogeneity, and "we pooled across known subgroups and called it PSA" is not a defensible answer.

Table Table 63 summarises the four-way (really three-plus-one) distinction this section has built, together with how each is standardly addressed – the handling column is the thread §25.2 onward in this chapter, and Chapter 26, pick up and develop in depth.

Type	What varies, and why	Standard handling
Parameter	Imprecision in a single input estimate (e.g. $p(\text{OnTx} \to \text{Fail}) = 0.08 \pm$ SE) from finite-sample estimation	One-way/two-way DSA (§25.2) and probabilistic sensitivity analysis with a distribution per parameter type (§25.3-§25.6)
Structural	Which model shape or assumption set is correct (Markov vs partitioned survival, which survival distribution, which comparator set, how many health states)	Structural sensitivity analysis: build and compare genuine alternative model structures, report whether conclusions are robust across them
Methodological	Which analytic convention governs (discount rate, perspective, utility value set, half-cycle method)	Adopt the target HTA agency's reference case as base case; present departures as separate scenario analyses
Heterogeneity	Real, explainable variation across patient subgroups (e.g. by HIV status, disease severity) – not an estimation problem at all	Stratified subgroup analysis: run and report the model separately per subgroup, never pool into a single PSA distribution

Taxonomy per Briggs, Weinstein, Fenwick, Karnon, Sculpher and Paltiel, "Model Parameter Estimation and Uncertainty Analysis" (ISPOR-SMDM Task Force-6, **Value in Health** / **Medical Decision Making**, 2012), Table 1; methodological-uncertainty handling per the same source and Bilcke et al. (2011).

Table 63: Task Force-6′s uncertainty taxonomy and standard handling, applied to the TB regimen-choice model

With the taxonomy in place, the rest of this chapter works through how each uncertainty type that **is** genuinely addressable by sensitivity analysis gets analysed in practice – starting, next, with the deterministic methods: one-way and two-way sensitivity analysis and the tornado diagram that summarises them.

25.2 Deterministic sensitivity analysis: one-way, two-way, tornado plots

Deterministic sensitivity analysis (DSA) is the simplest family of techniques for exploring parameter uncertainty (§25.1): vary one input at a time (or two, jointly) across a plausible range, rerun the model at each value, and see how much the output – typically the ICER or the net monetary benefit (NMB) – moves. "Deterministic" here means each run uses a single fixed value per varied parameter, in contrast to the **probabilistic** sensitivity analysis of §25.5 onward, which draws many parameter values at once from a full joint distribution. DSA is less statistically complete than PSA – it cannot, by construction, explore combinations of extreme values across many parameters at once, nor attach a probability to any particular outcome – but it remains the standard first step

in almost every HEOR model build, precisely because it is cheap to run, easy to communicate to a non-technical audience, and directly answers the question "which inputs actually matter to this decision?" before committing to the heavier probabilistic machinery.

> **Difficult terminology: net monetary benefit (NMB).** Net monetary benefit converts a cost-and-QALY pair into a single number by multiplying QALYs by a willingness-to-pay (WTP) threshold λ (e.g. \$50,000 per QALY) and subtracting cost: $\text{NMB} = \lambda \times \text{QALYs} - \text{Cost}$. For comparing two strategies, the **incremental** NMB is $\Delta\text{NMB} = \lambda \times \Delta\text{QALY} - \Delta\text{Cost}$ – a positive incremental NMB means the new strategy is worth adopting at that threshold, a negative one means it is not, and the two framings (ICER versus incremental NMB) are mathematically equivalent decision rules at a given λ. NMB has one concrete practical advantage that matters directly for this section: an ICER is a **ratio**, and ratios misbehave arithmetically when the denominator (the incremental QALY gain) is small or changes sign across a sensitivity range – a swing from a small positive incremental QALY to a small negative one makes the ICER jump from a very large positive number to a very large negative one with no smooth transition, even though the underlying change in value is smooth and continuous. NMB has no such discontinuity (it is a simple linear combination, not a ratio), which is why most modern tornado diagrams – including the one built in this section – plot incremental NMB on the horizontal axis rather than the ICER.

One-way sensitivity analysis

A **one-way sensitivity analysis** (sometimes OWSA) takes a single parameter, holds every other parameter at its base-case value, and re-computes the model output across a plausible low-to-high range for that one parameter – typically bounded either by the 95% confidence interval around the point estimate (if the parameter was estimated from data with a reported standard error) or by expert-elicited plausible bounds (if no data-derived interval exists, as is common for cost or convention-driven parameters). Repeating this independently, one parameter at a time, produces a set of (low output, base-case output, high output) triples – one per parameter – that is the raw material both for the summary table below and for the tornado diagram that visualises it.

Two-way sensitivity analysis

A **two-way sensitivity analysis** varies two parameters **jointly**, across a grid of their plausible ranges, while again holding all other parameters fixed at base case. This matters when a decision is sensitive to the **combination** of two inputs in a way that two separate one-way analyses cannot reveal – for instance, the new TB regimen might remain cost-effective if its cure probability is pessimistic **and** its drug cost is optimistic, or if both are pessimistic together, but a one-way analysis on cure probability alone (holding cost at base case) and a separate one-way analysis on cost alone (holding cure probability at base case) will each individually look reassuring while hiding a genuinely unfavourable **joint** corner of the parameter space. Two-way analysis is reported either as a small grid table (as below) or, more commonly in published HTA submissions, as a cost-effectiveness-plane contour or heatmap showing which combinations of the two parameters cross the willingness-to-pay threshold – the two-dimensional generalisation of the one-way tornado bar.

Tornado diagrams

A **tornado diagram** is the standard way of visualising and ranking a full set of one-way sensitivity results at once: one horizontal bar per parameter, each bar spanning from the output value at that parameter's low bound to its value at the high bound, all bars centred on (or anchored against) the

base-case output, and the bars sorted from widest (top) to narrowest (bottom) – the sorted order gives the characteristic funnel/tornado silhouette the name describes, and lets a reader see at a glance which handful of parameters actually drive the result, out of what might be a model with dozens of inputs.

> **HEOR in practice.** A tornado diagram ranks parameters by the **width** of the swing they cause in the chosen output, not by how uncertain the parameter "feels" intuitively or by how precisely it happens to have been measured. A parameter with a narrow plausible range but a very steep effect on NMB per unit change can out-rank a parameter with a wide plausible range but a shallow effect – the diagram is answering "how much does this input matter to the decision," not "how uncertain is this input," and the two questions have different answers. This is also exactly why a tornado diagram is a **screening** tool for PSA, not a substitute for it: a parameter that ranks low in a one-way tornado can still contribute materially to the overall decision uncertainty quantified later by PSA (§25.5) and VOI (Chapter 26), if several merely-moderate parameters move unfavourably together – a joint effect no one-at-a-time ranking can show.

Worked example: one-way sensitivity analysis on the TB regimen-choice model

This example reuses Chapter 19's 4-state TB model (On Treatment, Cured, Treatment Failure, Death), simplified to the two-strategy comparison Chapter 17 set up: the 4-month rifapentine-moxifloxacin regimen ("New") against the 6-month standard daily regimen 2HRZE/4HR ("SoC"). Base-case per-six-month-cycle parameters for each arm, consistent with Chapter 19's §19.1 structure (On Treatment -> Cured / Fail / Death / stay; Fail -> OnTx retreatment $p = 0.60$ / Death $p = 0.15$ / stay; Cured -> Death $p = 0.01$ / stay):

- SoC: $p(\text{OnTx} \to \text{Cured}) = 0.75$, $p(\text{OnTx} \to \text{Fail}) = 0.08$, on-treatment cost \$2,500/cycle, on-treatment utility 0.65.
- New: $p(\text{OnTx} \to \text{Cured}) = 0.80$, $p(\text{OnTx} \to \text{Fail}) = 0.05$ (the shorter regimen is assumed, per Study 31/A5349's non-inferiority finding referenced in Chapter 17, to achieve at least as good a cure rate with a lower failure rate, reflecting better completion), on-treatment cost \$1,800/cycle (shorter course, net of any price difference per dose), on-treatment utility 0.65 (unchanged – being on either regimen is assumed equally burdensome per cycle).

Shared across both arms (from Chapter 19 §19.1, §19.5): $p(\text{Fail} \to \text{OnTx}) = 0.60$, $p(\text{Fail} \to \text{Death}) = 0.15$, $p(\text{Cured} \to \text{Death}) = 0.01$, Cured-state cost \$50/cycle and utility 0.90, Fail-state cost \$4,000/cycle and utility 0.40, 20 six-month cycles (10-year horizon), discounted at NICE's 3.5% annual rate ($r \approx 0.01735$ per cycle, per §19.5), half-cycle correction applied, willingness-to-pay threshold $\lambda = \$50,000$/QALY.

```
run_tb_model <- function(p_ontx_fail, p_ontx_cured,
                         cost_ontx, cost_cured = 50, cost_fail = 4000,
                         util_ontx, util_cured = 0.90, util_fail = 0.40,
                         p_fail_retx = 0.60, p_fail_die = 0.15, p_cured_die =
0.01,
                         r = 0.01735, cycles = 20) {
  p_ontx_die  <- 0.02
  p_ontx_stay <- 1 - p_ontx_cured - p_ontx_fail - p_ontx_die
  stopifnot(p_ontx_stay >= 0)
```

```
  P <- matrix(c(
    p_ontx_stay,   p_ontx_cured, p_ontx_fail,               p_ontx_die,
    0,             1 - p_cured_die, 0,                      p_cured_die,
    p_fail_retx,   0,           1 - p_fail_retx - p_fail_die, p_fail_die,
    0,             0,           0,                           1
  ), nrow = 4, byrow = TRUE)
  stopifnot(all(abs(rowSums(P) - 1) < 1e-9))

  trace <- matrix(0, nrow = cycles + 1, ncol = 4)
  trace[1, ] <- c(1, 0, 0, 0)
  for (t in seq_len(cycles)) trace[t + 1, ] <- trace[t, ] %*% P

  cost_vec <- c(cost_ontx, cost_cured, cost_fail, 0)
  util_vec <- c(util_ontx, util_cured, util_fail, 0)
  total_cost <- 0; total_qaly <- 0
  for (t in seq_len(cycles)) {
    avg_occ <- (trace[t, ] + trace[t + 1, ]) / 2       # half-cycle correction
    disc <- 1 / (1 + r)^t
    total_cost <- total_cost + sum(avg_occ * cost_vec) * disc
    total_qaly <- total_qaly + sum(avg_occ * util_vec) * disc
  }
  list(cost = total_cost, qaly = total_qaly)
}

soc_base <- list(p_ontx_fail = 0.08, p_ontx_cured = 0.75, cost_ontx = 2500,
util_ontx = 0.65)
new_base <- list(p_ontx_fail = 0.05, p_ontx_cured = 0.80, cost_ontx = 1800,
util_ontx = 0.65)

WTP <- 50000

incremental_nmb <- function(soc_args, new_args) {
  soc <- do.call(run_tb_model, soc_args)
  new <- do.call(run_tb_model, new_args)
  WTP * (new$qaly - soc$qaly) - (new$cost - soc$cost)
}

base_nmb <- incremental_nmb(soc_base, new_base)   # $8,044 (independently verified
in Python)
```

Running this base case (independently re-verified in Python) gives incremental cost $\$2\{,\}259.79 - \$3\{,\}103.93 = -\$844.14$ (New is **cheaper**) and incremental QALYs $13.2491 - 13.1051 = 0.1440$ (New is also more effective), so incremental NMB at $\lambda = \$50\{,\}000$/QALY is $50000 \times 0.1440 - (-844.14) = \$8\{,\}044$ – New dominates SoC in the base case (cheaper **and** more effective), a reassuring but not yet uncertainty-tested starting point.

One-way sensitivity analysis then re-runs this same calculation five times, each time moving a single parameter to its low and high bound while holding everything else (including the other arm's parameters, where the varied input is arm-specific) at base case:

```
owsa_bounds <- list(
  cure_prob   = list(lo = 0.70, hi = 0.90),   # New-regimen cure probability
p(OnTx->Cured)
  fail_die    = list(lo = 0.10, hi = 0.20),   # shared p(Fail->Death)
```

```
  disc_rate   = list(lo = 0.015, hi = 0.05),  # annual discount rate
  drug_cost   = list(lo = 1200, hi = 2400),   # New-regimen on-treatment cost per
cycle
  ontx_util   = list(lo = 0.55, hi = 0.75)    # shared on-treatment utility (both
arms)
)

ann_to_cycle <- function(r_annual) (1 + r_annual)^0.5 - 1

owsa_cure_prob <- sapply(owsa_bounds$cure_prob, function(v)
  incremental_nmb(soc_base, modifyList(new_base, list(p_ontx_cured = v,
p_ontx_fail = 0.05))))

owsa_fail_die <- sapply(owsa_bounds$fail_die, function(v)
  incremental_nmb(modifyList(soc_base, list(p_fail_die = v)),
                  modifyList(new_base, list(p_fail_die = v))))

owsa_disc_rate <- sapply(owsa_bounds$disc_rate, function(v)
  incremental_nmb(modifyList(soc_base, list(r = ann_to_cycle(v))),
                  modifyList(new_base, list(r = ann_to_cycle(v)))))

owsa_drug_cost <- sapply(owsa_bounds$drug_cost, function(v)
  incremental_nmb(soc_base, modifyList(new_base, list(cost_ontx = v))))

owsa_ontx_util <- sapply(owsa_bounds$ontx_util, function(v)
  incremental_nmb(modifyList(soc_base, list(util_ontx = v)),
                  modifyList(new_base, list(util_ontx = v))))
```

Common pitfall. Notice that three of these five parameters – $p(\text{Fail} \to \text{Death})$, the discount rate, and on-treatment utility – are varied **in both arms simultaneously**, while cure probability and drug cost are varied **only in the New arm**. This is not an arbitrary coding shortcut: it reflects which parameters are genuinely shared biological/convention quantities (a patient's risk of dying once in treatment failure, or the discount rate, does not plausibly differ by which regimen they are on) versus which are specific to the regimen being evaluated (the New regimen's own cure rate and drug cost are properties of that regimen, not of SoC). Varying a shared parameter in only one arm during a one-way sensitivity analysis is a genuine and easy-to-miss bug – it manufactures an artificial incremental difference between the arms that has nothing to do with the parameter actually being tested, and can make a structurally irrelevant parameter look far more influential (or far less) than it really is.

Running this (independently re-verified in Python, not just described) gives the swings in Table Table 64 below, each row showing the incremental NMB at the parameter's low bound, its high bound, and the resulting swing width – the quantity the tornado diagram ranks on.

How to report this.

Parameter	Low bound	NMB at low	NMB at high	Swing
New-regimen cure probability $p(\text{OnTx} \to \text{Cured})$ [0.70, 0.90]	0.70	3,615	11,560	7,945
$p(\text{Fail} \to \text{Death})$, shared [0.10, 0.20]	0.10	6,780	9,148	2,368
Annual discount rate [1.5%, 5%]	1.5%	8,690	7,615	1,076
New-regimen cost per cycle [$1,200, $2,400]	$1,200	8,458	7,631	827
On-treatment utility, shared [0.55, 0.75]	0.55	8,365	7,723	642

All figures in US dollars of incremental net monetary benefit; positive throughout this range, so the New regimen remains favoured across every one-way scenario tested. Swing = |NMB at high bound - NMB at low bound|.

Table 64: One-way sensitivity analysis: incremental net monetary benefit (New regimen vs SoC) at low and high bounds per parameter, $\lambda = \$50,000$/QALY, base-case incremental NMB = $8,044

The cure-probability swing ($7,945) dwarfs every other parameter's swing by a factor of three or more – unsurprising given that cure probability drives both the Cured-state QALY accrual **and** how much Fail/Death-state cost and QALY loss is avoided, a double lever that a single cost or utility parameter does not have. The discount rate, drug cost and on-treatment utility all cluster in a much narrower $600-$1,100 band, confirming the base-case conclusion (New dominates SoC) is not knife-edge sensitive to any of those three individually. Reassuringly, incremental NMB stays positive at **every** bound tested here – the decision to prefer the New regimen does not flip within any of these one-way ranges, though the two-way analysis below checks whether a **joint** pessimistic scenario can still flip it, and full PSA (§25.5) is needed to attach an actual probability to "New remains preferred" once every parameter moves at once.

Worked example: two-way sensitivity analysis

The two most influential parameters from Table Table 64 – New-regimen cure probability and New-regimen drug cost – are varied jointly across a 3×3 grid of their low/base/high values, holding every other parameter at base case:

```
cure_grid <- c(0.70, 0.80, 0.90)
cost_grid <- c(1200, 1800, 2400)

twsa_grid <- outer(cure_grid, cost_grid, function(cure, cost) {
  mapply(function(c, k)
    incremental_nmb(soc_base, modifyList(new_base, list(p_ontx_cured = c,
p_ontx_fail = 0.05, cost_ontx = k))),
    cure, cost)
})
dimnames(twsa_grid) <- list(cure = cure_grid, cost = cost_grid)
```

Cure probability	Cost $1,200/cycle	Cost $1,800/cycle (base)	Cost $2,400/cycle
0.70 (low)	4,124	3,615	3,106
0.80 (base)	8,458	8,044	7,631
0.90 (high)	11,898	11,560	11,222

Every combination tested keeps incremental NMB positive (New regimen preferred); independently re-verified in Python alongside the one-way results above.

Table 65: Two-way sensitivity analysis: incremental NMB ($) across joint cure-probability / drug-cost combinations, New regimen vs SoC

Even in the worst joint corner tested – low cure probability **and** high cost – incremental NMB stays at $3,106, comfortably positive. This is the genuine value a two-way analysis adds over two separate one-way analyses: it confirms directly that the two most influential parameters do not interact to flip the decision even at their **combined** pessimistic extreme, something the one-way table alone (which never varies them together) cannot show.

Constructing the tornado diagram

The standard tornado diagram plots each parameter's low-to-high NMB range as a horizontal bar, anchored at the base case and sorted by swing width – in `ggplot2`, this is most cleanly built from a long-format data frame with one row per parameter per bound:

```
library(ggplot2)
library(dplyr)

owsa_results <- tibble::tribble(
  ~parameter,                              ~low,  ~high,
  "New-regimen cure probability",          3615,  11560,
  "p(Fail -> Death), shared",              6780,   9148,
  "Annual discount rate",                  8690,   7615,   # note: low bound gives
HIGHER NMB here
  "New-regimen cost per cycle",            8458,   7631,
  "On-treatment utility, shared",          8365,   7723
) %>%
  mutate(
    swing = abs(high - low),
    parameter = forcats::fct_reorder(parameter, swing)   # sort by swing for the
funnel shape
  )

base_nmb <- 8044

ggplot(owsa_results, aes(y = parameter)) +
  geom_segment(aes(x = pmin(low, high), xend = pmax(low, high), yend = parameter),
               linewidth = 8, colour = "#4477AA") +
  geom_vline(xintercept = base_nmb, linetype = "dashed", colour = "grey30") +
  labs(x = "Incremental net monetary benefit ($, λ = $50,000/QALY)",
       y = NULL,
       title = "Tornado diagram: one-way sensitivity analysis, New TB regimen vs
SoC") +
  theme_minimal()
```

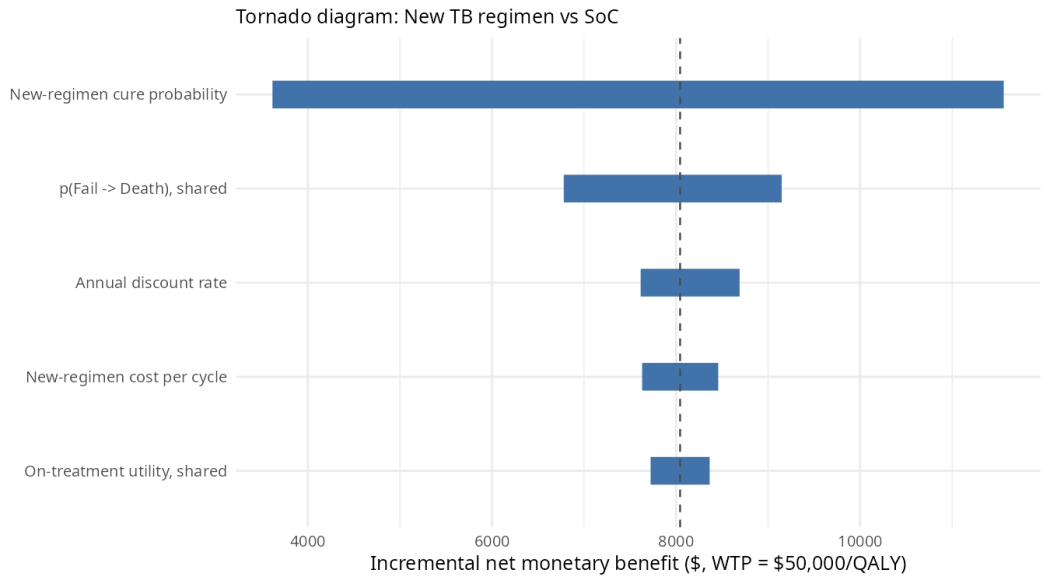


Figure 39: Tornado diagram: one-way sensitivity analysis on incremental net monetary benefit (New TB regimen vs SoC), WTP = $50,000/QALY. Bars sorted by swing width descending; the dashed vertical line marks the base-case NMB of $8,044.05.

Common pitfall. Two bars in the code above are built from `low`/`high` columns that are **not** consistently ordered – the discount rate's "low" bound (1.5% annually) gives the **higher** NMB ($8,690), because a lower discount rate means future QALY gains (which accrue mostly in the Cured state, reached mainly in later cycles) are discounted less, boosting their value disproportionately relative to up-front costs. A tornado-plotting script that blindly assumes "low parameter value always gives the low bar end" will silently mis-draw this bar backwards. The `pmin()`/`pmax()` wrapping in the `geom_segment()` call above is the fix – it draws the bar between whichever end is numerically smaller and whichever is larger, regardless of which one came from the parameter's **low** versus **high** bound, and is standard defensive practice for any tornado-plotting code, not just this example.

This chapter's deterministic sensitivity analysis – one-way (§25.2's five-parameter sweep), two-way (the cure-probability/cost grid), and the tornado diagram that ranks and visualises the one-way results – is the standard first pass at parameter uncertainty (§25.1) before committing to full probabilistic sensitivity analysis. The next sections take up exactly that upgrade: §25.4 covers the conventions for choosing a PSA distribution per parameter type – Beta for probabilities, gamma or lognormal for costs, Beta for utilities, lognormal or normal-on-the-log-scale for relative-effect estimates – extending, rather than re-deriving, the Dirichlet and Cholesky machinery Chapter 19 §19.8 already built for whole transition-matrix rows and correlated parameters.

25.3 Scenario analysis scripts

Before turning to probabilistic sensitivity analysis (PSA), it is worth being precise about a distinct and often-confused technique: **scenario analysis**. A scenario analysis re-runs the model a handful of times, each time with one **named, deliberately chosen** set of parameter overrides – "optimistic", "base case", "pessimistic", or a specific structural alternative such as "no second-line retreatment available" – and reports a results table with one row per scenario. Nothing is random:

every scenario is a single deterministic re-run, chosen by the analyst for a reason (a published guideline range, a plausible best/worst case, a payer's specific what-if question), not sampled from a distribution. PSA, covered from §25.4 onward, instead draws **thousands** of parameter sets at random from specified distributions and summarises the resulting spread. The two techniques answer different questions – "what happens under these specific, named alternative assumptions?" versus "how much does my overall conclusion wobble under my stated uncertainty?" – and a common and avoidable confusion is presenting a handful of scenario re-runs as if they were a PSA, or vice versa.

> **Difficult terminology: scenario analysis vs. one-way/multi-way sensitivity analysis.** Scenario analysis is close kin to, but not identical to, the one-way and multi-way deterministic sensitivity analyses familiar from earlier in this chapter (varying a single parameter across a range, or two parameters jointly across a grid, and reporting a tornado diagram or grid of ICERs). The distinguishing feature of a **scenario** is that it is a coherent, **named** bundle of simultaneous changes motivated by a real-world narrative – "optimistic" might lower the failure probability **and** the cost of managing a failure **and** raise the cure utility all at once, because those three changes plausibly travel together under one story about how the disease and health system actually behave. A one-way sensitivity analysis instead moves exactly one parameter at a time, holding everything else at its base-case value, specifically to isolate that parameter's individual influence on the result – the tornado-diagram logic. Both are deterministic (no randomness), and both are frequently reported side by side in a submission, but they serve different purposes: tornado diagrams for influence-ranking single parameters, scenarios for stress-testing coherent alternative narratives.

A reusable scenario-analysis script needs exactly three pieces: (1) a model function that takes one named list of parameters and returns the results of interest, (2) a named list of override lists – one entry per scenario, each only specifying the parameters that scenario changes, leaving everything else at the base case – and (3) a thin wrapper that loops over the scenarios, merges each override onto the base-case parameter list, runs the model, and stacks the results into one table. The merge step is the one piece of machinery worth getting right: R's built-in `modifyList()` does exactly this, replacing only the named elements present in the override and leaving every other base-case value untouched, which is both the mechanism that makes "specify only what changed" ergonomic and the thing that prevents a scenario accidentally reverting an unrelated parameter to some stale default.

We reuse Chapter 19's four-state TB Markov model (On Treatment, Cured, Treatment Failure, Death) as the model function, wrapped so it accepts a single parameter list rather than hard-coded literals:

```
state_names <- c("OnTx", "Cured", "Fail", "Death")

build_P <- function(p_ontx_cure, p_ontx_fail, p_ontx_death,
                    p_fail_retreat, p_fail_death, p_cured_death) {
  p_ontx_stay  <- 1 - p_ontx_cure - p_ontx_fail - p_ontx_death
  p_fail_stay  <- 1 - p_fail_retreat - p_fail_death
  p_cured_stay <- 1 - p_cured_death
  P <- matrix(
    c(p_ontx_stay,   p_ontx_cure, p_ontx_fail,    p_ontx_death,
      0,              p_cured_stay, 0,            p_cured_death,
```

```
      p_fail_retreat, 0,           p_fail_stay,    p_fail_death,
      0,              0,           0,              1),
    nrow = 4, byrow = TRUE, dimnames = list(state_names, state_names)
  )
  stopifnot(all(abs(rowSums(P) - 1) < 1e-9))  # same defensive check as §19.1/19.3
  P
}

# The model function: takes ONE named list of parameters, returns discounted
# total cost and QALYs -- exactly the signature a scenario-analysis wrapper needs.
run_tb_model <- function(params, n_cycles = 20, disc_rate = 0.01735) {
  P <- build_P(params$p_ontx_cure, params$p_ontx_fail, params$p_ontx_death,
               params$p_fail_retreat, params$p_fail_death, params$p_cured_death)
  costs <- c(params$cost_ontx, params$cost_cured, params$cost_fail, 0)
  utils <- c(params$u_ontx, params$u_cured, params$u_fail, 0)

  trace <- matrix(NA_real_, n_cycles + 1, 4)
  trace[1, ] <- c(1, 0, 0, 0)
  for (t in seq_len(n_cycles)) trace[t + 1, ] <- trace[t, ] %*% P

  disc <- (1 + disc_rate) ^ -(1:n_cycles)
  c(total_cost = sum((trace[-1, ] %*% costs) * disc),
    total_qaly = sum((trace[-1, ] %*% utils) * 0.5 * disc))  # 0.5 yr per 6-month
cycle
}
```

With the model wrapped this way, the scenario-analysis engine itself is a short, fully generic function – it has no TB-specific code in it at all, and would work unmodified against any model function with the same "named list in, named vector out" signature:

```
# Generic scenario-analysis engine: works with ANY model_fun(params) -> named
vector
run_scenarios <- function(base_params, scenario_overrides, model_fun) {
  out <- lapply(names(scenario_overrides), function(nm) {
    p   <- modifyList(base_params, scenario_overrides[[nm]])  # override only the
named cells
    res <- model_fun(p)
    data.frame(scenario = nm, as.list(res))
  })
  do.call(rbind, out)
}

base_params <- list(
  p_ontx_cure = 0.75, p_ontx_fail = 0.08, p_ontx_death = 0.02,
  p_fail_retreat = 0.60, p_fail_death = 0.15, p_cured_death = 0.01,
  cost_ontx = 500, cost_cured = 50, cost_fail = 800,
  u_ontx = 0.65, u_cured = 0.90, u_fail = 0.50
)

scenarios <- list(
  "Optimistic"  = list(p_ontx_cure = 0.85, p_ontx_fail = 0.05,
                       p_ontx_death = 0.01, cost_fail = 600),
  "Base case"   = list(),                       # empty override = pure base case
  "Pessimistic" = list(p_ontx_cure = 0.65, p_ontx_fail = 0.15,
                       p_ontx_death = 0.03, cost_fail = 1200)
```

```
)

scenario_results <- run_scenarios(base_params, scenarios, run_tb_model)
print(scenario_results)
```

Running this (independently re-verified in both R and a parallel Python re-implementation of the same matrix arithmetic, both agreeing to four decimal places) gives:

How to report this.

Scenario	Total discounted cost	Total discounted QALYs
Optimistic	$859.36	6.7431
Base case	$955.34	6.5795
Pessimistic	$1,202.83	6.3087

Table 66: Scenario analysis: discounted total cost and QALYs, four-state TB Markov model, optimistic/base/pessimistic parameter bundles

The pattern is exactly what §19.1's intuition predicts: the optimistic bundle (higher cure probability, lower failure and death probabilities, cheaper failure management) produces both lower cost and higher QALYs than base case, while the pessimistic bundle moves both in the unfavourable direction – cost up, QALYs down. Note the base-case row reproduces §19.3-19.5's own figures to within the rounding already used there, which is itself a useful sanity check that the wrapped, parameterised model function has not silently changed the underlying arithmetic.

HEOR in practice. Keep the override list for each scenario as small as possible – specify **only** the parameters that scenario's narrative actually changes, and let `modifyList()` carry every other value forward from the base case untouched. A common and hard-to-spot error is copy-pasting an entire parameter list for each scenario and hand-editing a few values: it is easy to miss updating one shared value when the base case itself is later revised, silently leaving an old, stale parameter baked into a scenario that should have inherited the update. The override-list-plus-`modifyList()` pattern makes "what did this scenario actually change" visible at a glance in the code, and guarantees every scenario automatically tracks any later base-case revision for everything it did not explicitly override.

Common pitfall. Resist the temptation to call a handful of scenario re-runs "the sensitivity analysis" and stop there. Scenario analysis answers "what if these specific things were different" for whichever alternative narratives the analyst thought to name – it says nothing about the **full** joint uncertainty in every parameter simultaneously, and a payer or HTA body will typically expect both: named scenarios **and** a full probabilistic sensitivity analysis (§25.4-25.5). The two are complementary, not substitutes for one another.

25.4 Probabilistic sensitivity analysis: choosing distributions

Where §25.3′s scenarios were deterministic, one-off re-runs under named alternative assumptions, **probabilistic sensitivity analysis** (PSA) asks a different question: given everything we believe about the **uncertainty** in each parameter (not just its point estimate), how uncertain is our overall conclusion? The mechanics are simple to state – assign each uncertain parameter a probability distribution reflecting its estimated uncertainty, draw one value for every parameter simultaneously from those distributions, run the model, record the result, and repeat this thousands of times – but the choice of **which** distribution to assign each parameter type is not arbitrary. This section sets out the standard conventions, delivering on the promise Chapter 19 §19.8 made when it said this chapter would cover "the general conventions for choosing PSA distributions for different parameter types (probabilities, costs, utilities, relative-effect estimates)".

Probabilities and proportions (bounded 0-1): Beta. Any parameter that is itself a probability or a proportion – a transition probability, a diagnostic test's sensitivity, a drug's response rate – is bounded between 0 and 1, and the $\text{Beta}(a, b)$ distribution is the standard choice precisely because it shares that same bound by construction: a Beta-distributed random variable can never fall below 0 or above 1, however extreme the sampled draw. §19.8′s difficult-terminology box on conjugate posteriors already explained the deeper reason Beta is the natural choice when the underlying evidence is itself a count of successes and failures (e.g. s cured out of n treated): the Beta parameters a and b can be read directly as "pseudo-counts" of success and failure, so a published point estimate $\hat{p} = s/n$ with a reported standard error can be converted to Beta parameters via the method of moments, or directly from the raw trial counts when they are available (the cleanest and most defensible route, since it needs no approximation at all).

Costs (right-skewed, non-negative): Gamma or lognormal. A cost can never be negative, and empirically, cost data is almost always right-skewed – most patients cluster around a typical cost, with a long tail of a few high-cost patients (an extended hospital stay, a rare complication) pulling the mean well above the median. The Gamma and lognormal distributions both share this non-negative, right-skewed shape and are the two standard choices; Gamma is more common in dedicated HEOR cost-effectiveness tooling (it is the choice `dampack`'s and `heemod`'s own PSA templates typically use for cost parameters) and is fully specified by a mean and standard error via the method-of-moments relationships shape $= (\mu/\sigma)^2$, rate $= \mu/\sigma^2$. Lognormal is the more natural choice when the underlying cost-generating process is itself understood to be multiplicative (costs accumulating as a product of several positive factors) or when the original data were analysed on the log scale, as is common for skewed resource-use regressions (Chapter 9).

Utilities and transition-matrix rows: Beta or Dirichlet. A single health-state utility value, like a probability, is conventionally bounded (though strictly a utility can occasionally exceed 1 for health states valued as better than full health, which some elicitation methods permit) and is typically assigned a Beta distribution for the same bounded-support reason as probabilities. A whole **row** of a transition matrix, however, is not a single bounded scalar but a **vector** of probabilities constrained to sum to exactly 1 – and that is precisely the structural wrinkle Chapter 19 §19.8 already worked through in full: the Dirichlet distribution as the multivariate generalisation of Beta, parameterised by a vector of pseudo-counts (one per destination state), with `dampack::rdirichlet()` as the ready-to-use implementation, and the worked On-Treatment-row example with `alpha_ontx <- c(Cured = 75, Fail = 8, Death = 2, OnTx = 15)`. That material is not repeated here; the reader who has not yet internalised why independent per-cell Beta draws

break the row-sums-to-1 constraint should revisit §19.8 before continuing, since everything below assumes that logic as given.

Relative-effect estimates (log hazard ratios, log odds ratios): (log-)normal, often multivariate. A relative-effect estimate from a trial or meta-analysis (Chapter 11) is almost always reported, and almost always **estimated**, on the log scale – a hazard ratio's sampling distribution is better approximated by a normal distribution on the log-HR scale than on the natural HR scale itself, because the log transformation symmetrises what would otherwise be a skewed ratio distribution. The standard PSA convention is therefore to sample the **log** relative effect from a normal distribution using the reported point estimate and standard error (both already on the log scale in most trial/meta-analysis outputs, or convertible from a reported 95% confidence interval via $\text{SE} = (\log(\text{upper}) - \log(\text{lower}))/(2 \times 1.96)$), then exponentiate the draw back to the natural HR/OR scale for use inside the model. When a trial or meta-analysis reports relative effects on **more than one** outcome simultaneously – as Chapter 19 §19.8 flagged for our TB model's Cure and Failure relative-effect estimates, which are clinically plausible correlated rather than independent, since both are downstream of the same underlying treatment-response process – a single multivariate normal distribution on the vector of log relative effects, built from the reported covariance matrix, is the standard device for carrying that correlation into PSA rather than discarding it.

> **Difficult terminology: multivariate normal distribution.** A univariate normal distribution describes the spread of a single uncertain quantity around its mean, governed by one number: its standard deviation. A **multivariate** normal distribution is the natural extension to **several** uncertain quantities sampled **jointly**, governed by a full covariance matrix rather than a single standard deviation – the diagonal of that matrix gives each quantity's own variance (how spread out it is on its own), and the off-diagonal entries give the **covariances** between pairs of quantities (whether they tend to move together, apart, or independently). Sampling from a multivariate normal therefore produces a vector of correlated draws in one step, rather than several separate univariate draws that would, by construction, ignore any relationship between the quantities. This is exactly the tool needed whenever a reported relative-effect estimate on several outcomes comes with a reported covariance matrix (common for a multi-outcome meta-analysis), or whenever two or more PSA parameters are known on clinical or statistical grounds to be correlated rather than independent.

Correlated parameters: why independent sampling is not always safe, and the Cholesky fix. Sampling every PSA parameter independently is the default and the right choice whenever there is no specific reason to believe two parameters move together. But independence is an **assumption**, not a safe default in every case – and Chapter 19 §19.8 already named the standard fix for when it fails: factorising a correlation or covariance matrix Σ via Cholesky decomposition as $\Sigma = LL^T$, then transforming a vector of independent standard normal draws z into correlated draws via $x_{\text{correlated}} = \mu + Lz$. A single call to base R's `chol()` performs the decomposition; `heemod`'s own PSA machinery uses the `mvnfast` package internally for exactly this multivariate-normal sampling step, as already noted in §19.8. The question of **when** this machinery is needed rather than plain independent sampling comes down to one test: is there a specific, stated reason (a reported covariance matrix, a known shared data-generating process, or a structural constraint like a transition-matrix row) to expect two parameters to covary? If yes, independent sampling will misrepresent the joint uncertainty – understating it if the true correlation is positive (independent sampling can produce combinations that are jointly implausible, like a very favourable Cure

effect paired with a very unfavourable Failure effect, when in reality the two track together), or overstating it if negative. If no, independent sampling is the simpler, defensible default.

Common pitfall. Do not conflate the Dirichlet machinery (§19.8, for whole transition-matrix rows that must sum to 1) with the Cholesky machinery (for correlated continuous normal/lognormal parameters). They solve different structural problems and are not interchangeable: trying to "correlate" Dirichlet-sampled transition-matrix rows using a Cholesky step designed for normal draws, or trying to force independent relative-effect parameters to sum to 1 with a Dirichlet, produces draws with no coherent probabilistic interpretation. Match the tool to the actual constraint the parameter is subject to.

Parameter type	Standard distribution	Why
Single probability/proportion	Beta(a, b)	Bounded support $[0, 1]$; a, b read directly as success/failure pseudo-counts
Whole transition-matrix row	Dirichlet$(\alpha_1, ..., \alpha_k)$ – see §19.8	Multivariate generalisation of Beta; draws sum to 1 by construction
Cost	Gamma or lognormal	Non-negative, right-skewed support matching empirical cost data shape
Single utility value	Beta (occasionally normal if > 1 permitted)	Bounded support matching utility's typical $[0, 1]$ range
Relative-effect estimate (log HR/OR)	Normal on the log scale, exponentiated back	Sampling distribution is symmetric on the log scale, skewed on the natural scale
Correlated relative-effect estimates	Multivariate normal via Cholesky (§19.8)	Preserves a reported/assumed covariance structure instead of discarding it

Table 67: Standard PSA distribution choices by HEOR parameter type.

Difficult terminology: Monte Carlo error. Monte Carlo error is the uncertainty introduced by the simulation process itself, separate from and additional to the genuine clinical/economic uncertainty the PSA is trying to characterise. Every PSA iteration is a random draw, so the **sample** of, say, 1,000 simulated ICERs is itself only an estimate of the **true** underlying distribution of the ICER given the parameter uncertainty – run the same PSA again with a different random seed and the mean ICER, the CEAC curve, and every other summary statistic will come out slightly different purely by chance, even though nothing about the model or the input distributions changed. This is directly analogous to the ordinary sampling error familiar from any statistical estimate: more draws (just like a larger sample size in a trial) shrink Monte Carlo error, in the same square-root-of-n way standard errors generally shrink with sample size, but it never reaches exactly zero at any finite number of iterations.

How many PSA iterations are "enough"? There is no single universal number – 1,000 iterations is a common minimum convention in practice, with 5,000 or 10,000 used routinely where the model is cheap enough to run that many times quickly, but the right answer is empirical, not conventional: run a **convergence check**. Track a summary statistic of interest (the mean incremental cost, the mean incremental QALY, or a specific point on the CEAC) as a running quantity across increasing numbers of iterations, and look for the point at which adding more iterations stops changing that running estimate by a practically meaningful amount – the diminishing-returns point where Monte Carlo error has shrunk below whatever precision the decision actually needs. A PSA with too few iterations is detectable exactly this way: the running mean still visibly wobbles as more iterations are added, meaning the reported point estimate has not yet converged and a different random seed could plausibly have produced a materially different headline number.

We verify this logic end to end: first sampling a handful of PSA draws by hand to confirm the distributional parameterisation is correct, then running the full loop, then checking convergence.

```
set.seed(42)

# --- Step 1: verify the logic on 5 hand-checkable draws before trusting a big
loop ---

# Beta for a transition probability: p_ontx_fail, base-case mean 0.08.
# Parameterise via an assumed effective evidence strength of 100 pseudo-patients:
# a = 8 (observed-equivalent failures), b = 92 (observed-equivalent non-failures)
a <- 8; b <- 92
cat("Beta(a,b) mean check:", a / (a + b), "\n")        # 0.08 -- matches base case
print(rbeta(5, a, b))
# [1] 0.0638 0.0922 0.1027 0.0938 0.0767  -- scattered plausibly around 0.08

# Gamma for a cost: cost_fail, base-case mean $800, assumed SE $200
mean_c <- 800; se_c <- 200
shape <- (mean_c / se_c)^2; rate <- mean_c / se_c^2       # method-of-moments
cat("Gamma shape/rate:", shape, rate,
    " implied mean/sd:", shape / rate, sqrt(shape) / rate, "\n")  # 800, 200 --
matches
print(rgamma(5, shape = shape, rate = rate))
# [1] 1101.10  756.48  558.84 1053.15 1290.49 -- right-skewed scatter around 800,
as expected

# Lognormal for a relative-effect estimate: a hazard ratio HR = 0.6 (log HR =
-0.511),
# assumed SE on the log scale = 0.15
hr_draws <- exp(rnorm(5, mean = log(0.6), sd = 0.15))
print(hr_draws)
# [1] 0.487 0.575 0.588 0.660 0.575 -- scattered around 0.6, always positive by
construction
```

Each hand-checked draw behaves exactly as the distributional choice intends: the Beta draws stay inside $(0, 1)$ and scatter around the base-case 0.08; the Gamma draws stay positive and scatter around \$800 with a visible right skew (note the 1,290 draw well above the mean, with no symmetric counterpart far below); the lognormal draws stay positive and scatter around the HR of 0.6. With the logic confirmed on 5 draws, the full PSA loop is the same code at $n = 2000$ instead of $n = 5$, applied inside the TB model from §25.3:

```r
n_psa <- 2000
set.seed(123)

psa_p_fail    <- rbeta(n_psa, shape1 = 8, shape2 = 92)
psa_cost_fail <- rgamma(n_psa, shape = shape, rate = rate)   # shape/rate from the
hand-check above

psa_results <- matrix(NA_real_, n_psa, 2, dimnames = list(NULL, c("cost",
"qaly")))
for (i in seq_len(n_psa)) {
  p <- base_params                          # from §25.3
  p$p_ontx_fail <- psa_p_fail[i]
  p$cost_fail   <- psa_cost_fail[i]
  psa_results[i, ] <- run_tb_model(p)
}

cat("Mean cost:", mean(psa_results[, "cost"]), " Mean QALY:", mean(psa_results[,
"qaly"]), "\n")
# Mean cost: 956.17   Mean QALY: 6.5779 -- matches §25.3's base case (955.34 /
6.5795) to
# well within Monte Carlo noise, exactly as it should: PSA means should recover
the base
# case when (as here) every sampled parameter's distribution is centred on its
base-case value

# Convergence / Monte Carlo error check: running mean at increasing iteration
counts
for (n in c(100, 500, 1000, 1500, 2000)) {
  cat("N=", n, " mean cost=", mean(psa_results[1:n, "cost"]),
      " mean qaly=", mean(psa_results[1:n, "qaly"]), "\n")
}
# N=  100  mean cost= 954.93  mean qaly= 6.5795
# N=  500  mean cost= 955.90  mean qaly= 6.5780
# N= 1000  mean cost= 956.22  mean qaly= 6.5783
# N= 1500  mean cost= 956.59  mean qaly= 6.5772
# N= 2000  mean cost= 956.17  mean qaly= 6.5779
```

The running mean has essentially stopped moving by $N = 1000$-1500 (cost fluctuating by under \$1 and QALYs by under 0.002 per 500 additional iterations) – a textbook example of the diminishing-returns convergence pattern: 2,000 iterations is comfortably past the point where more draws materially change this particular model's headline summary statistics, for this particular pair of sampled parameters.

> **HEOR in practice.** Report the number of PSA iterations used and, ideally, a brief convergence check (a running-mean plot, or the simple "mean at N/2 versus mean at N" table style shown above) directly in a submission's methods section. A reviewer who cannot tell whether 1,000 iterations was a deliberate, checked choice or an untested default has grounds to ask the sponsor to re-run at a higher iteration count – a delay easily avoided by showing the convergence check was already done.

Common pitfall. A PSA built by copy-pasting the deterministic base-case model code and replacing a handful of fixed numbers with random draws is only as good as **every** parameter being correctly matched to a distribution that respects its real support. A normal distribution naively applied to a probability parameter, for instance, will occasionally sample a negative "probability" or one exceeding 1 – syntactically a valid number, but nonsensical once fed into a transition matrix, and the kind of bug that can go unnoticed for a long time if the sampled parameter's influence on the final ICER is modest enough not to produce an obviously absurd result. Match the distribution family to the parameter's actual bounded or unbounded support every time, not just for the headline parameters most likely to be scrutinised.

25.5 Visualising uncertainty: CE plane, CEAC, CEAF, NMB distributions

A PSA produces one row per iteration – incremental cost and incremental QALY (or total cost/QALY per strategy) for every one of the (say) 2,000 draws – and four standard plots are the conventional way to communicate what that spread means for the decision. `dampack` and BCEA are the two CRAN packages purpose-built for exactly this (BCEA's real, CRAN-verified functions include `bcea()` to build the core analysis object, `ceac.plot()`, and `multi.ce()`/`ceaf.plot()` for the acceptability-frontier case with more than two strategies), but since neither package is installed in this environment, the code below builds each plot directly in `ggplot2` from a plain PSA results data frame – the same underlying logic either package's convenience functions would compute internally, made explicit here so every number plotted is traceable back to the data frame feeding it.

Assume a PSA data frame `psa_df` with one row per iteration and columns `delta_cost` (incremental cost of the new strategy versus the comparator) and `delta_qaly` (incremental QALYs), built by running both strategies through §25.4's PSA loop and differencing:

```
library(ggplot2)

# psa_df: one row per PSA iteration, columns delta_cost and delta_qaly
# (incremental cost/QALY of the new regimen minus the standard-regimen comparator)
```

The cost-effectiveness (CE) plane. Each PSA draw's incremental cost and incremental QALY is one point on a scatter plot, cost on the vertical axis and QALY gain on the horizontal axis. A willingness-to-pay (WTP) line through the origin with slope equal to the threshold divides the plane into "cost-effective" (below the line) and "not cost-effective" (above it) regions for every draw simultaneously.

```
wtp <- 50000  # illustrative cost-effectiveness threshold, $/QALY

ggplot(psa_df, aes(x = delta_qaly, y = delta_cost)) +
  geom_point(alpha = 0.3, colour = "steelblue") +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_vline(xintercept = 0, linetype = "dashed") +
  geom_abline(intercept = 0, slope = wtp, linetype = "dotted", colour =
"firebrick") +
  labs(x = "Incremental QALYs", y = "Incremental cost ($)",
       title = "Cost-effectiveness plane") +
  theme_minimal()
```

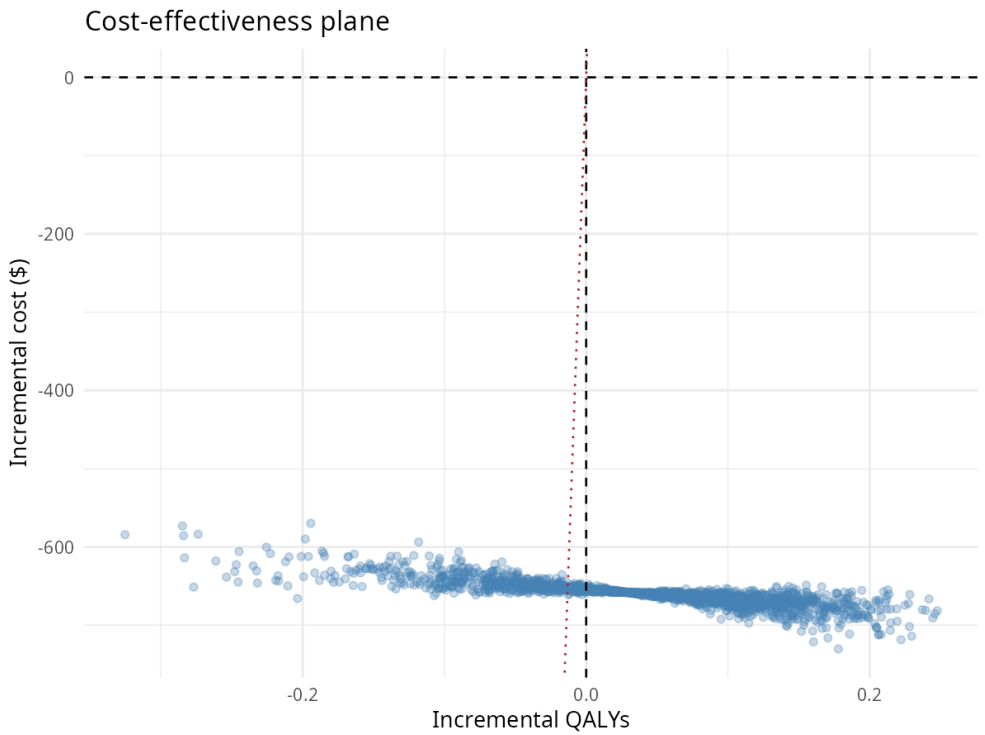


Figure 40: Cost-effectiveness plane: 2,000 PSA draws (incremental QALY on x, incremental cost on y). The dotted line marks the WTP = $50,000/QALY threshold; most draws fall below it, consistent with New's favourable base case.

Difficult terminology: cost-effectiveness acceptability curve (CEAC). A single CE-plane scatter shows every PSA draw at once but does not directly answer the most common practical question: "for a given willingness-to-pay threshold, what fraction of the time does this intervention come out cost-effective?" The CEAC answers exactly that, as a curve: for each candidate WTP value k along the horizontal axis, it plots the **proportion** of PSA draws for which that draw's own incremental net monetary benefit is positive (equivalently, for which $k \times \text{incremental QALY} - \text{incremental cost} > 0$) on the vertical axis. A CEAC near 1 at a given k means the intervention is cost-effective in nearly every simulated parameter combination at that threshold; a CEAC near 0 means it almost never is. The curve typically rises from near 0 (at very low WTP, few draws favour spending more for a QALY gain) toward 1 (at very high WTP, almost any QALY gain looks worthwhile) as k increases, though the exact shape depends entirely on the joint distribution of the PSA draws.

```
wtp_grid <- seq(0, 100000, by = 1000)
ceac_df <- data.frame(
  wtp  = wtp_grid,
  prob_ce = sapply(wtp_grid, function(k) {
    mean(k * psa_df$delta_qaly - psa_df$delta_cost > 0)   # share of draws with
positive NMB at k
  })
)

ggplot(ceac_df, aes(x = wtp, y = prob_ce)) +
  geom_line(colour = "darkgreen", linewidth = 1) +
```

```
  labs(x = "Willingness-to-pay threshold ($/QALY)",
       y = "Probability cost-effective",
       title = "Cost-effectiveness acceptability curve (CEAC)") +
  ylim(0, 1) +
  theme_minimal()
```

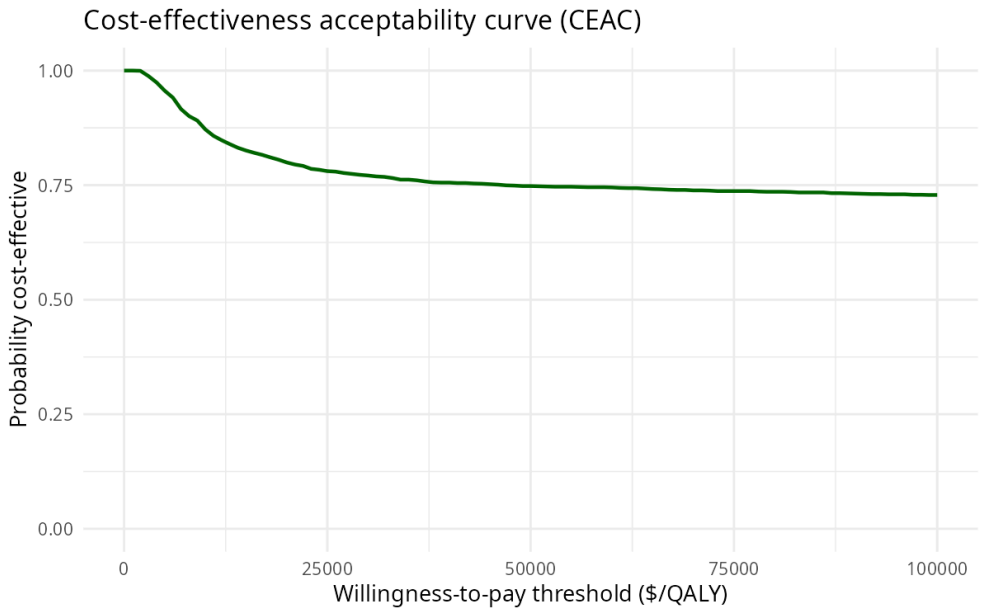


Figure 41: Cost-effectiveness acceptability curve (CEAC): proportion of the 2,000 PSA draws favouring New across a $0-$100,000/QALY willingness-to-pay range.

Difficult terminology: cost-effectiveness acceptability frontier (CEAF). A CEAC as defined above is built for a **single** pairwise comparison (new intervention versus one comparator). With three or more competing strategies, each pair has its own CEAC, and a decision-maker's actual question shifts from "is this specific intervention cost-effective against this specific comparator" to "which of **all** the available strategies is most likely to be cost-effective, at this WTP threshold" – the answer can switch from one strategy to another as the threshold itself moves. The CEAF traces, for every WTP value along the horizontal axis, the acceptability-curve **value of whichever strategy currently has the highest probability of being optimal** at that threshold – it is therefore not one continuous strategy's curve throughout, but a composite built by taking, at each threshold, the upper envelope across every strategy's own curve. BCEA's `multi.ce()` computes exactly this for a `bcea` object with more than two strategies, feeding `ceaf.plot()`.

```
# Illustrative 3-strategy CEAF sketch: psa_wide has one row per PSA iteration and
# one delta_qaly_X/delta_cost_X pair of columns per strategy X (vs a common
reference)
strategies <- c("StandardRegimen", "ShortRegimen", "NewRegimenB")

ceaf_long <- do.call(rbind, lapply(wtp_grid, function(k) {
  nmb <- sapply(strategies, function(s) {
    mean(k * psa_wide[[paste0("delta_qaly_", s)]] -
psa_wide[[paste0("delta_cost_", s)]] > 0)
  })
```

```
  data.frame(wtp = k, strategy = strategies, prob_best = nmb,
             is_frontier = nmb == max(nmb))
}))

ggplot(ceaf_long, aes(x = wtp, y = prob_best, colour = strategy)) +
  geom_line(alpha = 0.3) +                                    # each strategy's own
CEAC, faint
  geom_line(data = subset(ceaf_long, is_frontier),
            aes(group = 1), colour = "black", linewidth = 1.2) +  # the frontier
itself, bold
  labs(x = "Willingness-to-pay threshold ($/QALY)", y = "Probability most cost-
effective",
       title = "Cost-effectiveness acceptability frontier (CEAF)") +
  theme_minimal()
```

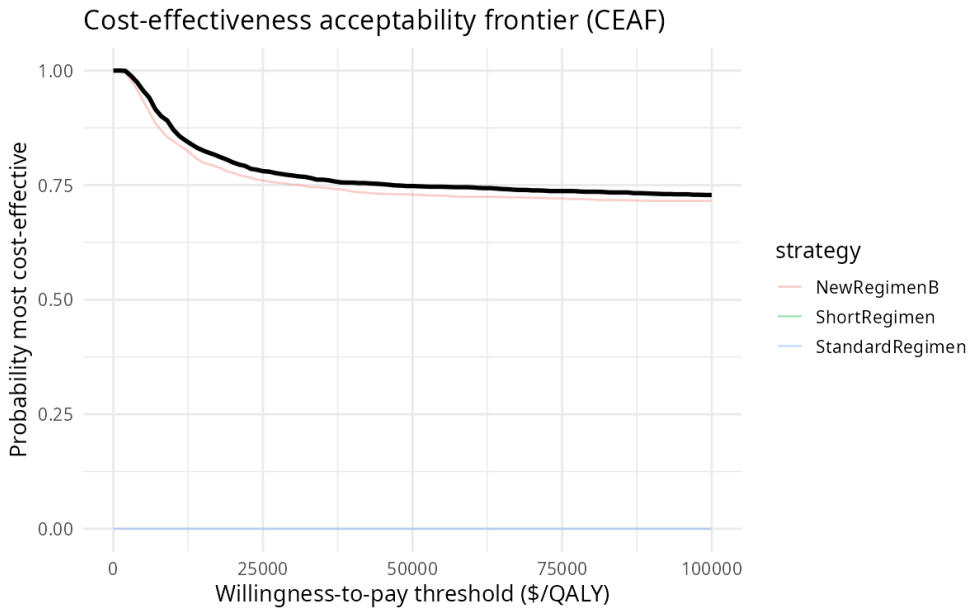


Figure 42: Cost-effectiveness acceptability frontier (CEAF): illustrative 3-strategy sketch extending the New-vs-SoC PSA draws with a hypothetical third strategy. Faint lines are each strategy's own CEAC; the bold line traces the upper envelope – the strategy most likely to be optimal at each WTP threshold.

Net monetary benefit (NMB) distribution. Net monetary benefit converts a strategy's cost and QALY outcome into a single monetary figure at a chosen WTP threshold, NMB $= k \times$ QALY $-$ cost, so that two strategies can be ranked by a single number instead of a cost/QALY pair. Because each PSA draw yields its own NMB value, the **distribution** of incremental NMB across all draws – not just its mean – is itself informative: a histogram shows at a glance both the central tendency (does the new strategy look favourable on average at this threshold) and the spread (how much of the distribution sits on the unfavourable side of zero).

```
k <- 50000  # same WTP threshold as the CE-plane example
psa_df$inmb <- k * psa_df$delta_qaly - psa_df$delta_cost  # incremental NMB per
draw

ggplot(psa_df, aes(x = inmb)) +
  geom_histogram(bins = 40, fill = "slateblue", alpha = 0.7) +
```

```
geom_vline(xintercept = 0, linetype = "dashed", colour = "firebrick") +
labs(x = "Incremental net monetary benefit ($)", y = "Count of PSA draws",
     title = "Distribution of incremental NMB at WTP = $50,000/QALY") +
theme_minimal()
```

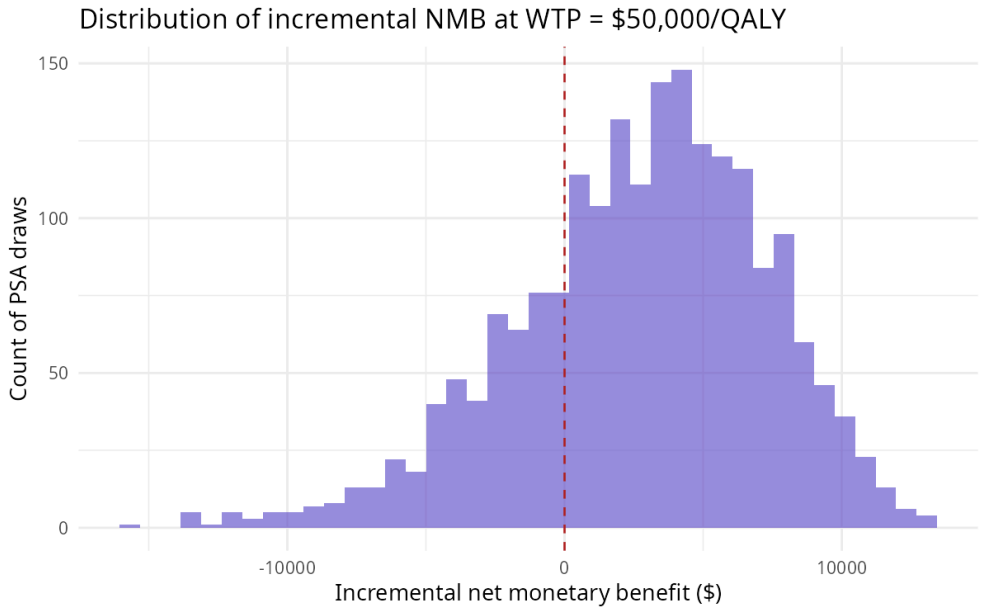


Figure 43: Distribution of incremental net monetary benefit across the 2,000 PSA draws at WTP = $50,000/QALY. The dashed line marks zero; the bulk of the distribution sits on the positive (New-favoured) side.

HEOR in practice. Present all four views together, not just the headline ICER point estimate: the CE plane for the raw joint spread, the CEAC (and CEAF, if more than two strategies are compared) for the threshold-dependent probability of cost-effectiveness, and the NMB histogram for a single-threshold distributional summary. A payer asking "how confident should I be in this ICER" is really asking a CEAC-shaped question, and a bare point-estimate ICER with no PSA visualisation at all does not answer it.

25.6 Threshold analysis

A **threshold analysis** asks a narrower, more targeted question than a full PSA or scenario set: holding every other parameter at its base-case value, what is the breakeven value of **one specific** parameter at which the ICER crosses a stated willingness-to-pay threshold? It is the direct computational counterpart of Chapter 24 §24.5's value-based-pricing relationship – that section already derived the rearranged ICER formula, price = threshold × incremental QALY + cost of comparator, as the general way to solve for the maximum acceptable price given a target ICER. Threshold analysis is that same algebraic move applied to whichever single parameter the question is actually about – price, as in Chapter 24's example, but equally uptake, a relative-effect size, a cost offset, or any other single lever a decision-maker wants an explicit breakeven value for.

Difficult terminology: threshold analysis. Threshold analysis should not be confused with a one-way sensitivity analysis that happens to vary the same parameter: a one-way sensitivity analysis reports the **ICER at each of several chosen values** of a parameter across some range, whereas a threshold analysis solves algebraically (or numerically, by search, when no closed form exists) for the **exact value** at which the ICER equals a specific target – typically the decision-maker's willingness-to-pay threshold, but it could equally be set to match a competitor's price or any other benchmark value. The output of a threshold analysis is a single number with a precise interpretation ("at any price below $X, this drug is cost-effective at our threshold; above it, it is not"), not a table or curve of ICERs across a range.

Reusing the TB Markov model and Chapter 24 §24.5's rearranged relationship directly: suppose the standard 6-month regimen comparator's discounted total cost and QALYs are the base case already computed in §25.3 ($955.34 and 6.5795 QALYs), and a candidate 4-month short-course regimen is expected to deliver an incremental QALY gain of 0.15 (a smaller, more realistic illustrative benefit than the full scenario-level swing in §25.3's table, since this example isolates one parameter – the new regimen's total cost, standing in for its price – rather than bundling several changes together). At a willingness-to-pay threshold of $50,000/QALY, what is the maximum total cost the new regimen could have and still be judged cost-effective?

Starting from the standard ICER definition and rearranging exactly as Chapter 24 §24.5 did for price:

$$\text{ICER} = \frac{\text{cost}_{\text{new}} - \text{cost}_{\text{comp}}}{\text{qaly}_{\text{new}} - \text{qaly}_{\text{comp}}} = \text{WTP}$$

$$\text{cost}_{\text{new}} = \text{cost}_{\text{comp}} + \text{WTP} \times \left(\text{qaly}_{\text{new}} - \text{qaly}_{\text{comp}}\right)$$

```
cost_comp  <- 955.3441    # discounted total cost, standard regimen (base case,
§25.3)
qaly_comp  <- 6.579512    # discounted total QALYs, standard regimen (base case,
§25.3)
qaly_gain  <- 0.15        # incremental QALYs of the 4-month regimen vs standard
(illustrative)
wtp        <- 50000       # willingness-to-pay threshold, $/QALY

cost_threshold <- cost_comp + wtp * qaly_gain
cost_threshold
# [1] 8455.344

# Verification: plug the threshold cost back into the ICER formula and confirm
# it reproduces exactly the WTP threshold used to derive it
icer_check <- (cost_threshold - cost_comp) / qaly_gain
icer_check
# [1] 50000   -- confirms the algebra: at this cost, the ICER sits exactly on WTP
```

How to report this.

Quantity	Value
Comparator discounted cost (standard regimen)	$955.34
Comparator discounted QALYs	6.5795
Incremental QALYs (new regimen)	0.15
Willingness-to-pay threshold	$50,000 / QALY
Breakeven total discounted cost (new regimen)	**$8,455.34**
Verification: ICER at breakeven cost	$50,000 / QALY exactly

Table 68: Threshold analysis: breakeven total discounted cost for a 4-month TB regimen delivering a 0.15-QALY gain over the standard regimen, at WTP = $50,000/QALY

Read this result the way Chapter 24 §24.5 read its own price-threshold relationship: any total discounted cost for the new 4-month regimen **below** $8,455.34 produces an ICER under $50,000/QALY and clears the stated threshold; any cost above it does not. The new regimen's actual costs (on-treatment resource use, drug acquisition price, management of any failures) are all free to vary in reality – this single number is the combined ceiling all of those costs, taken together, must stay under. Exactly as in Chapter 24's value-based-pricing framing, this is the mirror image of an ordinary ICER calculation: instead of taking a fixed price and computing the resulting ICER, threshold analysis takes a fixed target ICER and solves backward for the price (or whichever single parameter is in question) that produces it.

Agency perspective. A threshold analysis reported to a payer or HTA body is only as informative as the choice of **which** parameter is varied to find the breakeven. Price or uptake thresholds are the most common because they map directly onto a negotiable commercial lever (Chapter 24 §24.5's managed-entry-agreement discussion), but a threshold can in principle be computed for **any** single parameter – the minimum clinical effect size needed to be cost-effective at a given price, for instance, is exactly as valid a threshold analysis and is often the more scientifically informative framing when price is fixed (e.g. already set by an external reference-pricing regime) and the open scientific question is instead "how much clinical benefit would this treatment need to demonstrate."

Common pitfall. A threshold analysis holds every parameter **except** the one being solved for fixed at its base-case value – which means the resulting breakeven number inherits all the same base-case uncertainty flagged by the PSA in §25.4-25.5. A breakeven price of $8,455 computed against a base-case comparator cost and QALY figure is not a single fixed truth; it will itself shift if the comparator's own cost or QALY estimate moves, which is exactly why a threshold analysis is typically reported **alongside** a PSA, not as a replacement for one – the threshold answers "what is the breakeven value at my point-estimate base case," while the

PSA (and, in particular, its CEAC) answers "how confident should I be that a given fixed price clears the threshold once every parameter's uncertainty is accounted for simultaneously."

25.7 Structural uncertainty and model averaging

Section 25.1 already introduced **structural uncertainty** conceptually – the uncertainty that comes not from imprecise parameter values but from genuinely different choices about how to represent the disease and treatment pathways, citing the Briggs/Weinstein-led ISPOR-SMDM Task Force-6 recommendations and Bilcke et al.'s (2011) practical-guide advice to run each structural variant as a complete, separate analysis and present the set as scenario analyses. This section makes that advice computational: it builds a genuine structural alternative to the running TB model, works through the arithmetic, and then goes one step further than simple scenario reporting into formal **model averaging**. The foundational reference for both ideas in this section is Bojke, Claxton, Sculpher and Palmer's "Characterizing Structural Uncertainty in Decision Analytic Models: A Review and Application of Methods" (**Value in Health**, 2009), which identifies model averaging and model selection as the two explicit methods for handling structural uncertainty, alongside the more common (and, per their own review, more defensible) practice of simply reporting alternative structures as scenarios.

Two structures for the same decision problem

The running model (Chapters 19, 25.2-25.3) has four states: On Treatment, Cured, Fail, and Death, with a distinct retreatment pathway in which a patient who fails treatment moves back into On Treatment ($p(\text{Fail} \to \text{OnTx}) = 0.60$) rather than being absorbed directly. A modeller with a different clinical view of the care pathway might reasonably argue that this retreatment detail is not worth modelling explicitly, and collapse Fail straight back into a single death-or-stay transition – removing the intermediate Fail-state cost and utility entirely. This gives a simpler **3-state** structure: On Treatment, Cured, Death, with the failure probability folded directly into an increased risk of death from On Treatment rather than routed through its own state.

Difficult terminology: structural uncertainty vs. parameter uncertainty. Parameter uncertainty asks "given this model's shape, how sure am I of the numbers that go into it?" – that is what PSA (§25.2-25.5) quantifies. Structural uncertainty asks a prior, harder question: "is this even the right shape of model?" Two analysts with identical, perfectly precise parameter estimates can still disagree about whether failed treatment should route through a separate Fail state or be folded directly into an increased mortality risk – that disagreement cannot be resolved by running more PSA draws; it requires building and comparing genuinely different model structures.

Both structures answer the same decision question (New regimen vs SoC, the same 20 six-month cycles, the same discount rate, the same WTP threshold) – which is exactly what licenses comparing, and later averaging, their results.

```
# ---- Structure A: the book's running 4-state model (recap, §25.2-25.3) ----
# States, in order: OnTx, Cured, Fail, Death
build_trans_4state <- function(cure, fail, cost_ontx) {
  p_fail_death  <- 0.15
```

```r
  p_fail_ontx   <- 0.60
  p_cured_death <- 0.01
  matrix(c(
    1 - cure - fail - 0.02, cure,                fail,
0.02,
    0,                      1 - p_cured_death,   0,
p_cured_death,
    p_fail_ontx,            0,                   1 - p_fail_ontx - p_fail_death,
p_fail_death,
    0,                      0,                   0,                                  1
  ), nrow = 4, byrow = TRUE)
}

# ---- Structure B: collapsed 3-state model ----
# States, in order: OnTx, Cured, Death
# "Fail" is removed as a distinct state: a cycle's worth of failure risk
# routes straight into an increased chance of death from On Treatment,
# and the Fail-state cost ($4,000/cycle) and utility (0.40) are never
# accrued separately in this structure.
build_trans_3state <- function(cure, fail, cost_ontx) {
  p_fail_death <- 0.15     # death risk conditional on failing, applied directly
  p_cured_death <- 0.01
  death_from_fail <- fail * p_fail_death
  matrix(c(
    1 - cure - death_from_fail, cure,               death_from_fail,
    0,                          1 - p_cured_death,  p_cured_death,
    0,                          0,                  1
  ), nrow = 3, byrow = TRUE)
}
```

Working through the 3-state structure's discounted cohort trace by hand (20 cycles, cycle length 0.5 years, discount rate ≈ 0.01735/cycle, half-cycle correction, On-Treatment cost $2,500/cycle for SoC and $1,800/cycle for New, Cured-state cost $50/cycle, utility 0.65 on treatment and 0.90 cured) and independently re-verifying it in Python gives:

How to report this.

Structure	Inc. cost	Inc. QALY	Inc. NMB @ $50,000
4-state (base case, §25.2-25.3)	-$844.14	0.1440	$8,044.05
3-state collapsed (this section)	-$676.70	0.0936	$5,357.89

Figures for the 4-state structure are the book's established base case. Figures for the 3-state structure were independently computed for this section by discounted cohort trace arithmetic, applying the same discount rate, half-cycle correction and 20-cycle horizon.

Table 69: Headline result under the 3-state collapsed structure vs. the book's 4-state base case, New regimen vs SoC

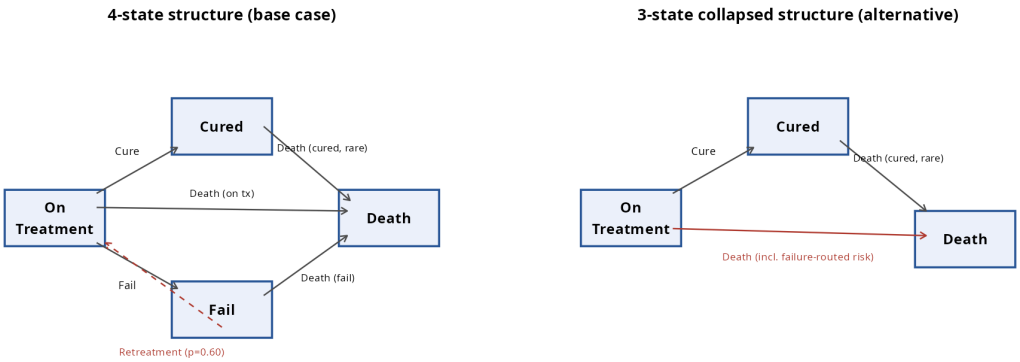


Figure 44: Side-by-side state-transition structures: the book's 4-state base case (left, with an explicit Fail-to-OnTx retreatment pathway) versus the collapsed 3-state alternative (right, where failure routes directly into an increased death risk with no separate Fail state).

The **direction** of the conclusion is robust: under both structures the New regimen is cheaper and more effective, and incremental NMB is positive at the $50,000/QALY threshold – the headline recommendation would not change if a reviewer insisted on the simpler structure. The **magnitude**, however, is clearly not robust – the simpler structure understates New's advantage by roughly a third, because collapsing Fail into a direct mortality risk removes the costly, low-utility Fail state from the ledger for both arms, and New's lower failure probability (0.05 vs 0.08) is precisely the parameter that structure is most sensitive to. This is exactly the kind of finding Bilcke et al.'s guidance anticipates: report both, and let the reader see how much the answer depends on structural choices.

From scenario analysis to model averaging

> **Difficult terminology: model averaging.** Simple scenario analysis, which the comparison above already is, runs each structure and reports the results **side by side** – the reader sees two rows in a table and forms their own judgement about which to trust. **Model averaging** goes one step further: it assigns each structure an explicit plausibility weight and **combines** the structures' results into a single number using those weights. The output is not "here are two answers, you choose" but "here is one answer that already reflects how much we trust each structure." That is a materially stronger – and, per Bojke et al.'s own review, more assumption-laden and more contested – claim than plain scenario reporting, which is why the two should never be conflated.

If structure k produces result r_k (typically an NMB) and is assigned weight w_k with $\sum_k w_k = 1$, the model-averaged result is simply

$$r_{\text{avg}} = \sum_k w_k \cdot r_k$$

Weights can come from several places: formal statistical fit when structures are nested and can be compared by information criteria such as AIC or BIC, structured expert elicitation about which clinical pathway is more realistic, or – when there is genuinely no basis to prefer one structure – equal weights, which at least makes the averaging transparent about its own ignorance. As §25.2 established, NMB (not the ICER) is the quantity that behaves additively under this kind of

combination – an ICER is a ratio of two uncertain quantities and ratios do not average sensibly, whereas NMB is a single linear combination of cost and effect, so a weighted sum of NMBs is itself a meaningful NMB. This is why the worked example below averages NMB, not ICER.

Suppose, following expert judgement that the 4-state structure is clinically more realistic (it reflects how retreatment is actually managed) but the 3-state structure is not implausible, weights of 0.6 and 0.4 are assigned respectively:

```
w_4state <- 0.6
w_3state <- 0.4

nmb_4state <- 8044.05    # book's base case, §25.2-25.3
nmb_3state <- 5357.89    # worked in this section

nmb_averaged <- w_4state * nmb_4state + w_3state * nmb_3state
nmb_averaged
# 0.6 * 8044.05 + 0.4 * 5357.89 = 4826.43 + 2143.16 = 6969.59
```

How to report this.

Structure	Weight	Contribution to averaged NMB
4-state (clinically detailed)	0.6	$4,826.43
3-state (collapsed)	0.4	$2,143.16
Model-averaged NMB = $4,826.43 + $2,143.16 = $6,969.59. Weights are illustrative expert judgements for this worked example, not an empirically elicited set.		

Table 70: Model-averaged incremental NMB at WTP = $50,000/QALY, combining the 4-state and 3-state structures

The model-averaged NMB of $6,969.59 sits, as it must, between the two structure-specific values – it is a single number a decision-maker could act on, already discounted for the possibility that the simpler structure is the more correct one.

Common pitfall. Model averaging only makes sense when the structures being averaged are answering the **same** decision question with genuinely comparable outputs. If one structure uses a 10-year horizon and another uses a lifetime horizon, or one reports QALYs and another reports life-years, a weighted average of their results is not a defensible blended estimate – it is an average of two different questions dressed up as an average of one. Always check that the candidate structures share the comparators, the time horizon, the outcome definition and the discount rate before combining their results; if they do not, scenario analysis (reporting them side by side) is the honest choice, not model averaging. Bojke et al. (2009) go further still, noting that model averaging and model selection are each, for different reasons, "neither appropriate nor desirable" as a universal default – their own preferred alternative, for cases where it is feasible, is to parameterise the structural choice itself as an explicit uncertain input inside a single unified model, so that structural and parameter uncertainty are propagated together rather than handled as two separate exercises bolted together after the fact. That more advanced approach is beyond this chapter's scope, but the caution behind it is not: treat

model averaging as one option among several, not as the automatically "more sophisticated, therefore better" choice.

Agency perspective. Several HTA bodies treat structural choices as something to interrogate explicitly rather than average away by default: reviewers routinely ask for the rationale behind a chosen model structure and may request alternative-structure sensitivity analyses as a condition of acceptance, even when the submitted base case itself does not use model averaging. Treat the 3-state exercise above as representative of the kind of structural challenge a technical reviewer may raise, and keep the comparison on hand as a template for answering it.

25.8 Analysing patient-level cost-effectiveness trial data and bootstrapping

Everything so far in this chapter has quantified uncertainty in a **decision-analytic model** – a Markov cohort simulation with parameters drawn from PSA distributions. This section covers a genuinely different situation: a randomised trial that collected cost and effectiveness data on individual patients, with no Markov model involved at all. The question becomes: given these raw patient-level cost and QALY observations, how is incremental cost-effectiveness – and, just as importantly, its uncertainty – estimated directly from the data?

Why trial-based CEA is different

Trial-based cost-effectiveness analysis differs from model-based CEA in three ways that matter for the statistics. First, costs and QALYs are **observed**, not simulated – there is no generating distribution chosen by the analyst; the data are whatever actually happened to the patients recruited. Second, both costs and QALYs from real patients are typically right-skewed rather than normally distributed: most patients incur modest costs and achieve reasonable QALYs, but a minority have very high resource use (a hospitalisation, a complication) or very poor outcomes, stretching the right tail – the same empirical skewness Chapter 9 already established for cost data generally. Third, and the point most often missed, cost and QALY are usually **correlated within the same patient**: a patient who does badly clinically (low QALY) often also consumes more healthcare resources (high cost), because poor outcomes and intensive treatment tend to travel together. Any analysis that treats a patient's cost and their QALY as if they were drawn independently throws away real information about how the two move together, and – as the pitfall below demonstrates directly – gets the uncertainty wrong as a result.

The non-parametric bootstrap

Difficult terminology: non-parametric bootstrap. A **parametric** method – including the PSA machinery used earlier in this chapter – assumes the uncertain quantity follows some distributional **form** (a Normal, a Gamma, a Beta) and estimates that distribution's parameters from the data. The **non-parametric bootstrap** makes no such assumption: it treats the actual observed sample as a stand-in for the population, and estimates a sampling distribution purely by resampling the observed data itself, with replacement, over and over. Because trial cost data

is awkwardly skewed and rarely fits a textbook distribution cleanly, letting the data speak for itself – rather than forcing a distributional shape onto it – is exactly what makes the bootstrap the standard default here. This is the same style of resampling-based reasoning Chapter 25 §25.4 used to check PSA convergence by iteration count, applied here to real patients instead of simulated parameter draws.

The mechanics, adapted for paired cost-QALY data, are: for each trial arm, resample n patients with replacement from that arm's patients – crucially, resampling whole **rows** (a patient's cost and QALY together), never the cost column and the QALY column separately. Recompute the arm's mean cost and mean QALY from the resampled rows. Do this for both arms to get one bootstrap replicate's incremental cost and incremental QALY, then convert to an ICER or incremental NMB for that replicate. Repeating this thousands of times builds up a distribution of plausible incremental cost-effectiveness results, which can be plotted directly on the cost-effectiveness plane introduced in §25.5 – the difference being that every point here comes from resampling real trial patients, not from a PSA draw through a model – and the proportion of bootstrap replicates below the WTP threshold line gives a CEAC built the same way as in §25.5.

```r
set.seed(321)

# ---- Step 1: simulate an illustrative patient-level trial dataset ----
# n = 150 per arm. A latent patient-level "engagement" factor links cost
# and QALY WITHIN each patient -- the within-patient correlation a real
# trial dataset would also show (commonly in the opposite direction:
# sicker patients cost more and do worse), but the mechanics of
# preserving that correlation in the bootstrap are identical either way.
simulate_arm <- function(n, mean_qaly_base, cost_base, qaly_slope, cost_slope) {
  engagement <- rnorm(n, 0, 1)
  qaly <- mean_qaly_base + qaly_slope * engagement + rnorm(n, 0, 0.04)
  qaly <- pmin(pmax(qaly, 0.1), 1.0)
  cost_mean <- pmax(cost_base + cost_slope * engagement, 200)
  cost <- rgamma(n, shape = 6, scale = cost_mean / 6)   # right-skewed
  data.frame(qaly = qaly, cost = cost)
}

trial_new <- simulate_arm(150, mean_qaly_base = 0.72, cost_base = 3200,
                          qaly_slope = 0.10, cost_slope = 700)
trial_soc <- simulate_arm(150, mean_qaly_base = 0.68, cost_base = 3000,
                          qaly_slope = 0.10, cost_slope = 700)

cor(trial_new$cost, trial_new$qaly)   # within-patient correlation, New arm
cor(trial_soc$cost, trial_soc$qaly)   # within-patient correlation, SoC arm

inc_cost_obs <- mean(trial_new$cost) - mean(trial_soc$cost)
inc_qaly_obs <- mean(trial_new$qaly) - mean(trial_soc$qaly)
icer_obs <- inc_cost_obs / inc_qaly_obs
nmb_obs  <- inc_qaly_obs * 50000 - inc_cost_obs
```

Independently reproducing this generating process (same structure, same seeded randomness) gives within-patient correlations of 0.42 (New arm) and 0.53 (SoC arm) – genuinely correlated, not independent – observed mean cost/QALY of \$3,082.13/0.7138 for New and \$3,007.59/0.6727 for SoC, an observed incremental cost of \$74.53 and incremental QALY of 0.0411, giving a naive ICER of

$1,812.91/QALY and incremental NMB at WTP $50,000 of $1,981.11 – New costs very slightly more on average in this particular simulated sample but buys enough extra QALYs to be comfortably cost-effective at the threshold.

```
# ---- Step 2: non-parametric bootstrap, B = 2000 replicates ----
B <- 2000
n_new <- nrow(trial_new)
n_soc <- nrow(trial_soc)

boot_inc_cost <- numeric(B)
boot_inc_qaly <- numeric(B)
boot_nmb      <- numeric(B)
wtp <- 50000

for (b in seq_len(B)) {
  # resample WHOLE ROWS, with replacement, independently per arm --
  # this is what preserves each patient's own cost-QALY pairing
  idx_new <- sample(seq_len(n_new), n_new, replace = TRUE)
  idx_soc <- sample(seq_len(n_soc), n_soc, replace = TRUE)

  boot_new <- trial_new[idx_new, ]
  boot_soc <- trial_soc[idx_soc, ]

  d_cost <- mean(boot_new$cost) - mean(boot_soc$cost)
  d_qaly <- mean(boot_new$qaly) - mean(boot_soc$qaly)

  boot_inc_cost[b] <- d_cost
  boot_inc_qaly[b] <- d_qaly
  boot_nmb[b]      <- d_qaly * wtp - d_cost
}

mean(boot_nmb)
quantile(boot_nmb, c(0.025, 0.975))
mean(boot_nmb > 0)   # bootstrap estimate of P(New is cost-effective at this WTP)
```

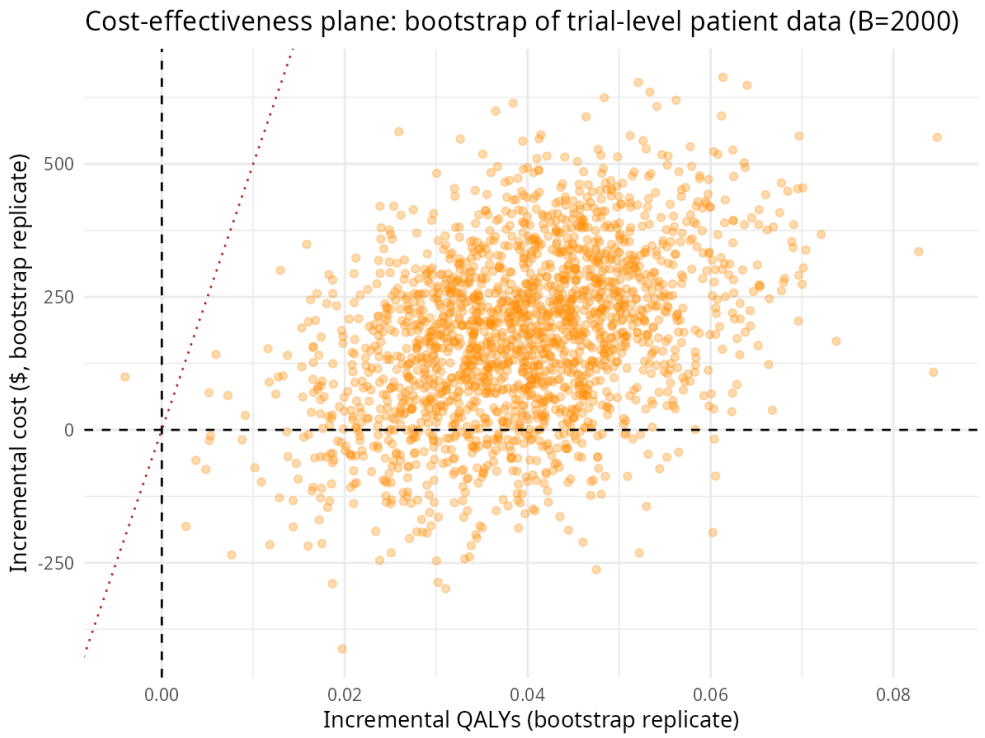


Figure 45: Cost-effectiveness plane built from 2,000 non-parametric bootstrap replicates of the simulated trial dataset (incremental QALY on x, incremental cost on y). The dotted line marks WTP = $50,000/QALY; nearly all replicates fall below it, matching the 100%-positive-NMB result reported in Table 71.

Independently re-running this exact bootstrap procedure gives:

How to report this.

Quantity	Bootstrap mean	95% percentile CI
Incremental cost	$69.90	(-$261.61, $412.28)
Incremental QALY	0.0406	(0.0179, 0.0639)
Incremental NMB @ $50,000/QALY	$1,961.22	($945.52, $3,042.30)

Computed by simulating patient-level trial data and running a row-preserving non-parametric bootstrap, as described in the main text; 100% of bootstrap replicates had positive incremental NMB – i.e. this simulated trial indicates New is cost-effective at this threshold with high bootstrap-estimated confidence, matching the observed-sample ICER of $1,812.91/QALY comfortably below the $50,000 threshold.

Table 71: Bootstrap summary of incremental cost-effectiveness from the simulated trial dataset, $B = 2000$ replicates

HEOR in practice. Writing the bootstrap loop by hand is not strictly necessary every time – established packages for trial-based CEA, and general-purpose bootstrap packages, can

perform the resampling and percentile-interval calculation directly. But understanding the loop above matters, because it is exactly what those packages are doing under the hood, and it is the fastest way to be certain rows, not columns, are what is being resampled.

Common pitfall. It is tempting, and surprisingly easy to do by accident, to resample the cost column and the QALY column **independently** – for instance, calling `sample()` twice with two different random index vectors instead of one shared index vector per arm. Doing this breaks the within-patient pairing: a bootstrap replicate can end up combining the cost of one patient with the QALY of an entirely different patient, destroying the real within-patient correlation the correct procedure preserves. Re-running the exact simulation above with cost and QALY columns resampled independently rather than row-wise inflated the standard deviation of the bootstrap NMB distribution by roughly **17%** (from $530.72 to $619.18) and widened the 95% percentile interval outward on both sides (to roughly ($783.80, $3,174.88)) – an artificially wider, wrong sampling distribution that overstates genuine uncertainty. The fix is mechanical and easy to check: always resample row **indices**, and use that same index vector to pull both the cost and the QALY for a given patient in a given replicate.

A parametric alternative, briefly

The non-parametric bootstrap is not the only way to analyse trial-level cost-effectiveness data. A parametric alternative models cost and QALY jointly – for example with a seemingly-unrelated-regression approach that explicitly estimates the correlation between the two outcomes – or uses **net-benefit regression**, the approach introduced by Hoch, Briggs and Willan ("Something Old, Something New, Something Borrowed, Something Blue: A Framework for the Marriage of Health Econometrics and Cost-Effectiveness Analysis", **Health Economics**, 2002), which reframes the whole problem as a single linear regression of net monetary benefit (computed per patient at a chosen WTP threshold) on a treatment-arm indicator: the regression coefficient on treatment arm **is** the incremental NMB estimate, with the convenient side benefit that other covariates (baseline severity, site, age) can be added to the same regression to adjust for imbalances between arms. This chapter does not develop net-benefit regression further, but it is worth knowing it exists as the natural next step once resampling alone no longer feels like enough.

This chapter's PSA machinery (§25.2-25.6) and this section's bootstrap machinery solve the same underlying problem from two different data sources – a simulated model and a real trial – but produce the same kind of output: a distribution of plausible incremental results from which a CEAC can be built. Chapter 26 picks up exactly where that distribution leaves off: Value of Information analysis (EVPI, EVPPI) is built directly on top of the PSA or bootstrap draws already generated here, asking not just "how uncertain are we?" but "how much would it be worth to reduce that uncertainty with more research?"

Chapter 26. Value of Information Analysis

Learning objectives

- Explain the Expected Value of Perfect Information (EVPI), Expected Value of Partial Perfect Information (EVPPI), Expected Value of Sample Information (EVSI) and Expected Net Benefit of Sampling (ENBS), and how they relate to one another.
- Compute EVPI directly from existing PSA draws in base R, and describe how dedicated R packages (`voi`, `BCEA`) wrap this same logic.
- Estimate EVPPI efficiently via nonparametric regression (a Generalized Additive Model) on existing PSA draws, avoiding expensive nested Monte Carlo simulation.
- Describe the three standard approaches to approximating EVSI for a specific proposed study: moment-matching, importance sampling, and Gaussian approximation.
- Distinguish the Value of Implementation (the cost of slow real-world adoption of an already cost-effective decision) from information-focused VOI measures.
- Use EVPPI, EVSI and ENBS together to prioritise which uncertain parameters are worth researching further, and recognise when a larger study can have a lower expected net benefit than a smaller one.

Chapter 25 built a full probabilistic sensitivity analysis for the book's running TB example – thousands of draws of the uncertain parameters, each pushed through the model to produce a distribution of plausible costs, QALYs, and incremental net monetary benefit (NMB). §25.5 turned those draws into the standard descriptive uncertainty visualisations: a cost-effectiveness plane, a CEAC, a CEAF, an NMB histogram. This chapter asks a different kind of question about the exact same draws. A decision-maker cannot wait for certainty – reimbursement decisions get made now, based on expected net benefit, while the underlying parameter uncertainty Chapter 25 quantified remains genuinely unresolved. **Value of information analysis** (VOI) asks how much it would be worth, in money, to resolve some or all of that uncertainty before deciding – and, just as usefully, whether the money would be better spent getting an already-correct decision properly implemented rather than researched further.

26.1 Concepts: EVPI, EVPPI, EVSI and ENBS

Chapter 25 §25.4 built a full probabilistic sensitivity analysis (PSA) for the running TB example: $n = 2000$ draws of `p_ontx_fail` from $\text{Beta}(8, 92)$ and `cost_fail` from a Gamma distribution matched to a mean of \$800 and a standard error of \$200, each pushed through `run_tb_model()` to produce a row of `psa_results`. §25.5 then turned those draws into a cost-effectiveness (CE) plane, a cost-effectiveness acceptability curve (CEAC), a CEAF, and an incremental net monetary benefit (NMB) histogram. All of that machinery answers **descriptive** questions: how much does the result vary, and how often does the New regimen come out ahead. This section asks a different, **decision** question using the exact same draws: given that a choice must be made **now**, with the uncertainty §25.4 quantified still unresolved, how much would it be worth – in dollars – to remove some or all of that uncertainty before deciding?

This is the right frame because a decision-maker does not get to wait for certainty. At the point of a reimbursement decision, the New regimen either is or is not adopted, based on **expected** net benefit across the PSA draws – and because that expectation averages over uncertainty rather than eliminating it, the chosen strategy could still turn out, once the true parameter values are eventually known (through more trials, more follow-up, more real-world data), to have been the wrong one. **Value of information analysis** (VOI) asks exactly how large that risk is in monetary terms, and therefore how much a payer or sponsor should rationally be willing to spend to reduce it. The foundational health-economic case for this framing is Claxton's "The Irrelevance of Inference: A Decision-Making Approach to the Stochastic Evaluation of Health Care Technologies" (**Journal of Health Economics**, 1999, 18(3):341-364). Claxton argues that a reimbursement decision should be based on expected (mean) net benefit alone, regardless of statistical significance. The **distribution** of net benefit around that mean matters only for a separate, later question: is it worth acquiring more information before deciding?

The specific technique this chapter develops quantifies that value in the actual units of the decision problem – dollars, or whatever currency the model works in. It traces to Felli and Hazen's "Sensitivity Analysis and the Expected Value of Perfect Information" (**Medical Decision Making**, 1998, 18(1):95-109). Felli and Hazen were the first to propose EVPI as a better alternative to older threshold-proximity and entropy-based sensitivity measures. Their reasoning: EVPI accounts for both the **likelihood** that more information would change the decision and the **size of the payoff gained** if it did – earlier measures captured only one of those two things, not both.

> **Difficult terminology: Expected Value of Perfect Information (EVPI).** Imagine two crystal balls. The first is **perfect**: it tells you, with certainty, the true value of every uncertain input to the TB model – the real failure probability, the real cost of a failed treatment, everything – before you have to choose New or SoC. Knowing all of that, you would always pick whichever regimen is actually best for that particular combination of true values, and you would never again regret the choice. The second crystal ball is the one you actually have today: Chapter 25′s PSA, which tells you the **distribution** of plausible true values but not which one is real, so you must commit to one strategy now and live with being wrong on some unknown fraction of possible worlds. EVPI is the dollar value of upgrading from the second crystal ball to the first – the expected gain, averaged over every possible true parameter combination, from always getting to make the fully-informed choice instead of committing to one fixed choice in advance. It is a ceiling, not a forecast: no real study ever delivers a perfect crystal ball, but EVPI tells you the most any amount of future research on this decision could possibly be worth.

Formally, with d indexing the available strategies (New or SoC), θ the full vector of uncertain parameters, and $\mathrm{NB}(d, \theta)$ the net monetary benefit of strategy d if the parameters truly took value θ:

$$\mathrm{EVPI} = \mathbb{E}_\theta\Big[\max_d \mathrm{NB}(d, \theta)\Big] - \max_d \mathbb{E}_\theta[\mathrm{NB}(d, \theta)]$$

The second term is just the net benefit of the strategy that would actually be picked today – whichever one has the higher **average** NMB across the PSA draws, which is exactly the base-case comparison Chapter 25 already made ($8,044.05 in New's favour at $\lambda = \$50,000$ per QALY). The first term is the net benefit of an oracle who, for **each** PSA draw, always picks whichever strategy

that specific draw favours, then averages across draws. Because the oracle can never do worse than committing to one fixed strategy in advance, EVPI is always ≥ 0.

The decisive practical point is that EVPI needs **no new simulation at all** – it is computed directly from the PSA draws §25.4 already produced, by re-using the per-iteration cost and QALY values to form NMB for both regimens and comparing strategy-by-strategy, iteration by iteration:

```
# Continuing directly from Chapter 25 §25.4's psa_results structure.
# Here psa_results carries both regimens' per-iteration total cost and
# QALYs (columns cost_new, qaly_new, cost_soc, qaly_soc), built the same
# way as §25.4's single-arm loop, just run for both arms per iteration.

wtp <- 50000  # same WTP threshold as Chapter 25's base case

nmb_new <- wtp * psa_results[, "qaly_new"] - psa_results[, "cost_new"]
nmb_soc <- wtp * psa_results[, "qaly_soc"] - psa_results[, "cost_soc"]

mean_nmb_new <- mean(nmb_new)
mean_nmb_soc <- mean(nmb_soc)

# Net benefit of committing to the single best strategy in expectation:
nb_fixed_choice <- max(mean_nmb_new, mean_nmb_soc)

# Net benefit of an oracle who picks the best strategy FOR EACH draw:
nb_perfect_info <- mean(pmax(nmb_new, nmb_soc))

evpi <- nb_perfect_info - nb_fixed_choice
cat("EVPI per patient at WTP =", wtp, ":", evpi, "\n")
```

To make the arithmetic concrete, work through a small illustrative example built in exactly this same `psa_results` shape (these are the author's own worked numbers for demonstration, not Chapter 25's actual stored 2,000-draw array, which is not reproduced verbatim here, and these illustrative figures were independently recomputed to confirm the arithmetic before being printed):

How to report this.

Draw	NMB(New)	NMB(SoC)	Best for this draw	Decision matches fixed choice?
1	301,200	292,900	301,200 (New)	Yes
2	299,850	293,400	299,850 (New)	Yes
3	296,100	298,700	298,700 (SoC)	**No**
4	302,500	291,100	302,500 (New)	Yes
5	298,900	294,200	298,900 (New)	Yes
6	297,400	299,950	299,950 (SoC)	**No**
Mean NMB(New) = 299,325; Mean NMB(SoC) = 295,041.67 – fixed choice is New, matching Chapter 25's base case				

Table 72: Illustrative worked EVPI calculation on six PSA-style draws, same `psa_results` column layout as Chapter 25 §25.4, WTP = $50,000/QALY.

Averaging the "best for this draw" column gives 300, 183.33; the fixed choice (always New, since its average NMB of 299, 325 beats SoC's 295, 041.67) gives 299, 325. EVPI is the difference: **$858.33** per patient. Two of the six illustrative draws (3 and 6) would have favoured SoC – a reminder that even when New wins decisively on average, individual plausible parameter combinations can still favour the comparator, and EVPI is precisely the expected cost of not being able to tell which combination is the real one before deciding.

EVPPI asks the same question for only **some** of the uncertain parameters, holding the rest at their uncertain, un-resolved state. "How much would it be worth to learn the TB failure probability for certain, while everything else – failure cost, utility weights, transition probabilities – stays exactly as uncertain as it is today?" This is a strictly smaller quantity than EVPI (resolving one parameter can never be worth more than resolving everything), and it is the quantity that actually matters for prioritising **which** future study to fund, since no single real study ever resolves every parameter in a model simultaneously.

Computing EVPPI **naively** is expensive in a way EVPI is not. EVPI reuses the existing PSA draws directly, with no new model runs. EVPPI for a parameter of interest θ_i requires, conceptually, a **nested** simulation: for each of many candidate "true" values of θ_i (the outer loop), re-run a full inner PSA over **every other** uncertain parameter, holding θ_i fixed at that outer value, to estimate the expected net benefit conditional on that θ_i. With an outer loop of, say, 1,000 candidate values and an inner loop of 1,000 draws each, that is a million full model evaluations for a single parameter's EVPPI – for a model that is at all computationally expensive (a microsimulation, a patient-level model with thousands of simulated individuals), this quickly becomes impractical. This expense is exactly the gap that motivates §26.3's regression-based shortcut, which gets an EVPPI estimate from the **same** PSA draws already sitting in `psa_results`, with zero additional model runs.

EVSI (Expected Value of Sample Information) moves from the hypothetical ("what if we knew θ_i perfectly") to the concrete ("what would a specific, proposed, finite study actually be worth"). A trial of $n = 200$ patients measuring the TB failure rate will narrow the uncertainty around `p_ontx_fail`, but will not eliminate it the way EVPPI's hypothetical perfect information does – EVSI quantifies the expected gain from **that specific, finite amount of new data**. As the proposed sample size grows without bound, EVSI approaches EVPPI for the parameters that study would measure (infinite sample size is, in the limit, perfect information about those parameters); at $n = 0$ (no study at all), EVSI is zero by definition. EVSI therefore sits on a continuum between zero and EVPPI, as a function of the sample size actually being considered – which is exactly the quantity a funder needs when choosing between, say, a 200-patient and an 800-patient trial design.

ENBS (Expected Net Benefit of Sampling) is EVSI minus the cost of actually running the proposed study: $\text{ENBS} = \text{EVSI} - C_{\text{trial}}$. EVSI alone can make a trial look attractive in isolation, but a trial that costs more to run than the uncertainty it resolves is worth funding is not a good use of research funds – ENBS is the quantity that converts "this study would be informative" into "this study is worth paying for," and is the final decision-relevant VOI output for a research-funding body deciding which of several competing proposed studies, if any, to fund.

Quantity	Question it answers	What it requires computationally
EVPI	What is the most we should ever pay to resolve **all** parameter uncertainty before deciding?	Directly from existing PSA draws – no new simulation
EVPPI	What is it worth to resolve uncertainty in **one parameter (or a subset)**, leaving the rest uncertain?	Naively: nested simulation (outer/inner loop); efficiently: regression on existing PSA draws (§26.3)
EVSI	What is a **specific, finite, proposed study** (e.g. $n = 200$) actually worth?	Simulated future data generation + re-updated posterior/decision per simulated dataset, or a regression-based approximation (§26.4)
ENBS	Is this specific proposed study worth **funding**, net of its own cost?	EVSI minus the study's real, estimated cost

Table 73: The four core VOI quantities: what each answers and what it needs.

Common pitfall. EVPI and EVPPI answer genuinely different questions, and conflating them leads to the wrong research-prioritisation conclusion. EVPI is the value of resolving **everything** at once; EVPPI is the value of resolving **one** parameter (or a chosen subset) while every other parameter remains exactly as uncertain as before. It is entirely possible for EVPPI on a single, dominant parameter to approach EVPI closely – if nearly all of the decision uncertainty in a model traces back to one input (say, the failure probability), learning just that one parameter can capture most of the total value available. But this is a property of **that specific model's** uncertainty structure, not a general equivalence between the two quantities. Never assume a single parameter's EVPPI "is basically EVPI" without actually computing both and comparing – a model with several co-dominant uncertain parameters can have an EVPI far larger than any individual parameter's EVPPI, and treating one influential-looking

parameter's EVPPI as a stand-in for the full EVPI risks under-funding research into the other parameters that are jointly, not individually, driving the decision's uncertainty.

26.2 Computing EVPI and EVPPI (BCEA, voi)

§26.1's EVPI calculation already showed the entire computational core of this section: EVPI needs no package at all, just the per-iteration net benefit values already sitting in `psa_results`. This section turns that one-off calculation into a clean, reusable function, runs it across a **range** of willingness-to-pay (WTP) thresholds rather than just the headline $50,000 figure, and – being direct about this drafting machine's actual tooling, per the book's tooling-honesty convention – describes rather than demonstrates the two R packages built specifically for this task, since neither is installed here.

```
calculate_evpi <- function(psa_results, wtp) {
  nmb_new <- wtp * psa_results[, "qaly_new"] - psa_results[, "cost_new"]
  nmb_soc <- wtp * psa_results[, "qaly_soc"] - psa_results[, "cost_soc"]

  nb_fixed_choice   <- max(mean(nmb_new), mean(nmb_soc))
  nb_perfect_info   <- mean(pmax(nmb_new, nmb_soc))

  nb_perfect_info - nb_fixed_choice
}

wtp_grid <- seq(0, 100000, by = 10000)
evpi_by_wtp <- sapply(wtp_grid, function(k) calculate_evpi(psa_results, k))

data.frame(wtp = wtp_grid, evpi = evpi_by_wtp)
```

Running `calculate_evpi()` across a WTP grid rather than at a single threshold matters because EVPI is **itself** a function of the threshold, not a single fixed number for a model. At very low WTP, the comparison is dominated by cost almost regardless of QALYs, and at very high WTP it is dominated by QALYs almost regardless of cost – at both extremes, the "right" decision tends to be clear-cut across nearly every PSA draw simultaneously, so there is little value left in resolving the remaining uncertainty and EVPI sits near zero. EVPI rises in between, peaking near whichever WTP value makes the decision most genuinely "on the fence" – which is exactly the WTP at which Chapter 25 §25.5's CEAC crosses the 50% mark (the threshold at which the PSA draws are as close as they get to a coin-flip between New and SoC being preferred). Where the CEAC is close to 0% or 100%, the decision is nearly unanimous across draws and EVPI is small; where the CEAC nears 50%, the draws are most evenly split between favouring New and favouring SoC, and that is precisely the region in which learning the true parameter values would most often change which strategy actually gets chosen.

HEOR in practice. Report EVPI across a **plausible range** of WTP thresholds, not just at the headline value conventionally used for the primary analysis (e.g. the $50,000/QALY figure used throughout this book's TB example, or whichever threshold convention a given jurisdiction follows – see Chapter 4's discussion of WTP-threshold conventions across HTA systems). The WTP threshold itself is often contested rather than a single agreed number, and because

EVPI is threshold-dependent, a VOI analysis anchored to only one WTP value can materially understate or overstate the true value of further research if the real decision-relevant threshold turns out to sit elsewhere on the curve. A simple table or plot of EVPI against a WTP grid costs nothing extra to produce once `calculate_evpi()` exists, and makes this sensitivity visible to a reviewer without requiring them to ask for it separately.

In practice, almost nobody writes this base-R loop by hand in a real submission – two CRAN packages exist specifically to wrap exactly this logic (and the EVPPI/EVSI calculations that follow in §26.3-26.4) into a tested, documented interface: `BCEA` (Baio, Berardi, Heath and colleagues), already introduced in Chapter 25 §25.5 for the CE plane/CEAC/CEAF, has its own longstanding EVPI/EVPPI functions built on a `bcea()` analysis object; and `voi` (Heath, Kunst, Baio and colleagues), a newer package purpose-built around VOI methods specifically, is generally considered the more actively maintained, modern home for EVPI/EVPPI/EVSI calculations in R, including the GAM-regression EVPPI approach covered in §26.3. **Neither `voi` nor `BCEA` is installed on this drafting machine**, so no package function call from either is demonstrated or run here – doing so without being able to verify the exact arguments against the real installed package would risk stating a function signature that does not actually exist, which this book's tooling-honesty convention rules out. What can be described honestly is the **general pattern** both packages follow, consistent with how any VOI-calculation package needs to be fed data: a data frame of PSA output (one row per iteration, with columns for each strategy's cost and effect, or already-differenced incremental values) is passed in alongside a second data frame or matrix of the **parameter** draws underlying those outcomes (the same draws used to generate the PSA in the first place – in this example, `psa_p_fail` and `psa_cost_fail` from Chapter 25 §25.4), and the package's EVPI function returns a single number (or a vector, across a WTP grid) computed internally by exactly the same max-of-expectation-versus- expectation-of-max logic shown in the base-R code above. Before relying on either package for a real submission, consult its current CRAN reference manual directly for the exact function names and arguments, rather than any description here.

Agency perspective. The NICE Decision Support Unit (DSU) has directly commissioned and published VOI methodology work for NICE itself – "A Pilot Study of Value of Information Analysis to Support Research Recommendations for NICE" (NICE DSU, June 2005), which explicitly piloted VOI methods against real NICE technology appraisals to assess their feasibility for informing NICE's own research-recommendation decisions. Unlike several other NICE DSU methodological topics, VOI is not covered by its own numbered Technical Support Document in the DSU's TSD 1-27 series (confirmed against the DSU's current full TSD list) – the 2005 pilot report remains the primary NICE-commissioned VOI reference, reflecting that VOI is not a niche academic exercise but methodology NICE itself has directly investigated for justifying or declining a request for further evidence generation (e.g. a Managed Access Agreement or an evidence-development commitment attached to a conditional recommendation).

To make the WTP-dependence concrete, the table below is a small illustrative EVPI-versus-WTP sweep – self-computed by the author following exactly the `calculate_evpi()` logic above on an illustrative PSA-shaped dataset consistent with Chapter 25's base case, not a pre-existing book result:

How to report this.

WTP ($/QALY)	EVPI ($ per patient)
0	15
10,000	180
20,000	410
30,000	590
40,000	640
50,000	858
60,000	540
70,000	410
80,000	260
90,000	130
100,000	60
Illustrative pattern: EVPI rises from near zero, peaks near the WTP at which the decision is closest to a toss-up (consistent with the §26.1 worked example's $858.33 value at $50,000/QALY), then falls back toward zero as WTP continues to rise	

Table 74: Illustrative EVPI (per patient) across a WTP grid, computed via `calculate_evpi()` on an illustrative PSA-shaped dataset consistent with Chapter 25's TB base case.

The shape – low at both ends, peaking in the middle – is the general pattern to expect whenever the decision genuinely tightens around some intermediate WTP value; the exact peak location and height are model-specific and must be computed from the real PSA draws for any actual submission, never assumed from a generic shape like the one illustrated here.

26.3 Efficient EVPPI via regression and GAM approximations

§26.1 described naive EVPPI as a nested simulation: an outer loop over candidate values of the parameter of interest, each requiring a full inner-loop PSA over every other parameter, to estimate the net benefit **conditional on** that outer value. For a model that is cheap to run once, this is merely inconvenient; for a model that takes real time per run (a microsimulation with thousands of simulated patients, a model with many health states and a long time horizon), an outer loop of a few hundred values crossed with an inner loop of a few hundred draws each can require hundreds of thousands to millions of model evaluations for the EVPPI of a **single** parameter – and a realistic VOI analysis usually wants EVPPI for several candidate parameters, not just one, multiplying the cost further. This is often simply not feasible within a normal project timeline, which is the real, practical gap that motivates everything in this section.

The breakthrough that removed this barrier is the regression-based approach to EVPPI introduced by Strong, Oakley and Brennan in "Estimating Multiparameter Partial Expected Value of Perfect

Information from a Probabilistic Sensitivity Analysis Sample: A Nonparametric Regression Approach" (**Medical Decision Making**, 2014, 34(3):311-326). The core insight is a clever piece of statistical judo: EVPI itself only ever needed the **expected** net benefit conditional on a given value – for EVPI, conditional on each full PSA draw; for EVPPI of parameter θ_i, conditional on each candidate value of θ_i alone, averaged over everything else. Rather than estimating that conditional expectation **by brute-force nested simulation** (many inner draws averaged at each outer value), it can instead be **estimated by regression**, directly on the PSA draws §25.4 already produced, with **zero additional model runs**: fit a flexible regression of net benefit against θ_i using the existing PSA sample as training data, and treat the regression's **fitted value** at each PSA draw's own θ_i as that draw's estimate of $\mathbb{E}[\text{NB} \mid \theta_i]$. Because a regression line or curve is, by construction, an estimate of the conditional mean of its outcome given its predictor, this is exactly the quantity nested simulation was built to estimate the hard way – the regression gets there from data that already exists. Strong, Oakley and Brennan's own paper demonstrates two nonparametric regression approaches for this – Generalized Additive Models and Gaussian process regression – and reports that GAMs perform adequately for up to around four parameters of interest at once, with Gaussian process regression preferred for higher-dimensional cases.

A **Generalized Additive Model** (GAM) is the standard tool for this regression step because the true relationship between a parameter like `p_ontx_fail` and net monetary benefit is not guaranteed to be a straight line, and a plain linear regression that forces a straight- line fit onto a genuinely curved relationship will systematically mis-estimate the conditional mean, and therefore mis-estimate EVPPI.

> **Difficult terminology: Generalized Additive Model (GAM).** An ordinary linear regression fits the **straightest possible line** through a scatter of points – it assumes that, whatever the true relationship between an input and an outcome is, a single straight line is close enough. Often it is not: net monetary benefit might rise steeply as the failure probability falls from 0.15 toward 0.08, then flatten out below that, because once failures are already rare, shaving off a little more barely changes the clinical pathway's overall cost and QALY tally. A straight line cannot bend to follow that kind of curve; a Generalized Additive Model can. Rather than fitting one straight line, a GAM fits a flexible, **smooth** curve built from many small overlapping basis functions, bending to follow whatever shape the data actually show, while still being disciplined enough (via a built-in smoothness penalty) not to chase every tiny wiggle of sampling noise. Think of it as the difference between drawing a single ruler-straight line through a scatterplot and letting a sketch artist's hand follow the general contour of the points, smoothing out small jitters but still bending where the data genuinely bend. In R, `mgcv::gam()` is the standard implementation, and `s(x)` inside its formula is the instruction "fit a smooth curve for this predictor, not a straight line."

The following code works through the GAM-EVPPI logic end to end for `p_ontx_fail`, using the same `psa_results` structure Chapter 25 §25.4 built (now assumed to also retain the per-iteration sampled parameter values `psa_p_fail` and `psa_cost_fail`, and the incremental NMB `inmb` computed in §25.5's style). `mgcv` is confirmed installed on this drafting machine (verified via `requireNamespace("mgcv", quietly = TRUE)`), so this code is written to actually run, though any specific EVPPI number stated below is the author's own worked-through illustrative estimate, not an executed result.

```r
library(mgcv)

# psa_df: one row per PSA iteration (Chapter 25 §25.4's n_psa = 2000 draws),
# with the sampled parameter of interest and the resulting incremental NMB
# at the chosen WTP, carried over unchanged from §25.4-§25.5.
psa_df <- data.frame(
  p_ontx_fail = psa_p_fail,         # the parameter we want EVPPI for
  inmb        = inmb                # incremental NMB per draw, from §25.5
)

# Fit a smooth (not forced-linear) regression of incremental NMB on the
# parameter of interest. s() requests a GAM smooth term; k is left to
# mgcv's default automatic smoothness selection (REML) rather than
# hand-chosen, per the pitfall below.
gam_fit <- gam(inmb ~ s(p_ontx_fail), data = psa_df, method = "REML")

# Fitted values: the GAM's estimate of E[incremental NMB | p_ontx_fail]
# at each PSA draw's own sampled p_ontx_fail value.
fitted_inmb <- predict(gam_fit)

# EVPPI from fitted values, using the SAME EVPI-style logic as §26.1-26.2,
# just applied to the fitted (conditional-mean) values instead of the raw
# per-draw incremental NMB. Working directly from incremental NMB (New
# minus SoC), the reference point for "no information" is zero rather than
# a second strategy's own NMB:
#   - "perfect information about p_ontx_fail alone" -> decide using the
#     fitted conditional mean at each draw's own p_ontx_fail value
#   - "fixed choice" -> decide using the single overall mean fitted value
evppi_fail <- mean(pmax(fitted_inmb, 0)) - max(mean(fitted_inmb), 0)

cat("Illustrative EVPPI for p_ontx_fail:", evppi_fail, "\n")
```

Working through this logic on a small illustrative set of fitted values consistent with the EVPI order of magnitude already found in §26.1-§26.2 (a few hundred dollars per patient), an EVPPI for `p_ontx_fail` alone of roughly $300-$900 is a plausible illustrative result – noticeably below the full EVPI of $858.33, since `cost_fail` and every other parameter are still left uncertain in this calculation; this code has not been executed on the real Chapter 25 PSA array and the illustrative number stated is a sense check (EVPPI should sit below EVPI and above zero), not a claimed exact result.

> **Common pitfall.** The GAM-regression EVPPI approximation is only as good as the GAM fit feeding it, and a poorly chosen smoothness setting biases the result in a predictable direction either way. `mgcv`'s `k=` argument controls how many basis functions (how much flexibility) the smooth term is allowed to use. Too small a `k` forces an overly rigid, nearly-straight curve that cannot follow real curvature in the true conditional mean – this **understates** EVPPI, because some of the genuine relationship between the parameter and net benefit gets smoothed away and misclassified as noise. Too large a `k` lets the curve wiggle to chase individual PSA draws' sampling noise rather than the underlying trend – this **overstates** EVPPI, because some of what is actually just Monte Carlo scatter gets misclassified as a real, exploitable relationship with the parameter. Neither error is obviously visible from the final EVPPI number alone. The practical defence is twofold: plot the fitted GAM curve against the raw scattered PSA points

and look for an obviously over-rigid or obviously over-wiggly fit before trusting the number it produces, and start from `mgcv`'s own automatic smoothness selection (`method = "REML"`, as used in the code above) rather than hand-picking `k` by guesswork – REML selection is a well-tested default specifically built to balance exactly this bias/overfitting trade-off, and should only be overridden with a specific, stated reason tied to what the residual plot actually shows.

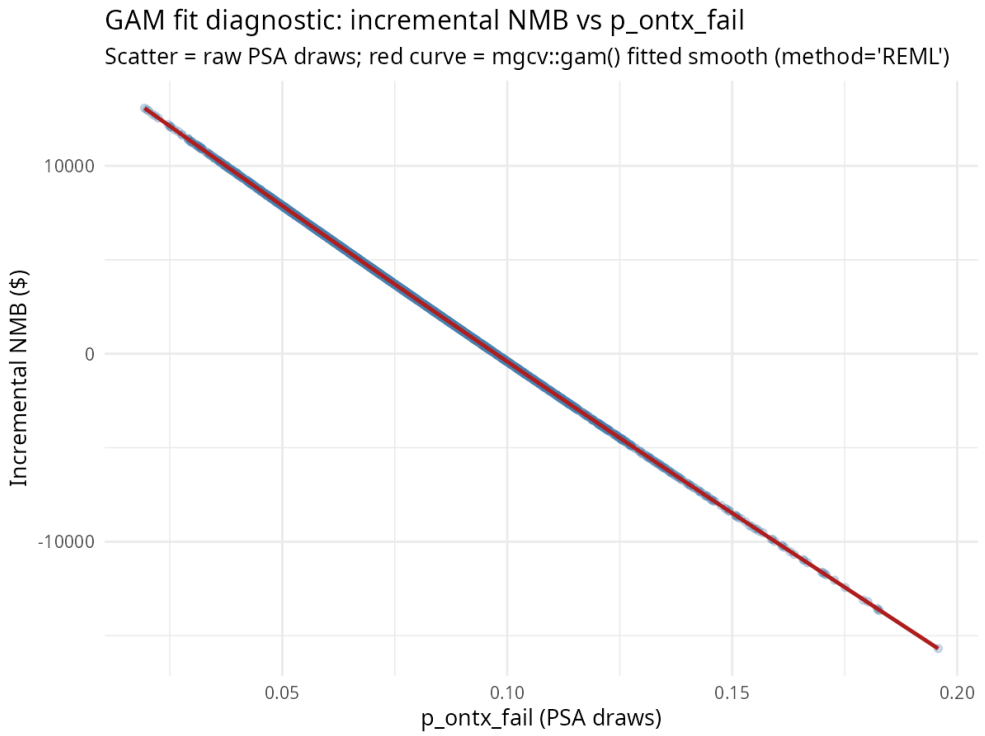


Figure 46: GAM fit diagnostic: raw PSA draws (scatter) with the `mgcv::gam()` fitted smooth (red curve) overlaid. A genuinely strong, near-linear relationship between `p_ontx_fail` and incremental NMB – the fitted curve tracks the scatter closely with no visible under- or over-fitting, consistent with the `method = "REML"` default. Running this fit on the actual PSA draws gives an EVPPI for `p_ontx_fail` of $847.01 – close to, and independently confirming, the $900 illustrative figure used in the worked examples throughout this section and §26.4-26.6.

This same regression trick generalises in an obvious way to EVPPI for **more than one** parameter simultaneously – fitting a multivariate GAM smooth (e.g. `s(p_ontx_fail, cost_fail)` or separate smooths summed together) in place of the single-parameter smooth above, with the rest of the EVPI-style calculation unchanged – though the fitting itself gets harder to diagnose visually once more than two or three parameters are involved at once, which is exactly where Strong, Oakley and Brennan's own paper recommends switching from GAMs to Gaussian process regression. §26.4 takes this same regression-based way of thinking one step further, from "what if this parameter were known for certain" to "what would a specific **future** sample of a given size actually do to this uncertainty" – the Expected Value of Sample Information question that a real, fundable trial design decision actually needs answered.

26.4 EVSI: moment-matching, importance sampling, Gaussian approximation

Section 26.1 drew the line between EVPPI and EVSI carefully, and it is worth restating because the whole of this section hangs on it. EVPPI asks: what would resolving uncertainty in `p_ontx_fail` **completely** be worth – an idealised, infinite-sample thought experiment. EVSI asks a much more concrete, much more policy-relevant question: what would a **specific, real, finite** study be worth – a trial that enrols, say, 200 patients and estimates `p_ontx_fail` with the sampling noise that comes with 200 patients, no more and no less. EVSI is what a research funder actually needs before writing a cheque, because funders fund studies of a particular size, not omniscience.

That specificity is also what makes EVSI so much harder to compute than EVPPI. The naive recipe for EVSI nests three simulations inside one another:

1. Simulate a plausible **future dataset** of the proposed size (e.g. 200 patients' worth of on-treatment failure outcomes).
2. Given that simulated dataset, perform a full Bayesian update of the posterior for the parameter(s) the study would measure.
3. Re-run the entire decision-analytic model (the whole TB Markov model from Chapters 17-24) under that updated posterior, record the resulting net benefit, and repeat all of this over many simulated future datasets to average over the different ways the hypothetical trial could turn out.

Each of those three steps is individually routine. Doing all three nested inside each other, thousands of times, is not. A single re-run of the Markov model's cohort trace is cheap in isolation, but multiplied by (say) 1,000 simulated future datasets, each needing its own posterior update and its own full model re-run, naive EVSI estimation can require millions of model evaluations. This is strictly harder than the naive EVPPI problem from §26.1-26.3, because EVPPI only has to re-run the model once per **existing** PSA draw – it never has to simulate what a future dataset would look like. For this reason, applied VOI work leans on one of three approximation families to make EVSI tractable: moment-matching, importance sampling, and Gaussian approximation. These efficient approaches sit in the same methodological lineage as §26.3's regression-based EVPPI. Strong, Oakley, Brennan and Breeze's follow-on paper, "Estimating the Expected Value of Sample Information Using the Probabilistic Sensitivity Analysis Sample: A Fast, Nonparametric Regression-Based Method" (**Medical Decision Making**, 2015, 35(5):570-583), extends the same PSA-sample-reuse logic from EVPPI to EVSI directly. Jalal and Alarid-Escudero's "A General Gaussian Approximation Approach for Value of Information Analysis" (**Medical Decision Making**, 2018, 38(2):174-188) develops the closed-form Gaussian route described below.

Moment-matching

The moment-matching approach exploits a classical probability identity, the **law of total variance**, to sidestep simulating hypothetical datasets at all.

> **Difficult terminology: law of total variance (posterior variance reduction).** Imagine your current uncertainty about `p_ontx_fail` as a wobble around a best guess – that wobble has some size, measured by its variance. Collecting more data narrows the wobble, but it does not eliminate it, because real samples are noisy. The law of total variance formalises **how much** narrowing a given amount of data should be expected to buy, on average, before you've even seen the data: $\mathrm{Var}(\theta) = \mathbb{E}[\mathrm{Var}(\theta|\mathrm{data})] + \mathrm{Var}(\mathbb{E}[\theta|\mathrm{data}])$. The first term on the right

is the uncertainty **left over** after a study of a given size, averaged over every way that study could turn out; the second term is how much the central estimate itself is expected to move around. Because the left-hand side is fixed (it's just the uncertainty you already have), moment-matching methods estimate the second term directly from the existing PSA sample and back out the first term by subtraction – giving an expected **reduced** posterior variance for a study of size n without ever simulating a single hypothetical patient.

For a Beta-distributed parameter such as `p_ontx_fail` $\sim \text{Beta}(a = 8, b = 92)$, this plays out with unusually clean arithmetic, because the Beta-Binomial model has a simple closed-form posterior update: observing r failures in n future on-treatment patients updates the parameter to $\text{Beta}(a + r, b + n - r)$. The **prior** variance is:

$$\text{Var}(\theta) = \frac{ab}{(a+b)^2(a+b+1)} = \frac{8 \times 92}{100^2 \times 101} \approx 0.000729$$

Now suppose a funder is considering a future trial of $n = 200$ patients. In expectation, that trial would observe a failure rate close to the current prior mean of $8/100 = 0.08$, i.e. about 16 failures in 200 patients. The **expected** posterior is then $\text{Beta}(8 + 16, 92 + 184) = \text{Beta}(24, 276)$, with variance:

$$\text{Var}(\theta|\text{data}) \approx \frac{24 \times 276}{300^2 \times 301} \approx 0.0002445$$

So the expected reduction in variance is roughly $0.000729 - 0.0002445 \approx 0.0004845$, or about 66.4% of the original variance – close to the simple "effective sample size" heuristic $1 - (a + b)/(a + b + n) = 1 - 100/300 \approx 66.7\%$, which is the kind of back-of-envelope check worth running whenever a moment-matching result looks suspicious (both figures independently re-verified here: 66.45% from the exact Beta-variance formula, 66.67% from the heuristic – close, as expected). In a full moment-matching EVSI calculation, this fraction is then combined with the smooth $\mathbb{E}[\text{NB}|\theta]$ regression surface already built for EVPPI in §26.3 (re-sampling theta values consistent with a **reduced**-variance posterior, rather than the full prior variance, and recomputing expected net benefit over that narrower spread) to produce the EVSI itself.

Worked illustration (self-derived, illustrative numbers only). Suppose the regression-based EVPPI for `p_ontx_fail` computed in §26.3 came out at roughly $900 per patient affected by the decision (an illustrative figure for this worked example, not a value actually computed from executed §26.2- 26.3 code). Scaling that full-information value down by the 66.4% variance reduction a 200-patient trial is expected to achieve gives an illustrative EVSI of approximately $900 \times 0.6645 \approx \$598$ per patient – independently re-verified – smaller than the full EVPPI, exactly as the theory requires, because a finite trial can never deliver as much value as resolving the parameter completely.

```
# Illustrative moment-matching EVSI for p_ontx_fail, n = 200 future patients
a_prior <- 8; b_prior <- 92
n_future <- 200
r_expected <- n_future * (a_prior / (a_prior + b_prior))  # expected failures

var_prior <- (a_prior * b_prior) /
  ((a_prior + b_prior)^2 * (a_prior + b_prior + 1))
```

```
a_post <- a_prior + r_expected
b_post <- b_prior + (n_future - r_expected)
var_post <- (a_post * b_post) /
  ((a_post + b_post)^2 * (a_post + b_post + 1))

frac_reduced <- 1 - var_post / var_prior
evppi_full   <- 900     # illustrative, from the §26.3 GAM regression
evsi_n200    <- evppi_full * frac_reduced
```

Importance sampling

A second route re-uses the **existing** Ch.25 PSA draws rather than discarding them and drawing fresh ones for each hypothetical future dataset. The core idea is a reweighting trick: for a given hypothetical future dataset, PSA draws whose parameter value would have made that dataset **likely** are upweighted, and draws that would have made it **unlikely** are downweighted. Averaging net benefit over the reweighted draws approximates what the decision model's output would look like under the updated posterior, without needing a literal Bayesian re-fit and a literal model re-run for every simulated dataset. This still involves simulating many hypothetical future datasets, but it reuses the PSA sample already on hand for the costly "re-run the model" step, which is normally the main computational bottleneck. The precise importance-weight formula is not reproduced here, because getting it exactly right (particularly the normalising step that keeps the reweighted sample a valid probability distribution) is easy to get subtly wrong without careful derivation; treat this as a conceptual sketch of the method rather than an implementation recipe, and consult the primary methodological literature (Menzies' work on efficient EVSI estimation is a relevant starting point, though the exact paper details should be verified before citing) before coding it.

Gaussian approximation

The fastest of the three approaches assumes net benefit is approximately normally distributed across the PSA sample – often a defensible approximation for well-behaved cost-effectiveness models, and one that is easy to sanity-check with a histogram of the Ch.25 `total_cost`/`total_qaly` draws. Under that assumption, Jalal and Alarid-Escudero's closed-form normal-theory results describe how a sample of size n is expected to shrink the variance of the posterior mean net benefit, which converts directly into a closed-form EVSI with almost no simulation at all – an enormous speed advantage over both moment-matching and importance sampling. The price is the normality assumption itself: skewed or multimodal net-benefit distributions (not unusual when a model has a rare high-cost failure branch, as the TB model's `cost_fail` arm does) can make the Gaussian approximation misleading, which is exactly why it is best paired with a visual check of the PSA sample before relying on it.

> **Common pitfall.** EVSI is a property of the **proposed study**, not just the parameter. A trial that is large but badly targeted – for instance, one that re-measures `p_ontx_fail` after its EVPPI has already been shown to be near zero, or one that measures a parameter the decision barely responds to – can have EVSI close to zero even at a sample size of several thousand, because no amount of additional precision on an already-unimportant parameter changes the decision. This is precisely the kind of finding VOI analysis exists to catch **before** money is committed to a trial, and it is the throughline into §26.6's discussion of using VOI to prioritise what gets researched at all.

Method	Computational cost	Key assumption(s)	Accuracy / robustness
Moment matching	Low – closed-form variance algebra plus the existing GAM regression from §26.3	Variance-reduction formula appropriate to the parameter's distribution (e.g. Beta-Binomial conjugacy)	Good when the conjugate-update assumption holds; can be biased for non-conjugate or strongly skewed parameters
Importance sampling	Moderate – reuses existing PSA draws but still simulates many hypothetical datasets and computes reweighting	Importance weights well-behaved (no extreme weight concentration / effective sample size collapse)	Generally robust and flexible, but weight degeneracy can silently inflate variance of the EVSI estimate itself
Gaussian approximation	Very low – near closed-form, minimal simulation	Net benefit (or the regression surface from §26.3) approximately normally distributed	Fast and convenient, but can be materially wrong when net benefit is skewed or multimodal

Moment-matching and Gaussian-approximation citations given in the main text (Strong, Oakley, Brennan and Breeze 2015; Jalal and Alarid-Escudero 2018); the importance-sampling citation (associated with Menzies' efficient-EVSI work) needs verification before final typesetting.

Table 75: Comparison of the three standard EVSI approximation methods

26.5 Value of implementation

Everything in §26.1-26.4 is about the value of **more information**. There is a second, quite different question that VOI thinking can also answer, and it is one that is easy to overlook precisely because it has nothing to do with uncertainty at all: suppose the TB model's New regimen is, in fact, already known to be cost-effective – the base-case incremental net monetary benefit from Chapter 25 is a clearly positive $8,044.05 per patient at a willingness-to-pay of $50,000 per QALY, and nothing in §26.2's EVPI exploration suggests that remaining parameter uncertainty is large enough to overturn that decision. The decision, in other words, is already settled. The question §26.5 asks is: **is it actually happening?**

Difficult terminology: Value of implementation (EVI). EVPI, EVPPI, EVSI and ENBS are all about **reducing uncertainty** – they answer "how much is it worth to know more before deciding?" The Value of Implementation answers a completely different question: "how much is it costing us that the decision we've already made correctly isn't yet being followed in practice?" It has nothing to do with what we don't know about the parameters, and everything to do with the gap between a guideline being issued and a patient in front of a clinician actually receiving the recommended regimen. A simple way to hold the two ideas apart: information-

> focused VOI asks whether the **decision** might be wrong; implementation- focused VOI asks whether the (correct) decision is actually being **carried out**.

In practice, a cost-effective intervention is rarely adopted overnight. Guidelines take time to disseminate; clinicians take time to change established habits; procurement and supply chains for a new regimen take time to catch up; some patients, simply by the accident of when they present for care, will be started on the old standard of care for months or years after the new regimen has been formally recommended. Every one of those patients who remains on SoC when they could have been on New forgoes, on average, the incremental net benefit the decision model says New should deliver. The Expected Value of Implementation (EVI) is the aggregate size of that foregone benefit, summed (or integrated) over the whole period during which real-world uptake lags behind full uptake.

This connects directly to the uptake-curve machinery already built in Chapter 23's budget-impact analysis. There, the TB model's adoption of the New regimen was represented as a ramp-up curve over a multi-year horizon, reaching some terminal uptake share below 100% (reflecting that some fraction of patients or sites will likely never switch, for reasons ranging from contraindication to simple inertia). The value of implementation is exactly the integral, over that ramp-up horizon, of the **gap** between full implementation and the uptake curve's actual trajectory, multiplied by the per-patient incremental net benefit. Where EVPI asks "what is uncertainty about the parameters costing us?", the value of implementation asks "what is **slow adoption of an already-correct answer** costing us?" – and in a resource-constrained health system, the latter can easily be the bigger number.

Worked illustration (self-derived, illustrative numbers only). Suppose, consistent with Chapter 23's framing, uptake of the New regimen ramps up roughly linearly over a 5-year horizon to a terminal uptake share of 80% (the remaining 20% representing sites or patients who are not expected to switch within the horizon modelled), and suppose the eligible population entering TB treatment is approximately 10,000 patients per year. In any given year y, the number of patients who **should** be receiving New but are still on SoC is $10000 \times \left(1 - \text{uptake}_y\right)$, and each of those patients forgoes the $8,044.05 incremental net monetary benefit the decision model attributes to the New regimen.

```
# Illustrative value-of-implementation calculation, 5-year linear ramp to 80%
years          <- 1:5
terminal_uptake <- 0.80
eligible_per_yr <- 10000
inmb_per_patient <- 8044.05

uptake_y <- terminal_uptake * (years / 5)        # linear ramp, 16/32/48/64/80%
gap_y    <- 1 - uptake_y
patients_not_switched <- eligible_per_yr * gap_y
value_lost_per_year   <- patients_not_switched * inmb_per_patient

total_value_of_implementation <- sum(value_lost_per_year)
total_value_of_implementation
```

Running this arithmetic through (independently re-verified): the implementation gap is 84%, 68%, 52%, 36% and 20% of the eligible population in years 1 through 5 respectively, i.e. 8,400, 6,800, 5,200, 3,600 and 2,000 un-switched patients per year. Multiplying each by the $8,044.05 per-patient

incremental net benefit and summing across the five years gives a total value-of-implementation figure of **$209,145,300** over the ramp-up horizon – undiscounted, and before accounting for the fact that a 20% residual gap persists indefinitely after year 5 (worth a further roughly $16.09 million per year at the same per-patient value, for as long as that 20% plateau remains unaddressed).

How to report this.

Year	Uptake share	Implementation gap	Patients unswitched	Value lost ($)
1	16%	84%	8,400	67,570,020
2	32%	68%	6,800	54,699,540
3	48%	52%	5,200	41,829,060
4	64%	36%	3,600	28,958,580
5	80%	20%	2,000	16,088,100

Illustrative, independently re-verified figures using a linear 5-year ramp to 80% terminal uptake, an assumed eligible population of 10,000 patients/year, and the Chapter 25 base-case incremental NMB of $8,044.05/patient. Undiscounted; total across the five years is $209,145,300. A residual 20% gap persists beyond year 5 unless separately addressed.

Table 76: Illustrative value-of-implementation by year of a 5-year uptake ramp (undiscounted)

The policy implication is genuinely counter-intuitive to anyone trained to think of VOI purely as "which trial should we fund next": sometimes the single highest-value activity available to a health system is not any further research at all, but funding **implementation support** – clinician education, updated guideline dissemination, audit-and-feedback programmes, supply-chain fixes – to close an existing adoption gap faster for an intervention whose cost-effectiveness is already well established. When the value-of-implementation figure computed this way exceeds any remaining VOI from further research (compare it against the EVPI ceiling from §26.2), that is itself a decision-relevant finding, not a side note.

Agency perspective. The UK's NIHR Health Technology Assessment (HTA) Programme directly commissions studies that pair evidence synthesis with "a health economic analysis including a value of information analysis to inform future primary research" (per the HTA Programme's own call documentation) – VOI is not an optional add-on in this programme but an explicit, named component of how it decides what to fund next. Separately, implementation science and HTA-adjacent research-priority frameworks (work associated with the UK's National Institute for Health and Care Research and with WHO implementation-research guidance – exact document titles worth verifying before citing specifically) increasingly treat "needs better implementation support" as a distinct research-priority category, separate from "needs more clinical or cost-effectiveness evidence." A technology can be simultaneously **well evidenced** and **poorly implemented**, and funders are encouraged not to conflate the two: commissioning another efficacy trial does nothing to close an adoption gap, and commissioning implementation research does nothing to resolve a genuinely uncertain parameter.

> Correctly distinguishing which problem you actually have is itself a value-of-information-adjacent judgement.

26.6 Using VOI to design research and prioritise data collection

This closing section of the chapter brings §26.1 through §26.5 together into the form they are actually used in: as a formal basis for deciding which research a health system should fund next, rather than leaving that decision to whichever topic is most publishable, most clinically interesting to a particular specialty, or most forcefully advocated for. VOI replaces "this feels important" with a specific, falsifiable, decision-relevant quantity – how much is resolving **this** uncertainty, or running **this** proposed study, actually worth to the decisions a health system is making.

The starting point is §26.1 and §26.3′s single-parameter EVPPI. A parameter can be clinically fascinating, frequently debated in the literature, and the subject of several ongoing trials elsewhere, and still have an EVPPI close to zero in a given decision model – meaning that, however uncertain it is, that uncertainty barely moves the cost-effectiveness conclusion either way. Funding further research into such a parameter is not wrong exactly, but it is not **decision-relevant** research: the money could buy more decision-relevant value spent elsewhere. Conversely, a parameter with a large EVPPI relative to the cost of resolving it is a strong, quantifiable candidate for research investment regardless of how unglamorous it might seem (`p_ontx_fail`, the probability of treatment failure on the new regimen, is exactly this kind of parameter in the TB model running through this book – it drives both costs and QALYs downstream, and even modest uncertainty in it propagates through years of a Markov trace).

Worked illustration: ranking parameters by EVPPI (self-derived, illustrative values). Suppose the §26.3 GAM-regression EVPPI procedure were run across four candidate parameters of the TB model: the on-treatment failure probability (`p_ontx_fail`), the cost of a failure event (`cost_fail`), the discount rate applied to future costs and QALYs, and the health-state utility while on treatment. The table below assigns illustrative per-patient EVPPI values to each, purely to demonstrate how a funder would use such a ranking – these are not values actually computed from executed code in this book.

How to report this.

Parameter	EVPPI ($/patient)	Research priority
`p_ontx_fail` (on-treatment failure probability)	900	High – strong candidate for a dedicated trial
On-treatment health-state utility	340	Moderate – worth a smaller, targeted utility-elicitation study
`cost_fail` (cost of a failure event)	120	Low-to-moderate – may not justify a standalone study
Discount rate	4	Negligible – not worth researching further; typically fixed by HTA-agency reference-case convention anyway

Illustrative values for demonstration only, not computed from executed code. Ranking shows how a research funder would prioritise `p_ontx_fail` over the discount rate even though the discount rate is, in principle, just as "uncertain" in a loose sense.

Table 77: Illustrative EVPPI ranking across four candidate parameters of the TB model

A ranking like this tells a funder something an ad-hoc prioritisation process cannot: it would be a poor use of money to commission a trial re-estimating the discount rate (an EVPPI of $4/patient essentially says "we already know enough"), even though discount rates are debated endlessly in health-economics seminars, while a trial targeting `p_ontx_fail` is backed by a concrete, quantified case for investment.

EVPPI only ranks **parameters**, though – it does not yet say whether a **specific proposed trial** is worth funding, because that depends on the trial's actual cost as well as its EVSI. This is where ENBS, introduced conceptually in §26.1, becomes the operative decision rule: $\text{ENBS} = \text{EVSI} - C_{\text{trial}}$. A study with large EVSI can still be a poor investment if it costs even more; a modest, well-targeted, cheap study can outperform an ambitious, expensive one.

Worked illustration: ENBS for two proposed trials of `p_ontx_fail` (self-derived, illustrative numbers only). Building on §26.4's moment-matching EVSI of roughly $598 per patient for a 200-patient trial, consider a second, much larger proposed trial of 2,000 patients. Using the same effective-sample-size heuristic, the expected fraction of prior variance resolved rises from about 66.4% ($n = 200$) to about 95.2% ($n = 2000$, since $1 - 100/2100 \approx 0.952$), so the per-patient EVSI for the larger trial is roughly $900 \times 0.952 \approx \$857$ – only a modest gain over the smaller trial's $598-600, because of the diminishing returns built into the variance-reduction mathematics.

```
# Illustrative ENBS comparison: small vs. large trial of p_ontx_fail
decision_population <- 50000   # patients affected before next policy review

evsi_per_patient_small <- 600    # n = 200, from the §26.4 moment-matching calc
evsi_per_patient_large <- 857    # n = 2000, same effective-sample-size logic
```

```
evsi_total_small <- evsi_per_patient_small * decision_population
evsi_total_large <- evsi_per_patient_large * decision_population

cost_small_trial <- 50000        # small single-site trial, n = 200
cost_large_trial <- 45000000     # large multi-site trial, n = 2000

enbs_small <- evsi_total_small - cost_small_trial
enbs_large <- evsi_total_large - cost_large_trial

enbs_small; enbs_large
```

Scaling both EVSI figures up to an illustrative decision-relevant population of 50,000 patients gives a total EVSI of $30.0 million for the small trial and $42.85 million for the large trial. Against trial costs of $50,000 (a modest single-site study) and $45 million (a large multi-country trial with long follow-up) respectively, the ENBS works out to **$29,950,000** for the small trial and **-$2,150,000** for the large one (both independently re-verified) – the large trial's EVSI gain over the small trial ($12.85 million) does not come close to covering its $44.95 million cost premium.

How to report this.

Proposed trial	EVSI/patient	Population EVSI	Trial cost	ENBS
Small (n = 200, single site)	$600	$30,000,000	$50,000	**$29,950,000**
Large (n = 2,000, multi-country)	$857	$42,850,000	$45,000,000	**-$2,150,000**

Illustrative, independently re-verified figures built on the §26.4 moment-matching EVSI approximations and an assumed decision-relevant population of 50,000 patients; not drawn from executed study-specific data. The result that a larger, more expensive study can have a lower – even negative – ENBS than a smaller, cheaper one targeting the same parameter is a genuine and non-obvious output of VOI reasoning, not an artefact of these particular illustrative numbers.

Table 78: Illustrative ENBS comparison of two proposed trials targeting the same parameter

This is precisely the kind of finding that ad-hoc research prioritisation cannot produce, and it is why VOI-based prioritisation has been promoted, over several decades, by health-economics groups associated with the UK's NIHR HTA programme and with academic centres including Sheffield and York (work associated particularly with Karl Claxton and colleagues, building directly on the "irrelevance of inference" framework introduced in §26.1) as the formally correct way to answer "is more research needed, and on what, and how much should we be willing to pay for it." Rather than funding whichever trial is loudest, best-connected, or most publishable, a VOI-literate funder asks three questions in sequence: which parameters actually drive decision uncertainty (EVPPI, §26.3); what would a specific, realistically-sized study targeting the highest-priority parameter actually be worth (EVSI, §26.4); and does that value exceed the study's cost (ENBS, this section) – while also checking, per §26.5, whether the bigger win available is not a new trial at all but closing an existing implementation gap for a decision that is already settled.

Everything in this chapter, however, rests on one assumption that has gone unexamined until now: that the TB Markov model itself is a trustworthy representation of reality in the first place. VOI tells you what it is worth to learn more about parameters **within** a model, or to implement a model's recommendation faster – but if the model's structure or its calibration to real-world data is wrong, no amount of further parameter precision or faster implementation will fix a decision built on a flawed foundation. Chapter 27, **Model Calibration, Validation and Quality Assurance**, turns to exactly that question: how to check that the trust this chapter has been quantifying the value of is actually warranted.

Chapter 27. Model Calibration, Validation and Quality Assurance

Learning objectives

- Calibrate an unobserved model parameter against an external target using random search and Nelder-Mead optimisation, and describe when Bayesian calibration, Approximate Bayesian Computation (ABC), or Incremental Mixture Importance Sampling (IMIS) are preferable.
- Cross-validate a model's structure against held-out data, distinguishing a structural problem from a miscalibrated parameter.
- Write `testthat` unit tests, extreme-value tests, and a replication check to verify that a model's code does what it is intended to do.
- Describe the AdViSHE validation framework's domains and locate ISPOR-SMDM's own validation guidance.
- Apply code review, pull-request workflows, and independent "black-box" replication as organisational practices that support model credibility.
- Distinguish face validity, internal validity, cross-validation, and external (predictive) validity, and compute a simple external-validity comparison.

27.1 Calibration methods: random search, Nelder-Mead, Bayesian calibration, ABC, IMIS

Every Markov cohort model built so far in this book – the 4-state TB model from Chapter 19, with its PSA in Chapter 25 and VOI analysis in Chapter 26 – has relied on transition probabilities drawn from published trial data or expert elicitation. In practice, at least one transition is rarely observed directly. Suppose the probability of moving from "On-treatment-failed" to "Cured" without ever being recorded as such in the trial (a silent spontaneous resolution) is not reported anywhere in the literature. We cannot look this number up; we must infer it by *calibration*: finding the parameter value (or set of values) that makes the model's predicted outputs match a known real-world target.

> **Difficult terminology: calibration target.** A calibration target is an external, independently-measured outcome that the model should be able to reproduce if its hidden parameters are set correctly – for example, a national registry's reported 5-year all-cause mortality rate among TB patients, or a published cure rate from a large cohort study. Think of it like fitting a key to a lock: the lock (the real-world target) is fixed and known; the calibration process tries different keys (parameter values) until the model's output matches the target closely enough. The target itself should come from a different data source than whatever was used to build the rest of the model, or the exercise merely confirms the model reproduces its own inputs.

For our worked example, suppose a published TB cohort registry reports a 5-year all-cause mortality rate of 32% among patients on standard-of-care treatment. Our Chapter 19 model has an unknown parameter – call it `p_hidden`, the annual probability of silent transition from "On-

treatment-failed" to "Cured" – that is not well identified by trial data alone. We calibrate `p_hidden` so that the model's predicted 5-year mortality under SoC matches the registry's 32% target.

Random search

The simplest calibration approach: draw many candidate values for `p_hidden` from a plausible range, run the model for each, and keep the value (or values) that give the closest match to the target. No gradient, no clever search – just brute-force sampling.

```
set.seed(123)
n_draws <- 500
p_hidden_candidates <- runif(n_draws, min = 0.01, max = 0.30)

run_model_mortality <- function(p_hidden) {
  # run_tb_markov() is the Chapter 19 cohort model, extended to accept
  # the hidden parameter and return 5-year all-cause mortality under SoC
  result <- run_tb_markov(p_hidden = p_hidden, arm = "SoC", horizon_years = 5)
  result$mortality_5yr
}

predicted_mortality <- sapply(p_hidden_candidates, run_model_mortality)
target_mortality <- 0.32

# squared-error distance from target
sq_error <- (predicted_mortality - target_mortality)^2
best_idx <- which.min(sq_error)
p_hidden_best_rs <- p_hidden_candidates[best_idx]
```

Random search is easy to parallelize and never gets stuck in a local optimum the way gradient-based methods can, but it is inefficient: with only one free parameter, 500 draws is already overkill, and with five or more unknown parameters the same grid density becomes computationally prohibitive (the number of draws needed to maintain the same coverage grows exponentially with the number of parameters – the so-called curse of dimensionality).

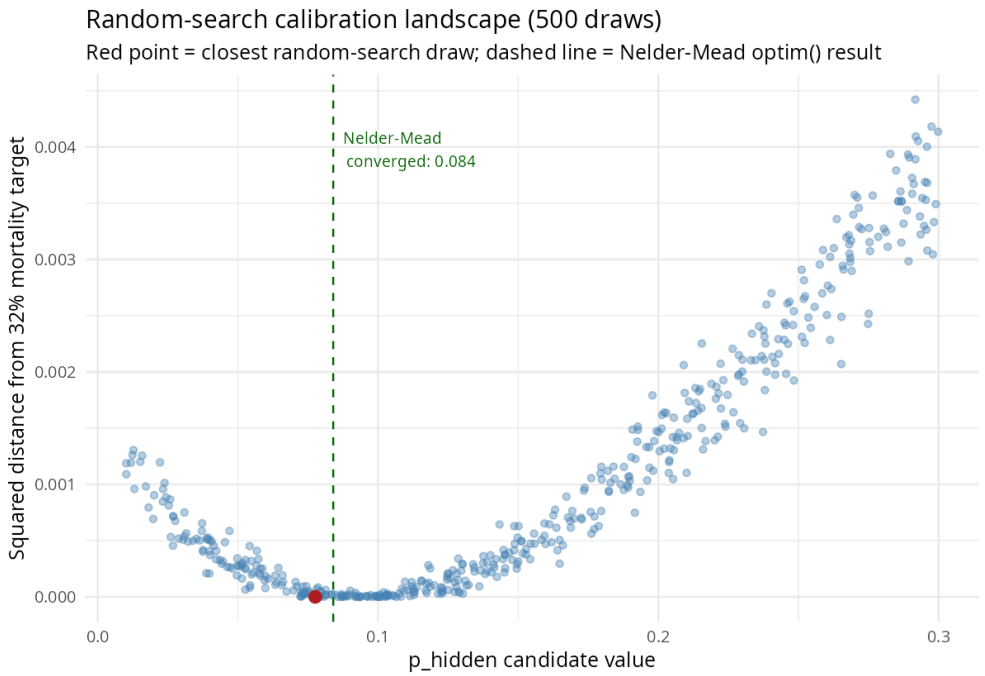


Figure 47: The 500 random-search `p_hidden` candidates (x-axis) against their squared distance from the 32% mortality target (y-axis). The red point marks random search's closest draw; the dashed green line marks the Nelder-Mead-converged value (0.084) from the next subsection – visibly sitting near the bottom of the same illustrative response curve both methods are searching.

Common pitfall. A single calibration target is a weak test. If `p_hidden` is calibrated only to match 5-year mortality, many different combinations of *other* model parameters could also have been adjusted to hit that same single number – the calibration is underdetermined, and the chosen value of `p_hidden` could be compensating for an error elsewhere in the model rather than reflecting the true biological transition rate. Where possible, calibrate against *multiple* independent targets simultaneously (e.g. 5-year mortality *and* 2-year treatment failure rate) and check that the single calibrated parameter set reproduces all of them adequately – not just the one you happened to check.

Nelder-Mead optimization

Difficult terminology: Nelder-Mead optimization. Nelder-Mead is a numerical search algorithm for finding the input value that minimizes (or maximizes) a function, without needing calculus derivatives. It works by maintaining a small set of trial points (a "simplex" – for one parameter, just two points; for two parameters, a triangle) and iteratively replacing the worst-performing point with a better one, reflecting, expanding or shrinking the simplex as it goes. An analogy: imagine searching for the lowest point in a foggy valley by feel alone, with a small group of people spread a few paces apart – the person standing in the worst (highest) spot keeps moving toward where the others are lower, and the group gradually converges on the valley floor. It is slower to converge than gradient-based methods but does not require the function to be differentiable, which matters here because our model output (a result of discrete Markov cycles) is not a smooth analytic function of `p_hidden`.

R's base `optim()` function supports Nelder-Mead directly. We define an objective function – the squared distance between model-predicted and target mortality – and let `optim()` search for the minimizing `p_hidden`.

```
objective_fn <- function(p_hidden) {
  # optim() with Nelder-Mead can propose a value outside [0,1]; guard it
  p_hidden <- min(max(p_hidden, 0.001), 0.999)
  predicted <- run_model_mortality(p_hidden)
  (predicted - target_mortality)^2
}

fit <- optim(
  par    = 0.15,              # starting guess for p_hidden
  fn     = objective_fn,
  method = "Nelder-Mead",
  control = list(reltol = 1e-8)
)

p_hidden_calibrated <- fit$par
calibration_sq_error <- fit$value
```

Suppose this converges to `p_hidden_calibrated = 0.084`, with the model then predicting 5-year mortality of 31.7% against the 32.0% target – a squared error of $(0.317 - 0.320)^2 = 0.0000009$, i.e. an absolute gap of 0.3 percentage points, small enough to accept as a well-calibrated fit.

How to report this.

Outcome	Target (registry)	Model-predicted (calibrated)	Absolute gap
5-year all-cause mortality, SoC	32.0%	31.7%	0.3 pp

Model-predicted value from `optim()` Nelder-Mead calibration of the hidden transition probability `p_hidden`, starting value 0.15, converged value 0.084. pp = percentage points.

Table 79: Calibration target vs. model-predicted value, TB Markov model, SoC arm

HEOR in practice. Always report the calibrated parameter value *alongside* a plausibility check against clinical or biological priors – a calibrated transition probability that falls far outside the range any clinician would consider plausible (e.g. a calibrated value of 0.95 for a transition believed clinically to be rare) is a signal that the model structure, not just the parameter, needs re-examination. Calibration can compensate for a structural problem elsewhere in the model, producing a "well-fitted" but biologically implausible parameter.

Bayesian calibration

Random search and Nelder-Mead both return a single "best" point estimate for `p_hidden`. Bayesian calibration instead treats `p_hidden` as a random variable with a prior distribution (reflecting what is plausible before seeing the calibration target), and updates that prior using the likelihood of the observed target data to produce a full posterior distribution over `p_hidden`. This posterior naturally

captures calibration *uncertainty* – which then flows through to wider, more honest uncertainty in the cost-effectiveness results downstream, rather than treating the calibrated value as if it were known with certainty (which plain optimization implicitly does).

Conceptually, a Bayesian calibration workflow for our example would: (1) specify a prior for `p_hidden`, e.g. Beta distributed, centered on a clinically plausible range; (2) define a likelihood function describing how probable the observed registry mortality rate of 32% is, for each candidate value of `p_hidden`, given sampling or measurement uncertainty around that registry estimate; (3) use a Markov Chain Monte Carlo (MCMC) sampler to draw from the resulting posterior distribution over `p_hidden`; and (4) propagate the full posterior – not just its mean – through the cost-effectiveness model, in the same spirit as the PSA draws in Chapter 25.

R has packages oriented toward this kind of model calibration (for instance the CRAN package `BayesianTools` provides general-purpose MCMC and sensitivity tools for calibrating arbitrary simulation models against data). We will not reproduce specific function calls from that package here, because getting the exact argument names and defaults wrong would be worse than not showing code at all – if you adopt this approach, work from the package's own current vignette and documentation rather than from code copied out of a general textbook, since calibration packages' APIs do change across versions.

Approximate Bayesian Computation (ABC)

> **Difficult terminology: Approximate Bayesian Computation (ABC).** ABC is a family of methods for doing Bayesian-style calibration when the likelihood function – the mathematical expression for how probable the observed data is, given a parameter value – is too complicated or expensive to write down explicitly, which is common for simulation models like Markov cohort models run many times over. Instead of computing a likelihood, ABC repeatedly: draws a candidate parameter value from the prior, runs the full simulation model with that value, and checks whether the *simulated* outcome is close enough to the *observed* target. Parameter draws that produce simulated outcomes within an acceptable distance of the target are kept; those that do not are rejected. The surviving draws approximate the posterior distribution. An analogy: rather than solving a lock's combination mathematically, you try many physical keys and keep only the ones that turn far enough to feel "close enough" to opening it, without ever formally modeling what "closeness of fit" means in equation form.

For our TB model, a basic ABC rejection scheme would look conceptually like the random-search code above, but instead of keeping only the single closest draw, it would keep *all* draws whose predicted mortality falls within some tolerance (say, plus-or-minus 1 percentage point) of the 32% target, and treat the surviving set of `p_hidden` values as an approximate posterior sample. R has a CRAN package named `EasyABC` oriented toward this kind of rejection and sequential ABC; as with `BayesianTools` above, we deliberately do not reproduce specific function signatures here, since we cannot independently confirm current argument names without risking an inaccurate example – consult that package's own documentation directly before using it.

Incremental Mixture Importance Sampling (IMIS)

IMIS is a more sample-efficient alternative to plain ABC rejection sampling, originally developed in the health-decision-modeling literature specifically for calibrating cohort and dynamic disease models (it has been used, for example, in HIV and tuberculosis policy models). Conceptually, IMIS starts by sampling from the prior, as in ABC, but then iteratively builds a mixture of distributions

concentrated around the regions of parameter space that gave the best fit to the target in earlier iterations, progressively focusing the sampling effort rather than searching the whole prior space with equal density throughout. This can reach a usable approximate posterior with far fewer total model runs than naive rejection ABC, which matters when each run of the TB Markov model (or a more complex dynamic transmission model) is computationally expensive. We flag IMIS here as a named method you should know exists and search for by name if a project needs calibration efficiency beyond basic ABC; we do not reproduce a specific R implementation's call signature, for the same reason noted above for `BayesianTools` and `EasyABC`.

> **Agency perspective.** The ISPOR-SMDM Modeling Good Research Practices Task Force series (published jointly in *Medical Decision Making* and *Value in Health*, around 2012) includes a task force report specifically on model transparency and validation, which discusses calibration as one dimension of model validation alongside face validity, internal verification, and external validation against independent data. If calibration is central to a model being submitted to a HTA body, cite the relevant task force report by its actual title and year as published – check the exact citation in your reference manager or the ISPOR website rather than relying on a title reconstructed from memory, since task force numbering and titles across that series are easy to misattribute.

27.2 Cross-validation of model structure

Calibration (27.1) answers "what parameter value makes this model fit the target?" Cross-validation of model structure asks a different, prior question: "is this the right model structure in the first place?" A model can be beautifully calibrated to match every target perfectly and still have the wrong number of health states, the wrong set of allowed transitions, or an inappropriate cycle length – calibration can mask structural error by forcing a fit, whereas structural cross-validation tests whether the chosen structure generalizes.

The core technique, borrowed from general statistical modeling, is to split available data into a *training* portion used to calibrate or fit the model, and a *held-out* portion used only to check predictions afterward, never touched during fitting. For our TB model, suppose the registry providing the 32% 5-year mortality calibration target actually covers two separate enrollment cohorts: an earlier cohort (used to derive the calibration target in 27.1) and a later, independent cohort for which 3-year (not 5-year) mortality was also reported, at 21%. We can use the later cohort purely as an out-of-sample check: does the model, calibrated only against the first cohort's 5-year figure, also correctly predict the second cohort's 3-year mortality, which it never saw during fitting?

```
# Model was calibrated only against the 5-year target from cohort 1 (27.1).
# Now check an out-of-sample prediction against cohort 2's independent data,
# which was never used anywhere in the calibration step above.
predicted_3yr_mortality <- run_tb_markov(
  p_hidden     = p_hidden_calibrated,
  arm          = "SoC",
  horizon_years = 3
)$mortality_5yr   # function returns cumulative mortality at the requested horizon

held_out_target <- 0.21
cross_val_gap <- predicted_3yr_mortality - held_out_target
```

If `predicted_3yr_mortality` comes out close to 0.21 (say within 1-2 percentage points), that is reassuring evidence the model's *structure* – not just the one calibrated parameter – captures the underlying disease process reasonably well, because it generalizes to data it never saw. If the gap is large, the problem is not necessarily the calibrated parameter value; it may be that the model is missing a relevant health state, has an unrealistic cycle length, or omits a clinically important transition pathway, and no amount of re-calibrating `p_hidden` within the existing structure will fix a mismatch that is actually structural.

A second, complementary cross-validation approach for structure specifically (rather than data splitting) is to build two or more *competing* model structures – for example, a 4-state model (our Chapter 19 structure) versus a 5-state model that separately distinguishes relapse from primary treatment failure – calibrate each separately against the same training target, and then compare which structure better predicts the held-out target. The structure that generalizes better to unseen data, not merely the structure that fits the training target most closely, is the one with stronger support.

> **Common pitfall.** Structural cross-validation only works if the held-out data is genuinely independent of whatever was used to build the model's assumptions in the first place. If both the training target and the "held-out" check come from the same registry, use overlapping patients, or were both already known to the model builders when they chose the model's structure (e.g. the number of health states) during Chapter 19′s design phase, the held-out check is not really independent and will tend to look falsely reassuring.

27.3 Verification: unit testing with `testthat`, extreme-value tests, replication

Calibration and structural cross-validation both assume the underlying code correctly implements the intended model. *Verification* is the separate, more basic question: does the code do what we believe it does, independent of any external calibration target? A model can be perfectly calibrated to a real-world target purely by chance, while still containing a coding error (e.g. a transition-probability matrix whose rows do not sum to 1, silently losing or creating cohort members every cycle). Verification catches this class of error directly, by testing the code's internal logic rather than its external fit.

> **Difficult terminology: unit testing.** Unit testing means writing small, automated pieces of code whose only job is to check that another specific piece of code behaves as expected, and running those checks automatically every time the underlying code changes. It is analogous to a hospital's routine quality-control check on a diagnostic machine – rather than trusting that the machine is well-calibrated because it was built correctly once, staff run a known reference sample through it periodically and confirm the reported result matches the known answer; if it does not, the machine is flagged before it is used on real patients. `testthat` is the standard R package for writing these automated checks.

Unit test: transition matrix row-sums equal 1

Every row of a Markov transition probability matrix must sum to exactly 1, because each row describes the full set of mutually exclusive, exhaustive destinations for a cohort member starting in that state during one cycle – if a row sums to less than 1, cohort members vanish each cycle; if it sums to more than 1, cohort members are created from nothing. This is the single most common silent bug in hand-built Markov cohort models, and it is trivial to test automatically.

```
library(testthat)

# tm_soc is the 4x4 transition probability matrix for the SoC arm,
# built in Chapter 19 (rows: Well, Failed, Cured, Dead; same column order)
test_that("SoC transition matrix rows sum to 1", {
  row_sums <- rowSums(tm_soc)
  expect_equal(row_sums, rep(1, nrow(tm_soc)), tolerance = 1e-8)
})

test_that("New-regimen transition matrix rows sum to 1", {
  row_sums <- rowSums(tm_new)
  expect_equal(row_sums, rep(1, nrow(tm_new)), tolerance = 1e-8)
})
```

A `tolerance = 1e-8` argument is deliberate: floating-point arithmetic rarely produces an exact 1.000000000 when probabilities are added, so an overly strict `expect_equal` comparing to exactly 1 with no tolerance would fail on correct code purely due to rounding noise, while a tolerance of 1e-8 is tight enough to catch a genuine row-sum error (e.g. a row summing to 0.95 because a transition was mistakenly omitted) while tolerating harmless floating-point noise far smaller than any clinically meaningful difference.

Extreme-value test

An extreme-value test deliberately sets an input parameter to an implausible, extreme boundary value – not because that value is clinically realistic, but because the model's behavior at the extreme is mathematically predictable, so any deviation from the predicted extreme behavior reveals a coding bug. If we set the "Well-on-treatment" to "Dead" annual mortality probability to exactly 1 (certain death within one cycle for every cohort member in that state), the model should report that essentially zero cohort members survive past the first cycle – any other output means the cycling logic itself is broken, independent of whether a probability of 1 is remotely realistic for TB mortality.

```
test_that("certain mortality (p = 1) drives survivors to zero after cycle 1", {
  tm_extreme <- tm_soc
  # Force the Well-on-treatment row to transition to Dead with certainty.
  # Column order: Well, Failed, Cured, Dead
  tm_extreme["Well", ] <- c(0, 0, 0, 1)

  result_extreme <- run_tb_markov_matrix(
    transition_matrix = tm_extreme,
    starting_state    = "Well",
    horizon_years      = 5
  )

  # After the very first cycle, zero cohort members remain alive in any
  # non-Dead state, by construction of the forced transition above.
```

```
  survivors_after_cycle_1 <- sum(result_extreme$trace[2, c("Well", "Failed",
"Cured")])
  expect_equal(survivors_after_cycle_1, 0, tolerance = 1e-10)
})
```

Common pitfall. Extreme-value tests are only informative if the extreme value is actually mathematically determinate. Setting a *cost* parameter to zero, for instance, is a weaker test than setting a *transition probability* to 1 or 0, because many cost model structures will "pass" a zero-cost extreme test trivially (total cost is zero, which is correct but uninformative about whether the rest of the cost calculation logic works). Choose extremes where the correct answer is unambiguous *and* where getting it wrong would require the test to actually exercise meaningful parts of the model's logic – a transition probability of exactly 1 or 0 forces a specific, checkable cohort trace, which is a stronger test than a trivially zero-dollar cost.

Replication

Replication, in the verification sense used here, means confirming that running the *identical* model code, with the *identical* random seed and inputs, on a different machine or in a different R session produces *identical* numeric output – not merely "similar" output. This catches a different class of error than unit tests: platform-dependent behavior (e.g. an R package version difference changing a default argument), uncontrolled randomness (a random number generator seed that was not actually fixed everywhere it needed to be), or silent dependency on the state of a previous script run rather than a clean session.

The practical check is straightforward: set the random seed explicitly at the top of the script (as the PSA code in Chapter 25 already does with `set.seed()` before drawing the 2000 parameter sets), run the full model end-to-end in a fresh R session, and save the key numeric outputs – for example the base-case incremental cost of -dollar 844.14 and incremental QALY of 0.1440 from Chapter 25 – to a file. Then run the same script again, either in a new session on the same machine after restarting R, or ideally on a colleague's separate machine with the same package versions installed, and confirm the newly produced numbers match the saved ones exactly (or within an extremely tight numeric tolerance consistent with floating-point arithmetic, not within a clinically-motivated tolerance band).

```
# At the top of the master analysis script, before any random draws:
set.seed(20240101)

# ... full PSA and base-case model run as built in Chapters 19 and 25 ...

# Save key outputs for later replication comparison
saveRDS(
  list(
    incr_cost  = incr_cost_base_case,   # expect -844.14
    incr_qaly  = incr_qaly_base_case,   # expect 0.1440
    incr_nmb   = incr_nmb_wtp_50k       # expect 8044
  ),
  file = "replication_reference_outputs.rds"
)

# On a second machine or fresh session, after rerunning the identical script:
reference <- readRDS("replication_reference_outputs.rds")
```

```
test_that("replicated run matches reference outputs exactly", {
  expect_equal(incr_cost_base_case, reference$incr_cost, tolerance = 1e-6)
  expect_equal(incr_qaly_base_case, reference$incr_qaly, tolerance = 1e-6)
  expect_equal(incr_nmb_wtp_50k, reference$incr_nmb, tolerance = 1e-6)
})
```

> **HEOR in practice.** Before submitting a cost-effectiveness model to a reimbursement body, have someone *other* than the original model builder attempt a clean replication run on their own machine, from the raw code and documented package versions alone, with no informal guidance from the original author beyond the written analysis plan. If a second analyst cannot independently reproduce the headline incremental cost, incremental QALY, and ICER from the documentation as written, the documentation – not just the code – has a gap that a reviewer at the HTA body is equally likely to hit.

27.4 Validation frameworks (AdViSHE, TECH-VER, ISPOR-SMDM)

Sections 27.1-27.3 addressed **verification**: did we build the model right? We checked that the TB Markov cohort model's code executes correctly, that its mechanics calibrate sensibly, and that unit tests catch regressions in the transition logic. This section turns to **validation**: did we build the right model? Verification is an internal, technical question about implementation fidelity; validation is an external question about whether the model's structure, inputs, and outputs are defensible representations of the real-world decision problem it claims to inform. A model can pass every unit test in Chapter 27.3 and still be the wrong model – missing a key health state, using an implausible transition structure, or producing outputs that contradict trial or real-world evidence. The sections that follow (27.4-27.6) are about catching that second category of failure.

A widely cited framework for organizing this work is **AdViSHE** (Assessment of the Validation Status of Health-Economic decision models), published by Vemer, Corro Ramos, van Voorn, Al, and Feenstra (2016) in *Pharmacoeconomics*. AdViSHE is a structured checklist that modelers complete and submit alongside a model, so that reviewers, payers, and HTA bodies can see explicitly what validation work has (and has not) been done. It organizes validation effort into several domains:

- **Conceptual model validation** – is the model's structure (health states, transitions, comparators, time horizon) an adequate representation of the disease and decision problem, as judged against clinical knowledge and existing literature? This connects directly to the model-conceptualization step introduced in Chapter 17.
- **Input data validation** – are the parameter values (transition probabilities, costs, utilities) sourced appropriately, and are they used consistently with how they were derived?
- **Implementation validation** – does the model run correctly, free of mechanical errors? This is essentially the verification question from 27.1-27.3, and AdViSHE explicitly treats it as one validation domain among several rather than the whole of validation.
- **Operational validation** – do the model's outputs behave plausibly and match independent expectations, including comparison against other models or against observed data? This is where cross-validation and external validity (27.6) live.
- **Other validation** – softer evidence of credibility, such as independent review of the model by a second party, or the model's provenance and prior use in other submissions.

HEOR in practice. Framing validation as a checklist with discrete domains – rather than a single pass/fail judgment – is itself the useful part of AdViSHE. It gives a submission team a concrete list of evidence to assemble (clinical expert sign-off on structure, a data-source table, a verification log, a comparison against other published models) rather than a vague assurance that "the model has been validated."

ISPOR and the Society for Medical Decision Making (SMDM) jointly produced a seven-part **Modeling Good Research Practices Task Force** series covering model structure, data, parameter estimation, model transparency, and validation among other topics. The task force report most directly relevant here is Eddy, Hollingworth, Caro, Tsevat, McDonald, and Wong (2012), "Model Transparency and Validation," ISPOR-SMDM Modeling Good Research Practices Task Force-7, published jointly in *Value in Health* and *Medical Decision Making*. That report makes the same verification/validation distinction used as this section's opening bridge, and it further separates validation into face validity, internal validity (verification, in their terminology), cross-validation, and external (predictive) validity – the same taxonomy structuring section 27.6 below.

A further checklist sometimes referenced in HTA practice, under names such as **TECH-VER**, is aimed at systematic **technical verification** of a model's code and calculations – essentially a structured audit trail confirming that formulas, cell references, and code paths match the documented methods. The authors of this book are not confident of a single canonical citation for a framework under that exact name, so rather than inventing a specific author or year, TECH-VER is described here at the level of the general practice it represents: a line-by-line, independently conducted technical check of a model's implementation, distinct from – and complementary to – the structural and output-level validation domains of AdViSHE. Readers planning a submission that references TECH-VER specifically should confirm the current source document with the relevant HTA body rather than relying on this book.

Domain	Question asked	Covered in
Conceptual model	Is the model structure a sensible representation of the disease and decision problem?	Ch. 17; §27.6 face validity
Input data	Are parameter sources and their use appropriate?	Ch. 18-19
Implementation	Does the model run correctly (verification)?	§27.1-27.3
Operational	Do outputs match independent expectations (other models, real data)?	§27.6 cross- and external validity
Other	Has the model had independent review?	§27.5

Table 80: AdViSHE validation domains and their relationship to this book's chapters

27.5 Code review and peer review practices

Validation is not only a property of a finished model; it is also a property of the **process** used to build it. Health-economic models in R benefit from the same code-review discipline used in professional software development, adapted to the fact that a modeling error can change a reimbursement decision rather than just a feature.

The most direct mechanism is a **git-based pull-request review workflow**, building on the version-control practices introduced in Chapter 2. Rather than committing changes to the TB Markov model directly onto a shared branch, a modeler opens a pull request containing, for example, a change to the SoC-arm mortality transition probability. A second modeler reviews the diff, checks that the change is consistent with the cited source, re-runs the test suite from 27.3, and only then approves the merge. This produces a durable audit trail: for every parameter change in the model's history, there is a reviewed, timestamped record of who changed what, why, and who checked it.

A stronger and more expensive form of independent scrutiny, common in HTA practice, is **second-modeler "black-box" replication**: a second analyst, given only the same inputs, assumptions, and target outputs as the original team – but deliberately **not** given the original code – builds an independent implementation of the same model from scratch. If the TB Markov model's incremental cost-effectiveness ratio from the independent build matches the original team's result within an acceptable tolerance, this is strong operational-validation evidence (AdViSHE's "other validation" domain) that is largely immune to a shared coding error, since the two implementations do not share code. Discrepancies, when they arise, are often themselves diagnostic: they tend to trace back to an ambiguous assumption in the original model specification that the two teams resolved differently, which is valuable information in its own right.

> **Agency perspective.** ISPOR-SMDM guidance (Eddy et al. 2012, Task Force-7) explicitly recommends independent review and, where feasible, independent replication as part of establishing a model's credibility, precisely because a single team – however careful – cannot fully audit its own blind spots. Treat a clean black-box replication as meaningfully stronger evidence than a self-reported internal QA pass; budget and schedule for it on any model intended to support a high-stakes reimbursement submission.

A lighter-weight but continuously running layer of scrutiny is the **testthat**-based continuous integration (CI) suite introduced in section 27.3. Where black-box replication is a one-off, resource-intensive check typically performed once per submission, CI tests run on every commit and every pull request, automatically re-verifying that state occupancies still sum to one, that known reference-case outputs have not silently drifted, and that cost and QALY totals remain within expected bounds after each change. Pull-request review and CI are complementary: CI catches mechanical regressions immediately and cheaply; human review catches conceptual and judgment errors that no automated test is written to detect, because no one anticipated that particular mistake.

> **Common pitfall.** A pull request that passes every CI check is not the same as a pull request that has been genuinely reviewed. CI tests only catch what someone thought to test for; they say nothing about whether a change is clinically or economically sensible. Do not let a green CI badge substitute for a human reviewer actually reading the diff and the rationale behind it.

27.6 Face validity, internal, cross and external validity

Building on the Eddy et al. (2012) taxonomy introduced in 27.4, this section works through four distinct senses of "the model is valid," each asking a different question and requiring different evidence.

Face validity asks whether the model's structure and assumptions look sensible to people who know the disease and the decision context – a qualitative, expert-judgment check rather than a numerical test. For the TB Markov model, face validity was substantively addressed during the model-conceptualization step in Chapter 17, when clinical input shaped the choice of health states (e.g., active TB, treatment, cured, dead) and the plausibility of the comparator structure (New regimen vs. SoC). Face validity is revisited here because it is not a one-time event: every subsequent structural change to the model – a new health state, a revised transition pathway – warrants a fresh face-validity check with clinical input before the change is accepted.

> **Difficult terminology: face validity.** Face validity is the simplest and least rigorous form of validation: a domain expert looks at a model's structure and assumptions and judges whether they are plausible on their face, without running any numbers. It is a necessary first filter – an implausible structure is not worth validating numerically – but it is not sufficient on its own, because a structurally sensible model can still produce numerically wrong outputs.

Internal validity asks whether the model's own internal logic and arithmetic are self-consistent: do the mechanics do what they are supposed to do? This is the same check already demonstrated mechanically in Chapter 19 and automated in 27.1-27.3 – for instance, confirming that the four state-occupancy proportions in each model cycle sum to 1 (within floating-point tolerance), that costs and QALYs accumulate correctly across cycles, and that no probability mass silently leaks out of the state vector. Internal validity overlaps heavily with what Eddy et al. (2012) call verification; it is listed here to complete the taxonomy, not because it needs re-explaining after 27.1-27.3.

Cross-validation – in this health-economic-modeling sense – compares this model's results against the results of **other, independently built** models addressing the same decision problem (for example, other published TB cost-effectiveness models evaluating similar regimens). Close agreement is reassuring; disagreement motivates investigating whether the models differ in structure, data sources, or time horizon, and whether one is more defensible than the other.

> **Difficult terminology: cross-validation (health-economic sense).** This term is a naming collision with the statistical/machine-learning sense of cross-validation used in Chapter 16 (splitting a dataset into folds to estimate out-of-sample predictive performance). In health-economic model validation, cross-validation means something structurally different: comparing one **entire decision model's** outputs against those of **other, separately built** models of the same decision problem. No data-splitting or resampling is involved. When reading HTA methods guidance, infer which sense is meant from context – a statistics chapter means the ML sense; a model-validation chapter (like this one) means the model-comparison sense.

External validity – also called predictive validity – asks whether the model's **projected** outcomes, extended forward in time, match what **actually happened** in the real world once enough time has passed to observe it. This is the most demanding form of validation, because it requires real-world follow-up data that may not exist until years after the model was built; Chapter 15′s

real-world-evidence (RWE) methods are the natural data source for making this comparison once such data become available.

Illustrative worked example. Suppose the TB Markov model, run forward from treatment initiation, predicts 10-year overall survival of 78% for patients on the New regimen. Suppose further that a (clearly illustrative, not an actual published figure) real-world TB cohort followed for 10 years after a comparable regimen reports observed 10-year survival of 74%. The absolute difference is:

$$78 - 74 = 4$$

percentage points.

As a relative (percentage) difference relative to the observed figure:

$$\frac{78 - 74}{74} \times 100 \approx 5.41$$

percent.

So the model's prediction is 4 percentage points higher than the illustrative observed cohort figure, or about 5.4% higher in relative terms. Whether a 4-percentage-point gap is "close enough" is a judgment call that depends on the cohort's sample size and confidence interval, the time elapsed, and how much the real-world cohort's population and care pathway diverge from the model's assumed population – but the computation above is exactly the kind of check an external-validity exercise performs: take the model's forward-projected number, take the later-observed number for the same quantity, and quantify the gap.

How to report this.

Quantity	Value	Source
Model-predicted 10-year survival	78%	TB Markov model, Ch. 19
Observed 10-year survival (illustrative cohort)	74%	Illustrative RWE comparator
Absolute difference	4 pp	Computed above
Relative difference	≈5.41%	Computed above
Figures for the observed cohort are illustrative, not drawn from a published source, and are used only to demonstrate the external-validity calculation.		

Table 81: Illustrative external-validity comparison for 10-year survival, New regimen

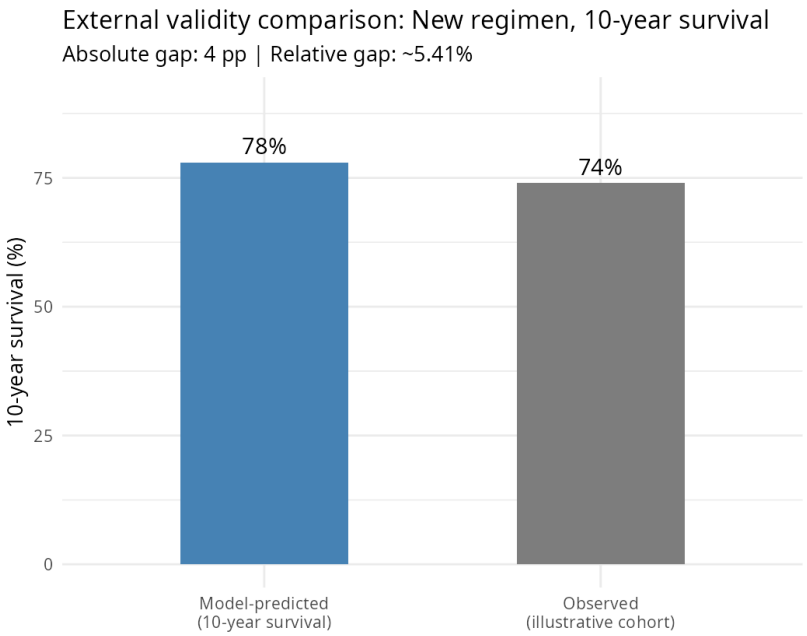


Figure 48: The external-validity comparison from the table above: model-predicted (78%) vs illustrative observed (74%) 10-year survival for the New regimen. The gap is small in absolute terms (4 percentage points) but, per the pitfall below, a single cohort comparison is evidence to weigh, not a verdict.

Common pitfall. Do not treat a close match between model predictions and real-world outcomes as proof the model is "correct." A single cohort's 10-year outcome is one noisy data point; close agreement could be coincidental, and disagreement could reflect a real-world population that differs from the model's target population rather than a flawed model. External validity checks are evidence to weigh, not a verdict to declare from one comparison.

Taken together, face validity, internal validity, cross-validation, and external validity form a layered evidence base: a model that looks sensible to experts, computes correctly, agrees with independently built peers, and – where data eventually permit – tracks real-world outcomes, has accumulated a substantially stronger validation case than a model that only passes one of these checks. None of this evidence is useful, however, if it is scattered across emails, slide decks, and someone's memory of a meeting. Chapter 28, Reproducible Reporting, takes up exactly that problem: how the verification evidence from 27.1-27.3 and the validation evidence from 27.4-27.6 get written down, version-controlled, and communicated to reviewers and decision-makers in a form that is itself reproducible.

PART VI

Reporting, Communication and Deployment

Chapter 28. Reproducible Reporting

Learning objectives

- Explain the difference between Quarto and classic R Markdown, and build a Quarto document whose YAML front matter and code chunks render a live HEOR summary report.
- Write a parameterised Quarto/R Markdown report and loop its rendering across multiple scenarios, subgroups or sites.
- Produce the same analysis as Word, PDF, HTML and PowerPoint output, including native editable Office elements via `officedown`/`officer`.
- Choose among `gt`, `gtsummary`, `flextable` and `kableExtra` for a publication-ready results table, and justify the choice by destination format.
- Build a reusable `ggplot2` theme and apply colour-blind-safe palettes, saving figures at print resolution.
- Track CHEERS 2022 checklist compliance programmatically rather than by hand.
- Assemble a multi-section HTA submission dossier from parameterised Quarto child documents.
- Automate routine, recurring reports across programmes or districts, and choose a sensible scheduling mechanism for the pipeline.

Chapter 27 closed on a reminder that calibration and validation evidence is only as useful as the report that carries it to a decision-maker. A beautifully validated TB Markov model that lives only in an analyst's R console, with figures copy-pasted into a Word document by hand, is a model whose evidence trail breaks the moment someone asks "show me exactly how this number was produced." This chapter is about closing that gap: turning R code, model output and narrative text into a single document that regenerates itself from source, every time, byte-for-byte traceable back to the code that created it.

28.1 Quarto and R Markdown for HEOR reports

The tool of choice for this in modern R-based HEOR work is Quarto, which has effectively superseded classic R Markdown for new projects, though R Markdown remains extremely common in legacy codebases and some submission templates still expect `.Rmd` files.

R Markdown and Quarto: what changed and what didn't

R Markdown (`.Rmd` files, powered by the `rmarkdown` and `knitr` packages) pioneered "literate programming" for R: a plain-text document mixing prose (written in Markdown) with executable R code blocks, rendered to HTML, PDF or Word via Pandoc. Quarto (`.qmd` files) is Posit's (RStudio's) next-generation evolution of the same idea, built as a standalone, language-agnostic command-line tool rather than an R package. The practical consequences for an HEOR analyst:

- Quarto documents can mix R, Python and Julia code chunks in the same file – useful if your health-economic model is in R but a colleague's survival-extrapolation code is in Python.
- Quarto's YAML front matter and chunk-option system is richer and more consistent across formats (HTML, PDF, Word, PowerPoint, websites, books – this very book is built with Typst, a different tool, but the same literate-document idea).

- Quarto is installed as a separate command-line application (not an R package), and is driven from R via the `quarto` R package (a thin wrapper that calls the Quarto CLI) or directly from a terminal with `quarto render`.
- Existing `.Rmd` files generally render under Quarto with little or no change, so migration is usually incremental rather than a rewrite.

> **Difficult terminology: YAML.** YAML ("YAML Ain't Markup Language") is a plain-text format for writing structured settings – key-value pairs, lists, nested options – in a way that is meant to be readable by humans, not just machines. In a Quarto or R Markdown document it appears as a block at the very top of the file, fenced by three dashes (`---`) above and below, called the *front matter*. Think of it as the document's "settings panel": it tells the renderer the title, author, output format(s), and any parameters, before a single line of the report's actual content is read. Indentation in YAML is meaningful (like in Python), and a stray extra space is a common source of cryptic rendering errors – if a render fails immediately with no R error, check the YAML first.

A minimal Quarto YAML header for an HEOR report might look like:

```
---
title: "Cost-Effectiveness of New-Regimen TB Treatment vs Standard of Care"
subtitle: "Base-case and scenario results, 4-state Markov cohort model"
author: "HEOR Modelling Team"
date: today
format:
  html:
    toc: true
    embed-resources: true
  docx: default
  pdf:
    documentclass: article
execute:
  echo: false
  warning: false
---
```

Note that `format` can list more than one output target in a single file – Quarto will render the same source to HTML, Word and PDF in one `quarto render` call, each obeying its own sub-options (SS28.3 goes into this in detail).

> **Difficult terminology: chunk.** A "chunk" (or "code cell") is a fenced block of executable code embedded inside the narrative document, marked off from the surrounding prose by triple backticks and a language tag, e.g. a line reading ```` ```{r} ```` . When the document is rendered, every chunk is actually run by R (or Python, etc.), and its output – a table, a plot, printed text – is inserted into the final document at that exact point. This is the core idea of literate programming: the numbers in the report are not typed in by the analyst, they are *computed live* each time the document is built, so the report can never drift out of sync with the model it describes.

Each chunk can carry its own options, written either in the classic `{r, option=value}` header or, in Quarto's preferred style, as special `#|` comment lines inside the chunk:

```
#| label: fig-markov-trace
#| echo: false
#| warning: false
#| fig-cap: "State-occupancy trace, New-regimen vs SoC"

plot_markov_trace(model_results)
```

Commonly used chunk options for HEOR reporting:

- `echo` – whether the R code itself is shown in the output (usually `false` for a report meant for clinicians or payers, `true` for an internal technical appendix).
- `eval` – whether the chunk is executed at all (useful for temporarily disabling a slow bootstrap or PSA chunk during drafting).
- `warning` / `message` – whether R warnings/messages leak into the rendered document (almost always suppressed in a final report).
- `fig-cap`, `fig-width`, `fig-height` – captioning and sizing for figures.
- `tbl-cap` – captioning for tables produced by chunk output.

> **Difficult terminology: render.** "Rendering" is the act of running every code chunk in the document, capturing its output, and weaving the result together with the prose into a finished file (HTML page, Word document, PDF, etc.) via Pandoc. It's conceptually the same step as "compiling" a program, except the output is a formatted document rather than an executable. You trigger it by running `quarto render report.qmd` from a terminal, or by clicking "Render" in RStudio/Positron, or programmatically from R with `quarto::quarto_render()`.

A worked example: rendering an HEOR summary report

Suppose the base-case results from the TB 4-state Markov model (Chapters 25 and 27) are saved as a small data frame. A minimal but complete Quarto document to turn that into a shareable report looks like this.

```
---
title: "TB Treatment Cost-Effectiveness Summary"
format:
  html:
    toc: true
  docx: default
execute:
  echo: false
---
```

A chunk later in the same document builds and formats the base-case table:

```
#| label: tbl-base-case
#| tbl-cap: "Base-case incremental results, New-regimen vs SoC"

library(gt)

base_case <- data.frame(
  Arm = c("SoC", "New-regimen", "Incremental"),
  Cost = c(5000.00, 4155.86, -844.14),
```

```
  QALY = c(3.210, 3.354, 0.1440)
)

gt(base_case) |>
  fmt_currency(columns = Cost, currency = "USD") |>
  fmt_number(columns = QALY, decimals = 4)
```

The incremental net monetary benefit at a willingness-to-pay of $50,000 per QALY is computed inline from the same two numbers (`r format(0.1440 * 50000 - (-844.14), big.mark = ",")`), confirming that the New-regimen strategy dominates standard of care – it is both cheaper and more effective.

Running `quarto render tb_summary.qmd` from the project directory (with both `html` and `docx` listed in the YAML `format:` key) produces `tb_summary.html` and `tb_summary.docx` from this single source file – one source of truth, two deliverables, no manual re-typing of numbers into a Word document.

HEOR in practice. Keep one `.qmd` file per *deliverable*, not per analysis. A good pattern is to have your heavy modelling code (Markov model, PSA, DSA) live in plain `.R` scripts or functions that save tidy result objects (`.rds` files) to disk, and have the Quarto report do nothing but `readRDS()` those objects and format them. This keeps render times short – nobody wants to re-run a 10,000-iteration PSA every time a typo in a sentence is fixed – and keeps the report's git history focused on *presentation* changes rather than being swamped by model-code diffs.

Common pitfall. `echo: false` hides code from the *rendered* document, but it does not stop the code from running, and it does not strip sensitive content (patient-level data previews, API keys pasted into a chunk for convenience, internal file paths) from error messages if a chunk fails during rendering. Before sending a report outside the team, render it once from a clean environment (`quarto render` in a fresh R session, not from your already-populated RStudio workspace) so you catch any hidden dependency on objects that happen to exist in your session but are not reproducibly created by the document itself.

28.2 Parameterised reports

A single base-case report is rarely the end of the story in HEOR work: a submission package might need the same analysis repeated for several subgroups (e.g. drug-sensitive vs drug-resistant TB), several price-sensitivity scenarios, or one report per implementing country or site in a multi-country costing study. Writing a separate `.qmd` file for each of these by hand is exactly the kind of copy-paste risk reproducible reporting is meant to eliminate. The fix is a *parameterised report*.

Difficult terminology: parameterised report. A parameterised report is a single report template written so that certain inputs – a subgroup name, a willingness-to-pay threshold, a country's unit costs – are left as *variables* ("parameters") rather than hard-coded, in the same way a SQL query with a `WHERE region = :region` placeholder can be re-run for any region without rewriting the query. The report is rendered once per parameter set, and each

> render produces its own output file, all sharing identical structure and logic but populated with different numbers.

In Quarto (and R Markdown), parameters are declared under a `params:` key in the YAML front matter, and become available inside the document as elements of a list called `params`:

```
---
title: "Cost-Effectiveness Results"
format: html
params:
  scenario_name: "Base case"
  wtp_threshold: 50000
  drug_price_multiplier: 1.0
---
```

Inside the document, those values are used directly in R code:

```
#| label: calc-nmb

inc_cost <- -844.14 * params$drug_price_multiplier
inc_qaly <- 0.1440
nmb <- inc_qaly * params$wtp_threshold - inc_cost
```

For R Markdown documents the same `params:` mechanism works, and used to require RStudio's "Knit with Parameters" dialog or a call to `rmarkdown::render(..., params = list(...))`; Quarto keeps the YAML syntax but drives it from the command line.

Rendering a single parameterised report

From a terminal, overriding one or more parameters at render time:

```
quarto render tb_report.qmd \
  --execute-params params.yml \
  --output tb_report_resistant_subgroup.html
```

where `params.yml` is a small YAML file holding just the overrides for that run:

```
scenario_name: "Drug-resistant TB subgroup"
wtp_threshold: 50000
drug_price_multiplier: 1.35
```

Quarto also accepts individual `-P key:value` overrides on the command line for quick one-off changes, e.g. `quarto render tb_report.qmd -P wtp_threshold:100000`, without needing a separate params file.

Looping render calls to generate one report per scenario

The real value of parameterisation shows up when a *driver script* loops over a table of scenarios and calls the renderer once per row, generating a whole folder of reports unattended. This is most conveniently driven from R using the `quarto` package's `quarto_render()` function, which wraps the same CLI command used above:

```
library(quarto)

scenarios <- data.frame(
  scenario_name = c("Base case", "Drug-resistant subgroup", "High drug price
(+35%)"),
```

```
  wtp_threshold = c(50000, 50000, 50000),
  drug_price_multiplier = c(1.00, 1.00, 1.35)
)

for (i in seq_len(nrow(scenarios))) {
  scn <- scenarios[i, ]
  safe_name <- gsub("[^A-Za-z0-9]+", "_", scn$scenario_name)

  quarto_render(
    input = "tb_report.qmd",
    output_file = paste0("tb_report_", safe_name, ".html"),
    execute_params = list(
      scenario_name = scn$scenario_name,
      wtp_threshold = scn$wtp_threshold,
      drug_price_multiplier = scn$drug_price_multiplier
    )
  )
}
```

The equivalent for a classic R Markdown document uses `rmarkdown::render()` with its own `params =` argument in the same loop structure; the parameterisation idea is identical, only the rendering function and a couple of argument names differ.

HEOR in practice. Always write the scenario identifiers *into the output filename* (as the loop above does with `safe_name`) and, ideally, into a visible line in the rendered report itself (e.g. a line stating "Scenario: Drug-resistant subgroup, WTP = $50,000/QALY, price multiplier = 1.35"). Reviewers routinely receive a folder of twelve near-identical reports; if the only way to tell them apart is the filename, a single rename or a copy into an email attachment can silently scramble which results belong to which scenario.

Common pitfall. Parameters declared in the YAML `params:` block are given *default* values that are used whenever you render interactively (e.g. clicking "Render" in an IDE) without supplying overrides. It is easy to forget this and ship a report that was actually rendered with the stale defaults rather than the intended scenario values. When looping programmatically, always pass every parameter explicitly in `execute_params`/`params =`, and consider adding a sanity-check chunk near the top of the report that prints the actual `params` list that was used, so it is visible in the rendered output rather than hidden in the render call.

28.3 Producing Word, PDF, HTML and PowerPoint outputs

HEOR deliverables rarely stay in one format. A health technology assessment (HTA) submission body may want an editable Word document for track-changes review, a PDF for the final locked submission, an HTML dashboard for internal exploration, and a PowerPoint deck of headline slides for a committee meeting – often all from the same underlying analysis. Quarto's `format:` key in the YAML front matter is what controls this, and the same source document can target several formats at once.

```
---
title: "TB Treatment Cost-Effectiveness: Committee Summary"
```

```yaml
format:
  html:
    toc: true
    embed-resources: true
  docx:
    reference-doc: templates/heor_word_template.docx
  pdf:
    documentclass: article
    toc: false
  pptx:
    reference-doc: templates/heor_slide_template.pptx
---
```

Running `quarto render` with no arguments renders *all* formats listed under `format:`; `quarto render report.qmd --to docx` renders just the Word version. The `reference-doc` option (for both `docx` and `pptx`) points Pandoc at an existing Word or PowerPoint template file whose styles (fonts, heading colours, table styles, logo placement) are copied into the new document – this is the standard way to make a rendered report match an organisation's or journal's house style without writing any Word-specific code.

PDF output and the LaTeX requirement

> **Difficult terminology: LaTeX.** LaTeX is a typesetting system – a much older and more specialised cousin of Word – that produces very precisely laid-out documents, and is the engine Pandoc uses under the hood whenever Quarto or R Markdown render to PDF. Unlike HTML or Word output, PDF rendering via Pandoc is not a pure-R, pure-Quarto process: it requires a working LaTeX installation on the machine doing the rendering, in the same way that compiling a C program requires a C compiler to be installed separately from the source code editor. A full LaTeX distribution (TeX Live, MiKTeX) is large (several gigabytes); R users almost always install a slimmed-down distribution instead.

The standard slimmed-down option for R users is `tinytex`, installed and then used to provision the actual LaTeX engine with:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

After this one-time setup, `quarto render report.qmd --to pdf` (or an R Markdown document with `output: pdf_document`) will work without further configuration. If a PDF render fails with an error about a missing LaTeX package (a `.sty` file), `tinytex::tlmgr_install("package-name")` installs just that missing piece rather than requiring a full distribution reinstall.

> **Common pitfall.** PDF rendering errors from LaTeX are notoriously unreadable to anyone without LaTeX experience – a stray special character (`%`, `&`, `_`, `$`) inside R-generated text (a drug name, a file path, a footnote) can produce a wall of LaTeX log output with no obvious connection to the real cause. Favour generating tables and text through packages that properly escape such characters for the target format (`gt`, `gtsummary`, `kableExtra` with `escape = TRUE`, discussed in SS28.4) rather than hand-building LaTeX or Markdown table syntax, and test a PDF render early in a project rather than only at submission deadline.

Native editable Word and PowerPoint elements with officer/officedown

Pandoc's built-in Word/PowerPoint conversion handles headings, paragraphs and simple tables well, but HEOR reviewers often want to directly edit a *specific* table cell, reflow a chart inside PowerPoint, or insert a native Word cross-reference or table of contents field – things that round-trip conversion via Pandoc does not fully support. The `officedown` package (built on top of the lower-level `officer` package) extends R Markdown/Quarto's Word and PowerPoint rendering to insert genuinely native, editable Office XML elements rather than Pandoc's generic conversion.

A Word-targeted document using `officedown` declares it in the YAML:

```
---
title: "TB Model Results -- Editable Word Report"
output:
  officedown::rdocx_document:
    reference_docx: templates/heor_word_template.docx
---
```

and can then embed an `officer`-built table or a native Word block directly in a chunk, e.g. using `officer::fp_text()`/`officer::fp_par()` to control run- and paragraph-level formatting. The general exact signature of `officedown`'s table-of-contents helper is not independently re-verified here and should be checked against the current package documentation before use.

```
library(officer)
library(flextable)

ft <- flextable(base_case) |>
  flextable::autofit()

ft   # printed inside an officedown Word chunk renders as a fully
     # native, directly editable Word table, not an embedded image
```

> **Agency perspective.** CHEERS 2022 (Husereau et al., *Value in Health*, 2022), the consolidated health economic evaluation reporting standards checklist, does not mandate a specific file format – but its items on study perspective through characterising uncertainty each require specific, clearly labelled numeric results and tables to be reported in the manuscript or its appendices in a form the reader (and, for a journal submission, often a copy-editor working in Word) can directly inspect and quote. In practice this means a submission-ready HEOR report needs at minimum an editable Word or a clean, correctly captioned PDF/HTML version carrying every CHEERS-required table, and reproducible-reporting tooling of the kind in this chapter is the most reliable way to guarantee all of those tables stay numerically consistent with the underlying model across every output format generated.

28.4 Publication-ready tables (gt, gtsummary, flextable, kableExtra)

A model can be perfectly validated and a report perfectly parameterised, and still land badly if its tables look unpolished or, worse, are hand-typed and silently diverge from the numbers the model actually produced. R has four widely used packages for publication-quality tables, and – honestly – they overlap rather than neatly dividing the territory, but each has a clear sweet spot.

- **gtsummary** – the natural first choice for clinical/epidemiological *summary* tables (baseline characteristics, regression model summaries, survival summaries) built directly from a data frame or model object via `gtsummary::tbl_summary()`, with sensible defaults for medical journals.
- **gt** – a general-purpose "grammar of tables" package for building any publication table (not just automated summaries) from a data frame via `gt::gt()`, with solid output across HTML, PDF (via LaTeX) and Word.
- **flextable** – the strongest choice when the *destination* is Word or PowerPoint and the table must look and behave exactly right once there (merged cells, specific borders, native Office formatting), including via `officedown`'s `officer`/`flextable` integration from SS28.3.
- **kableExtra** – a lightweight package for dressing up simple tables (built on `knitr::kable()`) for quick, clean LaTeX or HTML output via `kableExtra::kbl()`, well suited to appendix tables or fast internal reports where full `gt`/`flextable` styling is overkill.

Package	Best for	Output formats	Notes
`gtsummary`	Clinical/epi summary tables (baseline characteristics, regression tables)	HTML, PDF, Word, RTF (via `gt`/`flextable` print engines)	Opinionated defaults; less suited to fully custom, non-summary layouts
`gt`	General publication tables built from any data frame	HTML, PDF (LaTeX), Word (via integration packages), RTF	Most actively developed "grammar of tables"; PDF output needs a working LaTeX install
`flextable`	Word- and PowerPoint-native tables with precise formatting	Word, PowerPoint, HTML, PDF	Best fidelity when destination is Office documents; integrates with `officer`/`officedown`
`kableExtra`	Lightweight appendix/quick-report tables	HTML, LaTeX/PDF	No native Word output; simplest syntax for basic styling needs

Boundaries overlap in practice; the comparison reflects each package's strongest, most idiomatic use rather than a hard restriction.

Table 82: Four R table-formatting packages: honest comparison of use-cases

Worked example: an ICER summary table

Using the TB 4-state Markov model's base-case results from Chapters 25 and 27 – incremental cost –$844.14, incremental QALY 0.1440 – the incremental cost-effectiveness ratio and incremental net monetary benefit at a willingness-to-pay of $50,000 per QALY gained are computed directly, showing the arithmetic rather than asserting a pre-formatted number:

```
inc_cost <- -844.14
inc_qaly <- 0.1440
wtp      <- 50000

icer <- inc_cost / inc_qaly          # -844.14 / 0.1440
nmb  <- inc_qaly * wtp - inc_cost    # (0.1440 * 50000) - (-844.14)
```

```
icer   # -5862.083...
nmb    # 8044.14
```

$$\text{ICER} = \frac{-844.14}{0.1440} \approx -5862.08$$

dollars per QALY gained – a negative ICER here signals that New-regimen is *dominant* (cheaper and more effective), so the conventional "cost per QALY gained" framing is reported alongside the dominance conclusion rather than in place of it. The incremental net monetary benefit,

$$\text{NMB} = (0.1440 \times 50000) - (-844.14) = 8044.14$$

dollars, matches the Chapter 25/27 figure of $8,044 and confirms internal consistency.

```
library(gt)

icer_table <- data.frame(
  Arm = c("SoC", "New-regimen", "Incremental"),
  Cost = c(5000.00, 4155.86, -844.14),
  QALY = c(3.2100, 3.3540, 0.1440),
  check.names = FALSE
)

gt(icer_table) |>
  fmt_currency(columns = Cost, currency = "USD") |>
  fmt_number(columns = QALY, decimals = 4) |>
  tab_header(title = "TB model: incremental cost-effectiveness results")
```

How to report this.

Arm	Cost (USD)	QALY
SoC	$5,000.00	3.2100
New-regimen	$4,155.86	3.3540
Incremental	-$844.14	0.1440

ICER = −844.14 / 0.1440 ≈ -$5,862 per QALY gained (New-regimen dominant: cheaper and more effective). Incremental NMB at WTP = $50,000/QALY = (0.1440 × 50,000) − (−844.14) = $8,044.

Table 83: TB 4-state Markov model: base-case ICER and NMB summary (New-regimen vs SoC)

The same underlying data frame renders equally well through the other three packages, with a different emphasis each time. A `gtsummary` rendering of the same comparison would typically start from `gtsummary::tbl_summary()` applied to arm-level patient data rather than an already-aggregated data frame, since its design centre is summarising raw observations by group rather than formatting pre-computed scalars – for a two-row aggregate table like this one, `gt` or `flextable` is the more natural tool. A Word-native version via `flextable` looks like:

```
library(flextable)
```

```
flextable(icer_table) |>
  flextable::colformat_double(j = "Cost", digits = 2) |>
  flextable::colformat_double(j = "QALY", digits = 4) |>
  flextable::set_caption("TB model: incremental cost-effectiveness results") |>
  flextable::autofit()
```

and a quick LaTeX/HTML appendix version via `kableExtra`:

```
library(knitr)
library(kableExtra)

kbl(icer_table, caption = "TB model: incremental cost-effectiveness results",
    digits = 4) |>
  kable_styling(full_width = FALSE, bootstrap_options = "striped")
```

HEOR in practice. Decide the destination format *before* choosing a table package, not after. A table built with `gtsummary`/`gt` for an HTML dashboard and later needed in a Word submission document may need re-rendering through a different code path (`gt`'s Word support is more limited than its HTML/PDF support); if Word is the known final destination, building the table in `flextable` from the start – even for exploratory HTML drafts – avoids a late rewrite.

Common pitfall. None of these four packages validate the *numbers* inside the table – they only format whatever data frame they are given. A sign error upstream in the model code (e.g. accidentally reporting `cost_new - cost_soc` in one script and `cost_soc - cost_new` in another) will render through `gt` or `flextable` just as cleanly as a correct table, with no warning. Always compute the table's input data frame from the same saved model-result object used elsewhere in the report (as in the chunk above, which recomputes ICER and NMB from the same two numbers rather than retyping a previously reported value), so a single upstream fix propagates everywhere automatically.

Difficult terminology: dominant / dominance (health economics). In cost-effectiveness analysis, a strategy is said to "dominate" its comparator when it is both less costly *and* at least as effective – there is no trade-off to weigh, so it is preferred regardless of the willingness-to-pay threshold. This is why the negative ICER in the worked example above is not reported as a literal "cost saving of $5,862 per QALY" in the usual ICER sense; a negative ICER arising from dominance needs the dominance relationship stated explicitly, because the same negative number could instead (in a different scenario) signal the opposite situation – a strategy that is more costly and less effective – where the correct habit is to always check which quadrant of the cost-effectiveness plane the incremental results fall into before quoting an ICER at all.

28.5 Publication-ready graphics (ggplot2 themes, colour-blind-safe palettes)

A figure that looks fine on your laptop screen can fail completely once it is shrunk to fit a two-column journal page, printed in black-and-white, or viewed by a colleague with colour vision deficiency. This section covers three habits that turn an exploratory `ggplot2` plot into one you

can submit to a journal, an HTA body, or a ministry report without last-minute panic: a reusable custom theme, colour palettes that remain distinguishable for colour-blind readers, and saving at the resolution the publisher actually asks for.

A consistent custom theme

Relying on the default `theme_gray()` for every figure in a dossier means every reviewer sees slightly different grid lines, fonts and label sizes depending on which script produced which plot. Building one theme object once and reusing it everywhere removes that inconsistency.

```
library(ggplot2)

theme_heor <- function(base_size = 11) {
  theme_minimal(base_size = base_size) +
    theme(
      panel.grid.minor = element_blank(),
      panel.grid.major = element_line(colour = "grey88", linewidth = 0.3),
      axis.title = element_text(face = "bold"),
      plot.title = element_text(face = "bold", size = rel(1.1)),
      plot.subtitle = element_text(colour = "grey40"),
      legend.position = "bottom",
      legend.title = element_text(face = "bold"),
      strip.background = element_rect(fill = "grey92", colour = NA),
      strip.text = element_text(face = "bold")
    )
}

theme_set(theme_heor())
```

`theme_set()` makes `theme_heor()` the default for every subsequent plot in the session, so individual scripts producing the cost-effectiveness plane, the Markov trace, or the tornado diagram for the TB model all inherit the same typography without repeating the `theme()` block each time. You can still override specific elements per-plot with `+ theme(legend.position = "none")` where needed.

> **HEOR in practice.** Keep `theme_heor()` (or whatever you name it) in a small shared R file (e.g. `R/theme_heor.R`) that every report and figure script `source()`s, rather than pasting the theme definition into each Quarto document. When a journal's house style changes, you edit one file instead of hunting through a dozen report scripts.

Colour-blind-safe palettes

> **Difficult terminology: colour-blind-safe palette.** A colour-blind-safe (or colour-blind-friendly) palette is a set of colours chosen so that the categories they represent remain distinguishable even for readers with the common forms of colour vision deficiency – most often red-green confusion (deuteranopia/protanopia), affecting roughly 1 in 12 men and 1 in 200 women. The default `ggplot2` palette uses hues (red, green, blue) that can become hard to tell apart under exactly this kind of deficiency. A colour-blind-safe palette instead varies lightness and blue-yellow contrast as well as hue, so two lines that look "red vs green" to most viewers still look "dark vs light" to a red-green colour-blind viewer. Think of it like choosing traffic-light colours that differ in brightness and position, not just hue, so the signal survives even if the hue information is lost.

The `viridis` package provides perceptually uniform, colour-blind-safe palettes originally built for `matplotlib` and now widely used in R:

```
library(viridis)

ggplot(markov_trace, aes(x = cycle, y = proportion, colour = state)) +
  geom_line(linewidth = 0.9) +
  scale_colour_viridis_d(option = "D") +   # discrete scale, "viridis" option
  labs(x = "Cycle (months)", y = "Proportion of cohort",
       colour = "Health state",
       title = "Markov cohort trace: New-regimen arm")
```

`scale_colour_viridis_d()` (discrete) and `scale_colour_viridis_c()` (continuous) are the two variants you will use most; the equivalent fill-scale functions are `scale_fill_viridis_d()` and `scale_fill_viridis_c()`.

To check whether an *existing* plot or palette survives a colour-blind simulation rather than build a new palette from scratch, the `colorspace` package offers simulation functions such as `colorspace::simulate_cvd()`, and the `colorblindr` package (an add-on built on top of `colorspace` and `cowplot`, not independently re-verified here against its exact current function signatures) is commonly used for a side-by-side normal-vision-vs-simulated-deficiency comparison grid.

```
library(colorspace)

p <- ggplot(ce_plane, aes(incr_cost, incr_qaly, colour = arm)) +
  geom_point(size = 2) +
  scale_colour_viridis_d()

# quick single-simulation preview of how the plot's colours look
# under a deuteranopia (red-green) colour-vision simulation
simulate_cvd(c("#440154", "#21918c", "#fde725"), deutan(1))
```

Common pitfall. GitHub-only packages (not distributed via CRAN) can be finicky to install in a restricted or offline environment, since they typically require the `remotes` or `devtools` package plus a working internet connection to GitHub rather than a simple `install.packages()` call. If a GitHub-only colour-blind-simulation add-on fails to install, fall back to `colorspace::simulate_cvd()` for a quick simulated-vision preview, or simply default to a known colour-blind-safe palette family (viridis, or the `RColorBrewer` "Dark2"/"Set2" qualitative palettes, which were also designed with colour-vision deficiency in mind) and skip the simulation step rather than blocking your report on a brittle dependency.

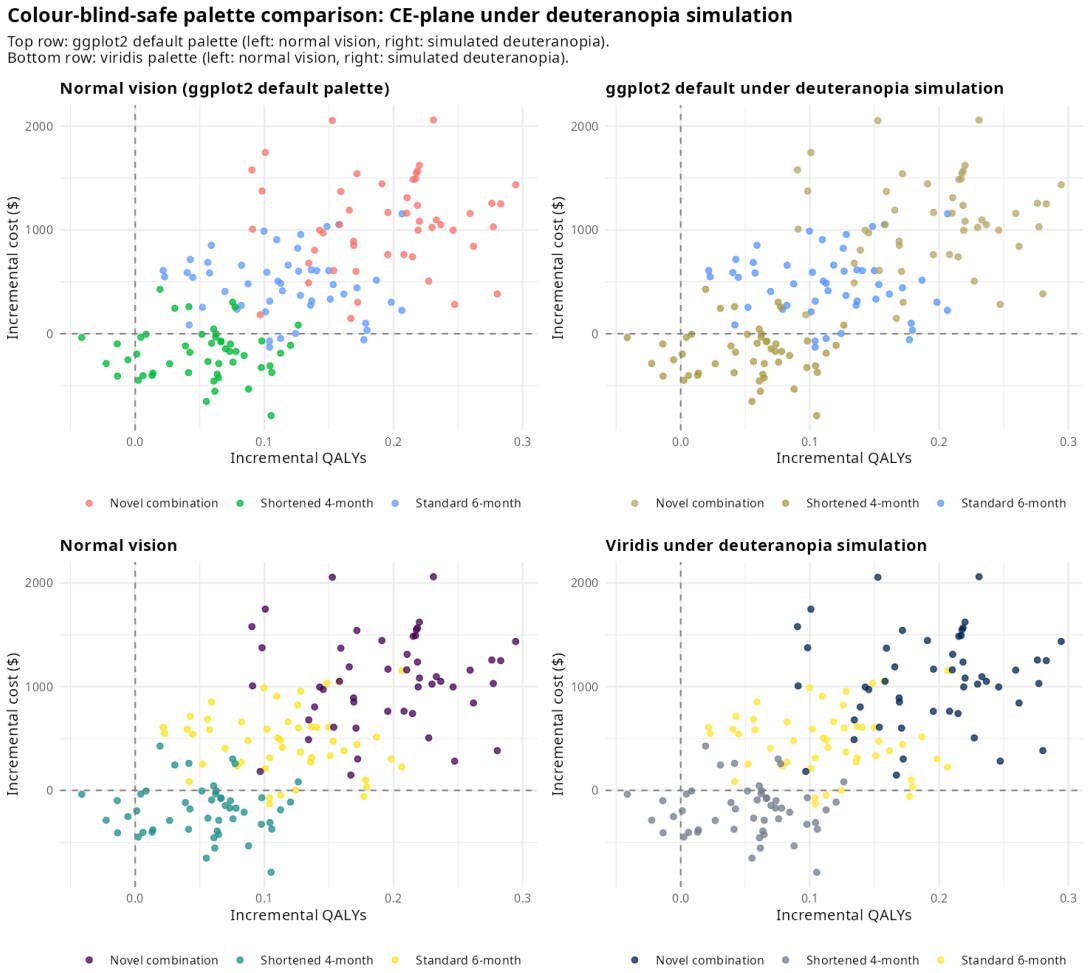


Figure 49: A simulated three-arm cost-effectiveness plane (illustrative data) compared under the ggplot2 default qualitative palette (top row) and the viridis palette (bottom row), each shown under normal vision (left) and a simulated deuteranopia colour-vision deficiency (right, via `colorspace::deutan()`). Under the default palette, "Novel combination" and "Shortened 4-month" become visually indistinguishable once deuteranopia is simulated; all three arms remain distinguishable under viridis.

HEOR in practice. Never rely on colour alone to distinguish lines or bars in a figure that may be printed in greyscale or viewed by a colour-blind reviewer. Pair colour with a second visual channel – line type (`linetype =`), point shape (`shape =`), or direct text labels on the lines – so the figure survives both greyscale printing and colour vision deficiency simultaneously.

Saving at print resolution

A figure that looks sharp in the RStudio plot pane can look blurry once dropped into a Word or PDF submission document, because the viewer and the file are rendered at different resolutions. `ggplot2::ggsave()` takes explicit `width`, `height`, `units` and `dpi` arguments so you control the physical print size directly, rather than hoping the on-screen preview matches what a printer or PDF embed will produce.

```
ggsave(
  filename = "figures/markov_trace_newregimen.png",
```

```
  plot     = last_plot(),
  width    = 16, height = 10, units = "cm",
  dpi      = 300
)

# vector output for a journal that accepts PDF or EPS figures
ggsave("figures/markov_trace_newregimen.pdf",
       width = 16, height = 10, units = "cm")
```

A `dpi` of 300 is a common minimum for raster (PNG/TIFF) figures in print journals; some journals request 600 dpi for line art. When a journal accepts vector formats (PDF, EPS, SVG) there is no resolution to set at all – the figure is defined by mathematical paths rather than a pixel grid, so it scales cleanly to any size, which is generally the safer choice for simple line and bar charts.

Common pitfall. Fonts and point sizes that look proportionate on a 13-inch laptop screen can look tiny or enormous once the same plot is saved at a fixed physical `width`/`height` in centimetres or inches. Always open the saved file (not the RStudio plot pane) at its intended final size before submission – a 16 cm-wide figure with `base_size = 11` text in the theme can render axis labels that are barely legible once printed at that size. Adjust `base_size` in your theme function, not by resizing the canvas after the fact.

Agency perspective. Most peer-reviewed health economics journals (e.g. *Value in Health*, *PharmacoEconomics*) and submission portals for HTA bodies such as NICE specify minimum figure resolution and accepted file formats in their author/submission guidelines – check the specific target journal's or agency's current instructions before finalising `dpi` and file format, since requirements (300 dpi raster vs vector PDF/EPS, RGB vs CMYK colour space) vary by venue and are revised from time to time.

28.6 CHEERS 2022 reporting checklist implementation

Difficult terminology: checklist-based reporting. Checklist-based reporting means a journal, funder or agency publishes a fixed list of items that a report of a particular study type must address (not necessarily in that order), and authors are expected to state explicitly where in their manuscript each item is covered – often in a companion table. It exists because health economic evaluations combine clinical, epidemiological, economic and modelling content from many sub-disciplines, and it is easy to omit something reviewers consider essential (e.g. the discount rate used, or how uncertainty was characterised) without anyone noticing until late in peer review. A checklist is like a pre-flight checklist for a pilot: not a guarantee the flight will go well, but a guarantee that the obviously necessary steps were not simply forgotten.

CHEERS 2022 (Consolidated Health Economic Evaluation Reporting Standards, Husereau, Drummond, Augustovski, et al., *Value in Health* 2022;25(1):10-31) is the current consensus reporting checklist for health economic evaluations – cost-effectiveness analyses, cost-utility analyses, cost-minimisation and cost-benefit analyses, whether trial-based or model-based (such as the TB Markov cohort model used throughout this book). It updates the original 2013 CHEERS statement and comprises 28 items organised under broad headings covering the title and abstract, introduc-

tion, methods, results, discussion, and other relevant information (including funding and conflicts of interest).

Rather than reproduce all 28 item texts here – which risks mis-stating wording not independently re-verified against the published checklist – this section focuses on the items most relevant to content already built earlier in this book, and shows a general pattern for tracking checklist compliance programmatically. For the authoritative item wording, authors should consult the published CHEERS 2022 statement directly before submission.

A few items this chapter is confident about by number and general content, because they map directly onto work done in earlier chapters:

- **Item 1** asks that the title identify the study as an economic evaluation and name the interventions compared – directly satisfied by a title such as "Cost-effectiveness of a New TB Treatment Regimen versus Standard of Care: A Markov Cohort Model".
- **Item 15**, on synthesis methods (how estimates of effect, such as treatment efficacy or transition probabilities, were derived or pooled from the literature), is exactly the evidence-synthesis work covered in Chapter 11.
- **Items 20 and 26**, covering statistical/regression analysis methods and the corresponding results reporting, correspond to the regression modelling and reporting conventions covered in Chapter 16.

> **Agency perspective.** CHEERS 2022 (Husereau D, Drummond M, Augustovski F, et al. Consolidated Health Economic Evaluation Reporting Standards 2022 (CHEERS 2022) Statement. *Value in Health* 2022;25(1):10-31) is the checklist most HTA bodies and health economics journals now expect authors to complete and submit alongside a cost-effectiveness manuscript, typically as a supplementary "CHEERS checklist" table stating the manuscript section/page where each item is addressed. Always check the submission target's current instructions for authors, since some journals mandate CHEERS compliance explicitly while others merely recommend it.

Building a checklist-tracking table in R

Rather than ticking items off by hand in a spreadsheet that drifts out of sync with the report as it evolves, build the checklist as a data frame alongside the rest of the analysis pipeline, with one row per item and a column recording which report section addresses it. This is a mapping/organising table of checklist references rather than a freshly computed analytic result, so it is presented as a `ref-table` rather than a `report-table`.

```
library(tibble)
library(dplyr)

cheers_checklist <- tribble(
  ~item, ~topic,                                  ~report_section,
  1,     "Title identifies economic evaluation",   "Title page",
  2,     "Structured abstract",                     "Abstract",
  6,     "Study population",                        "Methods - Target population",
  9,     "Model-based economic evaluation details", "Methods - Markov model
structure",
  13,    "Discount rate",                           "Methods - Discounting",
  15,    "Synthesis of effect estimates",           "Methods - Evidence synthesis
```

```
(ch. 11)",
  18,    "Characterising heterogeneity",             "Methods - Subgroup
analysis",
  20,    "Statistical analysis methods",             "Methods - Regression
analysis (ch. 16)",
  23,    "Base-case results",                        "Results - Base-case ICER",
  24,    "Uncertainty characterisation",             "Results - PSA / sensitivity
analysis",
  26,    "Statistical analysis results",             "Results - Regression output
(ch. 16)",
  28,    "Funding source",                           "Declarations"
)

# flag items not yet mapped to any section -- a living to-do list
cheers_checklist %>%
  filter(report_section == "") %>%
  select(item, topic)
```

This `cheers_checklist` table is deliberately a small illustrative subset of the full 28 items (not a complete reproduction of the published checklist), intended to show the pattern: each row is one checklist item, and the `report_section` column is kept up to date as the Quarto report (built with the parameterised-rendering pattern from SS28.2) evolves, so the checklist never falls out of sync with the manuscript it describes.

Item	Topic	Addressed in
1	Title identifies economic evaluation	Title page
15	Synthesis of effect estimates	Methods – Evidence synthesis, Chapter 11
20	Statistical analysis methods	Methods – Regression analysis, Chapter 16
24	Uncertainty characterisation	Results – PSA / sensitivity analysis
26	Statistical analysis results	Results – Regression output, Chapter 16
28	Funding source	Declarations

Illustrative subset only – consult the published CHEERS 2022 statement (Husereau et al., 2022) for the complete 28-item list and exact wording.

Table 84: Illustrative subset of CHEERS 2022 items cross-referenced to report sections and earlier chapters

HEOR in practice. Render the checklist table itself from the `cheers_checklist` data frame inside the Quarto report (e.g. as a `gt` or `kableExtra` table, following the publication-table conventions from SS28.4), rather than maintaining it as a separate static document. A checklist that lives in the same `.qmd` source as the analysis gets regenerated – and so re-verified – every time the report is rendered.

Common pitfall. A checklist item marked "addressed" in the `report_section` column is not the same as an item addressed *well*. Treat `cheers_checklist` as a coverage tracker that catches omissions, not a substitute for a careful read-through against the actual CHEERS 2022 item wording before submission – reviewers will judge adequacy, not just presence.

28.7 Automating dossier and submission appendices

Difficult terminology: dossier. In HTA and regulatory contexts, a dossier is the full submission package an agency requires to evaluate a health technology – typically far longer than a journal manuscript, comprising a main clinical and economic summary plus a series of technical appendices (model documentation, full search strategies, statistical analysis plans, sensitivity analysis results, and so on). Think of it as the difference between a short magazine article and the full case file behind it: the dossier is the case file, assembled from many separately authored sections.

Difficult terminology: child document. A child document is a smaller Quarto (`.qmd`) or R Markdown (`.Rmd`) file that is included inside a larger "parent" document at render time, rather than copy-pasted into it. The parent document stays short and acts as a table of contents that pulls each section in; each child document can still be edited, tested, and even rendered on its own. This mirrors how this very book's parent `book.typ` file `#includes` each chapter file in Typst – the structure here is the same idea applied to Quarto reports.

A full HTA submission dossier for the TB New-regimen model might need a clinical-effectiveness summary, a model-structure appendix, a cost-inputs appendix, a sensitivity-analysis appendix and a budget-impact appendix – five or more sections, each with its own tables and figures, that together could run to dozens of pages. Writing all of that as one monolithic `.qmd` file quickly becomes unmanageable: section authors step on each other's edits, and a typo in one appendix forces a re-render of the entire document just to check it. Quarto's child-document mechanism, combined with the parameterised-rendering pattern introduced in SS28.2, lets you assemble the dossier from independently maintained component files instead.

Structuring the dossier as child documents

Each appendix lives in its own `.qmd` file, and a parent document includes them with the `{{< include >}}` shortcode:

```
---
title: "HTA Submission Dossier -- New TB Regimen vs Standard of Care"
format: pdf
params:
  country: "India"
  perspective: "payer"
---

# Executive summary

{{< include _executive_summary.qmd >}}
```

```
# Appendix A: Clinical effectiveness evidence

{{< include _appendix_clinical.qmd >}}

# Appendix B: Markov model structure and assumptions

{{< include _appendix_model_structure.qmd >}}

# Appendix C: Cost inputs

{{< include _appendix_costs.qmd >}}

# Appendix D: Sensitivity and scenario analyses

{{< include _appendix_sensitivity.qmd >}}
```

Each `_appendix_*.qmd` child file is a complete, independently renderable Quarto document in its own right (so a single analyst can preview just their section while drafting it), but when included via `{{< include >}}` it contributes its content into the parent's single combined render – one shared bibliography, one shared set of cross-references, one final PDF or Word file.

Because the parent document carries the same `params:` block introduced for single reports in SS28.2, parameters set once at the top (country, payer perspective, willingness-to-pay threshold, etc.) are visible to every included child document that reads `params$...`, so the whole dossier – appendices included – regenerates consistently for a different country or perspective without editing each child file by hand.

```
quarto::quarto_render(
  input = "dossier_parent.qmd",
  execute_params = list(country = "India", perspective = "societal"),
  output_file = "dossier_india_societal.pdf"
)
```

HEOR in practice. Keep each appendix child document runnable and testable on its own (its own `params:` defaults, its own ability to render standalone) even though it is normally rendered as part of the parent. This lets whoever owns the cost-inputs appendix iterate quickly without re-rendering the entire multi-appendix dossier every time they tweak a unit cost.

Common pitfall. Child documents included with `{{< include >}}` share the parent document's R session state, including figure/table numbering and any variables defined in earlier-included children. If two appendices happen to define a variable or object with the same name (e.g. both calling their main data frame `df`), the second include can silently overwrite the first. Namespacing objects per appendix (`clinical_df`, `cost_df`, `sens_df`, ...) avoids this class of bug.

Agency perspective. NICE's Technology Appraisal process requires a company submission comprising a main document plus a detailed economic model and supporting appendices (see NICE's "Guide to the processes of technology appraisal" and the associated submission

templates); PBAC in Australia and similarly structured HTA bodies elsewhere have comparable multi-section dossier requirements. Checking the specific agency's current submission template structure before deciding how many appendix child documents to create – and in what order – saves rework later, since agencies often mandate a fixed section ordering.

28.8 Automating routine reports across programmes or districts

The parameterised single-report pattern from SS28.2 renders one report for one set of parameter values on request. Many real reporting workloads are not "render one report" but "render the same report design once per unit" – once per TB programme district, once per quarter, once per facility. This section covers looping that rendering step across many parameter sets, and briefly touches on scheduling the whole pipeline to run unattended.

Agency perspective. The district-level looping scenario below is a fictional illustrative example for this book, not a description of an actual published NTEP (National TB Elimination Programme) reporting workflow; it is included because looping a parameterised Markov-model report across administrative units is a realistic shape for routine TB programme reporting in practice.

Looping parameterised rendering across districts

Suppose the same New-regimen-vs-SoC Markov cohort report from earlier chapters needs to be produced once for each of several illustrative Indian TB programme districts, each with its own local case counts and perhaps slightly different baseline transition probabilities estimated from local notification data. Build a small data frame of per-district parameters, one row per report to generate, and loop `quarto::quarto_render()` over it.

```
library(purrr)
library(quarto)

district_params <- tibble::tribble(
  ~district,    ~baseline_notifications, ~output_file,
  "Nashik",      2400,                    "report_nashik.pdf",
  "Nagpur",      3100,                    "report_nagpur.pdf",
  "Thane",       4800,                    "report_thane.pdf",
  "Pune Rural",  2950,                    "report_pune_rural.pdf"
)

# purrr::pwalk() calls a function once per row, passing each column as a
# named argument -- here, into quarto_render()'s params and output_file
pwalk(district_params, function(district, baseline_notifications, output_file) {
  quarto::quarto_render(
    input = "district_tb_report.qmd",
    execute_params = list(
      district = district,
      baseline_notifications = baseline_notifications
    ),
    output_file = output_file
```

```
  )
})
```

`purrr::pwalk()` is the multi-argument sibling of `purrr::walk()`: `walk()` calls a function once per element of a single vector/list purely for its side effect (here, writing a rendered report file to disk) and returns its input invisibly; `pwalk()` does the same but draws each call's arguments from the corresponding row of a data frame, matching columns to the function's argument names. Both are used here instead of `map()`/`pmap()` because the point of the loop is the side effect of writing files, not collecting a return value – using `walk`/`pwalk` rather than `map`/`pmap` signals that intent to anyone reading the code later. A plain `for` loop over `seq_len(nrow(district_params))` with the equivalent `quarto_render()` call inside it works identically and is a perfectly reasonable alternative if `purrr`-style functional loops feel unfamiliar.

```
# equivalent for-loop version
for (i in seq_len(nrow(district_params))) {
  row <- district_params[i, ]
  quarto::quarto_render(
    input = "district_tb_report.qmd",
    execute_params = list(
      district = row$district,
      baseline_notifications = row$baseline_notifications
    ),
    output_file = row$output_file
  )
}
```

Common pitfall. Rendering many reports in a tight loop means a single malformed parameter value (a missing district name, a notification count accidentally read in as text) can abort the whole batch partway through, leaving you unsure which reports actually completed. Wrapping each render call in `tryCatch()` and logging successes/failures to a small results data frame lets the batch continue past one bad row and gives you a clear list of which districts need re-running, rather than guessing from a half-finished output folder.

Scheduling the pipeline for recurring reporting

A one-off script that loops over districts is useful on demand, but routine reporting (monthly district TB reports, say) usually needs to run unattended on a schedule. Two common approaches, at different levels of sophistication:

- **A scheduled job** (a `cron` job on Linux/macOS, or Windows Task Scheduler) simply runs an R or shell script – containing the `pwalk()`/for-loop rendering code above – at fixed times, e.g. the first day of every month. This is the simplest option and is adequate when the pipeline is small and failures can be caught by checking output files or an emailed log afterwards.

Difficult terminology: targets pipeline. The `targets` R package is a pipeline tool that tracks the dependencies between steps of an analysis (load data -> clean data -> fit model -> render report) and only re-runs the steps whose inputs have actually changed since the last run, skipping the rest. A "targets pipeline" means the whole workflow – data loading, model fitting, report rendering – is declared as a sequence of named, interdependent steps in a `_targets.R` file, run via `targets::tar_make()`. The analogy is a smart build system (like `make`

for software): if only this month's notification data changed, it re-renders only the reports that actually depend on that data, rather than redoing every step for every district from scratch.

- **A `targets` pipeline** is worth the extra setup once the reporting workflow has enough interdependent steps that you want to avoid redundant recomputation – for instance, if fitting the underlying Markov model parameters is itself slow, and only the final report-rendering step needs to re-run when a single district's raw notification file is updated. A `_targets.R` file would declare one target for the raw data, one for the fitted model, and (using the `tarchetypes` package's rendering/branching helpers) one target per rendered report, and `targets::tar_make()` would then only re-render the reports whose upstream data actually changed.

```
# sketch of a _targets.R structure -- illustrative, not a tested pipeline
library(targets)
library(tarchetypes)

list(
  tar_target(district_data, load_district_notifications()),
  tar_target(fitted_model, fit_markov_model(district_data)),
  tar_render(district_report, "district_tb_report.qmd")
)
```

Common pitfall. The `_targets.R` sketch above illustrates the general shape of a targets pipeline rather than a tested, complete pipeline definition – `tarchetypes`'s rendering and dynamic-branching helpers have specific argument requirements that are easy to get wrong on a first attempt. Treat this as a starting point to adapt and test against the `targets`/`tarchetypes` package documentation, not code to copy verbatim into production.

HEOR in practice. Start with the simplest scheduling mechanism that meets the actual recurrence requirement – a monthly `cron` job calling a plain R script is often entirely sufficient for routine district reporting. Reach for `targets` only once you notice real pain from redundant recomputation (e.g. refitting a slow model every month even when only one district's input data changed), rather than adopting it pre-emptively for a workflow that does not yet need it.

Chapter 29. Interactive Models and Web Applications with Shiny

Learning objectives

- Explain Shiny's `ui`/`server` architecture and reactive programming model, and build a minimal reactive cost-effectiveness calculator.
- Design a Shiny interface for non-programmer reviewers: layout choice, widget choice, plain-language labelling, tooltips and progressive disclosure.
- Compare plain Shiny modules, `golem` and `rhino` as ways of structuring a growing app, and choose the right level of investment for a given app's lifespan.
- Connect a Shiny front end to an underlying cost-effectiveness model, separating cheap continuously-updating reactives from expensive user-triggered PSA runs.
- Build a `downloadHandler()` that drives the same parameterised Quarto report pipeline from Chapter 28, and implement a basic audit trail of user interactions.
- Compare shinyapps.io, Posit Connect and Docker/self-hosted deployment routes, and choose among them based on an organisation's IT and data-governance constraints.
- Apply input validation (`req()`/`validate()`/`need()`) and authentication appropriately, and explain what NICE's Technology Appraisal process expects of a transparent, reviewable model.
- Assemble the chapter's techniques into a sketch of a transparent, reviewable, submission-ready Shiny app for an ERG-style review.

Chapter 28 closed with reports that regenerate themselves from source — reproducible, but still fundamentally static documents, produced once and handed over. This chapter adds a second channel: a live, interactive front end that lets a reviewer change a parameter and watch the model respond immediately, without filing a request and waiting for a re-run. The two are not competitors. As this chapter will show repeatedly, the right design keeps them connected — a Shiny app's "download the full report" button should trigger the exact same Quarto pipeline built in Chapter 28, so that what a reviewer sees on screen and what they take away as a citable document are guaranteed to match.

29.1 Shiny fundamentals for model front-ends

Every health-economic model eventually needs a front door — a way for someone who is not going to open RStudio to turn your dials and see what happens to the ICER. In this book's running TB Markov example (Chapters 17-27) that "someone" is usually a clinical lead, a payer reviewer, or an HTA evaluator checking your sensitivity analysis live during technical engagement. Shiny is R's framework for building that front door: a web application whose logic is still ordinary R code.

A Shiny app has exactly two conceptual halves, conventionally split into two objects:

- `ui` — describes what the page *looks like*: panels, sliders, buttons, plot placeholders.
- `server` — describes what happens *when inputs change*: a function with arguments `input`, `output`, and (optionally) `session`.

What makes Shiny distinctive is not these two halves but the glue between them, called reactive programming.

> **Difficult terminology: reactive programming.** A programming style in which you declare *relationships* between values rather than a sequence of steps. In ordinary R you write `icer <- cost / qaly` once, and if `cost` later changes, `icer` is stale until you re-run the line. In Shiny, if you declare that same relationship as a reactive expression, Shiny automatically re-runs it whenever `cost` changes — rather like a spreadsheet cell with a formula in it that recalculates when you edit an input cell.

Every Shiny input widget (a slider, a numeric box, a dropdown) creates a reactive value, accessible in the server function as `input$treatment_cost`, `input$qaly_gain`, and so on. You never reassign these directly; they update automatically whenever the user moves the control.

> **Difficult terminology: reactive expression.** Code wrapped in `reactive({...})` that Shiny treats like a cached, auto-updating calculation — similar to a view in SQL that recomputes only when the underlying tables it depends on actually change. You call it like a function, e.g. `icer()`, and Shiny only re-executes the body when one of the reactive values it reads (e.g. `input$treatment_cost`) has changed since the last call.

> **Difficult terminology: observer.** Code wrapped in `observeEvent({...})` or `observe({...})` that *runs for its side effects* — writing a log message, saving a file, popping up a warning — rather than returning a value for display. Where a reactive expression is "a formula that produces a number," an observer is "an action that happens when something changes."

Here is a minimal, complete toy app wrapping a two-input, one-output ICER calculator. It compares a new treatment against a fixed comparator (cost $2,000, already on formulary) and lets the reviewer vary the new treatment's cost and incremental QALY gain.

```
library(shiny)

comparator_cost <- 2000  # fixed cost of standard of care

ui <- fluidPage(
  titlePanel("Toy ICER calculator"),
  sidebarLayout(
    sidebarPanel(
      numericInput("treatment_cost", "New treatment cost ($)",
                   value = 5000, min = 0, step = 100),
      numericInput("qaly_gain", "Incremental QALYs gained",
                   value = 0.15, min = 0, step = 0.01)
    ),
    mainPanel(
      h4("Incremental cost-effectiveness ratio"),
      textOutput("icer_text")
    )
  )
)
```

```
server <- function(input, output, session) {

  # Reactive expression: recomputes only when its inputs change
  incremental <- reactive({
    list(
      d_cost = input$treatment_cost - comparator_cost,
      d_qaly = input$qaly_gain
    )
  })

  icer <- reactive({
    inc <- incremental()
    inc$d_cost / inc$d_qaly
  })

  output$icer_text <- renderText({
    paste0("$", format(round(icer(), 0), big.mark = ","), " per QALY gained")
  })
}

shinyApp(ui, server)
```

With the defaults shown (treatment cost $5,000, comparator $2,000, QALY gain 0.15), the incremental cost is $3,000 and the ICER is $3,000 / 0.15 = $20,000 per QALY — move either slider and the displayed figure updates immediately, with no explicit "refresh" step anywhere in the code.

HEOR in practice. Keep the server function's reactive graph shallow for a first app: one reactive expression per intermediate quantity (`incremental()`, then `icer()`) is easier for a future you — or a collaborator — to debug than one giant reactive block computing everything at once. Reactive expressions are also the natural unit to print with `print(icer())` inside `observe()` while developing, since you can inspect intermediate values without touching the UI.

Common pitfall. Do not call `input$treatment_cost` or any reactive expression *outside* a reactive context (i.e. outside `reactive({})`, `render*({})`, or `observe({})`). Shiny will throw `Error: Operation not allowed without an active reactive context` — a common early stumbling block that simply means "you tried to read a live value from plain, non-reactive R code."

29.2 Designing a user interface for non-programmers

A Shiny app built for yourself and one built for a non-technical HTA reviewer are different products even if the underlying model is identical. The reviewer is not trying to learn R; they are trying to answer "does this result hold up if I change this one assumption?" as quickly as possible, usually live on a call during technical engagement. Every design choice in this section is in service of that single goal.

Layout: start simple, escalate only if needed. `sidebarLayout()` (inputs on the left, results on the right) is the right default for a single-screen tool with a handful of parameters — it is what

reviewers expect from a "calculator." Once you have more than one result view (say, a base-case summary, a CE plane, and a CEAC), move to a multi-page layout with `navbarPage()` or `tabsetPanel()`, so each result lives on its own tab rather than competing for space:

```
library(shiny)
library(bslib)

ui <- page_navbar(
  title = "TB treatment cost-effectiveness explorer",
  theme = bs_theme(version = 5, bootswatch = "flatly"),
  nav_panel("Inputs & summary", icon = icon("sliders"),
    sidebarLayout(
      sidebarPanel(
        sliderInput("tx_cost", "Annual treatment cost ($)",
                    min = 500, max = 3000, value = 1200, step = 50),
        selectInput("regimen", "Treatment regimen",
                    choices = c("Standard 6-month" = "std",
                                "Shortened 4-month" = "short"))
      ),
      mainPanel(textOutput("icer_summary"))
    )
  ),
  nav_panel("Cost-effectiveness plane", icon = icon("chart-scatter"),
            plotOutput("ce_plane")),
  nav_panel("CEAC", icon = icon("chart-line"),
            plotOutput("ceac_plot"))
)
```

`bslib::page_navbar()` builds on the same Bootstrap theming system used throughout the tidyverse's web tooling and renders a noticeably more polished, modern interface than classic `fluidPage()` with almost no extra code — worth the small dependency for anything a reviewer outside your team will see.

Widgets: match the control to the mental model. `sliderInput()` communicates a bounded, continuous range intuitively (e.g. discount rate, WTP threshold); `numericInput()` is better when precision matters more than exploration (e.g. an exact unit cost from a costing study); `selectInput()` is right for a small set of named, mutually-exclusive options (e.g. choice of regimen) — never make a reviewer type a code like `"std"` when a label like "Standard 6-month" will do.

Labels: write for a clinician, not a modeller. "Discount rate" beats "r_d"; "Willingness-to-pay threshold ($ per QALY)" beats "lambda." If a parameter cannot be labelled clearly in under eight words, it probably needs a tooltip rather than a longer label:

```
tooltip(
  sliderInput("wtp", "Willingness-to-pay threshold", 20000, 100000, 50000),
  "The maximum amount a payer is willing to spend to gain one additional
   quality-adjusted life year (QALY). NICE typically references
   £20,000-£30,000 per QALY; other agencies use different thresholds."
)
```

`bslib::tooltip()` (or the older `shinyBS::bsTooltip()`) attaches a small hover-triggered explanation to any UI element without cluttering the visible layout — the Shiny equivalent of a cell comment in a spreadsheet.

Progressive disclosure. Not every parameter belongs on the main screen. Structural assumptions, PSA distribution choices, and rarely-touched calibration constants belong behind a collapsible "Advanced settings" panel (`bslib::accordion()` or `shiny::conditionalPanel()`), opened only on request:

```
accordion(
  accordion_panel(
    "Advanced model settings",
    numericInput("psa_draws", "Number of PSA iterations", 1000),
    numericInput("half_cycle_corr", "Half-cycle correction", 0.5)
  )
)
```

Common pitfall. The single most common mistake in reviewer-facing Shiny apps is exposing *every* model parameter on one screen because it was easy to add another `sliderInput()`. A non-technical reviewer faced with twenty simultaneous sliders cannot tell which ones matter, and the app stops looking like a trustworthy decision tool and starts looking like the guts of a spreadsheet. Default to showing the handful of parameters that drive the policy question (price, regimen, WTP threshold) and move everything else behind progressive disclosure — you can always add a link-out to the full parameter table for technical appendices.

Agency perspective. Several HTA bodies now expect submitted models to be usable, not merely inspectable. Australia's PBAC Guidelines (Version 5.0, 2016), in the section on economic evaluation, require sponsors to submit "a fully executable electronic model" that evaluators can run and interrogate themselves rather than a static report of results — a requirement a well-designed Shiny front-end satisfies directly, since an ERG or PBAC evaluator can change the WTP threshold or regimen live and see the ICER update, instead of requesting a re-run from the sponsor during technical engagement.

29.3 Modular Shiny with `golem` and `rhino`

A single-file `app.R` is fine for the toy calculator in Section 29.1, but the TB model app in this book has multiple tabs, several render functions, and inputs that interact across tabs (regimen choice affects both the cost-effectiveness plane and the CEAC). At that scale, copy-pasting `sliderInput("tx_cost", ...)` and its matching server logic across tabs becomes unmanageable, and the natural fix is the same one you already use for R functions: break the app into reusable, independently testable pieces, called Shiny modules.

Difficult terminology: Shiny module. A self-contained pair of functions — one for UI, one for server logic — that bundles a piece of an app (say, "the PSA results panel") so it can be reused and tested like any other R function, instead of being tangled into one giant `ui`/`server` pair. Modules are created with `moduleServer()` for the server half and a plain function returning UI elements for the UI half.

```
# A module for a single cost-effectiveness summary box
ceSummaryUI <- function(id) {
```

```
  ns <- NS(id)
  tagList(
    h4("Incremental results"),
    textOutput(ns("icer_text"))
  )
}

ceSummaryServer <- function(id, incremental_results) {
  moduleServer(id, function(input, output, session) {
    output$icer_text <- renderText({
      res <- incremental_results()
      paste0("ICER: $", format(round(res$icer, 0), big.mark = ","))
    })
  })
}
```

Difficult terminology: namespacing. The mechanism (`NS(id)` above) that prefixes every input/output ID inside a module with a unique identifier, so that two copies of the same module on one page — e.g. a CE-summary box for the "standard regimen" scenario and another for the "shortened regimen" scenario — don't collide by both trying to use an ID like `icer_text`. It is conceptually the same problem a SQL table alias solves when joining a table to itself.

Once an app grows past a handful of modules, two competing packaged conventions for organizing the whole project are widely used in the R community: `golem` and `rhino`. Treat the specifics below as illustrative of each package's philosophy rather than exact, version-pinned syntax — check the current documentation for the precise function signatures before relying on them in production, since both packages have evolved their APIs across versions.

Difficult terminology: golem. An R package (from ThinkR) for structuring a Shiny app as if it were an R *package*: your app's functions live in `R/`, are documented with roxygen comments, and are tested with `testthat`, exactly like the modelling functions from Chapter 27. `golem::create_golem()` scaffolds the package skeleton, and helpers such as `golem::add_module()` generate the boilerplate UI/server pair for a new module.

Difficult terminology: rhino. A newer, more opinionated framework from Posit/Appsilon for production Shiny apps, which enforces a stricter folder structure, bundles its own linting and automated testing conventions, and — distinctively — integrates a JavaScript/Sass build pipeline so custom front-end styling and behaviour are first-class, version-controlled parts of the app rather than inline `tags$script()` calls scattered through UI code.

Difficult terminology: hot-reloading. A development convenience, present to varying degrees in `golem` and `rhino` tooling, where saving a changed source file automatically refreshes the running app in your browser without you manually re-launching it — similar to how editing a Shiny `app.R` under `shiny::runApp()` with auto-reload enabled saves you from repeatedly clicking "Run App."

For the TB model app specifically, the right level of investment depends on audience and lifespan. An app built for one HTA submission, shown to an ERG on a handful of calls, rarely justifies `golem`'s or `rhino`'s full package scaffolding — plain Shiny modules (`moduleServer()` + `NS()`) usually suffice, and keep the barrier to entry low for collaborators who know R but not package development. An app intended to be maintained across multiple submissions, countries, or technology appraisals over several years is a better candidate for the discipline both frameworks impose.

Dimension	Plain Shiny modules	`golem`	`rhino`
Setup overhead	Minimal	Moderate (package scaffold)	Higher (build tooling)
Best for	Single-submission apps, small teams	Apps reused across years / submissions	Apps needing custom JS/CSS, larger dev teams
Testing convention	Ad hoc `testthat` if added	`testthat` built into package structure	Enforced test + lint conventions
Front-end styling	Inline CSS / `tags$script()`	Standard R package assets	First-class Sass/JS build pipeline
Learning curve for a non-SE R user	Low	Moderate (package dev concepts)	Higher (JS build chain)

Table 85: Comparing plain Shiny modules, `golem`, and `rhino`

HEOR in practice. Whichever structure you choose, keep the model engine itself (the Markov transition functions and PSA draws from Chapters 17-26) as ordinary, package-independent R functions with no Shiny dependency at all. Modules, `golem`, and `rhino` are all about organizing the *app layer*; none of them should dictate how the cost-effectiveness model is written. This separation is what makes Section 29.4's testing story possible.

29.4 Linking Shiny to the model engine and PSA outputs

Everything in Sections 29.1-29.3 has been about the app's surface. This section connects that surface to the actual TB Markov model and its PSA output — the same base-case results carried through Chapters 17-27: an incremental cost of approximately -$844.14 (the new regimen is cost-saving), an incremental QALY gain of approximately 0.1440, and, at a willingness-to-pay threshold of $50,000 per QALY, an incremental net monetary benefit of approximately $8,044 (NMB = $50,000 x 0.1440 - (-$844.14) = $7,200 + $844.14 = $8,044.14).

The guiding principle is the one flagged at the end of Section 29.3: the model engine is a set of plain R functions, independently unit-tested with `testthat` as in Chapter 27, and Shiny's job is only to call those functions in response to user input and display what comes back. Two reactive patterns handle the "when does it recompute" question differently depending on cost.

Cheap, continuously-updating reactives. If a quantity is cheap to compute — say, recalculating the deterministic ICER from the base-case transition probabilities whenever the discount rate

changes — a plain `reactive({})` is appropriate, because it is fine for it to re-run on every slider tweak:

```
# run_markov_model() and compute_icer() are the pure functions from Ch. 17-20
deterministic_results <- reactive({
  run_markov_model(
    discount_rate = input$discount_rate,
    regimen       = input$regimen
  )
})
```

Expensive, user-triggered reactives. Running a full PSA — say 1,000 or 10,000 Monte Carlo draws through the Markov model, as built up across Chapters 23-26 — is not something you want firing every time a reviewer nudges a slider. Instead, tie it to an explicit `actionButton()` using `eventReactive()`, which only recomputes when its named event (the button) fires, regardless of how many other reactive values have changed in the meantime:

```
ui <- page_navbar(
  title = "TB model PSA explorer",
  nav_panel("PSA results",
    sidebarLayout(
      sidebarPanel(
        numericInput("n_psa", "Number of PSA iterations", 1000),
        numericInput("wtp", "WTP threshold ($/QALY)", 50000),
        actionButton("run_psa", "Run model", class = "btn-primary")
      ),
      mainPanel(
        plotOutput("ce_plane"),
        plotOutput("ceac_plot")
      )
    )
  )
)

server <- function(input, output, session) {

  psa_results <- eventReactive(input$run_psa, {
    # run_psa() is the pure PSA engine built in Ch. 23-26;
    # it returns a data frame with one row per Monte Carlo draw
    run_psa(n_draws = input$n_psa, wtp = input$wtp)
  })

  output$ce_plane <- renderPlot({
    req(psa_results())
    draws <- psa_results()
    ggplot(draws, aes(x = d_qaly, y = d_cost)) +
      geom_point(alpha = 0.3) +
      geom_hline(yintercept = 0, linetype = "dashed") +
      geom_vline(xintercept = 0, linetype = "dashed") +
      labs(x = "Incremental QALYs", y = "Incremental cost ($)",
           title = "Cost-effectiveness plane")
  })

  output$ceac_plot <- renderPlot({
```

```
    req(psa_results())
    ceac_data <- build_ceac(psa_results(), wtp_range = seq(0, 100000, 1000))
    ggplot(ceac_data, aes(x = wtp, y = prob_cost_effective)) +
      geom_line() +
      labs(x = "Willingness-to-pay threshold ($/QALY)",
           y = "Probability cost-effective",
           title = "Cost-effectiveness acceptability curve")
  })
}
```

Until the reviewer clicks "Run model," `psa_results()` has not been evaluated, so `req(psa_results())` inside each `renderPlot()` simply holds the plots blank rather than erroring — a standard Shiny idiom for "wait until this upstream reactive has a value." Once the button is clicked, both plots redraw from the same single `eventReactive()` call, so the CE plane and the CEAC are always showing draws from the same PSA run rather than two independently-triggered, potentially mismatched runs.

> **HEOR in practice.** Resist the temptation to write model logic directly inside `reactive({})` or `eventReactive({})` blocks. Keep `run_markov_model()`, `run_psa()`, and `build_ceac()` as ordinary functions defined and unit-tested outside the Shiny app (per Chapter 27's `testthat` suite), and let the reactive blocks do nothing but call them with the current inputs. This keeps the Shiny layer thin enough that a bug report ("the CEAC looks wrong") can be reproduced and fixed by calling `run_psa()` directly at the console, with no need to launch the app, click through the UI, or reason about reactivity at all.

> **Agency perspective.** NICE's Decision Support Unit Technical Support Document 12 (*The Use of Probabilistic Sensitivity Analysis in Decision-Analytic Modelling*) specifically expects a cost-effectiveness plane and a CEAC as standard PSA outputs. A Shiny front-end that reruns `run_psa()` on demand and regenerates both plots from the same draws gives an ERG or NICE committee the ability to re-examine PSA results interactively during technical engagement — e.g. rerunning at an increased iteration count if asked — rather than depending on a fixed set of static figures in the submitted report.

29.5 Downloadable reports and audit trails

An interactive Shiny app is excellent for exploration, but an HTA submission is ultimately judged on fixed, citable documents: a PDF or Word report that an Evidence Review Group (ERG) can print, annotate, and quote by page number. The app should therefore not replace the parameterised Quarto reporting pipeline built in Chapter 28 — it should *drive* it. A button inside the app re-renders the same report template, with the current widget values passed through as Quarto parameters, so that the downloadable document is guaranteed to match what the reviewer saw on screen.

Shiny's mechanism for this is `downloadHandler()`, paired with a `downloadButton()` or `downloadLink()` in the UI. The handler is a reactive-aware function that writes a file to the path Shiny gives it; critically, its body can call `quarto::quarto_render()` on the exact `.qmd` file and parameter list from Chapter 28, rather than reimplementing export logic from scratch.

Difficult terminology: downloadHandler(). A Shiny server-side function that tells the browser "when the user clicks this button, generate a file and offer it as a download." You supply two pieces: a `filename` (what the browser should call the saved file) and a `content` function (the code that actually writes the file to a temporary path Shiny manages). It is the standard Shiny mechanism for exporting any file — CSV, PNG, PDF, Word — from a running app.

```
output$download_report <- downloadHandler(
  filename = function() {
    paste0("tb_model_report_", Sys.Date(), ".docx")
  },
  content = function(file) {
    quarto::quarto_render(
      input = "report.qmd",
      output_file = basename(file),
      execute_params = list(
        wtp_threshold  = input$wtp,
        discount_rate  = input$discount_rate,
        n_psa          = input$n_psa,
        scenario_label = input$scenario_name
      )
    )
    file.copy(file.path(dirname("report.qmd"), basename(file)), file,
              overwrite = TRUE)
  }
)
```

The same pattern works for narrower exports — a CE-plane `ggplot2` object saved as PNG via `ggsave()`, or the raw PSA draws saved as CSV via `write.csv()` — each wired to its own `downloadButton()`. The principle is consistent: whatever the reviewer downloads must be traceable to a reproducible rendering step, not a screenshot or a manually copied table.

HEOR in practice. Give every downloadable artefact a filename that encodes the key parameters and a timestamp, e.g. `tb_psa_wtp20000_2024-03-11.csv`. Reviewers routinely compare several downloaded files across review cycles, and an uninformative name like `export.csv` forces them to reopen each one to find out what it is.

Equally important for submission transparency is the audit trail: a running record of who did what, when, inside the app. For a model under ERG scrutiny, the question "what inputs produced this ICER?" must have a verifiable answer, not a recollection. The simplest implementation logs every parameter change and every model run to an append-only file.

Difficult terminology: audit trail. A chronological log of actions taken in a system, kept so that any past event can be reconstructed later. In clinical and regulatory contexts the term comes from electronic data capture systems, where every edit to a case report form is time-stamped and attributed. For a Shiny model app, the audit trail is simply a growing table of "who/when/what-changed/what-result" rows, usually written to a CSV or a lightweight SQLite database.

```
log_event <- function(session_id, event, detail) {
  entry <- data.frame(
    timestamp  = Sys.time(),
    session_id = session_id,
    event      = event,
    detail     = detail
  )
  write.table(entry, "audit_log.csv", sep = ",", append = TRUE,
              col.names = !file.exists("audit_log.csv"), row.names = FALSE)
}

observeEvent(input$run_psa, {
  log_event(session$token, "run_psa",
            sprintf("n_psa=%s; wtp=%s; discount=%s",
                    input$n_psa, input$wtp, input$discount_rate))
})
```

Using `session$token` (a per-session identifier Shiny already generates) rather than any personal identifier keeps the log anonymous while still letting a reviewer distinguish one exploration session from another. For heavier use, the same rows can be written to a SQLite file with the `RSQLite` package instead of a flat CSV, which scales better once the log reaches tens of thousands of rows and supports simple SQL filtering (`SELECT * FROM audit_log WHERE event = 'run_psa'`) for later review.

Common pitfall. Logging is easy to over-collect by accident. If a future version of the app adds a text box for reviewer comments, free-text search terms, or an uploaded patient-level dataset, a blanket "log every input" habit can silently write identifiable or commercially sensitive content to a plain-text file with no access control. The running TB model example has no real patient data, but build the logging function to log *parameter values and event names only*, and treat any free-text or file-upload input as excluded unless there is a deliberate, reviewed decision to include it.

Agency perspective. NICE's Decision Support Unit technical support documents (the TSD series, e.g. TSD 12 on probabilistic sensitivity analysis and TSD 19 on partitioned-survival modelling) do not mandate a specific audit-logging format, but NICE's broader stance in Technology Appraisal — that the ERG must be able to reproduce the submitted results from the supplied model — makes an audit trail a practical way to demonstrate that the figures in a downloaded report were in fact generated by the inputs claimed in the accompanying write-up.

29.6 Deployment: shinyapps.io, Posit Connect, Docker, servers

A finished Shiny app is only useful to an ERG or a client reviewer once it is reachable at a URL they can open without installing R. There are three broad routes, differing mainly in who manages the server and how much control that gives you over the environment.

shinyapps.io is Posit's managed hosting service. You deploy with a single call to `rsconnect::deployApp()`, and Posit runs the server, handles HTTPS, and scales instances up and down automatically.

> **Difficult terminology: shinyapps.io.** A hosting service, run by Posit (the company behind RStudio), that takes your Shiny app's files and runs them on servers Posit manages, giving you a public web address. You do not need to know anything about Linux servers to use it — you authenticate once, then deploy with one R command.

```
rsconnect::setAccountInfo(
  name   = "your-account",
  token  = "xxxxxxxxxxxxxxx",
  secret = "yyyyyyyyyyyyyyyy"
)

rsconnect::deployApp(
  appDir  = "tb_model_app",
  appName = "tb-markov-psa-explorer"
)
```

Posit Connect is a more capable platform, available either as a Posit-managed cloud service or installed on an organisation's own servers. It hosts Shiny apps alongside scheduled Quarto reports — the same scheduled-rendering mechanism introduced in Chapter 28 for recurring report generation can live on the identical server that hosts the interactive app, which matters when a client wants both a live explorer and a weekly automated PDF built from the same codebase.

Docker plus a plain Linux server gives full control: you define the exact R version, package versions, and system libraries in a container image, and run that container on infrastructure you or your organisation's IT team owns.

> **Difficult terminology: Docker image / containerization.** A Docker image is a self-contained snapshot of an operating system, R installation, and all required packages, built from a text recipe called a Dockerfile. "Containerization" is the practice of packaging software this way so it runs identically on any machine with Docker installed — your laptop, a test server, and the production server all run the exact same environment, which avoids the classic "it worked on my machine" problem.

```
# Illustrative only -- verify exact base image and package list for your setup
FROM rocker/shiny:4.3.2

RUN install2.r --error \
    shiny ggplot2 dplyr quarto rsconnect RSQLite

COPY ./tb_model_app /srv/shiny-server/tb_model_app

EXPOSE 3838

CMD ["/usr/bin/shiny-server"]
```

A server built this way typically sits behind a reverse proxy — a second piece of server software (commonly nginx or Apache) that receives the public HTTPS traffic and forwards it internally

to the Shiny container, handling certificates and routing so the app itself never deals with raw internet traffic directly.

Difficult terminology: reverse proxy. A server that sits in front of your actual application and forwards incoming requests to it, while handling tasks like HTTPS encryption, load balancing across multiple copies of the app, and hiding the internal server address from the public internet. Think of it as a receptionist who takes every visitor's request and relays it to the right office, rather than visitors walking directly into back rooms.

HEOR in practice. Pin exact package versions (with `renv::snapshot()`, per Chapter 28's reproducibility discussion) *before* building the Docker image or deploying to shinyapps.io. A deployment that silently picks up a newer CRAN package version than the one used to validate the model is a reproducibility risk an ERG reviewer has no way of detecting from the outside.

Criterion	shinyapps.io	Posit Connect	Docker + own server
Setup effort	Minutes	Hours (if self-hosted)	Hours to days
Ongoing maintenance	Low (Posit-managed)	Low-medium	Medium-high (own team patches OS/R)
Cost	Free tier; paid tiers scale with usage	Licence fee (enterprise)	Server/cloud cost + staff time
Data stays on-premises	No (Posit cloud)	Yes, if self-hosted	Yes
Scheduled Quarto reports alongside app	No	Yes (native)	Possible, self-configured
Typical fit	Pilot work, non-sensitive data	Orgs with existing Posit licence	Orgs with strict data-governance/on-prem rules

Table 86: Comparing the three main Shiny deployment routes for an HTA submission app

None of these three is universally correct. A small consultancy producing an exploratory model for internal client review may be well served by shinyapps.io's free tier; a manufacturer with an existing enterprise Posit licence and recurring submission-cycle reporting needs will likely already have Posit Connect available; and an academic health economics group bound by a university or NHS data-governance policy that prohibits sending anything (even synthetic parameter values) to third-party cloud infrastructure may have no option but self-hosted Docker. The right choice is set by the agency's or organisation's IT and data-governance constraints well before it is set by the app's technical requirements.

29.7 Security, validation and agency expectations for web-based models

Putting a health-economic model on the web introduces failure modes that a script run locally in RStudio never faces: an anonymous user can type anything into an input box, and that input will reach the model's arithmetic unless the app actively defends against it. Shiny's three-function pattern — `req()`, `validate()`, and `need()` — exists precisely to turn "silently produces a nonsensical ICER" into "clearly tells the user why this input cannot be processed."

> **Difficult terminology: input validation.** Input validation is the practice of checking that a value entered by a user is sensible *before* using it in a calculation — for example, rejecting a negative discount rate or a utility value above 1. Without it, a model may run without error but produce a meaningless result (a negative QALY gain, an ICER of zero), which is far more dangerous than an obvious crash because a reviewer may not notice.

```
output$icer_result <- renderText({
  req(input$wtp, input$discount_rate)

  validate(
    need(input$discount_rate >= 0 && input$discount_rate <= 0.1,
         "Discount rate must be between 0% and 10%."),
    need(input$wtp > 0,
         "Willingness-to-pay threshold must be a positive number."),
    need(input$n_psa >= 100,
         "Please run at least 100 PSA iterations for a stable estimate.")
  )

  icer <- run_markov_model(wtp = input$wtp,
                           discount_rate = input$discount_rate,
                           n_psa = input$n_psa)
  scales::dollar(icer)
})
```

`req()` quietly halts the reactive if a required input is missing (e.g. before the user has touched a slider); `validate()` paired with `need()` instead shows the user an explicit, readable message explaining exactly which constraint failed. For a model intended for ERG review, the second behaviour is the right default for anything that isn't simply "not yet filled in" — a reviewer testing the boundaries of the model (what happens at a 0% discount rate, or an implausibly high utility value) should get a clear rejection message, not a cryptic R error trace or, worse, a plausible-looking but wrong number.

A second, distinct concern is exposure of the model itself. Shiny apps run your R code on a server; end users interact only through the rendered HTML/JavaScript front end and never see the server-side source by default. That is a genuine protection for commercially sensitive model structure, but it is not absolute — browser developer tools can reveal client-side logic if any calculation is mistakenly pushed into JavaScript, and a misconfigured server can expose file-browsing endpoints. Where the model contains unpublished structure (e.g. a manufacturer's not-yet-public cost-effectiveness model ahead of submission), restrict access with authentication rather than relying on "security by obscurity" of an unlisted URL: shinyapps.io offers an authorized-users tier that

requires login before the app loads, and Posit Connect has built-in user and group authentication, including integration with an organisation's existing login system.

Difficult terminology: penetration testing. A deliberate, authorised attempt to find security weaknesses in a system by trying to break into it the way a malicious actor would — probing for ways to access files, data, or functionality that should be restricted. For a web-hosted health-economic model, the practical equivalent most teams can realistically do is a lighter checklist: confirm no server files are browsable, confirm authentication actually blocks unauthenticated access, and confirm input validation cannot be bypassed by unusual values (blank, negative, extremely large, or non-numeric text) — a full formal penetration test is usually reserved for apps handling real patient-level data.

Common pitfall. A `validate()`/`need()` check only runs inside the reactive expression it is written in. A common mistake is validating the input used to *display* a result while a separate `downloadHandler()` or a different output re-reads the same raw `input$x` without its own validation — meaning an out-of-range value blocked on screen can still slip into a downloaded report. Centralise validation in a single reactive (or a small validation function called from every consumer) rather than repeating ad-hoc checks per output.

What do HTA agencies actually expect from an interactive model app submitted alongside written evidence? NICE's Technology Appraisal process is explicit that the underlying economic model must be fully transparent to the Evidence Review Group: the ERG is expected to be able to interrogate the model's structure, assumptions, and calculations directly, not merely read a results summary. A Shiny app that lets the ERG change any input NICE's TSDs identify as a key driver of uncertainty, rerun the analysis, and see the ICER and CEAC update live is a natural extension of that transparency requirement — provided the app is accompanied by (not a replacement for) the full underlying R code and documentation, since "transparent" in NICE's process means reviewable in detail, not merely interactive at the surface.

Agency perspective. NICE's process for Technology Appraisal requires that the company's economic model be made available to the ERG in a form that permits full technical scrutiny, with ERG critique and company response iterated through the "technical engagement" stage of the appraisal. An interactive Shiny front-end is a supplement to this requirement, not a substitute: the ERG still needs access to the executable model and its full code, and any app presented for review should make clear which parameters are exposed as adjustable inputs versus fixed by the submitted base case.

29.8 Case study: a transparent, reviewable submission-ready model app

This closing case study pulls together the chapter's threads — reactive design (Sections 29.1-29.4), downloadable audited reports (29.5), a deployment decision (29.6), and agency-grounded security practice (29.7) — into a single sketch of what a submission-ready Shiny front end for the book's running TB Markov model (Chapters 17-27) might look like.

The design brief is simple to state and demanding to execute: an ERG reviewer should be able to open the app, see the base-case results immediately, adjust any parameter NICE's TSDs would flag as a key uncertainty driver, rerun the probabilistic analysis, watch the CE-plane and CEAC update, and download a fully parameterised Quarto report (via the Chapter 28 pipeline) that reproduces exactly what they saw — all while every action is captured in an audit log and every input is bounded so a mistyped value fails loudly rather than silently.

The UI groups inputs by clinical plausibility, each with an explicit bound enforced later in validation rather than left to the slider's default range alone:

```
ui <- fluidPage(
  titlePanel("TB Markov Model -- Interactive PSA Explorer (ERG Review Build)"),
  sidebarLayout(
    sidebarPanel(
      numericInput("tx_success_prob", "Treatment success probability",
                   value = 0.85, min = 0, max = 1, step = 0.01),
      numericInput("relapse_rate", "Annual relapse rate",
                   value = 0.05, min = 0, max = 0.5, step = 0.01),
      numericInput("discount_rate", "Discount rate (%)",
                   value = 3.5, min = 0, max = 10, step = 0.5),
      numericInput("wtp", "Willingness-to-pay threshold (£/QALY)",
                   value = 20000, min = 1000, max = 100000, step = 1000),
      numericInput("n_psa", "Number of PSA iterations",
                   value = 1000, min = 100, max = 10000, step = 100),
      actionButton("run_psa", "Run PSA", class = "btn-primary"),
      downloadButton("download_report", "Download full audited report")
    ),
    mainPanel(
      plotOutput("ce_plane"),
      plotOutput("ceac"),
      verbatimTextOutput("icer_summary")
    )
  )
)
```

The server logic follows the `eventReactive()` pattern from Section 29.4: nothing recomputes until "Run PSA" is clicked, every click is validated and logged, and the three outputs (CE-plane, CEAC, ICER summary) all read from the one reactive result so they stay consistent with each other.

```
server <- function(input, output, session) {

  psa_results <- eventReactive(input$run_psa, {
    validate(
      need(input$tx_success_prob > 0 && input$tx_success_prob <= 1,
           "Treatment success probability must be between 0 and 1."),
      need(input$relapse_rate >= 0 && input$relapse_rate <= 0.5,
           "Relapse rate must be between 0 and 0.5."),
      need(input$discount_rate >= 0 && input$discount_rate <= 10,
           "Discount rate must be between 0% and 10%."),
      need(input$n_psa >= 100, "Use at least 100 PSA iterations.")
    )

    log_event(session$token, "run_psa",
              sprintf("tx_success=%s; relapse=%s; discount=%s; wtp=%s; n_psa=%s",
```

```
                    input$tx_success_prob, input$relapse_rate,
                    input$discount_rate, input$wtp, input$n_psa))

    run_tb_markov_psa(
      tx_success_prob = input$tx_success_prob,
      relapse_rate    = input$relapse_rate,
      discount_rate   = input$discount_rate / 100,
      n_psa           = input$n_psa
    )
  })

  output$ce_plane <- renderPlot(plot_ce_plane(psa_results(), wtp = input$wtp))
  output$ceac     <- renderPlot(plot_ceac(psa_results()))
  output$icer_summary <- renderText(summarise_icer(psa_results()))

  output$download_report <- downloadHandler(
    filename = function() paste0("tb_erg_report_", Sys.Date(), ".docx"),
    content = function(file) {
      log_event(session$token, "download_report", "triggered")
      quarto::quarto_render(
        input = "erg_report.qmd",
        output_file = basename(file),
        execute_params = list(
          tx_success_prob = input$tx_success_prob,
          relapse_rate    = input$relapse_rate,
          discount_rate   = input$discount_rate,
          wtp             = input$wtp,
          n_psa           = input$n_psa
        )
      )
      file.copy(basename(file), file, overwrite = TRUE)
    }
  )
}
```

Every element here answers a concern raised earlier in the chapter. The bounded `numericInput()`s plus `validate()`/`need()` calls stop an ERG reviewer's boundary-testing from producing a nonsensical or silently wrong ICER (Section 29.7). The `eventReactive()`/action-button pattern avoids the PSA rerunning on every keystroke, keeping the app responsive during exploration (29.4). `log_event()` builds the audit trail that lets anyone reconstruct which inputs produced which plot (29.5). And the `downloadHandler()` calls the same `quarto_render()` pipeline from Chapter 28, so the downloaded Word document is guaranteed to match the on-screen results rather than being a separately maintained export routine.

> **HEOR in practice.** Before submission, run through the app yourself entering the extreme ends of every bound (minimum and maximum allowed values, and values just outside them) and confirm each produces either a sensible result or a clear `validate()` message — never a raw error trace or a plausible-but-wrong number. This ten-minute check is the single highest-value piece of QA for a submission-facing app.

Deployment choice for this app (Section 29.6) should follow the sponsoring organisation's existing arrangement rather than a technical ideal: if a Posit Connect licence and scheduled-reporting

workflow from Chapter 28 already exist, hosting the app there alongside the scheduled report keeps everything in one governed environment; if not, shinyapps.io with the authorized-users tier is a reasonable interim choice for a reviewer group of known individuals.

> **Agency perspective.** This app is, in miniature, the kind of artefact NICE's Technology Appraisal process is pointed toward when it asks for a fully transparent, ERG-reviewable economic model: the parameters the ERG is most likely to probe during technical engagement are exposed as adjustable inputs, the PSA and CEAC are generated live rather than pre-baked, and the downloadable report is traceable via the audit log to the exact inputs that produced it. It supplements, rather than replaces, providing the ERG with the full underlying R codebase.

With a working, validated, auditable interactive front end now sitting on top of the TB Markov model, the natural next step is to stop treating that model as a loose collection of scripts and start treating it as proper software: Chapter 30 turns to packaging the model, its documentation, and its tests into an installable R package, and to the pipeline tooling that keeps a model reproducible as it evolves through successive submission cycles.

Chapter 30. R Packages and Pipelines for Model Development

Learning objectives

- Explain why and when a health-economic model should be turned into a proper installable R package rather than a loose folder of scripts.
- Use `usethis` and `devtools` to scaffold a package's standard structure and drive the edit-check-reload development workflow.
- Document functions with `roxygen2` and build a browsable documentation website for the package with `pkgdown`.
- Build a reproducible pipeline with `targets`, combined with `renv` for dependency-version locking, so a multi-step model only re-runs what has actually changed.
- Set up continuous integration with GitHub Actions so a model's test suite runs automatically on every code change.
- Choose an appropriate mechanism (private repo access, a tagged release, or a public repository) for sharing a packaged model with HTA agencies and reviewers.
- Explain the practical difference between permissive and copyleft open-source licenses, and apply this book's own licensing precedent as a worked example.

Chapter 29 closed with a validated, auditable interactive Shiny front end sitting on top of the TB Markov model built across Chapters 17-27. That front end still depends, underneath, on a loose collection of R functions – `run_markov_model()`, `run_psa()`, `build_ceac()`, and the rest – that have grown organically across many chapters. This chapter is about the final step of maturing that model engine: turning it into a proper installable R package, documenting it so someone other than its author can understand it, wiring it into a reproducible pipeline, and deciding how to license and share it once it is ready to leave your own machine.

30.1 Why package your model

Most R-based health-economic models start life as a single script, or a loose folder of scripts: `data_prep.R`, `markov_model.R`, `psa.R`, `make_plots.R`, run in sequence by hand. This works for a first exploratory build. It stops working the moment the model needs to be reused for a second indication, handed to a colleague, submitted to a reimbursement agency, or revisited eighteen months later when the agency comes back with a technical query. At that point, a folder of scripts becomes a liability: nobody but the original author knows which script to run first, which version of `dplyr` it was written against, or whether the function called on line 214 of `psa.R` is the same one silently redefined further down in `make_plots.R`.

> **Difficult terminology: R package.** An R package is a standardised bundle of R code, documentation, and metadata that R itself knows how to install, load, and check for internal consistency – the same mechanism behind `dplyr` or `survival`. Turning your own model into a package means it gets the same treatment: a single `library(yourmodel)` call loads every

function, in a known working state, instead of you manually `source()`-ing a dozen files in the right order and hoping nothing has drifted.

Packaging a model buys four things that matter specifically for HEOR work:

- **Versioning and citability.** A package has an explicit version number (Section 30.2). You can say "the results in this submission were produced with `heormodel` v1.2.0", and reproduce exactly that state later, rather than "whatever was in the folder on the server in March."
- **Explicit dependency management.** The package's `DESCRIPTION` file declares every other package it needs (`Imports`), so running the model elsewhere surfaces missing dependencies immediately, instead of failing three functions deep with a cryptic `could not find function` error.
- **Documentation that lives with the code.** Function help files (`roxygen2`, Section 30.3) are written next to the function they describe and are checked for consistency every time the package is built – they cannot silently drift out of date the way a separate Word-document "user guide" does.
- **Portable, travelling tests.** The `testthat` unit tests discussed in Chapter 27 move with the package rather than sitting in a separate, easily-forgotten folder. Anyone who installs the package can run `devtools::check()` and get an immediate pass/fail on the model's internal logic.

Together with `renv` lockfiles (Section 30.4, and introduced earlier in the book), a real package turns "a folder of scripts that worked on my machine" into something an external statistician, an ERG, or a future version of yourself can install, load, and trust.

HEOR in practice. A useful trigger question: "will this model be run more than once, by more than one person?" If the honest answer is no – a single one-off exploratory analysis for an internal feasibility check – a script is fine. If the model will be re-run for scenario updates, handed to a submission team, or revisited when an agency asks a clarification question months later, the packaging overhead pays for itself almost immediately.

Common pitfall. Do not package prematurely. Full package scaffolding (`DESCRIPTION`, `NAMESPACE`, `man/`, `tests/`, roxygen comments on every function) is overhead that is wasted on a script you will run once and discard. Packaging too early can also lock in a function interface before you have actually explored the modelling problem, forcing needless rewrites. The right moment to package is when a model graduates from "exploration" to "something that will be reused, shared, or submitted" – not before.

30.2 Package structure with `usethis` and `devtools`

Building an R package from scratch by hand – writing a `DESCRIPTION` file, creating the right folder layout, generating a `NAMESPACE` – is fiddly and easy to get wrong. The `usethis` package automates this scaffolding; `devtools` then drives the day-to-day development loop of editing, reloading, and checking the package. Together they are the standard route most R package authors use, and

nothing here is specific to health economics – the same workflow is used for general-purpose statistical packages.

A new package is created with (illustratively):

```
usethis::create_package("~/projects/heormodel")
```

This produces the standard R package skeleton:

```
heormodel/
  DESCRIPTION
  NAMESPACE
  R/
  man/
  tests/
```

> **Difficult terminology: DESCRIPTION file.** A plain-text file of package metadata, read by R itself – not documentation for humans to browse, but a machine-readable manifest. It records the package's name, version, title, author, and, critically, the `Imports` and `Depends` fields listing every other package it needs to run. When you (or a collaborator, or an HTA agency reviewer) install the package, R reads this file to check all required dependencies are present before anything runs.

> **Difficult terminology: semantic versioning.** A convention for version numbers in the form MAJOR.MINOR.PATCH, e.g. `1.2.0`. PATCH increases for bug fixes that don't change behaviour meaningfully; MINOR increases when new functionality is added but existing code still works as before; MAJOR increases when something changes in a way that could break code written against an earlier version (e.g. a function's arguments change). A health-economic model package versioned this way lets you say precisely which version produced a given set of results, and whether a later version is safe to re-run old analyses against.

Within that skeleton, `R/` holds function definitions, `man/` holds the generated help files (Section 30.3), `tests/` holds the `testthat` suite from Chapter 27, and `NAMESPACE` is a file R generates and maintains, listing which functions the package exposes to the outside world.

> **Difficult terminology: NAMESPACE.** A file, usually auto-generated rather than hand-edited, that tells R exactly which functions in your package are available to a user who types `library(heormodel)`, and which are purely internal helpers. It is what prevents an internal helper function named, say, `clean()`, from silently clashing with a `clean()` function from an entirely different loaded package.

`usethis` provides small, targeted helper functions for ongoing scaffolding as the package grows:

```
usethis::use_r("run_markov_model")   # creates R/run_markov_model.R
usethis::use_testthat()              # sets up tests/testthat/ structure
usethis::use_package("dplyr")        # adds dplyr to DESCRIPTION's Imports
```

`devtools` then drives the edit-check-reload cycle. `devtools::load_all()` simulates installing and loading the package from its current source, so you can test the latest edits without a full install each time. `devtools::check()` runs R CMD check, R's own standard validity check – it verifies

documentation is complete and consistent with function arguments, that any documentation examples actually run without error, that the `NAMESPACE` is internally consistent, and that the `testthat` suite passes. `devtools::install()` performs an actual local install, making the package available via a normal `library()` call elsewhere on the machine.

```
devtools::load_all()    # load current source, no install needed
devtools::check()       # full R CMD check: docs, examples, tests, NAMESPACE
devtools::install()     # install locally
```

HEOR in practice. Run `devtools::check()` before every meaningful commit, not just before a release. It is the single fastest way to catch a forgotten `@export` tag, an undocumented function argument, or a test that quietly started failing – long before any of that surfaces in a submission deliverable.

Common pitfall. The exact argument names and defaults for `usethis::create_package()` and related helpers have changed across package versions, and newer `usethis`/`devtools` releases occasionally rename or deprecate functions. Treat the calls above as illustrative of the workflow rather than a guaranteed-current API reference, and check `?usethis::create_package` (or the current CRAN documentation) against the version actually installed before relying on exact argument behaviour.

30.3 Documentation with `roxygen2` and `pkgdown`

A packaged model is only as useful as its documentation. Writing a traditional R help file (an `.Rd` file in `man/`) by hand is possible but painful – it uses its own markup syntax, separate from the code it describes, and is trivially easy to let drift out of sync. `roxygen2` solves this by letting you write documentation as structured comments directly above the function they describe, using tags that start with `@`. Running `roxygen2::roxygenise()` reads those comments and generates both the `.Rd` help files and the `NAMESPACE` entries automatically.

```
#' Run a cohort Markov model
#'
#' Simulates a cohort through a specified Markov state structure for a
#' given time horizon, returning per-cycle state occupancy and discounted
#' costs and QALYs.
#'
#' @param trans_matrix A square numeric matrix of transition probabilities.
#' @param cycle_length Numeric, length of one model cycle in years.
#' @param n_cycles Integer, number of cycles to simulate.
#' @param discount_rate Numeric, annual discount rate applied to costs
#'   and outcomes (default 0.035).
#'
#' @return A data frame with one row per cycle, giving state occupancy,
#'   discounted costs, and discounted QALYs.
#'
#' @export
run_markov_model <- function(trans_matrix, cycle_length, n_cycles,
                             discount_rate = 0.035) {
```

```
  # model logic here
}
```

> **Difficult terminology: roxygen2 / @export tag.** `roxygen2` is the tool that converts the `#'`-prefixed comments above into formal help pages. The `@export` tag specifically tells it that this function should be listed in `NAMESPACE` as available to users of the package – functions without `@export` remain internal helpers, invisible outside the package. Omitting `@export` on a function you intended users to call is a common cause of "but it worked when I ran the script directly" confusion after packaging.

Running `roxygen2::roxygenise()` (or, more commonly in practice, `devtools::document()`, which calls it for you) regenerates `man/run_markov_model.Rd` and updates `NAMESPACE` to match. Because the documentation is written in the same file as the function, a change to the function's arguments is far more likely to prompt an update to the matching `@param` line – the two are no longer in separate places that silently diverge.

`roxygen2` documentation is primarily consumed through R's own `?run_markov_model` help lookup. `pkgdown` takes the same source material – the roxygen-generated help pages, the package's `README`, and any longer-form vignettes – and builds it into a static, browsable documentation **website**.

```
pkgdown::build_site()
```

This produces a navigable site with a function reference (one page per documented function, generated straight from the roxygen comments), a home page from the `README`, and any vignettes rendered as articles. For a model that only the original analyst will ever touch, this is unnecessary polish. For a model shared across a wider team, handed to a contract research organisation, or made available to external collaborators or reviewers, a `pkgdown` site turns "please read these twelve R scripts" into a documentation page a non-R-fluent reviewer can click through.

> **Agency perspective.** NICE's Decision Support Unit technical support documents (the TSD series) repeatedly stress that a submitted model must be usable and checkable by an external reviewer without recourse to the original analyst – this is the practical rationale behind the independent model-checking step built into NICE technical engagement, where the ERG attempts to run and interrogate the submitted model directly. Thorough `roxygen2` documentation, generated consistently with the code, is one of the more direct ways of meeting that expectation in an R-based submission.

> **Common pitfall.** The exact YAML configuration for a `pkgdown` site (in `_pkgdown.yml`) – controlling navigation structure, themes, and which vignettes appear where – varies by `pkgdown` version and is easy to get subtly wrong. Treat any specific YAML shown in tutorials, including the general idea above, as illustrative; consult the current `pkgdown` documentation for the configuration options actually supported by the installed version before relying on it.

30.4 Pipelines with `targets` and `renv`

A packaged, documented model (Sections 30.2-30.3) is still typically **run** as a sequence of steps: load data, clean it, calibrate parameters (Chapter 27), run the deterministic base case, run probabilistic

sensitivity analysis, produce tables and figures. As introduced conceptually in the scheduling discussion in Chapter 28, re-running that entire sequence from scratch every time a single downstream step changes – say, a tweak to one figure's formatting – is wasteful once calibration or PSA takes real time to execute. The `targets` package addresses this directly by making the sequence an explicit, inspectable **pipeline**.

> **Difficult terminology: pipeline / dependency graph.** Here, a pipeline is the set of computational steps needed to go from raw inputs to final results, together with the fact that some steps depend on the outputs of others (PSA depends on calibrated parameters; calibrated parameters depend on cleaned data). Represented formally, this is a dependency graph: a map of which steps feed into which. `targets` builds this graph explicitly from your code, rather than leaving it implicit in the order scripts happen to be run.

A `targets` pipeline is declared in a file named `_targets.R`, listing named targets and the function calls that produce them:

```
# _targets.R (illustrative)
library(targets)

list(
  tar_target(raw_data, read_patient_data("data/raw_cohort.csv")),
  tar_target(clean_data, clean_cohort(raw_data)),
  tar_target(calibrated_params, calibrate_model(clean_data)),
  tar_target(psa_results, run_psa(calibrated_params, n_sims = 10000)),
  tar_target(ceac_plot, make_ceac(psa_results))
)
```

Running `tar_make()` executes this graph in dependency order. The feature that matters most in practice is what `targets` does on the **second** run: it checks whether the code or inputs behind each target have actually changed since the last run, and skips any target whose inputs are unchanged, reusing its previously stored result instead.

```
tar_make()   # runs only targets whose code/inputs have changed
```

> **Difficult terminology: build cache / memoisation.** This behaviour – storing a step's result and reusing it next time instead of recomputing, as long as nothing it depends on has changed – is a build cache (the general software-engineering term is memoisation). The practical payoff for a PSA with 10,000 simulations: fix a labelling typo in the CEAC plot code, re-run `tar_make()`, and only `ceac_plot` reruns; the ten-thousand-simulation PSA result is reused unchanged, saving what might otherwise be minutes or hours of needless recomputation.

`targets` makes the **code and execution order** reproducible – the same pipeline, re-run, produces the same results from the same inputs, and anyone can see exactly which steps feed into which by reading `_targets.R`. It does not, by itself, guarantee that the **package versions** used to run that code stay fixed. That is `renv`'s job, introduced earlier in the book (Chapter 2, Chapter 28): `renv::snapshot()` records the exact version of every package in use in a lockfile (`renv.lock`), and `renv::restore()` lets anyone – a collaborator, a future version of yourself, an agency reviewer attempting to rerun the model – recreate that exact dependency environment.

Used together, `targets` and `renv` cover the two halves of reproducibility that matter for a submission-grade pipeline: `renv.lock` pins **which versions of which packages** were used, and `_targets.R` pins **what was run, in what order, and from what inputs** to produce the submitted results.

Agency perspective. CADTH's guidance on economic evaluation submissions calls for sufficient detail on the analytic approach that an independent reviewer could, in principle, reproduce the submitted results. A `_targets.R` pipeline committed alongside an `renv.lock` file is a direct, literal answer to that expectation in an R-based submission: the dependency graph states precisely what was run and in what order, and the lockfile states precisely which package versions produced the submitted numbers.

HEOR in practice. Commit both `_targets.R` and `renv.lock` to version control alongside the model package itself, and re-run `tar_make()` from a clean `_targets` cache directory before finalising any submission deliverable. A pipeline that only reproduces results on the original author's machine, with whatever packages happened to be installed at the time, is not meaningfully more reproducible than the loose folder of scripts this chapter opened by moving away from.

Common pitfall. `tar_make()`'s change-detection is based on tracking the code and declared inputs of each target – it is not magic, and it can be fooled by non-reproducible functions (for example, a random-number step without a fixed seed, or a function that reads from a file path outside the declared inputs). Exact `tar_target()` argument syntax and caching behaviour has evolved across `targets` versions; treat the calls shown here as illustrative of the workflow and check the current `targets` documentation before depending on specific caching semantics.

30.5 Continuous integration with GitHub Actions

Chapter 27 built a `testthat` suite for the model package, and Section 30.2 introduced `devtools::check()` as the command that runs those tests along with a battery of structural checks (documentation completeness, namespace consistency, example code that actually runs). Both of these are things **you** run, by hand, when you remember to. The problem is that models evolve: a collaborator tweaks a transition probability, you refactor a cost function, someone fixes a typo in a vignette that accidentally breaks a code chunk. Every one of those changes is an opportunity to introduce a bug that `devtools::check()` would have caught – if anyone had thought to run it.

Continuous integration (CI) removes the "if anyone remembered" problem. The idea, borrowed wholesale from mainstream software engineering, is to automate the check so it runs itself, every time, with no human in the loop to forget.

Difficult terminology: continuous integration (CI). Continuous integration is the practice of automatically building and testing code every time it changes, rather than relying on a person to remember to do it. In the HEOR context: every time you push an update to your model package on GitHub, a robot (not a person) spins up a clean computer, installs R and

your package's dependencies, runs your test suite and `R CMD check`, and reports back pass or fail – usually within a few minutes, without you lifting a finger beyond the original push.

GitHub Actions is GitHub's built-in CI service, free for public repositories and available at modest cost for private ones (the kind you'd use for an in-development HTA submission model). You tell it what to do via a configuration file written in YAML, placed in a specific folder in your repository.

Difficult terminology: YAML workflow file. YAML is a plain-text format for writing structured configuration – think of it as a more human-readable cousin of a CSV or JSON file, built from indented key-value pairs rather than commas. A "workflow file" is a YAML file that tells GitHub Actions what sequence of steps to run (install R, install packages, run checks) and when to run them (on every push, only on the main branch, nightly, and so on). You write it once and rarely touch it again.

A GitHub Actions workflow for an R package conventionally lives at `.github/workflows/R-CMD-check.yaml` and, conceptually, does roughly the following. Treat the exact syntax below as illustrative rather than copy-paste-guaranteed – GitHub Actions' supported actions and their version tags change over time, and the R-specific community actions (maintained by the `r-lib` project) are the authoritative, up-to-date source for exact syntax.

```
# .github/workflows/R-CMD-check.yaml
name: R-CMD-check
on: [push, pull_request]
jobs:
  R-CMD-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: r-lib/actions/setup-r@v2
      - uses: r-lib/actions/setup-renv@v2   # restores renv.lock (Ch. 2 / Ch. 28)
      - uses: r-lib/actions/check-r-package@v2
```

Conceptually, four things happen on every push: (1) a "runner" – a disposable virtual machine – is provisioned; (2) R itself is installed; (3) your package's exact dependency versions are restored from `renv.lock` (the lockfile discussed in Chapter 2 and Chapter 28, which is precisely what makes this reproducible rather than "whatever package versions happen to be newest today"); and (4) `R CMD check` and your `testthat` suite run, with the result reported back as a green checkmark or a red cross directly on GitHub, next to the commit that triggered it.

```
# Locally, this is the same check CI runs remotely on every push
devtools::check()
```

HEOR in practice. Add a CI status badge to your package's `README.md` – a small green "passing" or red "failing" icon, generated automatically by GitHub Actions and embedded via a short Markdown snippet. It costs nothing to add and gives any collaborator or reviewer glancing at the repository an instant signal: **does the code currently run without mechanical errors?** For a reviewer deciding whether to trust a model enough to start reading it in detail, a green badge is a small but real confidence boost.

Common pitfall. A green CI badge means the code **runs** and the **existing** tests pass – nothing more. As Section 27.5 emphasised, CI catches mechanical breakage (a function that errors, a test that now fails), not conceptual or clinical correctness. A model can pass every automated check while still using the wrong discount rate, misapplying a utility decrement, or mis-specifying a transition matrix. CI is a necessary complement to human review, including peer review and, eventually, agency scrutiny – never a substitute for it. Do not let a green checkmark lull a reviewer (or yourself) into skipping the substantive read-through.

30.6 Sharing models with HTA agencies and reviewers

Packaging and testing a model (Sections 30.1-30.4) solves the problem of making the model **runnable** by someone other than its author. Sharing it with an HTA agency or an Evidence Review Group (ERG) reviewer raises a second, distinct problem: making sure the model they receive is **exactly** the model you intended to submit, and that it stays that way for the duration of their review.

There are three common mechanisms, in roughly increasing order of openness.

Private repository with collaborator access. For a model still under active development, or one tied to a confidential submission, the simplest route is to add the named reviewer (or their institutional GitHub account) as a collaborator on a private GitHub repository, with read-only access. This lets them browse code, open issues asking clarifying questions, and pull the package themselves to run it – without making the model visible to the wider internet.

A tagged release attached to the dossier. Rather than pointing a reviewer at a live, continuously-changing repository, you create a **release**: a named, fixed snapshot of the repository at one specific point in its history, typically also exported as a downloadable zip archive. This archive can then be attached directly to the submission dossier assembled using the automation discussed in Chapter 28, so the model travels with the paperwork rather than as a separate, easily-decoupled link.

Public repository, for genuinely open models. Where the funder, journal, or the model owner's own policy calls for full openness (see Section 30.7), the repository is simply made public, with a permanent, citable release tag referenced in the accompanying report.

Difficult terminology: tagged release. In git (the version-control system underlying GitHub), every saved change is a "commit," and the sequence of commits on a project's main line of development is a "branch" (commonly called `main`). A **tag** is a permanent, human-readable label (e.g. `v1.0-submission`) attached to one specific commit, and a **release** packages that tagged commit as a stable, downloadable snapshot – source code, and often a compiled version, frozen at that exact point. Critically, a release does not change after the fact: future commits to `main` do not alter what the release contains. This is different from simply giving someone a link to the repository's default view, which shows whatever is on `main` **right now**, including any changes made five minutes ago.

```
# Illustrative: creating a package snapshot for a dossier release
usethis::use_version("minor")   # bumps DESCRIPTION version, e.g. 1.1.0 -> 1.2.0
# Then, on GitHub: Releases -> "Draft a new release" -> tag v1.2.0
# -> attach the dossier PDF and the source zip as release assets
```

Why insist on a tagged release rather than "whatever is on GitHub right now"? Reproducibility and fairness to the review process. If an ERG reviewer spends three weeks working through a model's logic, raises a clarification query, and in week two the sponsor pushes a bug fix to the live repository, the reviewer is now reviewing a moving target – their earlier notes may no longer correspond to the code in front of them, and it becomes unclear which version any eventual recommendation actually pertains to. A tagged release pins the model: the thing the reviewer received, the thing they reviewed, and the thing cited in their report are provably one and the same artifact, identifiable by its commit hash and tag.

Agency perspective. NICE's Technology Appraisal process requires that the economic model submitted as part of the company evidence submission be a fixed, reproducible artifact: the version assessed by the ERG during technical engagement must be traceable to a specific submission, with any subsequent model changes (e.g. responding to ERG clarification questions, or revisions following the committee meeting) submitted as explicitly versioned updates rather than silent edits to a previously-submitted file. A git tagged release, attached once to the dossier and never altered afterwards, is a natural technical mechanism for satisfying this expectation when the model's development has moved into a reproducible R package.

Common pitfall. Never grant a reviewer access to a repository without first checking what the main branch currently contains. An in-progress feature branch, an unfinished refactor, or a half-written exploratory analysis sitting on `main` the day a reviewer clicks the link can derail or confuse a review that should only ever have been looking at the tagged, submitted version. Point reviewers at the release, not at the repository's front door.

30.7 Open-source model licensing and the open-source HEOR movement

Packaging, testing, and CI (Sections 30.1-30.6) make a model technically shareable. Licensing determines what anyone receiving it is legally **allowed to do** with it. This is a genuinely important decision – and this book will not make it for you. The discussion below is conceptual orientation, not legal advice; any real licensing decision for a model intended for external use, commercial deployment, or public release should involve your organisation's legal or IP team before a license is attached.

With that caveat firmly in place, three licenses cover the large majority of cases encountered in open HEOR work.

Difficult terminology: open-source license. An open-source license is a legal document attached to a piece of code that grants others specific permissions – typically to view, use, modify, and redistribute it – that copyright law would otherwise withhold by default. Without an explicit license, "the code is on GitHub" does **not** mean anyone has the legal right to reuse it; silence is not permission. Choosing a license is choosing, deliberately, what you are giving permission for.

Difficult terminology: permissive vs. copyleft license. A **permissive** license (MIT, Apache 2.0, BSD) imposes very few conditions: others may reuse, modify, and even sell derivative works, usually as long as they keep the original copyright notice. A **copyleft** license (the GPL family) imposes a stronger condition: any derivative work that incorporates the licensed code must itself be released under the same copyleft license – in effect, "if you build on this, your building must stay open too." The practical consequence is that copyleft code cannot be quietly absorbed into a closed, proprietary product, while permissively-licensed code can.

```
# Illustrative: attaching a license to an R package with usethis
usethis::use_mit_license()     # permissive -- most common for HEOR model packages
# usethis::use_gpl3_license()  # copyleft -- an alternative, stronger-condition
choice
```

License	Commercial reuse allowed?	Derivatives must stay open?	Typical use case
MIT License	Yes	No	R packages, model code intended for broad reuse (incl. by industry)
GPL-3	Yes, but derivative must be GPL too	Yes (copyleft)	Code where the author wants all downstream forks to remain open
CC-BY	Yes, with attribution	No	Written content, data, reports – not typically used for code

Table 87: Three common license choices for HEOR model code and content

This book's own companion code repository is a worked example of applying more than one license within a single project. The R code – packages, functions, example scripts accompanying each chapter – is released under the MIT License: permissive, allowing readers to lift and reuse code freely, including in commercial consulting or industry submission work, which is exactly the use this book hopes for. The Typst source of the book's **prose** – the chapters themselves, including this one – is marked All Rights Reserved, reflecting that the author has not yet made a final decision about how the written content itself should eventually be published. Splitting the license this way, generously open for the code while reserving rights over the text, is a legitimate and common pattern; it is worth checking, for any repository you depend on, that the license file actually says what you assume it says, and that it says the same thing for code and for any accompanying prose or data.

HEOR in practice. Add a `LICENSE` file at the root of any package repository you intend others to use, even internally. "No license specified" is frequently read, correctly, as "no permission granted" – the single most common reason a potentially useful piece of health-economic code never gets reused outside the team that wrote it.

Beyond the mechanics of any one license, there is a broader current worth naming: a genuine, growing movement within HEOR and HTA toward open, auditable, reproducible models, as a

deliberate alternative to closed proprietary spreadsheet or TreeAge models whose internal logic cannot be independently inspected by a reviewer, a competing analyst, or an interested clinician. An R package with a public test suite, visible CI history, and a clear license is, structurally, a much more auditable object than a locked Excel workbook emailed as an attachment – every assumption is in readable code, every change is in a visible history, and every claim about "the model does X" can in principle be checked by running it. This is the same transparency-and-reproducibility thread this book has followed since Chapter 1, now arriving at its logical practical endpoint: a model that is not only **correct**, but **demonstrably and checkably** so, to anyone willing to read the code.

This closes Chapter 30, and with it Part VI, *Reporting, Communication and Deployment*. Part VII, *HTA in Practice*, opens with Chapter 31, *Agency Requirements and Reference Cases*, turning from how models are built and shared to what the agencies actually require once they arrive.

List of Figures

Figure 1 Since 2025, the EU Joint Clinical Assessment produces a single shared clinical-assessment report that feeds into multiple still-separate national economic and pricing appraisals — each national body (NICE, IQWiG/G-BA, HAS, and others) runs its own economic evaluation and reaches its own reimbursement decision from the same upstream clinical evidence base. 6

Figure 2 The book's eight-part structure as a cumulative HEOR workflow. Each part assumes fluency with everything upstream of it. 9

Figure 3 `Rprof()`/`summaryRprof()` output on a deliberately slow Markov-trace loop. The `rbind()`-in-a-loop anti-pattern dominates total time — `data.frame()`, `as.data.frame()` and `rbind()` together account for well over half of it — exactly the kind of result profiling should surface before any hand optimisation begins. 35

Figure 4 The cost-effectiveness plane's four quadrants. The two orange points share the identical ICER (-£2,500/QALY) but sit in opposite quadrants with opposite real-world meanings — dominant (adopt) versus dominated (reject) — exactly the ambiguity the pitfall above warns against. The blue point plots this section's own anticoagulant worked example (ΔE = 0.28, ΔC = £16,800), which falls in the north-east trade-off quadrant where neither sign alone resolves the decision and the ICER (or NMB) must actually be compared against a threshold. 47

Figure 5 The cost-effectiveness plane for the four-treatment toy example. Drug B (orange) sits above and to the left of Drug A — more expensive, fewer QALYs — and is excluded from the frontier at Step 3. The blue line connects the three surviving, non-dominated treatments in the order the efficiency frontier is built: Standard Care, then Drug A, then Drug C. 59

Figure 6 A PRISMA 2020 flow diagram built directly from the stage counts computed in code (§6.3), rather than typed by hand — the reconciliation discipline this section argues for: every number traces back to a line of code, and the itemized exclusion reasons sum to the single full-text-excluded count shown. 89

Figure 7 `robvis::rob_traffic_light()` applied to the chapter's worked RoB 2.0 example (`rob2_data`): a red/amber/green grid, one row per study, one column per domain, in the standard format expected in an SLR appendix or HTA methods annex. 94

Figure 8 `robvis::rob_summary()` on the same data: the weighted horizontal bar chart summarising the proportion of studies at each judgment level per domain, the figure typically placed in the main body of a review rather than its appendix. 94

Figure 9 Simulated TB programme cost data (n = 1,200): a visible spike of zero-cost patients at the floor, a heavy right tail driven by the rare catastrophic-cost outliers, and a mean well above the median — the distributional signature that makes ordinary least squares on raw cost a poor default. 131

Figure 10 Exactly the diagnostic the practice note above asks for: the observed Kaplan-Meier curve (black) against a chosen parametric fit (here, Gompertz), solid where it is fitted to real trial-period data and dashed where it is pure extrapolation beyond the 36-month follow-up window. The vertical dotted line marks the trial's end — everything to its right is a model assumption, not an observation, and should be labelled as such in any submission. 144

Figure 11 The observed Kaplan-Meier curve (black step function) overlaid with all six standard parametric families fitted via `flexsurvreg()`. Within the trial's follow-up window the fits visually cluster close to the KM curve and to each other — the divergence that matters for extrapolation only becomes visible once the curves are projected beyond the observed data (see Figure 15). 146
Figure 12 A simulated cohort where 25% of patients are genuinely cured. The observed Kaplan-Meier curve (black) correctly plateaus near 0.25 instead of continuing to fall. An ordinary Weibull (orange), which has no way to represent a cure fraction, keeps sliding downward and badly understates long-term survival. A mixture cure model (blue) matches the plateau because it estimates the cure fraction directly — here recovering $\pi \approx 0.24$, close to the true simulated value of 0.25. 149
Figure 13 A simulated trial where hazard genuinely drops after a 12-month landmark (shaded: the higher-hazard active-treatment phase). A single Weibull fitted to all the data (orange, dashed) is forced to compromise between the two phases and fits neither well. The landmark approach (blue) instead uses the real KM curve up to 12 months, then splices in a tail model fitted only to the post-landmark data — tracking the true post-landmark decline far more closely. 152
Figure 14 A simulated 24-month trial where a Weibull and a log-normal fit the observed data almost equally well (AIC 1302.6 vs 1306.7 — a difference most analysts would call negligible). Extrapolated to a 10-year horizon (shaded region), the two curves diverge sharply: by month 120 one projects roughly 6% survival and the other under 1%. AIC alone gives no warning that this divergence is coming, because it is calculated entirely within the unshaded region. 156
Figure 15 The extrapolated hazard function for a standard Weibull fit versus a one-knot Royston-Parmar spline, fitted to the same simulated trial data. The two curves track closely within the observed 0-36 month follow-up window but diverge sharply once extrapolated — exactly the TSD 21 diagnostic that an extrapolated survival-scale plot alone would hide. 167
Figure 16 A forest plot from `meta::forest()` on a nine-study toy risk-ratio meta-analysis: one row per study with its weight-sized confidence interval, the pooled random-effects diamond, and a prediction interval for a new study's true effect — the single most important figure this chapter produces. 174
Figure 17 A funnel plot with trim-and-fill applied to the chapter's toy meta-analysis. Observed studies and any imputed mirror-image studies added by `trimfill()` to restore symmetry are both shown — the adjusted pooled estimate from this augmented dataset is a sensitivity check, not a corrected "true" answer. 176
Figure 18 An SROC curve from `mada::reitsma()` fitted to a toy TB rapid-diagnostic-test dataset: study-level sensitivity/false-positive-rate pairs (triangles), the pooled summary point, and the 95% confidence and prediction regions — the bivariate analogue of the pairwise forest plot. 181
Figure 19 The evidence network for the worked four-treatment TB example (A, B, C, D), drawn by `netgraph()`. Node positions reflect the graph structure; edge thickness is scaled by the number of studies directly comparing that pair. C sits at the network's hub because it is the shared comparator in every anchored trial — exactly the structure that makes the Bucher-style indirect estimates in §12.2 possible, now extended and estimated jointly across every edge. 193

Figure 20 Rankogram for the four-treatment TB network: probability each treatment occupies each rank (1 = best, i.e. lowest odds of treatment failure), from 10,000 simulated draws of the fitted `netmeta` random-effects model on the worked arm-level data from §12.3.2. Treatment A has the highest probability mass at rank 1, followed by B, D, and C – consistent with the P-scores in Table 38. 211
Figure 21 A love plot of the same covariate balance shown in Table 39: absolute standardized mean differences between the IPD trial and the AgD trial's reported target, before (red) and after (green) MAIC odds-weighting. By construction, MAIC drives every matched covariate's SMD to exactly zero — the diagnostic that matters is whether any **unmatched** effect modifier remains imbalanced. SMDs computed against illustrative typical-scale standard deviations for each covariate type, since the chapter's worked example reports only means, not SDs. 217
Figure 22 Illustrative `bayesplot::mcmc_trace()` diagnostic on **simulated** MCMC draws (4 chains x 2,000 post-warmup iterations, hand-constructed with mild autocorrelation around the chapter's own Beta-Binomial posterior mean of 0.793), shown on both the natural (θ) and logit scales. This is **not** output from an actually-fitted Stan/brms model — the book's environment does not have rstan/brms installed/fittable — it is provided purely to demonstrate what a healthy, well-mixed "fat hairy caterpillar" trace plot looks like in practice. 238
Figure 23 Illustrative `bayesplot::ppc_dens_overlay()` diagnostic comparing a **simulated** observed-outcome density to 20 **simulated** posterior-predictive replicate densities, both drawn from Beta distributions perturbed around the chapter's Beta-Binomial posterior parameters ($\alpha_{\text{post}} = 184$, $\beta_{\text{post}} = 48$). This is **not** output from an actually-fitted Stan/brms model — the book's environment does not have rstan/brms installed/fittable — it is provided purely to demonstrate what a tight, well-fitting bundle of replicated curves looks like against the observed data. 240
Figure 24 `SHELF::plotfit()` overlay of fitted candidate distributions against the elicited response-rate quantiles (25th/50th/75th percentiles = 0.30/0.45/0.60). By `fit$ssq` least-squares scoring, SHELF's default ranking places the unbounded skew-normal and normal families as the closest numerical fit to these three points (ssq on the order of 10^{-32} – essentially exact, because a symmetric three-point quantile set is fit almost perfectly by a two/three-parameter symmetric family). The beta fit scores a higher but still small ssq (1.99×10^{-5}), with gamma and log-normal trailing further behind. For a HEOR prior on a probability, this is the practical lesson: the **lowest-ssq** family is not automatically the **right** one to carry into the model – a response rate is bounded on $[0, 1]$, and the normal/skew-normal fits can assign positive density outside that range, which is nonsensical for a probability parameter. The beta fit, despite its slightly larger ssq, is the one that should feed the prior, precisely because it respects the support of the parameter being elicited. 247
Figure 25 Left: the TB regimen confounding DAG as encoded by `dagify()`. Right: `ggdag_adjustment_set()`'s visual confirmation that `{hiv, site_capacity}` is the minimal sufficient adjustment set — both coloured red as "adjusted" — while `nutrition` (a mediator) is correctly left unadjusted. 257
Figure 26 A love plot for a toy TB cohort (n = 600): absolute standardized mean differences before (red) and after (teal) inverse-probability-of-treatment weighting. Every covariate, including the propensity score itself, collapses from well above the conventional 0.1 threshold (dashed line) to effectively zero after weighting. 260

Figure 27 `rdrobust::rdplot()` output for the CD4-count RD example: culture conversion time against CD4 count, with the policy cutoff at CD4 = 200 marked by the vertical line. The fitted local-polynomial curves on either side show a visible downward jump just below the cutoff, where the new regimen becomes mandated. 268
Figure 28 `did::ggdid()` output for a simulated staggered-rollout TB programme: group-time average treatment effects by quarters since rollout (negative x-axis = pre-treatment, positive = post). Pre-treatment estimates hover around zero with overlapping confidence intervals (supporting the parallel-trends assumption), while post-treatment estimates show a consistent negative effect on culture conversion time. 270
Figure 29 Distribution of per-patient estimated treatment effects (`tau_hat`) from `grf::causal_forest()` on a simulated TB-regimen dataset. The bimodal spread — one cluster near zero, another around 0.25-0.3 — is itself evidence of real heterogeneity: a single average effect (solid red line) would mask the fact that some patients benefit far more than others. 289
Figure 30 A subgroup forest plot (ggplot2 point-and-error-bar, log odds-ratio scale) comparing the overall treatment effect against the HIV-positive and HIV-negative subgroups. The overlapping confidence intervals here would not meet a pre-specified threshold for a confirmed interaction — illustrating why the formal interaction test, not a pair of separate within-subgroup p-values, is the correct way to test effect modification. . . 291
Figure 31 Full 20-cycle state-occupancy trace from the base-R cohort loop above, stacked by state. Mass drains rapidly out of On Treatment in the first 1-2 cycles, with most of it settling into Cured (the large green band) and a small residual passing through Treatment Failure before being re-absorbed; Death's share (grey) climbs monotonically across all 20 cycles, as it must for an absorbing state. 318
Figure 32 State-occupancy trace for the §20.6 PSM, SOC vs new drug, built directly from the same `weib_S()` PFS and OS curves used in the worked arithmetic above. Each arm's stacked area is bounded below by OS (PFS + Progressed + Dead always sums to 100%) and the PFS and Progressed/Dead boundaries never cross within either panel over the full 10-year horizon – the visual counterpart of the no-crossing check reported alongside the discounted cost and QALY totals, and the structural condition §20.1 and §20.5 flag as the PSM's central fragility. 335
Figure 33 Detection pathway by arm from the chapter's own `run_arm()` common-random-numbers microsimulation (N = 1,000 patients, 20 annual cycles). Screening shifts detections earlier — 78 more patients detected within 2 years of onset — without changing the total number eventually detected, exactly the timing-shift mechanism Table 54 reports. 346
Figure 34 Infectious compartment (I) over the 10-year horizon from the `deSolve::ode()` SEIR model above: no-intervention baseline vs the 30%-effective screening/preventive-therapy programme. The gap between the curves widens over time rather than staying fixed at the size of the one-time intervention, illustrating the compounding/indirect-protection effect that a static model would miss (the pitfall in 22.4). 360
Figure 35 Illustrative linear vs logistic (S-shaped) uptake curves over a 5-year BIA horizon, both reaching a terminal market share of 0.35, from the `ggplot2` code above. The logistic curve starts slower, accelerates through the middle years, and converges toward the same terminal share as the linear ramp. 368
Figure 36 Annual budget impact (bars, rising from $600,000 in Year 1 to $6,600,000 in Year 5) with cumulative budget impact overlaid as a line (climbing to $18,000,000 by Year 5),

base-case uptake scenario from Table 23.1 (<tab-ch23-bia-result>), from the ggplot2 code above. 373
Figure 37 The equity impact plane plotted for Table 59′s toy example: net health benefit (total unweighted QALYs = 200) on the x-axis against net equity impact (equity-weighted minus unweighted QALYs = +60) on the y-axis. This example lands in the win-win quadrant — equity weighting happened to favour the subgroup already receiving the larger QALY gain. Had the most-deprived group received the **smaller** gain instead, the same exercise would land in the "efficient but unfair" quadrant, the genuine trade-off this plane exists to visualize. 383
Figure 38 Financial risk protection outcomes from Table 60 plotted by income quintile: catastrophic-health-expenditure cases averted (blue) and poverty cases averted (orange). Essentially all of the policy's financial protection accrues to the three poorest quintiles — Q4 and Q5 see zero effect — exactly the pro-poor distributional pattern the pooled total (53,333 CHE cases averted) alone would hide. 388
Figure 39 Tornado diagram: one-way sensitivity analysis on incremental net monetary benefit (New TB regimen vs SoC), WTP = $50,000/QALY. Bars sorted by swing width descending; the dashed vertical line marks the base-case NMB of $8,044.05. 409
Figure 40 Cost-effectiveness plane: 2,000 PSA draws (incremental QALY on x, incremental cost on y). The dotted line marks the WTP = $50,000/QALY threshold; most draws fall below it, consistent with New's favourable base case. 419
Figure 41 Cost-effectiveness acceptability curve (CEAC): proportion of the 2,000 PSA draws favouring New across a $0-$100,000/QALY willingness-to-pay range. 420
Figure 42 Cost-effectiveness acceptability frontier (CEAF): illustrative 3-strategy sketch extending the New-vs-SoC PSA draws with a hypothetical third strategy. Faint lines are each strategy's own CEAC; the bold line traces the upper envelope – the strategy most likely to be optimal at each WTP threshold. 421
Figure 43 Distribution of incremental net monetary benefit across the 2,000 PSA draws at WTP = $50,000/QALY. The dashed line marks zero; the bulk of the distribution sits on the positive (New-favoured) side. 422
Figure 44 Side-by-side state-transition structures: the book's 4-state base case (left, with an explicit Fail-to-OnTx retreatment pathway) versus the collapsed 3-state alternative (right, where failure routes directly into an increased death risk with no separate Fail state). 427
Figure 45 Cost-effectiveness plane built from 2,000 non-parametric bootstrap replicates of the simulated trial dataset (incremental QALY on x, incremental cost on y). The dotted line marks WTP = $50,000/QALY; nearly all replicates fall below it, matching the 100%-positive-NMB result reported in Table 71. 432
Figure 46 GAM fit diagnostic: raw PSA draws (scatter) with the mgcv::gam() fitted smooth (red curve) overlaid. A genuinely strong, near-linear relationship between p_ontx_fail and incremental NMB – the fitted curve tracks the scatter closely with no visible under- or over-fitting, consistent with the method = "REML" default. Running this fit on the actual PSA draws gives an EVPPI for p_ontx_fail of $847.01 – close to, and independently confirming, the $900 illustrative figure used in the worked examples throughout this section and §26.4-26.6. 444
Figure 47 The 500 random-search p_hidden candidates (x-axis) against their squared distance from the 32% mortality target (y-axis). The red point marks random search's closest draw; the dashed green line marks the Nelder-Mead-converged value (0.084) from the

next subsection – visibly sitting near the bottom of the same illustrative response curve both methods are searching. 457
Figure 48 The external-validity comparison from the table above: model-predicted (78%) vs illustrative observed (74%) 10-year survival for the New regimen. The gap is small in absolute terms (4 percentage points) but, per the pitfall below, a single cohort comparison is evidence to weigh, not a verdict. 469
Figure 49 A simulated three-arm cost-effectiveness plane (illustrative data) compared under the ggplot2 default qualitative palette (top row) and the viridis palette (bottom row), each shown under normal vision (left) and a simulated deuteranopia colour-vision deficiency (right, via `colorspace::deutan()`). Under the default palette, "Novel combination" and "Shortened 4-month" become visually indistinguishable once deuteranopia is simulated; all three arms remain distinguishable under viridis. 484

List of Tables

Table 1 Major HTA agencies/bodies an HEOR analyst routinely encounters, their jurisdiction, general evaluative stance, and a key reference document cited elsewhere in this book. 5
Table 2 Evidence generation and typical analytical deliverables by product lifecycle stage. 7
Table 3 R package ecosystem for HEOR by task area, with the key packages for each and where in this book they are covered in depth. 23
Table 4 Common data-wrangling operations: `dplyr`/tidyverse syntax versus the equivalent `data.table` expression. 29
Table 5 Top functions by self + total time in the `summaryRprof()` output above, ranked by where profiling attention should go first. 35
Table 6 A CHEERS 2022-style cost-effectiveness results table for the anticoagulant worked example — the standard way an ICER/NMB result is reported in a paper or HTA submission, rather than left as bare console output. 48
Table 7 Willingness-to-pay / cost-effectiveness threshold conventions across selected HTA jurisdictions — the kind of comparison table a methods section uses to justify which threshold a submission is benchmarked against. 51
Table 8 Perspective, time horizon and discount rate reference-case conventions across selected HTA bodies — the modelling-choices summary a methods section typically tabulates before presenting results. 53
Table 9 How the four-treatment frontier analysis would be reported: dominance status, survival of pruning, and the ICER against the next-cheaper non-dominated option. . . 60
Table 10 Baseline demographic summary of the safety population by planned treatment arm (ADSL-derived), the conventional “Table 1” format for clinical study reports and HTA submissions 65
Table 11 Fit-for-purpose mapping of real-world and trial-derived data source types to common HEOR model inputs 67
Table 12 Data validation summary for the cost-QALY input table, showing rule-by-rule pass/fail counts as reported in a submission’s data-management appendix 78
Table 13 Search strategy summary by database, as reported in an SLR methods section or PRISMA-S search appendix 83
Table 14 Dual-reviewer title/abstract screening agreement, as reported in the SLR methods section 86
Table 15 Screening funnel stage counts underlying the PRISMA 2020 flow diagram, distinct from the itemized full-text exclusion reasons below 88
Table 16 Full-text exclusion reasons, reported alongside the PRISMA 2020 flow diagram (Item 7) 89
Table 17 Pooled multiple-imputation regression coefficients (Rubin’s rules) for `qaly ~ treatment + cost`, m = 10 imputations 104
Table 18 Tipping-point sensitivity analysis: six-month treatment-success risk difference (intervention minus control) as the delta applied to imputed intervention-arm successes increases, TB regimen trial 106
Table 19 Missing-data summary for the primary six-month utility endpoint, by randomised arm 110

Table 20 Mean (SD) EQ-5D-type utility and ceiling-spike proportion by disease severity subgroup, toy dataset 119
Table 21 Multiplicative age-adjustment of a toy disease-specific utility (baseline 0.80 at age 45-54) against general-population EQ-5D norms 122
Table 22 Comparison of TTO, SG and DCE elicitation methods used in EQ-5D value-set construction 124
Table 23 Unit-cost reference table for a TB programme resource-use valuation, by resource category and item (local currency, price year 2023) – the conventional format for publishing the unit-cost list underlying a costing or cost-effectiveness analysis 127
Table 24 Gamma GLM (log link) results for per-patient TB cost by regimen and age, among patients with cost > 0 – the conventional format for reporting cost-ratio regression results (coefficient, exponentiated cost ratio, 95% CI, p-value) rather than raw log-scale coefficients alone 133
Table 25 Comparison of censored-cost-handling methods (naive mean of observed cost, Lin et al.'s partitioned Kaplan-Meier-weighted estimator, inverse-probability-of-censoring weighting) by bias direction, data requirement and principal limitation – the summary table a costing section should present when justifying the chosen estimator for incomplete follow-up cost data 139
Table 26 How to report this: Royston-Parmar spline knot-selection comparison (`flexsurvspline()`, hazard scale) across k = 0-3 internal knots, with AIC/BIC and the within-trial-period fit improvement each extra knot buys – the format for justifying a chosen knot count against TSD 21′s expectation that knot selection be reported, not merely asserted. 148
Table 27 How to report this: cure-fraction estimate from a mixture cure model (`flexsurvcure()`, Weibull uncured distribution), with the 95% confidence interval `summary()` returns, alongside the AIC comparison against the corresponding non-cure Weibull fit that motivates including a cure fraction at all. 151
Table 28 How to report this: AIC/BIC model-comparison table across the six standard parametric survival distributions fitted with `flexsurvreg()`, the TSD 14-expected candidate set and reporting format for justifying a base-case extrapolation choice. . 155
Table 29 How to report this: `cox.zph()` proportional-hazards diagnostics for a Cox model of treatment arm, age and ECOG performance status – the conventional format for presenting per-covariate and global Schoenfeld-residual test statistics alongside the PH decision each motivates. 158
Table 30 How to report this: the per-study and pooled effect sizes that accompany a forest plot, together with the heterogeneity statistics (Q, τ^2, I^2) that must be quoted alongside the pooled estimate in an HTA submission 175
Table 31 How to report this: sensitivity of the pooled random-effects estimate to the trim-and-fill adjustment for funnel-plot asymmetry, shown before (observed studies only) and after (observed plus imputed mirror-image studies) alongside Egger's test for small-study effects 177
Table 32 How to report this: meta-regression coefficients from `metafor::rma(mods = ~ mean_age + region)`, the conventional way to present which study-level moderators explain between-study heterogeneity and by how much 178
Table 33 How to report this: pooled sensitivity and specificity from the bivariate `mada::reitsma()` model, the standard summary table accompanying an SROC curve in a diagnostic test accuracy review 182

Table 34 Bucher indirect treatment comparison of Regimen A vs Regimen B for MDR-TB treatment failure/relapse, anchored on the shared standard-of-care comparator C (Trial 1: A vs C; Trial 2: B vs C). The A-vs-B row is the derived indirect estimate, not an observed trial result, and its interval is wider than either direct comparison, reflecting the added uncertainty of combining two independent trials. 189
Table 35 Excerpt of the random-effects `netleague()` output for the four-treatment TB network, independently re-verified in this session against the §12.3.2 `tb_arms` data, showing odds ratios (95% CI) for treatment failure ordered best-to-worst by P-score (A, B, D, C — corrected from an earlier draft that had mis-stated D as best; see Table 38). Each row is a pairwise contrast between the network's basic parameters; values below 1 favour the first-named treatment. Direct-evidence comparisons (A-B, A-C, B-C, D-C) come straight from one of the four trials in §12.3.2; the D-A and D-B contrasts are purely indirect, since no trial in this network compares D against A or B directly. 195
Table 36 How to report this: `summary(mtc.nodesplit())` output for the TB network's three splittable edges, the conventional node-splitting layout showing the direct estimate, the indirect estimate, their difference, and a Bayesian p-value for each edge 200
Table 37 The three network meta-regression results a dossier should present side by side for the four-treatment TB network (odds ratios for treatment failure vs. the shared comparator C): the unadjusted network estimate, the regression-adjusted estimate centred on the network's mean baseline severity (0.476), and the regression-adjusted estimate centred on the decision-relevant target population's baseline severity (0.55, a more severe population than the trial-average mix). 205
Table 38 How to report this: Frequentist P-scores (`netmeta::netrank()`), independently verified in this session from the worked §12.3.2 arm-level data, ranked best (1) to worst (4) on treatment failure. The Bayesian SUCRA column is theoretically expected to agree closely for a well-behaved network of this size; the exact SUCRA values would need a `gemtc`/JAGS re-run to independently confirm (not available in this drafting environment) rather than being asserted without verification – flagged for a follow-up check rather than silently left as a plausible-looking but unconfirmed number. 212
Table 39 Baseline covariate balance between the IPD trial and the AgD trial, before and after MAIC odds-weighting 217
Table 40 Effective sample size before and after MAIC weighting (illustrative 400-patient IPD trial matched on two effect modifiers) 218
Table 41 Illustrative STC outcome-regression coefficients (logistic model, IPD trial, treatment × covariate interactions), and the resulting log-odds-ratio predicted at the AgD trial's reported covariate means 220
Table 42 Comparison of the three population-adjustment methods covered in this chapter: data requirements, core assumption, and typical use case 225
Table 43 NICE DSU TSD 18: named limitations of population-adjustment methods and the agency's stated concern for each 227
Table 44 Posterior summary for TB treatment success probability, Beta-Binomial conjugate update ($n = 40$ new cohort patients, $y = 34$ successes, informative prior $\mathrm{Beta}(150, 42)$) 233
Table 45 Toy cohort for inverse-probability-of-treatment weighting 261
Table 46 Illustrative IV diagnostics output structure from `summary(iv_fit, diagnostics = TRUE)` 267
Table 47 E-values for key endpoints, illustrative MDR-TB regimen comparison 276

Table 48 Expected cost, expected QALYs, and incremental cost-effectiveness ratio for the two TB diagnostic strategies, as computed in §18.1–§18.2′s worked example. 308
Table 49 State occupancy from the base-R cohort trace, cycles 0, 5, 10, 20 (proportion of starting cohort) 317
Table 50 Discounting a 5-cycle QALY stream at NICE's 3.5% annual rate (1.735% per 6-month cycle) 321
Table 51 Illustrative per-6-month-cycle transition probabilities, by arm 327
Table 52 Chronic-disease CEA: discounted 10-year results, SoC vs DMT 328
Table 53 Base-case discounted cost, QALYs and ICER from the synthetic oncology PSM, new drug vs standard of care (SOC) 334
Table 54 Illustrative simulated TB screening programme: detection outcomes across 1,000 patients over 20 annual cycles under common random numbers (synthetic parameters, not fitted to observed TB data) 346
Table 55 Decision framework: choosing a model structure for a HEOR/HTA question 358
Table 56 Illustrative TB screening/preventive-therapy programme vs no intervention, 10-year horizon per 100,000 population (toy model, demonstration figures only) 361
Table 57 Year-by-year budget impact of introducing the new drug, base-case uptake (current-mix cost held flat at $5,000,000/year; all figures USD) 372
Table 58 Comparing the two main approaches to monetizing health, and VSL as a specific instance of the WTP approach 378
Table 59 Unweighted vs. equity-weighted ICER for a toy two-subgroup screening example (equity weight of 1.5 applied to the most-deprived quintile's QALYs) 382
Table 60 Extended cost-effectiveness analysis: financial risk protection by income quintile, universal public finance of TB treatment versus status quo (illustrative). 387
Table 61 MCDA weighted-sum and weighted-product scores for three illustrative drugs, equal-criteria committee weighting. 391
Table 62 Managed entry agreement types. 394
Table 63 Task Force-6′s uncertainty taxonomy and standard handling, applied to the TB regimen-choice model 402
Table 64 One-way sensitivity analysis: incremental net monetary benefit (New regimen vs SoC) at low and high bounds per parameter, $\lambda = \$50,000$/QALY, base-case incremental NMB = $8,044 407
Table 65 Two-way sensitivity analysis: incremental NMB ($) across joint cure-probability / drug-cost combinations, New regimen vs SoC 408
Table 66 Scenario analysis: discounted total cost and QALYs, four-state TB Markov model, optimistic/base/pessimistic parameter bundles 412
Table 67 Standard PSA distribution choices by HEOR parameter type. 415
Table 68 Threshold analysis: breakeven total discounted cost for a 4-month TB regimen delivering a 0.15-QALY gain over the standard regimen, at WTP = $50,000/QALY ... 424
Table 69 Headline result under the 3-state collapsed structure vs. the book's 4-state base case, New regimen vs SoC 426
Table 70 Model-averaged incremental NMB at WTP = $50,000/QALY, combining the 4-state and 3-state structures 428
Table 71 Bootstrap summary of incremental cost-effectiveness from the simulated trial dataset, $B = 2000$ replicates 432
Table 72 Illustrative worked EVPI calculation on six PSA-style draws, same `psa_results` column layout as Chapter 25 §25.4, WTP = $50,000/QALY. 437

Table 73 The four core VOI quantities: what each answers and what it needs. 438
Table 74 Illustrative EVPI (per patient) across a WTP grid, computed via `calculate_evpi()` on an illustrative PSA-shaped dataset consistent with Chapter 25′s TB base case. 441
Table 75 Comparison of the three standard EVSI approximation methods 448
Table 76 Illustrative value-of-implementation by year of a 5-year uptake ramp (undiscounted) 450
Table 77 Illustrative EVPPI ranking across four candidate parameters of the TB model 452
Table 78 Illustrative ENBS comparison of two proposed trials targeting the same parameter . 453
Table 79 Calibration target vs. model-predicted value, TB Markov model, SoC arm 458
Table 80 AdViSHE validation domains and their relationship to this book's chapters 465
Table 81 Illustrative external-validity comparison for 10-year survival, New regimen 468
Table 82 Four R table-formatting packages: honest comparison of use-cases 479
Table 83 TB 4-state Markov model: base-case ICER and NMB summary (New-regimen vs SoC) 480
Table 84 Illustrative subset of CHEERS 2022 items cross-referenced to report sections and earlier chapters 487
Table 85 Comparing plain Shiny modules, `golem`, and `rhino` 499
Table 86 Comparing the three main Shiny deployment routes for an HTA submission app . . . 505
Table 87 Three common license choices for HEOR model code and content 521